



PowerArtist™

User Guide

Software Release 2011.2.1

Copyright Notice and Proprietary Information

No part of this document may be reproduced or transmitted in any form or by any means, electronic, or mechanical, for any purpose, without the express written permission of Apache Design, Inc. This manual and the program described in it are owned by Apache Design, Inc., a subsidiary of ANSYS, Inc., and may be used only as authorized in the license agreement controlling such use, and may not be copied except in accordance with the terms of this agreement.

© 2012 Apache Design, Inc.

Disclaimer

Apache Design, Inc. makes no warranty of any kind, expressed or implied, with respect to software or documentation, its quality, or performance. The information in this document is subject to change without notice and does not represent a commitment on the part of Apache Design, Inc. or ANSYS, Inc.

Trademarks and Registered Trademarks

ANSYS and any and all ANSYS, Inc. brand, product, service and feature names, logos and slogans are registered trademarks or trademarks of ANSYS, Inc. or its subsidiaries in the United States or other countries. All other brand, product, service and feature names or trademarks are the property of their respective owners.

Apache Design, Inc.

Subsidiary of ANSYS, Inc.

2645 Zanker Road

San Jose, CA 95134

Main: 408.457.2000

Fax: 408.428.9569

rtl-support@apache-da.com

sales@apache-da.com

www.apache-da.com

Table of Contents

CHAPTER 1	Introduction to PowerArtist	
	PowerArtist and Your Design Flow	1
	Available Tutorials to Get You Started	2
CHAPTER 2	Installing and Setting Up PowerArtist	
	Introduction	1
	Supported Operating Systems	1
	Installing the Software	1
	Distribution Tree	2
	PowerArtist Licensing.....	2
	Installing Licenses and FLEX/m	3
	Understanding License File Entries.....	3
	Support for Multiple License Servers	4
	Product vs. Feature Licenses	4
	Waiting for a Feature License	4
	Resolving Feature Checkout Problems	4
	Setting the UNIX Environment Variable for PowerArtist	4
CHAPTER 3	Using the PowerArtist Shell	
	Introduction	7
	Understanding the ptshell Command Options	7
	Invoking the GUI from the ptshell	9
	Reverting to the Legacy Shell	10
	ptshell Features.....	10
	Customizing Your PowerArtist Environment Using Initialization (INI) Files....	11
	Sample Initialization File.....	12
	Understanding Log Files and Key Files.....	12
	Understanding the Link Between Commands and the PowerCanvas	13

CHAPTER 4 PowerArtist Tutorial Part 1: Power Analysis

Introduction	15
Tutorial Organization	16
Copying the PowerArtist Tutorial Files	17
Contents of the Top-Level Tutorial Directory	17
Contents of the Analysis Directory	17
Understanding Command File Set-Up Methods	18
Running an RTL Full-Chip Power Analysis Using Command Files	19
Examining the Contents of the Tcl Command Files	20
Running the Tcl Scripts	26
Viewing Power Analysis Results in the PowerCanvas	31
Manipulating Your Design in the Schematic	32
Viewing Clock Trees	33
Waveform Cross-Probing	37
Running Full-Chip Analysis Using the PowerCanvas Wizards	38
Performing Design Inferencing Using the HDL Inferencing Wizard	38
Running Vector Analysis Using the Vector Wizard	42
Running the Average Power Wizard	45
Running the Time-Based Power Wizard	49
Running an RTL Block-Level Power Analysis	53
Basic Flow for Running the Block-Level Tutorial	53
Using the Command Files	54
Viewing the Block-Level Power Results in the PowerCanvas	59
Running the Block-Level Tutorial Using the Power Canvas Wizards	59
Running Power Analysis with Clock Gating	60
Using the Command Files	60
Using the PowerCanvas Wizards	60
Running a Power Analysis with a Mixed-Vt Cell Library	61
Using the Command Files	61
Using the PowerCanvas Wizards	63
Running Power Analysis with Voltage Islands	64
Using the Command Files	64
Using the PowerCanvas Wizards	65
Running a Power Analysis with Power Gating	66
Using the Command Files	66
Using the PowerCanvas Wizards	68

Running the Full-Chip Gate-Level Power Analysis Tutorial	69
Using the Command Files	69
Using the PowerCanvas Wizards	70
Understanding Power Analysis Results in the Text-Based Power Report	71
Internal Power Consumption	71
Pad Power Consumption	73
Clock Power Consumption	74
Clock Domain Power Consumption	77
Inferred Buffer Tree Power	80
Area	81
Net Frequencies	81
Power Consumption by Model/Gate Type	81
Summary	82

CHAPTER 5 **PowerArtist Tutorial Part II: Power Reduction**

Introduction	83
About PowerBots	84
Tutorial Organization	84
Copying the PowerArtist Tutorial Files	85
Contents of the Top-Level Tutorial Directory	85
Running the Reduction Tutorial Using Command Files	86
Setting Important Environment Variables	87
Basic Flow for Running the Reduction Tutorial	87
Using the Command Files	88
Reviewing Reduction Results in the PowerCanvas	95
Examining Simple Power Reductions	97
Examining Advanced Power Reduction Results	109
Examining Power Linter Reduction Results	118
Running the Reduction Tutorial Using the PowerCanvas Wizards	121
Using the HDL Inferencing Wizard	121
Using the Power Reduction Wizard	121
Using the RTL Rewrite Wizard	126
Reviewing the Output Reports	129
Reviewing the Power Reduction Report	129
Reviewing the Power Reduction Clock Gating Report	132

CHAPTER 6 Using the PowerCanvas

Introduction	139
Chapter Organization	139
Overview of the PowerCanvas Initial View	140
The PowerCanvas Menus	141
Using the Edit Menu	142
Using the Tools Menu	142
Using the View Menu	142
Using the Help Menu	143
PowerCanvas Dialogs	145
Using the Find Dialog	145
Using the Properties Dialog	148
Using the Smart Source Browser	151
Using the Hierarchy Browser	152
Column Header Definitions for the Power Table	153
Controlling the Hierarchy Browser From the Design Menu	153
Using the Schematic Display	156
Controlling the Schematic from the Schematic Menu	156
Using the Mouse to Zoom In and Out on the Schematic	159
Expanding the Schematic by Double Clicking Ports	159
Basic Clock Tree Manipulation	161
Using the Simple Reduction Dialog	163
Column Header Definitions for the Simple Reductions Dialog	163
Column Header Right-Click Pop-Up Menu	165
Simple Reduction Dialog Data Filters	166
The View Menu	167
The Detail Menu	168
Using the Simple Sorting and Filtering Features	169
Using the Advanced Sorting and Filtering Features	169
Using the Linter Reductions Dialog	172
Column Header Definitions for the Linter Reductions Dialog	172
Using the Prism Dialog	174
Categories of Implementation of Suggested Modifications	176
Definitions for the Bit Counters in the Prism Dialog	177
Column Header Definitions for the Prism Dialog	177
Filtering Prism Dialog Data	179
Using the Normal vs. Optimal Views	181

Using the Waveform Viewer	184
Supported Waveform Sources	184
Finding and Displaying Waveforms from an FSDB File	184
Manipulating the Waveform Display	188
Displaying Waveforms from a PTCL File	188
Displaying Waveforms from an HSPICE or Spectre Simulation	189
Using the Waveform Viewer Options Menu	189
Adding Notes to a Plot	193
Comparing Waveforms in the Same Plot	193
Opening Tabs in a Separate Window	193

CHAPTER 7 **Getting Your Design into PowerArtist**

Introduction	195
Chapter Organization	195
Command-Line Flow for Verilog	196
Command-Line Flow for VHDL	196
Command-Line Flow for Mixed-Language Designs	198
VHDL Designs with One or More Verilog Modules	198
Verilog Designs with One or More VHDL Modules	198
Case Sensitivity	199
Compiling Mixed-Language Designs in a Single Run	200
Creating Your Map Files	201
The synonym Statement	201
Running the wwvmkr Utility	202
Contents of the ptSourceFiles.tcl File	202
Metacomment Processing	203
Creating Custom VHDL Packages	204
Technique 1: Modifying Your Installation	204
Technique 2: Multiple Compile Scripts	204
Defining Libraries for Command-Line Use	206
HDL Advanced Topics	206
Using Power Macros	206
Overriding Parameter Settings for Top-Level VHDL or Verilog Modules	208
Controlling Array Inferencing	209

CHAPTER 8 **Preparing for Power Analysis**

Introduction	213
--------------------	-----

Chapter Organization	213
Estimating Net Capacitances	214
Back-Annotating Capacitance Using SPEF	215
Using Back-Annotated Load Capacitances for Primary Outputs	217
Specifying Default Output Load Capacitance Using a Command Option	217
Using Back-Annotated Wiring Capacitances for Local Nets	217
Specifying Wire Load Models	217
Using Apache Default Wire Load Models for Capacitance Analysis	220
Using PACE Technology Files During Power Analysis (Beta)	221
Using Clock Distribution Network Models	221
Understanding Priorities and Precedence When Using PACE Models	222
Using the SetClockGatingStyle Command	222
Additional Guidelines for Using PACE Models	223
Understanding Output File Changes due to PACE Models	223
Estimating Pin Capacitance	224
Handling Voltages in Liberty Format	225
Handling Designs with Multiple Libraries	225
Handling Designs with Multiple Power Supplies	227
Creating a Virtual Supply	227
Assigning a Virtual Supply to a Hierarchical Instance	227
Liberty Power Supply Support	228
Running RTL Mixed-Vt Power Analysis	229
Critical Liberty Leakage Attributes	229
Categorizing Cells for Multiple Vts	230
Default Cell Selection for Mixed-Vt Analysis	231
Sample Mixed-Vt Flow and Tcl File	231
Understanding Mixed-Vt Analysis Results in the Report File	232
Setting up Clock Power Analysis	233
Commands for Clock Power Analysis	233
How Clock Power Analysis Works	234
Controlling Forward Clock Tracing	235
Setting up Clock Gating for Power Analysis	237
Clock Gating Flow for Power Analysis	237
Performing Enhanced Clock Gating	238
Library Modeling of Integrated Clock Cells	239
Setting up Clock Gating for Power Reduction	239

	Clock Gating Flow for Power Reduction.....	239
	Clock Gating Algorithm.....	240
	Hierarchical Clock Gating.....	242
	Inferring Buffer Trees for Nets with High Fanout.....	242
CHAPTER 9	Analyzing Simulation Activity	
	Introduction	243
	Chapter Organization	244
	Defining Different Groups at Each Design Phase	244
	Understanding the Design Flow with Vector Analysis.....	245
	Creating Analysis Graphs	245
	Determining the Type of Vector Analysis to Run.....	245
	Running Vector Analysis Using Command Files.....	246
CHAPTER 10	Analyzing Simulation-Based Average Power	
	Introduction	249
	Overall Design Flow	249
	Chapter Organization	249
	Running a Power Analysis in Full Simulation Mode.....	250
	Controlling Your Average Power Analysis.....	250
	Sample CalculatePower Specification for Full Simulation Mode.....	252
	Running Analysis with Incomplete Simulation Data	253
	Performing Gate-Level Average Power Analysis	253
	Analyzing Large Gate-Level Designs	253
	Running Modal Analysis.....	253
	Understanding the Basics of the Detailed Power Report	256
	Available Power Report Formats.....	257
	Controlling the Contents of the Power Report.....	257
	Analyzing Average Power Using a SAIF File	269
	Flow Details.....	269
	Understanding Power Value Fluctuations Between SAIF and Other Formats ...	270
	Analyzing Average Power Using Partial Stimulus Files	270
	Name Mapping Flow	271
	Advanced Flows	273
	Re-Using a Stimulus File from a Previous Run.....	273

CHAPTER 11 Analyzing Time-Based Power

Introduction	275
Overall Design Flow	275
Chapter Organization	275
Understanding the Inputs for a Time-Based Power Analysis	276
Controlling Your Time-Based Power Analysis	276
Required Options for all Time-Based Analyses	276
Additional Arguments Required for Gate-Level Only	277
Additional Arguments Required for RTL and Mixed Analysis Only	277
Setting Timing Windows for Time-Based Power Analysis	278
Running the Analysis	279
Understanding and Reviewing Outputs and Results of the Time-Based Analysis ..	280
Contents of the ASCII Report File	280
Generating and Viewing Waveforms	281
Measuring the Distance Between Two Data Points	282
Using Results	285
Using Peak Power or Current Information in CoolTime	285
Monitoring Flop Clock Activity	285
Sample Flop Clock Activity Graphs	287

CHAPTER 12 Analyzing Vectorless Average Power

Flow Overview	291
Creating the VAF File	291
Setting the Frequency and Duty Cycle for Clock Nets	291
Setting the Frequency for Primary Inputs	292
Setting the Frequency for Ports of a Specific Type	292
Setting the Duty Cycle of Critical Control Signals that are Not Primary Inputs ..	292
Setting the Activity and Duty Cycle for Busses Driven by Tri-Stated Signals	293
Setting the Frequency and Duty Cycle on Leaf-Level Instances	293
Setting the Frequency for Memories	293
Handling Blackbox Instances	293
Understanding Common Warning Messages	294
Example 1: Mixing Commands or Using SetInstanceStimulus Commands	294
Example 2: Two Net Names that Resolve to the Same Net	294
Example 3: Conflicting Value Setting for the Same Object (Net/Port)	295

Example 4: Conflicts Caused by Wild Cards.....	295
CHAPTER 13 Examining and Implementing Power Reduction Opportunities	
Introduction	297
Chapter Organization	297
Recommended Flow for Implementing Power Reductions	298
Controlling Clock Domains During Reduction	299
Performing Power Reduction Analysis.....	301
Viewing Reduction Results	301
PowerBot Overview.....	302
Running PowerArtist Clock PowerBots	303
Generating Synthesis Constraints.....	304
Power Reduction PowerBots	309
Low-Activity Non-Enabled Register.....	309
Datapath Operator Isolation.....	311
Local Explicit Clock Enable	313
Split Memory Words (Beta)	318
Gate Memory Clock	326
Prism	328
Observability Don't Care	335
Power Linter PowerBots.....	339
Memory Power Linter	340
MUX Power Linter	341
Register Power Linter.....	348
Clock Enable Condition Linter.....	349
Rewriting Your RTL.....	351
Inputs.....	351
Flow for Rewriting the RTL.....	351
RewriteRTL Debug Switches	353
Directory Organization.....	353
Controlling the Format of Rewritten Code	354
Form of the Code Modifications for LNR	354
LNR Gate-Level Simulation	359
Form of the Code Modifications for GMC	359
Form of Code Modifications for Prism	362
General Form of Modifications for Different Code Types	366
Formal Verification of Rewritten Code.....	371
Errors, Warnings and Notes Generated by the Rewrite Process	374

	Running Hierarchical Rewrite.....	376
	Understanding the Hierarchical Rewrite Methodology	376
CHAPTER 14	Performing Functional Verification of RTL Changes	
	Introduction	381
	Using Standard Verification Tools by Defining ADS_PA_FV	381
	Generating a Rule File to use in the nCompare Utility (Beta)	382
	Performing Functional Simulation	386
CHAPTER 15	Analyzing the Effects of Power Gating with Proprietary Commands	
	Introduction	387
	Chapter Organization	387
	Required Inputs for Power Gating.....	388
	Defining Libraries	389
	Creating Source Files.....	390
	Special Option to the Elaborate Command.....	391
	Setting up Your Command File for Power Gating	391
	Defining Library Associations	391
	Defining Virtual Supplies	392
	Defining Power Domains.....	392
	Defining Retention Cell Mapping.....	392
	Sample Command File for a Power Gating Flow	392
	Performing Simulation-Based Power Analysis with Power Gating.....	393
	Performing Vectorless Power Analysis with Power Gating	393
	Creating a Vectorless Activity File	394
	Understanding the Output Reports for Power Gating Analysis	394
	Sample Report for Simulation-Based Analysis.....	394
	Sample Report for Vectorless Analysis	396
CHAPTER 16	Using Standard File Formats	
	Introduction	399
	Chapter Organization	399
	Using a CPF Input Flow (Beta)	400
	CPF Input Use Model	400
	Supported CPF 1.1 Commands	401
	Sample CPF Input	402

Sample PowerArtist Report Section for the CPF Flow	403
Using a CPF Output Flow (Beta).....	407
Mapping PowerArtist Tcl Commands to Generated CPF Commands	408
Quick Reference Checklist for CPF Output Flow	412
Sample Input and Output for a CPF Flow	414
Using a UPF Input Flow (Beta)	417
UPF Input Use Model	417
Supported UPF Commands and Options	418
Sample UPF Input	419
Sample PowerArtist Report Section for the UPF Flow	419
Using an SDC Input Flow	421
Using ReadSDC Output with a PowerArtist Command File	421
Mapping SDC commands to PowerArtist Tcl Commands	422
Sample SDC Input File	423
Sample Output Files	423

CHAPTER 17 **Acquiring Simulation Data**

Introduction	425
Chapter Organization	426
Using an FSDB Approach	426
Improving FSDB Performance	427
Improving FSDB Capacity	428
Generating a VCDe File from an FSDB File.....	428
Using the Standard VCD Approach	429
Writing a VCD File from a Verilog Simulator	429
Writing a VCD File from ModelSim.....	429
Writing a VCD File from the Cadence NC-Sim Simulator	430
Creating a Named Pipe to Manually Compress VCD Files	430
Using an IAF Approach	431
Acquiring Simulation Data in Palladium Flows	432
Troubleshooting Tips.....	434
Missing 'timescale in Verilog	434
Zero Length activities.iaf File.....	434
Problems with ModelSim	434
Zero Delay Simulation	435

CHAPTER 18 Generating Etcl Files for CoolTime

Introduction	437
Chapter Organization	437
Generating an Activity-Based Etcl File	438
Performing a Vector Analysis	438
Analyzing the Waveform	439
Controlling the Precision of the Data in the Etcl File	439
Generating the Etcl File	439
Using a Name Mapped Flow	440
Generating a Power-Based Etcl File	442
Automatically Selecting the Highest Activity Clock Cycle	442
Manually Selecting Single or Multiple Clock Cycles of Interest	443

CHAPTER 19 Writing OpenAccess Database Applications

Introduction	445
Chapter Organization	445
Introduction to OADB Programming	446
PowerArtist Netlist Properties	447
Instance Properties	447
Net Properties	448
Pin Properties	449
Module Properties	449
Hierarchical Instance Properties	450
Writing a Native OpenAccess Application	451
Contents of the Documentation Package	451
Basic Requirements for a Native OpenAccess Tcl Application	452
Flow for Writing a Native Tcl Application	452
Using a Script	453
Using the PowerArtist API to Write an OpenAccess Application	455
Getting Started	455
Using the Design Navigation Utilities	457
Using the Netlist Traversal Utilities	458
Using the Design Query Utilities	459
Using the Analysis Reporting Utilities	460
Using the Reduction Reporting Utilities	460
Using the Power Database Mapping Utilities	461

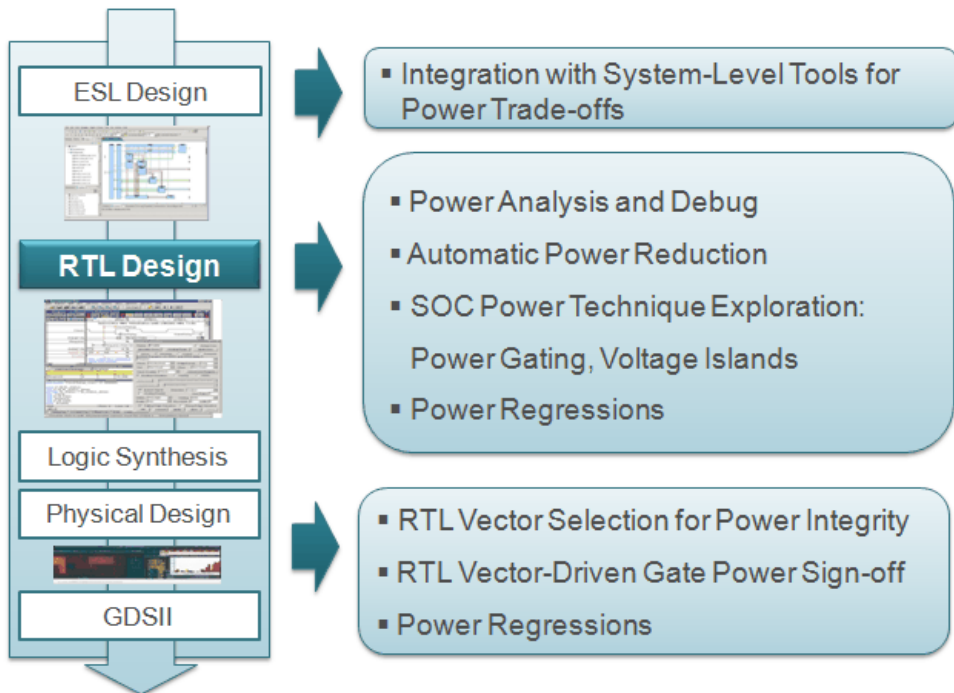
INDEX 463

Chapter 1

Introduction to PowerArtist

PowerArtist and Your Design Flow

PowerArtist is a complete RTL Design-For-Power (DFP™) environment with fully-integrated advanced analysis and automatic reduction. The following diagram illustrates how PowerArtist fits into your design flow.



Some of the highlights of the PowerArtist environment include:

- **RTL and Gate-Level Full-Chip Power Analysis**

You can analyze designs with multiple power supplies, mixed Verilog-Verilog2001-VHDL descriptions and embedded IP models. Power analysis includes both average power analysis and time-based analysis. For detailed information on

running these features, see [Chapter 10, Analyzing Simulation-Based Average Power](#) and [Chapter 11, Analyzing Time-Based Power](#).

- Automatic Analysis-Driven RTL Power Reduction

PowerArtist includes a comprehensive set of power reduction techniques, called PowerBots, that you can use to perform clock, memory and datapath power reduction. For complete details on the available types of power reductions, see [Chapter 13, Examining and Implementing Power Reduction Opportunities](#).

- Activity/Vector Analysis

PowerArtist's vector analysis capability allows you to visualize activity for an entire test suite, for any combination of modules in the design to quickly identify coverage problems and unexercised power modes. You can select small, power-intensive time slices for detailed power verification. For details on using vector analysis, see [Chapter 9, Analyzing Simulation Activity](#).

- An OpenAccess Database

PowerArtist stores power information in an OpenAccess database. The power information stored includes netlists, power reduction results and properties that represent calculated or extracted power data associated with the nets, instances and pins of your design. Additionally, PowerArtist provides a set of API that you can use to write an OpenAccess application. For more information, see [Using the PowerArtist API to Write an OpenAccess Application](#).

Available Tutorials to Get You Started

If you are a new user of PowerArtist, you might want to run through the analysis and reduction tutorials that are of interest to you.

- Available Analysis Tutorials

You can choose from any of the following analysis tutorials:

- [Running an RTL Full-Chip Power Analysis Using Command Files](#)
- [Running Full-Chip Analysis Using the PowerCanvas Wizards](#)
- [Running an RTL Block-Level Power Analysis](#)
- [Running Power Analysis with Clock Gating](#)
- [Running a Power Analysis with a Mixed-Vt Cell Library](#)
- [Running Power Analysis with Voltage Islands](#)
- [Running a Power Analysis with Power Gating](#)
- [Running the Full-Chip Gate-Level Power Analysis Tutorial](#)

- Available Reduction Tutorials

- [Running the Reduction Tutorial Using Command Files](#)
- [Running the Reduction Tutorial Using the PowerCanvas Wizards](#)

The tutorial files are available in the tar kit (POWERTHEATER_ROOT/tutorial).

Chapter 2

Installing and Setting Up PowerArtist

Introduction

This chapter provides instructions to enable you to install the software and have access to the PowerArtist application. You will need to perform the following three steps:

1. Install the software on one of the supported operating systems.
2. Install and point to the appropriate licenses.
3. Set a critical environment variable to point to the installation directory (and modify the PATH environment variable).

Supported Operating Systems

PowerArtist is supported on the following operating systems/platforms:

- 32-bit Red Hat Enterprise Linux (RHEL) 4.0 and 5.0
- 64-bit: RHEL 4 and 5 on Opteron and Xeon; Suse 9 and 10 on Xeon

If you have a relatively large design you should use a 64-bit platform. If you are not able to synthesize your design on a 32-bit Linux system, you should strongly consider using a 64-bit machine for your power analysis and reduction runs.

Installing the Software

To install the software on any supported OS, perform the following steps:

1. Decide where you want to put the distribution and change your working directory to that directory. The subdirectory `sequence/version/PowerTheater` will be created within your current working directory.
2. Uncompress and extract the software distribution by typing the following command. Note that the name of the zipped tar file is platform-dependent.

```
gunzip -c PowerTheaterversion.platform.tar.gz | tar xf -
```

Note that you can install multiple platform distributions in the same directory, as further discussed in the following section.

Distribution Tree

The Apache PowerArtist distribution is organized so that executables for multiple platforms can be installed in the same location. Therefore, platform-specific files are kept under directories with the following platform-specific names:

- `linux`—for 32-bit Linux, RHEL 4 and RHEL 5
- `linux-x86_64`—for 64-bit Linux for Opteron, RHEL 4, RHEL 5 and SUSE 9 and 10

Note, however, that the `PATH` environment variable must include the `$POWERTHEATER_ROOT/bin` directory. It need not have any platform-specific path elements, because a platform-independent script will invoke the correct executable.

The PowerArtist distribution consists of the following directories:

- `bin`—contains the executables. Set your `PATH` to include this directory.
- `doc`—contains the PDF documentation and the release notes.
- `examples`—contains some user examples as well as some examples oriented toward library developers.
- `lbin`—contains administrative programs that you will not want in your `PATH`. This includes the `FLEX/m` administration utilities and the PowerArtist license server.
- `lib`—contains library files needed by the power tools.
- `misc`—contains the source to the `wwvmkr` VHDL makefile program.
- `pthdl_src`—contains VHDL source files for the IEEE, STD, Synopsys and Vital libraries for the HDL analyzer, `pthdl` (run by the `Elaborate` command).
- `scripts`—contains Tcl files that implement all OADB utilities (for example, `reportPower`, `reportCGEfficiency`, etc.).
- `sfl_lib`—contains the SFL model file (`sfl_lib.dat`) and generic technology library information.
- `tutorial`—contains files for running both command-line and GUI-based tutorials in PowerArtist.
- `vhdl_src`—contains VHDL source files for the IEEE, STD, Synopsys and Vital libraries for the legacy VHDL analyzer, `wwvhdl`.

PowerArtist Licensing

This section includes the procedure for installing licenses and provides information on various licensing topics, including troubleshooting tips. Note that the software version number (for example, 2011.2.1) used in the following examples will change

with every software release. Furthermore, these examples assume that you installed the PowerArtist distribution tar kit in the /pkg directory.

Installing Licenses and FLEX/m

Use the following process to install licenses and FLEX/m® from Flexera:

1. Obtain a license file (license.dat) from Apache Design.
2. Install the license.dat file.

If you have other FLEX/m-licensed software, you may want to append the license.dat file to your existing license.dat file. If you do not, then simply install it into the /pkg/sequence/version/PowerTheater/lib/ directory.

3. Set the LM_LICENSE_FILE environment variable to point to the installed license.dat file, for example:

```
setenv LM_LICENSE_FILE /pkg/sequence/2011.2.1/PowerTheater/lib/license.dat
```

4. If you have a demo license, the installation is complete.

You have a demo license if the string “DEMO” appears in the license.dat file.

5. Edit the license.dat file to specify the full pathname to the vendor daemon. Replace the path specified in the DAEMON line with the following:

```
/pkg/sequence/2011.2.1/PowerTheater/lbin/apacheda
```

6. Start the license server by running the following command:

```
/pkg/sequence/2011.2.1/PowerTheater/lbin/lmgrd \  
-c /pkg/sequence/2011.2.1/PowerTheater/lib/license.dat \  
-l /pkg/sequence/2011.2.1/PowerTheater/lbin/lmgrd.log
```

Note that any user can do this—you don’t have to be “root”.

The -c option specifies the name of the license file.

The -l option specifies the name of the log file.

Understanding License File Entries

A typical license file entry looks like this:

```
FEATURE sptPowerGating apacheda 2012.10 15-feb-2012 20 \  
START=27-apr-2011 SIGN="0320 733C 1702 CC6B A286 A8B0 DB8A \  
9839 82CC 4E33 7903 E64F 0A34 744F 8A17 929D BB22 FDE2 C415 \  
C3BF 8BFC"
```

The first number, 2012.10, is the last supported build date for programs using this feature. In this case, all programs built on or before October, 2012 will be allowed to check out this feature. This is often referred to as the “maintenance end date”, indicating that no *new* software built after this date will run with this license feature.

The second date, 15-feb-2012, is the last day this feature can be checked out. In this case, no programs will be allowed to check out this feature after February 15, 2012. This is often referred to as the expiration date.

Support for Multiple License Servers

PowerArtist supports multiple FLEX/*m* license servers. See the FLEX/*m* documentation for details on how to set up multiple license servers.

Product vs. Feature Licenses

PowerArtist licenses are categorized into product licenses and feature licenses. Product licenses “own” feature licenses. This means that you can perform power analysis steps in parallel only if you have multiple product licenses.

Waiting for a Feature License

If you do not have a required feature license available to you immediately, you can instruct the software to wait for a license to become available. This feature is controlled by the `-wait_for_license` option to most of the Tcl commands. For example:

```
Elaborate -wait_for_license true
```

will wait for the license that controls elaboration to become available rather than exiting when one is not immediately accessible.

On the product level, the ptshell also has an option to wait for a product license (such as an SOC or an -artist license, for example:

```
ptshell -SOC -wait
```

waits for the SOC product license to become available rather than exiting immediately with a message indicating that a license is not available.

Resolving Feature Checkout Problems

If you get random license checkout problems, change your license file’s port number to something else in case there is a conflict with another vendor’s daemon.

Example

```
SERVER hostname 000f1f64dc32 7788
```

In this example, 7788 is the port number.

Setting the UNIX Environment Variable for PowerArtist

To configure PowerArtist for your use, you (and all other users) need to set the `POWERTHEATER_ROOT` environment variable to the proper installation directory.

On the UNIX command line, set up your environment by typing the following:

```
setenv POWERTHEATER_ROOT /your_path/version/PowerTheater  
set path =($POWERTHEATER_ROOT/bin $path)
```


Chapter 3

Using the PowerArtist Shell

Introduction

The PowerArtist shell (ptshell) is a fully functional Tcl shell, from which all other PowerArtist commands must be invoked. You can use any standard Tcl command inside of this shell. When you start up the shell, you will be informed as to which product you are running, and then receive a ptshell prompt. Once you get the prompt, you can run any of the PowerArtist commands.

Understanding the ptshell Command Options

This command launches the PowerArtist Tcl shell (ptshell). You can source Tcl scripts or type a sequence of commands from within the ptshell. Use the “exit” command to close the ptshell.

Syntax

```
ptshell [-artist | -SOC] [-cmd tcl_command] | [-tcl tcl_file]
        [-wait]
```

Options

-artist

Consumes a license at the PowerArtist level. You will get a ptshell prompt with this option.

-SOC

Consumes a license at the SOC level. You will get a ptshell prompt with this option. This is the default. If you have no sptSOC license and you have specified neither the -artist or -SOC options, ptshell will attempt to consume an sptArtist license.

-cmd *tcl_command*

Executes the specified Tcl command and then exits with the result.

`-tcl tcl_file`

Sources a file containing Tcl commands and then exits with the result. These commands may also include PowerArtist commands.

`-wait`

Waits for the appropriate product license to become available rather than exiting immediately with a message indicating that a license is not available.

Example 1

`ptshell`

This command invokes the PowerArtist shell. At the resulting ptshell prompt, you may type any Tcl command or PowerArtist command. This does not specify a license level so -SOC is assumed. Therefore, you will be able to execute all PowerArtist-PT commands. You will not, for example, be able to perform a power reduction analysis.

Example 2

`ptshell -artist`

This command invokes the PowerArtist shell. The -artist option allows you to run all PowerArtist features (including power reduction analysis and RTL rewrite).

Example 3

`ptshell -artist -tcl run_Elaborate.tcl`

This consumes licenses at the Power Artist level and then runs the Tcl script Elaborate.tcl. A sample Elaborate.tcl file might be:

```
source ptSourceFiles.tcl
Elaborate -scenario_file my.scn \
  -top top \
  -synlib_files {mylib.lib}
```

When the [Elaborate](#) command completes, the ptshell will be exited.

Example 4

`ptshell -artist`

`ptshell % source Elaborate.tcl`

This series of commands does the same thing as the command in example 3. except that you will remain in the ptshell when the Elaborate script is complete. that it does not exit the ptshell.

Example 5

```
ptshell
ptshell % source ptSourceFiles.tcl
ptshell % Elaborate -scenario_file my.scn -top top -synlib_files
           {mylib.lib}
ptshell % exit
```

When you type ptshell followed by a carriage return, you get a ptshell prompt, which defaults to “ptshell %”. You can type a sequence of commands at this point.

Example 6

```
ptshell
ptshell % help command
```

As in example 5, when you type ptshell followed by a carriage return, you get a ptshell prompt. You can then type “help command” at the ptshell prompt to print a list of commands that you can use in a PowerArtist command file/Tcl script.

Example 7

```
ptshell -cmd "CalculatePower -help" > cphelp
```

This prints help information for the [CalculatePower](#) command. It is first sent to stdout which is then re-directed to a file named “cphelp”. Whenever you want to run a Tcl command from the ptshell, you must enclose the command in quotes as shown here; otherwise you will get an error.

Invoking the GUI from the ptshell

You can invoke the PowerCanvas GUI from ptshell using the PowerCanvas command:

```
ptshell % PowerCanvas
```

You can optionally use the -pdb option to specify the power database to load:

```
ptshell % PowerCanvas -pdb top.pdb
```

You can also bring up the GUI from the UNIX command line as shown in the following examples.

Example 8

```
PowerArtist-PT
```

This is the recommended way to invoke the PowerArtist-PT PowerCanvas if you want (or have) only the analysis capabilities of the PowerCanvas GUI.

Example 9

PowerArtist

This is the recommended way to invoke the PowerArtist PowerCanvas with full analysis and reduction capabilities.

Reverting to the Legacy Shell

If you want to use the legacy ptshell, you need to specify the PT_DISABLE_APSH environment variable:

```
setenv PT_DISABLE_APSH 1
```

This setting reverts to the legacy ptshell.

ptshell Features

The ptshell is a fully functional Tcl shell from which you can invoke any standard Tcl command. In addition, ptshell supports a set of TCSH-like features for command editing, file completion, and more. The ptshell has the following features:

- Command-line help for command file commands.
 - You can type “help *command_name*” such as “help CalculatePower”. You can specify a partial command name with standard glob-style wild cards such as “help Calc*” or “help *db*”.
 - You can type “*command_name* -help” such as “Elaborate -help” in the ptshell to get a list of options for the [Elaborate](#) command.
 - You can generate an alphabetical list of all accepted PowerArtist Tcl commands (or pt_set variables) by typing “help command” (or “help variable”) at the ptshell prompt.
- An auto-generated history file—As each PowerArtist command is executed, it is echoed into a ptshell.history file in your current working directory. This allows you to run Tcl programs that execute commands that are then captured in the history file. You can then source this file (using the Tcl source command) to re-run the exact sequence of commands.
- Command/file name completion with the Tab key—the shell will expand a command to the next unique character and provide a list of all possible commands/file names to complete the entry.

- Command-line editing—use the left and right arrows, control characters and escape sequences to edit the current command on the command line. The following table lists the keystroke combinations and their associated actions.

Key/Key Combination	Action
Ctrl-a	Goes to the beginning of the line
Ctrl-b	Moves back one character
Ctrl-d	Deletes current character
Ctrl-e	Goes to end of the line
Ctrl-f	Moves forward one character
Ctrl-k	Yanks and deletes until the end of the line
Ctrl-n	Gets the next command from history
Ctrl-p	Gets the previous command from history
Ctrl-y	Pastes the yanked buffer
Esc-b	Moves back one word
Esc-f	Moves forward one word
Left arrow key	Moves back one character
Right arrow key	Moves forward one character
Up arrow key	Gets the previous command from history
Down arrow key	Gets the next command from history

- Recalling commands from history—use the up and down arrows to scroll through commands that were previously executed in the current session. By default, 20 commands are saved in history. Use “history keep 50” to increase the history size to 50.
- An initialization file (.ini) file that allows you to customize your ptshell run (see the next section for details).

Customizing Your PowerArtist Environment Using Initialization (INI) Files

You can use .ini files to customize your ptshell setting at multiple levels. You can use this file to change the default names of the log file and the key file (see, [Understanding Log Files and Key Files](#) for details). You can also specify a custom ptshell prompt. You can place the .ini files in different locations to customize at

different levels. The shell sources the .ini files in the following order (the last one taking precedence over previously read files):

1. \$POWERTHEATER_ROOT/ptshell.ini (default .ini file)
This default .ini file is read first. Further .ini files (if present) will modify the settings in this file.
2. \$HOME/ptshell.ini file, if present, is read next.
You can use this .ini file to customize the shell for you as a user.
3. \$PWD/ptshell.ini file, if present, is read next.
You can use this .ini file to customize the shell for a particular run.
4. The file pointed to by the \$PTSHELL_INI variable, if present, is read last.
You will typically use this .ini file to customize the shell for a particular project.

Sample Initialization File

The following sample initialization (.ini) file includes ptsh_* variables that you can use to customize your PowerArtist shell.

```
#+++++
# Sample Initialization File
#+++++
set ptshLogFile my_ptshell.log
set ptshKeyFile my_ptshell.key
set ptshPrompt "my_ptshell %"
#+++++
```

Understanding Log Files and Key Files

When you run ptshell, you will always get a .key file and a .log file that capture session information in different ways:

- ptshell.log
This file captures all tool outputs for your current session. This includes commands run, output generated (including the content of each of the individual *command_name.log* files), the contents of any sourced files, etc. You can change the name of this file using the ptshLogFile Tcl variable in your .ini file.
- ptshell.key
This file captures commands executed from the ptshell. You can source this key file to re-run the exact sequence of commands from the recorded session. You can change the name of this file using the ptshKeyFile Tcl variable.

Understanding the Link Between Commands and the PowerCanvas

You can invoke the PowerCanvas, which is the PowerArtist graphical user interface, by specifying different commands to ptshell (see examples 3-5). As you will see in later chapters, the PowerCanvas contains the following major blocks of functionality:

- Wizards that guide you through a process to perform major tasks such as HDL elaboration, vector analysis, power calculation or power reduction. They help you set up the various task options and then execute one or more of the key PowerArtist commands.
- Features that help you diagnose power bugs in your design and understand the output of various automated reduction algorithms.

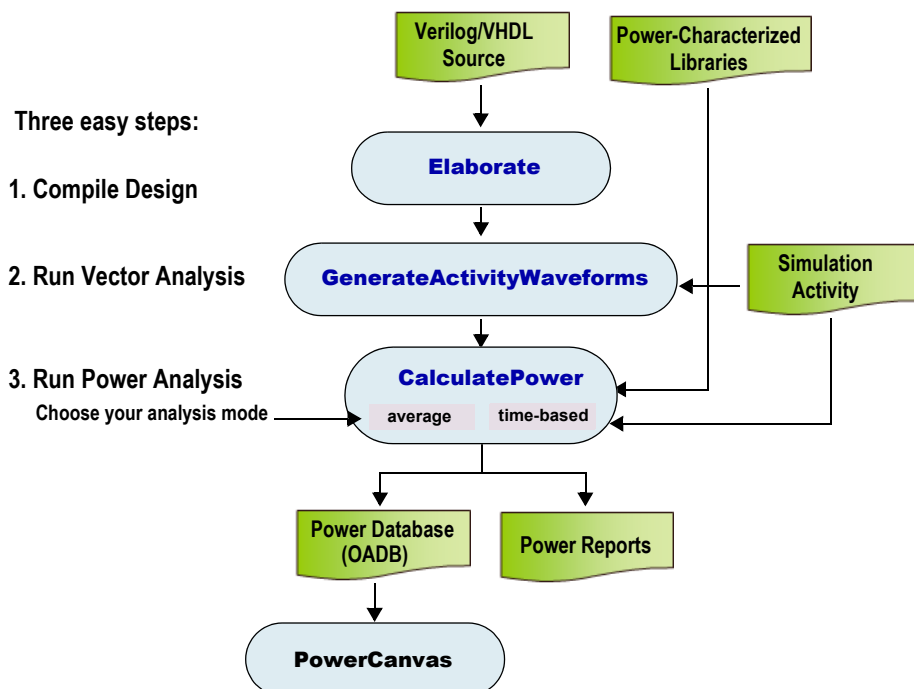
In this way, the PowerArtist commands are closely linked to the PowerCanvas wizards.

Chapter 4

PowerArtist Tutorial Part 1: Power Analysis

Introduction

This tutorial shows you how to run both average and time-based power analysis using a batch flow and a PowerCanvas flow. It includes the following types of analyses: full-chip, block-level, clock gating, mixed-vt, voltage_islands, power gating and gate-level full chip. Running power analysis is a simple three-step process, as shown by the following graphic:



The main steps in this power analysis flow are controlled by the following commands:

- **Elaborate**—creates the scenario file for Verilog, VHDL and mixed-language designs.
- **GenerateActivityWaveforms**—analyzes the activity file and produces waveform files representing the activity in your design.
- **CalculatePower** -analysis_type average—performs an average power analysis.
- **CalculatePower** -analysis_type time_based—performs a time-based power analysis.

There are three goals to the tutorial:

- To introduce you to the PowerArtist command file setup.
- To introduce you to the PowerCanvas GUI.
- To present sample power-aware design flows for different types of analysis.
- To guide you through interpreting the results.

Note that this tutorial does not cover power reduction. To run the power reduction tutorial, see the [PowerArtist Tutorial Part II: Power Reduction](#).

Tutorial Organization

The following topics are covered in this tutorial:

- [Copying the PowerArtist Tutorial Files](#)
- [Understanding Command File Set-Up Methods](#)
- [Running an RTL Full-Chip Power Analysis Using Command Files](#)
- [Viewing Power Analysis Results in the PowerCanvas](#)
- [Running Full-Chip Analysis Using the PowerCanvas Wizards](#)
- [Running an RTL Block-Level Power Analysis](#)
- [Running Power Analysis with Clock Gating](#)
- [Running a Power Analysis with a Mixed-Vt Cell Library](#)
- [Running Power Analysis with Voltage Islands](#)
- [Running a Power Analysis with Power Gating](#)
- [Running the Full-Chip Gate-Level Power Analysis Tutorial](#)
- [Understanding Power Analysis Results in the Text-Based Power Report](#)
- [Summary](#)

Copying the PowerArtist Tutorial Files

There are two tutorials in the /tutorial directory. It is recommended that you copy the entire contents of the tutorial directory to a local run directory:

```
cp -r $POWERTHEATER_ROOT/tutorial .
```

The analysis design is a mixed-language RTL design of a 64-bit PCI controller. The bulk of the design is captured in Verilog with one file, representing the fifo controller, captured in VHDL.

Contents of the Top-Level Tutorial Directory

The following table lists the contents of the entire tutorial directory:

Directory Name	Description
analysis	Contains the analysis tutorial files. These tutorials teach you how to perform HDL inferencing, vector analysis and average and time-based power analysis at both the RTL and gate levels of abstraction.
libraries	Contains all of the technology libraries required to run the tutorials.
settings/ global_setting.tcl	This file illustrates how you can establish site-wide tool settings that are shared among users. These settings are leveraged in all the tutorials.
reduction	Contains the reduction tutorial files. This tutorial teaches you how to perform an RTL power reduction analysis. This tutorial is not described in this chapter. For instructions on running the reduction tutorial, see PowerArtist Tutorial Part II: Power Reduction .

For the analysis tutorials, you will need all but the /reduction directory.

Contents of the Analysis Directory

The analysis directory contains a number of sub-directories that each run a different type of analysis tutorial as shown in the following table.

Directory Name	Description
full_chip	Runs a full-chip RTL tutorial that shows how to run all of the critical analyses in PowerArtist. You will be running simulation-based analyses here.
block	Runs a tutorial on a sub-block of the full-chip design in the full_chip directory. You will be performing a vectorless analysis here.
clock_gating	Shows you how to run clock gating analysis using the full_chip design.

Directory Name	Description
mixed_vt	Shows you how to run analysis using mixed VT using the full_chip design.
voltage_islands	Shows you how to run multiple voltage-domain analysis using the full-chip design.
power_gating	Shows you how to run power gating analysis using the full-chip design.
full_chip_gate	Shows you how to run a gate-level analysis on the full-chip design, which has been synthesized to gates.

It is recommended that you run these tutorials in the order they are listed in this table. In addition to the tutorial sub-directories, there is also a `design_data` directory that contains design data that is used by all of the analysis tutorials. It includes both gate and RTL source files and simulation data.

Understanding Command File Set-Up Methods

Before starting this tutorial, it's important that you understand the ways in which you can set up your Tcl command files. It is recommended that you specify the command file commands with command-line options that determine the inputs for the commands, the type of analysis they run and the output files generated.

Recommended Method Sample Script

```
# Elaborate Design
Elaborate -top top -elaborate_write_power_db true -blast_regfile all
```

This script runs the `Elaborate` with the `-top`, `-elaborate_write_power_db` and `-blast_regfile` options specified. Alternatively, you could specify these command options as standalone `pt_set` variables. You would then specify the command without options. The following command file excerpt uses this set-up method. You would specify these variables before the command to which they apply.

Alternative Method Sample Script

```
# Elaborate Design
pt_set top top
pt_set elaborate_write_power_db true
pt_set blast_regfile all
...
Elaborate
```

In this sample file, each of the `pt_set` variables specified before the `Elaborate` command determine how the command will run. The `Elaborate` command is then called last. Using this method, you can specify one `pt_set` variable that affects multiple commands.

Note that the `pt_set` variable names are the same as the command options but without the preceding dash (-), therefore, “-blast_regfile” becomes “`pt_set blast_regfile`”. For a complete list of `pt_set` variables, see [Alphabetical List of `pt\_set` Variables](#) in the *PowerArtist Reference Manual*.

Running an RTL Full-Chip Power Analysis Using Command Files

To begin the full-chip tutorial, change directories to your local `full_chip` tutorial directory, for example:

```
cd tutorial/analysis/full_chip
```

The following table lists the files in this directory.

File Name	Description
README	Provides the basic flow for running this tutorial.
ptshell.ini	Initialization file specific to the full-chip tutorial. This file sets the <code>ptshell</code> prompt and specifies the names of the <code>.log</code> and <code>.key</code> files for this tutorial.
<i>command.tcl</i> files	Tcl scripts meant to be sourced from within the <code>ptshell</code> that perform various tasks related to power analysis (for example, design elaboration, vector analysis, average power analysis, etc.) These are identical to the operations you perform using the interactive wizards.
ptSourceFiles.tcl	Contains a list of VHDL libraries and VHDL files in your design in the order in which they must be analyzed and elaborated. It consists of AddLibrary and CompileFile commands. For details on this file, see ptSourceFiles.tcl File Format .
project_settings.tcl	Sources a global Tcl file containing site-wide tool settings and the <code>libraries.tcl</code> file containing library information. Using a single file like this to source other files will save time and reduce keystrokes. This file is also specified in the HDL Inferencing wizard.
power_aware.tcl	Contains the power-aware commands required to run the tutorial. This file contains all of the commands that constrain the power analysis of the design. Constraints can include: -voltage and power domains -mixed VT -clock gating -the libraries to be used for each sub-block of the design

File Name	Description
analysis_wizard.tcl	This file is used by the power analysis wizards to source the project_setting.tcl file and the power_aware.tcl file.
clock.tcl	Contains commands that define the clocks in the design.
txrx.vc	Verilog startup file for the full chip that tells the HDL elaborator which Verilog files must be loaded for your design.
libraries.tcl	Contains commands that define your Liberty libraries.
/.ads_power_config	This hidden directory contains a persistent representation of what you enter in the PowerCanvas wizards. The next time you invoke the same wizards, all of the data you previously entered will be pre-loaded in the forms. This allows you to rapidly perform similar operations.
run_Elaborate.tcl run_VectorAnalysis.tcl run_CalculateAveragePower.tcl run_CalculateTimeBasedPower.tcl	Run scripts to invoke from the Unix shell. As an alternative method, you could run the tutorial by typing the following commands at the Unix prompt: ptshell -tcl run_Elaborate.tcl ptshell -tcl run_VectorAnalysis.tcl ptshell -tcl run_CalculateAveragePower.tcl ptshell -tcl run_CalculateTimeBasedPower.tcl

Examining the Contents of the Tcl Command Files

1. Open the ptshell.ini file. This file contains shell initialization settings for the full-chip tutorial.

```
set ptshKeyFile full_chip.key
set ptshLogFile full_chip.log
set ptshPrompt "full_chip_ptshell %"
```

As you run the tutorial, PowerArtist generates a key file and log file (both with a “full_chip_” prefix) in your run directory. These files are useful for debugging or recreating a previous run. For more information on these files, see [Understanding Log Files and Key Files](#). In addition, the ptshell prompt for this tutorial will have an “full_chip_” prefix. Note that this .ini file overrides the default .ini file that comes with the tarkit. For more information on the initialization file, see [Customizing Your PowerArtist Environment Using Initialization \(INI\) Files](#).

2. Open the project_settings.tcl file. This file is shown here:

```
# Obtain global site-wide settings
source ../../settings/global_settings.tcl

# Specify the technology libraries
source libraries.tcl
```

This file obtains site-wide tool settings (from the global_setting.tcl file) and reads-in library information (from the libraries.tcl file). The libraries.tcl file contains a series of [ReadLibrary](#) commands. Creating this type of file saves you time and requires fewer keystrokes.

3. Open the power_aware.tcl file. This file contains different setting depending on the type of analysis you are running. For the full-chip tutorial, this file contains clock gating commands and MonitorInstances commands. An excerpt from this file is shown here:

```
#####
# Clock Gating Commands

SetClockNet -name top.clk -mode infer -gate_clock yes
SetClockNet -name top.pci_clk -mode infer
SetClockNet -name top.tck -mode infer
SetClockGatingStyle -clock_cell_attribute latch_posedge_precontrol
SetClockBuffer -type root -name SEQCLKBUF4MTH -library hvt -fanout 2
SetClockBuffer -type branch -name SEQCLKBUF8MTH -library hvt -fanout 2
SetClockBuffer -type leaf -name SEQCLKBUF8MTH -library hvt -fanout 60

#####
# Instances to monitor in the design

MonitorInstances -name top.core1.t1 -all 1
MonitorInstances -name top.core1.r1 -all 1
```

4. Make special note of the following information in this Tcl file:
 - The [SetClockNet](#) command defines the three clocks in the design and specifies that clock gating be performed (-gate_clock yes) for the clock named “top.clk”. For top.pci_clk it specifies “-mode infer” tells PowerArtist to infer to the clock tree in this design. This is essential to achieve accurate full-chip RTL Power Analysis.
 - The [SetClockGatingStyle](#) command specifies the value of the clock_gating_integrated_cell attribute, which in this case latch_posedge_precontrol. During the clock inferencing step of any power analysis, PowerArtist searches for this attribute to determine candidate clock gating cells.

- The [SetClockBuffer](#) commands specify three types of clock buffers: root, branch and leaf. when you want to infer a clock tree. These are required when you want to infer a clock tree.
 - The [MonitorInstances](#) command tells PowerArtist to generate the power report that lists all of the children instances for top.core1.t1 and top.core1.t2.
5. Open the Elaborate.tcl file. This script compiles and elaborates the design using the [Elaborate](#) command. It generates a scenario file which is a binary representation of the elaborated design. This file is shown here:

```
# Elaborate the design
set design top

# Read in the design
source ptSourceFiles.tcl

Elaborate \
    -elaborate_write_power_db true \
    -elaborate_log ElaborateBatch.log \
    -power_db_name $design.Batch.pdb \
    -scenario_file $design.Batch.scn \
    -top $design \
    -verilog_2001 true \
    -verilog_startup_file txrx.vc
```

6. Make special note of the following information in this command file:
- The “set design top” is a simple standard Tcl variable that sets the top module to “top”. This defined “design” variable is used in the rest of the script.
 - Since the full-chip design is a mixed design that has some Verilog and some VHDL, this script sources the ptSourceFiles.tcl and specifies the -verilog_startup_file option to the [Elaborate](#) command.
 - The -elaborate_write_power_db option writes out a power database (.pdb) file. When Elaborate runs, it creates this power database.

This power database is an OpenAccess™ database (OADB) that contains a hierarchical netlist representing your design as well as power analysis and reduction values. PowerArtist uses this database to generate hierarchical tree and schematic displays of your design, and to store power analysis and reduction information. You are able to write your own applications that access the netlist and power-related information. For more information, see [Writing OpenAccess Database Applications](#).
 - The resulting scenario file, as specified with the -scenario_file option, will be named top.Batch.scn (since \$design is set to “top”).

7. Open the `GenerateActivityWaveforms.tcl` file. The commands and variables in this file run full-chip vector analysis:

```
# Perform Vector Analysis

# Define the vector groups for the design

DefineGroup top { top }
DefineGroup core { top.core1 }
DefineGroup pci { top.core1.p1 }
DefineGroup rxchan { top.core1.r1 }
DefineGroup txchan { top.core1.t1 }

set design top

GenerateActivityWaveforms \
    -activity_file ../design_data/rtl_sim/activities.vcd \
    -activity_waveform_group_list { top core pci rxchan txchan } \
    -activity_waveform_graph_type frequency_per_interval \
    -activity_waveform_interval_size 15160ps \
    -activity_waveform_number_of_intervals 400 \
    -activity_waveform_start_time 6071580ps \
    -fsdb_output_file top_activity.fsdb \
    -log GenerateActivityWaveformBatch.log \
    -scenario_file top.Batch.scn \
    -top_instance txrx_tst.top1
```

8. Make special note of the following information in this command file:
 - Simulation data in the form of a VCD file is supplied as the `activities.vcd` file. Full-chip vector analysis requires some form of simulation data. You can use any one of three approaches to acquire simulation data: the Intermediate Activity File (IAF) approach, the standard VCD approach, and the FSDB approach. In the IAF approach, special-purpose code provided by Apache is linked into your simulator to create the `.iaf` file when your simulation is run. In the standard VCD approach, no special code is linked but standard VCD commands used with your simulator are used to produce the VCD file. In the FSDB approach, you generate an FSDB file using the Verdi™ product from SpringSoft to obtain activity information from your simulator of choice. FSDB files are excellent ways to store your activity data because they represent the information in a more compressed manner than other formats. See [Chapter 17, Acquiring Simulation Data](#) for detailed information on how to create the simulation activity files.
 - The `DefineGroup` command defines groups (top, core, pci, rxchan and txchan). These groups will have separate waveforms in the resulting FSDB file that captures the average activity of all the nets local to those instances as a function of time.

- The [GenerateActivityWaveforms](#) command performs the vector analysis. In this case, it sets the interval size to 15160ps, the number of intervals to 400 and begins the analysis at 6071580ps. The -activity_waveform_group_list option specifies the five groups created earlier by the DefineGroup command.
9. Open the CalculateAveragePower.tcl file (as shown below). The settings in this file run the full-chip average power analysis.

```
#Calculate the power of the design

set design top
CalculatePower -analysis_type average \
  -activity_file ../design_data/rtl_sim/activities.vcd \
  -average_report_file $design.AverageBatch.rpt \
  -average_report_options agip \
  -average_write_power_db true \
  -compress_gaf true \
  -default_output_load 3.9e-11 \
  -detailed_vertical_report true \
  -finish_time 12135580ps \
  -gaf_file $design.AverageBatch.gaf \
  -calculate_log CalculatePowerAverageBatch.log \
  -mode_file txrx.mode \
  -power_db_name $design.AverageBatch.pdb \
  -save_clock_trees_netlist true \
  -scenario_file $design.Batch.scn \
  -start_time 6071580ps \
  -top_instance txrx_tst.top1 \
  -use_scan_flops true \
  -vertical_report_instances $design \
  -wireload_library hvt
```

10. Make special note of the following information in this command file:
- The [CalculatePower](#) command is set to run average analysis (-analysis_type average). All of the -average* options are specific to this type of analysis.
 - Simulation data is provided by the activities.vcd file.
- The quality of the simulation data provided to PowerArtist is a major factor in the quality of the power analysis. Accurate power analysis for typical usage of a design requires typical simulation data. The CalculatePower command analyzes the simulation results and generates compressed summary data in the top.AverageBatch.gaf file. This file will then be used for power analysis in this run and subsequent runs of CalculatePower to save time from having to re-analyze the VCD file each time you run the software.
- It will generate a detailed vertical report of analysis results.
 - It will run a modal (-mode_file) analysis using the txrx.mode file. The design in this tutorial has different modes of operation and it useful to know the power

consumed in each mode. Typical modes of operation for chips are standby, sleep, receive, transmit, interrupt, etc. By getting a good understanding of the modal power of your chip, you can gain a better understanding of the battery needs for your applications. For details on the format of the mode file, see [Mode File Format](#).

- The log file that this analysis generates will be named `CalculatePowerAverageBatch.log`.
 - The `-save_clock_trees_netlist true` option generates power database schematics for the clock tree in the design. Without this setting, you will not be able to view your clock tree in the PowerCanvas.
 - The power database that this analysis generates will be named `top.AverageBatch.pdb`. This is the file you will load into the PowerCanvas.
 - The analysis looks for wire load models in the Liberty library with the logical library name `hvt`.
 - Scan flops will be used to perform analysis (this is the default behavior).
11. Open the `CalculateTimeBasedPower.tcl` file (as shown below). The settings in this file run the full-chip time-based power analysis. Many of these options are identical to those required to perform the average power calculation.

```
# Calculate the power of the design
set design top
CalculatePower -analysis_type time_based \
  -active_edge positive \
  -activity_file ../design_data/rtl_sim/activities.vcd \
  -fsdb_output_file $design.TimeBasedBatch.fsdb \
  -default_output_load 3.9e-11 \
  -finish_time 12135580ps \
  -calculate_log CalculateTimeBasedPowerBatch.log \
  -num_clock_cycles 400 \
  -power_db_name $design.TimeBasedBatch.pdb \
  -reference_clock top.clk \
  -save_clock_trees_netlist true \
  -scenario_file $design.Batch.scn \
  -start_time 6071580ps \
  -time_based_report_file $design.TimeBasedBatch.rpt \
  -time_based_write_power_db true \
  -top_instance txrx_tst.top1 \
  -use_scan_flops true \
  -wireload_library hvt
```

12. Make special note of the following information in this command file:
- The `CalculatePower` command is set to run a time-based analysis (`-analysis_type time_based`). All of the `-time_based*` options apply to this type of analysis.

- The simulation time from 6071580ps to 12135580ps will be broken up into intervals of 400 clock cycle blocks. Each 400 cycle block will result in an average power estimation therefore generating a power-over-time waveform. This waveform is saved in the top.TimeBasedBatch.fsdb file which you can view in the Apache Waveform Viewer.
- The log file that this analysis generates will be named CalculateTimeBasedPowerBatch.log.
- The power database that this analysis generates will be named top.TimeBasedBatch.pdb. This is the file you will load into the PowerCanvas.

Running the Tcl Scripts

Use the following steps to run the command file flow for the full-chip tutorial:

1. Type “ptshell” to start the PowerArtist shell. The prompt for this tutorial will be:

```
full_chip_ptshell %
```

As soon as the ptshell starts, you will see the following additional files in your run directory:

- full_chip.key—Records keystrokes run as part of this tutorial. You can re-run a session by sourcing this file.
- full_chip.log—Records log information (inputs and outputs) for this tutorial. capturing the output of standard out.

2. From the ptshell prompt, source the project_setting.tcl script to setup project-specific parameters and read your libraries.

```
full_chip_ptshell % source project_settings.tcl
```

Note: You can take advantage of the command completion feature of the ptshell. For example, you can simply type “source p” followed by the **Tab** key and to see a list of files that match. Type additional characters followed by **Tab** until the ambiguity is resolved. Using this feature will save you time and cut down the number of keystrokes required to run the command-line tutorial.

3. Source the Elaborate.tcl script to compile the design and create a PowerArtist scenario file:

```
full_chip_ptshell % source Elaborate.tcl
```

The ptshell can do limited checking on the correctness of the data. It can verify that files are readable. However, the arguments are not completely checked until the actual application gets spawned. As an example, if you specify a [CompileFile](#) command, the actual source file is not parsed until the [Elaborate](#) command is executed.

When the script is finished, you will see the following files in your run directory:

- ElaborateBatch.log—This file captures log information for the Elaborate command. If you get any unexpected results, you can look through this file to make sure that the settings listed are those that you expected to be set.
 - top.Batch.scn—This is a binary representation of the full-chip design. This scenario file is called later by the analysis scripts.
 - top.Batch.pdb—At this point in the flow, this PDB file contains the design hierarchy. You can use the PowerCanvas to explore this hierarchy at this point. The power analysis run will add to this file.
4. Exit the ptshell:


```
full_chip_ptshell % exit
```
 5. View and analyze the elaborated design in the PowerCanvas.
 - a. Invoke the PowerCanvas using either of the following commands:


```
% 1PowerArtist-PT -pdb top.Batch.pdb
```

or

```
% PowerArtist -pdb top.Batch.pdb
```

The hierarchy browser (at the top level) and the collapsed schematic of the elaborated design appears.

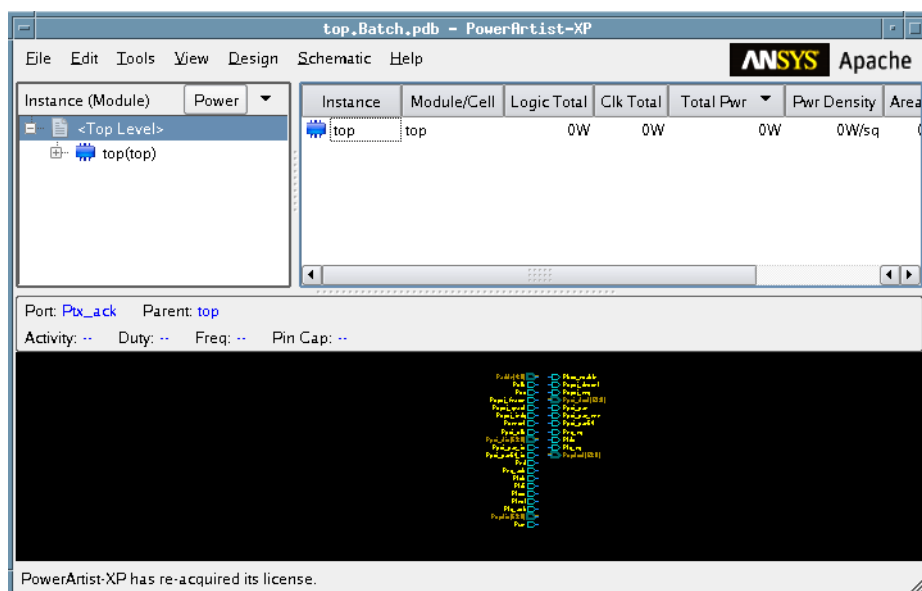


Figure 1 Display of top.Batch.pdb File (Elaborated Design)

1. If you are a returning PowerTheater user and you do not have a PowerArtist license, you will need to invoke the PowerCanvas GUI using the PowerArtist-PT command. Note that it does not have the full functionality of the PowerArtist license.

- b. Click the top(top) instance in the left-hand pane of the hierarchy browser. Click the + symbols next to the nested hierarchical instances to display their instances in the right-hand pane called the power table.
6. When you are finished examining the elaborated design, exit the PowerCanvas (select **File** > **Exit**).

7. Restart the ptshell:

```
% ptshell
```

8. Source the project_settings.tcl file again.

```
full_chip_ptshell % source project_settings.tcl
```

You must reload this file whenever you restart the ptshell in this tutorial flow.

9. Source the GenerateActivityWaveforms.tcl script to run vector analysis:

```
full_chip_ptshell % source GenerateActivityWaveforms.tcl
```

Vector Analysis shows how the activity varies over time for selected blocks in your design. Vector Analysis should be done before power analysis to ensure that the testbench has exercised the design as expected and can be used to identify a suitable time window to perform power analysis. Note that additional details are available in [Analyzing Simulation Activity](#).

When the script is finished, you will see the following files in your run directory:

- GenerateActivityWaveformBatch.log—This file captures log information for the GenerateActivityWaveforms command that controls vector analysis. If you get any unexpected results, you can look through this file to make sure that the settings listed are those that you expected to be set.
 - top_activity.fsdb—This is the FSDB file containing waveforms for the specified groups.
10. Exit the ptshell and restart the PowerCanvas GUI.
 11. Display the resulting waveforms from the vector analysis:
 - a. From the PowerCanvas main menu, select **Tools** > **Waveform Viewer**.
The Add Waveform Source menu appears.
 - b. Select **top_activity.fsdb** from the Files list on the right and click **OK**.
 - c. In the Waveform Name field, type an * to find all waveforms and click the **Search** button.
 - d. Select all of the waveform names, select "Place waveforms in: One plot" and click the **Add waveform to plot area** button.

The resulting waveforms show the average activity over time for the specified groups.

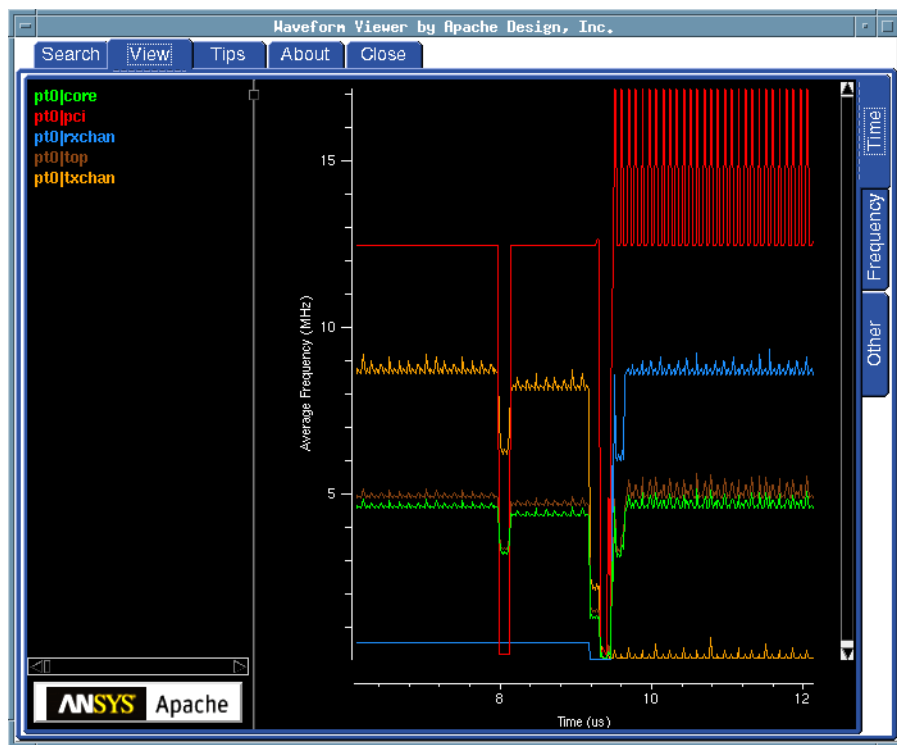


Figure 2 Vector Analysis Waveforms for Each Specified Group

Notice that the rxchan module (colored blue) has very little activity in the first half of the simulation then turns on. The inverse happens with the txchan instance (colored orange). This clearly indicates that the pci controller starts out transmitting packets of information and switches to receive packets. The average net activity in the controllers when the controller is either transmitting or receiving is about 9 MHz and less than 1 MHz when the chip is not in that mode of operation.

- Use the cursor to highlight an individual graph. Hover on the graph legend to minimize all but the selected graph. Hover on the graph itself to increase the thickness of the line.
- Move the cursor over a data point on a graph to display the coordinates.
- Hold down the middle mouse button and drag a red rectangle to zoom in to an area of interest.

- Right-click and hold anywhere on the waveform to display a pop-up menu. Use this menu to manipulate the waveform display. For details on all options available, see [Using the Waveform Viewer Options Menu](#).
 - Click the Tips tab at any point to see more tips on using the viewer.
12. Exit the PowerCanvas GUI and restart the ptshell.
 13. Source the project_settings.tcl file again.

```
full_chip_ptshell % source project_settings.tcl
```
 14. Source the power_aware.tcl file to read in the power constraints for this tutorial:

```
full_chip_ptshell % source power_aware.tcl
```
 15. Source the CalculateAveragePower.tcl script to run average power analysis:

```
full_chip_ptshell % source CalculateAveragePower.tcl
```

When the script is finished, you will see the following files in your run directory:

 - CalculatePowerAverageBatch.log—This file captures log information for the CalculatePower average run.
 - top.AverageBatch.pdb—This power database now includes average power analysis data.
 - top.AverageBatch.rpt—This is the a text-based power report. For small designs, you may find this format useful; for larger designs, you will want to use the PowerCanvas to review the results.
 16. Exit the ptshell.
 17. Review and analyze the results of the average power analysis in the PowerCanvas. For details see, [Viewing Power Analysis Results in the PowerCanvas](#).
 18. Restart the ptshell.
 19. Source the project_settings.tcl file again.

```
full_chip_ptshell % source project_settings.tcl
```
 20. Source the power_aware.tcl file to read in the power constraints for this tutorial:

```
full_chip_ptshell % source power_aware.tcl
```
 21. Source the CalculateTimeBasedPower.tcl script to run time-based power analysis:

```
full_chip_ptshell % source CalculateTimeBasedPower.tcl
```

When the script is finished, you will see the following files in your run directory:

 - CalculatePowerTimeBasedBatch.log—This file captures log information for the CalculatePower time-based run.
 - top.TimeBasedBatch.pdb—This power database now includes time-based power analysis data.
 - top.TimeBasedBatch.rpt—This is the a text-based power report for the time-based run.

22. Exit the ptshell.
23. Review and analyze the results of the time-based power analysis in the PowerCanvas. For details see, [Viewing Power Analysis Results in the PowerCanvas](#).

Viewing Power Analysis Results in the PowerCanvas

Once you have run any power analysis (average or time-based) you can view those results in the PowerCanvas.

1. Load the power database named **top.AverageBatch.pdb** that was created by the CalculateAveragePower.tcl script.

By default, the hierarchy browser is not colored.

2. To see the power results in the hierarchy browser, select **Design > Hierarchy > Colorize by > Power**. You can also color the power table by selecting **Design > Hierarchy > Colorize by > Power**. The hierarchy browser now shows the design colored for power as shown in the following figure:

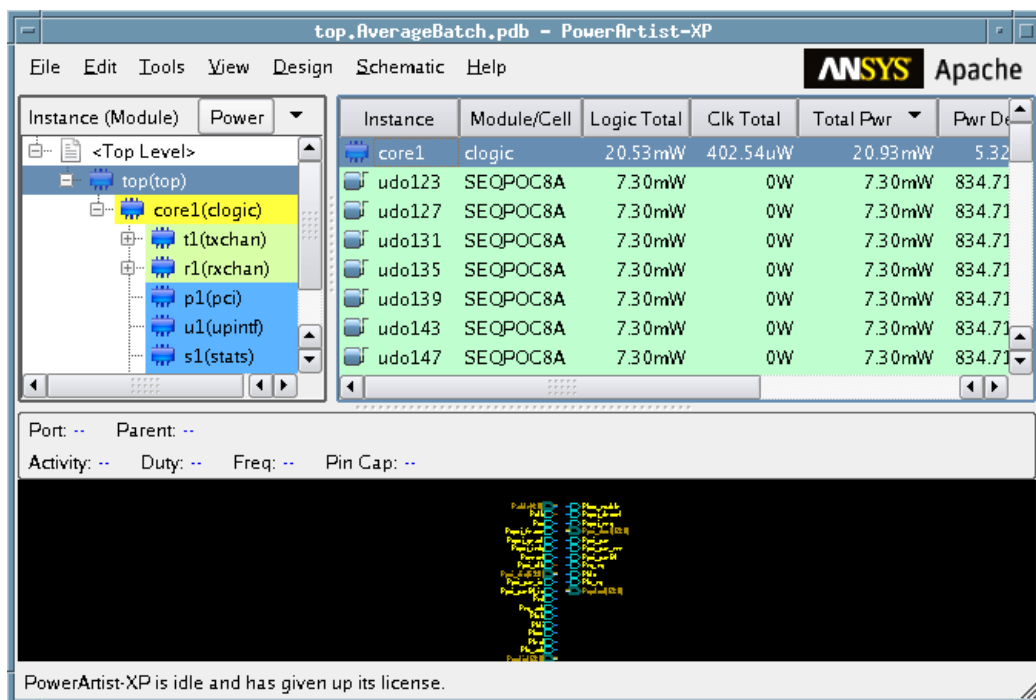


Figure 3 Display of top.AverageBatch.pdb (Average Power Analysis Results)

The colorizing follows a thermal spectrum. The more power consumed, the hotter the color (red indicating the most power consumed). The less power consumed, the cooler the tab. Select this tab and select the data you want to view from the

list; then click the **Add waveforms to plot area** button to display the waveform in the **View** tab.

Manipulating Your Design in the Schematic

This section gives you a basic understanding of how to probe your design/results using the schematic view. Once you have a basic understanding, you can experiment with your own design.

1. From the hierarchy browser, click on **t1/txchan** then right click and select **Show in Schematic**.

The t1 instance will display in the schematic.

2. Use your +/- key to zoom or do a press-drag-release to zoom in around t1, as shown in the following figure.

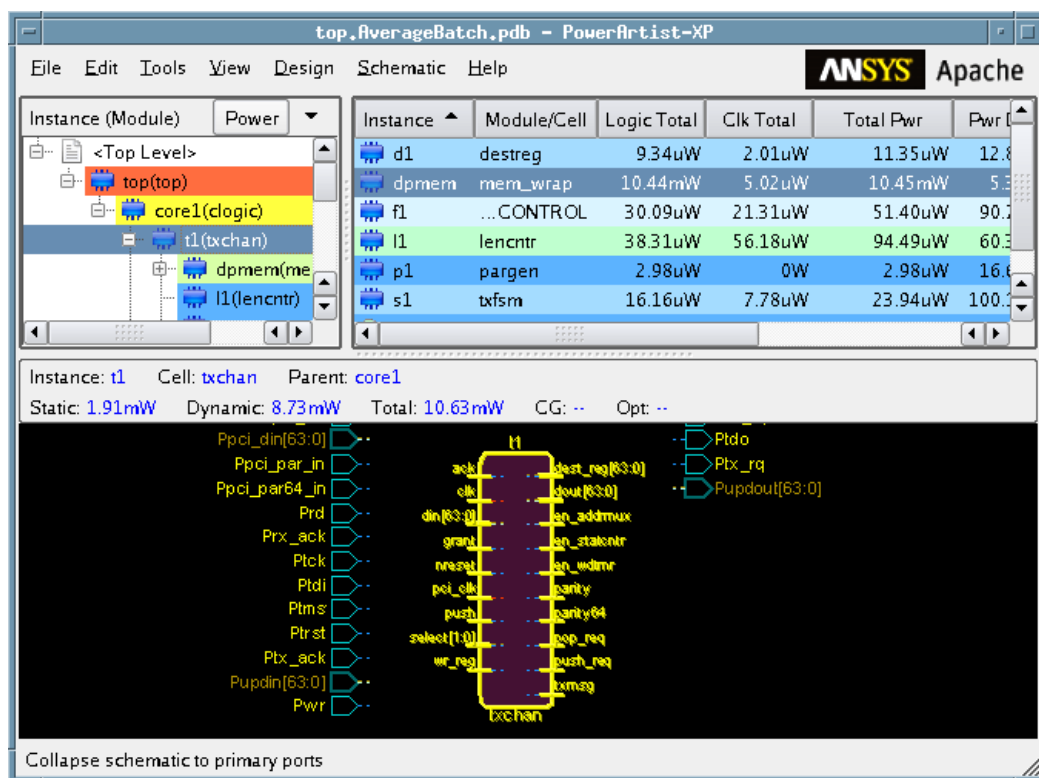


Figure 4 Schematic Display of Module Selected in the Hierarchy

3. Notice the dynamic information display just above the schematic. This display changes as you move your cursor over the elements in the schematic. This display includes information such as instance name, cell name, parent, static/

dynamic/total power, clock gating status, and optimization status. From this figure, you can see that instance t1 consumes a total of 10.63 mW of power.

4. Do a shift-double click on the border of t1 which will then show a detailed view of its schematic.
5. Do a shift-double click again to return to a view that only shows the primary ports. If you zoom-in, you will notice that the port stubs for single-bit pins are thinner than the port stubs for bus ports.
6. Double-click the bus port stub for din[63:0].
7. Click the wire coming from this port that is on the outside of the instance boundary and select **Schematic > Properties.** or right-click and select **Properties.**
8. To see all of the nets, click the + sign next to the txdin pin name.

The Properties dialog will appear as shown in the following figure.

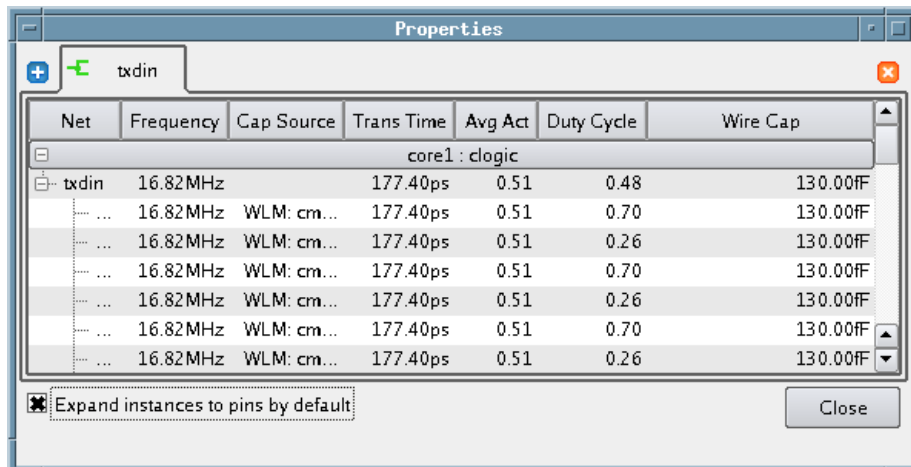


Figure 5 Properties Dialog for Net txdin

This dialog displays the frequency, capacitance source, transition time, average activity, etc. for each net in the bus. Once you have opened this dialog, you can click on different areas in the schematic and the dialog will update accordingly. You can use this dialog to view net information for both single-bit ports and bus ports. Furthermore, you can add tabs to this dialog to display other pins, nets, or instances. For complete details on how to use the Properties dialog, see [Using the Properties Dialog](#).

Viewing Clock Trees

The PowerCanvas allows you to view both the clock tree for the entire design or for a specific portion of the design. You can start with the full clock tree and zoom-in to

tree segments or you can start with the clock tree for a specific register and build the clock tree up from there. However, to be able to view clock trees you must save them while performing a CalculatePower or ReducePower command. Do this by specifying the “-save_clock_trees_netlist true” option as shown in the tutorial Tcl scripts.

If you want to see the clock tree for the entire design, you must first display a wire in the schematic that corresponds to the actual clock net. You will select the wire and then select a menu item that will generate the full clock tree corresponding to that highlighted wire. In the tutorial design, there are three clock nets: *top.clk*, *top.pci_clk* and *top.tclk*.

You can use the following process to generate the full clock net for *top.clk*.

1. Erase the schematic by doing a right-click and selecting **Collapse Full Schematic**. This goes back to the view with only the primary inputs displayed.
2. From the Tree display, select *core1/clogic*.
3. Right-click and select **Show in Schematic**.
4. Double-click the *clk* pin for *core1*.
This brings up a vendor gate named *sclk1* (SEQPIC), which in this case is a pad cell.
5. Select/highlight the wire that goes to that pad cell.
6. Right mouse click on that wire and select **Pin > Show Clock Tree** as shown in the following figure.

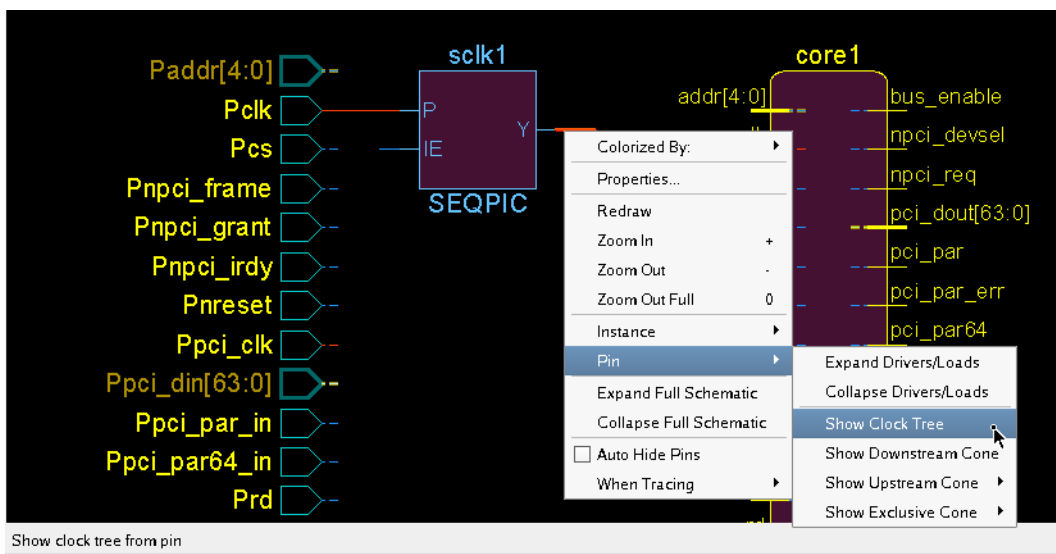


Figure 6 Displaying the Clock Tree for the Entire Design

This may take a few seconds. The entire clock tree will be displayed.

7. You can now zoom in to see specific segments.

Displaying the Clock Tree for a Specific Register

If you want to see the clock tree associated with a specific register, you need to first display the register in the schematic. You can then select the clock pin and select a menu item to generate the clock tree. The clock tree that is displayed will be the entire tree traced from the clock pin back up to the root buffer including any trace instances or inferred integrated clock gating cells.

You can use the following process to display the clock tree for a single register:

1. Erase the schematic by doing a right-click and selecting **Collapse Full Schematic**. This goes back to the view with only the primary inputs displayed.
2. Ensure that **Design > Power Table > Hide Registers** is not checked.
3. In the hierarchy, expand core1/clogic.
4. Select r1(rxchan) / s1(rxfsm) in the hierarchy view.
5. In the power table, scroll down to the register listing and select instance name CurSt (the first listed register) and select **Show in Schematic** from the pop-up menu.

Register core1.r1.s1.CurSt[0] displays in the schematic as shown in the following figure:

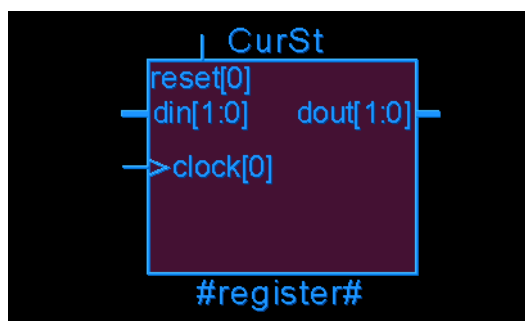


Figure 7 Zoomed-in Schematic Display of Register core1.r1.s1.CurSt

6. Click the clock[0] pin/stub and select **Pin > Show Clock Tree** from the pop-up menu.

7. The clock tree for that register will be displayed in a separate window as shown here:

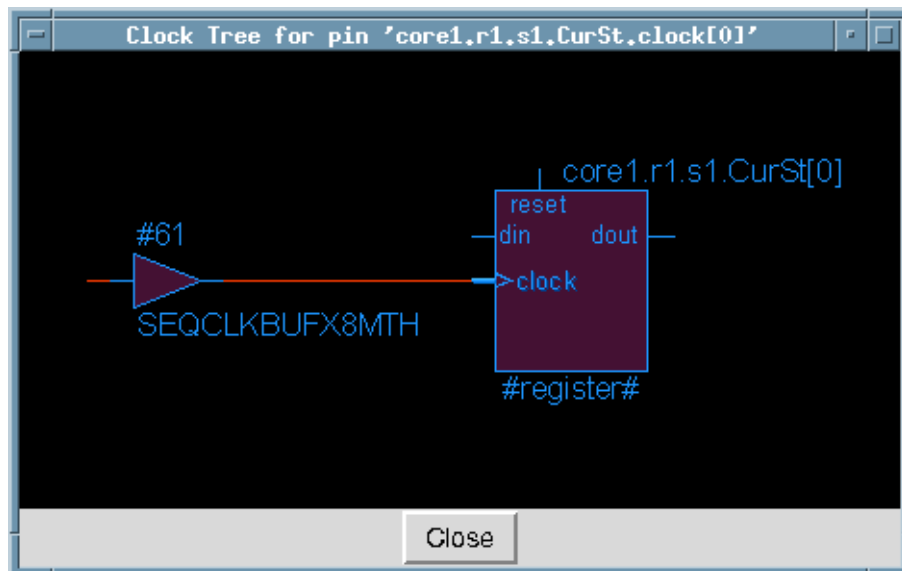


Figure 8 Clock Tree for Register core1.r1.s1.CurSt[0]

Once you have the clock tree for this register displayed, you can build up the clock tree by double clicking the red wires/nets.

8. Double click the red net segment going into the input of SEQCLKBUF8MTH buffer. You will see additional connections to the clock tree for register core1.r1.s1.CurSt[0] as shown in the following figure:

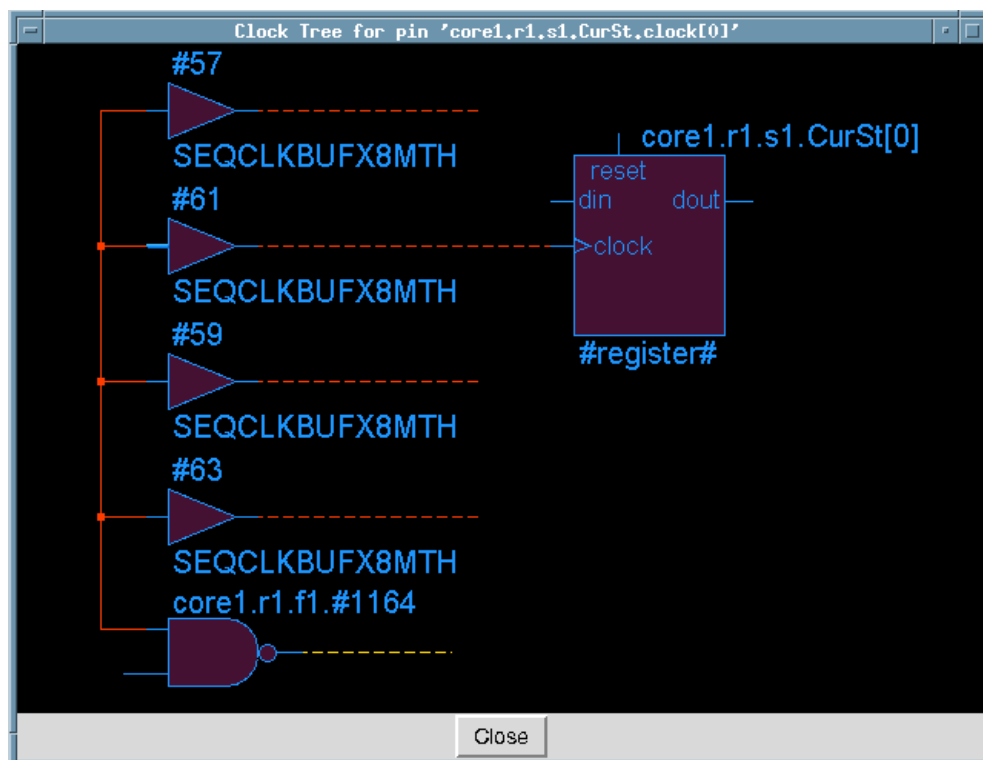


Figure 9 Additional Connections to the Clock Tree for Register core1.r1.s1.CurSt[0]

Waveform Cross-Probing

To perform waveform cross-probing from the schematic, first make sure that the waveform viewer is visible by selecting **Tools > Waveform Viewer**. Then load the FSDB or VCD file that contains the simulation data you want to see. When you select a pin, port or net in the schematic, PowerArtist searches the simulation data you loaded into the Waveform Viewer for data that matches the selected object's name. The matches are shown in the **Search** field. Simply select the waveforms you want to view and click the **Add waveforms to plot area** button. For more details, see [Using the Waveform Viewer](#).

Running Full-Chip Analysis Using the PowerCanvas Wizards

This section shows you how to elaborate your design, run vector analysis and a full-chip power analysis using the PowerCanvas wizards. The main goal of all the wizards is to introduce new users to the PowerArtist analysis tools. Specifically, the wizards are designed to do the following:

- Guide you through the steps required to perform a power analysis or reduction run.
- Show you the critical options that are required to perform even the most basic analysis. You can specify many additional options if you use the Tcl command file flow as described earlier in this tutorial.
- Show you how commands and command options are generated from the user interface and put into task-specific files in the `.ads_power_config` directory. You can then build your own run scripts based on these output files.
- As you become more experienced, you will likely migrate to running more of your analyses using the Tcl command files.

Note that each wizard corresponds to a PowerArtist Tcl command. For example, the HDL Inferencing wizard corresponds to the `Elaborate` command. The power analysis wizards both call the `CalculatePower` command. Therefore, the input fields correspond to the equivalent command options. For example, the “Top instance name” field, corresponds to the `CalculatePower -top_instance` option. When you are finished running the wizard, you can look at the files in the `./ads_power_config` directory to see the exact settings that you made in the wizards. These files are hidden so you are not tempted to edit them. Do not edit them.

Performing Design Inferencing Using the HDL Inferencing Wizard

The first step in any PowerArtist run is to inference the design. You can use the following process to run the HDL Inferencing wizard.

1. Invoke the PowerCanvas using either of the following commands:

```
% PowerArtist-PT  
  
or  
  
% PowerArtist
```

2. From the menu bar, select **Tools > HDL Inferencing**.

The HDL Inferencing wizard appears as shown in Figure 10.

In this full-chip tutorial, the design has both Verilog and VHDL and so both are checked. If you have a design that is a pure Verilog design, you would check the Verilog box and uncheck the VHDL box.

3. Verify that the `txrx.vc` simulation startup file is specified as the Verilog Command File. Note that if you click the right-most icon, a file selector is displayed. You can

then open the `txrx.vc` file. For additional information about this file, see [Verilog Startup File Format](#) in the *PowerArtist Reference* manual.

4. To edit a specified input file, use the following process:
 - a. Click the  icon to the right of the file specification. You will get an Xterm with the editor of your choice editing the file. You can use the `$EDITOR` environment variable to set the editor in advance.
 - b. If the input field is empty, a file browser will pop-up allowing you to choose or enter a file name for saving the edited file.
 - c. Close the file browser. You get an Xterm with your file open in the editor of your choice.
 - d. When you are done editing the file, simply close the editor.

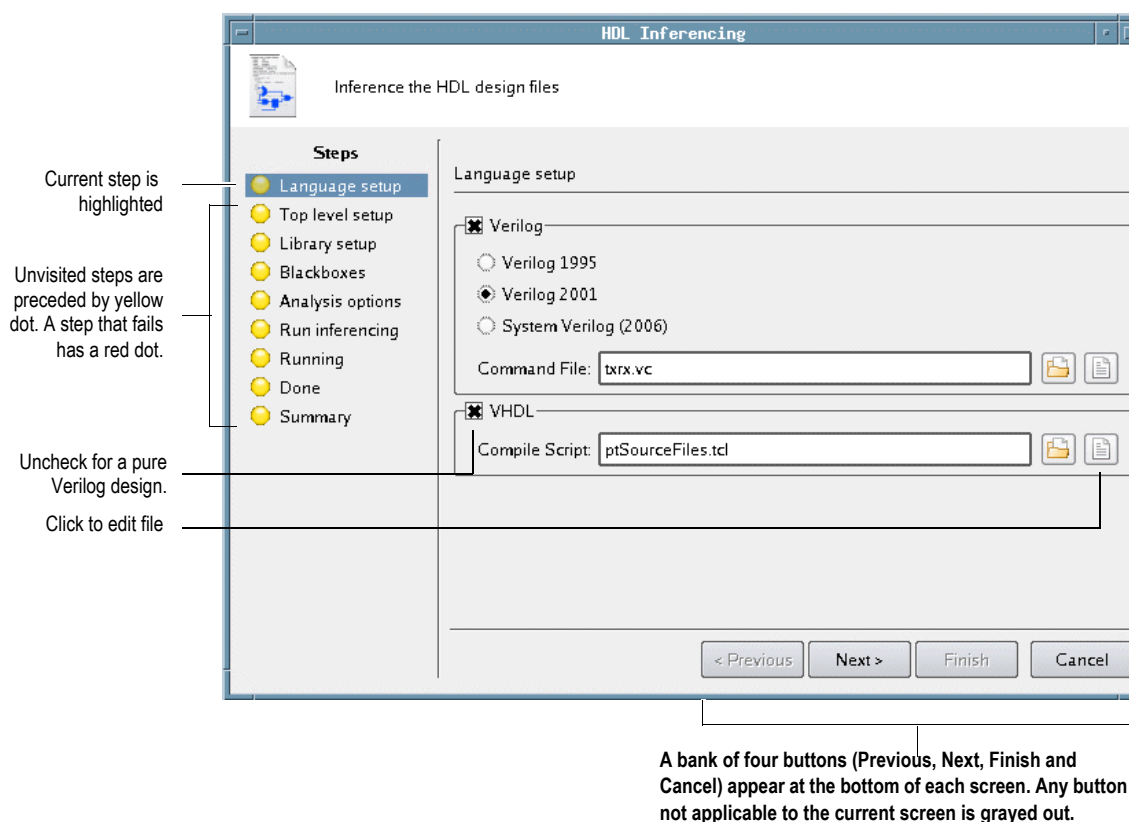


Figure 10 HDL Inferencing Wizard Language Setup Screen

5. Click **Next**.

The Top level setup screen appears. Note that you can also select the step directly from the list of steps on the left side of the window.

6. Verify that the top-level module has been set to “top”. The top-level parameters are used if you have Verilog defparam or VHDL generic override values you want to put in. To add a parameter, click the **Add** button and then type the name of the parameter and the value. To delete a parameter, select the parameter and value (Ctrl-click to select multiple parameters) then click the **Delete** button. This is not needed for this design.
7. Click **Next**.
The Library setup screen appears.
Use this screen to specify the Liberty libraries (this tutorial does not use them).
8. Click **Next**.
The Blackboxes screen appears.
9. Click **Next**.
The Analysis options screen appears.

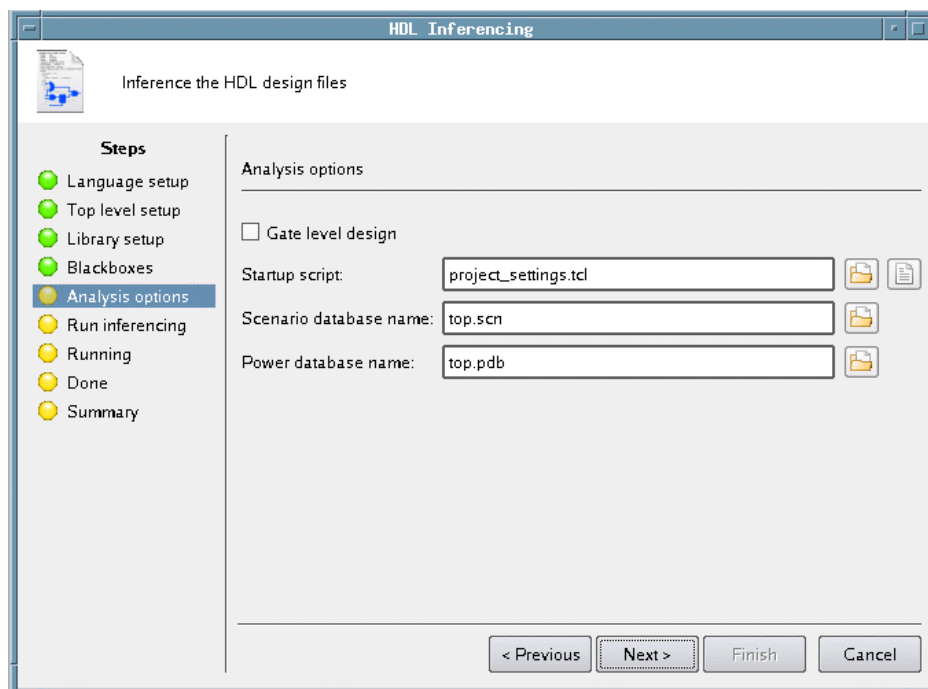


Figure 11 HDL Inferencing Wizard Analysis Options Screen

All of the fields on this screen are populated for this tutorial. Note that since this is an RT-level design, the “Gate level design” box is not checked.

10. Notice that there is a startup script specified for this tutorial called `project_settings.tcl`. You can create any Tcl script and apply it here to inferencing. The script can contain any number of command file Tcl commands.

11. Verify that the resulting scenario file name is set to top.scn.
12. Verify that the Power database field is set to top.pdb.
13. Click **Next**.

The Run the inferencing command screen appears.

14. Click the **Run** button to start the inferencing.

As the inferencing is running, you will see that an analysis report is printed interactively into the large text field. Error and warning messages are printed here. When the inferencing is complete, you will see the Done screen.

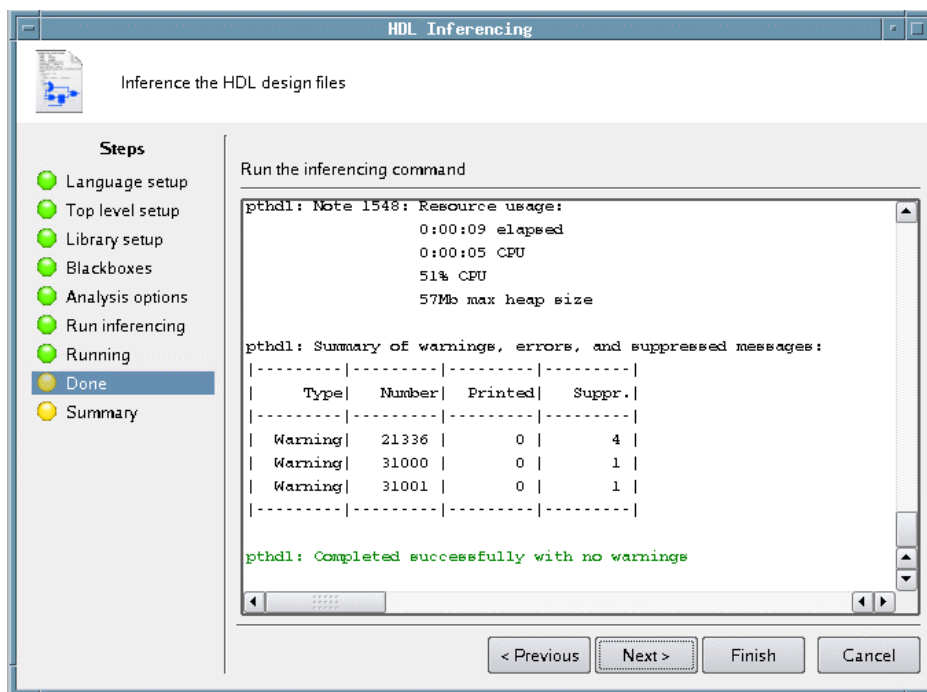


Figure 12 HDL Inferencing Wizard Log

Notice the summary table of error and warning messages.

15. Click the **Next** button to see a Summary of error and warning messages.
16. Click the **Finish** button to close the wizard.

This will bring you back to the main display where you will see a tree display of your tutorial design hierarchy. You can double-click on the modules to display any children. By default, inferred instances are not displayed. To display them, uncheck **Design > Power Table > Hide Inferred Instances**.

17. In your working directory, cd to the .ads_power_config subdirectory.

18. Locate the HDLInferencing file (use the “vi” editor to see the contents).

This file was generated by the HDL Inferencing wizard and contains all of the commands/options/variables settings that you just ran in the tutorial. You can use this file to generate your own Tcl command file to run future batch analyses.

Running Vector Analysis Using the Vector Wizard

You can use the Vector Analysis wizard to perform vector analysis in the PowerCanvas. To do so, use the following process:

1. From the PowerCanvas main menu, choose **Tools** > **Vector Analysis**.

This brings up the Define instance groups screen.

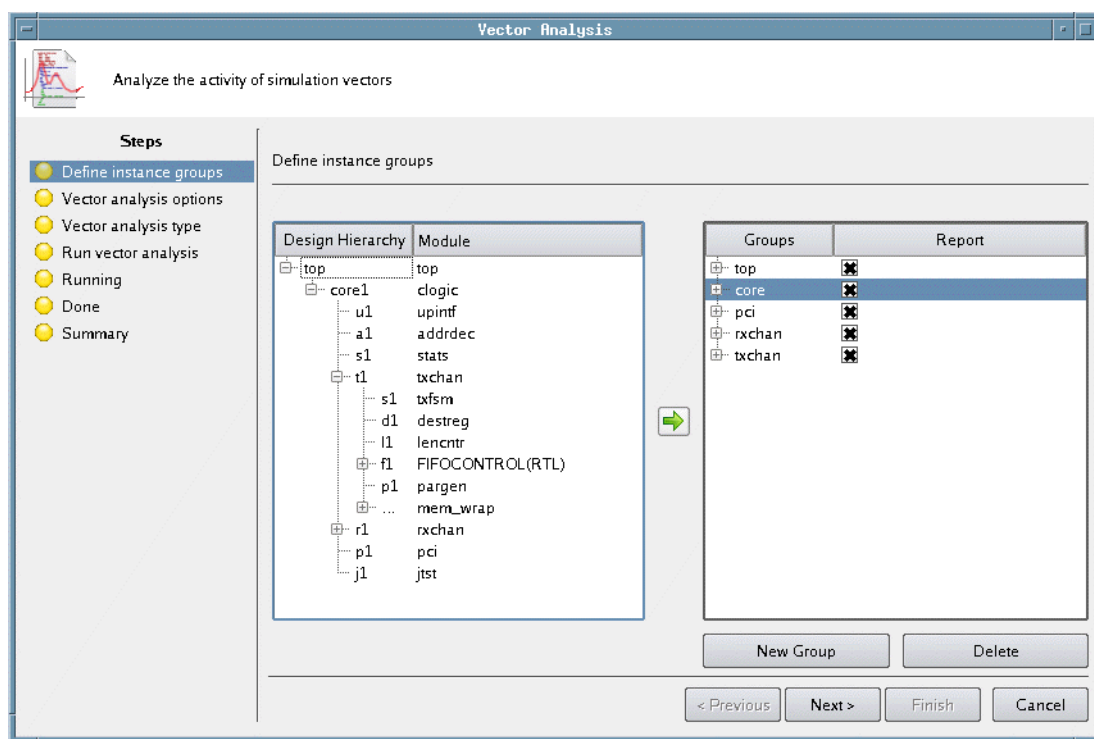


Figure 13 Vector Analysis Define Instance Groups Screen

You can use this screen to define groups of instances for vector analysis. It works the same way as the [DefineGroup](#) command used in the command-based flow. For the tutorial, there are five defined groups, all of which are checked to be included in the analysis.

2. Note the following navigation tips for this screen:
 - To see the instances included in a particular group, click the + icon to the left of the group name.
 - To exclude an instance group from the current analysis without deleting it, simply uncheck the appropriate **Report** box.
 - If you want to create a new group, you would use the following process:
 - a. Click the **New Group** button.
 - b. Type in the name for the new group where it says **<Group>**.
 - c. Select an instance from the Design Hierarchy box. Click the + sign next to the top module name to move down the hierarchy and select an instance. The groups automatically include all hierarchical instances underneath the instances you indicate. The design hierarchy displayed supports the top 3 levels of your design.
 - d. Click the  icon to include the selected instance in the new (highlighted) group. If you want to edit the group name, double click the group name in the Groups column.
 3. Maintain the current group settings and click **Next**.
- This brings up the Analysis options Screen.

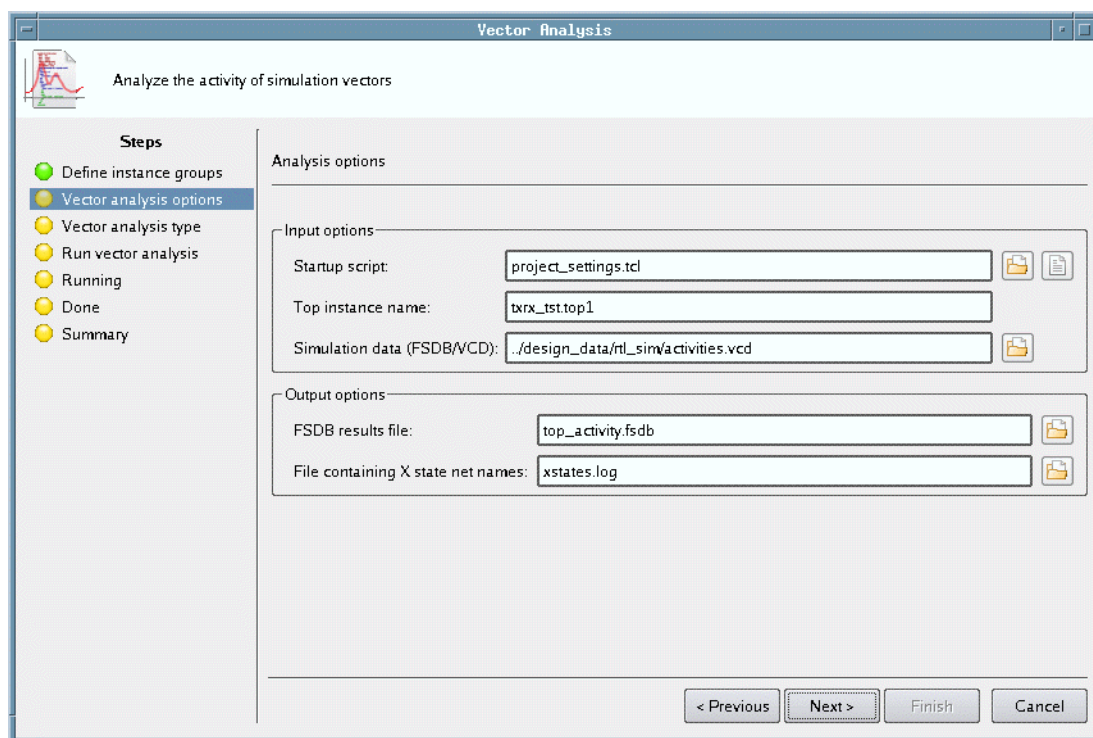


Figure 14 Vector Analysis Wizard Analysis Options Screen

In this step, you specify input/output options and analysis options for the current run.

4. Make note of the following input settings:
 - The startup script called `project_settings` is specified. This Tcl file sources another global settings Tcl file. You can specify any Tcl commands or variables you want to apply on a site-wide or project-wide basis.
 - The top instance name is the hierarchical path to the top level of the design in the VCD file. In this case, the top instance is `txrx_tst.top1`.
 - The VCD file used in this tutorial is `./design_data/rtl_sim/activities.vcd`.
5. Make note of the following output settings:
 - This vector analysis will generate an FSDB output file (`top_activity.fsdb`).
 - It will create a file called `xstates.log` which contains X state net names.
6. Click **Next**.

This brings up the Analysis type Screen.

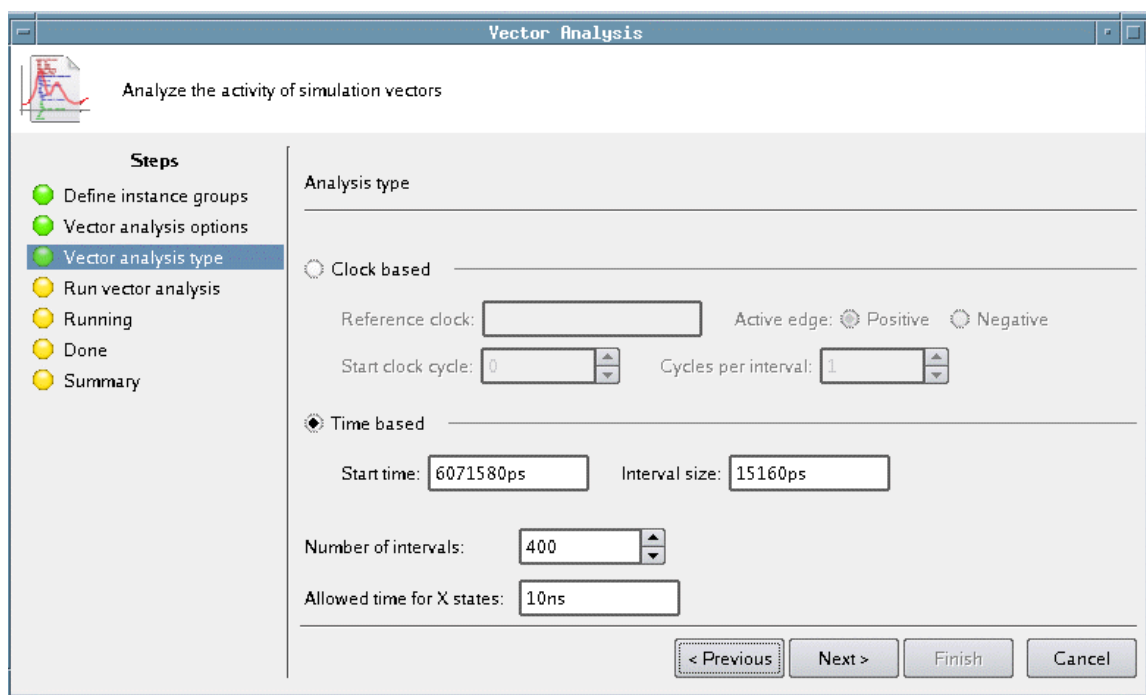


Figure 15 Vector Analysis Wizard Analysis Type Screen

This is where you select the type of vector analysis to run—clock-based or time-based. This tutorial runs a time-based analysis and so the clock-based is grayed-out.

7. Verify the following input settings:
 - Time based is selected.
 - Start Time: 6071580ps. This defines the simulation time step that forms the starting point for the analysis.
 - Interval size: 15160ps. This is the length of the interval in time steps.
 - Number of intervals: 400
 - Allowed time for X states: 10ns
8. Click **Next** and then **Run** to start the vector analysis.
9. When the analysis completes, click the **Finish** button to close the vector analysis screen.

Running the Average Power Wizard

You can use the Average Power Analysis wizard to run an average power analysis. To do so, use the following process:

1. From PowerCanvas main menu bar, select **Tools > Average Power Analysis**.
The Average Power Analysis wizard appears as shown below:

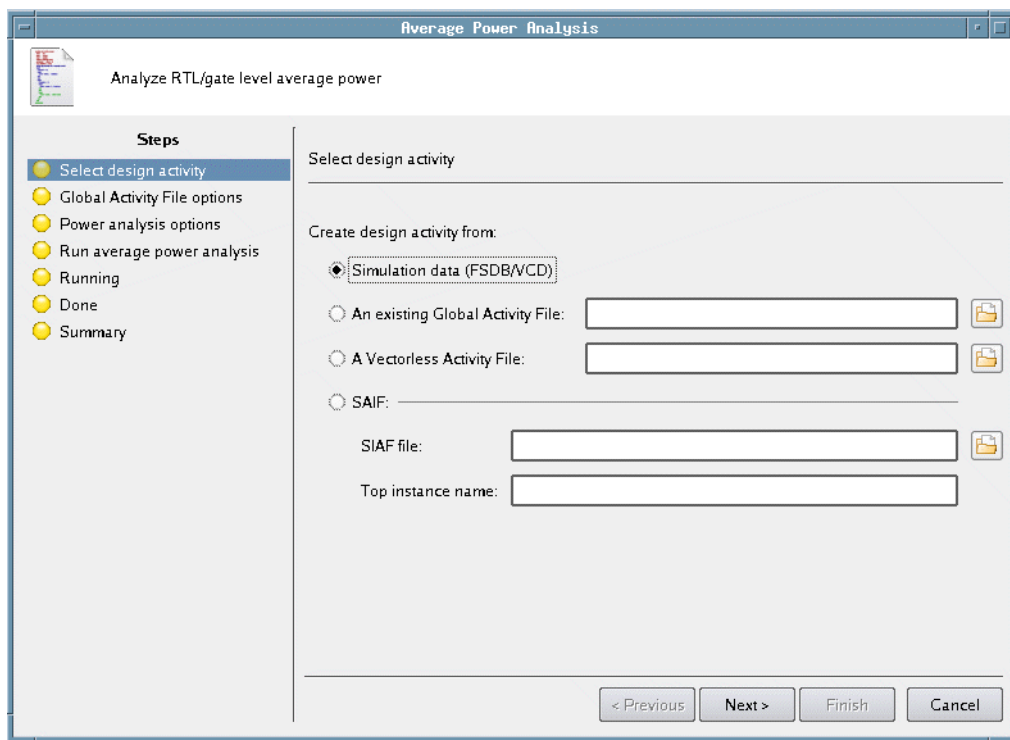


Figure 16 Average Power Analysis Wizard Select Design Activity Screen

This screen indicates that this is a simulation-based analysis. You could also perform a vectorless analysis or one that uses an existing GAF file or a Switching Activity Interchange Format (SAIF) file as input. For more information on using SAIF, see [Analyzing Average Power Using a SAIF File](#).

2. Click the **Next** button.

The Set Global Activity File Creation Options screen appears as shown in Figure 17. This is the screen where you specify input files and other options that affect the generation of a GAF file.

The screenshot shows the 'Average Power Analysis' wizard window. The title bar reads 'Average Power Analysis'. Below the title bar, it says 'Analyze RTL/gate level average power'. On the left, there is a 'Steps' panel with six items: 'Select design activity' (green circle), 'Global Activity File options' (yellow circle, selected), 'Power analysis options' (yellow circle), 'Run average power analysis' (yellow circle), 'Running' (yellow circle), 'Done' (yellow circle), and 'Summary' (yellow circle). The main area is titled 'Setup Global Activity File creation options'. It contains three sections: 'Input options' with 'Top instance name' (txrx_tst.top1) and 'Simulation data (FSDB/VCD)' (../design_data/rtl_sim/activities.vcd); 'Analysis options' with 'Start time' (6071580ps), 'Finish time' (12135580ps), and 'Allowed time for X states' (10ns); and 'Output options' with a checked 'Output compressed GAF' box, 'Global Activity File (GAF)' (top.gaf), and 'File containing X state net names' (xstates.log). At the bottom right are four buttons: '< Previous', 'Next >', 'Finish', and 'Cancel'.

Figure 17 Average Power Analysis Wizard Set Global Activity Creation Options Screen

3. Verify the following input settings:
- Top Instance Name: txrx_tst.top1
 - Simulation data (FSDB/VCD): ../design_data/rtl_sim/activities.vcd
 - Start Time: 6071580ps
 - Finish Time: 12135580ps
 - Allowed time for X states: 10ns
 - Output Compressed GAF box is checked, this option enables compressed GAF as output.

- Global Activity File (GAF): top.gaf
 - File containing X state net names: xstates.log
4. Click the **Next** button.

The Power Analysis Options screen appears as shown in Figure 18.

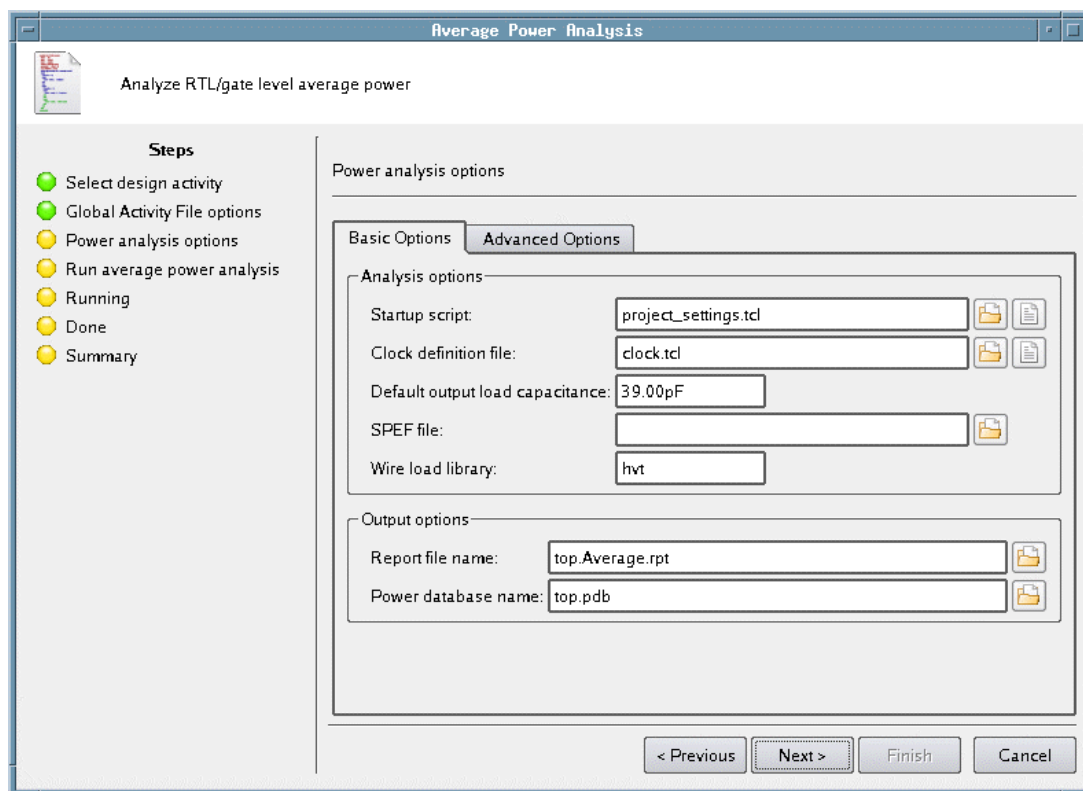


Figure 18 Average Power Analysis Wizard Set Power Estimation Options Screen

5. Verify the following analysis option settings:

- Startup script: project_settings.tcl
- Clock definition file: clock.tcl

This file contains clock definition information supplied with the SetClockNet, SetClockGatingStyle and SetClockBuffer commands.

Note: This average power wizard tutorial is the only one that specifies a clock definition file. In all of the other tutorials (command line or wizard-based), the clock definition commands are embedded in the power_aware.tcl file.

- Default output load capacitance: 39.00pF
- Wire load library: hvt

6. Verify the following Output options settings:
 - Report file name: top.Average.rpt
 - Power database name: top.pdb.
7. Click the **Advanced Options** tab.

This tab includes advanced settings on handling negative power numbers, separating internal load power, including register clock pin power in clock power and skipping net capacitance estimation. None of these options are used in this tutorial.
8. Click **Next** to bring up the Run screen and then click the **Run** button to perform the analysis.

As the analysis runs, you will see log information on the screen.
9. When the analysis completes, you will see a Summary table at the end of the log file window.

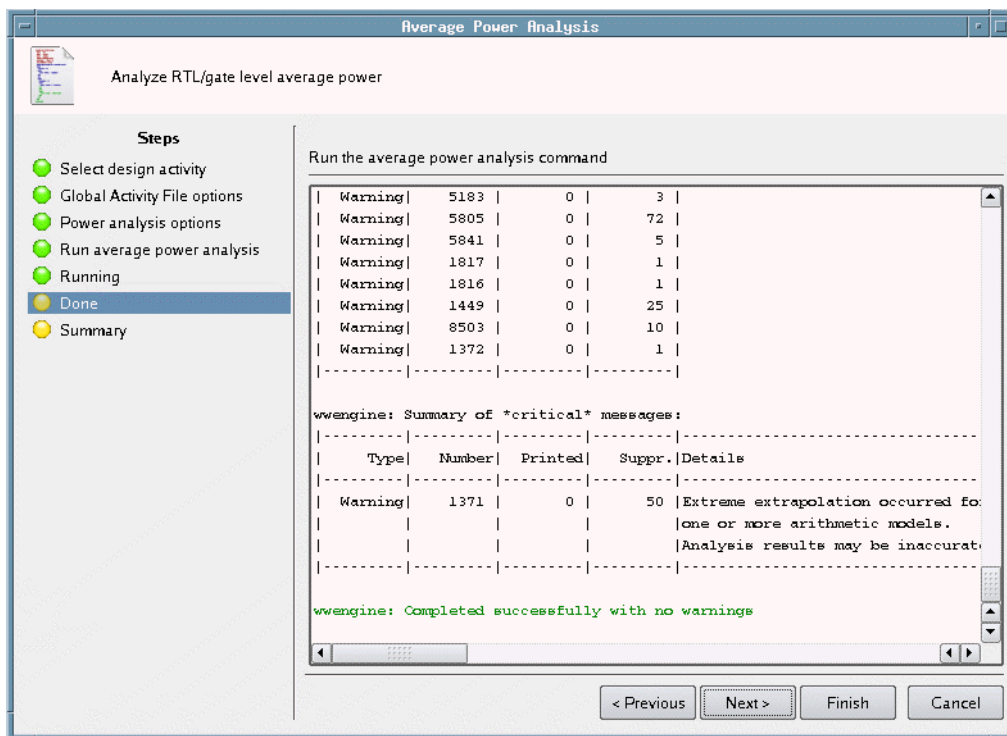


Figure 19 Average Power Analysis Wizard Log File and Summary Table

10. Click the **Finish** button.

This closes the wizard and brings you back to the main window.

Running the Time-Based Power Wizard

You can use the Time Based Power Analysis wizard to run time-based analysis. To do so, use the following process:

1. From the PowerCanvas main menu, choose **Tools > Time Based Power Analysis**.
The Time Based Analysis wizard appears as shown:

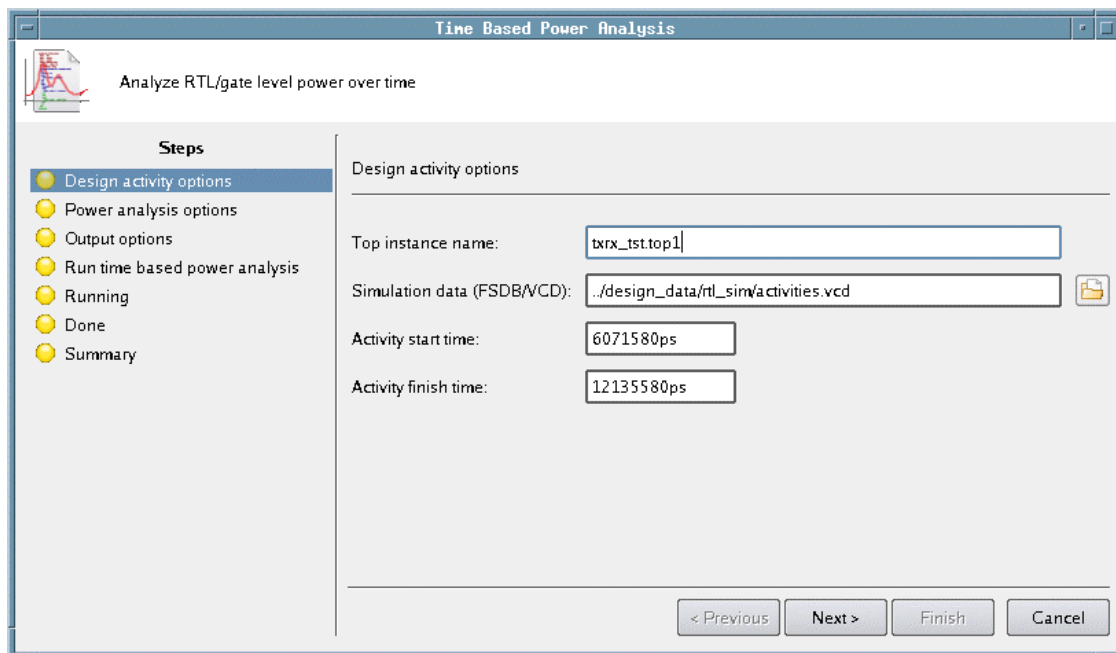


Figure 20 Time Based Power Analysis Wizard Design Activity Options Screen

2. Verify the following input settings:
 - Top Instance Name: txrx_tst.top1
 - Simulation data (FSDB/VCD): ../design_data/rtl_sim/activities.vcd
 - Activity Start Time: 6071580ps
 - Activity Finish Time: 12135580ps
3. Click the **Next** button.

The Power Analysis Options screen appears as shown in Figure 21.

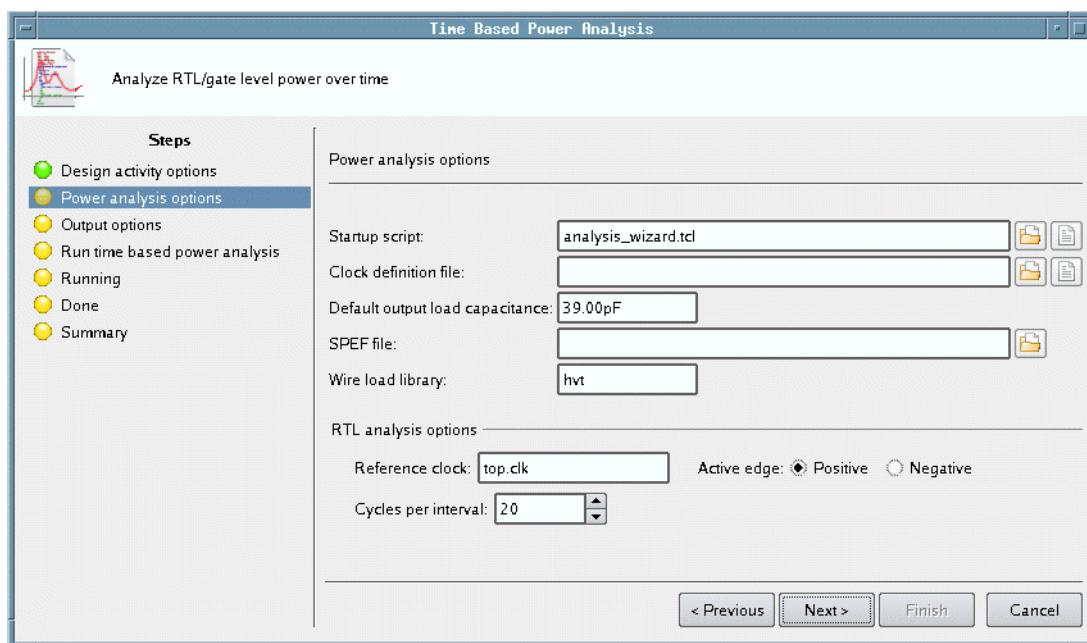


Figure 21 Time Based Power Analysis Wizard Power Analysis Options Screen

This is the screen where you specify the parameters for the time-based analysis.

4. Verify the following analysis option settings:

— Startup script: analysis_wizard.tcl

There is a significant difference between this tutorial and the average power tutorial. The time-based tutorial uses the analysis_wizard.tcl file. This file sources the power_aware.tcl (which contains the clock file commands) and the project_setting.tcl file.

— Clock definition file: blank

The clock file field is in the wizard to remind you of the importance of clock power to an accurate power analysis. The clock-related commands are no different from other power-aware commands and, therefore, can be included in the startup script file, as they are here.

— Default output load capacitance: 39.00

— SPEF: blank

— Wire load library: hvt

— Reference clock: top.clk

— Active edge: Positive

— Cycles per interval: 20

5. Click the **Next** button.

The Output Options screen appears as shown in Figure 22.

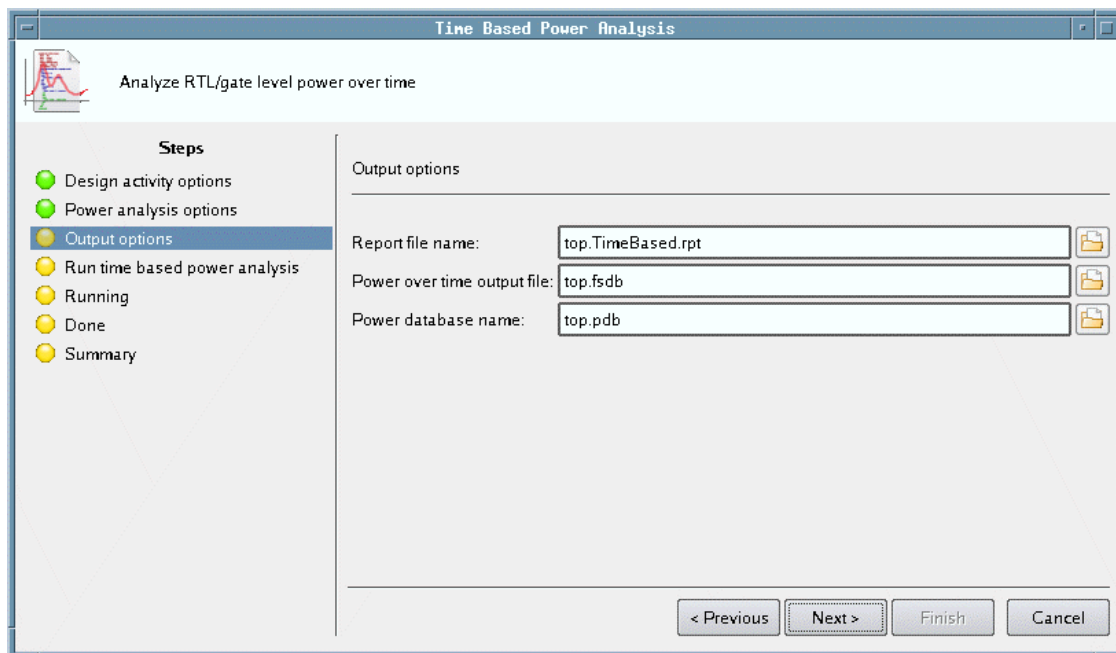


Figure 22 Time Based Power Analysis Wizard Output Options Screen

6. Verify the following output option settings:
 - Report file name: top.TimeBased.rpt
 - Power over time output file: top.fsdb.
 - Power database name: top.pdb
7. Click **Next** to bring up the Run screen and then click the **Run** button to perform the analysis.

As the analysis runs, you will see log information on the screen.

8. When the analysis completes, you will see a Summary table at the end of the log file window.

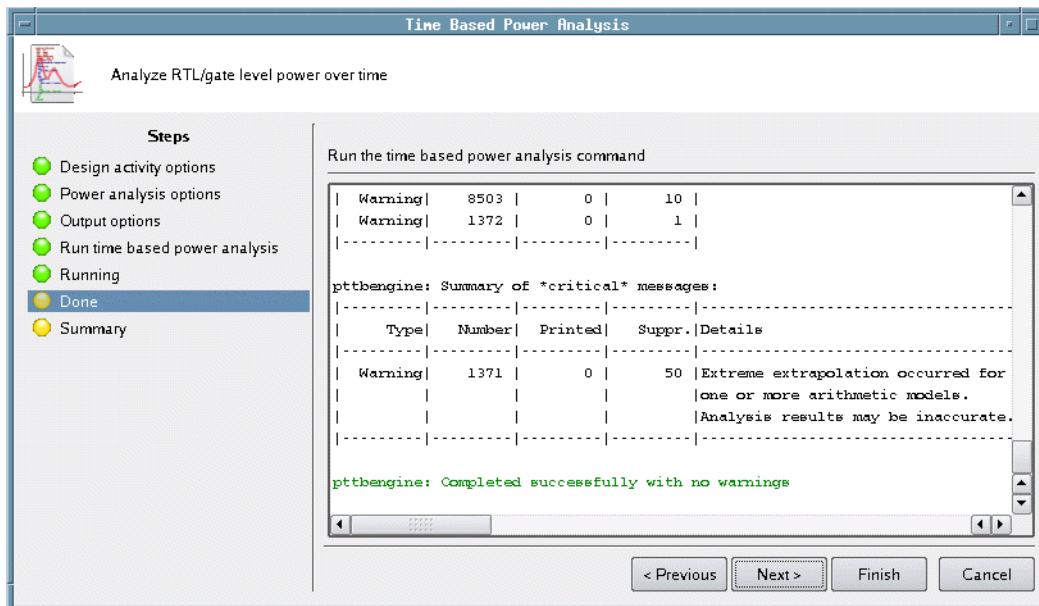


Figure 23 Time Based Power Wizard Log File and Summary Table

9. Click the **Finish** button.

This closes the wizard and brings you back to the main window.

Running an RTL Block-Level Power Analysis

This tutorial will show you how to run a vectorless average power analysis using PowerArtist Tcl command files. If you haven't already done so, copy the contents of the entire tutorial directory to a local directory.

For this tutorial, you will need the following block directory (in addition to the top-level /settings directory). The following table lists the contents of the /block directory.

File Name	Description
README	Provides the basic flow for running this tutorial.
ptshell.ini	Initialization file specific to the block-level tutorial. This file sets the ptshell prompt and specifies the names of the .log and .key files for this tutorial.
power_aware.tcl	Sources the technology libraries and specifies clock information using the SetClockNet command.
project_settings.tcl	Sources a global Tcl file containing site-wide tool settings and the libraries.tcl file containing library information. Using a single file like this to source other files will save time and reduce keystrokes. This file is also specified in the HDL Inferencing wizard.
Elaborate.tcl	Runs the elaboration step to create a scenario file and a power database.
CalculatePower.tcl	Runs block-level average power analysis.
rxchan.vc	Verilog startup file, for the rxchan block, that tells the Verilog compiler which Verilog files must be loaded for your design.
rxchan.vaf	Vectorless activity file for the block.
run_Elaborate.tcl run_CalculatePower.tcl	Run scripts to invoke from the Unix shell. As an alternative method, you could run the tutorial by typing the following commands at the Unix prompt: ptshell -tcl run_Elaborate.tcl ptshell -tcl run_CalculatePower.tcl

Basic Flow for Running the Block-Level Tutorial

The basic flow for this tutorial is as follows:

1. Type “ptshell” to start the ptshell. The prompt for this tutorial will be:
block_ptshell %
2. From the ptshell prompt, source the project_setting.tcl script to setup project-specific parameters and read your libraries.
block_ptshell % source project_settings.tcl

3. Source the Elaborate.tcl script to compile the design and create a PowerArtist scenario file and a power database:

```
block_ptshell % source Elaborate.tcl
```

4. Source the power_aware.tcl file to read power constraints:

```
block_ptshell % source power_aware.tcl
```

5. Source the CalculatePower.tcl file to run vectorless average power analysis:

```
block_ptshell % source CalculatePower.tcl
```

6. View the results in the PowerCanvas using the following command:

```
PowerArtist-PT
```

For details on this process see, [Viewing Power Analysis Results in the PowerCanvas](#).

Using the Command Files

This section describes each of the .tcl files for the block-level tutorial in detail and provides detailed steps for running these scripts.

1. Open the ptshell.ini file. This file contains shell initialization settings for the full-chip tutorial.

```
set ptshKeyFile block.key
set ptshLogFile block.log
set ptshPrompt "block_ptshell %"
```

As you run the tutorial, PowerArtist generates a key file and log file (both with a “block_” prefix) in your run directory. These files are useful for debugging or recreating a previous run. For more information on these files, see [Understanding Log Files and Key Files](#). In addition, the ptshell prompt for this tutorial will have a “block_” prefix. Note that this .ini file overrides the default .ini file that comes with the tarkit. For more information on the initialization file, see [Customizing Your PowerArtist Environment Using Initialization \(INI\) Files](#).

2. Open the project_settings.tcl file. This file is shown here:

```
# Obtain global site-wide settings
source ../../settings/global_settings.tcl

# Specify the technology libraries
source libraries.tcl
```

This file obtains site-wide tool settings (from the global_setting.tcl file) and reads-in library information (from the libraries.tcl file). The libraries.tcl file contains a series of [ReadLibrary](#) commands. Creating this type of file saves you time and requires fewer keystrokes.

3. Open the power_aware.tcl file. This file contains clock gating commands, as shown here:

```
#####  
# Clock Gating Commands  
  
SetClockNet -name rxchan.clk -mode infer  
SetClockNet -name rxchan.pci_clk -mode infer  
  
SetClockBuffer -type root -name SEQCLKBUF4MTH -library hvt -fanout 2  
SetClockBuffer -type leaf -name SEQCLKBUF8MTH -library hvt -fanout 2  
SetClockBuffer -type branch -name SEQCLKBUF8MTH -library hvt -fanout 2
```

4. Open the Elaborate.tcl file. The commands in this file compile your design and perform inferencing.

```
# Elaborate the design  
set design rxchan  
Elaborate \  
-elaborate_write_power_db true \  
-elaborate_log ElaborateBatch.log \  
-power_db_name $design.Batch.pdb \  
-scenario_file $design.Batch.scn \  
-top $design \  
-verilog_startup_file rxchan.vc
```

Much of this script is the same as that for the full-chip tutorial with the following differences:

- The design variable is set to “rxchan” to represent the block.
 - This block is pure Verilog and therefore has only the verilog startup file specified—rxchan.vc.
 - The resulting scenario file name will be rxchan.Batch.scn.
 - The resulting power database will be named rxchan.Batch.pdb.
5. Open the CalculatePower.tcl file. The variables and commands in this file run an average power analysis on the block-level design.

```
# Calculate the power of the design  
set design rxchan  
CalculatePower -analysis_type average \  
-average_report_file $design.Batch.rpt \  
-average_write_power_db true \  
-calculate_log CalculatePowerBatch.log \  
-power_db_name $design.Batch.pdb \  
-scenario_file $design.Batch.scn \  

```

```
-save_clock_trees_netlist true \
-vectorless_input_file $design.vaf \
-wireload_library hvt
```

6. Make special note of the following information in this command file:
 - The CalculatePower command is set to run an average analysis (-analysis_type average). All of the -average* options apply to this type of analysis.
 - The log file that this analysis generates will be named CalculatePowerBatch.log.
 - The power database that this analysis generates will be named rxchan.Batch.pdb. This is the file you will load into the PowerCanvas.
 - The -save_clock_trees_netlist true option generates power database schematics for the clock tree in the design. Without this setting, you will not be able to view your clock tree in the PowerCanvas.
 - The vectorless activity file will be called rxchan.vaf and will look like the following:

```
# Clocks:
#
# Name Frequency (Hz) Duty Cycle (Default: 0.5)
#
SetNetStimulus -net rxchan.clk -frequency 6.6e+07
SetNetStimulus -net rxchan.pci_clk -frequency 6.6e+07

# Primary IOs:
#
# Name Frequency (Hz) Duty Cycle (Default: 0.5)
#
SetNetStimulus -net {rxchan.din[0]} -frequency 1.40451e+07
SetNetStimulus -net {rxchan.din[1]} -frequency 1.40451e+07
<snip>
# Other Nets:
#
# Name Frequency (Hz) Duty Cycle (Default: 0.5)
#
SetNetStimulus -net {rxchan.wr_addr[0]} -frequency 1.75861e+07
SetNetStimulus -net {rxchan.wr_addr[1]} -frequency 8.5564e+06
<snip>
# Memories:
#
# Name Avg. Frequency(Hz) of Local Nets
#
SetInstanceStimulus -instance rxchan.dpmem.m0.m1 -frequency 2.76e+07
SetInstanceStimulus -instance rxchan.dpmem.m0.m2 -frequency 2.76e+07
<snip>
```

This is a portion of the .vaf file. You should be very careful setting the clocks, primary IO's, critical signals, such as enable signals, and your memories. The commands you will be using most often are the [SetNetStimulus](#) (for most nets) and [SetPortStimulus](#) or [SetInstanceStimulus](#) for memories in your design.

1. Type “ptshell” to start up the PowerArtist shell. The prompt for this tutorial will be:

```
block_ptshell %
```

As soon as the ptshell starts, you will see the following additional files in your run directory:

- block.key—Records keystrokes run as part of this tutorial. You can re-run a session by sourcing this file.
- block.log—Records log information (inputs and outputs) for this tutorial. capturing the output of standard out.

2. From the ptshell prompt, source the project_setting.tcl script to setup project-specific parameters and read your libraries.

```
block_ptshell % source project_settings.tcl
```

3. Source the Elaborate script to compile the design and create a PowerArtist scenario file and a power database for the elaborated design:

```
block_ptshell % source Elaborate.tcl
```

When the script is finished, you will see the following additional files in your run directory:

- ElaborateBatch.log—This file captures log information for the Elaborate command. If you get any unexpected results, you can look through this file to make sure that the settings listed are those that you expected to be set.
- rxchan.Batch.scn—The scenario file for the block-level design. The scenario file is a binary representation of your design.
- rxchan.Batch.pdb—At this point in the flow, this PDB file contains the design hierarchy. You can use the PowerCanvas to explore the design hierarchy at this point. The power analysis run will add to this file.

4. Exit ptshell:

```
block_ptshell % exit
```

5. View and analyze the elaborated block-level design in the PowerCanvas.

- a. Invoke the PowerCanvas using either of the following commands:

```
% PowerArtist-PT  
or  
% PowerArtist
```

- b. From PowerCanvas main menu, select **File > Load Design**.

From the file selection dialog, select the correct .pdb file (for example, **rxchan.Batch.pdb**) and click **OK**. The hierarchy browser (at the top level) and the collapsed schematic of the elaborated design appears.

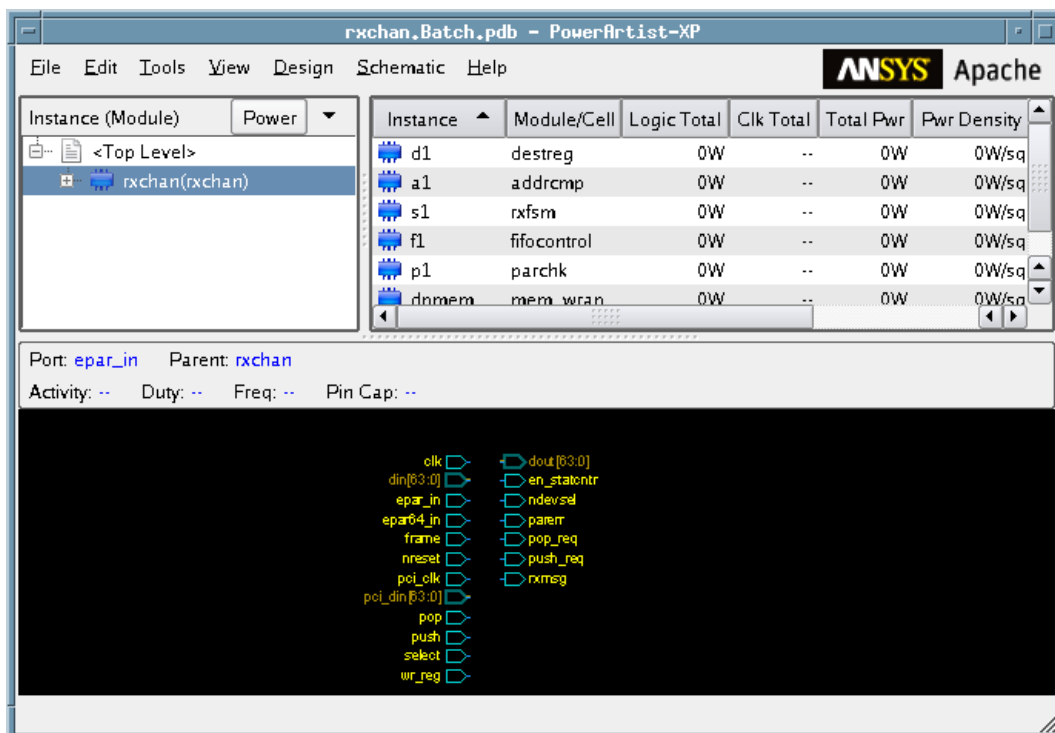


Figure 24 Display of rxchan.Batch.pdb File (Elaborated Design)

- c. Click the rxchan(rxchan) instance in the left-hand pane of the hierarchy browser.
- d. Click the + symbols next to the nested hierarchical instances to display their instances in the right-hand pane called the power table.
6. When you are finished examining the elaborated design, exit the PowerCanvas (select **File > Exit**).
7. Type "ptshell" to restart the ptshell.
8. Source the project_settings.tcl file again.

```
block_ptshell % source project_settings.tcl
```

You must reload this file whenever you exit and restart the ptshell in this tutorial flow.
9. Source the power_aware.tcl file to read power constraints for this tutorial:

```
block_ptshell % source power_aware.tcl
```
10. Source the CalculatePower.tcl file to run the block-level power analysis:

```
block_ptshell % source CalculatePower.tcl
```

When the script is finished, you will see the following additional files in your run directory:

- CalculatePowerBatch.log—A log file for the average power analysis run. It includes all of the command option specifications, warnings and notes generated by the analysis engine, CPU usage, etc. for the latest run.
- rxchan.Batch.pdb—A complete power database that includes the power analysis results, in this case, for average power analysis.
- rxchan.Batch.rpt—The power report in text format.

11. Exit the ptshell.

12. Review the power analysis results in the PowerCanvas.

Viewing the Block-Level Power Results in the PowerCanvas

Once you have run any power analysis (average or time-based) you can view those results in the PowerCanvas.

1. Bring up the PowerCanvas and load the power database named **rxchan.Batch.pdb** which was created by the CalculatePower.tcl script.

```
% PowerArtist -pdb rxchan.Batch.pdb
```

The hierarchy tree now shows the design colored for power. The colorizing follows a thermal spectrum. The more power consumed, the hotter the color (red indicating the most power consumed). The less power consumed, the cooler the color (blue indicating the least power consumed).

2. You can use the same steps outlined in [Viewing Power Analysis Results in the PowerCanvas](#) section to review the block-level power results. Since this block design is a small design, you can also review results in the text-based power report, rxchan.Batch.rpt.

Running the Block-Level Tutorial Using the Power Canvas Wizards

To run the block tutorial using the PowerCanvas wizards, you would use the same process described in the [Running Full-Chip Analysis Using the PowerCanvas Wizards](#). You simply need to start from the /block directory. The wizard fields will be pre-populated with the block-level data and input files.

Running Power Analysis with Clock Gating

To achieve accurate full-chip RTL power analysis, the clock tree and clock gating that will be added at the gate level must be taken into account. To provide this, PowerArtist can infer a clock tree and consider the effect of clock gating. This helps in early and accurate power estimates necessary for decisions that directly impact chip cost—packaging, routing layers, heat sink etc. Clock gating is an important power saving technique as clocks can consume 30-60% of the total chip power consumption.

To access the clock gating tutorial files, change directories to the `clock_gating` directory. The Tcl command files in this directory are similar to those in the `full_chip` directory. The difference is in the content of the `power_aware.tcl` file. You can run the clock gating analysis using either a command file flow or the PowerCanvas wizards. The clock gating commands are identical to those used in the full-chip design.

Using the Command Files

Use the following process to run power analysis with clock gating:

1. Change directories to the `clock_gating` directory.
2. Type “ptshell” to start up the PowerArtist shell. The prompt for this tutorial will be:

```
clock_gating_ptshell %
```

3. From the ptshell prompt, source the `project_setting.tcl` script to setup project-specific parameters and read your libraries.

```
clock_gating_ptshell % source project_settings.tcl
```

4. Source the `Elaborate.tcl` command to compile the design to create a scenario file:

```
clock_gating_ptshell % source Elaborate.tcl
```

When the script is finished, you will see the scenario file (`.scn`), the power database file (`.pdb`) and log files.

5. Source the `power_aware.tcl` file to read power constraints:

```
clock_gating_ptshell % source power_aware.tcl
```

6. Source the `CalculateTimeBasedPower.tcl` script to run time-based power analysis:

```
clock_gating_ptshell % source CalculateTimeBasedPower.tcl
```

When the script is finished, you will see several new files including an updated power database (`.pdb`), the time-based log file and the power report file (`.rpt`).

Using the PowerCanvas Wizards

To run the clock gating tutorial using the PowerCanvas wizards, you would run the same process as that described in the [Running Full-Chip Analysis Using the PowerCanvas Wizards](#). You simply need to start from the `/clock_gating` directory. The wizard fields will be pre-populated with the correct data and analysis specifications.

Running a Power Analysis with a Mixed-Vt Cell Library

PowerArtist allows you to estimate and explore leakage power savings with mixed-Vt cells. A mixed-Vt approach may be optimal since low-Vt cells offer faster timing, but have higher leakage while high-Vt cells have moderate timing with a lower leakage. PowerArtist currently supports a percentage-based mixed-Vt analysis.

To access the mixed-vt tutorial files, change directories to the `mixed_vt` directory. The Tcl command files in this directory are similar to those in the `full_chip` directory. The difference is in the content of the `power_aware.tcl` file. You can run power analysis using a mixed-Vt library using either a command file flow or the PowerCanvas wizards.

Using the Command Files

Use the following process to run power analysis with a mixed-vt cell library:

1. Change directories to the `mixed_vt` directory and open the `power_aware.tcl` file. An excerpt is shown here:

```
#####
# Mixed-VT Library Commands

SetLibrary -instance top -library {hvt RETENTION_EXAMPLE_LIB Memories}
SetLibrary -instance {top.core1.u1 top.core1.a1 top.core1.s1} -library {hvt lvt}

SetVoltageThreshold -group LOW_VT -pattern {*L}
SetVoltageThreshold -group HIGH_VT -pattern {*H}

#####
### Mixed Vt specific settings showing always on Virtual Supplies

SetVT -mode percentage -instance {top.core1.u1 top.core1.a1 top.core1.s1 } \
-vt_group {LOW_VT:30 HIGH_VT:70}

CreateVirtualSupply -supply Memories.vdd -virtual_supply MEMVDD -on 1
CreateVirtualSupply -supply hvt.vdd -virtual_supply HVTVDD -on 1
CreateVirtualSupply -supply lvt.vdd -virtual_supply LVTVDD -on 1

CreateVirtualSupply -supply VDDNW -virtual_supply RX_VDDNWS -on 1
CreateVirtualSupply -supply VDD -virtual_supply RX_VDDNWS -on 1
CreateVirtualSupply -supply VRET -virtual_supply RX_VRET -on 1
<snip>

CreateDomain -instance top.core1.r1 -virtual_supply {MEMVDD HVTVDD LVTVDD \
RX_VDDNWS RX_VRET}
CreateDomain -instance top.core1.t1 -virtual_supply {MEMVDD HVTVDD LVTVDD \
TX_VDDNWS TX_VRET}
```

```
MapRetentionCell -instance {top.core1.r1} -attribute CK_LOW
MapRetentionCell -instance {top.core1.t1} -attribute CK_LOW -tag NoMap \
-notag true

#####
# Clock Gating Commands

SetClockNet -name top.clk -mode infer -gate_clock yes
SetClockNet -name top.pci_clk -mode infer
SetClockNet -name top.tck -mode infer
SetClockGatingStyle -clock_cell_attribute latch_posedge_precontrol
SetClockBuffer -type root -name SEQCLKBUF4MTH -library hvt -fanout 2
SetClockBuffer -type branch -name SEQCLKBUF8MTH -library hvt -fanout 2
SetClockBuffer -type leaf -name SEQCLKBUF8MTH -library hvt -fanout 60
```

The commands shown here are required to incorporate a mixed-Vt library into a design. It contains the `SetLibrary` and `SetVt` commands as well as several iterations of the `CreateVirtualSupply` command to establish voltage domains. These will be later leveraged as part of the `power_gating` tutorial. It also contains the [SetVoltageThreshold](#), defined here for two groups—`LOW_VT` and `HIGH_VT`. For more explanation of the commands in this file see [Sample Mixed-Vt Flow and Tcl File](#).

2. Type “ptshell” to start up the PowerArtist shell. The prompt for this tutorial will be:

```
mixed_vt_ptshell %
```

3. From the ptshell prompt, source the `project_setting.tcl` script to setup project-specific parameters and read your libraries.

```
mixed_vt_ptshell % source project_settings.tcl
```

4. Source the `Elaborate.tcl` file to compile the design and create a PowerArtist scenario file and a power database:

```
mixed_vt_ptshell % source Elaborate.tcl
```

When the script is finished, you will see the scenario file (`.scn`), the power database file (`.pdb`) and log files.

5. Source the `power_aware.tcl` file to read power constraints:

```
mixed_vt_ptshell % source power_aware.tcl
```

6. Source the `CalculatePower.tcl` file to run average power analysis:

```
mixed_vt_ptshell % source CalculatePower.tcl
```

When the script is finished, you will see several new files including an updated power database (`.pdb`), the `CalculatePower.log` file and the power report file (`.rpt`).

7. View the results of this analysis by reviewing the top.Batch.rpt file. Locate the “Mixed-VT cells distribution” section as shown below:

7. Mixed-VT Cells Distribution			
=====			
Hier-Instance Name	VT Group Name	Specified Percentage	Number of Cells Selected
-----	-----	-----	-----
top.core1.u1			
	HIGH_VT	70	145
	LOW_VT	30	62

Total			207
top.core1.a1			
	HIGH_VT	70	58
	LOW_VT	30	25

Total			83
top.core1.s1			
	HIGH_VT	70	579
	LOW_VT	30	248

Total			827

For every hierarchical instance specified using the [SetVT](#) command (in the power_aware.tcl file) this report includes the total number of default cells selected for each of the specified VT groups. For example, 145 default cells selected for instance top.core1.u1 were HIGH_VT cells while 62 were LOW_VT cells. For additional information about this section of the report, see [Understanding Mixed-Vt Analysis Results in the Report File](#).

Using the PowerCanvas Wizards

To run the mixed-vt tutorial using the PowerCanvas wizards, you would do exactly the same process as that described in the [Running the Average Power Wizard](#). The only difference is the content of the power_aware.tcl file in that it contains command required for running a mixed-vt analysis. You simply need to start from the /mixed_vt directory. The wizard fields will be pre-populated with the correct data and analysis specifications.

Running Power Analysis with Voltage Islands

Voltage island techniques can be used to reduce the power on blocks that are not timing critical. PowerArtist provides analysis algorithms that predict power savings that could be achieved by assigning lower voltages for one or more blocks.

To access the required tutorial files, change directories to the `voltage_islands` directory. The Tcl command files in this directory are similar to those in the `full_chip` directory. The difference is in the content of the `power_aware.tcl` file. You can run the power analysis using voltage islands using either a command file flow or the PowerCanvas wizards.

Using the Command Files

Use the following process to run power analysis with `voltage_islands`:

1. Change directories to the `voltage_islands` directory and open the `power_aware.tcl` file. An excerpt is shown here:

```
#####
SetLibrary -instance top.core1 -library {hvt Memories}

#####
# Voltage Domain Commands

CreateVirtualSupply -supply hvt.vdd -virtual_supply VDDRX -voltage 0.9
CreateVirtualSupply -supply hvt.vdd -virtual_supply VDDTX -voltage 0.8

CreateDomain -instance top.core1.r1 -virtual_supply VDDRX
CreateDomain -instance top.core1.t1 -virtual_supply VDDTX

#####
# Clock Gating Commands

SetClockNet -name top.clk -mode infer -gate_clock yes
SetClockNet -name top.pci_clk -mode infer
SetClockNet -name top.tck -mode infer
SetClockGatingStyle -clock_cell_attribute latch_posedge_precontrol
SetClockBuffer -type root -name SEQCLKBUF4MTH -library hvt -fanout 2
SetClockBuffer -type branch -name SEQCLKBUF8MTH -library hvt -fanout 2
SetClockBuffer -type leaf -name SEQCLKBUF8MTH -library hvt -fanout 60

#####
# Instances to monitor in the design

MonitorInstances -name top.core1.t1 -all 1
MonitorInstances -name top.core1.r1 -all 1
```

Notice the two [CreateVirtualSupply](#) and the [CreateDomain](#) command specifications. These commands together create two domains (one for receiver and one for the transmitter) that are given different voltage values.

2. Type “ptshell” to start up the PowerArtist shell. The prompt for this tutorial will be:

```
voltage_islands_ptshell %
```

3. From the ptshell prompt, source the project_setting.tcl script to setup project-specific parameters and read your libraries.

```
voltage_islands_ptshell % source project_settings.tcl
```

4. Source the Elaborate.tcl command to compile the design and create a PowerArtist scenario file:

```
voltage_islands_ptshell % source Elaborate.tcl
```

When the script is finished, you will see the scenario file (.scn), the power database file (.pdb) and log files.

5. Source the power_aware.tcl file to read power constraints:

```
voltage_islands_ptshell % source power_aware.tcl
```

6. Source the CalculatePower.tcl file to run average power analysis:

```
voltage_islands_ptshell % source CalculatePower.tcl
```

When the script is finished, you will see several new files including an updated power database (.pdb), the log file (.log) and the power report file (.rpt).

7. You can now view the results of the analysis with voltage islands in the “Power Domain Summary” section of the top.Batch.rpt report file in this directory.

Using the PowerCanvas Wizards

To run the mixed-vt tutorial using the PowerCanvas wizards, you would do exactly the same process as that described in the [Running Full-Chip Analysis Using the PowerCanvas Wizards](#). The only difference is the content of the power_aware.tcl file in that it contains command required for running a power analysis with voltage islands. You simply need to start from the /voltage_islands directory. The wizard fields will be pre-populated with the correct data and analysis specifications.

Running a Power Analysis with Power Gating

Power gating is a technique that saves leakage power by turning off supplies to blocks that are inactive. You can use PowerArtist to accurately predict power savings due to the implementation of power gating.

To access the power gating tutorial files, change directories to the `power_gating` directory. The Tcl command files in this directory are similar to those in the `full_chip` directory. The difference is in the content of the `power_aware.tcl` file. You can run power analysis with power gating using either a command file flow or the PowerCanvas wizards.

Using the Command Files

Use the following process to run power analysis with power gating:

1. Change directories to the `power_gating` directory and open the `power_aware.tcl` file. This is the file that specifies the power gating commands. An excerpt is shown here:.

```
SetLibrary -instance top -library {hvt RETENTION_EXAMPLE_LIB Memories}
#####
# Power Gating Commands
# Not partitioning the design based on voltage islands.

CreateVirtualSupply -supply vdd -virtual_supply VDDR_X -on top.rx_rq
CreateVirtualSupply -supply vdd -virtual_supply VDDTX -on top.tx_rq

CreateVirtualSupply -supply VDDNW -virtual_supply RX_VDDNWS -on top.rx_rq
CreateVirtualSupply -supply VDD -virtual_supply RX_VDDNWS -on top.rx_rq
CreateVirtualSupply -supply VRET -virtual_supply RX_VRET -on (!top.rx_rq)

CreateVirtualSupply -supply VDDNW -virtual_supply TX_VDDNWS -on top.tx_rq
CreateVirtualSupply -supply VDD -virtual_supply TX_VDDNWS -on top.tx_rq
CreateVirtualSupply -supply VRET -virtual_supply TX_VRET -on (!top.tx_rq)

CreateDomain -instance top.core1.r1 -virtual_supply {VDDR_X RX_VDDNWS RX_VRET}
CreateDomain -instance top.core1.t1 -virtual_supply {VDDTX TX_VDDNWS TX_VRET}

MapRetentionCell -instance {top.core1.r1} -attribute CK_LOW

MapRetentionCell -instance {top.core1.t1} -attribute CK_LOW -tag NoMap -notag true
<snip>
```

In addition to clock definitions, this `power_aware.tcl` file also defines power domains for the power gating analysis using the [CreateVirtualSupply](#) and [CreateDomain](#) commands. These commands are required implement power gating in the design. Note that the [SetLibrary](#) here specifies a special retention cell

library called RETENTION_EXAMPLE_LIB to ensure that registers are available for the power domains.

2. Type “ptshell” to start up the PowerArtist shell. The prompt for this tutorial will be:

```
power_gating_ptshell %
```

3. From the ptshell prompt, source the project_setting.tcl script to setup project-specific parameters and read your libraries.

```
power_gating_ptshell % source project_settings.tcl
```

4. Source the Elaborate.tcl command to compile the design and create a PowerArtist scenario file:

```
power_gating_ptshell % source Elaborate.tcl
```

When the script is finished, you will see the scenario file (.scn), the power database file (.pdb) and log files.

5. Source the power_aware.tcl file to read power constraints:

```
power_gating_ptshell % source power_aware.tcl
```

6. Source the CalculateTimeBasedPower.tcl file to run time-based power analysis:

```
power_gating_ptshell % source CalculateTimeBasedPower.tcl
```

When the script is finished, you will see several new files including an updated power database (.pdb), the time-based log file and the power report file (.rpt).

7. Exit the ptshell.
8. Review and analyze the results of the time-based power analysis in the PowerCanvas. For details see, [Viewing Power Analysis Results in the PowerCanvas](#).
9. You can see the impact of power gating by viewing the waveform. To do so, perform the following steps:
 - a. Choose **Tools** > **Waveform Viewer** to open the graph in the Apache Waveform Viewer.
 - b. From the Files list, select either the **top.TimeBasedBatch.fsdb** file (from the command-line run) or the **top.fsdb** file (from the GUI run) and click the **OK**.
 - c. In the Waveform Name field, type an * to find all waveforms and click the **Search** button.
 - d. Select only the receive (the pt0|top.core1.r1) and transmit (pt0|top.core1.t1) waveform names, select “Place waveforms in: One plot” and click the **Add waveform to plot area** button.

The graph shows that the power of the receive channel is always zero in transmit mode and vice-versa. PowerArtist has essentially cut-off leakage and dynamic power for channels that are inactive in the appropriate modes.

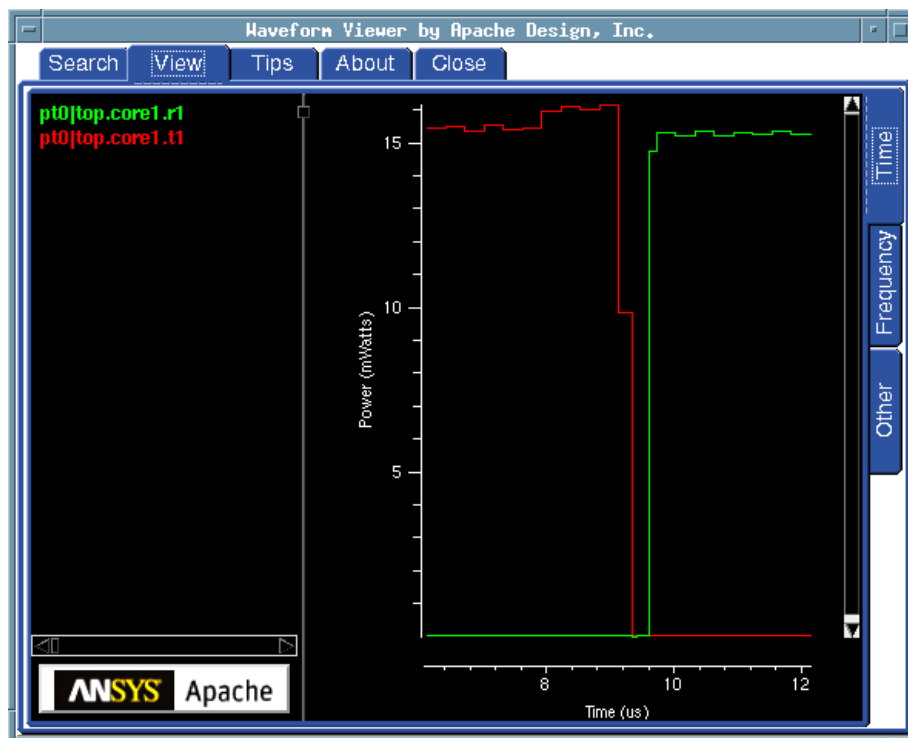


Figure 25 Power Waveform of Receive and Transmit Blocks

Using the PowerCanvas Wizards

To run the time-based power analysis with power gating using the PowerCanvas wizards, you would run the same process as that described in the [Running the Time-Based Power Wizard](#). The only difference is the content of the power_aware.tcl file in that it contains power gating commands. You simply need to start from the /power_gating directory. The wizard fields will be pre-populated with the correct data and analysis specifications.

Running the Full-Chip Gate-Level Power Analysis Tutorial

You can run gate-level analysis on a fully synthesized netlist/placed and routed design. This tutorial shows you how to perform different power analyses using a sample gate-level design. This design is a synthesized version of the RTL design you worked with in earlier sections of the tutorial; therefore, most of the inputs are the same. This section highlights only those inputs that are different for this tutorial. You can run either an average or a time-based power analysis to perform gate-level power verification (with or without back-annotated parasitics). This time-based analysis can be done for instantaneous power or current for different time steps.

Using the Command Files

Use the following process to run the gate-level tutorial:

1. Change directories to the `full_chip_gate` directory.

The `Elaborate.tcl` file is exactly the same as that used in the RTL full-chip tutorial. Additionally, the `VectorAnalysis.tcl`, `CalculateAveragePower.tcl` and `CalculateTimeBasedPower.tcl` files are nearly identical to those used in the full-chip RTL tutorial. The only difference is that they specify a gate-level VCD that comes from the `design_data/gate_sim` directory instead of the `design_data/rtl_sim` directory.

2. Type “ptshell” to start up the PowerArtist shell. The prompt for this tutorial will be:

```
full_chip_gate_ptshell %
```

3. From the ptshell prompt, source the `project_setting.tcl` script to setup project-specific parameters and read your libraries.

```
full_chip_gate_ptshell % source project_settings.tcl
```

4. Source the `Elaborate.tcl` command to compile the design and create a PowerArtist scenario file:

```
full_chip_gate_ptshell % source Elaborate.tcl
```

When the script is finished, you will see the scenario file (`.scn`), the power database file (`.pdb`) and log files.

5. Source the `GenerateActivityWaveforms.tcl` command to perform vector analysis on the gate-level design:

```
full_chip_gate_ptshell % source GenerateActivityWaveforms.tcl
```

Vector Analysis shows how the activity varies over time for selected blocks in your design. When the script is finished, you will see `GenerateActivityWaveformBatch.log` and the `top_activity.fsdb` in your directory. You can load the FSDB file into the Apache Waveform View to see the resulting waveforms.

6. Source the `power_aware.tcl` file to read in the power constraints for this tutorial:

```
full_chip_gate_ptshell % source power_aware.tcl
```
7. Run either an average or time-based power analysis using the corresponding Tcl script.
8. Exit the ptshell.
9. Review the results in the PowerCanvas or the text-based power report.

Using the PowerCanvas Wizards

To run the gate-level tutorial using the PowerCanvas wizards, you would use the same process as that described in the [Running Full-Chip Analysis Using the PowerCanvas Wizards](#). You simply need to start from the `/full_chip_gate` directory. The wizard fields will be pre-populated with the correct data and analysis specifications.

Understanding Power Analysis Results in the Text-Based Power Report

In addition to reviewing the power analysis results in the PowerCanvas, you can also review the results using the text-based power report (.rpt) file.

Both the average and time-based reports are divided into sections that each provide valuable information about the power analysis of your design. At the top of the report are the date and program version, along with values for numerous arguments and parameters. This section describes a sample average power report from the full-chip analysis tutorial.

The first headed section is an abstract of total power dissipation, divided into components of internal power, pad power, clock power and inferred buffer power.

1. Total power consumption
=====

Power contribution -----	Power (Watts)		
	Static -----	Dynamic -----	Total -----
Internal register power	4.36uW	705uW	710uW
Internal latch power	0W	0W	0W
Internal memory power	3.81mW	15.8mW	19.7mW
Other internal power	3.71uW	161uW	165uW
Total internal power	3.81mW	16.7mW	20.5mW
IP Core power	0W	0W	0W
Pad power	44.2mW	751mW	795mW
Clock power	144nW	440uW	440uW
Inferred Buffer power	139nW	4.9uW	5.04uW
Total power	48mW	768mW	816mW

Each of these categories is broken down into greater detail in the sections that follow.

Internal Power Consumption

This section of the power report shows the details of the internal power consumption.

2. Internal power consumption
=====

Note: (G) after either a register or 2-1 mux means this instance is affected by clock gating.
Note: (F) after model name means power of this instance has been forced by using SetPower command.
Note: (O) after model name means this instance has been partially/fully optimized.

Component	Model	Power (Watts)		
		Static	Dynamic	Total
-----	-----	-----	-----	-----
top	user	3.81mW	16.7mW	20.5mW
#1942	connect	0W	0W	0W
#1943	connect	0W	0W	0W
#1944	auto	0W	0W	0W
core1	user	3.81mW	16.7mW	20.5mW
<snip>				
Total internal power		3.81mW	16.7mW	20.5mW

For each module, this report provides several pieces of information:

- The hierarchical name of the HDL component.
- The Apache power model associated with the HDL object.
- The power consumed by that object. This is the key element provided by PowerArtist.

Using the frequency values computed for the ports and the type of power model associated with the object, PowerArtist can accurately estimate from the RTL the power consumed by the object.

Among other options, PowerArtist reports let you separate static and dynamic power estimates. The static power is generally independent of the frequency of the object, although for some modules, such as memories, the static power could depend on the value of control signals, such as chip select. For example, a ROM could consume a certain amount of static power when active, and a smaller amount of power in standby mode. PowerArtist computes the amount of time the ROM is in each mode, and determines the static power based on these modes.

Logic Optimization

PowerArtist performs straightforward logic optimization to help increase accuracy. This is becoming more important from a power analysis accuracy point of view because more RTL code is either being generated from ESL synthesis tools or is being designed for re-use. Both flows can result in unused registers in the design, which can then result in unused combinational logic. The analysis tools locate unused register bits and then trace logic to eliminate unused combinational logic. They also optimize logic that generates constant outputs.

Optimization depends on analyzing the inferencing of a module at every instantiation; therefore, optimization needs to happen when the power analyzers run. PowerArtist may optimize registers only in certain instantiations—not all instantiations. In the report, instances that have either been fully or partially optimized are marked with an (O) after the model name. This is similar to how gated logic is marked with a (G) and logic that has its power set by the SetPower command is marked with an (F).

Pad Power Consumption

This section of the report shows the power consumed by each pad instantiated in your design.

4. Pad power consumption					
=====					
			Power (Watts)		
Component	Pad type	Cap (F)	Static	Dynamic	Total
-----	--- ---	-----	-----	-----	-----
top.sclk1	SEQPIC	281fF	154uW	866uW	1.02mW
top.sclk2	SEQPIC	59.4fF	154uW	874uW	1.03mW
top.sclk3	SEQPIC	22.1fF	154uW	0W	154uW
top.udio	SEQPIC	132fF	154uW	0W	154uW
<snip>					
Total pad power			44.2mW	751mW	795mW

For each I/O pad, several pieces of information are shown.

- The hierarchical name of the pad in the HDL design.
- The name of the pad model in the chosen technology for that pad.
- The power consumed by that pad.
- The load capacitance attached to the output of the pad

Power consumed by pads is often a significant fraction of total power and PowerArtist uses specific models for each pad in a technology to accurately estimate the power. Power is estimated using the PowerArtist-computed frequency value for the output net, parameters of the pad, and a user-provided value for the off-chip capacitance driven by the net if it is an output.

Clock Power Consumption

This section of the text power report shows the power consumed by each clock net. Details about clock gating prediction and clock tree inferencing are included in this clock power report section

```
5. Clock power consumption
=====

top.clk                                     Instantiated   5.5056nW  1.276mW   1.276mW

Net properties:
  Area           :   1.35e-10
  Senses         :   1
  Frequency      :  65.303MHz
  Transition time :   0s
  Fanout capacitance : wire   1.7675pF*;pins   45.1fF
  Wire power     :   1.257mW
  Pin power      :  32.073uW

Wireload models used:
  not used, capacitance back annotated

Traced instances:
Library   Clock   Driven By   Driven Net   Driven      Power (Watts)      Traced
Model     Level   Net Number   Numbers     Loads       Static   Dynamic   Total   Instance
-----
BUFX20    1         0         38         1         146pW   69.1uW   69.1uW   top.core1.I1
BUFX20    2         38        37         2         146pW   68.9uW   68.9uW   top.core1.L2_I1
INVX12    3         37        36         1         146pW   160uW   160uW   top.core1.L3_I1
INVX12    4         36        35         1         146pW   155uW   155uW
<snip>
Total power                                     5.54nW  232mW   2.32mW

Traced nets:
Net       Frequency   Transition   Wire       Pin       Net Wire   Net Pin   Net
Number    -----   -----   ---       Cap       Power (W)   Power (W)   ---
-----
1         15.831MHz    0s         2.61fF*   0F         481nW       0W         top.1
2         65.963MHz    0s         156fF*   70.03fF    52.2uW      23.4uW     top.2
3         65.963MHz    0s         173fF*   55.98fF    57.9uW      18uW      top.3
4         65.963MHz    0s         221fF*   31.63fF    74.1uW     10.6uW     top.4
<snip>
Total                                7.41pF   2.61pF    2.47mW    873uW
<snip>
```

- For each clock net, the following information is provided:
- The hierarchical net name, the type of analysis, and the total power for the clock

tree. The analysis type will be either Instantiated, which means that it was traced, or Inferred meaning clock inferencing was done.

- Area occupied by the net.
 - Clock senses.
 - Frequency of the clock net.
 - Transition Time.
 - Fanout capacitance of the clock net (divided into contributions from wire and pin capacitances).
 - Power consumed as the net toggles (divided as wire and pin components).
 - The wire load model applied to the net (the first entry is the net name, the second entry is the wire load model applied to that net). If you assigned wire load models to sub-nets of the clock tree, they would be listed here as well.
 - Descriptions of the instances that were traced as part of the clock tree, which includes the following:
 - **Library Model:** the Liberty model name.
 - **Clock Level:** the depth in the clock tree for this particular instance
 - **Driven Net Numbers:** the net numbers from the “Traced nets” section that this instance drives. Most instances will drive one net, but some instances may drive multiple nets. The power portion is split into static/dynamic/total just like in the Internal Power Consumption portion of this report.
 - **Driven by Net Number:** the net number that was traced through to get to this particular instance. Note that the “Driven by” net may have Index 0. This means that this is the clock net you are either inferring or tracing.
 - **Driven Loads:** the number of loads this particular instance is driving.
 - **Traced Instance:** the full hierarchical instance name (listed last to accommodate a long name).
- Note that if the instance is a leaf buffer that is driving only clock pins, it is marked with an (L) in the Clock Level column.
- Descriptions of each clock net in the design. This section lists every net in the clock tree and contains the following fields:
 - **Net Number.** this number allows you to figure out what instance drives this net
 - **Frequency:** the toggling frequency of this net.
 - **Transition Time:** the slew time either back-annotated or calculated by the slew calculator, the back annotated slew values are annotated with an asterisk. The power portion is split into Pin Power and Wire Power.
 - **Wire Capacitance:** capacitance determine from a wire load model or a back-annotated value. In the case of a back-annotated value, the number is annotated with an asterisk to indicate that it was a user-specified value.
 - **Pin Cap:** the total capacitance of the pins driven by this net.

- **Net Wire Power** and **Net Pin Power**: the total power separated into pin and wire amounts
- **Net**: the full hierarchical instance name for this net.

Controlling the Clock Tracing Order

The order in which clocks are traced does not impact power calculations, but it does impact how the power is associated with a particular clock. The contents of the clock power section in your power analysis reports depends completely on the order in which clocks are traced. The order in which you specify your clock nets in the clock file and if you specify the `-frequency` option to the [SetClockNet](#) command completely control the order in which nets are traced in a predictable fashion.

Clocks specified with frequencies in the clock file are traced first during power analysis, starting from the highest frequency clock down to the lowest frequency clock. If two nets have equal frequencies, the net listed first in the clock file is traced first. Clocks with no specified frequency are traced last, in the order they are listed in the clock file.

Additional Information on Clock Gating

If you performed clock gating, as was done in the [Running Power Analysis with Clock Gating](#) section, the following additional information is included in the clock power report.

- All output nets of clock-gated registers are listed on their own line.
- For each clock-gating cell (ICGC), the cell type and enable signal and duty percentage are listed.
- For each clock-gating cell (ICGC), the static, dynamic, and total power usage is provided.
- A separate section called Clock Gating Summary at the end of the Clock power consumption section of the report. This section reports information on instantiated ICGCs. See the following sample:

```
Clock Gating Summary:
-----
Clock net: top.clk
Number of inferred clock gating cells: 10
Number of registers gated by inferred clock gating cells: 376
Number of instantiated clock gating cells: 0
Number of registers gated by instantiated clock gating cells: 0
Total number of gated registers: 376
Total number of ungated registers: 77
```

The counts are the total number of register bits—not the number of register instances (where an entire register bank is one instance). The sum of “Number of

gated registers” and “Number of registers gated by instantiated clock gating cells” may not be equal to the “Total number of gated registers”. This is because a register may have a block-level instantiated ICGC and have a local inferred ICGC as well. The “Total number of gated registers” section includes registers that are either gated by inferred or instantiated ICGCs or both. The “Total number of ungated registers” section includes those registers that are not gated by either inferred or instantiated ICGCs.

For more information on clock gating, see [Setting up Clock Gating for Power Analysis](#).

Clock Domain Power Consumption

PowerArtist provides a mechanism for generating report that shows power consumption per clock domain. If you specify the `-trace_domain` option to the [SetClockNet](#) commands, you will get an additional section in the power report named “Clock domain power consumption”. This report is broken into the following sub-sections:

- **Clock domain power summary:** This section lists the power of each clock domain and the total power of all clock domains. In addition, it lists “Untraced instance power” which is the power of all instances that do not belong to any of the specified clock domains. This power is broken down further in a later section called, “Untraced Instances details”.
- **Clock domain details:** This section is broken down by clock domain. Each listed clock net has the following sub-sections:
 - **Instance Summary:** This section lists the power contribution for the clock domain based on different components such as internal power, pad power, core power, clock power, internal load power, etc. The format is identical to the “Total power consumption” section.
 - **Traced instances:** This section records the power of all of the elements traced in the given clock domain. This includes the clock distribution network leading up to the clock inputs of flops as well as the elements traced through the Q outputs of the flops. The format of this section is identical to the “Clock Power Consumption” section of the report. Note that the sequential elements will all be at clock level 1. The clock distribution network will still be traced as part of the clock power section of the report.
- **Untraced instances details:** This section contains the instances summary for various categories like internal power, pad power, clock power, etc. as well as a list of the untraced instances of design

The following report excerpt shows a sample Clock Domain Power Consumption section:

```
<snip>

6. Clock domain power consumption
=====

Note: This section reports power consumption on clock domain basis.
      Instances and nets in this section are also included
      as part of other relevant sections of this report.

6.1 Clock domain power summary
=====

                                Power (Watts)
Clock domain net                Static  Dynamic  Total
-----
top.clk                         14.4mW   307mW   322mW
top.pci_clk                     10.3mW   432mW   442mW
top.tck                         154uW     0W    154uW

Total clock domain power        24.9mW   739mW   764mW
Untraced instance power        23.1mW   29.1mW   52.1mW

6.2 Clock domain details
=====
<snip>

2. Clock Net: top.pci_clk

Instance Summary:
=====

                                Power (Watts)
Power contribution                Static  Dynamic  Total
-----
Internal power
  Internal register power    1.53uW   326uW   327uW
  Internal latch power        0W       0W      0W
  Internal memory power       0W       0W      0W
  Other internal power        704nW   21.3uW   22uW
  Total internal power        2.23uW   347uW   349uW
IP Core power                 0W       0W      0W
Pad power                     10.3mW   431mW   442mW
Clock power                   21.7nW   217uW   217uW
Internal load power           0W       0W      0W

Total power                   10.3mW   432mW   442mW
```



```

Traced instances:
Library      Clock  Driven By  Driven Net  Driven      Power (Watts)      Traced
Model        Level Net Number Numbers  Loads          Static  Dynamic   Total   Instance
-----
register      1      0          1568        2          19.9nW    2uW      2.02uW  top.core
mux21         2    (L)  1568        1569        1          8.92nW   173nW    182nW   top.core
mux21         2    (L)  1567        938         1          8.92nW   173nW    182nW   top.core
connect_inv   4      1564       1565        0           0W       0W       0W      top.c(r)

<snip>

6.3 Untraced instances details:
=====

Power contribution      Power(Watts)
-----
Static  Dynamic   Total
-----
Internal power
  Internal register power      0W      0W      0W
  Internal latch power         0W      0W      0W
  Internal memory power        0W      0W      0W
  Other internal power         1.22nW   2.6nW   3.83nW
  Total internal power          1.22nW   2.6nW   3.83nW
IP Core power                  0W      0W      0W
Pad power                      23.1mW  29.1mW  52.1mW

Total power                    23.1mW  29.1mW  52.1mW

Untraced instances:

Component      Model      Power (Watts)
-----
Static  Dynamic   Total
-----
top      user      24.1mW  31.2mW  55.3mW
#b0      auto      0W      0W      0W
core1    user      1.08nW  6.2nW   7.28nW
#b0      connect   0W      0W      0W
j1       user      0W      0W      0W
#b0      connect_inv 0W      0W      0W
p1       user      0W      0W      0W
#b0      connect   0W      0W      0W

<snip>

```

Note: The sample report has been modified to fit the available space. The Traced Instance names have been shortened and only one of the clock instances (top.pci_clk) is shown. To see the full report, simply add the -trace_domain option to each of the SetClockNet specifications in the power_aware.tcl script and run the tutorial as usual.

Inferred Buffer Tree Power

This section describes the buffer inferencing that happens when a net exceeds the number of fanouts specified by the [SetHighFanoutNet](#) command. Its organization is identical to that of the equivalent clock buffer section. For each inferred buffer tree, the following information is supplied:

- Name of the net with a fanout that exceeds the limit specified by the [SetHighFanoutNet](#) command.
- The hierarchical instance being driven by the net.
- The frequency of the net toggles.
- The number of loads.
- A description of the fanout tree. This includes the cells that were used to build up the buffer tree, the library the cells came from, the counts of the cells, fanout limitations of the cells and the amount of power consumed by the cells. The capacitance seen in the buffer tree is also supplied because that is what controls the number of cells inferred.

```
6. Inferred buffer tree power
=====
Total dynamic power =      4.9uW
Total static power =     139nW

    Inferred Buffer Tree:
        Net name      : top.core1.t1.f1.WR_PTR[8]
        Driver instance: top.core1.t1.f1.WRCNTR.#1180
        Frequency     :   82.5KHz
        Number of Loads: 74
        Leaf Driver   :
            Cell      : SEQCLKBUF8MTH
            Library   : hvt
            Count     : 2
            Cell maximum fanout :    60
            Buffer power: static    8.26nW; dynamic    86.3nW
            Fanout capacitance: wire    740fF; pins    97.2fF

        Total capacitance : wire    740fF; pins    97.2fF
<snip>
```

Area

This section of the text power report shows the width and height, and number of registers and gates for each component. This report is generated if you use the “-average_report_options a” option to CalculatePower.

7. Area				
=====				
Component	Width	Height	Regs	Gates
-----	-----	-----	----	----
top	1.98K	1.98K	694	3070942
core1	1.98K	1.98K	694	3070942
a1	10.9	10.9	0	93
<snip>				
Total Counts			694	3070908
Total Net Routing Area	346K			
Total Area (Gates+Routing)	4.28M			

Net Frequencies

This section of the text power report shows the type, glitch, and frequency for each net. This report is generated if you use the “-average_report_options g” option to CalculatePower.

Net	Type	Glitch	Frequency
---	---	-----	-----
top.Pclk	PI	0.00%	63.2MHz
top.Pnreset	PI	0.00%	0Hz
top.Pcs	PI	0.00%	0Hz
top.Prd	PI	0.00%	0Hz

Power Consumption by Model/Gate Type

This section provides summary information for every type of instance in the design. This report is generated if you use the -vertical_report_instances option to CalculatePower. For inferred elements, the cell count equals the number of separate instances in the design and not the total bit width for bundled elements like registers and muxes. For vendor_gates, the cell count is the number of gates of that type in the design.

Power (Watts)					
Component	Model	Cell Count	Static	Dynamic	Total
-----	-----	-----	-----	-----	-----
top		2083	305nW	396mW	396mW

Register Power	523	76.3nW	10.1mW	10.1mW
Latch Power	0	0W	0W	0W
Memory Power	20	0W	369mW	369mW
Other Power	1540	229nW	16.2mW	16.2mW
AND2X1	18	2.62nW	377nW	379nW
AND3X1	6	875pW	30.7nW	31.6nW
AND4X1	66	9.62nW	1.5uW	1.51uW

Summary

- You can use PowerArtist power analysis at various stages of the design implementation:
- While designing your RTL
Use PowerArtist concurrently with RTL simulation to analyze power and identify hot spots. If the design consists of mixed RTL and gate-level models, use PowerArtist and apply simulation data as it becomes available for individual modules or subsystems to improve the analysis accuracy for those blocks.
 - After synthesizing
Once you have used PowerArtist at the RT-level to decide among design alternatives, you will begin to synthesize portions of your design. Since logic synthesis can substantially restructure control logic to minimize area or meet timing constraints, you might want to repeat power analysis using post-synthesis information. This can be done on a mixed RTL and gate-level design or on a fully synthesized gate-level netlist.
 - After place and route
Use PowerArtist with back-annotated capacitance values in a gate-level simulation to use parasitic information from layout in the analysis.

Chapter 5

PowerArtist Tutorial Part II: Power Reduction

Introduction

This tutorial takes you through power reduction on a full-chip mixed Verilog/VHDL design. You have two methods available to run the reduction tutorial—a scripted flow with command files or the PowerCanvas wizard flow. For either flow, you will run the same basic four steps:

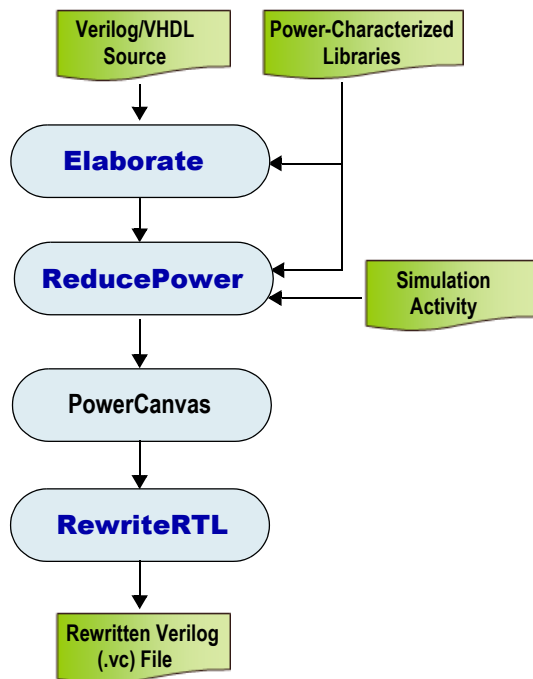
Four easy steps:

1. Compile Design

2. Run Power Reduction

3. Review Results/Schedule changes in PowerCanvas

4. Rewrite Scheduled RTL Changes



If you run power reduction using command files, you will most likely want to use the PowerCanvas to view the results and schedule design changes; therefore, it is

recommended that even if you intend to mainly use a command-line flow you still familiarize yourself with how the PowerCanvas works.

About PowerBots

Power reduction in PowerArtist is based on modules called “PowerBots”. These PowerBots scan your design looking for a specific design feature, such as an enable signal that can be constructed to turn off unobserved register toggles. A PowerBot performs an analysis of the topology of your design and activities on nets and then makes recommendations on potential changes you should make to your design. It reports either power savings numbers or power wastage numbers to help you make your decision. If it can determine an example change to make, it will provide code snippets as well. For details on the different PowerBots available, see [PowerBot Overview](#).

Tutorial Organization

The following subjects are covered in this chapter:

- [Copying the PowerArtist Tutorial Files](#)
- [Running the Reduction Tutorial Using Command Files](#)
- [Setting Important Environment Variables](#)
- [Basic Flow for Running the Reduction Tutorial](#)
- [Reviewing Reduction Results in the PowerCanvas](#)
- [Running the Reduction Tutorial Using the PowerCanvas Wizards](#)
- [Reviewing the Output Reports](#)
 - [Reviewing the Power Reduction Report](#)
 - [Reviewing the Power Reduction Clock Gating Report](#)

Copying the PowerArtist Tutorial Files

There are two tutorials in the tutorial directory. It is recommended that you copy the entire contents of the tutorial directory to a local run directory:

```
cp -r $POWERTHEATER_ROOT/tutorial .
```

Contents of the Top-Level Tutorial Directory

The following table lists the contents of the entire tutorial directory.

Directory Name	Description
reduction	Contains the reduction tutorial files. This tutorial teaches you how to perform an RTL power reduction analysis and is fully documented in this chapter.
analysis	Contains the analysis tutorial files. These tutorials teach you how to perform HDL inferencing, vector analysis and average and time-based power analysis. This tutorial is not described in this chapter. For instructions on running the reduction tutorial, see PowerArtist Tutorial Part 1: Power Analysis .
libraries	Contains all of the technology libraries required to run the tutorials.
settings/ global_setting.tcl	This file illustrates how you can establish site-wide tool settings that are shared among users. These settings are leveraged in all of the tutorials.

The reduction design is a slight variation of the full-chip design used in the analysis portion of the tutorial.

Running the Reduction Tutorial Using Command Files

To access the tutorial files, change directories to the `full_chip` directory:

```
cd tutorial/reduction/full_chip
```

The following table lists the contents in the `reduction /full_chip` directory:

Directories/File Names	Description
README	Provides the basic flow for running this tutorial.
ptshell.ini	Initialization file specific to the full-chip tutorial. This file sets the ptshell prompt and specifies the names of the <code>.log</code> and <code>.key</code> files for this tutorial.
<i>command.tcl</i> files	Tcl scripts meant to be run from within the ptshell that perform the different phases in the power reduction tutorial. These are identical to the operations you perform using the interactive wizards.
ptSourceFiles.tcl	Contains a list of VHDL libraries and VHDL files in your design in the order in which they must be analyzed and elaborated. It consists of AddLibrary and CompileFile commands. For details on this file, see ptSourceFiles.tcl File Format .
project_settings.tcl	A sample file that illustrates how you can incorporate site-wide tool settings into your design flow.
power_aware.tcl	Contains the power-aware commands required to run the tutorial. This file contains both design environment settings (libraries, wire load models, and memory definitions) and clock gating constraints.
analysis_wizard.tcl	This file is used by the reduction wizard to source the <code>project_setting.tcl</code> file and the <code>power_aware.tcl</code> file.
txrx.vc	Verilog startup file for the full chip that tells the Verilog compiler which Verilog files must be loaded for your design.
run_Elaborate.tcl run_ReducePower.tcl run_RewriteRTL.tcl	Run scripts to invoke from the Unix shell. As an alternative method, you could run the tutorial by typing the following commands at the Unix prompt: <pre>ptshell -tcl run_Elaborate.tcl ptshell -artist -tcl run_CalculatePower.tcl ptshell -artist -tcl run_RewriteRTL.tcl</pre>

In addition to the tutorial sub-directories, there is also a `reduction/design_data` directory (at the same level as `full_chip`) that contains design data for the full-chip design. It includes RTL simulation data (FSDB and VCD files) and RTL source data (Verilog and VHDL files).

Setting Important Environment Variables

To run this tutorial as it is written in this chapter, you must set the PT_APPLY_SMW variable to true (1).

```
setenv PT_APPLY_SMW 1
```

This enables the SplitMemoryWords PowerBot during power reduction. It is disabled by default. In addition, you must also set the PT_ENABLE_HIER_PRISM_REWRITE variable to true (1).

Basic Flow for Running the Reduction Tutorial

The basic flow for this tutorial is as follows:

1. Type “ptshell -artist” to start up the PowerArtist shell. The -artist option is required for rewrite as well as reduction analysis. PowerArtist reads in the ptshell.ini file at this point.

2. From the ptshell prompt, source the project_setting.tcl script to setup project-specific parameters and read your libraries.

```
reduction_full_chip_ptshell % source project_settings.tcl
```

3. Source the Elaborate.tcl script to compile the design and create a PowerArtist scenario file:

```
reduction_full_chip_ptshell % source Elaborate.tcl
```

4. Source the power_aware.tcl file to read in the power constraints for this tutorial:

```
reduction_full_chip_ptshell % source power_aware.tcl
```

5. Source the ReducePower scripts run power reduction analysis:

```
reduction_full_chip_ptshell % source ReducePower.tcl
```

Remember -artist option is required to perform a reduction analysis. If you forget the -artist option in step 1, you will get the following error message:

```
ptshell: Error 9690: Feature 'sptWSEngine' not enabled in 'PowerArtist PT'
```

6. View the results in the PowerCanvas and schedule changes manually (optional).

You could choose to just accept the changes that PowerArtist automatically accepts for you and skip this step.

7. Restart the ptshell.

8. Reload the power_aware.tcl file.

```
reduction_full_chip_ptshell % source power_aware.tcl
```

9. Source the RewriteRTL.tcl script to run an RTL rewrite:

```
reduction_full_chip_ptshell % source Rewrite.tcl
```

Note that you need to have a feature license for automated rewrite to be able to run this step.

Using the Command Files

This section describes each step in the command file flow and the contents of the .tcl command files in detail.

1. Open the `project_settings.tcl` file. This file is shown here:

```
# Obtain global site-wide settings
source ../../settings/global_settings.tcl

# Specify the technology libraries
source libraries.tcl
```

This file obtains site-wide tool settings (from the `global_setting.tcl` file) and reads-in library information (from the `libraries.tcl` file). The `libraries.tcl` file contains a series of [ReadLibrary](#) commands. Creating this type of file saves you time and requires fewer keystrokes.

2. Open the `power_aware.tcl` file and examine the contents as shown here:

```
#####
# Set up the design environment

SetLibrary -instance top -library {hvt RETENTION_EXAMPLE_LIB Memories}
SetWireLoadModel -name cmos65_wl10 -instance top -library hvt

#####
# Memory commands
DefineMemory -library Memories \
    -access_enable { WENA WENB } \
    -read_address { AA* AB* } \
    -write_address { AA* AB* } \
    -data { QA* QB* DA* DB* } \
    -memory_enable { CENA CENB } \
    -cell { DP*}

#####
# Clock gating commands

SetClockNet -name top.clk -mode infer -gate_clock yes
SetClockNet -name top.pci_clk -mode infer -gate_clock yes
SetClockNet -name top.tck -mode infer -gate_clock yes
SetClockGatingStyle -clock_cell_attribute latch_posedge_precontrol
SetClockBuffer -type root -name SEQCLKBUF4MTH -library hvt -fanout 2
SetClockBuffer -type branch -name SEQCLKBUF8MTH -library hvt -fanout 2
SetClockBuffer -type leaf -name SEQCLKBUF8MTH -library hvt -fanout 60
```

3. Make special note of the following information in this Tcl file:

- The [DefineMemory](#) command defines a memory that the [Gate Memory Clock](#) (GMC) PowerBot will analyze during reduction.
- The [SetClockNet](#) command defines the three clocks in the design and indicates that synthesis will perform clock gating on all of the clocks. Therefore, do not attempt to perform a standard [Clock Enable Condition Linter](#) (CEC) analysis.
- The [SetClockGatingStyle](#) command specifies the value of the `clock_gating_integrated_cell` attribute, which in this case `latch_posedge_precontrol`. During reduction analysis, PowerArtist searches for this attribute to determine candidate clock gating cells.
- The [SetClockBuffer](#) commands specify three types of clock buffers: root, branch and leaf. when you want to infer a clock tree. These are required when you want to infer a clock tree.

4. Open the Elaborate.tcl file as shown below:

```
# Elaborate the design
set design top

# Read in the design
source ptSourceFiles.tcl

Elaborate \
  -elaborate_write_power_db true \
  -elaborate_log ElaborateBatch.log \
  -power_db_name $design.Batch.pdb \
  -scenario_file $design.Batch.scn \
  -top $design \
  -verilog_2001 true \
  -verilog_startup_file txrx.vc
```

5. Make special note of the following information in this command file:

- The “set design top” is a simple standard Tcl variable that sets the top module to “top”. This defined “design” variable is used in the rest of the script.
- Since the full-chip design is a mixed design that has some Verilog and some VHDL, this script sources the `ptSourceFiles.tcl` and specifies the `-verilog_startup_file` option to the [Elaborate](#) command.
- The `-elaborate_write_power_db` option writes out a power database (.pdb) file. When Elaborate runs, it creates this power database.

This power database is an OpenAccess™ database (OADB) that contains a hierarchical netlist representing your design as well as power analysis and reduction values. PowerArtist uses this database to generate hierarchical tree and schematic displays of your design, and to store power analysis and reduction information. You are able to write your own applications that access the netlist and power-related information. For more information, see [Writing OpenAccess Database Applications](#).

- The resulting scenario file, as specified with the `-scenario_file` option, will be named `top.Batch.scn` (since `$design` is set to “top”).
6. Open the `ReducePower.tcl` file. The variables and commands in this file run a power reduction analysis using the [ReducePower](#) command.

```
Calculate and reduce the average power of the design
set design top
ReducePower \
  -activity_file ../design_data/rtl_sim/txrx_sleep.vcd.gz \
  -reduction_report_file $design.Batch.srpt \
  -reduction_report_options agip \
  -reduction_overwrite_power_db true \
  -compress_gaf true \
  -default_output_load 3.9e-11 \
  -finish_time 12135580ps \
  -gaf_file $design.Batch.gaf \
  -reduction_log ReducePowerBatch.log \
  -power_db_name $design.Batch.pdb \
  -reduction_hierarchy full
  -save_clock_trees_netlist true
  -scenario_file $design.Batch.scn \
  -skip_reduction_list {}
  -start_time 6014730ps \
  -top_instance txrx_tst.top1 \
  -use_scan_flops true \
  -wireload_library hvt
  -reduction_topology true \
  -memory_disable_edge_detection true
```

Make special note of the following information in this command file:

- The “`set design top`” is a simple standard Tcl variable that sets the top module to “top”. This defined “design” variable is used in the rest of the script.
- This reduction analysis uses a gzipped VCD file as activity information.
- The power reduction report will be called `top.Batch.srpt` and the clock gating report will be called `top.Batch_cg.srpt`.
- This reduction run will overwrite any previously generated power database.
- The reduction analysis starts at 6014730ps and finishes at 12135580ps.
- The power database will contain information on the clock tree(s) in the design. If you do not specify the `-save_clock_trees_netlist` option, by default, clock tree information will not be part of the power database.
- The scenario it reads in is the one that the `Elaborate` command creates called `top.Batch.scn`.

- By default, **Split Memory Words (Beta)** (SMW) is turned off as “smw” is the default value for the `-skip_reduction_list` option. The “`-skip_reduction_list {}`” entry in this script enables the SMW PowerBot for this tutorial.
 - The top instance in the design is specified as `txrx_tst.top1`. This provides the link between the VCD file hierarchy represented by `$scope` statements and the actual design. The `-top_instance` value must be equivalent to the `-top` module value specified to the `Elaborate` command.
 - The log file that this reduction generates will be named `top.ReducePowerBatch.log`.
 - Wireload libraries will be used during the power analysis. The Liberty logical library name that will contain the libraries is `hvt`.
 - This reduction runs in “topology mode” meaning that `-reduction_topology true`
7. Type “`ptshell -artist`” to start up the PowerArtist shell. The prompt for this tutorial will be:

```
reduction_full_chip_ptshell %
```

As soon as the `ptshell` starts, you will see the following additional files in your run directory:

- `reduction_full_chip.key`—Records keystrokes run as part of this tutorial. You can re-run a session by sourcing this file.
 - `reduction_full_chip.log`—Records log information (inputs and outputs) for this tutorial. capturing the output of standard out.
8. From the `ptshell` prompt, source the `project_setting.tcl` script to setup project-specific parameters and read your libraries.

```
reduction_full_chip_ptshell % source project_settings.tcl
```

Note: You can take advantage of the command completion feature of the `ptshell`. For example, you can simply type “`source p`” followed by the **Tab** key and to see a list of files that match. Type additional characters followed by **Tab** until the ambiguity is resolved. Using this feature will save you time and cut down the number of keystrokes required to run the command-line tutorial.

9. Source the `Elaborate.tcl` script to compile the design by running the following command:

```
reduction_full_chip_ptshell % source Elaborate.tcl
```

When the script is finished, you will see the following files in your run directory:

- `ElaborateBatch.log`—This file captures log information for the `Elaborate` command. If you get any unexpected results, you can look through this file to make sure that the settings listed are those that you expected to be set.
- `top.Batch.scn`—The scenario file for the design. The scenario file is a binary representation of your design.

- top.Batch.pdb—At this point in the flow, this PDB file contains the design hierarchy. You can use the PowerCanvas to explore the design hierarchy at this point. The power reduction run will change this database.
- 10. If you want, you can view the elaborated design (top.Batch.pdb) in the PowerCanvas. For details see, step 2 of [Running the Tcl Scripts](#) in the analysis tutorial.
- 11. Source the power_aware.tcl file to read in the power constraints for this tutorial:

```
reduction_full_chip_ptshell % source power_aware.tcl
```
- 12. Source the ReducePower script to run power reduction analysis:

```
reduction_full_chip_ptshell % source ReducePower.tcl
```

Remember -artist option is required to perform a reduction analysis.

When the script is finished, you will see the following additional files in your run directory:

 - ReducePowerBatch.log—A log file for the average power analysis run. It includes all of the command option specifications, warnings and notes generated by the analysis engine, CPU usage, etc. for the latest run.
 - top.Batch.srpt—The power reduction report in text format. This is a high-level report of the power numbers in your design. For details on this report, see [Reviewing the Power Reduction Report](#).
 - top.Batch_cg.srpt—The power reduction clock gating report. This report provides detailed information about the clock gating in this design. For details on this report, see [Reviewing the Power Reduction Clock Gating Report](#).

Also note that the top.Batch.pdb has changed since the Elaborate run.
- 13. Exit the ptshell.
- 14. Review the power analysis results in the PowerCanvas and modify the scheduled reduction opportunities. For details, see [Reviewing Reduction Results in the PowerCanvas](#).
- 15. Open the Rewrite.tcl file, an excerpt of which is shown below:

```
<snip>
RewriteRTL \
  -elaborate_write_power_db true \
  -rewrite_log RTLRewriteBatch.log \
  -output_rtl_dir_name ./rewrite.Batch \
  -memory_def_file mem_cr.tcl \
  -power_db_name $design.Batch.pdb \
  -rewrite_report_file rewriteBatch.rpt \
  -top $design \
  -verilog_2001 true \
  -verilog_startup_file txrx.vc
```

This command reads your power database file, reads your original Verilog identified by the `-verilog_startup_file` option and then creates a directory called `./rewrite.Batch` containing the rewritten RTL code files based on the reduction opportunities recommended by the PowerArtist power reduction (with any manual modifications you may have made). This directory must be empty and exist in the given location. The `-memory_def_file` option specifies a file containing memory definitions in case you want to automatically check your rewritten RTL (if you have GMC rewrite opportunities).

16. Type “ptshell” to restart the ptshell.

17. Source the `project_settings.tcl` file again.

```
reduction_full_chip_ptshell % source project_settings.tcl
```

You must reload this file whenever you exit and restart the ptshell in this tutorial flow.

18. Source the `Rewrite.tcl` script to perform RTL rewrite by running the following command:

```
reduction_full_chip_ptshell % source Rewrite.tcl
```

Note that you need to have a feature license for automated rewrite to be able to run this step. When the script is finished, you will see the following additional subdirectory and files in your run directory:

- `./rewrite.Batch`—The Directory containing the rewritten RTL code (PowerArtist does not overwrite your original RTL).
- `rewriteBatch.rpt`—This file records the names of the files that were automatically rewritten. It has the following syntax:

```
#  
# Original_RTL_File          Power_Optimized_RTL_File  
#  
original_file_name          rewritten_file_name  
...
```

By comparing the original and rewritten files using the UNIX “diff” command, you will be able to see the changes the rewrite process made to your design.

- `RTLRewrite.Batch.log`—A log file for the RTL rewrite process. It includes all of the command option specifications, warnings and notes generated by the rewrite process, CPU usage, etc. for the latest run. In the log file, you will find the following section indicating the changes that were written:

```
partlrewrite: Note 7079: Writing report file : rewriteBatch.rpt  
partlrewrite: Note 7079: Writing CheckRewrite's interface file :  
rewriteBatch.cr.tcl  
partlrewrite: Note 7099: Summary for design top  
                        Predicted Gated Bits (LNR) : 5  
                        Predicted Clock Gating Logic (LNR) : 1
```

```
Rewritten Gated Bits (LNR) : 5
Rewritten Clock Gating Logic (LNR) : 1
Predicted Gated Bits (Prism) : 69
Predicted Clock Gating Logic (Prism) : 3
Rewritten Gated Bits (Prism) : 69
Rewritten Clock Gating Logic (Prism) : 3
Predicted Gated Memory Instances (GMC) : 4
Rewritten Gated Memory Instances (GMC) : 4
partlrewrite: Note 7036: Writing power optimized rtl
partlrewrite: Note 7039: Writing out verilog startup file ./rewrite.Batch/
rewrite.vc
```

- This section describes the predicted rewrite opportunities and those PowerArtist was actually able to rewrite.

5. From the PowerCanvas main menu, click the **View** menu item and notice the last three menu items. Each of these menu items opens a unique dialog that is optimized to show the results of particular PowerBots:
 - **Simple Power Reductions:** from non-linter PowerBots that include [Low-Activity Non-Enabled Register \(LNR\)](#), [Gate Memory Clock \(GMC\)](#), [Local Explicit Clock Enable \(LEC\)](#), [Datapath Operator Isolation \(DOI\)](#), [Observability Don't Care \(ODC\)](#), and [Split Memory Words \(Beta\) \(SMW\)](#)
 - **Linter Power Reductions:** from linter PowerBots that include [Clock Enable Condition Linter \(CEC\)](#), [Memory Power Linter \(MEM\)](#), [Register Power Linter \(REG\)](#) and [MUX Power Linter \(MUX\)](#).
 - **Advanced Power Reductions:** reductions found by the [Prism](#) PowerBot. To review sample Prism results, see [Understanding the ASCII Output of the Prism PowerBot](#).

Note that in the simple and linter reduction dialogs, the opportunities use the acronyms for the PowerBots instead of their full names (that is, GMC instead of Gate Memory Clock).

Examining Simple Power Reductions

To view these results on a module by module basis, select **View > Simple Power Reductions**. This opens the dialog shown in the following figure.

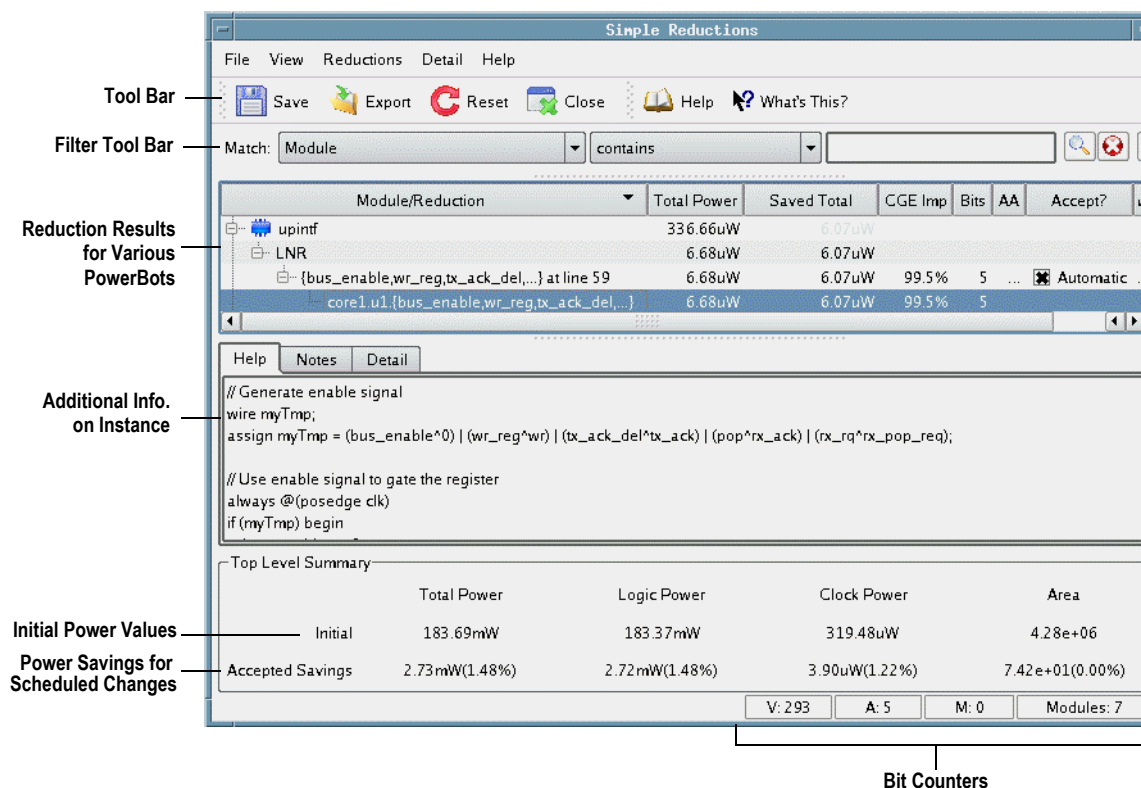


Figure 27 Simple Reductions Dialog

Note: All data that is shown in gray text is summarized data that consists of multiple parts. When you expand each entry by clicking the + signs, the values will display in black text. For definitions of each of the available column headers, see [Column Header Definitions for the Simple Reductions Dialog](#).

Note also that registers are displayed as either vectors or as a concatenated list of scalars.

Definitions for the Bit Counters in the Simple Reduction Dialog

In the bottom right corner of all reduction dialogs, in the status bar, you will see a set of four bit counters with the following definitions:

- V(iable)—the total number of bits that could be accepted for either manual or automatic rewrite.
- A(utomatic)—the total number of bits that have been accepted for automatic rewrite.

- M(anual)—the total number of bits that have been accepted for manual rewrite.
- Modules—the number of Verilog modules or VHDL architectures that are displayed in the dialog.

Navigating the Dialog

The toolbar at the very top can help you efficiently navigate this dialog. It has the following functions:

 Save	Saves the current state of the data, including any scheduled reductions.
 Export	Exports the data to a CSV file. For details on this process, see Exporting the Data to a CSV File .
 Reset	Restores the data back to the original state at the time you opened the dialog. Any data/changes you previously saved is retained.
 Close	Closes the dialog without saving any changes you made to the data. However, unless you quit PowerArtist all together, any changes that were in progress will continue to appear in the dialog when you re-open it. To apply these changes in preparation for an RTL rewrite, you must click the Save button.
 Help	Displays context-sensitive Help information.
 What's This?	Invokes a mechanism by which you can display information on each of the columns.

This toolbar is divided into two separate bars: File and Help. The File toolbar consists of the first four icons. The Help toolbar consists of the final two. You can click and drag the dotted line divider between the two bars to change their positions. If you right-click on the toolbar, a pop-up menu displays. This menu allows you to choose the parts of the toolbar that display and in what manner. This toolbar is part of the Linter and Prism dialogs as well.

Match pull-downs on the left hand side of the filter tool bar should be self explanatory and represent simple ways to filter the reduction data.

Using the What's This? Feature

The What's this? feature provides additional information on the definitions of column headers/categories on the reduction dialogs. To use it, simply click on the What's

This? icon on the tool bar and then on any column header, as shown in the following figure.

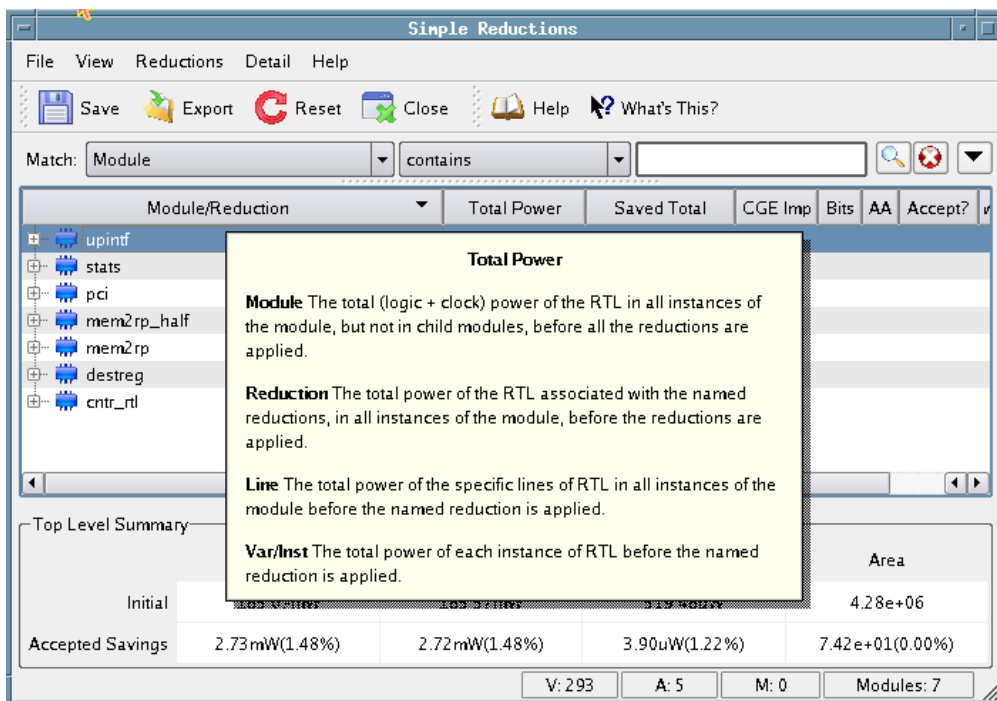


Figure 28 What's This? Pop-up Definition for Total Power

For high-level definitions of each column header, see [Column Header Definitions for the Simple Reductions Dialog](#).

Manipulating the Data

If a module with potential reductions is instantiated multiple times, you will get a better idea of the contribution to the overall power saving if you can see power savings for all instances of the module together. To allow you to do this, PowerArtist organizes the available reductions by module. The power saving for each instantiation is shown in rows that are colored light blue.

For example, if you expand out **pci > LNR > pci_dout[63:0] at line 69**, you will see that there is one instance of this change, as shown in the following figure:

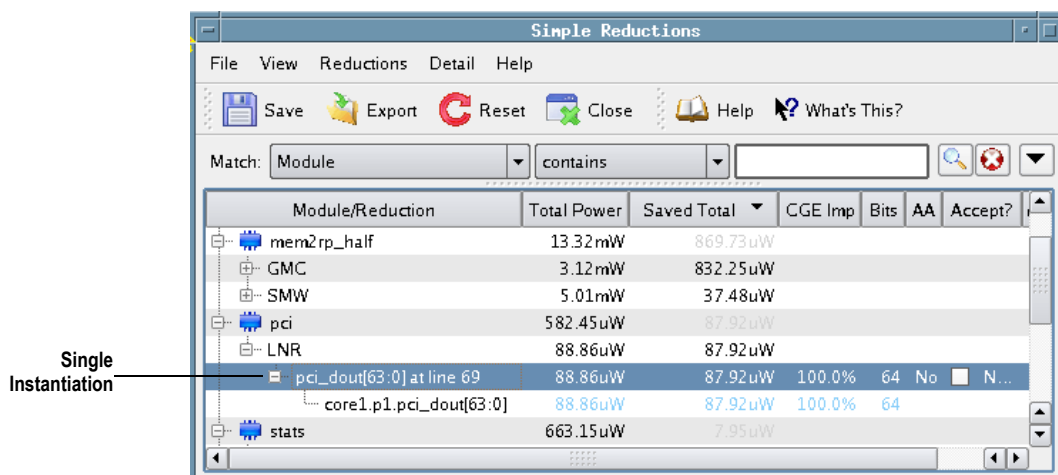


Figure 29 Sample of LNR PowerBot Expanded to Show Instances

Note that you will have to expand the width of the Module/Reduction field by dragging the right field separator to see all of the text shown in this figure.

If you right-click any entry, a pop-up menu displays as shown in the following figure:

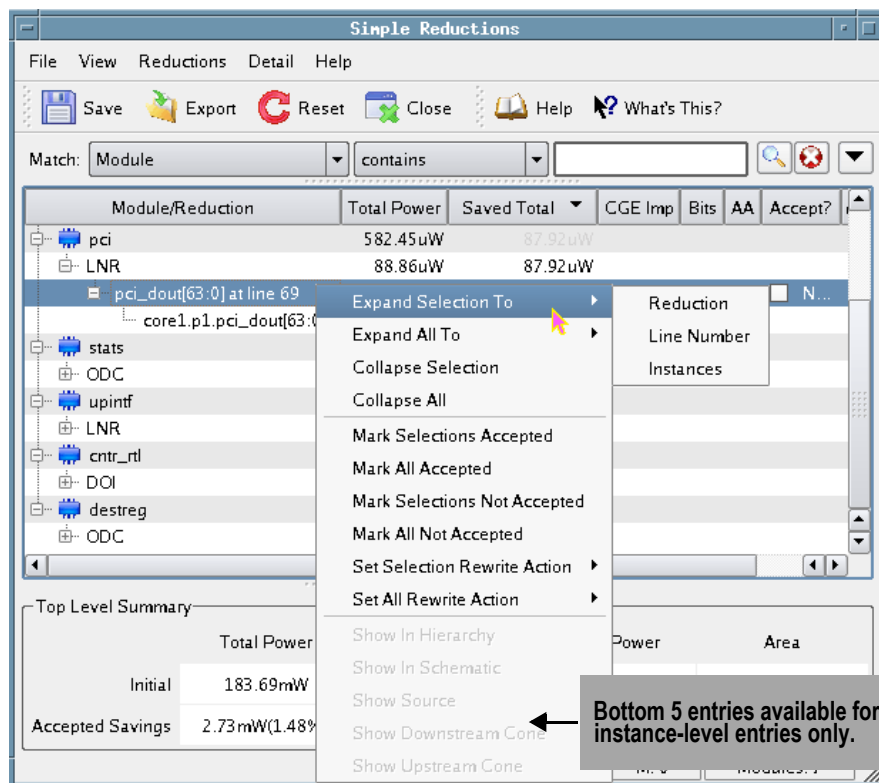


Figure 30 Pop-up Menu for Instance-Level Entry

If you are on an instance-level entry, this menu allows you to cross-probe, for example, to the hierarchy display, schematic or source and to trace cones of logic (upstream or downstream).

The two right-most columns as shown here (Accept and Rewrite?) help you manage decisions on whether or not to accept a particular reduction opportunity and if so, to implement them manually or automatically. For details, see [Scheduling Automatic RTL Changes](#).

Shortcuts for PowerBot Dialogs

The PowerCanvas also supports the following keyboard shortcuts for navigating any of the PowerBot dialogs.

- Up/Down arrow moves the cursor up and down the rows.
- Left/Right arrow for a row expands/collapses the levels of data displayed.
- Asterisk completely expands a row.

Using the Context-Sensitive Help Pages

If you click the **Help** button in the Simple Power Reductions dialog, you will see context-sensitive Help pages. If you click the **Help** button before clicking on any of the results, you will see the Reduction Index. This page lists each of the PowerBots on this dialog. From here, you can click on any of the links to get information on a specific PowerBot.

If you first click on a line in the reduction results and then you click the **Help** button, you will see help pages for that specific PowerBot. For example if you click on an LNR line in the reduction results, and then click the **Help** button, you will see a detailed help page for the Low Activity Enable PowerBot.

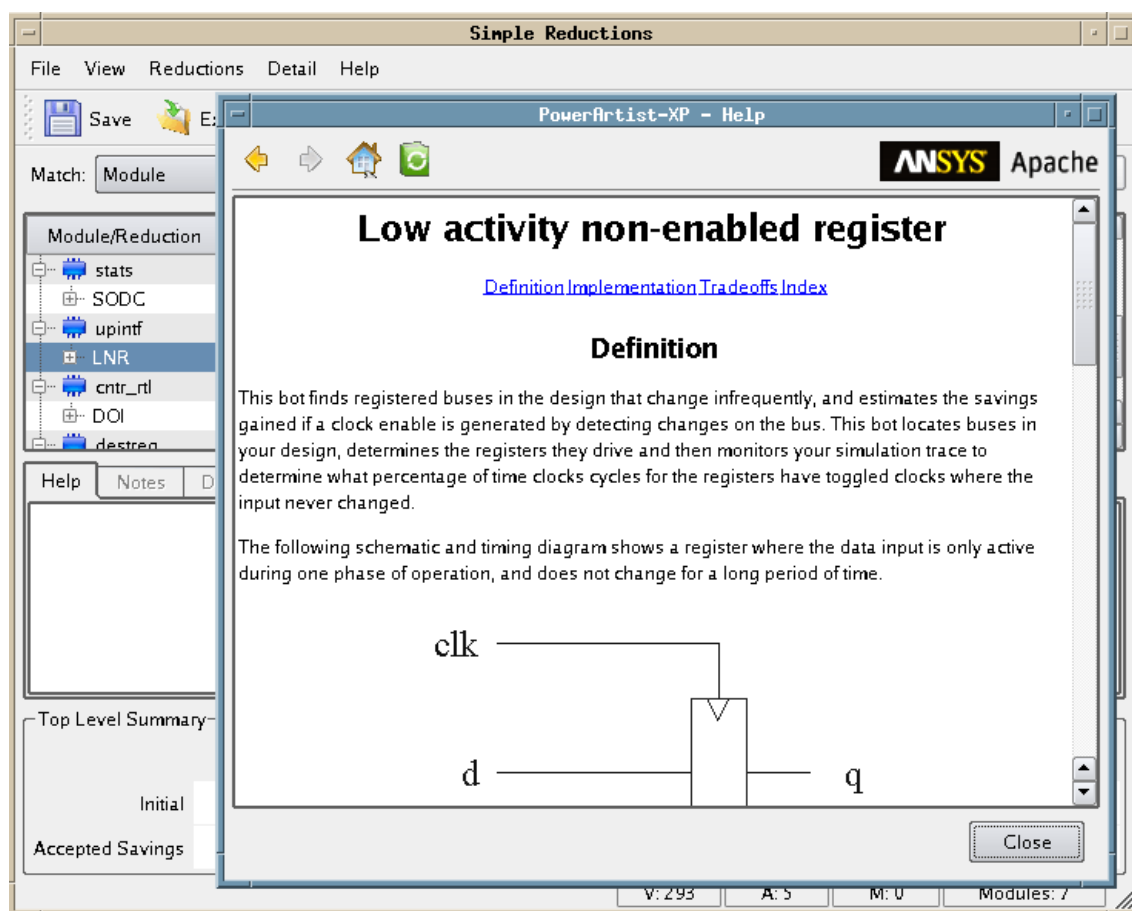


Figure 31 Sample Context-Sensitive Help Page for LNR PowerBot

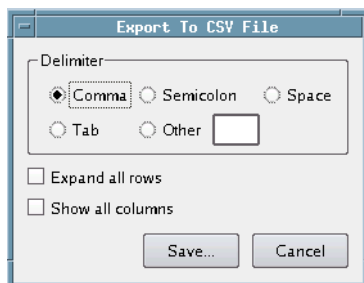
The Help pages work similarly in the Linter Reductions and Prism dialogs.

Exporting the Data to a CSV File

Use the following process to save the reduction data as a CSV file:

1. Click the **Export** button on the tool bar.

The Export to CSV File dialog appears as follows:



2. Select the appropriate delimiter (comma, semicolon, space, tab, other).
3. You can optionally dump rows fully expanded (without having to expand them in PowerCanvas). Additionally, you can show all columns or just the visible ones (default).
4. Click the **Save** button.
A file dialog appears.
5. Select an existing file or specify a file name and click the **Save** button.

The CSV file is written to disk.

You can then import this data into Excel™ or process it with UNIX scripts using a utility like awk or Tcl.

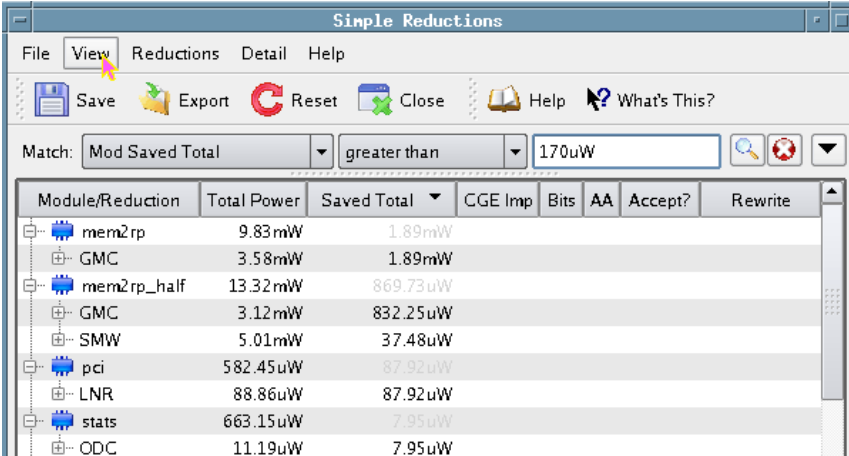
Filtering Reduction Results

You can use the filter tool bar to filter your data in a variety of ways. Once you have selected your filter criteria, use the  magnifying glass (or just hit the Enter key) or the  cancel icon to either perform or cancel the filtering, respectively. For example, you could perform the following filtering:

1. On the Simple Reductions dialog, in the Match: field, select **Mod Saved Total** from the first pull-down and **greater than** from the second.
2. Enter a value of 170uW and click the  button. This filters out all line-level reductions that do not reduce power by at least 170 uW.

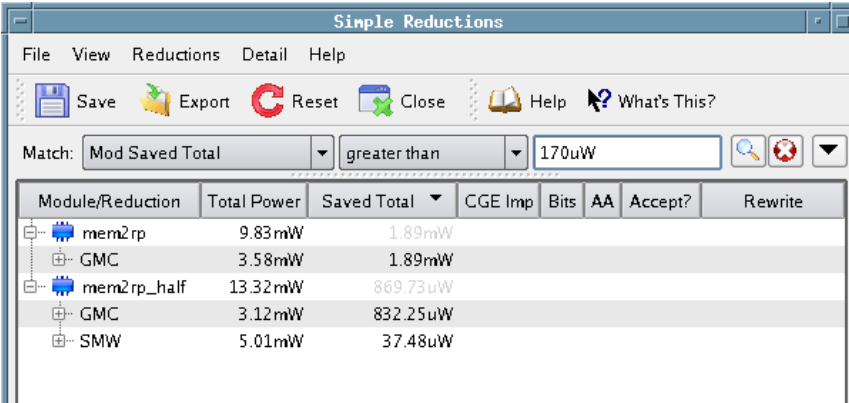
The following figures show the results before and after the filtering is applied.

Original Results



Module/Reduction	Total Power	Saved Total	CGE Imp	Bits	AA	Accept?	Rewrite
mem2rp	9.83mW	1.89mW					
GMC	3.58mW	1.89mW					
mem2rp_half	13.32mW	869.73uW					
GMC	3.12mW	832.25uW					
SMW	5.01mW	37.48uW					
pci	582.45uW	87.92uW					
LNR	88.86uW	87.92uW					
stats	663.15uW	7.95uW					
ODC	11.19uW	7.95uW					

Filtered Results



Module/Reduction	Total Power	Saved Total	CGE Imp	Bits	AA	Accept?	Rewrite
mem2rp	9.83mW	1.89mW					
GMC	3.58mW	1.89mW					
mem2rp_half	13.32mW	869.73uW					
GMC	3.12mW	832.25uW					
SMW	5.01mW	37.48uW					

Figure 32 Simple Reduction Results Before and After Filtering

The triangle on the far right allows you to do more complex filtering. You can build up boolean combinations of filters using the + and - buttons.

3. Take a close look at the reduction results.

Click on any of the column headers to sort the results in either ascending or descending order. To rearrange the column headers, click on a header and drag it left or right using the mouse.

4. Click the  icon to cancel the filtering and re-display all results.

For information on using the multi-level advanced filtering features, see [Using the Advanced Sorting and Filtering Features](#).

Using the Help, Notes and Detail Tabs

The third and bottom pane in all of the reduction dialogs contain three tabs: Help, Notes and Detail.

To see a Help message for a particular entry, do the following:

1. Expand the **pci > LMR > pci_dout[63:0] at line 69** row.
2. Select the **core1.p1.pci_dout[63:0]** line and click the Help tab in the bottom pane of this dialog (as shown in the following figure).

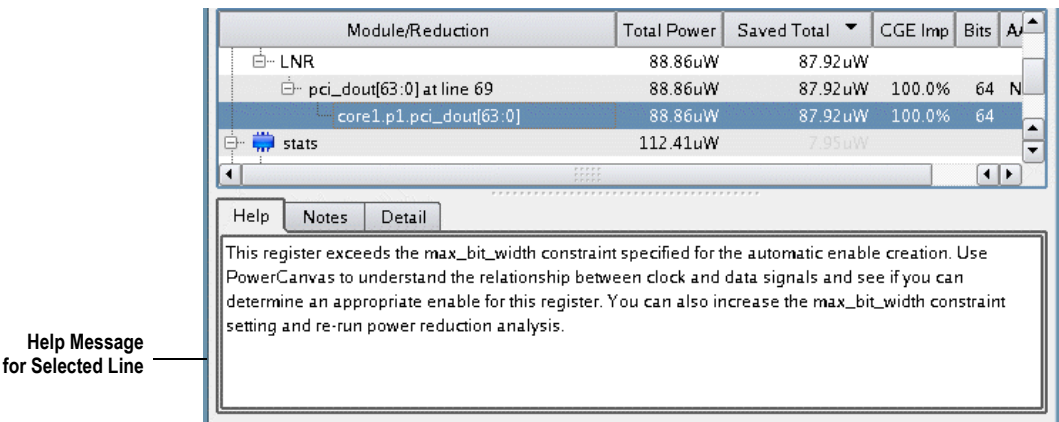


Figure 33 Sample Help Information for LNR PowerBot

The Help tab provides context-specific messages that tells you what you have to do to affect this particular change. The signal names are generally from the source code. On some occasions, you will see inferred net names that are a # followed by a number. Also notice how the Accept field is checked and marked for Automatic acceptance. The software automatically determines which reduction opportunities stand a very high probability of being successfully rewritten by the software. If you do not want a particular change done automatically, remove the check mark by clicking it again. For this tutorial, you can leave it as is.

3. Select a different row and notice how the Help changes to reflect the correct signals.

The Notes tab allows you to add notes for yourself that get stored in the OADB.

The Detail tab provides detailed information specific to a particular type of reduction. It contains a list of data output nets among other data. To see this, expand out the **pci > LNR > pci_dout[63:0] at line 69** and click on **core1.u1.pci_dout[63:0]**.

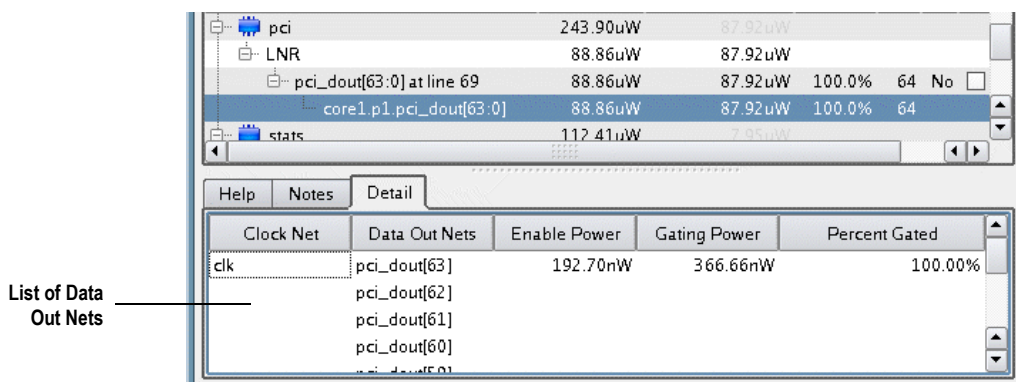


Figure 34 Detail Tab Data Out Nets

As an additional feature, you can cross-probe and trace instances and nets from the Detail tab. To do this, simply select the row (net) you want to cross-probe, right-click and make your selection from the pop-up menu. Note that, currently, you can not trace enable nets because they are generally complex boolean expressions of other signals.

Scheduling Automatic RTL Changes

Once you have examined all of the power reductions, you can select and schedule the changes you would like to implement. For this tutorial, all automatic RTL changes have already been accepted. This will not always be the case. You can use the following process to schedule a reduction opportunity for automatic rewrite.

1. Expand a reduction out completely. Use Expand Selection or Expand all from the right pop-up menu for this or do it manually.
2. On the far right, locate the Accept? column that is set to “no” for any source code line number you want to have rewritten automatically.
3. Click the checkbox to the left of the word “no”. A black X will cover the box and a drop-down menu will appear with “Manual” selected.
4. Select **Automatic**.
5. Right-click on any item in the header and select “Autosize all columns” from the pop-up menu.

Note also that the Rewrite? column has changed to scheduled. This means that if you save these results and do a rewrite, PowerArtist will automatically change

your Verilog code to reflect these changes. For details, see [Using the RTL Rewrite Wizard](#).

6. Locate the Power Saving Summary in the bottom pane.
In this power summary, the first row gives you the power starting point, meaning the current power for your design. Once you start accepting changes, the power goes down. The second row displays the sum of all of the savings due to the changes you accepted.
7. Click the **Save** button to save your results. Note that if you do not save your changes before starting the rewrite process, the changes you scheduled will not take effect.

Scheduling Multiple RTL Changes Simultaneously

The PowerCanvas allows you to accept multiple power savings opportunities at the same time using the following process:

1. In the Simple Reductions dialog, select **View > Expand All To > Line Number**.
This allows you to see all of the available opportunities for the data that is currently displayed in the dialog (any data previously filtered out will not be affected).
2. To accept all available reductions, select **Reductions > Mark All Accepted**.
Notice that the Accept column is now populated with “Manual” since the default is to schedule all changes for manual RTL updates.
3. Select **Reductions > Set All Rewrite Actions > Automatic**.
PowerArtist will reset all GMC, LNR and ODC reductions to “Automatic”. Note that only GMC, LNR and ODC opportunities can be done automatically in the Simple Reductions dialog. You can also schedule automatic rewrites for local reduction opportunities using the same methodology described here in the Prism dialog. You must make all other RTL changes manually; therefore, the entries for SMW, LEC, DOI, etc. will always be set to “Manual”.
4. Click the **Save** button to schedule the changes.
Note that if you do not save your changes before starting the rewrite process, the changes you scheduled will not take effect.

Selectively Rewriting LNR Opportunities Based on Bit Width

PowerArtist displays the number of bits in an inferred register that you can selectively rewrite. This allows you to select LNR opportunities based on bit width. For example, suppose you want to rewrite any LNR opportunity that is greater than 3 and less than 9 bits wide. You want to put these limits in place because you assert that for registers greater than 8 bits wide you would not want to generate a clock gating enable signal using exclusive ORs and an OR tree. Also, you do not want to clock gate any

register bank less than 4 bits wide. In order to do this, you would perform the following steps:

1. Clear out any filtering by clicking the stop sign icon on the far right side of the window.
2. Open the Simple Reduction dialog.
3. Click on the “More sort/filter options” black arrow button on the far right of the dialog.
4. Set the three filters as follows:
 - Reduction contains “LNR”
 - Bits less than “9”
 - Bits greater than “3” (or you could also filter on total power saving being greater than a given power value)

Make sure the “All” radio button is selected. The set-up should look like the following figure:

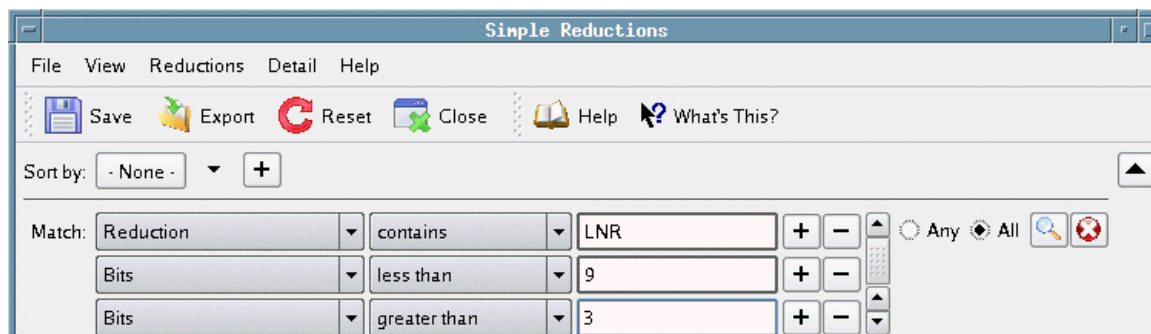


Figure 35 Multiple Filters Set for LNR Opportunities

5. Run the filter (click the magnifying glass icon).
6. Select **Reductions > Mark All Accepted**. Note the power saving in the summary area
7. **Select Reductions > Set All Rewrite Action > Automatic**
8. Click the **Save** button to schedule the changes.
9. Perform the rewrite as described in [Using the RTL Rewrite Wizard](#).

Notice that the LNR “at line” entries now group registers by name. This separates registers in the same Verilog always block by bit width. This allows you to rewrite selected registers within the same always block. If you see multiple “at line” instances sharing the same line number, this will be due to multiple instantiations of the containing Module.

Examining Advanced Power Reduction Results

This section describes reductions discovered by the Prism PowerBot, the different ways you can examine the schematic elements related to each opportunity and how to schedule automatic rewrite of local reductions. Note that for this tutorial, all local opportunities are already scheduled for automatic rewrite.

1. Select **Advanced Power Reductions > Prism**.

The Prism dialog appears displaying the results of the Prism PowerBot. It is important that you read the [Prism](#) PowerBot description to understand the specifics of this PowerBot. You can also read [Using the Prism Dialog](#) to see details of this dialog (including definitions for column headers, pop-up menus, etc.).

2. Click on each of the + signs at the beginning of each line (or click on a row that has a + sign and type the * character) to expand that row so that the results look like those in the following figure.

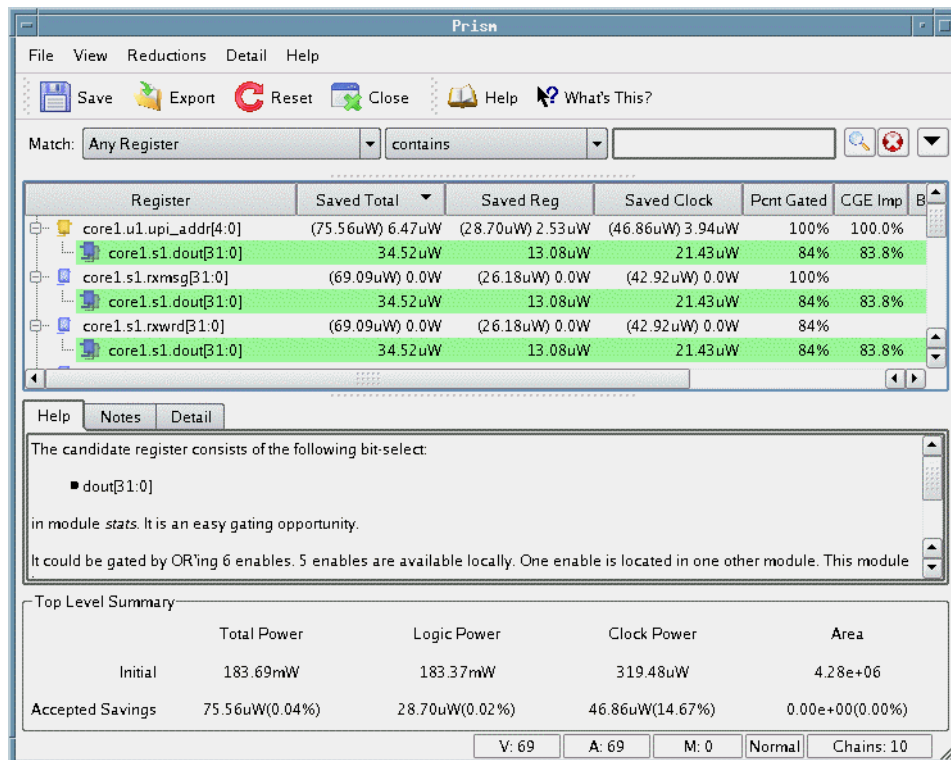


Figure 36 Prism PowerBot Reduction Results

Notice that the results in this figure are sorted by “Saved Total”.

3. Notice the icons at the beginning of each line. These provide an indication of the type of element in the register chain. The icons are:

-  A register gated in the RTL.
 -  A register for which an enable has been generated by XORing the inputs and outputs. If the register width is greater than min_width, the enable will also be used to gate it.
 -  A gated register with a strengthened enable.
 -  A single candidate register: This indicates that the chain is a simple register.
 -  Two registers whose outputs drive another register. This indicates that this register is one of at least two other registers that drive another register. This means that two or more registers share a common register chain. By selecting the + sign that precedes the icon, you can see the next element in the chain.
 -  Two stacked registers: This indicates the start of a common register chain. Somewhere else in the display is a row that has as the  icon. Rather than searching for that, you can right-click this row and select **Move Common Chain Here**.
4. Notice that there are different colors applied to the lines. The color key applied to the Prism results is as follows:
- White row: These are the gated registers that form the start of the potential enable propagated chain.
 - Green row: These are candidate registers that only have gated registers upstream. Green rows fall into the easy category.
 - Yellow row: These are candidate registers that have gated and/or candidate registers upstream. These fall into the medium category. Yellow rows are always under green rows.
 - Red row: The register instance in this row has unresolved dependencies, that is, the enable logic required to clock gate this register is partially known. This may happen when:
 - a register is driven by an ungated register in addition to being driven by a white, green or yellow register.
 - a register is driven by primary input in addition to being driven by a white, green or yellow register.
 - a register is driven by a macro (memory, etc.) in addition to being driven by a white, green or yellow register.
 - a register is driven by a delayed version of itself in a feedback loop in addition to being driven by a white, green or yellow register.

Categories of Implementation of Suggested Modifications

- easy—creating an enable for this register from the upstream gated registers is easy. You would just OR the enables and delay by a register.

- medium—creating an enable for these registers is not so easy since there is a candidate upstream; however, a simple inspection of the schematic may uncover an easy way to create one.
 - hard—creating an enable for these registers is probably not possible since upstream of this register are ungateable registers, macros, primary inputs, etc. However, by looking at the schematic, you may be able to easily identify an enable. For example, the upstream ungateable register might be an initialization flop that never changes state outside of reset.
5. Locate instance register `core1.t1.d1.dout[63:0]`. Since this instance is in a white row, you can easily see that it is a gated register that starts a register chain. Register `core1.p1.pci_dout[63:0]` is a candidate register. Click on this instance. You will see in the Help tab that this is a difficult one to rewrite—it is in a different clock domain and it has many sources upstream.
 6. Right-click on any column header and select “ungateable” from the pop-up menu. This will shown any ungateable elements. You may need to stretch the window to the right to see the ungateable column.
 7. Notice that for gated register `core1.t1.d1.dout[63:0]` there are 10 ungateable elements that you need to gate before you can potentially gate the `core1.p1.pci_dout[63:0]` candidate with the enable `core1.t1.d1.dout[63:0]` combined with other enables. There are two accepted methodologies for determining how to gate the ungated elements, which are described in the following sections.

Viewing the Upstream Cone of Logic for a Candidate Register in the Prism Dialog

This section shows you how to use the PowerCanvas to view the upstream cone of logic from a candidate register in the Prism dialog. Viewing the upstream cone of logic allows you to determine how to gate a candidate register with unknown enables.

1. Click on `core1.p1.pci_dout[63:0]` (a child instance of `core1.t1.d1.dout[63:0]`) and right-click to bring up the pop-up menu.
2. Select **Show Upstream Cone** > **Show Select Cones**.

In the schematic view, you will see the candidate register `pci_dout` displayed in green to the far right. This is the net named `core1.p1.pci_dout[63:0]` in the Prism results dialog.

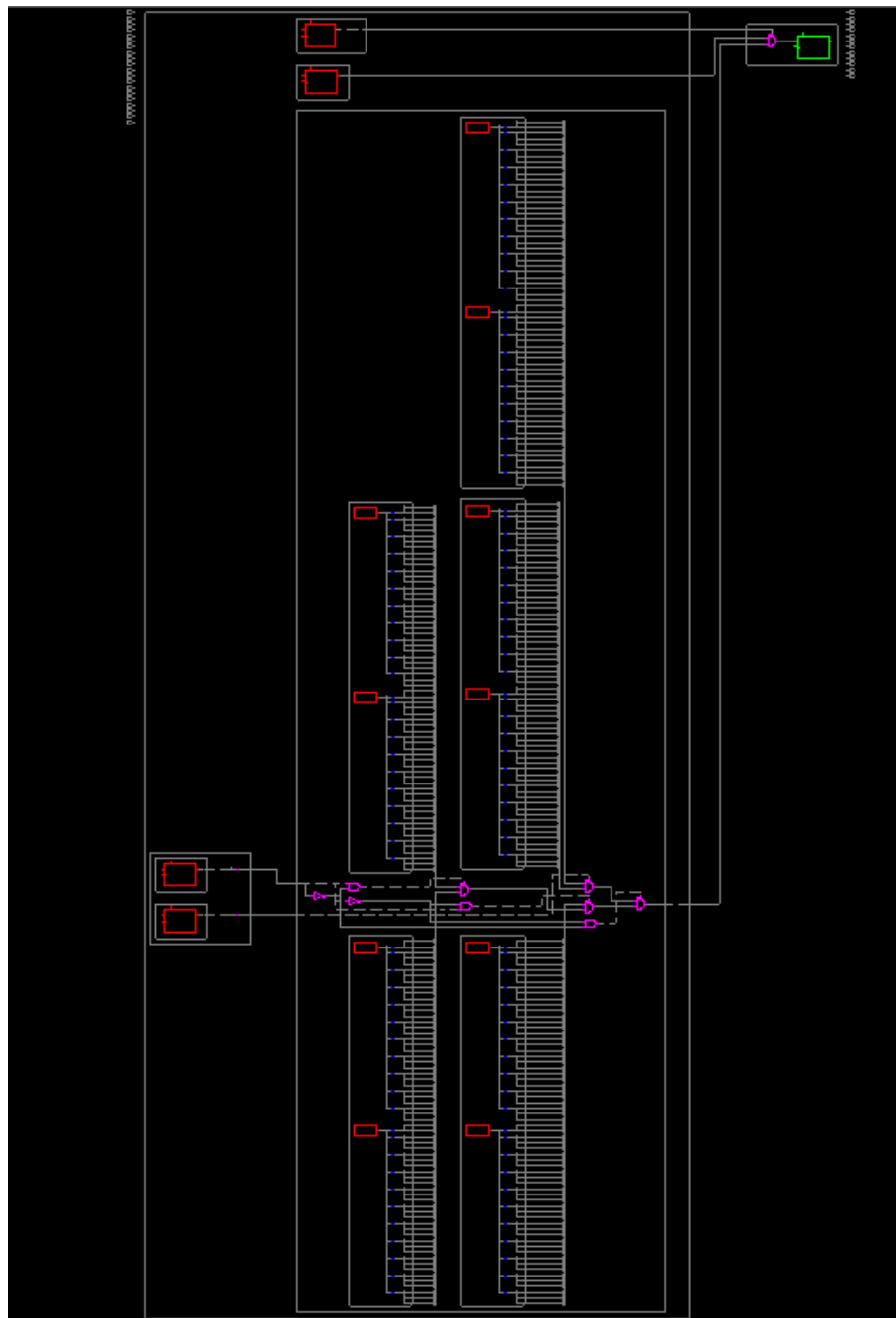


Figure 37 Upstream Cone of Logic for Candidate Register `pci_dout` in Schematic

3. Click on the schematic and right-drag the mouse around the green candidate register to zoom in on as shown in the following figure.

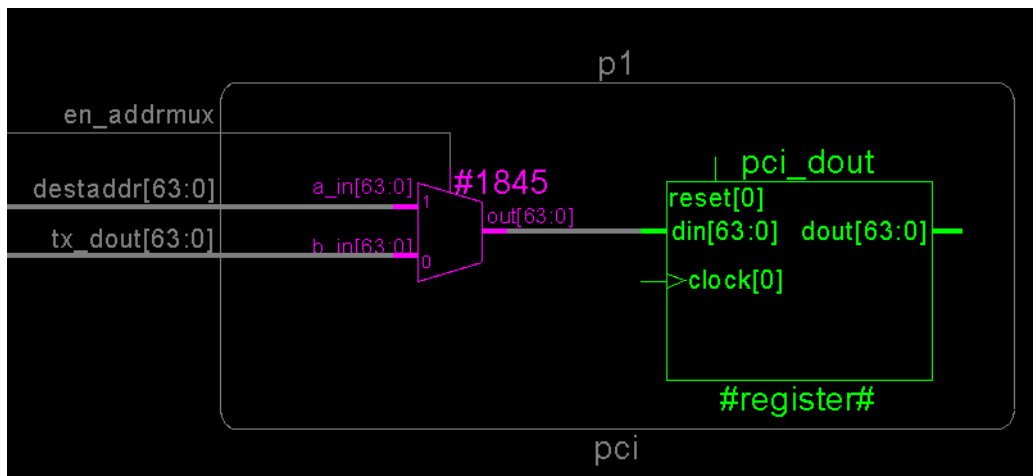


Figure 38 Close-up View of Candidate Register pci_dout and Adjacent MUX

Notice the element driving the data input of the candidate register pci_dout. This is a mux that takes two inputs: the output of the gated register and the muxed outputs of the memories. To gate this clock, you would have to gate the memories then create an enable signal by OR'ing all of the enable signals.

The GMC PowerBot already identified these as candidates and will be writing new code for you. You can use the chip select signal as the enable. Note that traces through pins when tracing cones of logic (as you can see in the above figure).

Viewing the Upstream Cone of Logic from an Input Pin of a Candidate Register

This process shows you how to use the schematic to display the upstream cone of logic from an input pin of a candidate register. This is another method you can use to determine how to gate a register.

1. Select **Collapse Full Schematic** from the schematic pop-up menu.
2. Position the cursor over core1.p1.pci_dout[63:0] in the Prism dialog and right click to bring up the pop-up menu.
3. Select **Show in Schematic**. Make sure your schematic is colored by connectivity (select **Schematic** > **Colorized by** > **Connectivity**).
Candidate register pci_dout will be the only element displayed.
4. Select (check) **Schematic** > **Auto Hide Pins**.

When you do tracing with this setting, all pins that are not explicitly traced will be suppressed in the resulting schematic.

5. Verify that **Schematic > When Tracing > Colorize DataPath** is not checked.
6. Verify that **Schematic > When Tracing > Colorize Select Path** is not checked.
7. Select **Properties** from the schematic pop-up.
The Properties dialog appears.
8. In the schematic, click a pin/port coming out of din[63].
The Properties dialog will be populated as shown in the following figure.

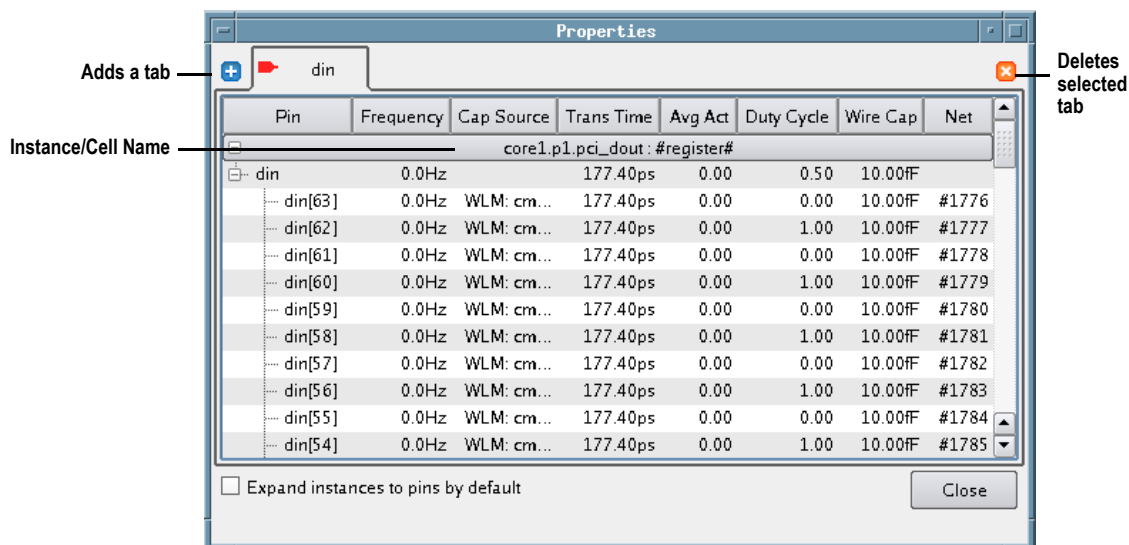


Figure 39 Properties Dialog

Make sure to click the + icon next to din to display all of the pins. You may need to make the column wider to see the full pin names shown above.

9. Click the second line in the dialog (din[63]).

10. Right-click and select **Show Upstream Cone** > **Show Select Cones** from the pop-up menu, as shown in the following figure.

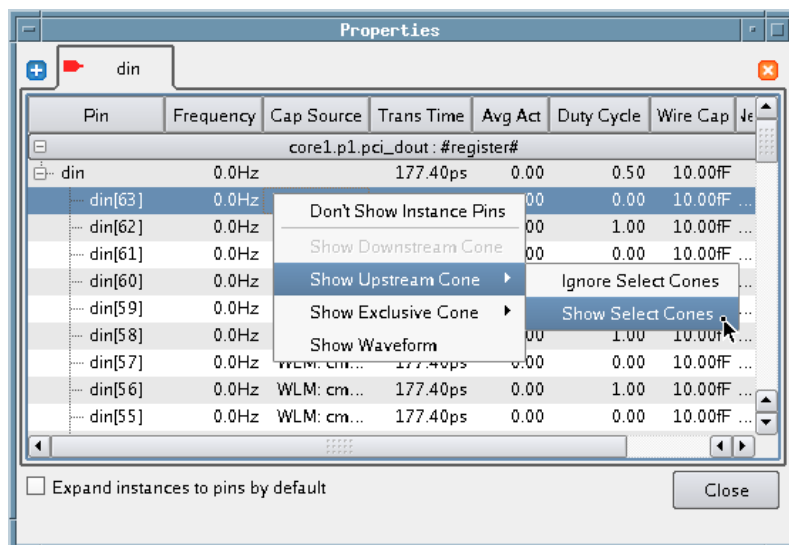


Figure 40 Showing Select Cones of Logic from the Properties Dialog

The upstream cone of logic will be displayed as shown in the following figure.

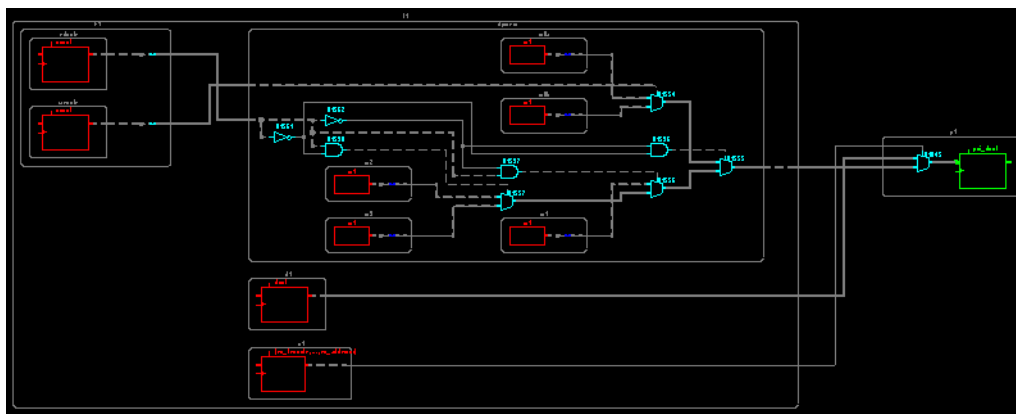


Figure 41 Upstream Cone for Input Pin din[63] of Register pci_dout

This shows you what is happening to one bit of the flow.

Tracing Select Lines Independently of Data Path Logic

You can also use the PowerCanvas to trace through select lines using the following process:

1. Select **Schematic > Collapse Full Schematic**.
2. From the Prism dialog, display the candidate register (core1.p1.pci_dout[63:0]) in the schematic, as described in the previous section.
3. Enable (check) **Schematic > When Tracing > Colorize DataPath**.
4. Enable (check) **Schematic > When Tracing > Colorize Select Path**.
5. Enable (check) **Auto Hide Pins**.
6. Select **Properties** from the schematic pop-up.
The Properties dialog appears with no data displayed.
7. In the schematic, click the pin/port coming out of din[63]. The Properties dialog will be populated.
8. Select the second line in the Properties dialog (din[63]). You may need to stretch out the Pin column header to see the full pin names.
9. Right-click and select **Show Upstream Cone > Show Select Cones** from the pop-up menu.

The upstream cone of logic will be displayed as shown in the following figure.

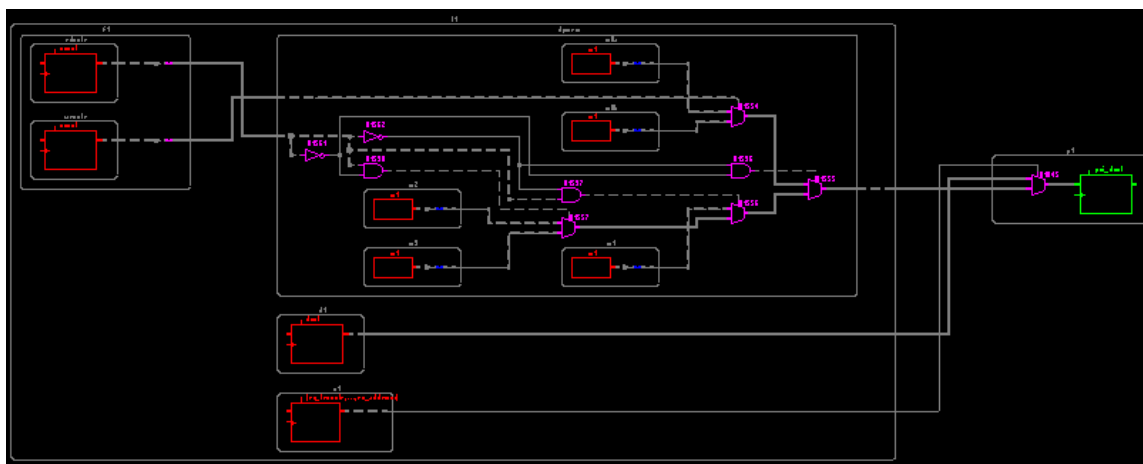


Figure 42 Upstream Cone for Input Pin din[63] of Register pci_dout (Partial View)

If you zoom-in, you can clearly see that the select line from the MUX immediately preceding the candidate register gets traced. You can trace the select line back to instance {en_lencntr,...,en_addrmux}. This notation tells you that this register is a bundle of scalar signals, two of which are shown: en_lencntr and en_addrmux. To keep the list short, any remaining scalars are replaced with "...".

Scheduling Advanced (Prism) Opportunities for Automatic Rewrite

You can schedule any of the Prism reduction opportunities for manual RTL rewrite. Additionally, you can perform automatic RTL rewrite on any reduction opportunity.

PowerArtist automatically selects the rewrite opportunities deemed “easy” where the enable signals can be generated from the RTL. Note that if you select an opportunity for automatic rewrite and it does not meet the criteria for enable generation, then PowerArtist will write information to the log file indicating that the scheduled update was not performed. The following figure shows three Prism opportunities scheduled for automatic RTL rewrite:

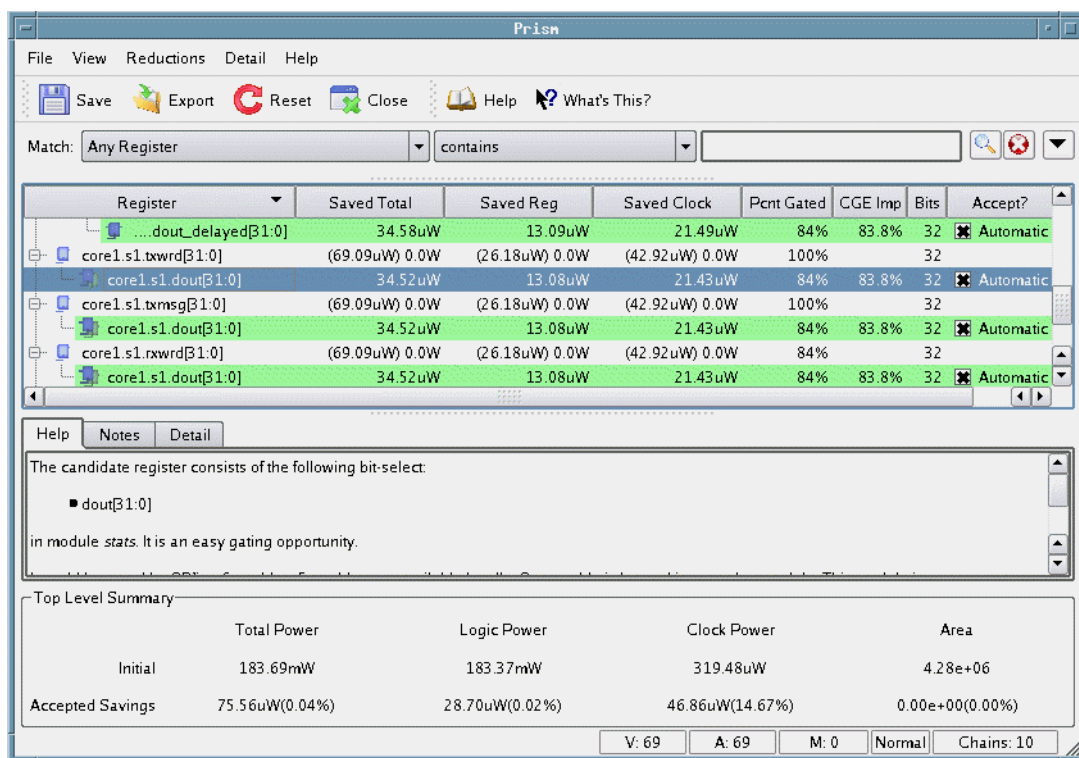


Figure 43 Prism Reduction Opportunities Schedule for Automatic RTL Rewrite

Note: You may need to stretch the Prism dialog to the right to see the Accept and Rewrite columns.

Once the opportunities have been scheduled for automatic rewrite, you then need to run the rewrite process either using the Rewrite.tcl script or using the PowerCanvas process as described in [Using the RTL Rewrite Wizard](#). The rewrite process generates a log file that records the status of the rewrite and updates the .pdb file for all of the opportunities that PowerArtist successfully rewrote.

The log file will contain messages similar to the following log file entries:

Example 1

```
partlrewrite: Note 7099: Summary for design top
                        Predicted Gated Bits (Prism) : 69
                        Predicted Clock Gating Logic (Prism) : 3
                        Rewritten Gated Bits (Prism) : 69
                        Rewritten Clock Gating Logic (Prism) : 3
```

In this case, Prism rewrote all of the requested enables. It can happen however, that not all enables are able to be rewritten. This will be indicated in the log file as well.

Example 2

```
partlrewrite: Note 7099: Summary for design top
                        Predicted Gated Bits (Prism) : 32
                        Predicted Clock Gating Logic (Prism) : 1
                        Rewritten Gated Bits (Prism) : 0
                        Rewritten Clock Gating Logic (Prism) : 0
```

In this case, because the rewritten values are both 0, there was no true opportunity for rewrite that could be accomplished automatically without significantly changing your RTL code.

Examining Power Linter Reduction Results

You will have linter reduction results if you included the linter PowerBots in your scripts. By default, all reduction types are run. This section describes the linter reduction opportunities in this tutorial.

1. Select **View > Linter Power Reductions**.

The Linter Reductions dialog will display as shown in the following figure.

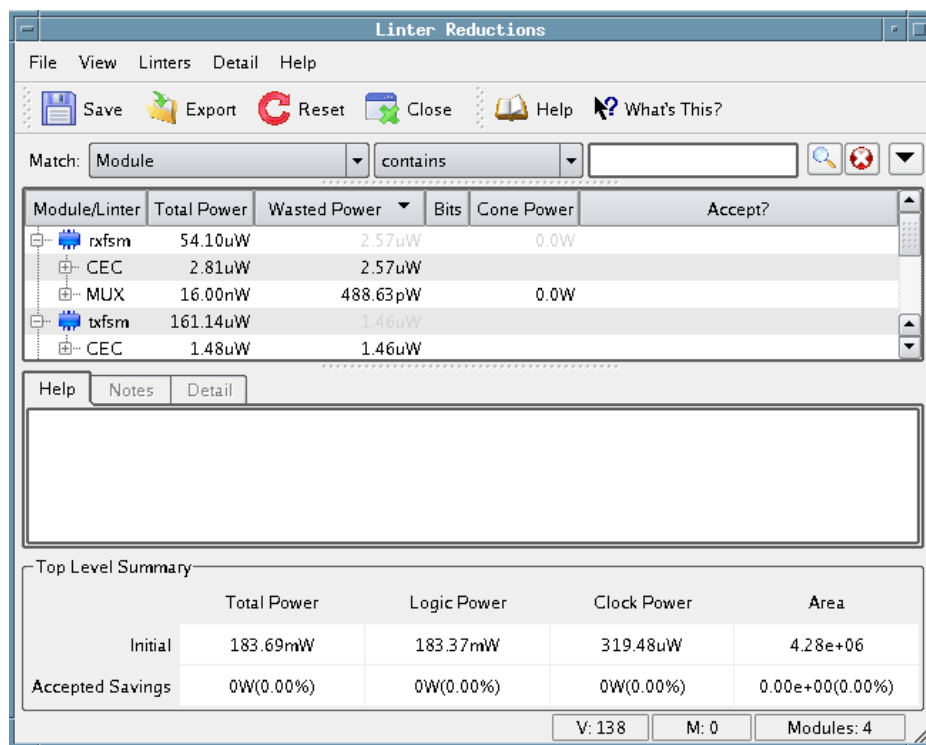
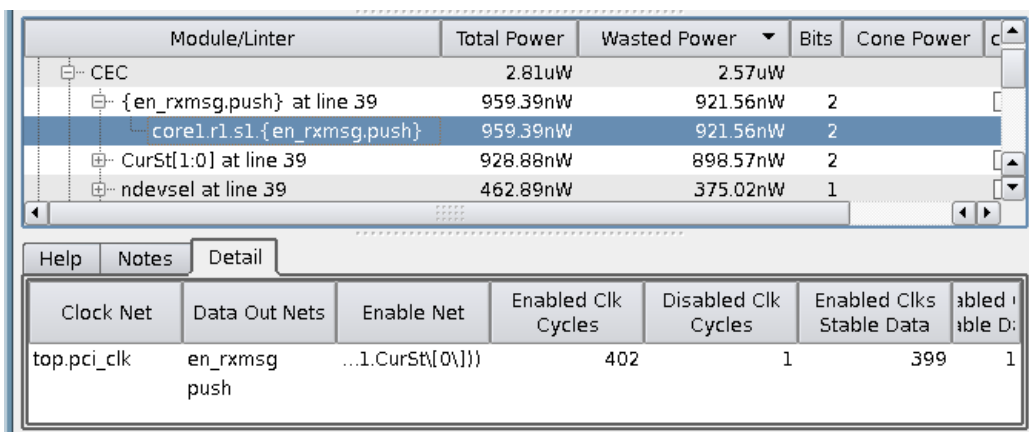


Figure 44 Linter Power Reductions Results

This dialog provides a list of wasted power results for each linter PowerBot. Note that in the tutorial, only the CEC bot is listed.

2. Navigate down to an instance in a [Clock Enable Condition Linter](#) (CEC) results line. You can either click the + symbols to move down one level at a time. Optionally, you can selected (in the Linter Dialog) **View > Expand All To Instances** or **Expand Selected To Line Instances** (if you have selected multiple lines).
3. Click (select) an instance line. You may want to select **Auto Size Column** from the right-click pop-up menu on the Module/Reduction header to display the full name of the instance.

- Click the **Detail** tab in second pane to see more details on the instance you selected, as shown in the following figure.



Module/Linter	Total Power	Wasted Power	Bits	Cone Power
CEC	2.81uW	2.57uW		
{en_rxmsg.push} at line 39	959.39nW	921.56nW	2	
core1.r1.s1.{en_rxmsg.push}	959.39nW	921.56nW	2	
CurSt[1:0] at line 39	928.88nW	898.57nW	2	
ndevsel at line 39	462.89nW	375.02nW	1	

Clock Net	Data Out Nets	Enable Net	Enabled Clk Cycles	Disabled Clk Cycles	Enabled Clks Stable Data	abled i able D:
top.pci_clk	en_rxmsg push	...1.CurSt[0\])	402	1	399	1

Figure 45 CEC Linter Power Reduction Results with Detail Tab Displayed

- To accept all linter reductions, click **Linters > Mark All Accepted**.

The Accepted Savings line in the bottom pane of this dialog will display the wasted power savings you will get with these changes. Note that you will need to manually implement all of the wasted power savings opportunities listed in this dialog.

- Click the **Save** button to save your changes in the database.

For high-level definitions of each column header, see [Column Header Definitions for the Linter Reductions Dialog](#).

Running the Reduction Tutorial Using the PowerCanvas Wizards

If you are a new user, you may want to run through the different steps in a power reduction analysis using the PowerCanvas wizards. After that, you will probably want to use the scripted flow documented in the beginning of this chapter. You use the PowerCanvas wizards to do the following tasks:

- Inference a mixed Verilog/VHDL design.
- Perform a power reduction analysis
- Review your results (for details see, [Reviewing Reduction Results in the PowerCanvas](#)).
- Automatically rewrite your RTL to implement some or all of the suggested reductions.

To invoke the PowerCanvas, type the following command:

```
% PowerArtist
```

Using the HDL Inferencing Wizard

As the first step in the full-chip reduction tutorial, you need to inference the design to create a new scenario file.

1. From the menu bar, select **Tools > HDL Inferencing**.

The HDL Inferencing wizard appears.

Note that all of the fields in this tutorial flow are pre-populated with data.

2. Follow the process as outlined in the [Performing Design Inferencing Using the HDL Inferencing Wizard](#) section of the analysis tutorial.

Using the Power Reduction Wizard

The Power Reduction wizard runs the [ReducePower](#) command; however it is not meant to replace the Tcl scripts. The results you get from using the wizard will vary slightly from the results of a command-based flow. Note that for this tutorial, you will not see Advanced Power Reductions, produced by the Prism PowerBot, if you run through the wizard.

If you are new to PowerArtist power reduction, you may want to run through a reduction using the Power Reduction wizard as described here. As you become more familiar with the tool, you will most likely want to generate Tcl scripts to run the reduction analysis for you. In this tutorial, all of the option specifications, input/output file names, etc. specified on the screens of the wizard correspond to those set in the ReducePower.tcl script.

Note that before you run power reduction with the wizard, you must have already run HDL Inferencing using the wizard to create the top.scn scenario file required for this

tutorial flow. For details on running this process, see [Performing Design Inferencing Using the HDL Inferencing Wizard](#).

1. Choose **Tools > Power Reduction Analysis**.

This brings up the Start screen of the RTL Reduction Wizard.

2. Verify that “Simulation Data (FSDB/IAF/VCD)” is selected.

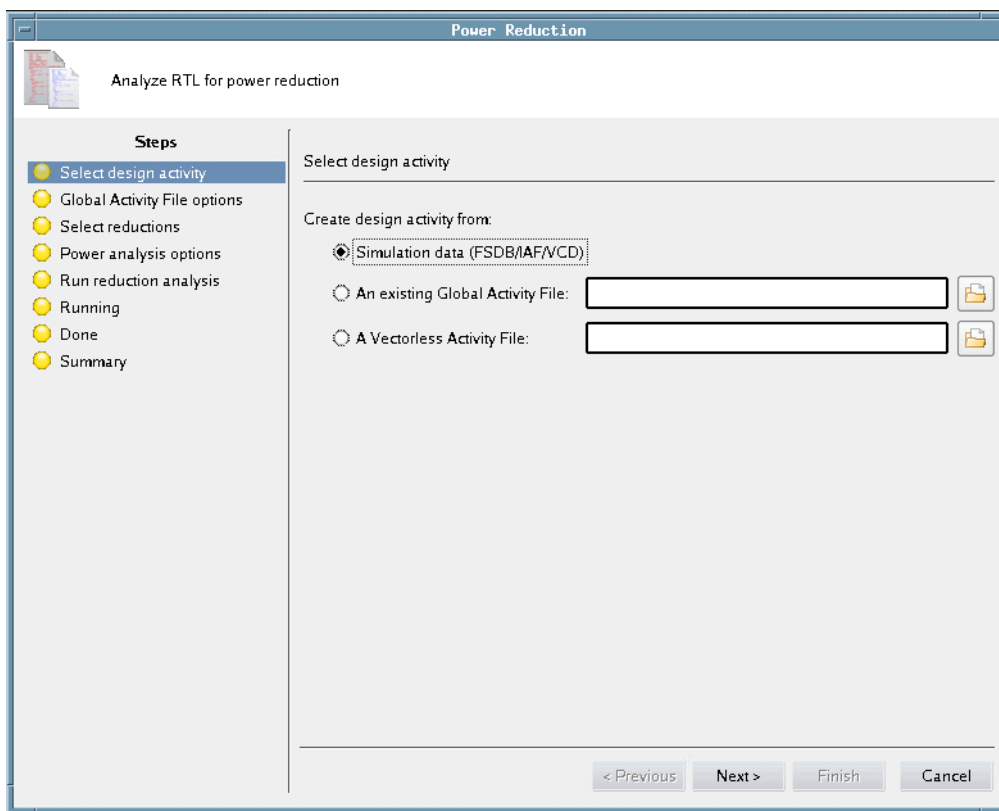


Figure 46 Power Reduction Wizard Start Screen

3. Click **Next**.

Because you are using full simulation mode for the tutorial, PowerCanvas displays the screen shown in the following figure. Use it to enter the options that are used

to convert simulation activity data to a global activity file. This screen is not displayed if you selected “An existing Global Activity File”.

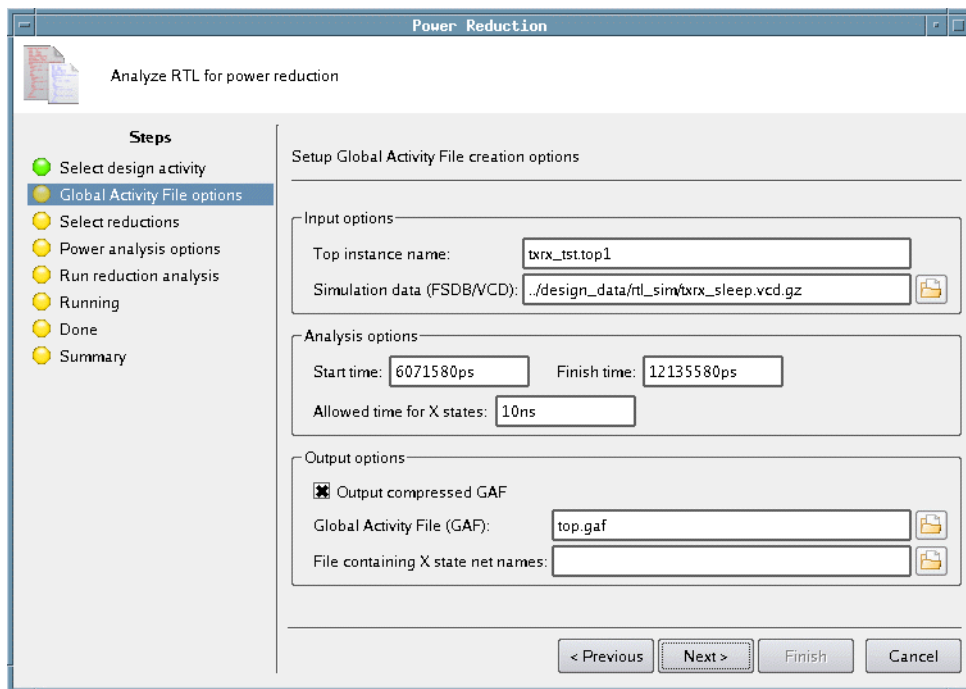


Figure 47 Power Wizard Setup Global Activity File Creation Options Screen

4. Click **Next**.

This brings up the Select reductions to analyze screen.

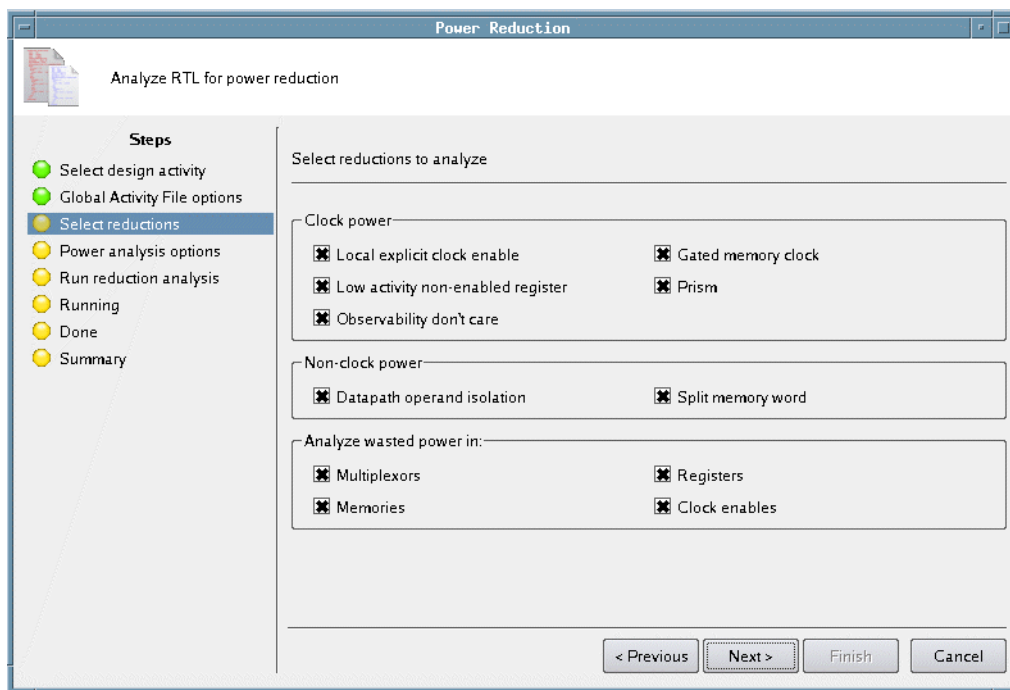


Figure 48 Power Reduction Wizard Select Reductions to Analyze Screen

You can choose from several different PowerBot reduction methods. You can choose one or many. For this tutorial, you want to run all of the PowerBots.

5. Check/enable the box for “Split Memory Word” (it is unchecked/disabled by default unless you previously set the PT_APPLY_SMW env. variable).

Note: For this tutorial, you will not get Prism results from the wizard flow—you need to use the command-line flow if you want to see hierarchical Prism reductions. For detailed descriptions of the PowerBots, see [PowerBot Overview](#).

6. Click **Next**.

This brings up the Power analysis options screen with the Basic Options tab displayed.

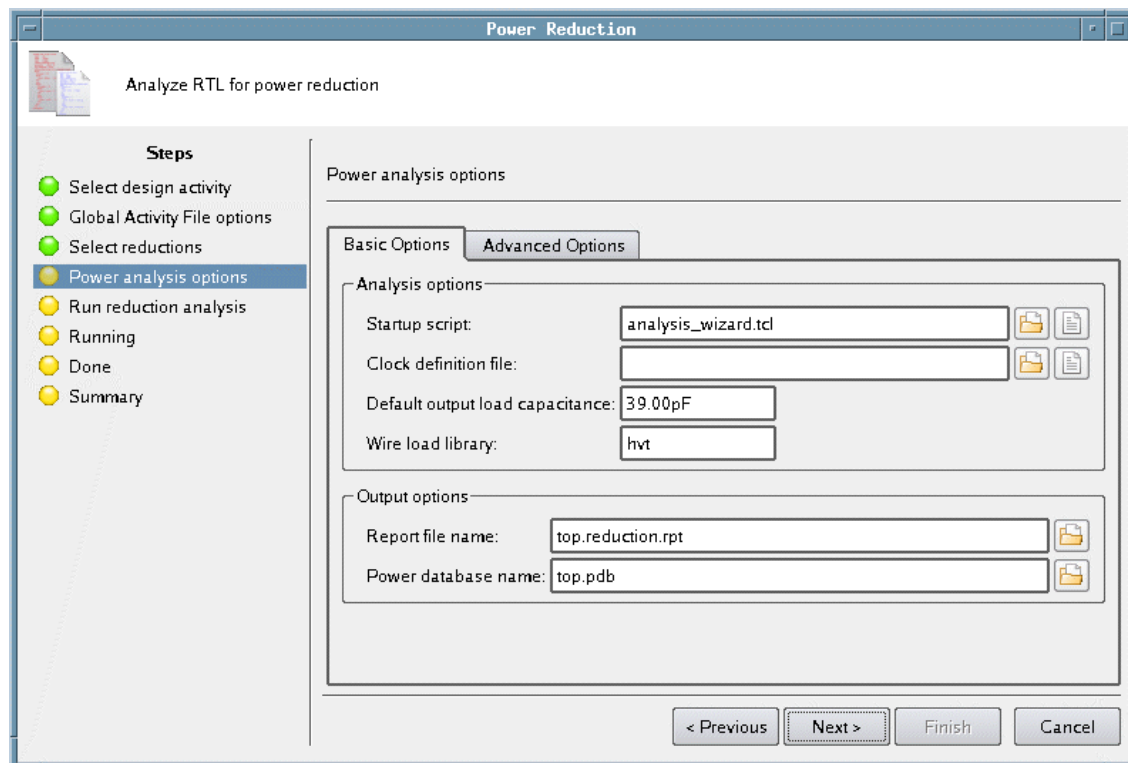


Figure 49 Power Reduction Wizard Power Analysis Options Screen

7. Click **Next**.

This brings up the Run the power reduction analysis screen.

8. Click **Run** to run the reduction.

While the reduction is running the left pane will move to the Running line and you will see log message scroll by in the right pane.

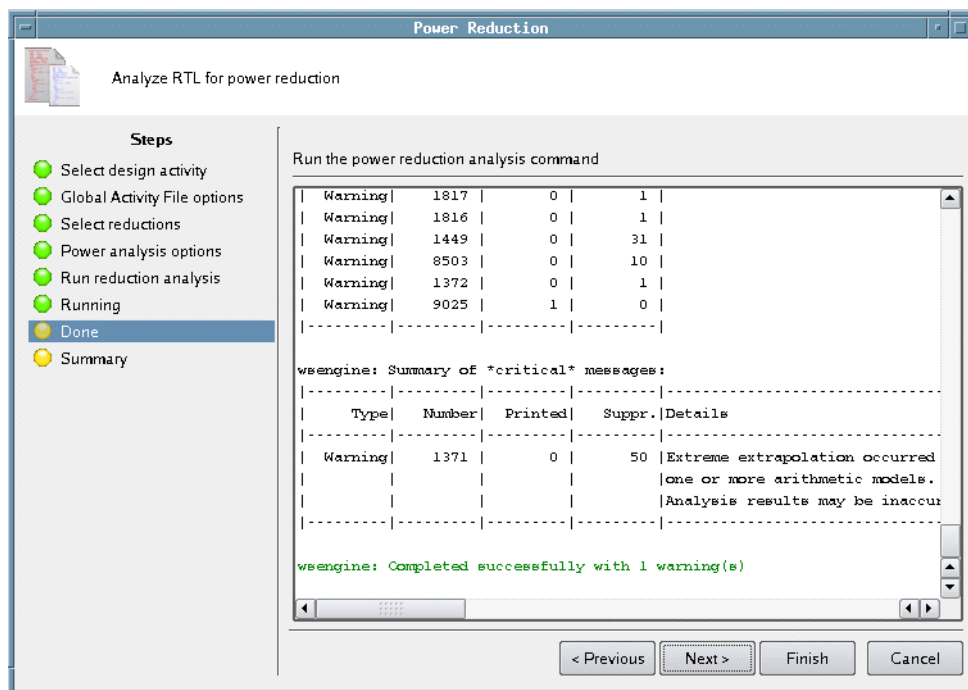


Figure 50 Power Reduction Wizard Finish Screen

The green text at the end of the log indicates that the run was successful.

- Click **Next** to view a summary (list of errors and warnings) for the reduction run.
- Click **Finish** to return to the PowerCanvas main screen.

Notice that the sub-items under the View menu are now visible. This is because the reduction results are now available for you to view and manipulate using the PowerCanvas. For details, see [Reviewing Reduction Results in the PowerCanvas](#).

Using the RTL Rewrite Wizard

The RTL Rewrite wizard runs the [RewriteRTL](#) command. Note that you need to have a feature license for automated rewrite to be able to run the rewrite wizard. The field entries in this wizard correspond directly to those specified in the Rewrite.tcl script described in the command file flow at the beginning of this chapter.

For this tutorial, all reduction opportunities that can be automatically rewritten are scheduled as such. You can change the scheduled opportunities using the Simple Reductions wizard. Just remember to save any changes that you make before closing that wizard if you want them to be reflected in the rewritten RTL.

1. Choose **Tools > Rewrite RTL**.

The RTL Rewrite wizard appears.

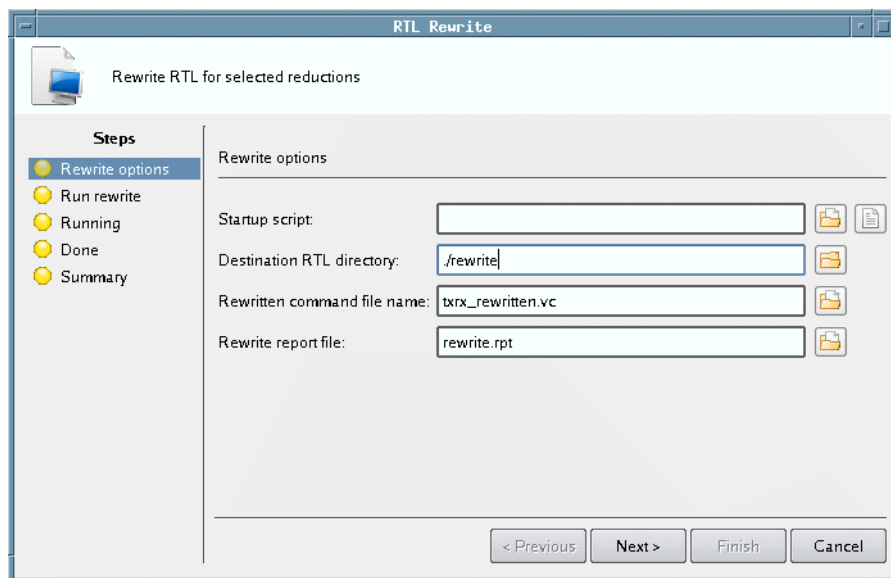


Figure 51 RTL Rewrite wizard Rewrite Options Screen

Note that PowerArtist will not overwrite your existing RTL. Instead, it will create a symbolic link to your source as recorded in your .vc file and any files included as well. Therefore, you need to specify an existing directory name. PowerArtist stores links to files that are not rewritten and real files that are rewritten.

2. Verify that the destination RTL directory is set to ./rewrite (the default name). This must be the name of an existing empty directory or a non-existent directory. To select an empty directory, you can click the file browser icon to the right of the text entry field. By default, the directory name is ./rewrite.
3. Verify that the Rewritten command file name is set to txrx_rewrite.vc.
This setting allows you to easily re-simulate your design with the changes you accepted earlier.
4. Verify that the rewrite report file is set to rewrite.rpt.
5. Click **Next** and then **Run**.

6. Review the resulting log window and find message number 2850. It says “Writing instrumented design ...” indicating that it has rewritten one file that corresponds to an LNR or GMC transformation you scheduled.

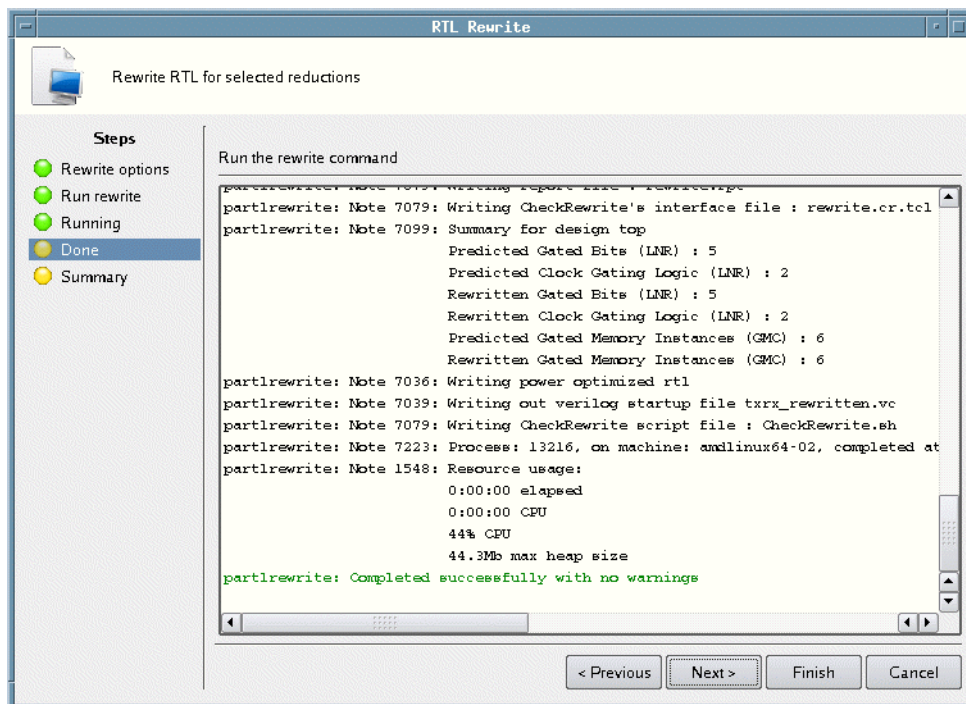


Figure 52 RTL Rewrite Wizard Done Screen/Rewrite Log

7. You can perform a “diff” between the original file and this file. You will see very few things have changed. The original formatting has been completely maintained. To see some samples of how the RTL code is changed, see [Form of the Code Modifications for LNR](#) and [Formal Verification of Rewritten Code](#).

Reviewing the Output Reports

PowerArtist generates two different output reports containing power reduction information: the *power reduction report* and the *power reduction clock gating report*. It is recommended that you review both of these reports as they contain different information.

Reviewing the Power Reduction Report

The power reduction report is a top-level summary of the reduction run. By default, this report file is named *scenario_filename.srpt*. You can control the name of this report using the `-reduction_report_file` command-line option. If you run through both the command file Tcl flow and the wizard flow in this tutorial, you will have two of these reports—*top.Batch.srpt* and *top.Reduction.srpt*, respectively. Other than some input file names, these report files are identical.

Understanding the Format of the Power Reduction Report

This report provides a breakdown of the power values in the design. It has a header section and four body sections. The header section contains information such as date run, program version information, all of the specified options and a summary of information used to perform the analysis. The body sections are as follows:

1. **Total Initial Power:** This supplies the initial total static and dynamic power for your design. The total power is split into three categories: Core, Memory and Design Power. The report records which types of RTL elements go into each category.
2. **Total Power Savings:** This summarizes the total savings reported for all of the PowerBots. The bulk of the PowerBots currently reduce dynamic power so those savings are split out. The total columns consider the impact on both static and dynamic power. The Auto Accept lines show what the algorithms determine are potentially able to be automatically rewritten. The Potential lines show the total savings you can potentially achieve if you implement all of the identified reduction opportunities. The percentage numbers are the savings for the given category (for example, Core) as a percentage of the Total power for that category from section 1.
3. **Total Final Power:** This section subtracts the auto-accepted power savings from section 2 from the Total Initial Power in section 1 and presents that as the total final power.
4. **Power Reduction by Technique:** This sections supplies the total savings for each PowerBot. The techniques are divided into those that affect Core power and those that affect Memory power. The savings are reported as a percentage of dynamic savings as well as total savings which factors in impacts on static power. Some particular points to note:

- The local explicit clock enable information describes what you would save running standard synthesis clock gating using constraints like minimum bit width. The PowerArtist constraints information explains what you may save if you supply the generated constraint file to your synthesis tools to further eliminate registers that may meet physical constraints but would not save you enough power to warrant clock gating them.
- The low activity non-enabled register information describes what you would save given the LNR constraints you supplied that were automatically accepted for rewrite versus the total potential savings you would get if all registers were considered.
- The Prism information describes what you would have saved given the constraints you supplied that were automatically accepted for rewrite versus the most you could save if you could create missing optimal enables. Comparing these two results gives you an indication of how far away you are from reaching the maximum you could save given the simulation data you supplied.

Sample Reduction Report File

The following excerpt is from the top.Batch.srpt file (generated by the command file flow) and shows the four body sections described in the previous section.

```
POWER REDUCTION POWER REPORT
=====
Date:                Fri Sep 23 12:22:25 2011
Program:             PowerArtist Power Reduction Engine (64 Bit Linux)
                   R2011.1.3PreAlpha (21 Sep 2011)
...
1. Total Initial Power
=====

                Core*      Memory      Design+
-----
Static :         9.12uW      3.97mW      50.1mW
Dynamic :        813uW      12.4mW      134mW
-----
Total :         822uW      16.4mW      184mW

* Core Power = Registers + Latches + Combinational Gates + Clocks - Black Boxes
+ Design Power = Core + Memory + Black Boxes + IO Pads

2. Total Power Savings
=====

                Dynamic Power Saving                Total Power Saving
                Core      Memory      Design      Core      Memory      Design
```

Auto Accept :	6.09uW	2.88mW	2.88mW	6.07uW	2.76mW	2.76mW
% Auto Accept :	0.75%	23.14%	2.16%	0.74%	16.80%	1.50%
Potential :	184uW	2.88mW	3.06mW	183uW	2.76mW	2.94mW
% Potential :	22.59%	23.14%	2.29%	22.30%	16.80%	1.60%
3. Total Final Power						
=====						
	Core	Memory	Design			

Static :	9.14uW	4.09mW	50.2mW			
Dynamic :	807uW	9.56mW	131mW			

Total :	816uW	13.7mW	181mW			
4. Power Reduction by Technique						
=====						
Core Techniques	Core			Core		
	Dynamic Saving			Total Saving		

Datapath operator isolation	:	1.14uW	0.14%	1.07uW	0.13%	
MUX Power Linter	:	489pW	0.00%	489pW	0.00%	
Register Power Linter	:	43.7nW	0.01%	43.7nW	0.01%	
Clock Enable Condition Power Linter	:	4.72uW	0.58%	4.72uW	0.57%	
Prism	:					
	Auto-accepted :	Topology based auto-accept#				
Using optimal clock enables	:	75.6uW	9.29%	75.6uW	9.19%	
Low activity non-enabled register	:					
	Auto-accepted (width <= 16bits)	:	6.09uW	0.75%	6.07uW	0.74%
	Potential :	94.2uW	11.59%	94uW	11.43%	
Observability Don't Care	:					
	Auto-accepted :	Topology based auto-accept#				
	Potential :	7.96uW	0.98%	7.94uW	0.97%	
# All viable reductions accepted regardless of power saving.						

Memory Techniques	Memory			Memory		
	Dynamic Saving			Total Saving		

Split memory words	:	158uW	1.27%	37.5uW	0.23%	
Gate Memory Clock	:					
	Auto-accepted :	2.72mW	21.88%	2.72mW	16.58%	
	Potential :	2.72mW	21.88%	2.72mW	16.58%	

Reviewing the Power Reduction Clock Gating Report

The clock gating report generating during power reduction provides a measure of the static and dynamic clock gating efficiency in the design. Static clock gating efficiency is a vector-independent measure of the clock gating coverage of the design. Dynamic clock gating efficiency is a vector-dependent measure of the clock gating effectiveness of the design. The report provides information on the number of bits that will be clock gated by synthesis as well as additional clock gating identified by the PowerArtist power reduction techniques. This report further breaks down the clock gating information on a per-clock and per-hierarchical instance basis. With the report providing both a high-level and a detailed view of the clock gating status in the design, you can quickly identify the right areas on which to focus your power reduction efforts.

The clock gating report name is generated by appending `_cg` to the base name you gave to the power reduction report (using the `reduction_report_file` option to `ReducePower`). In the reduction tutorial, the clock gating report is named `top.Batch_cg.srpt`.

Understanding the Power Reduction Clock Gating Report

This report has a header section and three body sections. The header section includes information such as the run date and program information (date and version). In addition, the header section includes a Note section that defines some of the symbols and terms used in this report. It provides information on how PowerArtist calculates certain values. For example, it explains that the static clock gating efficiency values are calculated as $(100 * (\text{gated_flops}/\text{total_flops}))\%$.

The top-level body sections the clock gating report are:

1. **Clock Gating Constraints:** This section provides the constraint values that were applied during the reduction analysis, for example, minimum register bank size, minimum register bank size for enhanced clock gating, and maximum clock gate fanout. It also lists the corresponding command-line option from which the constraint was defined. For example, `-min_bit_width` (to the [SetClockGatingStyle](#) command) or the `-reduction_min_bit_width` option (to the [ReducePower](#) command).
2. **Clock Gating Summary:** This section is broken into three subsections: *Clock Gating Summary for Flops*, *Clock Gating for Flops by Reduction* and *Clock Gating Summary for Latches*.

Clock Gating Summary for Flops

This section first provides basic statistic on the flops in the design—total flops, enabled flops, and inferred gated flops). It also lists the number of flops that were excluded from inferred gating, broken down by the reason for the exclusion. In addition, it provides three data points (before reduction, after reduction and

improvement percentages) for clock gating efficiency and clock gating of the flops in the design. Clock gating efficiency is broken out into static and dynamic values, which are calculated as follows:

Static Clock Gating Efficiency = $(100 * (\text{gated_flops}/\text{total_flops}))\%$.

Dynamic Clock Gating Efficiency = $(100 * (\text{gated_cycles}/\text{total_cycles}))\%$

For example, in the tutorial (sample shown on the following page) the static clock gating efficiency for flops before reduction was approximately 61% before reduction and 71% after reduction, which is a 10% improvement.

Clock Gating for Flops by Reduction

This section provides the numbers and percentages of flops in the design that were clock-gated by a particular reduction techniques (such as, Prism and Observability Don't Care). It further breaks down each category by how the gating was applied (Auto-accepted, Strengthened and Potential).

This section provides basic statistics on the latches in the design (total latches, inferred latches, instantiated latches) as well as information on clock gating efficiency and clock gating for the latches, which is very similar to that for the flops as described earlier.

3. **Clock Gating by Clock:** This section provides clock gating information on a clock-by-clock basis. For a given clock, this section summarizes the status of the clock gating. The Flop Summary section for each clock lists the total enabled flops, inferred gated flops, clocks excluded from inferred gating (broken out by reason for the exclusion), and total flops.
4. **Clock Gating by Instance:** This section, which is not printed by default, provides clock gating information on a clock-by-clock basis. If you want PowerArtist to generate this section, you need to specify the following option to the [ReducePower](#) command:

`-reduction_report_clock_gating_by_instance true`

For a given instance, this section summarizes the status of the clock gating in that instance and then it tells you for each register, how that register may or may not be clock gated. For more details on this section of the report, see [Sample Clock Gating by Instance Section](#).

Sample Power Reduction Clock Gating Report

The following excerpt is from the top.Batch_cg.srpt file (generated by the command file flow). Note that it does not include the Clock Gating by Instance section. See the [Sample Clock Gating by Instance Section](#) for a sample of that section.

```
POWER REDUCTION CLOCK GATING REPORT
=====
Date: Tue Nov 8 11:05:56 2011
```

```
Program:                PowerArtist Power Reduction Engine (64 Bit Linux) R2011.1.4PreAlpha
(08 Nov 2011)

Note:
  '-'                  : indicates unknown value
  Enabled              : Bits with a synchronous load-enable or a clock-enable
  Inferred Clock Gating : Bits that synthesis will add clock gates for
  Instantiated Clock Gating : Bits that are driven by instantiated clock gate cells
  Enhanced Clock Gating, ECG : Bits that synthesis will add clock gates for using weak
                              enables
  Static Clock Gating Efficiency : (100 * (gated_flops/total_flops))%
  Dynamic Clock Gating Efficiency : (100 * (gated_cycles/total_cycles))%

1. Clock Gating Constraints
=====

Minimum register bank size (min_bit_width)                : 3
Minimum register bank size for enhanced clock gating (min_bit_width_ecg) : 6
Maximum clock gate fanout (max_bit_width)                 : -
Minimum register bank size for applying clock gating reduction (reduction_min_bit_width) : 3
Maximum register bank size for applying LNR reduction (reduction_max_bit_width) :
16

2. Clock Gating Summary
=====

2.1 Clock Gating Summary For Flops

Total enabled flops      :      481 ( 63.46% )
Total inferred gated flops :      467 ( 61.61% )
Flops excluded from inferred gating
  Lack of enable        :      277 ( 36.54% )
  Bank width too small  :       14 (  1.85% )
  Instantiated flops    :         0 (  0.00% )
Total flops              :      758

                                Before Reduction    After Reduction    Improvement
                                -----
Clock Gating Efficiency  :
  Static :      ( 61.61% )      ( 71.37% )      ( 9.76% )
  Dynamic :      ( 58.21% )      ( 66.62% )      ( 8.41% )

Clock Gating             :
  Inferred :      467 ( 61.61% )    541 ( 71.37% )    ( 9.76% )
  Instantiated :         0 (  0.00% )    0 (  0.00% )    (  0.00% )
  Inferred and Instantiated :         0 (  0.00% )    0 (  0.00% )    (  0.00% )
  None :      291 ( 38.39% )    217 ( 28.63% )    ( 9.76% )

2.2 Clock Gating For Flops By Reduction
```


		Flops	Percent

Prism	:	69	9.10%
Low activity non-enabled register	:		
Auto-accepted (width <= 16bits)	:	5	0.66%
Potential	:	69	9.10%
Observability Don't Care	:		
Auto-accepted	:	0	0.00%
Strengthened	:	224	29.55%
Potential	:	224	29.55%
2.3 Clock Gating Summary For Latches			
Total latches	:	0	
Total inferred latches	:	0 (0.00%)	
Total instantiated latches	:	0 (0.00%)	
Clock Gating Efficiency	:		
Static	:	(-)	
Dynamic	:	(-)	
Clock Gating	:		
Instantiated	:	0 (0.00%)	
None	:	0 (0.00%)	
3. Clock Gating by Clock			
=====			
3.1 Clock Root : top.clk			
Clock Gating	:	On	
Frequency	:	66.5MHz	
Static Power	:	128nW (0.07%)	
Dynamic Power	:	173uW (99.93%)	
Total Power	:	173uW	
3.1.1 Clock Gating Summary For Flops			
Total enabled flops	:	377 (72.92%)	
Total inferred gated flops	:	376 (72.73%)	
Flops excluded from inferred gating	:		
Lack of enable	:	140 (27.08%)	
Bank width too small	:	1 (0.19%)	
Instantiated flops	:	0 (0.00%)	
Total flops	:	517	
		Before Reduction	After Reduction

Clock Gating Efficiency	:		Improvement

Static :	(72.73%)	(87.04%)	(14.31%)
Dynamic :	(68.50%)	(80.80%)	(12.31%)
Clock Gating :			
Inferred :	376 (72.73%)	450 (87.04%)	(14.31%)
Instantiated :	0 (0.00%)	0 (0.00%)	(0.00%)
Inferred and Instantiated :	0 (0.00%)	0 (0.00%)	(0.00%)
None :	141 (27.27%)	67 (12.96%)	(14.31%)
3.1.2 Clock Gating Summary For Latches			
Total latches :	0		
Total inferred latches :	0 (0.00%)		
Total instantiated latches :	0 (0.00%)		
Clock Gating Efficiency :			
Static :	(-)		
Dynamic :	(-)		
Clock Gating :			
Instantiated :	0 (0.00%)		
None :	0 (0.00%)		
<snip>			

Sample Clock Gating by Instance Section

The following section is produced when you specify the ReducePower -reduction_report_clock_gating_by_instance option (or the equivalent pt_set variable). In the reduction tutorial, this option is not set. If you want to generate this report, you can add this option to the ReducePower.tcl script.

This section provides clock gating information on a per-hierarchical instance basis. For a given instance, this section summarizes the status of the clock gating in that instance and then it tells you for each register, how that register may or may not be clock gated. In the sample report, instance top.core1.s1 has 224 total bits of which 160 will be clock-gated by synthesis. With PowerArtist reductions, an additional 64 bits can be clock gated, for an overall improvement of 23.95% on the dynamic clock gating efficiency (DCGE). This additional gating is identified by the PowerArtist Prism reduction technique for the two listed registers.

<snip>	
4. Clock Gating by Instance	
=====	
Note:	DCGE : Dynamic Clock Gating Efficiency
	ECG : Enhanced Clock Gating

<snip>	Flops		Initial		Final		Improvement		Instance
	Total	Enabled	Gated	DCGE	Gated	DCGE	Gated	DCGE	Name

	758	481	467	58.21	541	66.62	74	8.41	top
	758	481	467	58.21	541	66.62	74	8.41	-core1
	<snip>								
	Flops		Initial		Final		Improvement		Instance
	Total	Enabled	Gated	DCGE	Gated	DCGE	Gated	DCGE	Name

	224	160	160	69.15	224	93.11	64	23.95	--s1
Registers:									
Gating Technique			Width	Register Name (Clock)					
-----			-----	-----					
Inferred			32	txmsg[0:31] (top.clk)					
Inferred			32	txwrd[0:31] (top.clk)					
Inferred			32	rxmsg[0:31] (top.clk)					
Inferred			32	rxwrd[0:31] (top.clk)					
Inferred			32	wdtmr[0:31] (top.clk)					
Prism			32	dout[0:31] (top.clk)					
Prism			32	dout_delayed[0:31] (top.clk)					
<snip>									

Chapter 6

Using the PowerCanvas

Introduction

This chapter provides an overview of how to use the PowerArtist graphical user interface, called the PowerCanvas. It describes each of the menu items available and how to use them. It does not take you through a process step-by-step. To become familiar with how you should use the PowerCanvas, run either the [PowerArtist Tutorial Part 1: Power Analysis](#) or the [PowerArtist Tutorial Part II: Power Reduction](#). You should refer to this chapter if there is a particular aspect of the PowerCanvas that is not covered in the tutorial or you just want to know how one particular menu item works.

For details on starting the PowerCanvas, see [Using the PowerArtist Shell](#) and [Running an RTL Full-Chip Power Analysis Using Command Files](#).

Chapter Organization

The following topics are covered in this chapter:

- [Overview of the PowerCanvas Initial View](#)
- [Using the Hierarchy Browser](#)
- [Using the Schematic Display](#)
- [Using the Simple Reduction Dialog](#)
- [Using the Linter Reductions Dialog](#)
- [Using the Prism Dialog](#)
- [Using the Waveform Viewer](#)

Overview of the PowerCanvas Initial View

When you first bring up the PowerCanvas, as described in [PowerArtist Tutorial Part 1: Power Analysis](#), you will need to load your design by selecting **File > Load Design** and selecting your .pdb file. Alternately, you could use the “-pdb *file_name*” option to the PowerArtist command to automatically load the specified power database. Note that if you are loading a large design that may take some time, PowerArtist displays an hour glass busy indicator. When your design is loaded, the display will look like the following figure:

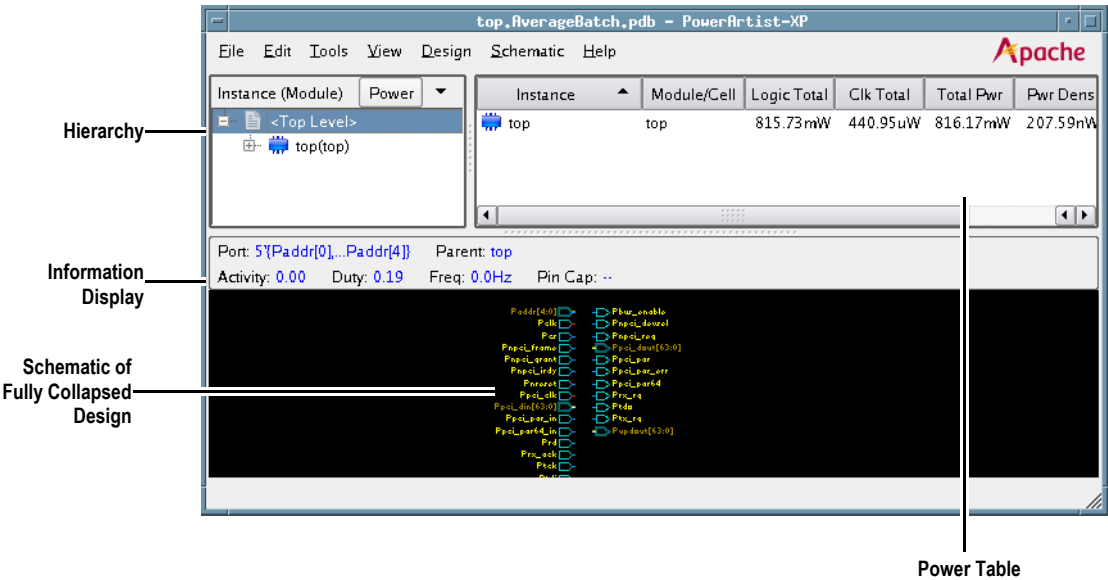


Figure 53 Initial Power Canvas Display (Tutorial Design)

The Power Canvas has two main sections—the hierarchy browser and the schematic display. You can use the Design menu items to change the look of the hierarchy browser and the schematic menu to do the same for the schematic display. For more information on the using the hierarchy browser, see [Using the Hierarchy Browser](#). For details on using the schematic display, see [Using the Schematic Display](#).

The PowerCanvas Menus

A snapshot of the menus available with the PowerArtist GUI—PowerCanvas—is shown in the following figure. Note that some menus are enabled only when certain conditions exist (for example, the Design and Schematic menus are available only if an inferred design is loaded).

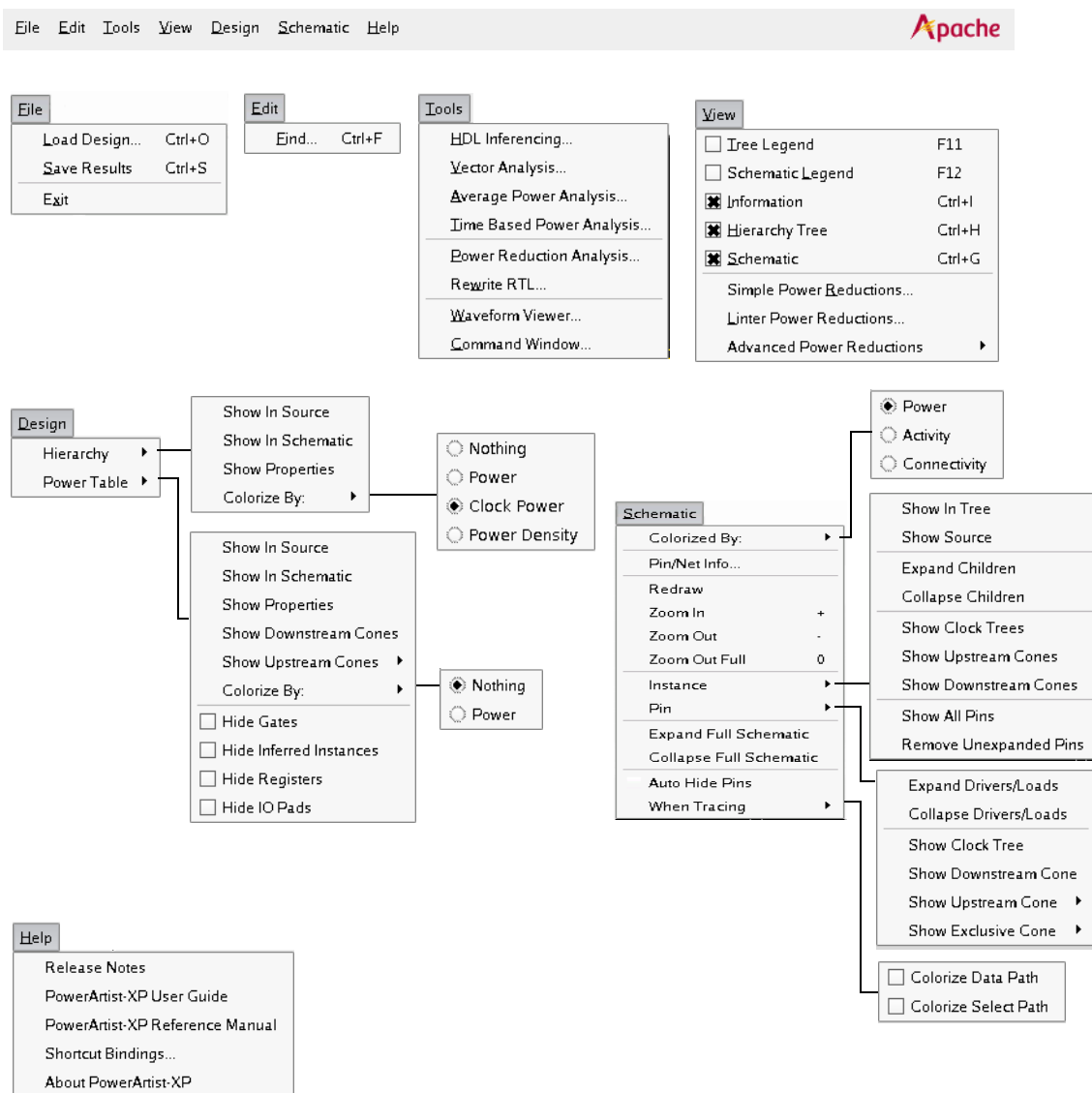


Figure 54 PowerCanvas Menus

Using the Edit Menu

The Edit menu has just one item, Find, which brings up the Find dialog. For detailed information on this dialog, see [Using the Find Dialog](#).

Using the Tools Menu

The Tools menu is divided into three sections. The first section includes items related to analysis: HDL Inferencing, Vector Analysis, Average Power Analysis, and Time-Based Power Analysis. For details on using these features, see [PowerArtist Tutorial Part 1: Power Analysis](#). The second section applies to reduction and includes Power Reduction Analysis and Rewrite RTL. Details on using these features see [PowerArtist Tutorial Part II: Power Reduction](#). The last section includes the Waveform Viewer and Command window. For information on how to use the waveform viewer, see [Using the Waveform Viewer](#).

Using the View Menu

The View menu has seven menu items, five of which control the display of paned windows: Schematic Legend, Hierarchy, Information and Schematic.

The last three items are used for viewing and manipulating power reduction analysis results. They appear only after a power reduction analysis is complete and the power database is loaded. These items are: Simple Power Reductions, Linter Power Reductions and Advanced Power Reductions. For details on using these dialogs, see [Examining Simple Power Reductions](#), [Examining Power Linter Reduction Results](#) and [Examining Advanced Power Reduction Results](#) in the reduction tutorial.

Using the Help Menu

The Help menu provides links to release notes, documentation, shortcut bindings and version information. If you select **Help > Shortcut Bindings**, PowerArtist displays the following menu, which describes various hot keys and their operations.

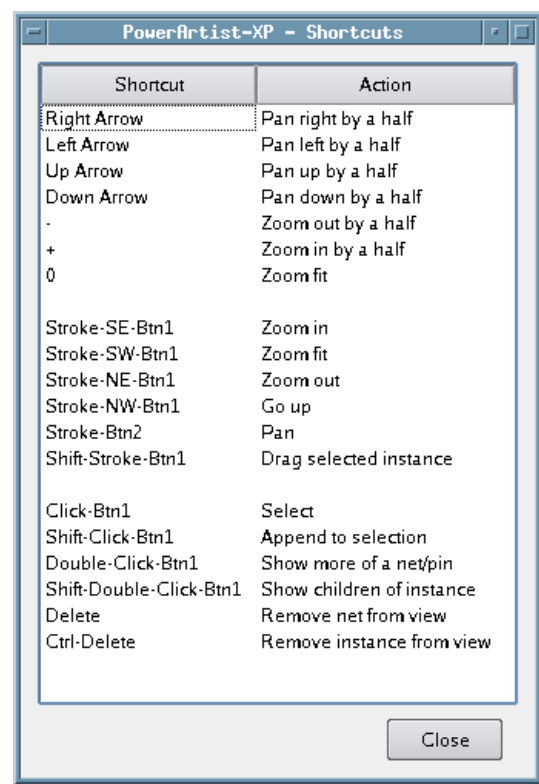


Figure 55 Schematic Viewer Shortcuts

Menu Item	Keyboard/Mouse Key or Key Combination	Function
Right Arrow	Right Arrow key	Pans to the right by half the window size
Left Arrow	Left Arrow key	Pans to the left by half the window size
Up Arrow	Up Arrow key	Pans up by half the window size
Down Arrow	Down Arrow key	Pans down by half the window size
-	- key	Zooms out by half the window size
+	Shift + =	Zooms in by half the window size
0	0 key	Zooms to fit the available window space

Menu Item	Keyboard/Mouse Key or Key Combination	Function
Stroke-SE-Btn1	Hold down left mouse button and drag in south east direction	Zooms in to area in selection box
Stroke-SW-Btn1	Hold down left mouse button and drag in south west direction	Zoom fit; fits the current window size
Stroke-NE-Btn1	Hold down left mouse button and drag in north east direction	Zooms out; zooms out proportional to the length of the stroke. The longer the stroke, the further out the zoom. As you drag the mouse, PowerCanvas displays the value of the zoom factor.
Stroke-Btn2	Click and hold the middle mouse button	Pan; moves around the design
Shift-Stroke-Btn1	Hold down Shift key and click left mouse button and drag across screen	Drag selected instance; performs a move This move is only operational for elements that are not nested inside the body of another instance.
Click-Btn1	Single-click left mouse button	Select; this focuses in the schematic window and selects any element that is under the mouse cursor.
Shift-Click-Btn1	Hold down Shift key while clicking left mouse button	Append to selection; adds clicked elements to your current selection
Double-Click-Btn1	Double-click left mouse button	Navigate; expands the connectivity for the clicked object to include additional connections. Clicking on a net will show additional pins for the net. Clicking on a pin will show the net for the pin.
Shift-Double-Click-Btn1	Hold down Shift key while double-clicking left mouse button	Shows the children of the clicked instance
Delete	Delete key	Removes the selected net from the view
Ctrl-Delete	Hold down the Control key and press the Delete key	Removes the selected instance from the view

PowerCanvas Dialogs

The PowerCanvas provides different dialogs that allow you to manipulate your data in a number of ways. You can search the power database and view and edit the source lines for selected elements.

Using the Find Dialog

The Find dialog allows you to do a number of different searches on your power database. Select **Edit > Find** from the PowerCanvas to bring up the Find dialog as shown in Figure 56.

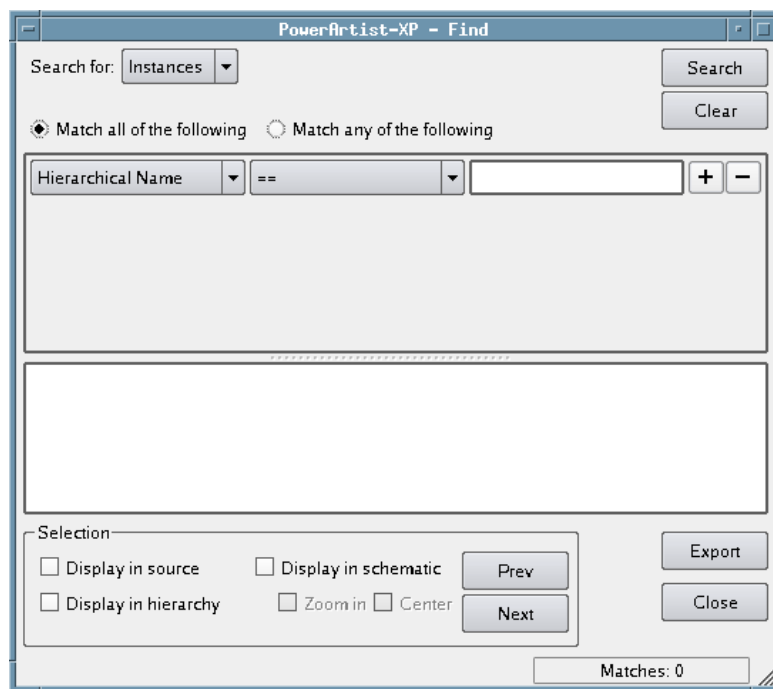


Figure 56 PowerCanvas Find Dialog

The Find dialog allows you to search the power database for information relating to instances, pins or nets. You control your search choice using the “Search For” pull-down menu. Once you have done this, you enter the search criteria, controlling how many criteria must be met using the + (add another one to the list), - (delete the last one from the list) and Clear (clear the entire list). The criteria depend on the object you select. If you choose instances, you can then search by name, hierarchical name, name, inferred name, cell name, total power, dynamic power, static power and clock gating. If you choose pins, you can search by hierarchical name, name, type

and capacitance. If you choose nets, you can search by hierarchical name, name, average activity, capacitance, duty cycle, and frequency.

For name-based searches, you do not need to use wild card characters. You can simply type in any part of the name and the find dialog will return all names that contain that string. You can then ask for either a match (==) or not a match (!=). Also, the name-based entries are case-insensitive. When using this dialog, it's important that you understand the difference between a *hierarchical name* and a *name*. A hierarchical name is the full hierarchical path name where levels of hierarchy are separated by a dot (.). The name is the portion of the name after the last (right-most) dot. Assume, for example, that you have the following strings in your design:

a.b.c

a.bc

abc

A name search of using “b” as the search string will return a.bc and abc. It will not match a.b.c because in that case “c” is the name, which does not match the pattern “b” (0 or more name elements followed by a “b” followed by 0 or more name elements). However, a hierarchical name search using “b” returns all three because a.b.c matches the given pattern.

When you are searching for numeric values, the values will all be floating point numbers but will be displayed using scientific notation. You will use the standard relational operators to control your search (<, <=, ==, >=, >). If you do not specify a unit when searching for a numeric value, Watts is assumed. You can specify, W (default) mW, uW, or nW.

You can search for instances by their clock gating type. If you select Instance and Clock Gating, then you can search for instances that match (or don't match) any of the following types: None, Inferred, Instantiated, Inferred and Instantiated and Register Bank.

Once you have defined your search criteria, you will then click the **Search** button to perform the search. Notice that the search button changes to a **Stop** button while the

search is working. If you want to stop the search at any time, you can click **Stop**. The following figure shows sample search criteria with results.

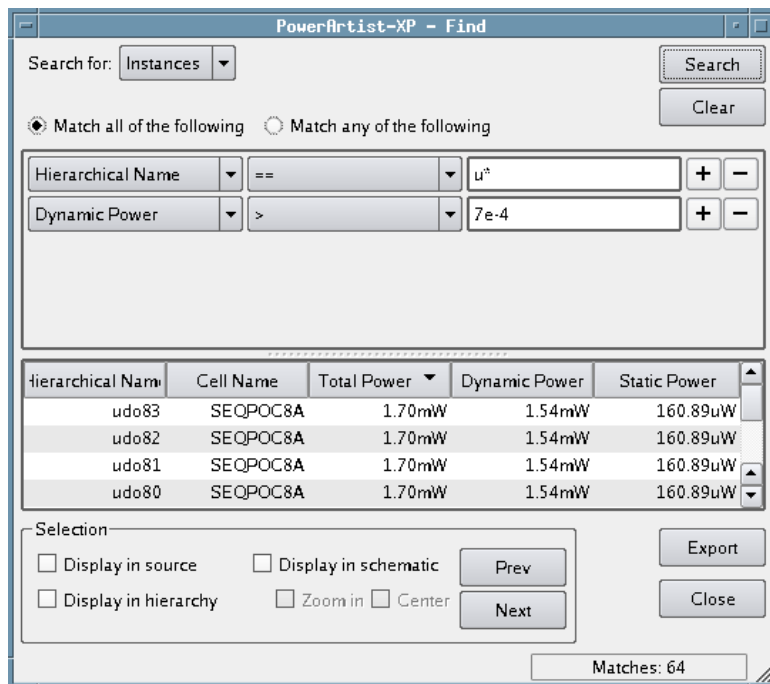


Figure 57 PowerArtist Find Dialog (Search with Multiple Criteria)

If the search goes on for more than several seconds you will see a progress meter in the status bar that indicates the percentage completion of the search.

Note: The first time you search the Instance, Net, or Pin category you will see a “busy indicator” because the Find dialog does not know how many items are in the design. After the first search, you will get a progress meter that indicates the percent of the search that has been completed.

This sample search, which uses the power database from the reduction tutorial, finds all hierarchical names (including inferred instances) that begin with u (or U) and have a dynamic power greater than 7e-04W (or .7mW). In this example, the search criteria matched 64 elements. You can use the **Previous** and **Next** buttons to step through the each element in the returned list. Notice the Selection box on the bottom left of the dialog. This box allows you to view the source for each element, the schematic entry, or the hierarchy browser entry. Moreover, you can zoom-in on or center the element in the schematic display. Note that you must first make the selection in this section before clicking on an element in the returned list. Once the selection is made,

for example, “Display in source” you can then double-click any element in the list to see the corresponding source.

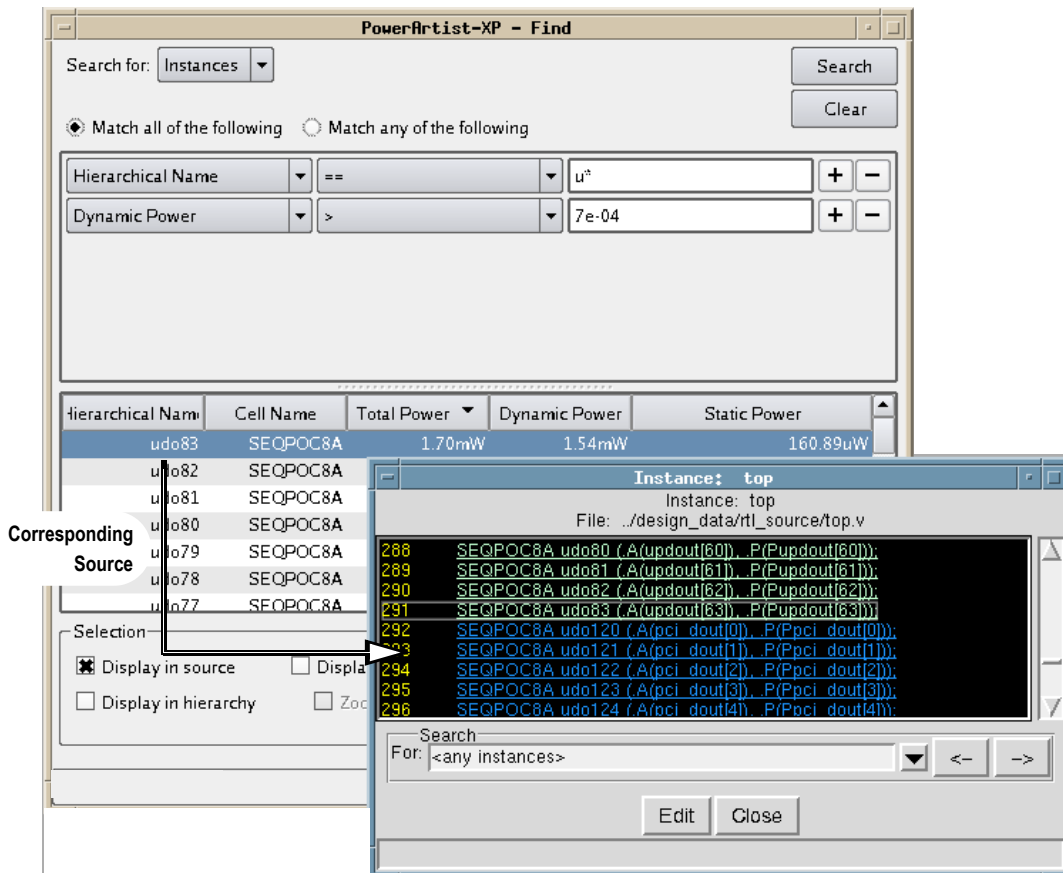


Figure 58 Displaying Source for Element in Returned Search List

You can click on any column header in the table of returned elements to sort the elements by either increasing or decreasing order according to that criteria. The first time you select a header, you will get an arrow showing the direction of the sort. The second time you select the header inverts the sort.

Using the Properties Dialog

You can bring up the Properties dialog from the Schematic top-level menu or from the schematic pop-up menu on the schematic display. This dialog displays a number of properties (power values, frequency, transition times, average, activity, duty cycle, wire capacitance, etc.) for nets, pins or instances in the design. You can do the following with this dialog:

- Select nets, pins or instances from the schematic or from the hierarchy browser.

- Add tabs to allow you to save data for items you have selected and then toggle between them. There is no limit on the number of tabs you can create. You can use the  and  icons to add or remove tabs, respectively. You can also right-click on the dialog heading to open a pop-up menu of tab actions. This menu has three items: **New Tab**, **Close Tab** and **Close Other Tabs**.
- Suppress (hide) any of the columns, by right-clicking a column header and disabling/enabling the columns from the pop-up menu that appears. To reorder the columns, click on and drag the headers.
- Cross-probe to the Apache Waveform Viewer. To do so, first open the Apache Waveform Viewer (select **Tools** > **Waveform Viewer**). Then, right-click a net/pin name and select **Show Waveform** from the pop-up menu. For full details, see the [Using the Waveform Viewer](#) section.
- Hide or change columns by right-clicking any column header and making an appropriate selection.
- Select multiple elements to display in the same tab. The following figure has three tabs, one for each element type.

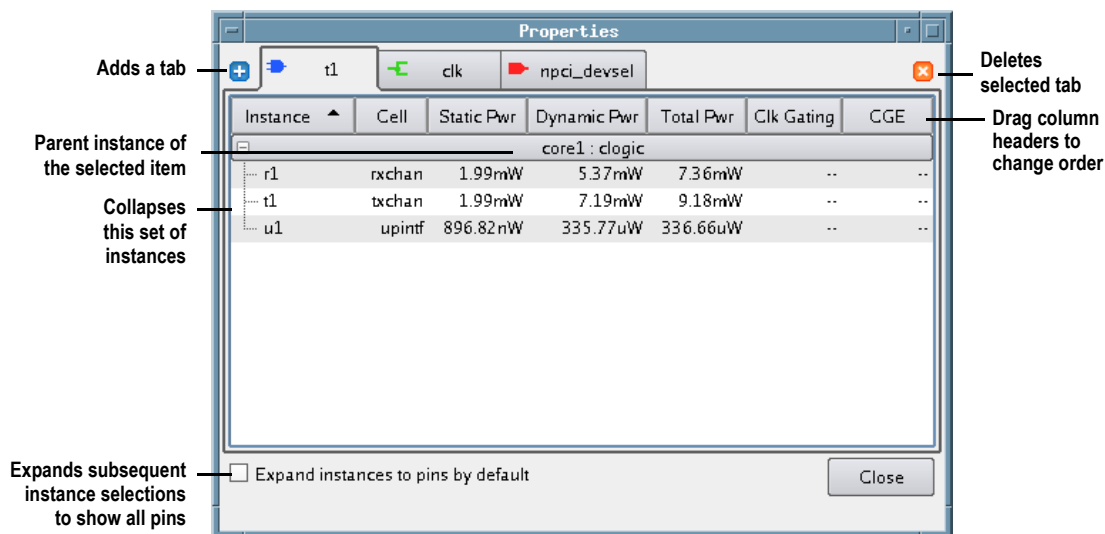


Figure 59 Properties Dialog with Multiple Tabs and Instances Displayed

Notice the different icons in the tab headings for each element. The  icon represents a net instance. The  icon represents a pin. The  icon represents an instance. You can display multiple nets, pins or instances (using Ctrl-select) within a single tab. Note, however, that you cannot display different elements in the same tab.

- Display the pins for any of the listed instances. To do so, right-click any row and select **Show Instance Pins** from the pop-up menu. If the tab is already showing pins when you right click a menu item, select **Don't Show Instance Pins** to view only the information for the parent instances. If you check the **Expand instances to pins by default** box, the pins will display whenever you select an instance.
- Display the upstream, downstream or exclusive cones of logic for an instance pin. To do so, simply select the appropriate entry from the right-click pop-up menu. The following figure shows how you would display the downstream cone of logic for the Y output pin on instance udi18.

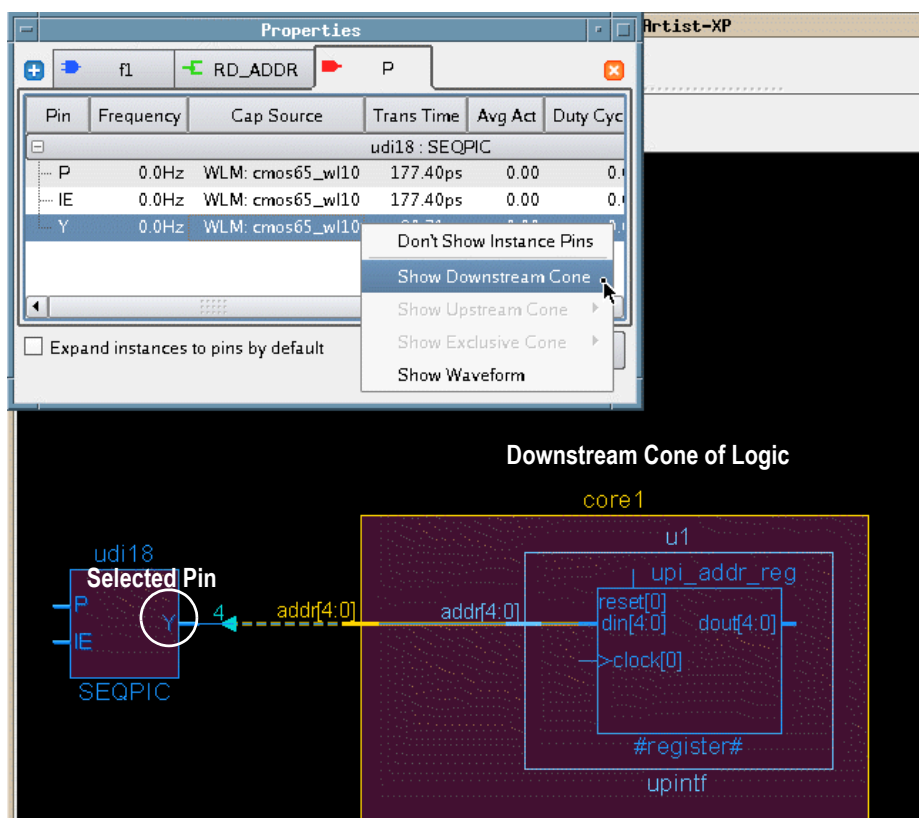


Figure 60 Displaying the Downstream Cone of Logic for an Output Pin

Using the Smart Source Browser

To view the source for an element in the hierarchy display, click the element and then select **Design > Hierarchy > Show in Source** or just **Show in Source** from the right-click pop-up menu. You can perform a similar operation from the schematic display. The Smart Source browser will appear as shown in the following figure.

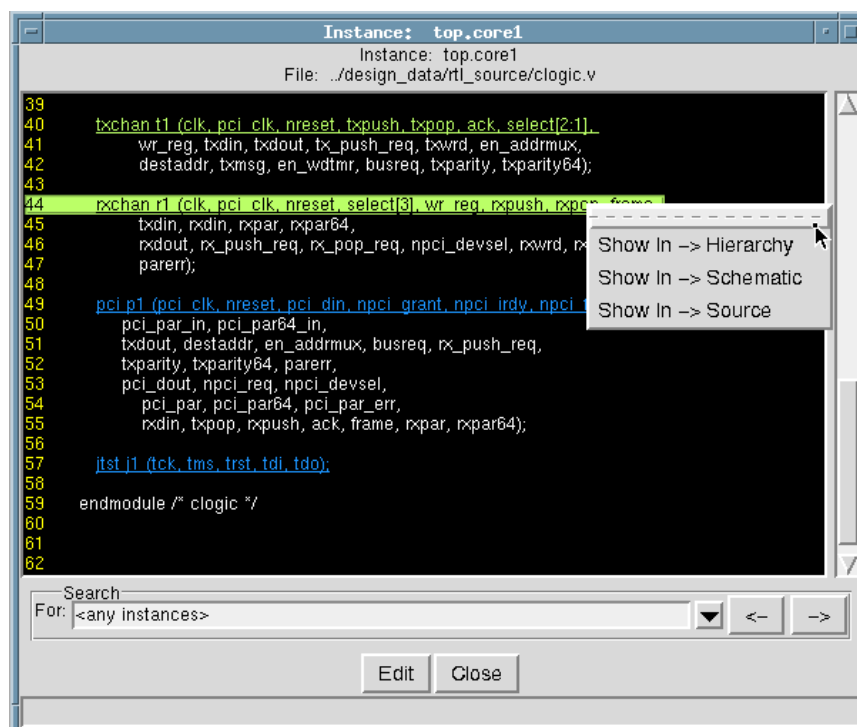


Figure 61 Smart Source Browser

Much like the coloring of the hierarchy and power table, the SmartSource browser color codes source lines according to the amount of power consumed by elements inferred in that line. The color given to a line of code reflects the sum of all elements inferred by that line. Once you select (click) a line, you can right-click to display the pop-up menu shown in Figure 61. The three options have the following functions:

- **Show in --> Hierarchy:** Displays the selected instance in the hierarchy browser.
- **Show in --> Schematic:** Displays the schematic elements associated with the selected line of source code.
- **Show in --> Source:** Brings up the source code of the selected line. If the line represents a module instantiation and the definition of that module exists in a file, an additional source code browser is displayed with the editor positioned at the

line where the module definition occurs. If the line represents inferred logic, then there is no additional source to display and the request is ignored.

You then have the choice of editing the source code. If you select the Edit button, the line you have selected comes up in the editor that you set the first time you selected the Edit button. The editor is placed in a read only mode. When you are done with the browser, click the **Close** button.

You can use the Search box at the bottom of the browser to search for occurrences of the string in the current file. The left and right arrow keys search forward and backward in the file. By default, it searches for any instance.

Using the Hierarchy Browser

This section describes the ways in which you can manipulate your design data in the hierarchy browser in the PowerCanvas. The Design menu controls the display of the hierarchy browser. The hierarchy browser has two panes: the left-hand side is the hierarchy and the right-hand side is the power table

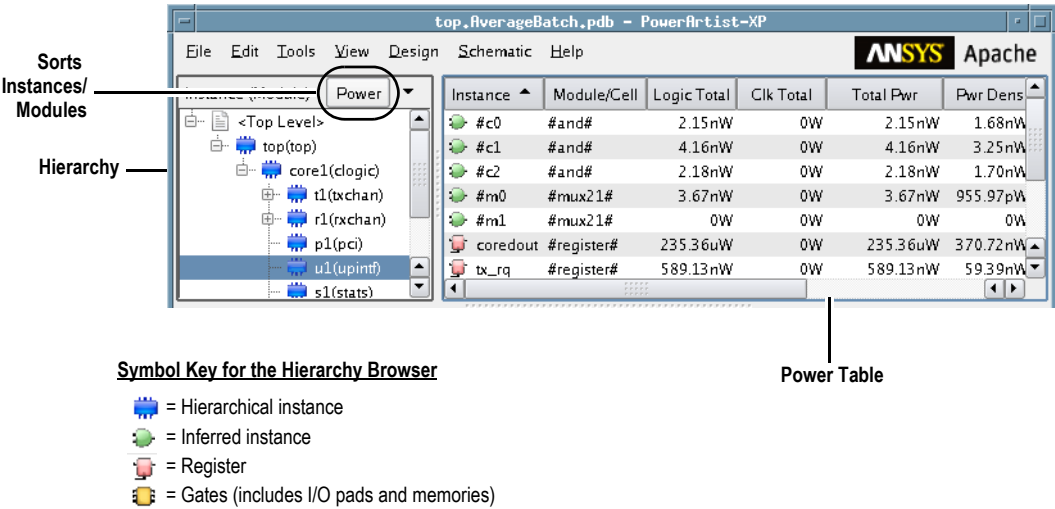


Figure 62 Hierarchy Browser

You can use the button (which by default shows “Power”) at the top right of the hierarchy view to sort the data. You can sort by Power, Power Density, Instance, Module or Area.

The power table displays the inferred instances, registers and gates that are within the hierarchical instance (module) that is highlighted in the hierarchy view in the left-hand pane.

Column Header Definitions for the Power Table

The power table display has the following columns:

- ***Instance:** This is the name of the instance inferred from the RTL.
- ***Module/Cell:** This is the name of the modules/cell in the design.
- **Logic Stat:** This is the total leakage power of the logic and inferred buffers.
- **Logic Dyn:** This is the total dynamic power of the logic (internal plus net power) and inferred buffers.
- ***Logic Total:** This is the sum of the leakage and dynamic power of the logic. The latter includes internal plus net power and inferred buffers.
- **Clk Stat:** This is the total leakage power of the clock tree enclosed by a hierarchical instance.
- **Clk Dyn:** This is the total dynamic power of the clock tree enclosed by a hierarchical instance.
- ***Clk Total:** This is the sum of the leakage and dynamic power of the clock tree enclosed by a hierarchical instance.
- **Total Stat:** This is the sum of the logic and clock tree leakage power.
- **Total Dyn:** This is the sum of the logic and clock tree dynamic power.
- ***Total Pwr:** This is the sum of the logic and clock tree power.
- ***Pwr Density:** This is the total instance power divided by the instance area.
- ***Area:** This is the instance area.
- ***Clk Gated:** For registers, it identifies whether it is clock gated or not.
- **CGE:** This is the clock gating efficiency value.
- ***Optimized:** Indicates whether the instance can be optimized away, for example, if it has no fanout.

* indicates that this field is enabled/displayed by default.

Note on Logic Instance Values: If the instance power under **Logic Stat**, **Logic Dyn** or **Logic Total** are zero, it may be because it forms part of a clock tree. If so, its power will be included there.

Note Clock Instance Values: If a leaf instance power value under **Clk Stat**, **Clk Dyn** or **Clk Total** is non-zero value, it's because it forms part of a clock tree so its logic power is included here.

Controlling the Hierarchy Browser From the Design Menu

The Design menu has two top-level menu items: Hierarchy and Power Table and is broken down the in the following way.

- Hierarchy

For the highlighted module/instance, you can perform the following actions:

- **Show in Source:** Opens a Smart Source browser on the source file containing the selected element. The browser will be positioned at the source line and colorized for power based on the module/instance from which you cross-probed. For details on using the Smart Source browser, see [Using the Smart Source Browser](#).
- **Show in Schematic:** Displays the selected instance in the schematic. You can then zoom-in to get a closer look. For more information see, [Using the Schematic Display](#).
- **Show Properties:** displays the selection in the Properties dialog. For more information, see [Using the Properties Dialog](#).
- **Colorize by:** Colors the hierarchy browser by the following:
 - Nothing (the default). This adds no color.
 - Power: Colorizes the modules in the hierarchy by the power results. The higher the power consumption, the hotter the color.
 - Clock Power: Colorizes the modules in the hierarchy by the clock power results. The higher the power consumption, the hotter the color.
 - Power Density: Colorizes the modules in the hierarchy by power density (power/area) as a percentage of the largest power density.

You can also select these items from the pop-up menu when you right-click on a module in the hierarchy.

- **Power Table**

For the selected instance, you can perform many of the same actions listed under the Hierarchy sub-menu and the following actions that apply only to selections in the power table (or apply differently to them):

- **Show Downstream Cones:** Displays the downstream cone of logic in the schematic for the selected inferred instance in the power table.
- **Show Upstream Cones:** Displays the upstream cones of logic in the schematic for the selected inferred instance in the power table. For upstream cones, you have the following additional display choices:
 - Ignore Select Cones/Show Select Cones
Ignores/shows cones of logic for upstream select signals for 2-1 muxes, unencoded muxes and tri-states.
- **Colorize by:** Colors the power table by the following:
 - Nothing (the default). This adds no color.
 - Power: Colorizes for power as a percentage of the parent instance.

You can also choose to hide different elements in the power table by checking the following options:

- **Hide Gates**
- **Hide Inferred Instances**

— **Hide Registers**

— **Hide IO Pads**

By default, all elements are displayed.

You can also select these items from the pop-up menu when you right-click on the power table.

Using the Schematic Display

The schematic display allows you to browse through a schematic representation of the loaded design. This section describes the ways in which you can manipulate the schematic view of your design in the PowerCanvas.

Controlling the Schematic from the Schematic Menu

The top-level Schematic menu offers a variety of options you can use to manipulate the schematic. This menu has the following options/sub-menus:

- **Colorize By...**

Colorizes the schematic by either Power, Activity or Connectivity.

- Power: Colorizes by the power results. The higher the power consumption, the hotter the color.

- Activity: Colorized by the activity of a given net. The more activity, the hotter the color.

- Connectivity: This is a special “power debug” mode that associates colors with instances of the schematic when you are tracing cones of logic. This is explained in more detail in [Viewing the Upstream Cone of Logic from an Input Pin of a Candidate Register](#).

- **Properties...**

Brings up the Properties dialog. This dialog displays information on pins, nets and instances selected from the schematic. This is especially useful to see all instances connected to a particular bus port. Multiple selections will show as many of the selected items as possible, even if they have different parents. You can save selection information by creating a tab. For more information on how to use this dialog, see [Using the Properties Dialog](#) and [Viewing Power Analysis Results in the PowerCanvas](#).

- **Redraw**

Redraws the current schematic.

- **Zoom In/Zoom Out/Zoom Out Full**

Zooms in/out or zooms to fit the full screen.

- **Instance**

This menu has the following sub-menus:

- Show In Hierarchy

Highlights the selected instance in the hierarchy browser.

- Show Source

Displays the selected instance source in the Source Code Browser. For details, see [Using the Smart Source Browser](#).

- Expand Children

Displays the schematic of all the child instances of the selected instance.

- Collapse Children
Replaces the schematic of the child instances of the selected instance with just the block symbol representing the parent instance.
- Show Clock Trees
Displays the clock trees for the selected instance. For details, see [Basic Clock Tree Manipulation](#).
- Show Downstream Cones
Displays all downstream cones of logic for the given instance.
- Show Upstream Cones
Displays all upstream cones of logic for the selected instance.
 - Ignore Select Cones/Show Select Cones
Ignore/shows cones of logic for upstream select signals for 2-1 muxes, unencoded muxes and tri-states.
- Show All Pins
Displays all pins of the selected instance.
- Remove Unexpanded Pins
Removes pins that have not been expanded from the schematic view. For example, when PowerArtist traces an instance in the schematic it does not show all of the pins of the instances so as to provide an uncluttered view. You can display all pins by selecting **Schematic > Instance > Show All Pins**. You can then do a trace from one of those pins or invoke the pin/net dialog. If you then want to collapse the instance to reduce clutter you can select **Schematic > Instance > Remove Unexpanded Pins**.
- Pin
This menu affects any pin selected in the schematic and has the following sub-menus:
 - Expand Drivers/Loads
Expands the drives/loads for the selected instance pin/port.
 - Collapse Driver/Loads
Removes the drives/loads for the selected instance pin/port.
 - Show Clock Tree
Displays the clock trees for the selected pin. For details, see [Basic Clock Tree Manipulation](#).
 - Show Downstream Cone
When selected for an output pin of an instance, it generates a schematic of the downstream cone of logic for that pin. It stops tracing at register boundaries or primary IOs.

— Show Upstream Cone

When selected for an input pin of an instance, it generates a schematic of the upstream cone of logic for that pin. It stops tracing at register boundaries or primary IOs. For an example using the reduction tutorial, see [Viewing the Upstream Cone of Logic from an Input Pin of a Candidate Register](#).

- Ignore Select Cones/Show Select Cones

Ignores/shows cones of logic for upstream select signals.

— Show Exclusive Cone

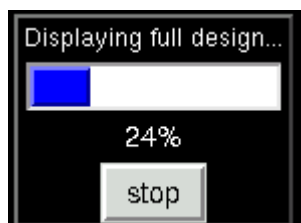
When selected for an input or output pin of an instance, it generates a schematic for the exclusive cone of logic for that pin.

- Ignore Select Cones/Show Select Cones

Ignore/shows exclusive cones of logic for select signals.

- **Expand Full Schematic**

Generates a full hierarchical schematic for your entire design. Before you select this command, consider the potential time this will take due to the size of the schematic that it will generate. When you do select this, the following progress meter will appear:



This will give you an idea of how long it will take to display the full design. If you decide you no longer want to display the full schematic, simply click the **stop** button.

- **Collapse Full Schematic**

Clears the schematic view and takes you back to the original schematic, which consists of the primary inputs and outputs.

- **Auto Hide Pins**

When selected, this toggle suppresses the display of unconnected pins for any subsequent schematic display. This feature is most useful when showing cones of logic as it will not display any of the pins that are not associated with the cone.

- **When Tracing**

This menu has two sub-menus:

— Colorize Data Path

Colorizes the data path while performing tracing.

— Colorize Select Path

Colorizes the path for select signals while tracing.

You can also select these items from the pop-up menu when you right-click on the schematic.

Using the Mouse to Zoom In and Out on the Schematic

By pressing and holding the left mouse button and moving the mouse, you will get different zoom operations. There are three supported directions:

- To the NorthEast (up and to the right): this zooms out by a factor that is proportional to the angle as well as the length of the stroke. You will see an annotation “zoom *value*” on the display.
- To the SouthEast (down and to the right): this does a window zoom where the first point is the upper left bounding box point and the second point is the lower right bounding box. You will see the annotation “zoom in” on the display.
- To the SouthWest (down and off to the left): this does a zoom fit. You will see the annotation “zoom fit” on the display.

Expanding the Schematic by Double Clicking Ports

Suppose you have pin-pointed a hierarchical instance that consumes a lot of power and you want to see the schematic for it. You can simply select that element in the hierarchy browser, right-click and select **Show in Schematic**. This will display the

schematic symbol body in the schematic with input ports listed on the left side and output ports on the right hand side, as shown in the following figure.

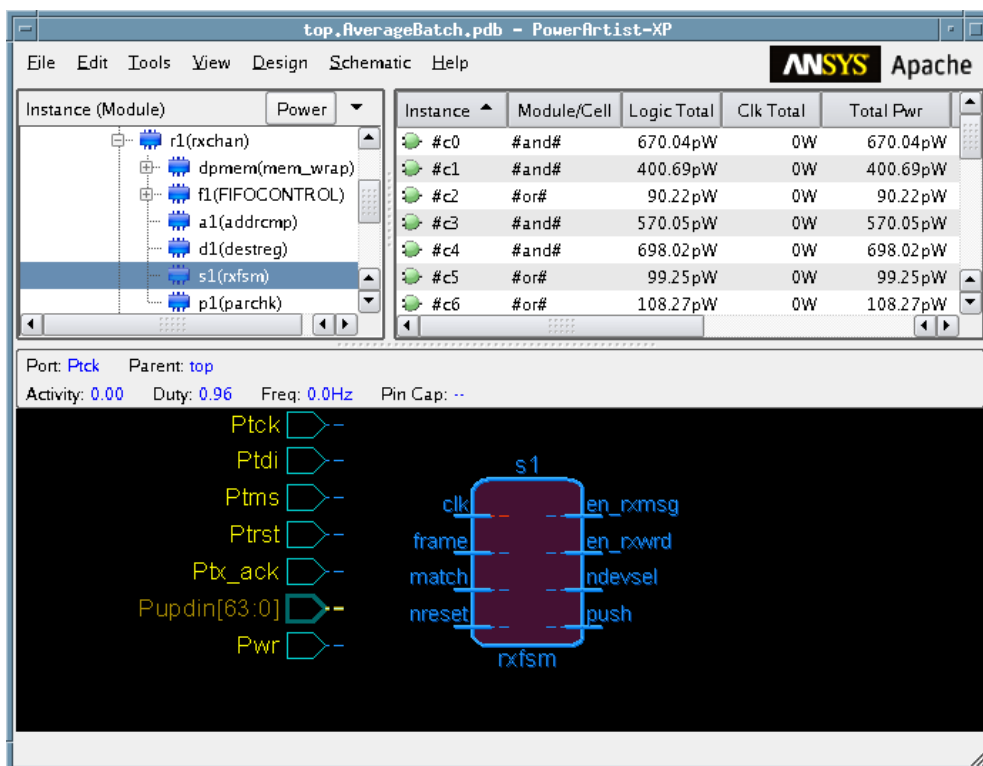


Figure 63 Hierarchical Schematic Body

Double clicking this element in the schematic while holding the shift key down expands the body into its equivalent schematic. You can then do a Zoom Fit to see the following figure.

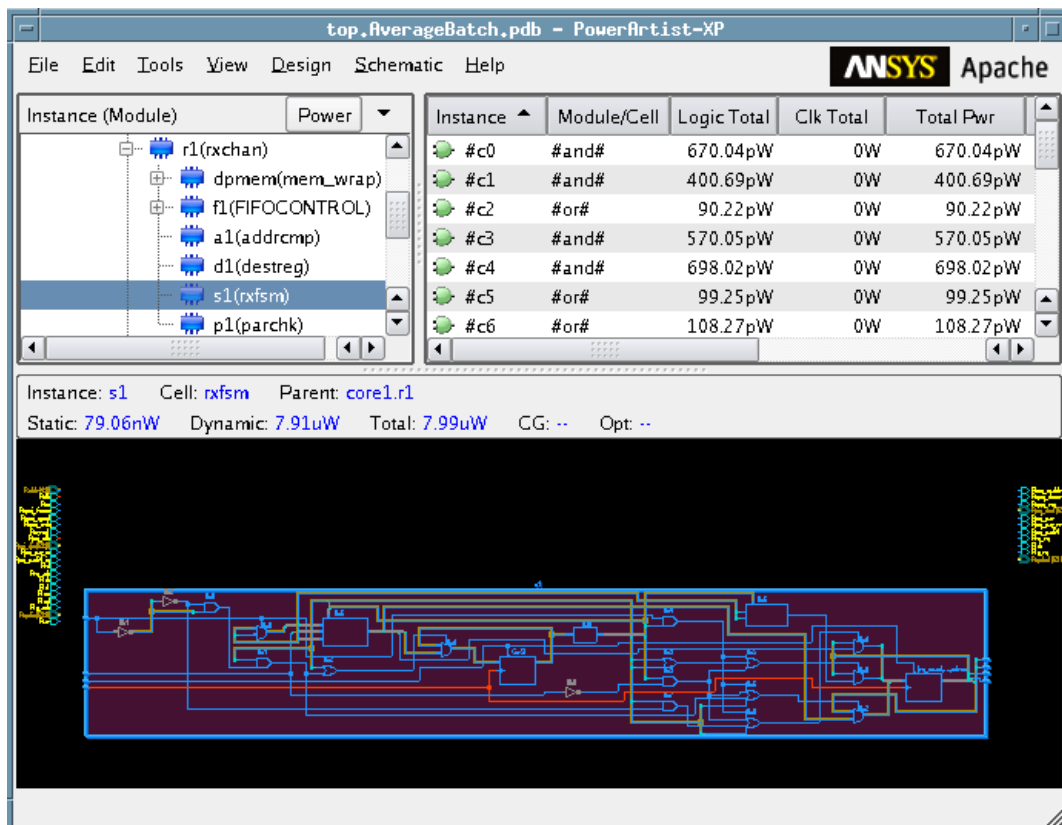


Figure 64 Nested Schematic for a Hierarchical Block

If you have done a power analysis, you can quickly find the nets that have high activity since they will be a non-blue color. Other elements are displayed in different shades of the thermal spectrum indicating that some of their powers are higher than others. If you want to see how a port of this module is connected to other modules, simply double-click on the port.

Basic Clock Tree Manipulation

To display a clock tree, you must first define your clock trees using the [SetClockNet](#) command. Second, you must set the `-save_clock_trees_netlist` option to [CalculatePower](#) or [ReducePower](#). Finally you must perform the power analysis (you will not be able to display clock trees after elaboration). Once you have performed these requisite steps, you can use the following process to display a clock tree:

1. From the schematic display, select and highlight one of three elements to display a clock tree:
 - The clock pin of an inferred register
 - The clock pin of an instantiated gate-level instance
 - The clock net defined using the SetClockNet command
2. If you select a pin, right-click to bring up the pop-up menu and select **Pin > Show Clock Tree**.

This creates a new schematic window of either the inferred clock tree for this element or the traced clock tree. If you select a pin, you will get the cone of logic that leads to that particular clock pin. If you select the clock net, you will see the entire clock tree generated in a new clock schematic.

Note that you cannot use the standard “less” and “more” schematic techniques previously described to see less or more of your clock tree. You can only traverse the clock distribution network in this view; that is, you do not have access to enable pins of integrated clock gating cells, for example. Figure 65 shows a clock tree for inferred register core1.s1.rxwrd.clock[0].

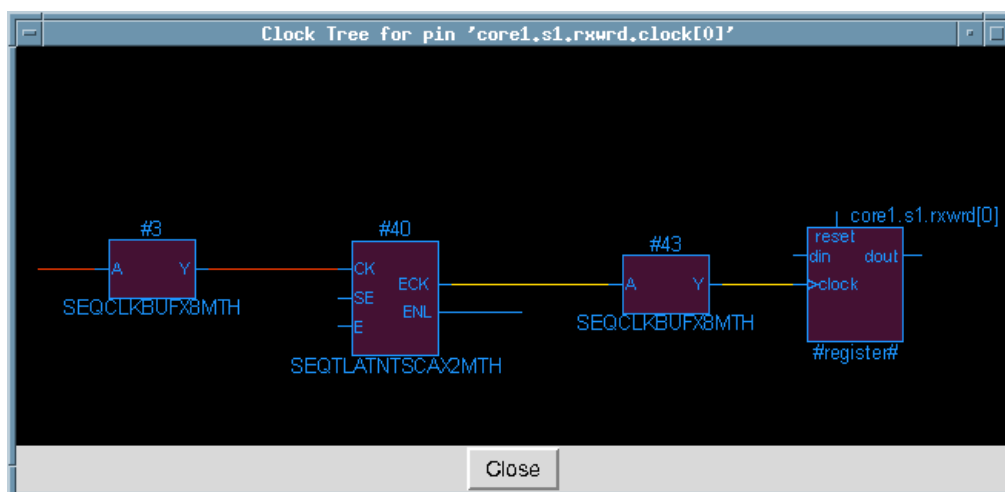


Figure 65 Inferred Clock Tree

This register name is a concatenated list of scalars.

Using the Simple Reduction Dialog

There are three separate dialogs for viewing power analysis results: Simple Reductions, Linter Reductions and Prism. This section describes the Simple Reduction dialog.

To view the results of the GMC, LNR, LEC, ODC, and DOI PowerBots, bring up the Simple Reduction dialog by selecting **View > Simple Power Reductions**. The Simple Reduction dialog will appear as shown in the following figure.

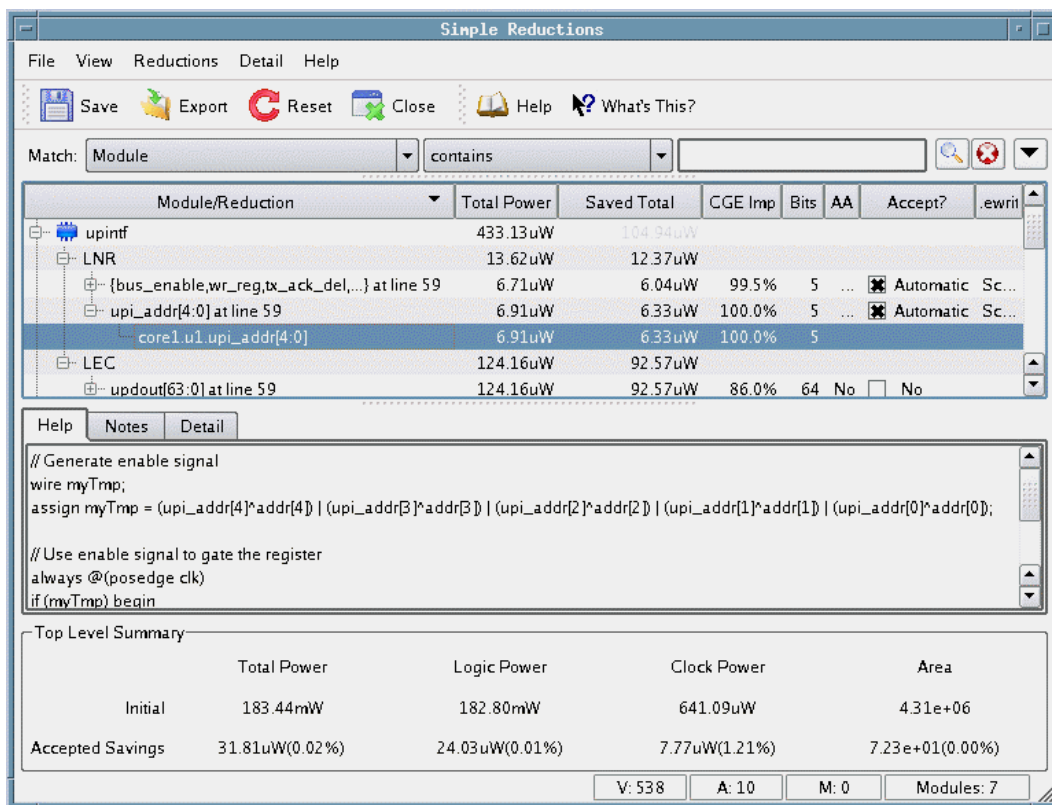


Figure 66 Simple Reductions Dialog

Column Header Definitions for the Simple Reductions Dialog

You can click on any column header in the Simple Reductions dialog to sort the data by that column. The following list provides a high-level definition of each column:

- **Module:** The name of the modules containing the reductions.
- **Reduction:** The type of reductions discovered in the given module. The line number of the RTL where the reduction has been found is also listed in this

column. Also, the name of the variable/instance associated with the reduction is listed in this column at the lowest level.

- **Instance:** The name of the instance inferred from the RTL.
- **Total Power:** The total of the logic and clock power of the RTL before any reduction takes place.
- **Logic Power:** This is all of the power that is not clock power. This include combinational logic, sequential logic (latches and register) and instantiated library elements.
- **Clock Power:** The power not including logic power. This will be the power of your clock distribution network including any inferred buffers, inferred integrated clock gating cells and any traced elements in your clock network.
- **Saved Total:** The total of the logic and clock power saved after all reductions (or the specific reduction) are applied.
- **Saved Logic:** The power of the RTL logic that is saved per module, reduction, etc.
- **Saved Clock:** The clock power that is saved after the reduction(s) are applied.
- **Pcnt Saved:** The total of the logic and clock power saved as a percentage of the total top-level power.
- **Ideal Saved Logic:** The power of the RTL logic saved after the reductions are applied but before subtracting the logic power required to implement the reduction.
- **Ideal Saved Clock:** The clock power that is saved after the reductions are applied but before subtracting the clock power required to implement the reduction.
- **Area Ovrhd:** The total area required to implement the reduction(s).
- **CGE Imp:** The clock gating efficiency (CGE) improvement percentage, which is the CGE value after reduction minus the CGE before reduction. This value is displayed for clock-based reductions only (that is, LEC, LNR and ODC in this dialog). The CGE for each instance of a register is displayed (see the blue text) and the “Line” row displays the average of all its children.
- **Bits:** The bit width of the register.
- **AA:** The AA (auto-accepted) column indicates whether or not the reduction was automatically accepted for rewrite. If a reduction is not auto-accepted, you can hover your mouse over the word “No” to see a tool tip that tells you why the reduction was not auto-accepted. Note that you can also filter on what has been automatically accepted. In the Simple Reductions dialog, the PowerBots that can be auto-accepted are: GMC, LNR and ODC.
- **Accept:** Ticking this checkbox will cause reductions that can be automatically rewritten to be scheduled (when Automatic is selected from the pull-down menu) and will add the power saved by any accepted reduction to the power summary table.

- **Rewrite:** The word “Scheduled” appears in this column when a reduction opportunity is accepted for “Automatic” rewrite.

Column Header Right-Click Pop-Up Menu

If you right-click on any of the column headers in the Simple Reductions dialog, you will see a pop-up menu. This menu shows you which column headers are already displayed [X] and which are not. You can turn the display of these columns off and on using this menu. Additionally, the first three selections in this menu have the following functions:

- **Auto Size Column:** Extends the selected column to allow you to see the entire contents of the column for all displayed reductions. Note that a ... in any column indicates that there is text that is not displayed. To see the text, you can auto-size the column.
- **Auto Size All Columns:** Extends all displayed columns to their full width or the maximum width set in the Column Width dialog for that column.
- **Column Width:** When you select this, you will see the following dialog:

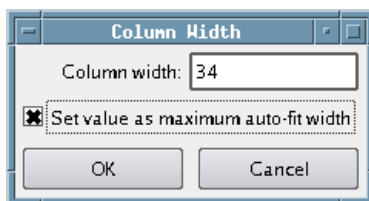


Figure 67 Column Width Dialog

In the input field, enter a specific width (in characters) and click **OK**. The column from which you invoked this dialog will adjust to the specified width.

If you specify a column width and check the “Set value as maximum auto-fit width” box, the specified value becomes the maximum width to which the **Auto Size Column** feature will expand. This is useful when instance names are very long and the **Auto Size Column** selection would otherwise make the columns too wide. Note that this maximum setting will persist in the PowerCanvas. If you want to remove the maximum setting, specify a value of 0 and click **OK**.

Simple Reduction Dialog Data Filters

You can use the “Match” filtering mechanism to filter the data in the Simple Reductions dialog. The available filters are described in the following table.

Table 1 Simple Reductions Dialog Data Filters

<i>Filters that compare against strings using contains/doesn't contain:</i>	
Module	Name
Reduction	Instance
<i>Filters that compare against numbers using standard relational operators:</i>	
Mod Total Power	Line Saved Clock
Red'n Total Power	Mod Pcnt Saved
Line Total Power	Red'n Pcnt Saved
Line Logic Power	Line Pcnt Saved
Line Clock Power	Line Ideal Svd Lgc
Mod Saved Total	Line Ideal Svd Clk
Red'n Saved Total	Line Area
Line Saved Total	Bits
Line Saved Logic	
<i>Filters that compare against</i>	
Auto-Accepted	Accepted
<i>Filters that compare against Scheduled, Rewritten, Failed</i>	
Rewrite	

The purpose of using these filters is to reduce the amount of data you are viewing at any point in time. For example, if you want to display only those reductions that were auto-accepted, you can filter on “Auto-Accepted is yes”:

1. Remove all filtering by clicking the  cancel icon.
2. Select **Reductions > Mark All Not Accepted**
3. Filter on “Auto-Accepted is yes”.
4. Select **Reductions > Mark All Accepted**.

You can use this type of filtering to restore the auto-accepted state of the reductions after manually accepting or rejecting reductions in the GUI and saving your changes.

The View Menu

The Simple and Linter Reduction dialogs have five menu items across the top of the dialog: File, View, Reductions, Detail and Help. The View menu allows you to expand or collapse collections (groups) of reductions at one time. Using this menu, you can do the following:

- Expand the view of selected items
- Expand the view of all items
- Collapse the view of selected items
- Collapse the view of all items

To expand/collapse selected items, you first need to select one or more items using the Shift-click or Ctrl-click method. With the selected items highlighted in blue, you can then click the View menu and expand/collapse the selections to your level of choice (Reduction, Line Number or Instances). The following figure shows you how you can expand multiple selections down to the line number.

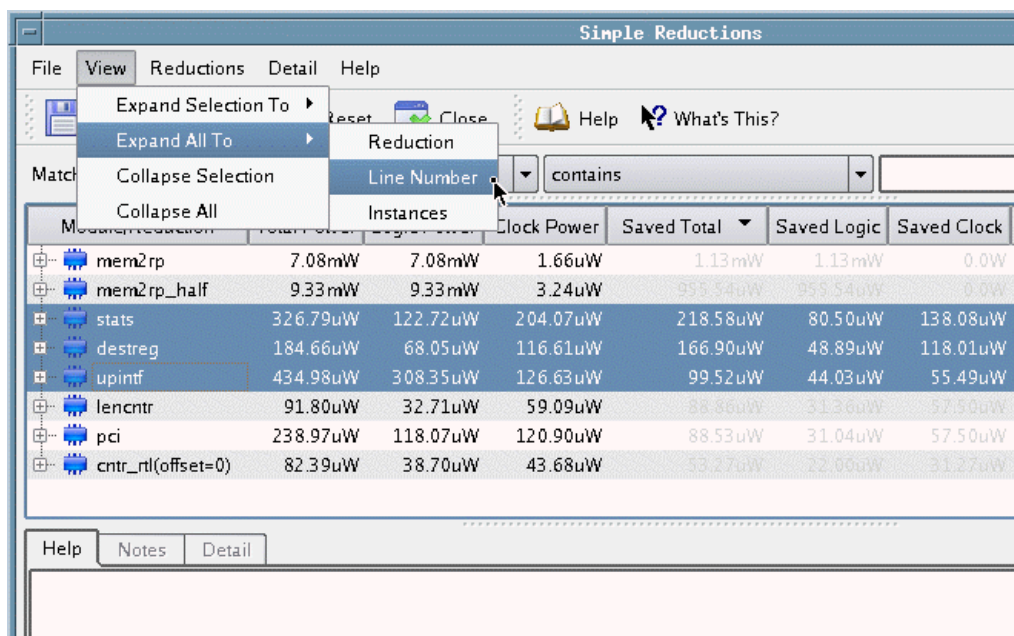


Figure 68 Expanding Selected Items to the Line Number

The “All” in the View menu items refers to all of the items currently displayed in the dialog (not all items in the database). If you previously filtered out some of the data, that filtered data is not included in the “All” operation.

For example, if you want to accept all LNR reductions you can do the following:

1. Filter out all reductions other than LNR. In the Match field, select “Reduction contains” and enter LNR in the input field. Click the magnifying icon to filter the data.
2. Use the Shift-click method to highlight all remaining lines in the dialog.
3. Click **Reductions > Mark all Accepted**.
4. Select **Reductions > Set All Rewrite Actions > Automatic**.
5. Click the **Save** button to schedule the reductions.

The Detail Menu

The Detail menu allows you to perform several different operations on an instance selected in the Detail tab (not from the reduction list). This menu provides the same options as the right-click menu that pops-up when you right click an instance name in the reduction list. The Details menu has one additional item—Show Waveform. You can also bring up these options by right-clicking on a net/instance in the Detail tab.

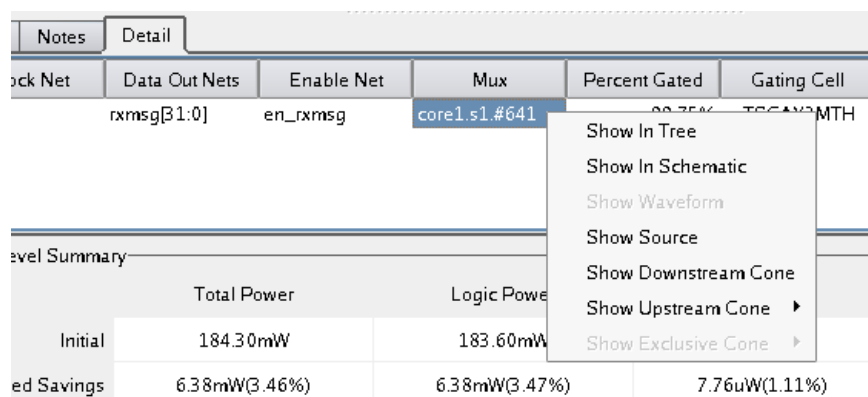


Figure 69 Right-Click Menu from Detail Tab

Using the Simple Sorting and Filtering Features

PowerArtist provides some simple features for sorting and filtering your data. In the simple mode, you can sort by simply clicking on any column header. If a column header does not show an arrow to the right of the column name, you can't sort by that column.

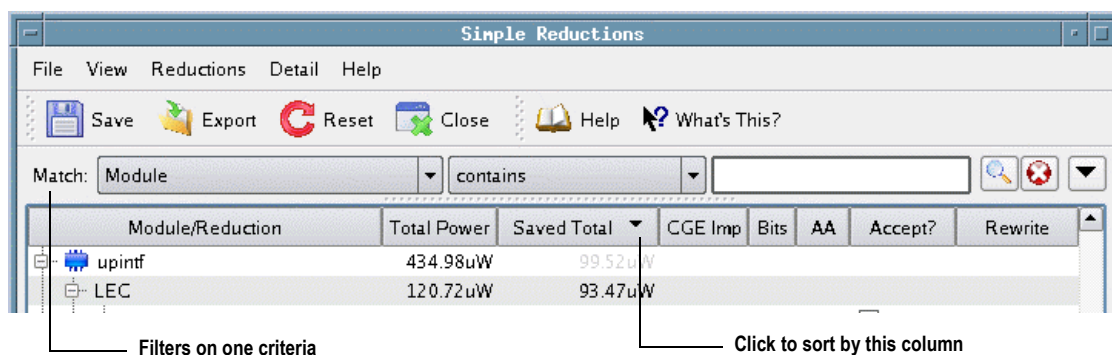


Figure 70 Simple Filtering Features

In the simple mode, you can see there is only one “Match” filter line. This allows you to filter on one set of criteria at a time. For example, you filter out any reductions that do not reduce total power by a given amount. For additional information and an example, see [Filtering Reduction Results](#) in the reduction tutorial.

Using the Advanced Sorting and Filtering Features

In addition to the simple sorting and filtering features, PowerArtist provides more advanced sorting and filtering features. To use the advanced sorting and filtering feature, click the  arrow icon on the far right of the Simple Reductions dialog. This expands the dialog to display new “Sort by” buttons and provides the ability to filter on multiple fields. For example, you can filter on “module” and whether power savings are greater than a specified value, then sort your data first by bits and then by module/reduction. To add another sort or filter field, simply click the  icon. You can add up to three sort fields and as many filters as you like.

In the following figure, there are two sorting fields and two filtering fields.

Sort by: Saved Total (dropdown) Accepted (dropdown) + (button) - (button)

Match:

- Reduction (dropdown) contains (dropdown) LNR (text input) + (button) - (button)
- Module (dropdown) contains (dropdown) p (text input) + (button) - (button)

Module/Reduction	Total Power	Clock Power	Saved Total
upintf	434.98uW	126.63uW	99.52uW
LNR	6.73uW	4.28uW	6.05uW
{bus_enable,wr_reg,tx_ack_del,...} at line 59	6.73uW	4.28uW	6.05uW
...l.u1.{bus_enable,wr_reg,tx_ack_del,...}	6.73uW	4.28uW	6.05uW
pci	238.97uW	120.90uW	88.53uW
LNR	89.49uW	57.86uW	88.53uW
pci_dout[63:0] at line 69	89.49uW	57.86uW	88.53uW
core1.p1.pci_dout[63:0]	89.49uW	57.86uW	88.53uW

Figure 71 Sample Multi-Level Advanced Sorting and Filtering

It is important to note that sorting is hierarchical, that is, it is applied within levels. Specifically, the sort feature is applied to the different levels in the following order:

1. Module level
2. Reduction level
3. Line level
4. Instance level

Given this sort order, if you sort on bits, you won't necessarily get the instance with the highest bits first.

Sorting Total Power by Module and Reduction

The following examples show how a total power sort is applied at the different levels of hierarchy in the simple reduction wizard. For example, when you sort by Total Power (by clicking the up arrow in the Total Power header) PowerArtist sorts each of the various levels—Module, Reduction, Line and Instance—as shown in the following figures.

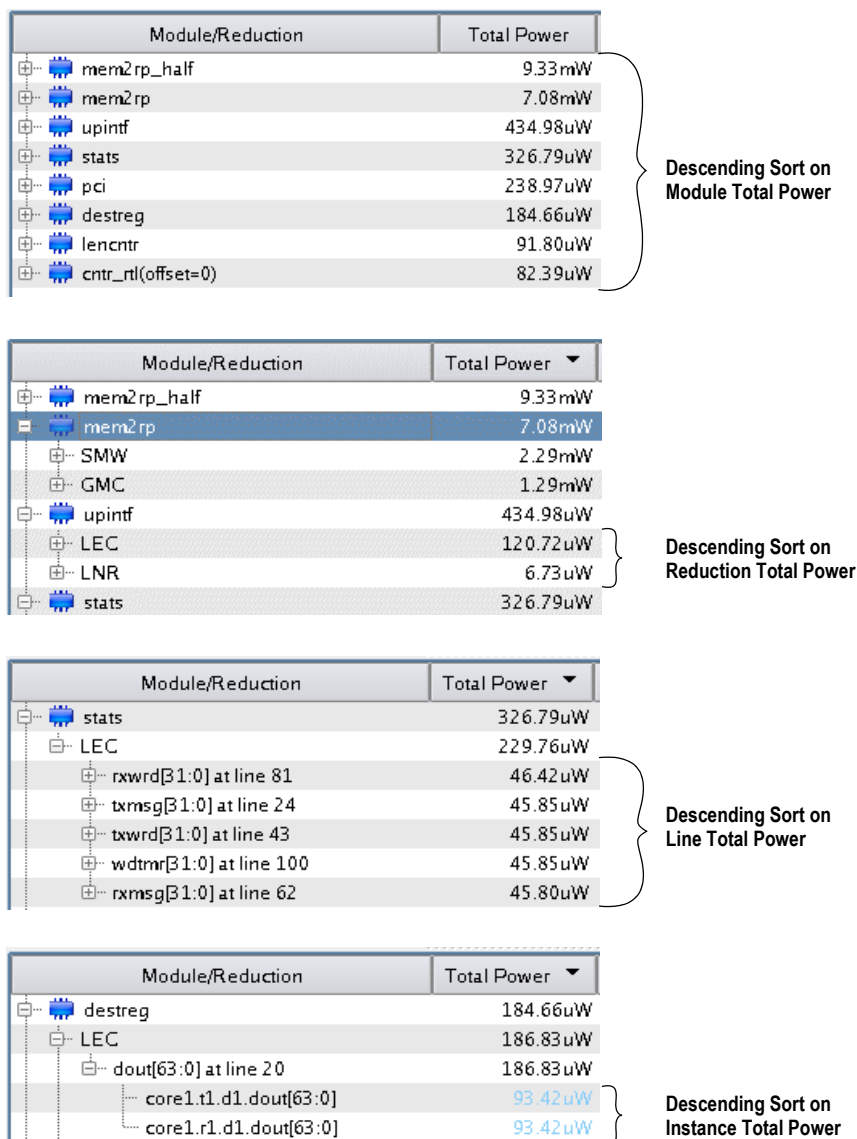


Figure 72 Sorting Total Power Values at Different Hierarchical Levels

Using the Linter Reductions Dialog

This section describes the Linter reductions dialog though many of the features described herein are also available, in a slightly modified form, in the other reduction dialogs.

To view the results of the Linter PowerBots, bring up the Linter Reductions dialog by selecting **View > Linter Power Reductions**. The Linter Reductions dialog will appear as shown in the following figure.

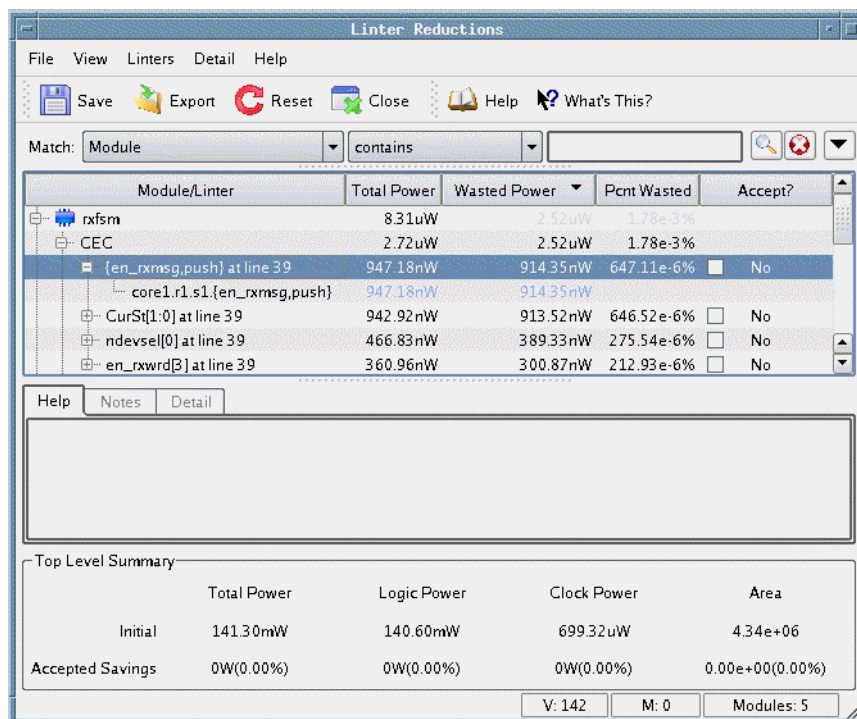


Figure 73 Linter Reductions Dialog

Similar to the Simple Reductions dialog, you can use the What's This feature to get information on each of the column headers. You can also use the Match sorting feature to sort the data.

Column Header Definitions for the Linter Reductions Dialog

You can click on any column header in the Linter Reductions dialog to sort the data by that column. The following list provides a high-level definition of each column:

- **Module/Linter:** The name of the module/linter containing the linter reduction.
- **Instance:** The name of the register instance inferred from the RTL.

For details on how you can use the data in this dialog to evaluate MUX power reduction opportunities, see [Understanding the PowerCanvas Data for the MUX Linter](#).

- **Total Power:** The total power, including wasted power.
- **Wasted Power:** The total wasted power of the RTL (per module, linter, line or instance as applicable).
- **Pcnt Wasted:** The total wasted power of the RTL in all instances (per module, linter or line) as a percentage of the total module power. For modules, it does not include child modules.
- **Bits:** The bit width of the instance associated with the linter. Note that you will need to expand to the line level to see this information.
- **Cone Power:** The total average power in the exclusive cones of all instances that experience wasted toggles (per module, linter or line).
- **Start Points:** The maximum number of start points in the exclusive cones over all instances that experience wasted toggles (per module, linter or line).
- **Accept:** Ticking this checkbox will cause reductions that can be automatically rewritten to be scheduled and for all reductions will cause the power saved to be accumulated in the power summary table.

Using the Prism Dialog

This section describes the Prism dialog though many of the features described herein are also available, in a slightly modified form, in the other reduction dialogs.

To view the results of the Prism PowerBot, bring up the Prism dialog by selecting **View > Advanced Power Reductions > Prism**.

The Prism dialog will appear as shown in the following figure.

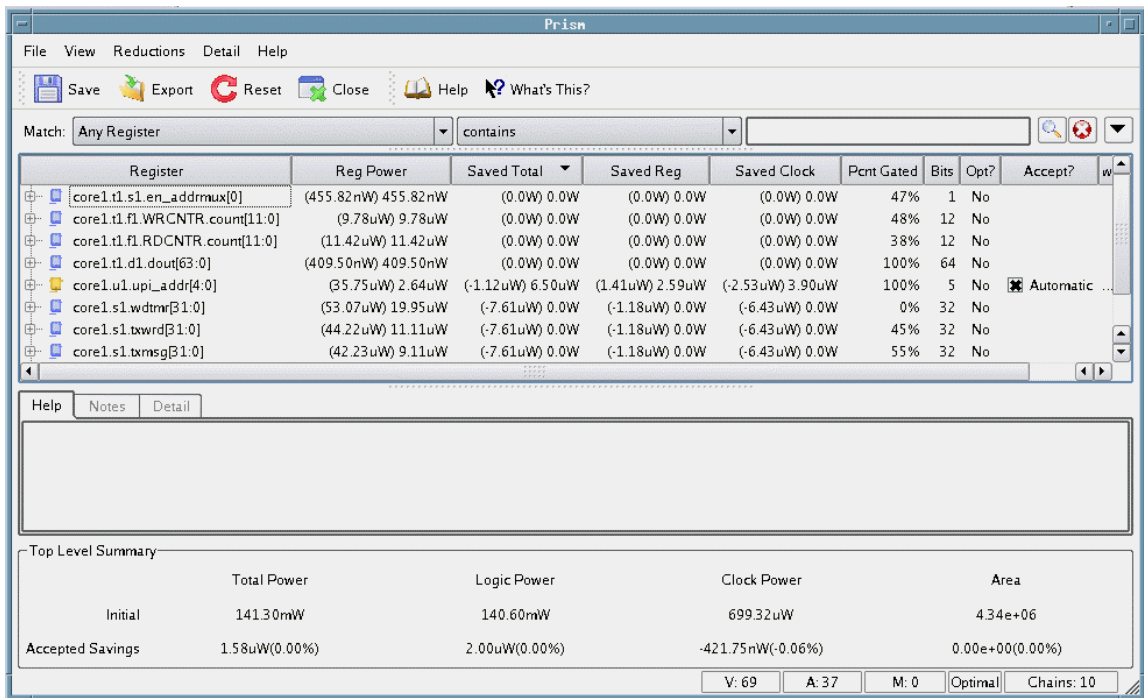


Figure 74 Prism Dialog (Initial View)

Click on each of the + signs at the beginning of each line to expand the results. You can also click on a row that has a + sign and click the * key to expand that row. When you do so, you will see that each line is color-coded, as shown in the following figure.

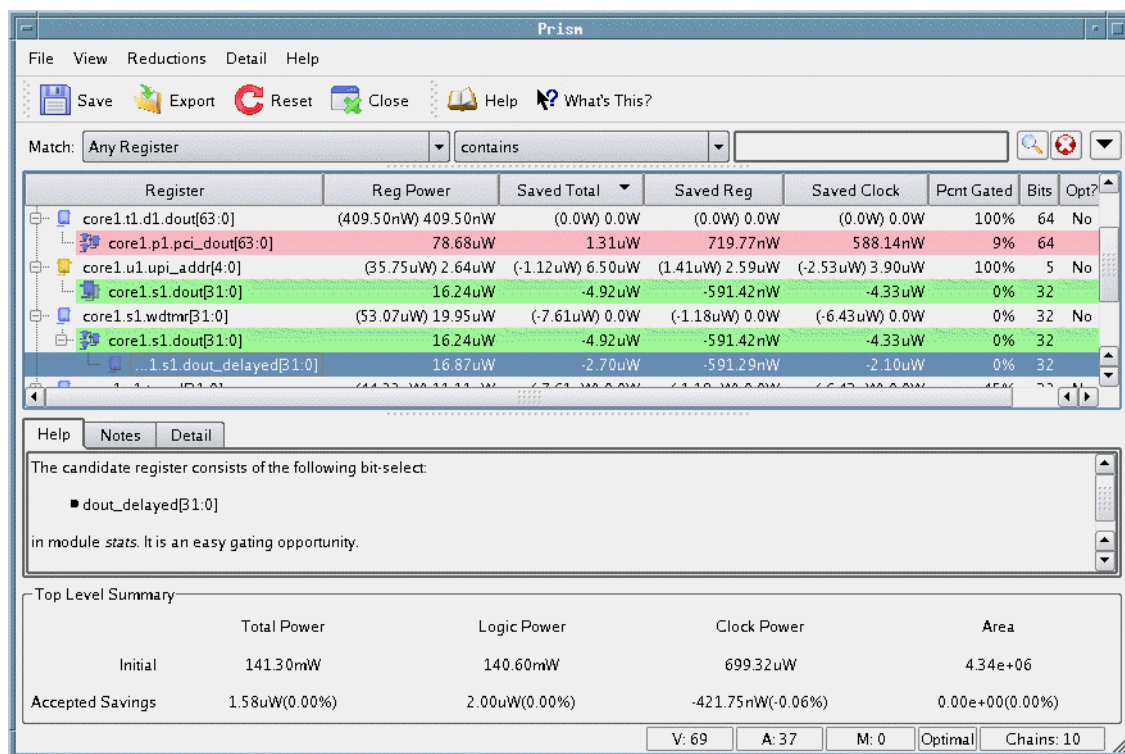


Figure 75 Prism Dialog (Expanded View)

The color key applied to the Prism results is as follows:

- White row: These are the gated registers that form the start of the potential enable propagated chain.
- Green row: These are candidate registers that only have gated registers upstream. Green rows fall into the easy category.
- Yellow row: These are candidate registers that have gated and/or candidate registers upstream. These fall into the medium category. Yellow rows are always under green rows.
- Red row: The register instance in this row has unresolved dependencies, that is, the enable logic required to clock gate this register is partially known. This may happen when:
 - a register is driven by an ungated register in addition to being driven by a white, green or yellow register.

- a register is driven by a primary input in addition to being driven by a white, green or yellow register.
- a register is driven by a macro (memory, etc.) in addition to being driven by a white, green or yellow register.
- a register is driven by a delayed version of itself in a feedback loop in addition to being driven by a white, green or yellow register.

The icons at the beginning of each line indicate the type of element in the register chain. The icons are:

-  A register gated in the RTL.
-  A register for which an enable has been generated by XORing the inputs and outputs. If the register width is greater than min_width, the enable will also be used to gate it.
-  A gated register with a strengthened enable.
-  A single candidate register: This indicates that the chain is a simple register.
-  Two registers whose outputs drive another register. This indicates that this register is one of at least two other registers that drive another register. This means that two or more registers share a common register chain. By selecting the + sign that precedes the icon, you can see the next element in the chain.
-  Two stacked registers: This indicates the start of a common register chain. Somewhere else in the display is a row that has as the  icon. Rather than searching for that, you can right-click this row and select **Move Common Chain Here**.

Categories of Implementation of Suggested Modifications

The Prism results can be put into three different categories: easy, medium and hard as described here:

- Easy—creating an enable for this register from the upstream gated registers is easy. You would just OR the enables and delay by a register.
- Medium—creating an enable for these registers is not so easy since there is a candidate upstream; however, a simple inspection of the schematic may uncover an easy way to create one.
- Hard—creating an enable for these registers is probably not possible since upstream of this register are ungateable registers, macros, primary inputs, etc. However, by looking at the schematic, you may be able to easily identify an enable. For example, the upstream ungateable register might be an initialization flop that never changes state outside of reset.

Definitions for the Bit Counters in the Prism Dialog

In the bottom right corner of all reduction dialogs, in the status bar, you will see a set of four bit counters with the following definitions:

- **V(iable)**—the total number of easy or medium bits. Note that it might not be possible to accept all of these because some might have red candidates upstream which could block an enable.
- **A(utomatic)**—the total number of bits that have been accepted for automatic rewrite.
- **M(annual)**—the total number of bits that have been accepted for manual rewrite.
- **M(odules)**—the number of Verilog modules or VHDL architectures that are displayed in the dialog.

Column Header Definitions for the Prism Dialog

You can click on any column header in the Prism Dialog to sort the data by that column. The following list provides a high-level definition of each column header that appears by default and those that you can enable (by right-clicking any column header).

The columns available by default are as follows:

- **Register:** For a gated register it is the name of the register where the enable originates. For a candidate register it is the name of the register to which the enable can be propagated.
- **Instance:** The name of the register instance inferred from the RTL.
- **Module:** The name of the module containing the register.
- **Clock Net:** The name of the clock driving the register.
- **Reg Power:** The power of the register before clock gating. The value in parentheses is the total power, before clock gating, of all the registers in the chain starting from the gated register.
- **Saved Total:** The total power (register + clock - penalty) saved by gating the clock to the register. The value in parentheses is the total power (register + clock - penalty) saved in the chain starting from the gated register if all of the registers in the chain were clock gated. Note that there might not be any savings if the clock gating percentage is too low, too few register bits are gated, or the logic required to gate the clock, an ICGC, consumes too much power.
- **Saved Reg:** The register power saved by gating the clock to the register. The value in parentheses is the total register power saved in the chain starting from the gated register if all of the registers in the chain were clock gated. Note that there might not be any savings if the clock gating percentage is too low or too few register bits are gated. Check the online help for a more detailed explanation.

- **Saved Clock:** The clock power saved by gating the clock to the register. The value in parentheses is the total clock power saved in the chain starting from the gated register if all of the registers in the chain were clock gated. Note that there might not be any savings if the clock gating percentage is too low, too few register bits are gated, or if the ICGC consumes too much power.
- **Ideal Saved Reg:** This is the total register power saved in the chain starting from the gated register if all of the registers in the chain were clock gated, but before subtracting the power in the enable propagation logic. For a candidate register, this is the register power saved by gating the clock to the register, before subtracting the power required to propagate the enable from the upstream registers.
- **Ideal Saved Clock:** The total clock power saved in the chain starting from the gated register if all of the registers in the chain were clock gated, but before subtracting the power due to the ICGCs and additional clock buffers. For a candidate it is the clock power saved by gating the clock to the register before subtracting the power due to the ICGC and additional clock buffers. Note that there might not be any savings if the clock gating percentage is too lower, too few register bits are gated, or if the ICGC consumes too much power.
- **Pcnt Gated:** The percentage of the time the clock to the register was clock gated. The value is (1 - duty cycle) of the feedback MUX select signal expressed as a percentage. For a gated register, the duty cycle is calculated from the simulation data. For a candidate register, the duty cycle is estimated by ORing the upstream enables. Sources that cannot be gated are ignored.
- **Area Ovrhd:** The total area required to implement the reduction(s).
- **CGE Imp:** The CGE improvement percentage, which is the CGE value after reduction minus the CGE before reduction. This value is displayed for all candidate registers. The root registers may or may not have a CGE value. If the root register has its enable strengthened or has a generated enable (these roots will have blue/yellow or just yellow icons) then the CGE improvement is displayed due to the strengthening or new gating. Note that if the register is a simple user-gated register, no CGE value is displayed since PowerArtist does not attempt to improve the CGE for these registers.
- **Bits:** The bit width of the register.
- **Opt:** Indicates (by yes or no) whether you can save more power by strengthening the gated register enable. If there are “yes” values in this column, you can switch to the Optimal view (if you are in the Normal view) to see the extra savings. To do this, simply select **View > Analysis > Optimal** (this is the default).
- **Gated Regs:** The number of upstream gated registers from which an enable can be propagated.

- **Candidates:** The number of upstream candidate registers from which an enable can potentially be propagated. A candidate that has a candidate upstream of it becomes a “medium” gating opportunity.
- **Ungateable:** The number of ungateable sources upstream. You need to inspect these sources to see if they will prevent you from clock gating the register. Ungateable sources makes a candidate a “hard” gating opportunity.
- **CDC:** false means this register does not cross clock domains—it is in one clock domain. True would mean that it is in more than one clock domain.
- **Local Enable:** The number of enables that are available in the same module as the candidate register.
- **Extrn Enable:** The number of enables that need to be routed to the same module as the candidate register from other modules.
- **Feeds Back:** Indicates whether the output from the candidate register feeds back to its input via another register. This might prevent you from gating the clock to the register.
- **Accept:** Ticking this checkbox will cause reductions that can be automatically rewritten to be scheduled (when Automatic is selected from the pull-down menu) and will add the power saved by any accepted reduction to the power summary table.
- **Rewrite:** The word “Scheduled” appears in this column when a reduction opportunity is accepted for “Automatic” rewrite.

Filtering Prism Dialog Data

In addition to sorting by table column, you can use the “Match” filtering mechanism to filter the data. These filters are described in the following table.

Table 2 Prism Dialog Data Filters

<i>Filters that compare against strings using contains/doesn't contain:</i>	
Any Register	Any Module
Any Instance	Any Clock Net
<i>Filters that compare against numbers using standard relational operators:</i>	
Any/Chain Reg Power	Any/All Bits
Any/Chain Saved Total	Any/All Gated Regs
Any/Chain Saved Reg	Any/All Candidates
Any /Chain Saved Clock	Any/All Ungateable
Any/Chain Ideal Reg	Any/All Local Enable

Table 2 Prism Dialog Data Filters

Any/Chain Ideal Clock	Any/All Extn Enable
Any Pcnt Gated	Chain Area Ovrhd
Any Area Ovrhd	
<i>Filters that compare against yes/no:</i>	
Any Feeds Back	All Accepted
Any Accepted	All Feeds Back
Any/All CDC	Chain Optimized

The purpose of using these filters is to reduce the amount of data you are viewing at any point in time. Filters starting with “Any...” mean that if a condition matches any register in a chain, the whole chain will be visible whereas filters starting with “All...” require that all registers in a chain match the condition for the chain to be visible. Filters starting with “Chain...” apply to the values associated with the first or gated register in the chain which are rolled-up values for the entire chain. Your goal is to reduce your power as much as possible with as little work as possible. There are several approaches to this problem.

Focus on a Module

You know by reviewing the hierarchy display that one module consumes more power than it should. You should then use the “Any Module” filter to examine that particular module. If the module has multiple instantiations you will see data for all the instantiations.

From the hierarchy browser again, you might realize that there is one particular instantiation of the module that is consuming more power than the others. You can then use the “Any Instance” filter to locate register chains involving that instance. As an example, you could use “contains” top.core1 to focus on all the register chains involving that module instance.

Focus on Clock Domains

You may be very concerned about the clock power for a particular clock. You had run PowerArtist and noticed that one clock domain consumed more power than you expected. Use the “Any Clock Net” filter with the full clock name to locate those register chains that are in the desired clock domain.

Focus on the Amount Saved

If you want to see the opportunities that provide the most savings choose Chain Saved Total and look for a number greater than some value.

Another way of looking at opportunities is by bit width. Generally, the wider the register bank, the greater the opportunity for power savings. Use the “Any Bits” filter and specify a bit width size using the “greater than” operand.

Focus on Easiest First

Making a source code edit where you do not have to cross module boundaries is the easiest alternative. This means that the enable signal you are missing is local to your module. Choose All Local Enable “greater than” 0 and the displayed opportunities will require edits to just one file.

You will also want to focus first on registers that have no ungatetable elements in their chain. This means that enables are all immediately available. Filtering on “All Ungatetable equals 0” is a good start because this will leave you with chains where the registers can be gated using upstream registers but bear in minds that some of these upstream registers might themselves be more difficult opportunities.

Focus on the More Difficult

Once you have done all the easy changes, and you still want to continue reducing power, you should remove from your view all the opportunities you have accepted. Filter on “Any Accepted No” to remove from view any chain with an accepted opportunity. To focus on chains crossing module boundaries, filter on Any Extn Enable “greater than” 0.

Using the Normal vs. Optimal Views

Designs often contain ineffective enables, which result in a substantial amount of wasted power. These enables are recognized by the CEC linter, which then estimates the resulting wasted power in the particular register they are enabling. These inefficient enables can also impact the quality of the Prism powerbot results. This is because Prism replicates existing enables to use as the enables of candidate registers later in the chain. If these replicated enables are recognized by CEC, you may be wasting power in both the gated register and in every register downstream that uses that enable. Therefore, by making that one enable more efficient, you will save even more power in the entire chain.

The Prism dialog has two different views—Normal and Optimal. The Normal view represents the power savings you would achieve given the simulation vectors used to perform the reduction analysis. The Optimal view represents the power savings you would achieve if your enable was “optimal”.

By examining the difference between the Normal and Optimal savings values, you will be able to see the impact that an inefficient enable has on multiple registers in your design. You can toggle between the two views using the **View > Analysis** menu. The status bar at the bottom right of the dialog shows the current mode as Analysis: type where type is Normal or Optimal (default).

The Opt?. column in the Prism dialog mean “Optimizable?”. If its value is “yes” then this chain has an inefficient enable which you can optimize to further reduce power. If its value is “no” then this chain has the best enable possible given the simulation vectors used to perform the reduction analysis. The dialog displays this field for the root (gated) register because this is the enable signal that will be leveraged through the chain. As with other dialog columns you may filter the data using this column’s value. For example, if you want to see only Prism opportunities that have enables that are inefficient and can be optimized, you would filter that data using “Chain Optimized is yes”.

Note that when you change the view between Normal and Optimal, PowerArtist recalculates all of the values in the dialog, including roll-up values, using the appropriate data. It also updates the summary pane.

Using the Optimized Results to Improve the Power in your Design

You will be able to view the optimal results in the Prism dialog if you run a reduction analysis with the Prism PowerBot using either the [ReducePower](#) command or the PowerCanvas wizard. Use the following flow to generate and view optimal results:

1. Run the [ReducePower](#) command with the Prism PowerBot enabled and open the Prism dialog in the PowerCanvas.
2. Ensure that the display is in the “Analysis: Optimal” mode by selecting **View > Analysis > Optimal**. Verify that you are in the Optimal mode by checking the status bar at the bottom right of the dialog.
3. Filter out the chains that need optimizing and chains with small bit width registers. You can do this by applying the following filters:
 - a. “Chain Optimized is no”
 - b. “Any Bits greater than 2”
4. Accept all of the easy chains by selecting **Reductions > Mark All Chains Accepted > Just Easy Candidates**.
5. Make a note of how much power is saved in the summary table.
6. To see just these chains set a single filter to “Any Accepted is yes”
7. Sort on Saved Total (by clicking the column header). You now have a list of register chains with readily available efficient enables that can be easily implemented.

8. Repeat this process on chains that would benefit from more efficient enables. To do this, first define the following two filters:
 - a. “Chain Optimized is yes”.
 - b. “Any Bit greater than 2”
9. Accept all the easy chains by selecting **Reductions > Mark All Chains Accepted > Just Easy Candidates**.
10. Make a note of how much the total savings has increased in the summary table. If this amount is substantial, then modify the previous filter to:
 - a. “Chain Optimized is yes”.
 - b. “Any Accepted is yes”
11. Sort on Saved Total (by clicking the column header). You now have a list of register chains with inefficient enables that if implemented would provide extra power savings.
12. If there are many chains of this sort it might be useful to see if it is worth the effort to create better enables to achieve the extra savings. To do this toggle the display to Normal by selecting **View > Analysis > Normal**. Make a note of how much the total savings in the summary table decreases. Based on this decrease, you can decide whether you want generate better enables or just use the existing enables.
13. At this point, you are now left with things that are potentially more difficult to implement. Follow your previously established methodologies.

Using the Waveform Viewer

This section focuses on how to use the Apache Waveform Viewer. You can use the waveform viewer to view output from PowerArtist or from external sources. Note that you can use the Tips tab at anytime to see some tips on performing common functions using the waveform viewer.

Supported Waveform Sources

You can view waveforms from the following sources:

- From an FSDB file.

The waveform viewer supports all FSDB files that PowerArtist supports.

The waveform viewer supports ETCL and PTCL files as well.

- From an HSPICE (CSDF or PSF) or a Spectre PSF file.

For details, see [Displaying Waveforms from an HSPICE or Spectre Simulation](#).

As a PowerArtist user, you will mostly be using FSDB files and potentially PTCL files. There is very little reason to use PTCL files since PowerArtist has the capability of generating power waveforms in the FSDB format. These will be much faster and take less disk space than their .ptcl equivalents.

Finding and Displaying Waveforms from an FSDB File

Use the following process to view waveforms stored in an FSDB file.

1. From the main PowerCanvas window, select **Tools > Waveform Viewer**.

The Waveform Viewer appears as a separate window with the Add Waveform Source window open.

2. From the Add Waveform Source window, select an FSDB file (or PTCL file). For demonstration purposes, you can use the FSDB file in the PowerArtist tutorial. Simply navigate to the \$POWERTHEATER_ROOT/tutorial/analysis/design_data/rtl_sim/directory and select the activities.fsdb file.

3. Click the **OK** button.

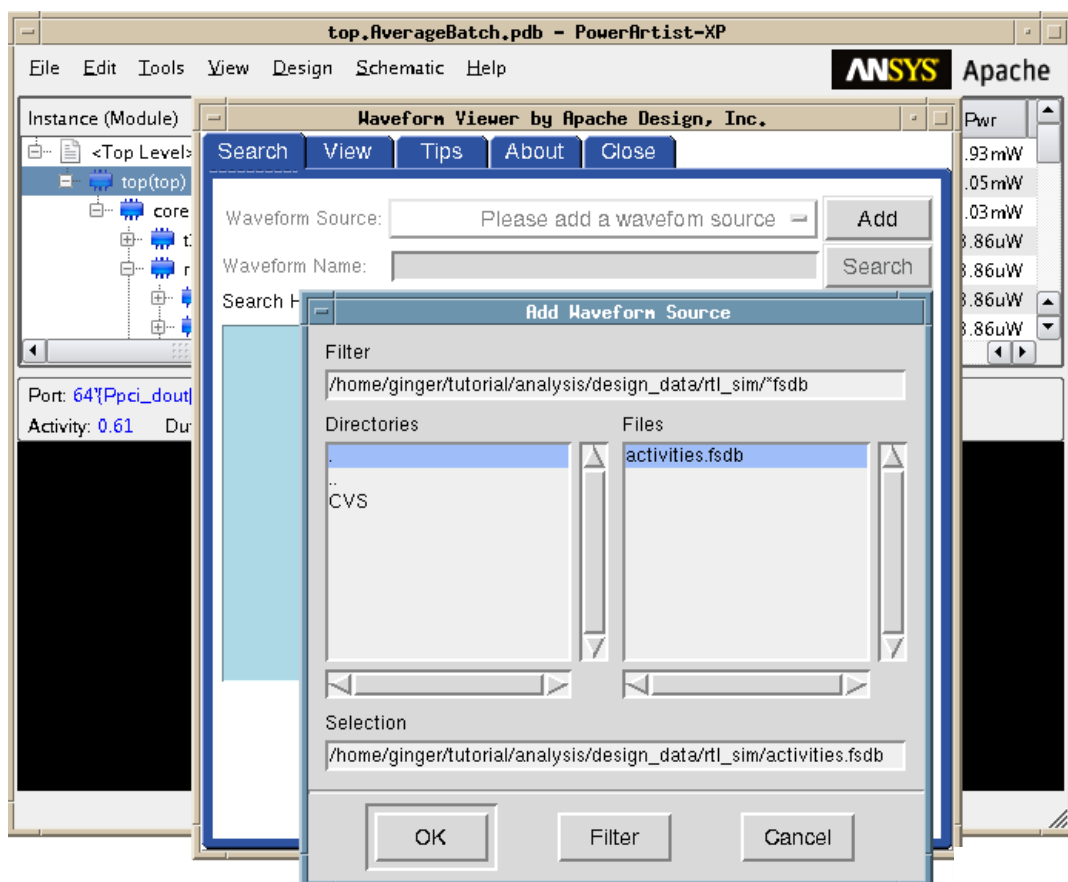


Figure 76 Selecting an FSDb File with the Waveform Viewer

4. To locate a signal name to view as a waveform, you have three choices:
 - Type in a search pattern in the Waveform Name entry field. The pattern is case-insensitive and can contain the * wild card character.
 - Cross-probe instances from the schematic or the Properties dialog.

When you select an instance in the schematic, the name will be transferred to the Waveform Name field with two modifications. The syntax will be:

.full_path_to_instance.

The *. is added to the beginning of all cross-probed nets/instance. This is due to the top-level hierarchy of the testbench that is reflected in the FSDb file. Adding a . * wild card at the beginning of the name will match any top-level string that begins the hierarchical net names (for example, testbench.top0.).

The `.*` is added to the end of the hierarchical name so that the waveform viewer finds all of the nets defined in the selected instance. It will recursively match all the nets defined by its children instances as well. Given the sample in Figure 76, if you selected `t1/txchan` the following name is added to the Waveform Name field in the Search tab:

`*.core1.t1.*`

— Cross-probe wires from the schematic or Properties dialog.

If you select a wire, the name of the associated net will appear in the Waveform Name field as:

`*.full_path_to_net`

Since there is a single net associated with the wire, there is no need for the `.*` at the end of the name. If you select a bus, the name generated will be:

`*.full_path_to_bus[*]`

If you select an inferred instance or net, then no cross-probing is done to the waveform browser since there can be no possible match in the FSDB file.

5. Once the Waveform Name entry field is filled-in, click the **Search** button to perform the search. Cross-probing performs an automatic search for you. The waveforms that are found, using the selected waveform source, will be listed in the box below the entry field.

The following figure shows the Search tab with a list of available waveforms.

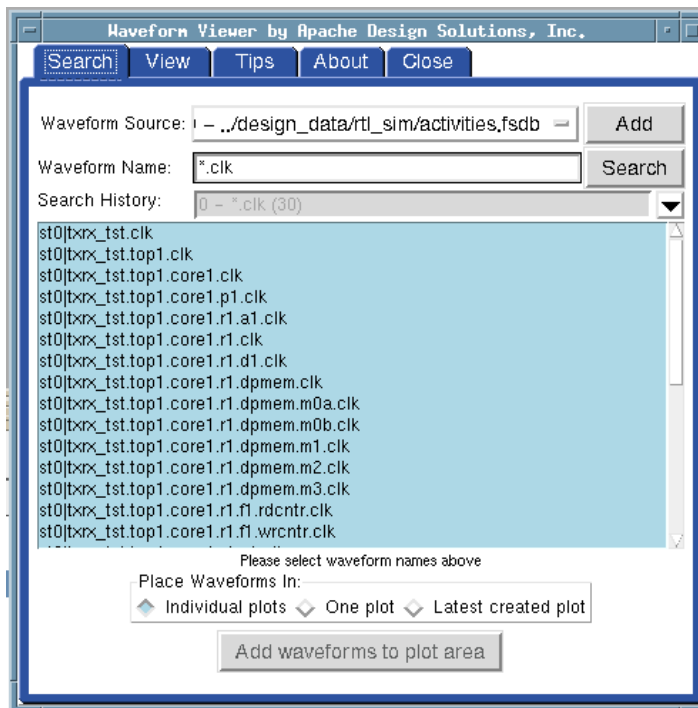


Figure 77 Waveform Viewer with a List of Available Waveforms

In this figure, the search pattern is *.clk which locates 30 waveforms. If your search returns more than 100 matches, the browser displays the first 100 matches and generates a warning that the remaining matches will not be displayed. The Search History selector menu contains a log of all searches in the current session. As you keep searching, this menu is updated, allowing you to quickly go back and forth between different search results. This eliminates the need to re-invoke a search for a given pattern. For any of the listed searches, the first number indicates the waveform source for which the search was executed, followed by the pattern that was entered, and finally the number of matches.

6. From the list of available waveforms, select the waveforms you would like to display, and click the **Add waveforms to plot area** button. If you want to select all of the listed waveforms, right-click on the list and select **Select All**.

If the external file size is large, it may take some time for the waveforms to appear. Note that although the option appears available, you cannot place more than one

digital waveform in a single plot (it will not look right). Figure 78 shows three waveforms in individual plots that were selected from the Properties dialog.

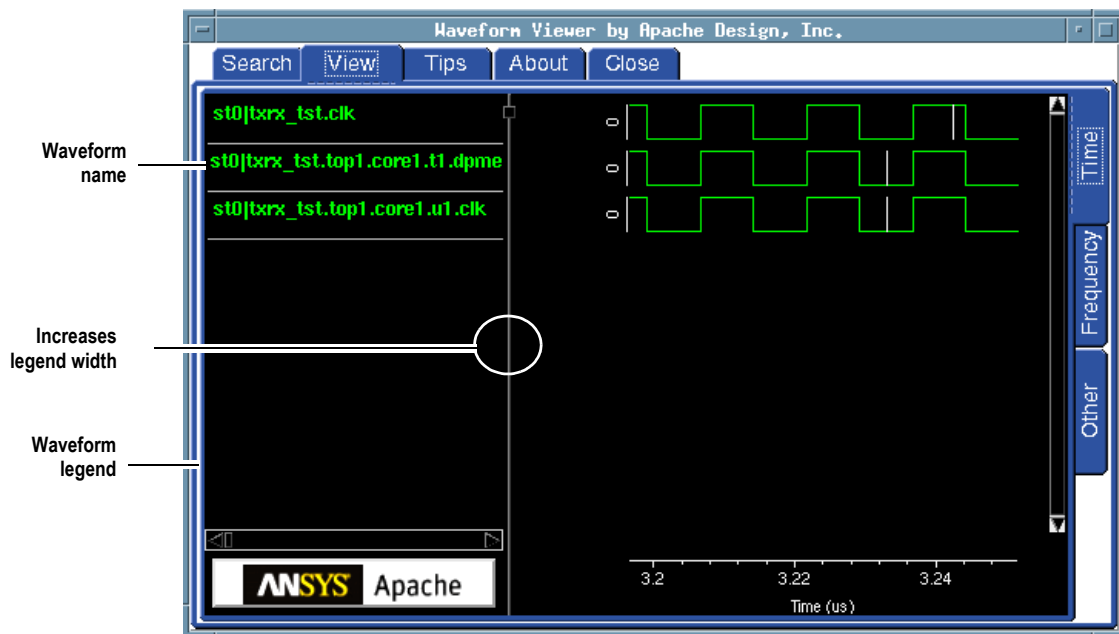


Figure 78 Waveform Display with Three Plots

7. Right click on any plot to bring up a menu of available operations. You can manipulate the display using the Display Options menu. For easier viewing, the plots in this figure were increased in width by 20% (**Display Options > Increase height of all plots by 20%**). For more information on options, see [Using the Waveform Viewer Options Menu](#).

Understanding the Waveform Name

There are four distinct elements in a waveform name:

- The first element indicates the type of waveform: u (unknown), p (power), s (state), and v (voltage).
- The second element represents the domain: t (ime) or f (requency).
- The third element is the index number assigned to the source of the waveform. This index is listed next to the source name on the Waveform Source drop-down list. The 0 index is used for the first waveform source. This index is increased by 1 for each additional source.

- The fourth element—the element that follows the pipe—indicates the name of the waveform as exists in the waveform source. The following figure breaks-down a sample waveform name.

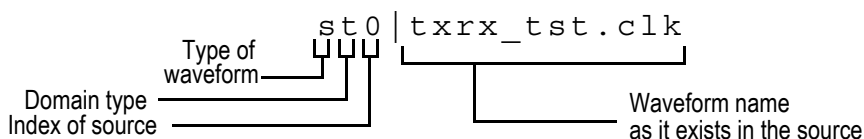


Figure 79 Break-Down of a Waveform Name

In this figure, the waveform is a state waveform in the time domain for a waveform called `txrx_tst.clk` in the first listed source (source index 0).

Manipulating the Waveform Display

To delete the waveform, double-click a waveform name.

To zoom-in on particular area of interest on a waveform, click the middle mouse button and hold it down while dragging the mouse to form a rectangle around the area of interest.

Displaying Waveforms from a PTCL File

The waveform viewer can display waveforms from a PTCL file. The analysis tutorial generates a `ptshell.ptcl` file. If you have this file available to you, you can view this file by adding it to the waveform source, as described in the previous section. Simply delete out the `**fsdb` from the end of the Filter field entry and replace it with `*ptcl` to see the available PTCL files.

Displaying Waveforms from an HSPICE or Spectre Simulation

The waveform viewer can display waveforms from an HSPICE or Spectre simulation. For HSPICE, the simulation output can be in either CSDF or PSF format.

To generate the output in CSDF format, you must include the following setting in your HSPICE simulation input file:

```
.options CSDF
```

Make sure that there isn't another output format option specified after this CSDF specification (for example, `.options POST`)—this would override the CSDF setting. Since a CSDF file is an ASCII file, displaying waveforms from very large CSDF files will be slow; however, you can speed up this process by using `.PROBE v(XXX)`

statements in your HSPICE simulation input file—to limit number of reported waveforms.

To generate the output in PSF format, you must include the following setting in your HSPICE simulation input file:

```
.options psf=2
```

Specifying '2' creates an ASCII PSF file, which the waveform viewer supports. Note that the waveform viewer does not support binary PSF files.

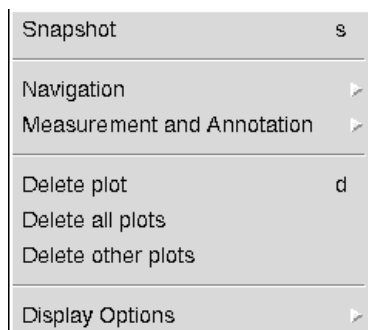
For Spectre, you need to set the following option:

```
psfascii
```

You can then load HSPICE or Spectre output files as you would for any source file.

Using the Waveform Viewer Options Menu

Once you have a waveform in the plot area, you can right-click anywhere on the plot to bring up the options menu. You must continue to hold the right mouse button to see the menu as shown below. Note that some menu items have single-character shortcuts, as noted on the right side of the options menu.



This main menu allows you to:

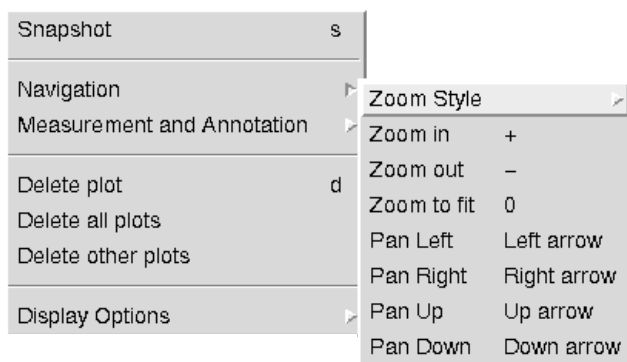
- Take a snapshot

To take a snapshot of the current waveform view, press the “s” key or right-click on the plot and select **Snapshot** from the options menu. You have the choice of either .png or .gif formats for the saved snapshot. The default format is .png due to it’s small file size.

- Delete a plot, Delete all plots, and Delete other plots (other than the current plot).

Using the Navigation Sub-Menu

From the options menu, select Navigation. The following sub-menu appears.

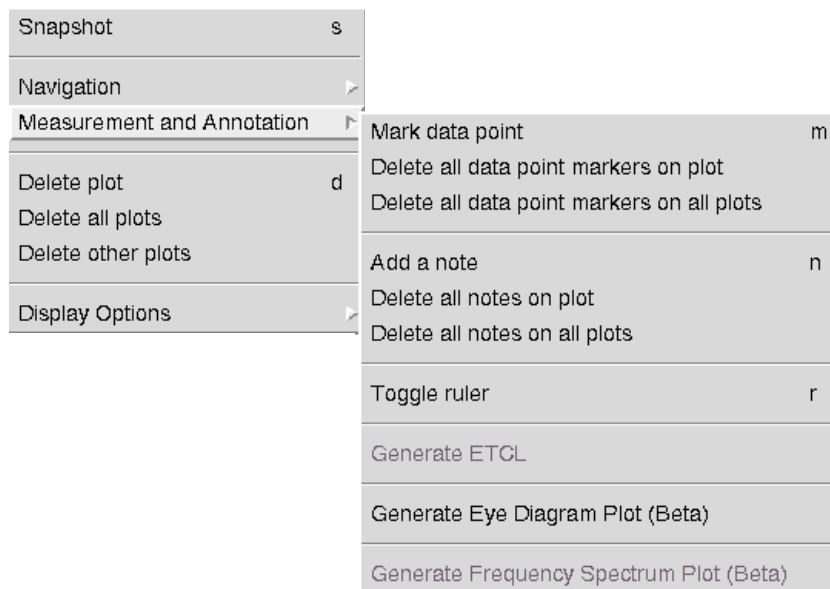


Using this sub-menu, you can do the following:

- Select a Zoom Style (Zoom x only, y only or x & y (default)).
- Zoom-in, Zoom-out, and fit area
- Pan a waveform in different directions.

Using the Measurement and Annotation Sub-Menu

From the options menu, select Measurement and Annotation. The following sub-menu appears:

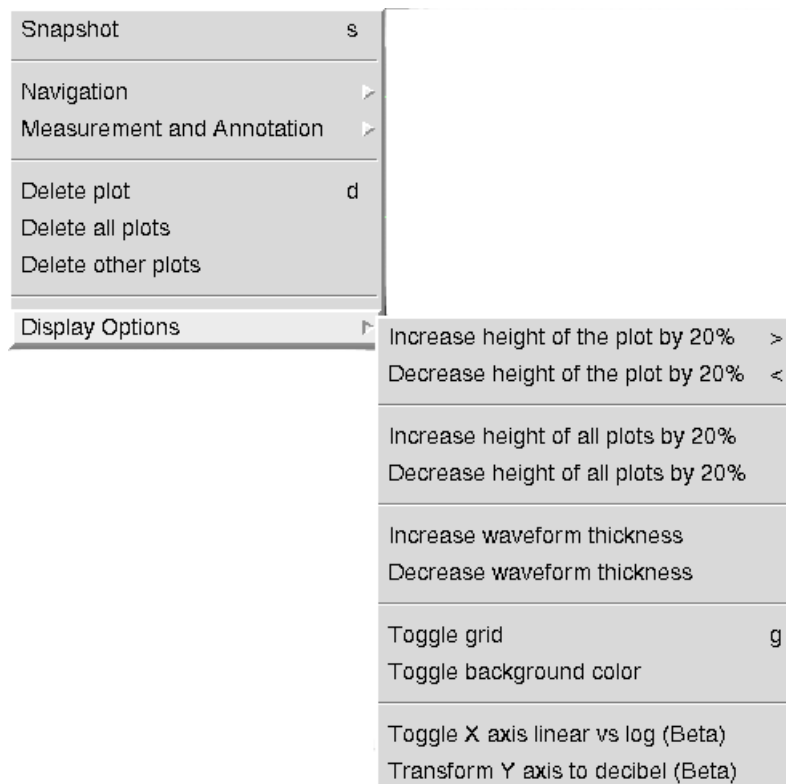


Using this sub-menu, you can also do the following:

- Mark a data point. Simply hover the cursor over any area on a waveform and press the “m” key or select Mark data point from this menu.
- Delete all data point markers on plot or on all plots
- Add a note (for more information see [Adding Notes to a Plot](#)).
- Delete notes on a plot.
- Toggle the ruler feature (for more information on using the ruler, see [Measuring the Distance Between Two Data Points](#)..

Using the Display Options Sub-Menu

To make changes to the way the waveforms display, select Display Options from the options menu. The following sub-menu appears:



Using this sub-menu, you can do the following:

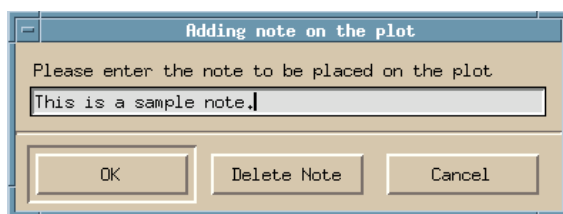
- Increase/decrease height of one/all plots by 20 percent.
- Increase/decrease waveform thickness.
- Toggle grid, which displays/hides a grid on the plot.
- Toggle background color, which switches the background from black to white and back again.
- Toggle x-axis linear vs. log (Beta), which allows you to look at the x-axis in linear values vs. log scale (for example, ticks would be 1,2,3,4 vs 1,10,100,1000).
- Transform Y axis to decibel (Beta). This is useful for generating bode plots from some analog signals.

Adding Notes to a Plot

At some point, you may want to share a snapshot of a waveform, either through e-mail or as a slide in a presentation. To save a snapshot of a particular waveform, You may want to explain a detail about the plot or highlight a particular area of interest.

Use the following process add notes to a waveform and take a snapshot:

1. Bring up the desired waveform view.
2. Press the “n” key or right-click on the plot to bring up the note window.
Alternately, you can right-click on any waveform and select **Measurements and Annotation** > **Add a note**.



3. Type in the note content and click **OK**.
The note will be added at the point where you originally clicked on the waveform.
Additional note manipulations available:
 - To move a note to a different location, simply click and drag it.
 - To edit the contents of a note, simply double-click it. Deleting all contents in a note will delete the note.
 - To delete a single note, double-click an existing note to bring up the note window and click the **Delete Note** button. To delete all notes from a plot, select **Measurements and Annotation** > **Delete all notes on plot** from the options menu.

Comparing Waveforms in the Same Plot

To put multiple waveforms in the same plot, you should select the **Place in one plot** checkbox on the Find Waveforms tab before clicking the **Add waveforms to plot area** button. If you already have individual plots, you can drag one waveform into the plot of another. To do so, click anywhere on a waveform and drag it to the other plot.

Opening Tabs in a Separate Window

To open a tab in a separate window (also referred to as a tearing tab), click the ----- perforation the tab titles. This will open the tab in a separate window. Closing the separate window will bring it back to its tab. This feature allows you to view the Find Waveforms tab and View Waveforms tab side-by-side and see the waveform

additions immediately as you click the Find button. This is more efficient than clicking back and forth between tabs.

If you are a new user, you might want to tear the Tips tab for quick reference while you are viewing the waveform tab. This is also useful for comparing time-domain waveforms on the time domain tab against their frequency spectrum counterpart on the frequency domain tab.

Chapter 7

Getting Your Design into PowerArtist

Introduction

This chapter describes the command-line flow for creating a scenario file for Verilog, VHDL and mixed-language designs. The designs may be RT level, gate level or mixed RTL and gate. The one common command that you will be using is the [Elaborate](#) command.

The scenario file is a binary representation of the hierarchical micro architectural netlist for your design. This chapter includes simple command examples to help you understand the flow. More complex examples of commands can be found in the `examples/command_files` directory.

You can create your scenario file from the command line (or from the PowerCanvas). For either approach, you must first make sure the PowerArtist design environment is set up correctly, as described in [Installing and Setting Up PowerArtist](#).

Chapter Organization

The following topics are covered in this chapter:

- [Command-Line Flow for Verilog](#)
- [Command-Line Flow for VHDL](#)
- [Command-Line Flow for Mixed-Language Designs](#)
- [Creating Your Map Files](#)
- [Creating Custom VHDL Packages](#)
- [Defining Libraries for Command-Line Use](#)
- [HDL Advanced Topics](#)

Command-Line Flow for Verilog

This section describes how to prepare Verilog design files in order to elaborate your design using Verilog files directly from the UNIX command line.

1. Ensure that the design successfully compiles in your target simulator and ideally functions the way you expect.
2. Run the [Elaborate](#) command to create a scenario file for your design.

Syntax

```
Elaborate -scenario_file scn_file -top top_module  
         -verilog_startup_file startup_file
```

where *startup_file* is the Verilog startup file, *scn_file* is the full scenario file name and *top_module* represents the root of your design.

Example

```
Elaborate -scenario_file my.scn -top top  
         -verilog_startup_file startup.vc
```

To get a list of all options available, see the *Elaborate* command in the *PowerArtist Reference Manual*.

3. At this point, the flow becomes language independent. You will use the [CalculatePower](#) and [ReducePower](#) commands to analyze your simulation information and either run a power analysis or perform a power reduction.

Note that you might want to create run scripts that run these commands for you instead of entering them directly on the command line. For sample scripts, see [Running an RTL Full-Chip Power Analysis Using Command Files](#).

Command-Line Flow for VHDL

Use the following process to perform power analysis using the command line for designs that include VHDL.

1. Ensure that the design successfully compiles in your target simulator and ideally functions the way you expect.
2. Generate your list of VHDL files that need to be compiled into the appropriate libraries. There are three ways to do this:
 - Automatically generate it using your internal processes just like you probably generate your simulation compile script.
 - Translate an existing command file format, like that used for your simulator of choice to the PowerArtist format.
 - Run the `map-file/makefile/ptCompileScript` option.

The flow described next covers the `map-file` approach.

- a. Build a mapping file that describes the names of all of your VHDL libraries and the files you want to compile into them. For more detail, see section [Creating Your Map Files](#).
 - b. Build “Makefiles” using the mapping file generated during step a by running the `wwvmkr` utility. These Makefiles define rules that compile your VHDL design units in the correct order to meet VHDL language requirements, where all design units must be compiled before they can be referenced. For more detail, see section [Running the wwvmkr Utility](#).
 - c. Run the `ptCompileScript` utility. This utility executes “make” on all of your makefiles created in step b, determines all of the libraries and files used in your design, generates a file, `ptSourceFiles.tcl`, that contains a series of `CompileFile` commands that compile each of your VHDL files into the correct library in the correct order. See [Contents of the ptSourceFiles.tcl File](#) for a sample of what this file looks like.
3. Generate a scenario file using the `Elaborate` command to analyze, elaborate, and infer your design.

Syntax

```
source ptSourceFiles.tcl
Elaborate -top top_module_name -scenario_file scenario_name
```

where `ptSourceFiles.tcl` is the file generated by `ptCompileFileScript`,
`top_module_name` is the VHDL top-module name and `scenario_name` is the full scenario file name.

Example

```
source ptSourceFiles.tcl
Elaborate -top top -scenario_file my.scn
```

4. At this point, the flow becomes language independent. You will use the [CalculatePower](#) and [ReducePower](#) commands to analyze your simulation information and either run a power analysis or perform a power reduction.

Note that you might want to create run scripts that run these commands for you instead of entering them directly on the command line. For sample scripts, see [Running an RTL Full-Chip Power Analysis Using Command Files](#).

Command-Line Flow for Mixed-Language Designs

Whether you compile on the command line, using a script or from the PowerCanvas, you need to ensure that your design files contain instantiations and declarations of modules (or entities) from different languages. In PowerArtist, you compile mixed-language design at one time, which allows you to leverage information from your mixed-language simulation environments.

VHDL Designs with One or More Verilog Modules

Before you compile a VHDL design that instantiates one or more Verilog modules, you must ensure that:

- The VHDL source files contain a component declaration and an instantiation for each Verilog module. The component must have the same name as the corresponding Verilog module. The component ports must have the same name, ordering, size and direction as the ports of the corresponding Verilog module.
- For example, your VHDL source files could contain the following component declaration for a Verilog module, LEAF:

```
-- component declaration for the verilog module LEAF
component LEAF
    port( I1, I2 : in bit_vector(1 downto 0 );
          OUTPUT : out bit_vector(1 downto 0 ));
end component;
```

- It would also contain an instantiation of LEAF:

```
-- component instantiation of LEAF
INST_LEAF : LEAF
    port map(I1(1 downto 0), I2(1 downto 0), OUTPUT(1 downto 0));
```

- The Verilog source files must contain definitions of the modules. For example, your Verilog source files could contain the following definition of LEAF:

```
module LEAF (I1, I2, OUTPUT );
input [1:0] I1, I2;
output [1:0] OUTPUT;
...
endmodule
```

Verilog Designs with One or More VHDL Modules

Before you compile a Verilog design that instantiates one or more VHDL modules, you must ensure that:

- The Verilog source files contain instantiations of VHDL entities/architectures. For the module type name in the instantiation, the user must escape the work library

name followed by the entity name and optionally append the architecture name in parentheses:

```
\work_lib_name.entity_name(arch_name)
```

- The following uses are also allowable as long as they are un-ambiguous and represent legal VHDL references:

```
entity_name(arch_name)  
entity_name
```

- The instance ports must have the same ordering, size and direction as ports of the corresponding VHDL entity. For example, your Verilog source files could contain the following instantiation for a VHDL module, LEAF:

```
module top_level(..., I1, I2, OUT);  
...  
input [1:0] I1, I2;  
output [1:0] OUT;  
...  
// work is the VHDL work library, LEAF the entity name,  
// and RTL is the corresponding architecture  
\work.LEAF(RTL) inst(.I1(I1), .I2(I2), .OUTPUT(O1));  
...  
endmodule
```

The VHDL source files must contain definitions of the modules. For example, your VHDL source files could contain the following definition of LEAF:

```
entity LEAF is  
    port( I1, I2 : in bit_vector(1 downto 0 );  
          OUTPUT : out bit_vector(1 downto 0 ));  
end;
```

Case Sensitivity

Case sensitivity is critical when matching design names supplied in external files like net names in SPEF files or clock files, or instance or module names in options such as -top. By default, the scenario database is constructed to be case sensitive if the design is entirely in Verilog, or case insensitive if the design is VHDL or combination of VHDL and Verilog.

If you have a mixed-language design and you make it case sensitive, all VHDL identifiers will be expected to be upper-case and Verilog identifiers will be kept as they were in the Verilog source files. In a mixed design, you might want to make it case sensitive if, for example, you happened to have two Verilog identifiers that differed only in case and you want to search for them in an application such as power reduction ([ReducePower](#)).

Compiling Mixed-Language Designs in a Single Run

Using the [Elaborate](#) command, you can compile both Verilog and VHDL source into a single scenario database file. With this method, PowerArtist searches first to match Verilog instances with a Verilog module definition, and then a VHDL entity-architecture pair. Likewise, PowerArtist will search first to match VHDL instances with a VHDL entity-architecture pair (if no configuration is specified) and then with a Verilog module.

Using a Command-Line Flow

The Elaborate command can read both a Verilog startup file (specified with the `-verilog_startup_file` option) and a script to compile VHDL source files (specified with the `-compile_script` option).

Example

```
Elaborate -top my_lib.my_design_unit -scenario_file my_chip.scn  
          -verilog_startup_file my_file.f -compile_script ptSourceFiles.tcl
```

Alternatively, you can specify Verilog files and VHDL files together using a series of `CompileFile` commands in a script passed to Elaborate using the `-compile_script` option.

Sample Script

```
CompileFile -type verilog -file my_verilog.v -2001 yes  
CompileFile -type vhd1 -library toplib -file my_top_vhdl.vhdl
```

```
AddLibrary STD  
CompileFile -type vhd1 -library STD -file standard.vhd  
AddLibrary IEEE  
CompileFile -type vhd1 -library IEEE -file std_1164.vhd  
AddLibrary WORK  
CompileFile -type vhd1 -library work -file mydesign.vhd
```

To see a complete syntax for the [CompileFile](#) command.

Using the PowerCanvas

If you are using the HDL Inferencing wizard in the PowerCanvas, on the Language setup screen be sure to check both the Verilog and VHDL boxes for a mixed-language design and continue as you would for any design. For details see, [Performing Design Inferencing Using the HDL Inferencing Wizard](#).

Creating Your Map Files

Each file that you supply to the [wvwmkr](#) utility must include the following:

- The names of your VHDL logical libraries.
- The UNIX file location of the compiled VHDL libraries and file dependency data used by your Makefiles.
- The VHDL file names to be compiled into your VHDL logical libraries.

Sample Mapfile Syntax

```
map LIB1 ./ww_lib1 \  
/home/tmiller/unit1 -i {unit1a.vhdl unit1b.vhdl} \  
/home/gramirez/stypes -i {type1.vhdl type2.vhdl}
```

This statement indicates the following:

- The keyword map indicates that a “map” statement is coming. Line continuations are indicated by a trailing backslash (\).
- LIB1 is the logical name of a VHDL library.
- LIB1 is being compiled into physical library location ./ww_lib1.
- There are two UNIX directories that contain VHDL source to be compiled into logical library LIB1: unit1 and stypes.
- For unit1, the files to be included (-i) from that directory are unit1a.vhdl and unit1b.vhdl. For stypes, the files to be included from that directory are type1.vhdl and type2.vhdl.

It is strongly recommended that you use the VHDL standard libraries supplied with the PowerArtist distribution and not the libraries provided with your simulator when running PowerArtist. Using the PowerArtist libraries ensures that the pragmas needed by the VHDL compiler have been inserted in the libraries.

The synonym Statement

If you are using ModelSim as your simulator, you may need to supply one additional map file statement. This simulator lets you create synonym libraries. Different logical library names may point to the same physical location. The synonym statement maps a new logical library name to a previously specified logical library name. For example, once IEEE appears in a map statement, it can be followed with the synonym statement:

```
synonym SYNOPSYS IEEE
```

Now SYNOPSYS and IEEE point to the same location. The order of the parameters is new name then old name.

Running the *wwvmkr* Utility

The PowerArtist [wwvmkr](#) utility reads in a map file—created either by hand, an automated process or by `make_mti_mapfile`—and builds makefiles that you use to compile your design.

Example

```
wwvmkr -m mapfile -w work
```

This command builds the Makefiles in your local working directory, following VHDL 93 rules. The top-level logical library is `work` (this is case-insensitive). You will next execute the [ptCompileScript](#), which calls the script `wwcompile` generated by your *wwvmkr* run.

Example

```
ptCompileScript
```

This will create a file called `ptSourceFiles.tcl` that you need to source before you specify the `Elaborate` command.

Contents of the *ptSourceFiles.tcl* File

The `ptSourceFiles.tcl` file is a Tcl file that contains the following sections:

- Definitions of all of the standard libraries and the source files that define them.
- `AddLibrary` commands that map logical library names to physical libraries. A typical `AddLibrary` command would be:

```
AddLibrary WORK /system/u/demouser/work
```

- `CompileFile` commands, providing file names, libraries, and VHDL standards (87/93) to apply when compiling. A typical `CompileFile` command would be:

```
CompileFile -file test.vhdl -library /system/u/demouser/work -87 yes
```

These commands are written so that they support the legacy flow of pre-compiled libraries. `CompileFile` can simply skip the physical-to-logical mapping and accept a logical library name:

```
CompileFile -file test.vhdl -library WORK -87 yes
```

Note that the library name in this case must match a logical library name previously defined in an `AddLibrary` command.

For complete syntax, see [ptSourceFiles.tcl File Format](#) in the *PowerArtist Reference Manual*.

Metacomment Processing

VHDL is a strongly typed language and, furthermore, since you can create many different types, frequent use is made of functions and procedures to provide type conversion and to overload arithmetic and Boolean operators for your defined types.

When Elaborate processes your design, it analyzes the VHDL into equivalent hardware elements ranging from simple gates to more complex blocks such as adders, memories etc. In many cases, sub-programs used to overload operators and to perform type conversions etc., contain VHDL which has no practical hardware implementation and if included in the elaboration process would impair the accuracy of the power analysis.

For example, the conversion of a `STD_LOGIC_VECTOR` to an `INTEGER` has to be performed with the `CONV_INTEGER` function to satisfy VHDL syntax and semantics, but from a hardware point of view the two types have the same representation and so the `CONV_INTEGER` function passes the `STD_LOGIC_VECTOR` argument bit for bit through to the `INTEGER` return value. To accomplish this without interfering with the VHDL simulation, use is made of special comments embedded in the VHDL source code. These special comments are called metacomments and are often of the form:

```
-- pragma keywords
```

For example, to pass a function argument of `STD_LOGIC_VECTOR` to an `INTEGER` can be achieved by including the following metacomment in the declaration section of `CONV_INTEGER`:

```
-- pragma BUILT_IN SYN_UNSIGNED_TO_INTEGER
```

When Elaborate encounters this metacomment, it passes the argument bit for bit to the return value and bypasses the body of the function. You can tell Elaborate to ignore a section of VHDL (for example, sections that might contain `TEXTIO` statements) as follows:

```
-- pragma translate_off
.. some VHDL to ignore..
-- pragma translate_on
```

For your convenience, PowerArtist can handle the class of metacomments used by Synopsys which are included in the packages they distribute. So if you use these libraries you don't need to be concerned about inserting your own metacomments to ensure that the elaboration process infers appropriate hardware from your VHDL.

Creating Custom VHDL Packages

PowerArtist supplies all the standard Synopsys and IEEE VHDL standard libraries as part of its distribution. These are all located in the following directory:

```
$POWERTHEATER_ROOT/pthdl_src
```

You might need to substitute these standard definitions with ones supplied as part of your company's standards. If you need to do this, you will have to provide the `Elaborate` command with the location of these new standard library files. There are two techniques to make this happen. The first and recommended technique requires you to modify the PowerArtist installation directory. The second requires you to compile `Elaborate` with multiple `compile_scripts`.

Technique 1: Modifying Your Installation

In the `pthdl_src` directory, you will notice two files:

- `stdlibs.87.map`
- `stdlibs.93.map`

Normally, these files are automatically read by the `wwwmkr` command depending on the language standard you are following: `stdlibs.87.map` for VHDL 87 designs or `stdlibs.93.map` for VHDL 93 designs.

These map files have identical syntax to the ones that you use to define the file/library matching for your own design. These map files define three standard libraries by default: IEEE, STD and SYNOPSYS. If you want to add or delete a file from one of these three standard libraries, just edit the appropriate map file line. For example, if you are a Synopsys user you might want to add support for Cyclone to their SYNOPSYS library for all VHDL 93 designs. In this case, you might want to modify the file in the following way:

```
map SYNOPSYS ww_synopsys $r -i syn_attr.vhd \  
    /my-cyclone-directory-path -i cyclone.vhd
```

Then run `wwwmkr` and `ptCompileScript` just like you have done in the past.

Technique 2: Multiple Compile Scripts

1. Using the `AddLibrary` and [CompileFile](#) commands, compile the base packages for your design into a script called, for example, `myBaseFiles.tcl`.

```
AddLibrary STD  
CompileFile -library STD -file mydir/standard.vhd  
AddLibrary IEEE  
CompileFile -library IEEE -file mydir/std_logic_1164.vhd
```


2. Create the compile script for the remainder of the design as follows:

```
wwvmkr          ## creates make files for VHDL design
ptCompileScript -c ## sources make files, creates ptSourceFiles.tcl
```

The `wwvmkr` run would be identical to that used by someone using the PowerArtist standard supplied libraries. When you run `ptCompileScript` with the `-c` option, PowerArtist generates a `ptSourceFiles.tcl` file that does not contain any information related to PowerArtist standard libraries. The `-c` suppresses this output.

3. Source the compile scripts and run the Elaborate command:

Example

```
source myBaseFiles.tcl
source ptSourceFiles.tcl
Elaborate
```

Each of the sourced files must follow the format for a standard compile script. The Elaborate command reads and processes the scripts in the listed order.

Consequently, due to VHDL semantic rules, any design unit you need defined must appear before it is referenced. In short, your standard libraries must get compiled before all other files and they must be in the correct order.

In the sample script in step 1, the `standard.vhd` file which is compiled into STD must be compiled before `std_logic_1164.vhd` is compiled into IEEE. Since these two files appear in `myBaseFiles.tcl` they are automatically compiled before the files defined in `ptSourceFiles.tcl`.

Keep in mind that this is a standard Tcl file. You can use standard Tcl features to locate your particular libraries using environment variables and Tcl variables. For example, the `myBaseFiles.tcl` script could be re-written as:

```
global env

set path $env(VHDL_STD_DIR)
AddLibrary STD
CompileFile -library STD -file $path/standard.vhd
AddLibrary IEEE
CompileFile -library IEEE -file $path/std_logic_1164.vhd
```

Defining Libraries for Command-Line Use

Technology-specific information is supplied using Liberty format library files. To specify these Liberty libraries, you can use one of two methods: the `-synlib_files` option or the [ReadLibrary](#) command.

You can use the `-synlib_files` option (available with several commands) to specify your Liberty library files, for example:

```
Elaborate -synlib_files {mylibrary1.lib mylibrary2.lib}  
-verilog_startup_file my.vc -scenario_file mychip.scn -top core
```

This example loads two libraries called `mylibrary1.lib` and `mylibrary2.lib`.

You can also use the [ReadLibrary](#) command to specify Liberty files, for example:

```
ReadLibrary mylibrary1.lib
```

If you specify such a large number of `ReadLibrary` commands that the command line generated to perform an elaboration, power calculation, power reduction or rewrite would exceed the Unix limits on command-line size, PowerArtist automatically generates a file called `ptshell.libs.opts` that contains the Liberty library names. This file then gets referenced using the `-i` option to various commands.

HDL Advanced Topics

This section provides information on several advanced HDL topics.

Using Power Macros

The PowerArtist elaborate process supports power macros. This allows you to replace HDL models written for simulation with more synthesis-aware and power-aware models without modifying your design source. This is useful for the following situations:

- You are using DesignWare™, which contains many models that are not synthesizable, and you want to create some models that more closely represent what would be synthesized.
- Your designers have used simulation models in their design to improve simulation performance and you wish to replace those models with versions that would normally be synthesized.

Apache provides a default set of power macro models that contain definitions to replace some of the simulation models from the base DesignWare™ library. The Elaborate command looks for these macro models in a power macro directory. The default location for this directory is

```
$POWERTHEATER_ROOT/pthdl_src/macros
```

You can provide a different directory name using the `Elaborate -macro_directories` option. This directory must contain:

- A set of Verilog source files that contain module definitions.
- A mapping file, named MacroMap, that specifies the name of each module to be replaced in the design and the name of the module with which to replace it. The name of each Verilog file should be the same as the name of the module plus the .v extension. The Verilog module must be port compatible with the module it is replacing. This also works for VHDL designs following standard conventions that ensure that a VHDL architecture can instantiate a component written in Verilog. For example, the macro directory can contain a macro map file that includes the following line:

```
originalAndGate myAndGate
```

In this example, the macro directory must also contain a Verilog file named `myAndGate.v`, which contains the module definition for the module `myAndGate`. The elaborate process will mark all occurrences of Verilog modules or VHDL entities of `originalAndGate`. It compiles `myAndGate.v` and replaces instances of `originalAndGate` with the model constructed from `myAndGate`. It also applies any parameters or generic values to this new model. The scenario database that [Elaborate](#) produces contains instances of `myAndGate` where `originalAndGate` had previously occurred.

Apache has modeled the following components that can be mapped onto the base DesignWare™ library:

- adder
- adder/subtractor
- decrementer
- incrementer
- subtractor
- absolute value
- simple multiplier
- 2-6 stage pipelined multipliers
- wallace tree multiplier
- partial multiplier

If you use DesignWare™, you do not have to do anything to take advantage of this feature. By default, PowerArtist automatically substitutes any modules it finds in the macros subdirectory. If you have created your own power macros and cannot install them into the macros subdirectory, then you must use the Elaborate `-macro_directories` option to point to the directory containing the power macros. If you use multiple directories, use a Tcl list to specify the directories, as shown in the following example:

```
Elaborate -macro_directories {dir1 dir2 dir3 ...}
```

Overriding Parameter Settings for Top-Level VHDL or Verilog Modules

The PowerArtist elaborate process includes an option you can use to assign a value to a top-level VHDL or Verilog module. The `-parameter_maps` option allows you to override parameter settings, including generics.

For example, suppose you have the following Verilog (or equivalent VHDL fragment:

```
module top(in,out);  
parameter size=2;  
input [size-1:0] in;  
output [size:0] out;  
assign out = in+1;  
endmodule
```

You could override the size parameter by adding the `-param_map` option to your Tcl command file:

```
Elaborate -param_map {size=4} ...
```

When PowerArtist elaborates the Verilog design, size would be set to 4. If you have multiple parameters you want to override, you can specify a Tcl list of parameters. For example:

```
Elaborate -param_map {p1=4 p2=5}
```

You can also use the `-param_map` option to set generics. Take, for example, the following VHDL fragment:

```
LIBRARY IEEE;  
USE IEEE.STD_LOGIC_1164.ALL;  
  
ENTITY TOP IS  
  GENERIC(SIZE : integer);  
  PORT ( AA, BB, TT : IN STD_LOGIC_VECTOR(SIZE-1 downto 0);  
        CC : OUT STD_LOGIC_VECTOR(SIZE-1 downto 0));  
END TOP;  
  
ARCHITECTURE A0 OF TOP IS  
  
BEGIN  
  CC <= BB AND TT;  
END A0;
```

Given this fragment, you could add the following lines to your Tcl command file to set the SIZE parameter:

```
Elaborate -param_map {size=4}
```

Controlling Array Inferencing

When a two-dimensional array is modeled in Verilog RTL, the array may be inferred in one of two ways: either as a single two-dimensional register file or latch file, or as a collection of individual state devices (registers and latches) with read and write address decoding logic. For large arrays, register or latch files provide the best balance of analysis accuracy versus capacity and power analysis performance. For small arrays, the PowerArtist power analyzers can better account for clock gating and other synthesis effects when inferencing maps an array to individual state devices.

The [Elaborate](#) command provides options that allow you to control array inferencing. You can use these options to tell PowerArtist to “blast” arrays into a combination of state devices and the supporting control logic rather than to infer a single register or latch file.

If you make extensive use of arrays and you plan to clock gate the registers that will ultimately be synthesized, then you should take advantage of these options to force bit blasting to occur in as many places as possible. The following options all use the name “regfile”. These options must also be used for instances that would ultimately be inferred as a latch file. There is no separate -blast_latchfile option as an example.

The options for bit-blasting are:

`-blast_regfile list | all`

Specifies either a Tcl list of 2-D arrays to bit-blast or all (all arrays will be bit-blasted).

`-preserve_regfile list | all`

Specifies either a Tcl list of 2-D arrays to be preserved or all (all arrays will be preserved).

`-min_regfile_bit_count int`

Specifies a minimum bit count (words x length) of 2-D arrays to be preserved as register files or latch files.

Default: 32

`-min_regfile_word_count int`

Specifies a minimum word count for 2-D arrays to be preserved as register files or latch files.

Default: 8

`-min_regfile_word_length int`

Specifies a minimum word length for 2-D arrays to be preserved as register files or latch files.

Default: 3

The format for 2-D arrays to be provided to either blast_regfile or preserve_regfile is:

module_name.array_name

If the 2D array is used in a named block, generate statement or other named scope in your Verilog, you must include that scope value. In this case, the name is of the form:

module_name.scope_name.array_name

Application of the Options

You need to specify these options to the Elaborate command. The options are applied by PowerArtist in the following sequence:

1. Any two dimensional arrays named with `-blast_regfile` or `-preserve_regfile` are marked to be blasted or preserved respectively.
2. If either “`-blast_regfile`” all or “`-preserve_regfile` all” is specified, then all remaining arrays are blasted or preserved as directed.
3. Otherwise, for each array, the word length, word count, and total number of bits (determined by word count times the word length) are compared with the minimum limits for register files. You may set these limits using the `-min_regfile_*` options. If any of these are lower than the specified or default limits, then the array is marked to be blasted.

Examples

The following Verilog examples show how you would translate the names in your Verilog to the names required in the bit blasting options. All of the examples show how the array names would be used in the `-preserve_regfile` option.

Example 1: Simple use

```
module top(clk, wen, wadr, wdata, radr, rdata);
    <snip> ...

    always @(posedge clk)
    begin
        reg [5:0] myRam [15:0];
        if (wen)
            myRam[wadr] = wdata;
        rdata = myRam[radr];
    end
end
```

In this case, the array name would be:

```
Elaborate -preserve_regfile top.myRam
```

Example 2: Named begin block

```
module top(clk, wen, wadr, wdata, radr, rdata);  
    <snip> ....  
  
    always @(posedge clk)  
    begin : myblock  
        reg [5:0] myRam [15:0];  
        if (wen)  
            myRam[wadr] = wdata;  
        rdata = myRam[radr];  
    end
```

In this case, the array name as specified with the `-preserve_regfile` option would be:

```
Elaborate -preserve_regfile top.myblock.myRam
```

Example 3: Nested named begin block

```
module top(clk, wen, wadr, wdata, radr, rdata);  
    <snip> ....  
  
    always @(posedge clk)  
    begin : myblock  
        begin : mysubblock  
            reg [5:0] myRam [15:0];  
            if (wen)  
                myRam[wadr] = wdata;  
            rdata = myRam[radr];  
        end  
    end
```

In this case, the array name as specified with the `-preserve_regfile` option would be:

```
Elaborate -preserve_regfile top.myblock.mysubblock.myRam
```

Example 4: Generate Statement

```
module top(clk, wen, wadr, wdata, radr, rdata);  
    <snip> ....  
  
    genvar i;  
    generate  
        for(i = 0; i < 4; i = i+1)  
        begin : blk  
  
            always @(posedge clk)  
            begin : myblock  
                begin : mysubblock  
                    reg [7:0] myRam [31:0];
```

```
        if (wen)
            myRam[wadr] = wdata[i*8+3:i*8];
            rdata[i*8+3:i*8] = myRam[radr];
        end // mysubblock
    end // myblock
end // blk
endgenerate
```

In this case, the array name as specified with the `-preserve_regfile` option would be:

`Elaborate -preserve_regfile top.blk.myblock.mysubblock.myRam`

Note that the scope name for the generate block must be specified without the `[index]` for each genvar iteration. The option applies to all generated copies of the block, so that all copies of “myRam” get preserved.

Notes, Warnings and Error Messages

With respect to register file and latch files, you will see two different Warning messages in your `Elaborate.log` file. If blasting occurs, you will see Warning 12000. An example is:

```
Warning 12000: design.v:12 3x4 memory myRam not recognized as dual-
port RAM. Will be bit-blasted.
```

If blasting does not occur, you will see Warning 2859. An example is:

```
Warning 2859: design.v:15 inferred regfile instance myblock:myRam
with data width of 6, address width 4.
```

The [Elaborate](#) command generates no Notes or Error messages specific to bit blasting. Note that in some cases, the colon “:” separator is printed for named scopes. The `Elaborate regfile` options require that scope names be specified using a dot “.” separator only.

Chapter 8

Preparing for Power Analysis

Introduction

This chapter describes the steps required before you can run power analysis or reduction. It includes information on setting net capacitance, handling designs with multiple power supplies and libraries and clock power specification.

Chapter Organization

The following topics are covered in this chapter:

- *Estimating Net Capacitances*
- *Using PACE Technology Files During Power Analysis (Beta)*
- *Estimating Pin Capacitance*
- *Handling Voltages in Liberty Format*
- *Handling Designs with Multiple Power Supplies*
- *Running RTL Mixed-Vt Power Analysis*
- *Setting up Clock Power Analysis*
- *Setting up Clock Gating for Power Analysis*
- *Setting up Clock Gating for Power Reduction*

Estimating Net Capacitances

Accurately estimating dynamic power in your design requires good estimates of your net capacitances; therefore, before you begin your power analysis, you need to determine how you will provide net capacitance information to PowerArtist. You can use any of the following methods:

- Back-annotate capacitances using SPEF. For details see, [Back-Annotating Capacitance Using SPEF](#).
- Back-annotate load capacitances for primary outputs using a PowerArtist capacitance file. You can get output load information based on the package you will use for your design. Capacitances files for this purpose should supply loads representing output load capacitances. For details, see [Using Back-Annotated Load Capacitances for Primary Outputs](#).
- Specify a default output load capacitance value using the `-default_output_load` option to [CalculatePower](#). For details see, [Specifying Default Output Load Capacitance Using a Command Option](#).
- Back-annotate capacitances for local signal nets using a PowerArtist capacitance file. RTL floorplanners may be able to generate signal capacitances. This format is most often used for RTL and mixed RTL/gate designs. If you are back-annotating signal capacitances, you can only choose one format. You cannot mix SPEF and wiring capacitances files. For details see, [Using Back-Annotated Wiring Capacitances for Local Nets](#).
- Specify a Power Artist Calibrator and Estimator (PACE) technology file (using the `-power_tech_file` option) to estimate net capacitance for RTL, mixed RTL/gate or gate-level designs. PACE files are also used for clock distribution network modeling. For details on how to use a PACE file see, [Using PACE Technology Files During Power Analysis \(Beta\)](#).
- Allow PowerArtist to use wire load models to estimate signal capacitances. This can be used for either RTL and mixed-RTL and gate or for pure gate-level netlists. You can set the wire load models using Tcl commands. For details see, [Specifying Wire Load Models](#).
- Allow PowerArtist to use default wire load models. For details see, [Using Apache Default Wire Load Models for Capacitance Analysis](#).

All nets not explicitly listed in a net capacitance file have their capacitance, length, and routing-area values estimated from capacitance models either available in your technology libraries or supplied automatically by PowerArtist. If you want no net capacitance estimation to be performed, then you must supply the “`-no_module_net_capacitances true`” option to the [CalculatePower](#) and [ReducePower](#) power commands.

If you are using one of the capacitance file formats to back-annotate capacitances onto your nets, PowerArtist performs two different checks. First, it generates a warning message for every net in your back annotation file that it can't find in the

scenario file. Second, it generates a warning message for every net in the scenario file that was not back-annotated after all back annotation files have been processed.

Back-Annotating Capacitance Using SPEF

This method is most often used for gate-level designs that have been placed and routed. Note that SPEF files can be gzipped. PowerArtist determines if a file is gzipped by examining the first bytes of the file (not by the file extension).

Note that only simple, lumped RC SPEF is supported.

If you only have one SPEF file that covers the entire design, use the `-spef_file` option with the [CalculatePower](#) command, as in the following example:

```
CalculatePower -scenario_file my_chip.scn -spef_file my_chip.spef
```

Otherwise, you will be using the hierarchical flow as described next.

Hierarchical SPEF Processing

You have the option of providing parasitic information using hierarchical SPEF files. PowerArtist supports two different flows:

- You can specify the list of SPEF files using the [SetSpefFiles](#) command and the top SPEF design name using the [SetTopSpef](#) command. The SPEF reader will then determine the hierarchy by going through the instantiations in the SPEF files (by looking at the “*DEFINE” statements), and then processing them.
- Alternatively, you can specify a hierarchy (hierarchical instance) in the design along with the SPEF file associated with it, using the [ReadParasitics](#) command.

The two flows are mutually exclusive. The SPEF reader will generate an error if the flows are mixed. To use the flow involving “*DEFINE” records, you will need to specify two separate commands: `SetSpefFiles` and `SetTopSpef`.

```
SetSpefFiles {file_name(s)}
```

This command provides the SPEF reader with all the files that you want to process (you can specify a single SPEF file). The SPEF reader performs a rapid “read” of these files to determine their associated design names. These design names must later be referenced in “*DEFINE” statements following the SPEF specification.

```
SetTopSpef top_design_name
```

This command tells the SPEF reader the name of the design that forms the root of your SPEF hierarchy. Typically, this maps to the top-level design unit in your design hierarchy. The design name is then used to find the top-level SPEF file, which is read. As the SPEF reader encounters *DEFINE statements, the other SPEF files are

located and read-in. This command is not required when you specify only one SPEF file.

Sample SetSpefFiles and SetTopSpef Commands

```
SetSpefFiles { top.spef middle.spef bottom.spef }  
SetTopSpef top
```

If you want to use a flow that follows SPEF back-annotation methodologies established by industry standard timing analysis products, then you need to include the [ReadParasitics](#) command in your PowerArtist command file (Tcl script).

```
ReadParasitics -path hierarchical_inst_name -file SPEF_file_name
```

This command associates a particular SPEF file with a specific hierarchical instance in the design. The instance name must be fully rooted (that is, it must contain the top module name). Also, in PowerArtist, periods (.) separate levels of hierarchy.

Sample ReadParasitics Commands

```
ReadParasitics -path mydesign -file ../spef/mydesign.spef  
ReadParasitics -path mydesign.mymodule0 -file ../spef/mymodule.spef  
ReadParasitics -path mydesign.myblock1 -file ../spef/myblock.spef
```

Do not use the SetSpefFiles or SetTopSpef commands if you are using the [ReadParasitics](#) command.

Handling SPEF Comments

To ignore C-style comments, you can include the [SetCCommentsIgnore](#) command. This command is identical to the -iscc option. This command indicates that C-style comments should be ignored in the SPEF file.

A sample line that contains a C-style comment is:

```
*NAME_MAP  
*1931 buffer/read_rcl/buf_rp_read/*suffix*/a
```

If you are ignoring C-style comments, the exact string is taken as the net name.

You need to be aware of the following points when using these new commands:

Important Notes

- Do not use the -spef -spef_file and any of the hierarchical SPEF commands together this will cause an error.
- This reader does an incremental back-annotation. If a net already has parasitics associated with it, the parasitics specified in the SPEF will be added to them.
- If a net specified in the SPEF file is not found in the design, a warning is reported.

Using Back-Annotated Load Capacitances for Primary Outputs

You can specify load capacitances for any primary output of the design by specifying the output pin name and capacitance for that pin. The format of the capacitance file is provided in the [Capacitance File Format](#) section of the *PowerArtist Reference Manual*. To specify a file containing load capacitances from the command line, use the `-load_file` command option as shown in the following example:

```
CalculatePower -scenario_file my_chip.scn -load_file my_chip_lds.cap
```

Specifying Default Output Load Capacitance Using a Command Option

You can specify default output load capacitance using the `-default_output_load` option to [CalculatePower](#).

```
CalculatePower -scenario_file my_chip.scn -default_output_load 3.9e-11
```

You should always specify output loads. This is especially true if you have pads instantiated in your design. If you supply a default load value using the `-default_output_load` option or have annotated your loads using the `-load_file` option, then you will not get messages about your primary output nets. Primary input nets will not be flagged as missing annotations since capacitances on primary inputs have no impact on power consumption.

Using Back-Annotated Wiring Capacitances for Local Nets

You also have the option of specifying a capacitance file that include capacitances for local nets including primary outputs. To specify a wiring capacitance file, use the `CalculatePower -capacitance_file` option.

```
CalcultePower -scenario_file my_chip.scn -capacitance_file my_chip_wirecaps.cap
```

Specifying Wire Load Models

Using Apache Tcl commands, you can set the wire load mode and models for your design that the analyzers will use for power analysis. You can specify wire load models at the following levels:

- The design level
- On an instance by instance basis hierarchically
- On a net by net basis

The following sections provide an overview of the various commands and precedence rules used to select the wire load models. You can specify these wire load models using either the [SetWireLoadMode](#), [SetWireLoadModel](#),

[SetWireLoadSelectionTable](#) and [SetCapEstimation](#) Tcl commands or the -wireload_library command-line option to [CalculatePower](#) or [ReducePower](#).

Using the Wire Load Model Tcl Commands

The following table provides a quick synopsis of the available wire load Tcl commands and the objects to which they apply:

Applicable Design Object	Tcl Command	Valid Assignment Values
Design	SetWireLoadMode	Top or Enclosed
Design, Hierarchical Module Instance or Net	SetWireLoadModel	wire_load model from library file
Design or Hierarchical Instance	SetWireLoadSelectionTable	selection table group from library file
Design or Hierarchical Instance	SetCapEstimation	wire_load model from Apache default library file

For details on how to use these Tcl commands in the command file, see [Estimating Pin Capacitance](#).

Rules for Estimating Wire Capacitance

Capacitance annotation takes precedence over capacitance estimation methods. This section discusses precedence rules for capacitance estimation only. The Liberty semantics define a hierarchy of defaulting rules that must be applied to determine how tools should estimate capacitance using wire_load models.

The software first has to determine and locate the library to be searched for wire load models. The software determines the correct library to be search by applying the following rules in the given order:

1. If a [SetWireLoadModel](#) command explicitly supplies a logical library name, use that library as the default library.
2. If there is no explicit specification of a logical library name and you have specified the -wireload_library option, use that library as the default library.
3. Otherwise, select as the default library the first library that contains wire_load models in the technology library list.

Once you have identified the library to search, then the commands themselves have a set of hierarchical rules. The rules are:

1. The wire_load mode dictates how wire_load models of children instances are assigned. If the mode is “top” assign the wire_load of the parent instance if a parent exists and no commands pertain to this instance.

2. If a `SetWireLoadModel` command has been specified for a particular instance, use that model.
3. If a `SetWireLoadSelectionTable` command has been specified, locate the `wire_load` selection table in the library, determine the area of the design, then locate the area range and then determine the `wire_load` model. If the area does not fall into a range specified by the selection table, choose the closest matching range and use that `wire_load` model.
4. If a `SetCapEstimation` command has been specified, locate the `wire_load` model in the Apache default capacitance file.
5. If no `SetWireLoadSelectionTable` command has been specified, look for the selection table specified by the `default_wire_load_selection` attribute in the default library. If such an attribute has been supplied, use the same rules as for `SetWireLoadSelectionTable` to determine the wire load model.
6. If the default library contains no `default_wire_load_selection` attribute but it does contain selection tables then:
 - start from the beginning of the library file.
 - find the first selection table that has a range with area limits that meet the area requirements.
 - if no such selection table exists, find the selection table that has the closest matching area range.
7. If the default library does not contain selection tables, then use the `wire_load` model specified by the `default_wire_load_model` attribute.
8. At this point, nothing has matched. Assume that the “`SetCapEstimation -technology 90 -scale 1`” command has been specified. Follow the `SetCapEstimation` rules to locate the `wire_load` model.

As you can see, the defaulting rules ensure that the power analyzers will always estimate capacitance for your design. When all else fails, you will get capacitance estimated using the default 90 nm technology libraries that come with the PowerArtist installation. This is discussed in the following section. If you want to suppress capacitance estimation completely, use the “`-no_module_net_capacitances true`” command option (to `CalculatePower` or `ReducePower`).

The wire load mode has default rules as well.

1. Use the mode specified by the `SetWireLoadMode` command.
2. If no such command has been specified, then use the `default_wire_load_mode` in the default library being searched for `wire_load` models.
3. If the `default_wire_load_mode` is not found, then the default “enclosed” is used.

Note

The power analyzers do not estimate the wiring capacitance of a net with no fanout. For example, a primary output has no fanout; the only pin on the net is the driver. Therefore, the estimators do not compute a wiring capacitance for it.

Using Apache Default Wire Load Models for Capacitance Analysis

Steps 4 and 8 in the previous section describe the conditions under which PowerArtist will use the default wire load library that comes with the PowerArtist installation. This section describes the reasons why you might want to use this built-in library and what it contains.

Some sub-90 nm technology libraries (and smaller) no longer contain wire load model information. Some technologists believe that wire load models are no longer effective at predicting capacitance to a high enough accuracy for them to be used for timing closure. Therefore, some companies no longer include wire load models in their Liberty files. However, wire load models still have a strong role to play in power analysis, especially at high levels of design abstraction.

If you do not have wire load models defined in your libraries and want to have wire capacitance taken into consideration during power analysis using PowerArtist, you can use the default wire load library that comes with the PowerArtist installation. This default library has models for various technology sizes. Apache generated these models based on experiences with a variety of technology sizes. It is recommended that you use these libraries rather than not consider capacitance at all.

You can find the library in `$POWERTHEATER_ROOT/sfl_lib/generic/seqcap.lib`.

The file contains `wire_load_selection` tables for 180, 150, 130, 90, 65, and 45nm technologies. To use this library for capacitance estimation, you must specify the [SetCapEstimation](#) command or have no `wire_load` model information in any of your technology libraries.

Unlike all of your other Liberty files, you do not need to specify the location of this file using the `-synlib_files` option. The analyzers automatically locate this library and insert it at the end of the library search path. Therefore, it is always available for use. This library is read-only, so you can not accidentally modify it. However, once you install the software you can modify it to create a site-specific set of technology defaults.

Using PACE Technology Files During Power Analysis (Beta)

Note on Using this Beta Feature

Before trying this beta feature/flow, please contact your Apache Design support team for assistance.

To specify a PACE technology file, you need to include the `-power_tech_file` command line option to the `CalculatePower` command in your run script. A PACE model file will contain both capacitance models and clock distribution network models. The capacitance models improve capacitance estimation accuracy. Similarly, the PACE clock distribution network models improve the modeling of distribution network topology and improve power analysis accuracy. The models apply to a pure gate level power analysis, mixed gate and RTL or pure RTL.

PACE models are generated by a CAD engineer using PowerArtist commands. For details on this process, see [Generating PACE Technology Files \(Beta\)](#) in the PowerArtist Library Developer Guide.

Using Clock Distribution Network Models

The clock distribution network PACE models are designed to replace the `SetClockBuffer` and `SetClockGatingStyle` commands as much as possible. Since they are frequency dependent, you must specify the `-frequency` option to the `SetClockNet` command when you specify a PACE model. You may also have to specify `SetAttribute` commands to enable/disable the “`dont_use`” attribute in your Liberty library to control which clock buffers or ICGCs may be chosen as a result of the electrical parameters stored in the PACE model. The following run script provides a good example:

```
SetClockNet -name clk1 -frequency 5e+08 -mode infer -clock_gate yes
SetClockNet -name clk2 -frequency 2e+08 -mode infer -clock_gate no
SetClockNet -name clk3 -frequency 1e+08 -mode infer -clock_gate yes

SetAttribute -cell CKENAIAX8 -library typical -name dont_use -value false
SetAttribute -cell BUFX16 -library typical -name dont_use -value true

CalculatePower -analysis_type average \
               -scenario_file top.scn \
               -synlib_files synlib.lib \
               -verilog_startup_file verilog_files.vc \
               -power_tech_file power.tech \
               -top top \
               -default_transition_time 64e-12 \
               -domain_frequency_cell_selection true
```

Note the following points about this run script that uses a PACE model:

- The `SetClockNet` `-frequency` option is specified for all clocks in the design.

- The [SetAttribute](#) commands in this script make certain that cell CKENAIAX8 is considered as an ICGC, while buffer BUFEX16 is not considered at all. You may also set the PT_PACE_CLK_DONT_USE environment variable to false to ignore the dont_use attribute when determining ICGCs and buffers from your Liberty libraries. The behavior is, when you specify:

```
setenv PT_PACE_CLK_DONT_USE false
```

the “dont_use : true” Liberty attributes is ignored. When you specify:

```
unsetenv PT_PACE_CLK_DONT_USE
```

the “dont_use : true” Liberty attribute is honored.

- The final line in this script is the CalculatePower specification that includes the PACE model specification by the -power_tech_file option and other required parameters. Note that you can also specify the -power_tech_file option to the ReducePower command.

Understanding Priorities and Precedence When Using PACE Models

When using a PACE model, you must recognize the following priorities:

- Manual back annotation methods take precedence over PACE models. This includes SPEF files, wire capacitance files, output load files and default output load capacitance.
- PACE models will take priority over any wire load model-based capacitance estimation technique you have applied. This includes [SetWireLoadModel](#), default wire load library lookups in Liberty files and the seqcap.lib default wire load model.

Also, it is your responsibility to determine that the PACE model is applicable to your design. At a minimum, you should ensure that the technology node is the same as the one used to create the PACE model.

Using the SetClockGatingStyle Command

You can use the [SetClockGatingStyle](#) command if you want explicit control over two parameters. First, you can specify the -min_bit_width constraint. This overrides any such value determined by the PACE model. Second, you can specify the -structure option, which has the following syntax:

```
-structure branch | leaf
```

This controls whether ICGCs directly fanout to registers (leaf) or whether ICGCs can fanout to buffers that then drive registers (branch). You can use this option to increase analysis accuracy if you understand what will happen during CTS and optimization of your design. By default, the branch or leaf value will be determined when the PACE model is created and then that value will be applied to your design.

You will see the impact of the `-structure` option in two different note messages that are printed to your log file during a power analysis:

```
NOTE 9937 Clock net <name> drives <int> enabled register bits.  
      These registers are being considered for clock gating.  
      <int> ICGCs (with fanout=<int>) will be used to clock  
      gate these.
```

```
NOTE 9944 Clock net <name> drives <int> enabled register bits.  
      <int> <branch|leaf> ICGCs (with fanout=<float>) were used to  
      clock gate <int> registers. <int> registers were left ungated.
```

All other parameters specified with the `SetClockGatingStyle` command are ignored when using a PACE model.

Additional Guidelines for Using PACE Models

PACE models affect the way some other PowerArtist commands are applied during power analysis. Keep the following guidelines in mind when writing scripts that use PACE models:

- Do not use the `SetClockBuffer` command with a PACE model. This command is ignored completely. You will see warning 9932 if `SetClockBuffer` is specified along with a PACE model file when performing a power analysis:

```
WARNING 9932: All SetClockBuffer command(s) in clock file foo.clk are  
             being ignored. Since clock power estimation is performed  
             using PACE file (aka power_tech_file).
```

- Do not specify the `SetCellDefaultFanout` command as it will have no effect when used with PACE models.
- Do not specify arc-based estimation (`"pt_set arc_based_estimation true"`) when using PACE models—you need to use the default pin-based estimation.

Understanding Output File Changes due to PACE Models

In addition to PACE-related log file messages, you will see additions to the report files due to using a PACE model. For example, if buffers were identified using a PACE model, the report file will have PACE explicitly identified, as in the following sample output:

```
PACE Inferred Buffer Tree:  
  Net name      : top.clk_2_0  
  Driver instance: top.U_2_0  
  Frequency     : 100MHz  
  Number of Loads: 16  
  PACE Root Driver :
```

```
Cell      : BUFX12
Library   : typical
Count     : 1
Cell maximum fanout :      8
Buffer power: static      1pW; dynamic      32.5uW
Fanout capacitance: wire    217fF; pins    17.4fF

PACE Branch Driver :
Cell      : BUFX12
Library   : typical
Count     : 2
Cell maximum fanout :      8
Buffer power: static      2pW; dynamic      65uW
Fanout capacitance: wire    434fF; pins    34.9fF

PACE Leaf Driver   :
Cell      : BUFX12
Library   : typical
Count     : 4
Cell maximum fanout :      8
Buffer power: static      4pW; dynamic     130uW
Fanout capacitance: wire    868fF; pins    69.8fF
```

Estimating Pin Capacitance

PowerArtist recognizes the following .lib attributes, which are used to calculate pin capacitance.

```
rise_capacitance_range (Min_rise, Max_rise)
```

```
fall_capacitance_range (Min_fall, Max_fall)
```

When you specify the `-interpret_pin_caps_as` option to the `CalculatePower/ReducePower` commands, PowerArtist uses the `rise_capacitance_range` and `fall_capacitance_range` attributes to calculate pin capacitances. The `-interpret_pin_caps` option accepts the following values: min, max and avg.

```
CalculatePower -scenario_file my_chip.scn -interpret_pin_caps min
```

The actual pin capacitance values are calculated in the following manner:

- min: pin capacitance = $(\text{Min_rise} + \text{Min_fall}) / 2$
- max: pin capacitance = $(\text{Max_rise} + \text{Max_fall}) / 2$
- avg: pin capacitance = $(\text{Min_rise} + \text{Max_rise} + \text{Min_fall} + \text{Max_fall}) / 4$

The value you select will determine the manner in which PowerArtist calculates pin capacitance. PowerArtist calculates pin capacitance using the following algorithm:

1. If `rise_capacitance_range` and `fall_capacitance_range` are present then calculate it using the min, max or avg algorithm

2. Otherwise use the value from the capacitance pin-level attribute
3. Otherwise use the rise_capacitance and fall_capacitance attribute values
4. Otherwise set the pin capacitance to 0

If the -interpret_pin_caps_as option is not specified, then PowerArtist calculates pin capacitance using the following algorithm:

1. Use the value from the capacitance pin-level attribute if present
2. Otherwise use the rise_capacitance and fall_capacitance attribute values
3. Otherwise use the rise_capacitance_range and fall_capacitance_range attribute values following the “avg” algorithm.
4. Otherwise set the pin capacitance to 0

In the PowerCanvas, you can control pin capacitance using the **Rise/Fall Capacitance Range Interpretation** field in the Advanced Options tab of the “Power analysis options” page of the Power Reduction wizard.

Handling Voltages in Liberty Format

Normally, the characterization voltage is taken from the nom_voltage or from the default_power_rail attribute within the power_supply group. The estimation voltage generally comes from the voltage set in the default_operating_conditions.

If no nom_voltage or default_power_rail values are supplied, the characterization voltage will come from values within the default operating conditions. If no default_operating_conditions are supplied, the estimation voltage comes from the nom_voltage or the default_power_rail attribute within the power_supply group.

To get the dynamic power, PowerArtist derates the dynamic energy number by the square of the estimation voltage divided by square of the characterization voltage. For details about characterization and estimation voltages, see [Liberty Power Supply Support](#).

Any voltage you supply using the -voltage option to ReducePower will become the estimation voltage for the entire design. Using -voltage is an obsolete technique that has been replaced with the [CreateVirtualSupply](#) command. If the estimation voltage does not equal the characterization voltage, voltage derating occurs.

Handling Designs with Multiple Libraries

If you have a hierarchical design, a Liberty library for one portion of the hierarchy might not be suitable for a different portion of the hierarchy. In general, you might want to do this because you are designing with multiple voltage domains or power domains. More specifically, you might want to do this because:

- You have different operating conditions that you want to specify for different modules.

- You have a different set of cells that you want to be considered for default cell selection for RTL power analysis.
- You synthesized your design to gates and used different libraries for various blocks in your design.

Choices like these impact the power analysis of your chip. In PowerArtist, you can assign Liberty libraries to hierarchical instances with libraries further down in the hierarchy, overriding those higher in the hierarchy. All children of the hierarchical instance inherit the library of the parent unless you specifically assign them their own power library. You will control these choices using the [SetLibrary](#) command.

Handling Designs with Multiple Power Supplies

Many designs use more than one power supply (or voltage rail). Common examples include:

- A design with two voltage islands: one supply for the core of the chip and the other for the I/Os.
- A design that uses power gating. The supplies to the power domains will need to be explicitly turned on and off due to the use of sleep signals to put various power domains of your chip in a stand by mode.

This section describes the ways in which PowerArtist Analysis supports designs with more than one power supply. In addition, this section describes the special support requirements for designs that require more than one power library (common in hierarchical designs).

PowerArtist provides general support for multiple power supplies by:

- Allowing you to define additional power supplies, which are referred to as virtual supplies.
- Allowing you to set virtual power supplies on an instance by instance basis.

Note that any changes to the power supply configurations should be completed after generating a scenario file and before running any of the Power Wizards (power analysis).

Creating a Virtual Supply

Creating virtual supplies is most useful for power gating applications. In this application, you can define one virtual supply per library supply rail, per power domain. This manner of creating virtual supplies supports the specification of on/off conditions for power gating applications. To create a virtual supply, you will use the [CreateVirtualSupply](#) command.

For voltage domain applications, you will at a minimum need the [SetLibrary](#) command. You might also use the [CreateVirtualSupply](#) command to create virtual supplies with different voltages than those defined as the default operating condition in the library. If you provide a voltage, it becomes the estimation voltage and then the library's characterization voltage will be derated using the new estimation voltage value. That new voltage will then be used to compute new energy and power numbers. This flow incorporates the standard PowerArtist derating technique.

Assigning a Virtual Supply to a Hierarchical Instance

If you have a mixed-voltage design, you might want to explore the effects of changing your voltage values. If your design is organized so that an entire hierarchical sub-section of your design is at the same voltage, you can set the correct values using the [CreateDomain](#) command.

Liberty Power Supply Support

For a library with `nom_voltage` and operating conditions, the characterization voltage is considered as the `nom_voltage` and the estimation voltage is picked up from the default operating conditions.

Similarly, if a library defines multiple supplies using the power supply attribute as

```
nom_voltage : 1;
power_supply() {
    default_power_rail : VDDlow;
    power_rail(VDDhigh, 1.32) ;
    power_rail(VDDlow, 1.32) ;
    power_rail(VSShigh, 0.00) ;
    power_rail(VSSlow, 0.00) ;
}
operating_conditions("BEST"){
    process : 0.70;
    temperature : 110;
    tree_type : balanced_tree;
    power_rail(VDDhigh, 1.00) ;
    power_rail(VDDlow, 1.00) ;
    power_rail(VSShigh, 0.00) ;
    power_rail(VSSlow, 0.00) ;
}
default_operating_conditions : "Best";
```

then by the fact that the `default_operating_conditions` have selected "Best", the estimation voltages for the supplies are taken as:

```
VDDhigh - 1V
VDDlow - 1V
VSShigh - 0V
VSSlow - 0V
```

and the power numbers are derated accordingly, based on the supply voltages defined in the `power_supply` construct. To get the dynamic power, PowerArtist derates the dynamic energy number by the square of the estimation voltage divided by square of the characterization voltage. Similarly, PowerArtist derates leakage power by the estimation voltage divided by the characterization voltage.

The Liberty `rail_connection` attribute is used to specify the power rails to which a cell is tied. Power for an instance of the cell will be derived from those power rails only.

Running RTL Mixed-Vt Power Analysis

Using cells characterized for different threshold voltages (Vt) is a critical way to control leakage power in low-power designs. Many gate-level synthesis and physical design tools have the ability to optimize your design to reduce your leakage power by replacing regular threshold voltages cells with cells that have a higher threshold voltage. This optimization is typically done to cells on paths that have positive timing slack.

To perform this optimization you must either have multiple libraries characterized at a single threshold level or libraries characterized for multiple thresholds. Some tools, such as PowerCompiler™, suggest that you get superior optimization results using libraries characterized for multiple thresholds. PowerArtist allows you to perform power analysis that takes mixed-Vt libraries into consideration and supports both library methodologies.

Critical Liberty Leakage Attributes

Synopsys' Library Compiler™ supports two attributes related to threshold voltage:

- At the library level:

```
default_threshold_voltage_group : "string" ;
```

- At the cell level:

```
threshold_voltage_group : "string" ;
```

Your libraries should be defined with these attributes; PowerArtist uses the supplied strings to differentiate threshold voltages for mixed-VT power analysis.

One use of these attributes in a library could be:

```
default_threshold_voltage_group : "HVT" ;
```

For this example, assume that the library provider has established conventions where HVT implies high threshold voltage devices and LVT implies low threshold voltage devices. In this case, the following example implies that unless a particular cell has a threshold_voltage_group that cell is a high Vt device. Another example for a mult-Vt library would be to have multiple cells with different threshold_voltage_group attributes:

```
cell (NAND2_HVT) {  
    threshold_voltage_group : "HVT";  
    ...  
}  
  
cell (NAND2_LVT) {  
    threshold_voltage_group : "LVT";  
    ...  
}
```

PowerArtist allows you to use Tcl commands to supply these attributes if they are missing from your libraries using the [SetVoltageThreshold](#) command (see [Categorizing Cells for Multiple Vts](#) next).

The [SetVT](#) command is used to assign different thresholds to hierarchical instances in your design. The methodology is based on the assumption that most customers have an idea of the typical spread between different Vt points in their modules. An example command is:

```
SetVT -mode percentage -instance {top.block1 top.block2} -vt_group {HVT:70 LVT:30}
```

In this example, all of the inferred elements that are children of top.block1 and top.block2 will have their default cells chosen from the libraries that are to be used for power analysis such that 70% of the default cells have a vt_group with the value HVT and 30% have the value LVT. The standard PowerCompiler defaulting is used; if the threshold_voltage_group is not found in a cell, then the default_threshold_voltage_group value is used. Instances that are not children of top.block1 and top.block2 will be assigned default cells without any consideration for the threshold voltage attributes.

It is possible to assign the SetVT command hierarchically. For instance, if one of the instances is top.block1.child1, this will override the percentage values set by the top.block1 SetVT command.

Categorizing Cells for Multiple Vts

If your libraries do not categorize cells using Liberty threshold voltage attributes, you need to categorize cells for different threshold voltages using PowerArtist's [SetVoltageThreshold](#) command.

```
SetVoltageThreshold -group LVT -pattern { *_TL1 *_TL2 }  
SetVoltageThreshold -group HVT -pattern { *_TH }
```

Given these sample settings, any cell names matching *_TL1 or *_TL2 will have their threshold_voltage_group value set to LVT. Additionally, any cell names matching *_TH will have their threshold voltage string set to HVT. The wild cards supported are the typical glob-style wild cards.

The SetVT example, in the previous section, assumes that the supplied libraries categorize the cells based on the Liberty threshold voltage attributes. If they are not categorized this way, you must use the [SetVoltageThreshold](#) command in addition to the SetVT command.

```
SetVoltageThreshold -group LVT -pattern { *_TL }  
SetVoltageThreshold -group HVT -pattern { *_TH }  
SetVT -mode percentage -instance {top.block1 top.block2} -vt_group {HVT:70 LVT:30}
```

Given these settings, any cells from the supplied libraries that are available for top.block1 and top.block2 and have names that match the pattern *_TL will be treated as part of the LVT threshold voltage group. Cells that match the pattern *_TH will be treated as part of the HVT threshold voltage group.

Default Cell Selection for Mixed-Vt Analysis

If you do not perform mixed-Vt analysis using the [SetVT](#) command, for each of the default cells, PowerArtist automatically chooses three cells from your .libs that can drive low, medium, and high loads.

With mixed-Vt analysis, PowerArtist selects three additional default cells for each cell type for each threshold group. This happens even if you specify only one mixed-Vt library. For example, suppose you have the following sequence of commands:

```
SetLibrary -instance top.block1 -library mixed-vt
SetVT -instance top.block1 -vt_group {HVT:50 REG:20 LVT:30}
```

Given these settings, for each of the default cells, PowerArtist searches for three candidate cells for each of the three voltage groups in the library “mixed-vt”.

Sample Mixed-Vt Flow and Tcl File

To prepare for mixed-vt power analysis, first create a file called, for example, mixed_vt.tcl.

```
# Mixed-Vt Lib and Threshold Settings#
SetLibrary -instance {top} -library {scmetro_cmos10lp_hvt_ff_1p1v_125c \
scmetro_cmos10lp_lvt_ff_1p32v_125c}
SetVoltageThreshold -group LOW_VT -pattern {*L}
SetVoltageThreshold -group HIGH_VT -pattern {*H}

##Mixed-Vt Specific Settings##
SetVT -mode percentage -instance {top} -vt_group {LOW_VT:30 HIGH_VT:70}
```

As an example of a complex Tcl file containing a variety of command file commands, consider the following case. If you want to perform power gating with mixed Vt analysis, combine the power gating related commands with the mixed-Vt commands. For this example, you could simply append the following commands after the commands in the previous section in the mixed_vt.tcl file.

```
#####
# Power Gating Commands
# rx_rq and tx_rq are sleep signals
CreateVirtualSupply -supply vdd -virtual_supply VDDRX -on top.rx_rq
CreateVirtualSupply -supply vdd -virtual_supply VDDTX -on top.tx_rq
CreateVirtualSupply -supply VDDNW -virtual_supply RX_VDDNWS -on top.rx_rq
CreateVirtualSupply -supply VDD -virtual_supply RX_VDDNWS -on top.rx_rq
```

```
CreateVirtualSupply -supply VRET -virtual_supply RX_VRET -on (!top.rx_rq)
CreateVirtualSupply -supply VDDNW -virtual_supply TX_VDDNWS -on top.tx_rq
CreateVirtualSupply -supply VDD -virtual_supply TX_VDDNWS -on top.tx_rq
CreateVirtualSupply -supply VRET -virtual_supply TX_VRET -on (!top.tx_rq)
CreateDomain -instance top.core1.r1 -virtual_supply {VDDR RX_VDDNWS RX_VRET}
CreateDomain -instance top.core1.t1 -virtual_supply {VDDTX TX_VDDNWS TX_VRET}
MapRetentionCell -instance {top.core1.r1} -attribute CK_LOW
MapRetentionCell -instance {top.core1.t1} -attribute CK_LOW -tag NoMap -notag true
```

Understanding Mixed-Vt Analysis Results in the Report File

PowerArtist will generate a section in the .rpt file when you run a power analysis or reduction and specify the SetVT command. For every hierarchical instance specified in the SetVT command, the report will include the total number of default cells selected for each of the specified Vt values.

Sample Report Section for Mixed-Vt Analysis

Mixed-VT Cells Distribution			
=====			
Hier-Instance Name	VT Group Name	Specified Percentage	Number of Cells Selected
-----	-----	-----	-----
top.block1			
	HVT	50	40
	REG	30	24
	LVT	20	16

Total			80

top.block2			
	HVT	50	20
	REG	30	12
	LVT	20	8

Total			40

If a particular default cell type could not be found from all of the specified threshold_groups in the SetVT command, then default cells of that type will be chosen independent of those threshold_groups, from the libraries that are set on that hierarchical instance. The total number of those default cells selected will be shown in an additional row where “Other VTs” is specified under the “VT Group Name” column. For example, suppose you specified the following SetVT command:

```
SetVT -mode percentage -instance {top.block1} -vt_group {HVT:90 LVT:10}
```

Assume that a few default cells could not be found for threshold_groups HVT and LVT, and therefore those cells are chosen from other VTs. In this case, the following report would be generated.

Mixed Vt Analysis Report with Other VTs Section

Hier-Instance Name	VT Group Name	Specified Percentage	Number of Cells Selected
top.block1			
	HVT	90	110
	LVT	10	13
	Other VTs	0	7
Total			130

Setting up Clock Power Analysis

You can set-up PowerArtist to perform clock power analysis. Clock power has two primary components:

- Power consumed by the clock buffer (internal cell power) driving the capacitive load
- Power due to the capacitive load

The capacitive load has two primary components:

- Load due to the cumulative input capacitance of the inputs of all clock loads
- Capacitance due to the wiring on the clock line

$$P_{clk} = (C_{fanout} + C_{wire})V^2f_{clk}$$

The C_{fanout} value is determined by analyzing the HDL code, coupled with input capacitance information from the technology file. The wiring capacitance (C_{wire}) is based on an estimate of the amount of wiring required to distribute the clock. C_{wire} therefore depends on the fanout of the net and is calculated from a wire load model in the specified libraries. You can either specify this wire load model for a clock net or allow PowerArtist to do the selection based on standard wire model selection rules.

Commands for Clock Power Analysis

PowerArtist provides a set of commands you can use to specify the clock net in the design and provide information on any clock buffers for which a clock tree is to be inferred. To enable clock power analysis, you must include the appropriate commands in your PowerArtist command file.

The two commands that you can use to define your clock net are: SetClockNet and SetClockBuffer (optional). The SetClockNet command specifies a clock net in the design and is required. The SetClockBuffer command specifies clock buffers for

which you want PowerArtist to infer a clock tree¹. If you set SetClockNet -mode option to “trace” the SetClockBuffer command is optional; it is required for inferred clock trees.

Sample Usage

Assume a scenario whereby you want to trace a clock named “tclk” which has special circuitry beginning with instance chip_top.mux21-a that you do not want to include in the power analysis. Also assume that the wire load model WL_05x5 should be applied to chip_top.tclk and all of the sub-nets that make up this traced clock net. Given these parameters, you would specify the following SetClockNet command in your command file:

```
SetClockNet -name chip_top.tclk -mode trace -stop_at_instance chip_top.mux21-a
```

Furthermore, if you want to infer a clock net, pciclock, you would also include a section similar to the following:

```
SetClockNet -name chip_top.pciclock -mode infer
SetClockBuffer -type root -name clkr -library lib1 -fanout 12
SetClockBuffer -type branch -name clkbr -library lib1 -fanout 10
SetClockBuffer -type leaf -name clkbl -library lib1 -fanout 8
```

If you want to set a wire load model to be used for the clock net defined, you can specify the SetNetProperty command for example:

```
SetNetProperty chip_top.tclk wireload_model WL05x5
```

How Clock Power Analysis Works

For clock power analysis on a net driving a large number of clock pins, PowerArtist first determines pin load due to all clock pins on the net, as well as capacitance due to wiring needed to drive all the clock pins.

If there is an instance driving the clock net, PowerArtist determines the maximum fanout for the instance output or the maximum capacitive load that output can drive.

For a gate-level instance, this information is obtained from the max_fanout or max_capacitance attributes for the cell output pin represented in Liberty format. If these attributes are not specified in the library, then you can use the [SetCellDefaultFanout](#) or the [SetDefaultFanout](#) command to specify the default fanout.

For an RTL instance, PowerArtist searches the output of the default NAND cell for max_fanout/max_capacitance information, which is then used on the instance

1. Tree is the only topology PowerArtist supports.

output. To specify the default fanout for an RTL instance, you have to use the `SetCellDefaultFanout` command on the default NAND cell.

Since the `SetCellDefaultFanout` command never overrides the value for `max_fanout` in your library, if you are performing an RTL power analysis, you could do either of the following:

- Find all cells that are two-input NANDs and using wild cards in one or more commands, set the defaults for just NAND gates.
- Do a complete wild card specification on cells and libraries. For example:

```
SetCellDefaultFanout -cell * -library * -fanout value
```

If there are no drivers for the net (that is, the clock net is a primary input) clock buffers are always inserted. Using the specified buffers and their fanouts, PowerArtist synthesizes a clock tree using the following process:

1. It calculates the number of leaf-level buffers to be inserted based on the number of clock pins on the net, and the specified fanout for the leaf buffer.
2. It checks the branch fanout to determine how many branch buffers are needed and then inserts branch-level buffers to drive the leaf buffers.

Using this method, PowerArtist continues to insert branch buffers, level after level, until a stage is reached in which a single root buffer can drive all of the next branch-level buffers.

Example

If there are 100 pins on a primary clock net (no driver) and the leaf buffer fanout is 4, then 25 leaf buffers are inserted at the leaf level. If the branch driver fanout is 10 and they have to drive 25 leaf buffers, 3 branch buffers are inserted. One root driver is then added to drive the three branch buffers.

The wiring capacitance from the root buffer to all leaf buffers is calculated using the wire load model for the clock net on which the buffers are inserted.

Controlling Forward Clock Tracing

PowerArtist performs forward clock tracing to isolate an instantiated clock path in a design and create a separate report showing the power consumed by the clock path. PowerArtist starts tracing forward from a net specified with the `SetClockNet` command in the clock nets file. Tracing stops when any of the following are encountered:

- A clock pin of a sequential device
- A primary output of the design
- Blackboxes

You can also control when the tracing stops by specifying either the `-stop_at_instance` or `-stop_at_cell` option for a given clock (for example, `chiptop.CLKIN`).

Stop at Instance Example

```
SetClockNet -name chiptop.CLKIN -mode infer -gate_clock yes  
  -stop_at_instance /  
  { chiptop.corepinmux.coretop.pbustop.syscfg_if.SYSPLL }
```

Given this example, PowerArtist stop tracing the CLKIN net when it reaches the specified instance.

If the cell type is PLL, you could use the following setting:

Stop at Cell Example

```
SetClockNet -name chiptop.CLKIN -mode infer -gate_clock yes  
  -stop_at_cell { PLL }
```

Tracing will continue through any instances specified with the [TraceThruInstance](#) or [TraceThruCell](#) commands.

Supported Timing Arcs

The following combinational timing arcs are supported:

- combinational
- combinational_rise
- combinational_fall
- three_state_disable
- three_state_disable_rise
- three_state_disable_fall
- three_state_enable
- three_state_enable_rise
- three_state_enable_fall

The following sequential timing arcs are supported:

- rising_edge
- falling_edge

RTL Designs without Timing Arcs

Tracing for RTL designs whose libraries do not contain any timing arcs is affected by the number of output pins on an instance: if there are three or more output pins, clock tracing will stop at that point. The inferred model types: decoder, comparator, adder, tri-state driver, unencoded_mux, regfile, latchfile, or multiplier will cause tracing to stop. When tracing fails, buffers leading up to the failure point will not be included in the clock report.

Estimating Instantiated Clock Tree Capacitance

If you have back annotated capacitances for your clock nets, PowerArtist will use those values; otherwise, it uses wire load models for all wires up to and including the leaf buffers. A wire load model for a net in the clock tree is selected based on the following rules:

1. If you supplied a wire load model for the clock net, that wire load model is used for the clock net as well as all of the sub-nets in the clock tree. (Sub-nets are nets between clock buffers in the clock tree.)
2. If you supplied a wire load model for any of the sub-nets, that wire load model is used for estimating the capacitance of the sub-net.
3. If no wire load model was supplied, PowerArtist uses standard area-based wire load model rules to determine the wire load model, as is done for any net in the design, to determine the wire load model assigned to the net.

It then uses the heuristic algorithm to estimate the capacitances of wires between the leaf buffers and the clock pins of sequential devices by inferring clock trees between them if needed.

Setting up Clock Gating for Power Analysis

PowerArtist allows you to select the conditions under which clock gating should be performed to conserve power. PowerArtist will take this information into consideration and perform clock gating during power analysis. The power report generated at the end of the analysis will include information on clock gating, specifying which registers were gated and providing details about which clock gating cells were used along with the resulting power numbers.

The key features of PowerArtist clock gating include:

- Clock gating is performed using integrated clock gating cells that are defined using the methodology established by Synopsys™.
- You control which integrated clock cells get selected from the library, the clock nets that will be gated, and the minimum bit width required for a register bank to be clock gated.
- Generated reports that describe the clock gating performed by PowerArtist.

Clock Gating Flow for Power Analysis

The flow for performing clock gating with power analysis in PowerArtist is as follows:

1. Define your integrated clock cells in your Liberty (.lib) libraries.
2. Add a section to your PowerArtist Tcl command file that indicates your clock gating decisions. For every net you want to gate, be sure to specify the -gate_clock option to the [SetClockNet](#) command:

```
SetClockNet -gate_clock yes ...
```

3. Run your PowerArtist command file to perform an average power analysis.
4. Review the generated report file (.rpt) to see the clock gating results.

See the [Running Power Analysis with Clock Gating](#) section in the tutorial for a sample power analysis with clock gating.

Performing Enhanced Clock Gating

One of the constraints for register-based clock gating is the required minimum bit width that register banks must satisfy before they will be clock gated. You set this constraint using the `-min_bit_width` option to the `SetClockGatingStyle` command. However, synthesis tools now are creating “weak” enables that extract common sub-expressions from enable expressions that don’t meet the minimum bit width constraint. For example, examine the following RTL:

```
always @(posedge clk)
    if (en1 and en2)
        out1 = in1;

always @(posedge clk)
    if (en1 and en3)
        out2 = in2;
```

Assuming the two register assignments involve a total of 3 bits, you could actually insert an ICGC that has `en1` as the enable signal. The output clock of the ICGC then becomes the clock input to the two register banks that have feedback muxes with `en2` as one select line and `en3` as the other. Assuming that `en1` is not enabled 100% of the time, then the clock going to the registers is toggling less often, saving power.

Just as synthesis tools have constraints controlling the use of weak enables, so does PowerArtist. There are two options that control this—one to the [SetClockNet](#) command and the other to the `SetClockGatingStyle` command.

- In the [SetClockNet](#), specify the following option:

```
-enhanced_cg true | false
```

The default is `false`. If you set this to `true`, weak enables will be used when possible. This option is valid only when you have also specified “`-gate_clock yes`”. If you do not specify “`-gate_clock yes`” and you do specify “`-enhanced_cg true`”, PowerArtist will generate the following message

```
1594: "Clock net XXX will not be clock gated..."
```

- In [SetClockGatingStyle](#), you need to specify the following option:

```
-min_bit_width_ecg int
```

The combined bit width of all register banks gated with this weak enable must be greater than or equal to this value. The default is $2 * \text{-min_bit_width}$ constraint.

Changes to the Power Analysis Report and the Power Database

If enhanced clock gating is performed, you will see the following changes to the *Clock Power Consumption* section in the power analysis report:

- There will be a new sub-section titled *Integrated Clock Cell Power for Enhanced Gating*. The format of this section is identical to the existing section, but it will contain information describing the enhanced clock gating performed.
- The *Clock Gating Summary* will have the following additional line:

Number of registers enhanced gated by inferred clock gating cells.

This represents the total number of register bits gated using weak enables.

In addition, in the power database (.pdb file) any registers that have had enhanced clock gating will be marked the same as other clock gated registers.

Library Modeling of Integrated Clock Cells

PowerArtist's clock gating feature takes advantage of the modeling methodology Synopsys has already defined for integrated clock cells. PowerArtist identifies integrated clock gating cells by looking for the `clock_gating_integrated_cell` attribute in the .lib file. It is recommended that these cells also have a `max_fanout` attribute. This attribute tells the algorithm how many loads the cell can drive.

PowerArtist searches all Liberty files specified using the `-synlib_files` option for integrated clock gating cells. There is no special switch for specifying the libraries containing clock gating cells.

Setting up Clock Gating for Power Reduction

Clock gating is performed entirely by the reduction engine controlled by the [ReducePower](#) command. No other command is involved in the process. To perform clock gating, PowerArtist must be told which nets need to be gated and the type of clock gating cells to be used for the gating. The clock nets that are to be gated are not limited to a particular module hierarchy. This is because PowerArtist is not limited to synthesizing a portion of your design, but can process the entire chip as a whole.

Clock Gating Flow for Power Reduction

If you want PowerArtist to perform clock gating during power analysis, you must perform the following steps:

1. Define your integrated clock cells in your power libraries.

2. Include the following commands in your PowerArtist command file or on the command line: [SetClockNet](#) and [SetClockGatingStyle](#). The [SetClockNet](#) command defines which clock nets should be gated among other things. The [SetClockGatingStyle](#) command defines the attributes that control how clock gating is performed.
3. Run the [ReducePower](#) command to perform power reduction.

Clock Gating Algorithm

During clock gating, PowerArtist performs the following steps, in the following order.

1. It reads in the libraries specified using the `-synlib_files` option.
2. It reads in the clock-related commands from your command file.
3. It searches the libraries for all cells containing the `clock_gating_integrated_cell` attribute whose value matches that supplied by the `-clock_cell_attribute` option to the [SetClockGatingStyle](#) command.

Note that if the library does not contain output maximum fanout information, you can specify it using the [SetCellDefaultFanout](#) command in the clock nets file.

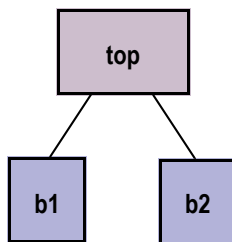
4. It identifies the elements that are suitable for clock gating. To be suitable:
 - It must be an SFL type of register. Register files cannot be clock gated.
 - The clock name must match that specified using the [SetClockNet](#) command (in the clock nets file) or is traceable from such a clock.
 - The `d_in` of the register must be the output of a 2-1 feedback mux and the `d_out` of the register must be an input to the same mux.
 - All registers sharing the same clock, enable and reset pins are gathered together. If the number of bits is greater than or equal to the `min_bit_width` value, they will be clock gated. However, if you specified a list of instances with the `-instance` or `-hierarchical` options to the [SetClockNet](#) command, only those instances will be gathered together and clock gated. For more information see, [Hierarchical Clock Gating](#).
5. It gates the selected registers using the following process:
 - It first identifies the best fitting integrated clock gating cell (ICGC) to drive the register bank. The ICGC depends on three attributes: 1) the `-gating_cells` and `-clock_cell_attribute` options to the [SetClockGatingStyle](#) command, 2) the `max_fanout` attribute in its cell description in your technology file and 3) the `-max_bit_width` option in the [SetClockGatingStyle](#) command.
 - If the `-gating_cells` or `-clock_cell_attribute` options are specified then the choice of ICGC is limited to those specified in the option values. These options establish the candidate ICGCs.

- If the `-max_bit_width` option is not specified then it identifies the best fitting candidate ICGC based on `max_fanout` to drive the total number of bits in the register bank.
 - If no ICGC is identified that can drive all the bits, PowerArtist will build a fanout tree based on the clock buffers specified in the clock nets file. Then the ICGC drives the fanout tree.
 - The load on the net driving the ICGC is reduced by the number of bits in the register bank. The load is then increased by one due to the added ICGC.
 - If the `-max_bit_width` option is specified, PowerArtist chooses an ICGC that best fits the specified value.
 - PowerArtist adds as many ICGCs in parallel that are required to drive the register bank load.
 - No buffer tree will ever get inferred in this case.
 - The load on the net driving the ICGC is reduced by the number of bits in the register bank. The load is then increased by the total number of ICGS added.
 - If your technology library uses `max_capacitance` rather than `max_fanout` for its ICGCs, then the `max_capacitance` value gets translated into a `max_fanout` value by dividing the `max_capacitance` value by the `input_capacitance` of the default flip-flop clock pin.
6. It performs an RTL average power analysis:
 - a. It determines the duty cycle of the select pin on the 2-1 mux driving the bits as the duty cycle of the enable pin on the integrated clock cell.
 - b. It uses the gate-level power model for the integrated clock cell to calculate the power consumed by the integrated clock cell.
 - c. It calculates power for the register bits that have been clock gated. The clock activity for these register bits is derated by the duty cycle for the enable pin in the clock gating cell. Likewise, the power for the feedback mux bits is reduced to zero, because these feedback components are eliminated from the circuit.
 7. It generates clock gating information in the power report (`activities.rpt`). It uses this information while performing various reductions. Clock tree and clock gating information is displayed in the PowerCanvas main window just above the tree display. If you display the clock tree, which comes up in a separate window, you can move your cursor over the different elements in the clock tree and the dynamic display in the main window will change as you move along. This will provide information such as Activity, Duty Cycle, Frequency, Net Capacitance, etc.

Hierarchical Clock Gating

This feature gives you control over where the integrated clock gating cells are to be inferred. By default, PowerArtist attempts to clock gate the entire design at one time. Any inferred register that shares the same clock net and enable signal controlling a feedback mux will be clock gated using the same integrated clock gating cell. A buffer tree then handles the potentially high fanout load.

However, if you are using a logic synthesis tool that performs clock gating and that has a capacity limitation, clock gating for the entire design cannot be implemented at one time. Instead, only the large blocks in the design that form the boundaries beyond which integrated clock gating cells cannot be shared will be clock gated. You can use the `-hierarchical` option to the [SetClockNet](#) command to overcome this particular situation. Assume the following design.



In this design, if you want to synthesize and clock gate block b2 separately from the remainder of the design, you would use the following command:

```
SetClockNet -name top.clk -hierarchical top.b2 -gate_clock yes
```

In addition, PowerArtist can perform clock gating on selective blocks in a design using the `-instance` option to the [SetClockNet](#) command.

In the same design, if you only want to clock gate block b2 but not clock gate b1, then use the following command:

```
SetClockNet -name top.clk -instance top.b2 -gate_clock yes
```

Both the `-hierarchical` and `-instance` options accept a Tcl set of instance names. At this time, they cannot accept wild cards.

Inferring Buffer Trees for Nets with High Fanout

PowerArtist can infer buffer trees for nets with high fanout. To use this feature, you will need to specify the [SetHighFanoutNet](#) and [SetBuffer](#) commands in the command file. For [SetHighFanoutNet](#), PowerArtist will infer a buffer tree for any net with at least the given number of fanouts. Use the [SetBuffer](#) command to specify the buffers used when inferring a buffer tree for these high fanout nets.

If you do not use this feature, during inferencing, you could get a tremendous capacitive load on a driving pin which would lead to the inferred element.

Chapter 9

Analyzing Simulation Activity

Introduction

PowerArtist has the ability to perform flexible activity analysis on time slices as small as a clock cycle. This is called vector analysis. Analyzing your simulation activity before you perform a power analysis allows you to:

- Determine if your testbench exercises your design correctly. A testbench for average power differs from a testbench for peak power.
- Potentially detect functional flaws that are difficult to see by looking at simple waveforms. For example, if your design should be in an “idle mode” and yet you see significant activity, this might signal a design error.

The primary method PowerArtist uses to achieve these goals is to provide you with a graph of activity over time. PowerArtist calculates activity data using the following equation:

$$\begin{aligned}\text{Activity} &= \frac{(\text{toggle_counts_of_all_nets}/\text{number_of_nets})}{\text{number_of_clock_cycles}} \\ &= \frac{(\text{toggle_counts_of_all_nets}/\text{number_of_nets})}{(\text{toggles_on_clock_signal}/2)}\end{aligned}$$

You run each simulation once while collecting activity data. Each run provides a graph for a specific slice of the simulation time, showing the activity for selected portions of the design on the Y axis and time on the X axis. The portions of the design are called groups.

Each group corresponds to one line waveform on the graph, and can consist of any number of hierarchical instances of your design. The activity analysis includes the children of the defined instance. It averages the activities of all nets in the design below the top level (as defined in the group). While activity is only an indicator of power, displaying the average activity for each of the top-level blocks highlights test

suites that fail to access all blocks on the chip at once, resulting in artificially low power results.

The term “activity” is used to describe the average activity of named nets in the group. Named nets are nets that have been explicitly specified in a simulation and appear in a simulation dump, which would include the boundary pins.

PowerArtist’s vector analysis tool has two modes of operation: simulation time-based and system clock-based. In simulation time-based mode, the average activity of the group is graphed over time. In system clock-based mode, the ratio of the average frequency of the group to that of the clock is reported.

Chapter Organization

The following topics are covered in this chapter:

- *Defining Different Groups at Each Design Phase*
- *Understanding the Design Flow with Vector Analysis*
- *Creating Analysis Graphs*

Defining Different Groups at Each Design Phase

As described in the introduction, vector analysis allows you to define arbitrary groups of hierarchical instances in the chip. PowerArtist computes the activity for each group, and displays the groups as individual lines in a graph. During different design phases, you might want to define different kinds of groups:

- System design: you may want to know the activity for all the memories on the chip, for overall clock activity or for overall I/O pad activity.
- RTL design: you may want to define each top-level block as its own group. In this way, the tool can display graphs of activity over time for each block. You can raise the activity coverage of your RTL test suite by making sure that the simulation patterns exercise all the blocks at once. You should also capture the activity of the primary inputs of your chip. You can compare these against similar activity graphs from your gate-level design to make sure you have the same testbench driving both. This is a useful check if you are doing RTL-to-gate correlation studies or comparisons.
- Gate-level design: define activities on the primary inputs of your chip. Verify that the activity is the same at the gate level as what you had seen at the RT level.

Understanding the Design Flow with Vector Analysis

The vector analysis tool fits seamlessly into your existing RTL design flow, which would consist of the following steps:

1. Simulate your design to create a simulation activity file in any of the supported formats.
2. Elaborate the design to create a PowerArtist scenario file.
3. Perform vector analysis to create analysis graphs of activity over time for specific groups of instances. For details see, [Creating Analysis Graphs](#).
4. Use the Apache Waveform Viewer to display the activity graphs.

At this point, you may want to use the tool's feedback to improve the simulation test suite for better coverage of high-activity conditions. See [Creating Analysis Graphs](#) for more information.

5. Once you have confidence that your testbenches exercise the design completely, you can use additional tools to find power peaks and graph power over time. See [Analyzing Time-Based Power](#) for additional information.

Creating Analysis Graphs

Before using the analysis tools, you should verify that your testbenches have good power coverage. Often, testbenches are designed for functional verification or fault coverage, and might not represent typical operation or worst-case power situations. PowerArtist's vector analysis tool can show you portions of the design where your testbenches do not produce enough activity. You can tune your testbenches, by adding tests or exercising more blocks in each test, so that your testbenches adequately cover all situations. If you don't, you risk overlooking real power problems.

Before you begin to find low-activity modules, you should already have a PowerArtist scenario file and a simulation file. See [Acquiring Simulation Data](#) for information on how to capture your simulation data. Once you are ready, you can run vector analysis using either a Tcl scripted flow with command files or using the PowerCanvas Vector Analysis wizard. The command-based flow is documented here. For information on using the wizard flow, see [Running Vector Analysis Using the Vector Wizard](#) in the tutorial.

Determining the Type of Vector Analysis to Run

There are two different types of vector analysis: *clock-cycle* mode and *time-based* mode. The type of analysis is determined by the value of the following option:

```
-activity_waveform_graph_type activity_per_cycle /  
    frequency_per_interval
```

Choose `activity_per_cycle` for clock-cycle mode or `frequency_per_interval` for time-based mode. When you perform a vector analysis, you should be aligning your intervals so that the dominant clock edge of your choice is at the first interval start time and that each interval is some multiple of your clock period. If you choose the `activity_per_cycle` mode this happens automatically for you. If you choose the `frequency_per_interval` mode, you have to choose the interval start time and the interval size carefully. The benefit of the `frequency_per_interval` mode is that it offers you more flexibility if you don't want to align your intervals with a clock edge.

Running Vector Analysis Using Command Files

Use the following process to run vector analysis and create your analysis graphs using a command file (Tcl script):

1. Specify the [DefineGroup](#) command to define groups of hierarchical instances in the chip, for example:

```
DefineGroup top "top"  
DefineGroup core "top.core1"  
DefineGroup pci "top.core1.p1"  
DefineGroup rxchan "top.core1.r1"
```

This example defines four groups named `top`, `core`, `pci` and `rxchan` that will have activity graphs generated for their associated instances.

2. Specify the [GenerateActivityWaveforms](#) command to run the vector analysis. You will use different command options depending on the type of vector analysis you want to run.

— Apply the following syntax to run a clock-cycle based activity analysis:

```
GenerateActivityWaveforms -activity_file file_name  
    -scenario_file file_name -top_instance inst_name  
    -activity_waveform_clock_name clock_name  
    -activity_waveform_clock_edge (pos | neg | auto)  
    -activity_waveform_cycles_per_interval int  
    -activity_waveform_graph_type activity_per_cycle  
    -activity_waveform_group_list group_list  
    -activity_waveform_log file_name  
    -activity_waveform_number_of_intervals int | all  
    -activity_waveform_start_clock_cycle int  
    -ptcl_output_file | -fsdb_output_file  
    -use_rtl_sim_data true | false
```

— Apply the following syntax to run a time-based activity analysis:

```
GenerateActivityWaveforms -activity_file file_name  
    -scenario_file file_name -top_instance inst_name
```

```
-activity_waveform_graph_type frequency_per_interval  
-activity_waveform_group_list group_list  
-activity_waveform_interval_size time  
-activity_waveform_log file_name  
-activity_waveform_number_of_intervals int  
-activity_waveform_start_time time  
-ptcl_output_file | -fsdb_output_file  
-use_rtl_sim_data true | false
```

The following Tcl command file shows a typical set up for a time-based vector analysis.

```
GenerateActivityWaveforms \  
-activity_file activities.iaf \  
-scenario_file txrx.scn \  
-top_instance top_instance txrx_tst.top1 \  
-activity_waveform_graph_type frequency_per_interval \  
-activity_waveform_group_list {top core pci rxchan} \  
-activity_waveform_start_time 100us \  
-activity_waveform_interval_size 1ns \  
-activity_waveform_number_of_intervals 1000 \  
-fsdb_output_file ${topModule}_vectors_full.fsdb \  
-activity_waveform_log Waveform.activity_vw.log \  

```

3. View the resulting FSDB or PTCL file in a waveform viewer. You can use the Apache Waveform Viewer (or the Verdi™ product from SpringSoft). A sample waveform is shown here:

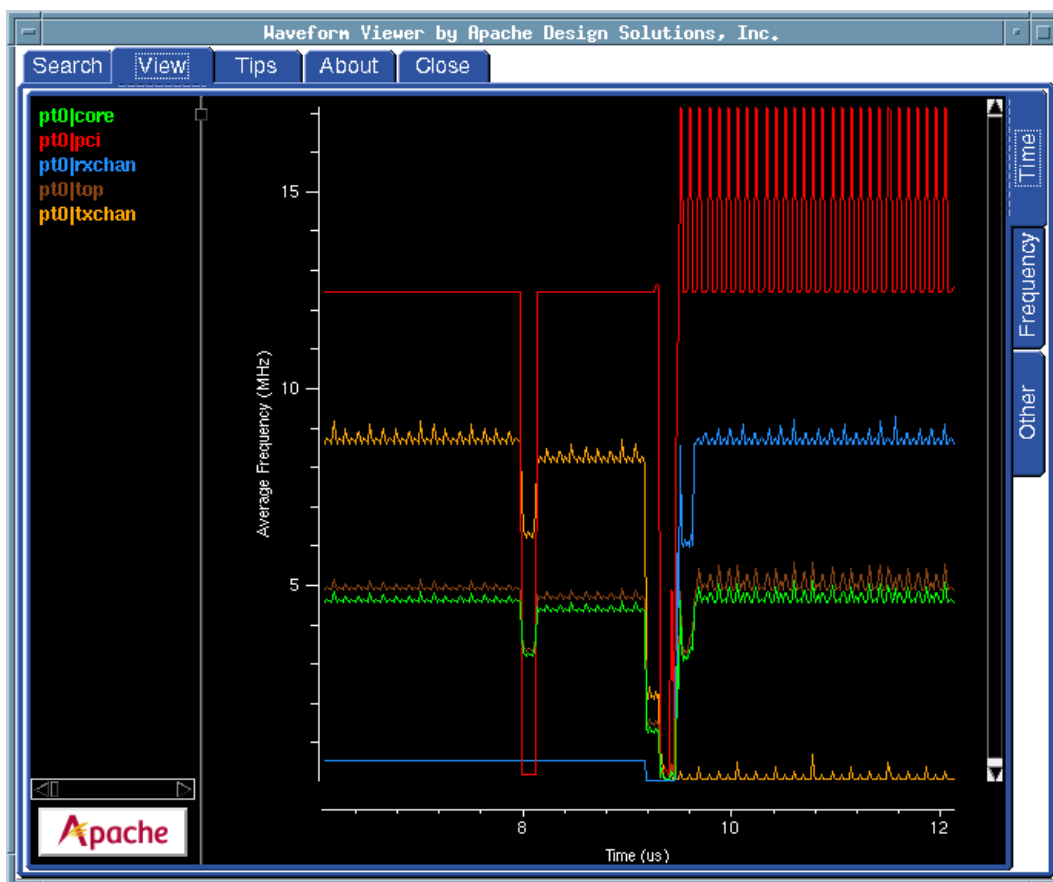


Figure 80 Sample Vector Analysis Waveforms for Five Specified Groups

Note that this is the same waveform generated by the full-chip analysis tutorial. For details on this flow and to try a sample run, see [Running an RTL Full-Chip Power Analysis Using Command Files](#) section in the analysis tutorial.

Chapter 10

Analyzing Simulation-Based Average Power

Introduction

You can run PowerArtist average power analysis on RTL, gate-level and mixed RTL and gate designs. Before you run average power analysis, it is highly recommended that you read [Chapter 8, *Preparing for Power Analysis*](#), which describes prerequisite steps you need to perform before you begin a power analysis. It is also highly recommended that you run through [PowerArtist Tutorial Part 1: Power Analysis](#) to learn how to run both a command-line flow and wizard-based flow (for beginners) for average power analysis.

Overall Design Flow

You should implement the following high-level design flow to perform time-based power analysis:

1. Simulate your design and generate simulation data in the FSDB, IAF, or VCD formats.
2. Build a scenario file using the [Elaborate](#) command. For details, see [Getting Your Design into PowerArtist](#).
3. Run vector analysis using the [GenerateActivityWaveforms](#) command. For details, see [Analyzing Simulation Activity](#).
4. Run the [CalculatePower](#) -analysis_type average command. Details for this process are documented in this chapter.
5. View the reports, in text format, created by the CalculatePower command or view the results, in the PowerCanvas, that you created by saving the power database.

Chapter Organization

The following topics are covered in this chapter:

- [Running a Power Analysis in Full Simulation Mode](#)
- [Running Analysis with Incomplete Simulation Data](#)
- [Performing Gate-Level Average Power Analysis](#)
- [Running Modal Analysis](#)
- [Understanding the Basics of the Detailed Power Report](#)
- [Controlling the Contents of the Power Report](#)
- [Analyzing Average Power Using a SAIF File](#)
- [Analyzing Average Power Using Partial Stimulus Files](#)
- [Re-Using a Stimulus File from a Previous Run](#)

Running a Power Analysis in Full Simulation Mode

Running power analysis in full simulation mode is done entirely by the [CalculatePower](#) command. This command first converts simulation activity data into a global activity file (GAF) and then runs the power analysis.

Controlling Your Average Power Analysis

As with all PowerArtist programs, you control the operation of your power analysis by specifying the appropriate commands in your PowerArtist command file. To perform an average analysis in full simulation mode, you need to specify the [CalculatePower](#) command with the required arguments:

- `-activity_file your_simulation_file`
This option specifies an input stimulus file generated by to a functional simulator run. The file may be in FSDB, VCD or IAF (generated by Apache PLI routines) format. You must specify either `-activity_file` or `-vectorless_input_file` for an average power analysis. CalculatePower automatically determines the type of the simulation activity file.
- `-gaf_file your_gaf_file`
Specifies the name for your generated GAF file.
- `-scenario_file your_scenario_file`
This option specifies the scenario file you generated using the [Elaborate](#) command. For more information, see [Getting Your Design into PowerArtist](#).
- `-synlib_files {file_name1 file_name2 ...}`
Adds the specified file or Tcl list of files to the list of Liberty technology files. You can also use the [ReadLibrary](#) command to specify which Liberty libraries to read.
- `-top_instance top_module_name`
This option specifies the instance in the simulation hierarchy that corresponds to the top-level instance in your scenario file. The `-top_instance` is specified as a

dotted name in which a dot is used as a hierarchical separator regardless of the simulator that was used (for example, testbench.corelogic_0).

It is also recommended that you specify some additional options to control your analysis in different ways.

- To control the point in time with which to begin power analysis, you can use the `-start_time` option.
- To stop the conversion of simulation data before the point when the simulation stopped, specify the stop time with the `-finish_time` option. This option is also needed if simulation time ended long after the last signal toggle occurred. Otherwise, your average power number will be incorrect.
- By using `-start_time` and `-finish_time`, you can perform a variety of “what-if” scenarios without having to re-create the simulation activity file.
- If you have a gate-level design, you will need to specify the `-gate_level_netlist` option. For details, see [Performing Gate-Level Average Power Analysis](#).
- If you have an IAF, VCD, or FSDB file that does not match your scenario file you will want to run with the `-mixed_sim_prob_estimation` option. For details, see [Running Analysis with Incomplete Simulation Data](#). You will also need to specify this option if you have a partial stimulus file. For details, see [Analyzing Average Power Using Partial Stimulus Files](#).
- To perform mode-dependent power analysis, specify the name of the mode file with the `-mode_file` option. For details see, [Running a Power Analysis in Full Simulation Mode](#).
- Simulations of large designs could result in a significant number of signals never leaving the X state. This is particularly true for gate-level designs. This could severely compromise the accuracy of your results. The [Transition Counting on Nets](#) section describes how you can control some of the effects for RTL designs. X states in gate-level designs will most likely cause power to be underestimated since power vectors modeled in your power library will not match transitions to and from X states. The CalculatePower command generates a warning such as:

```
wwgaf: Warning 3344: wwgaf encountered 54102 signal(s) that were in  
an X state for more than 10ns amount of time.
```

This means that an X state of duration greater than 10 ns occurred 54,102 times during the simulation run. This could be one signal being in an X state 54102 times or 54102 signals 1 time.

Seeing this message with an unexpectedly large number of signals is an indication that your test bench may not be sufficiently robust for performing power analyses. The duration may be controlled by the `-allowed_x_time` option.

Note on Pin-Based Estimation for Gate-Level Instances

There are two techniques that PowerArtist can use for power estimation of gate-level instances: pin-based and arc-based. By default, PowerArtist uses pin-based estimation which does not do arc monitoring. Arc monitoring requires the ports of the instances to be output into your simulation data file—not just the nets connected to the ports of the instance. This method is somewhat faster and consumes less memory than the arc-based method, at the cost of some accuracy. Arcs are still monitored for the following instances:

- IO pads
- instantiated memories
- non-memory cells containing bus pins
- macro combinational cells with 11 or more pins

To enable arc monitoring for additional cells and instances when using this option, use the [MonitorArcs](#) command. If you want PowerArtist to perform arc-based estimation for all gate-level instances, you can specify the “-arc_based_estimation true” option to [CalculatePower](#).

Sample CalculatePower Specification for Full Simulation Mode

To run a power analysis in full simulation mode, you may want to use a [CalculatePower](#) command such as the following:

```
set design top
CalculatePower -analysis_type average \
  -activity_file ../design_data/rtl_sim/activities.vcd \
  -average_report_file $design.AverageBatch.rpt \
  -average_report_options agip \
  -average_write_power_db true \
  -compress_gaf true \
  -default_output_load 3.9e-11 \
  -detailed_vertical_report true \
  -finish_time 12135580ps \
  -gaf_file $design.AverageBatch.gaf \
  -calculate_log CalculatePowerAverageBatch.log \
  -mode_file txrx.mode \
  -power_db_name $design.AverageBatch.pdb \
  -scenario_file $design.Batch.scn \
  -start_time 6014730ps \
  -top_instance txrx_tst.top1 \
  -use_scan_flops true \
  -vertical_report_instances $design \
  -wireload_library hvt
```


Running Analysis with Incomplete Simulation Data

If you have an IAF, VCD, or FSDB file that does not match your scenario file in some way, for example, some instance in the scenario file may be missing in the design you simulated because the source files have changed, you will need to run the [CalculatePower](#) command with the `-mixed_sim_prob_estimation` option. When you run with this option, only some of the nets have .gaf data associated with them, as opposed to full simulation mode in which all of the nets are assumed to have .gaf data available.

Performing Gate-Level Average Power Analysis

If you have a pure gate-level design, you will need to use the “`-gate_level_netlist true`” [Elaborate](#) option when you create a new scenario file. This option tells the tools that the design is meant to be gate level. If during scenario file creation Elaborate encounters RTL components, it will black box them and generate warning messages.

Analyzing Large Gate-Level Designs

If you are executing PowerArtist on the 32-bit Linux system, you are severely constrained by the size of design you will be able to run. On some Linux systems, this may be at best 2GB of memory and on others, it is certainly no more than 4GB. Therefore, it is recommended that for any meaningful sized, gate-level designs, you use 64-bit Linux or Solaris platforms exclusively.

Running Modal Analysis

Large designs often have several operational modes, such as standby, receiving, and transmitting. The power consumption for these modes will typically vary as different sections of the chip may be inactive, or even powered down, in each mode. Entering data into a mode description file enables PowerArtist to perform accurate RTL power analysis for each mode. You can print the mode analysis results or PowerArtist can compute a weighted average. Note that mode analysis is available at the RTL designs or mixed RTL and gate-level designs only. It is not available if you have a full gate-level design that you have signaled to PowerArtist-X using the `-gate_level_netlist` option.

A mode file contains design-specific information that the tools use to execute several power analyses corresponding to different operational modes of the design, and combine the results into one or more reports.

The format for a mode file meant for simulation-based power analysis is shown here.

```
$mode "boolean_expression"  
$report file_name  
$result file_name
```

Table 3 Elements Within the Mode File

Element	Description
<code>\$mode <i>boolean_expression</i></code>	This required keyword assigns a name to the operational mode you are defining. The <i>boolean_expression</i> can be a single signal name or a combination of signals in the design. The signal's hierarchical name is, in this case, with respect to the top most scope (or testbench scope) in the VCD—not the design's "top" name. You can use any of the standard boolean operators.
<code>\$report <i>file_name</i></code>	This optional keyword writes a report file for this mode to the specified file name. You can write this file to any directory by specifying a relative or absolute file path name; however, the "~" symbol is not supported in file path name.
<code>\$result <i>file_name</i></code>	This optional keyword writes a result file for this mode to the specified file name.

A sample mode file might look like the following:

```
$mode top.counting_off  
$report counting_off.rpt  
  
$mode top.counting_on  
$report counting_on.rpt
```

This file says the design is expected to be in one of two modes: `counting_off` or `counting_on`.

The name of the mode must be the name of a signal such that when it is asserted high, the system is assumed to be in this mode. In this case, the design is expected to find two signals: `top.counting_off` and `top.counting_on`. The names of the signals must be in the VCD file and be rooted at the topmost scope (or testbench scope) in the VCD.

For this mode file, PowerArtist performs two analyses, one for each mode. The results of the analyses are written to two separate files (`counting_off.rpt` and `counting_on.rpt`).

A second sample mode file might be:

```
$mode top.sys_request&&top.interrupt  
$report request-interrupt.rpt  
  
$mode "top.sys_request&&(!top.interrupt) "  
$report request-not_interrupt.rpt  
  
$mode "(!top.sys_request)&&top.interrupt"  
$report not_request-interrupt.rpt  
  
$mode "(!top.sys_request)&&(!top.interrupt) "  
$report not_request-not_interrupt.rpt
```

In this example, there are four modes. While processing the simulation data file, PowerArtist monitors `sys_request` and `interrupt`. When either of these two signals changes state, PowerArtist evaluates the various boolean expressions, determines which one is true and allocates the energy to that mode bucket.

There are two edge cases for this method of modal power. If the simulator enters a mode in which none of the mode signals are in a logic 1 state, the simulation activity for those simulation periods is ignored (that is, the activity will not contribute to the power of any mode). If more than one of the mode signals are in the 1 state at the same time, simulation determines that the system is in the mode that appears first in the mode file (counting_off in the first sample mode file).

Using a mode file containing n different modes results in $n+1$ report files. One report file is produced and named for each mode and one report file is produced for the entire simulation. This mode file produces a report file named for each of the modes, `counting_off.rpt` and `counting_on.rpt` and another report file representing the entire simulation, which is named as defined by the [CalculatePower](#) command.

To obtain the total power consumed during the two modes, you *cannot* add the power consumed by each mode. Instead, you must:

1. Convert each power to energy.
2. Add the two energy values together.
3. Convert to power.

This is equivalent to computing a weighted sum (weighted by the amount of time spent in each mode) of the two powers.

For more information on mode files and additional samples, see [Mode File Format](#) in the *PowerArtist Reference Manual*.

Understanding the Basics of the Detailed Power Report

The detailed power report is stored in an ASCII file (or as an HTML file if you request that using the `-html_report_title` option). Although some of the sections in this report are always included others are generated/formatted depending on the values you specify to the `-average_report_options` argument to `CalculatePower`. Major sections (often used) include the following:

- **Header:** includes:
 - Date and time report file was generated.
 - Version of PowerArtist used.
 - Library file used for the power calculation.
 - Assumed clock frequency.
 - Voltage values for the power supplies used.
 - Name of the activity file.
 - Names of any capacitance files.
 - How the power consumed in driving inter-module nets is reported.
 - Design and simulation prefix.
- **Total Power Consumption:** summarizes the results of the analysis.
- **Internal Power Consumption:** lists the power consumed by each individual leaf module in the design. The specific content of this section will depend, in part, on the options you specify the `CalculatePower -average_report_options` argument. For more information, see [Controlling the Contents of the Power Report](#).
- **Pad Power Consumption:** lists the power consumed by each individual IO pad in the design, along with the capacitive load “seen” by that pad.
- **Clock Power Consumption:** lists the following information:
 - The hierarchical net name, the type of analysis, and the total power for the clock tree. The analysis type will be either `Instantiated`, which means that it was traced, or `Inferred` meaning clock inferencing was done.
 - Area occupied by the net.
 - Clock senses.
 - Frequency of the clock net
 - Transition time of the clock net
 - Fanout capacitance of the clock net (divided into contributions from wire and pin capacitances).
 - Power consumed as the net toggles (divided as wire and pin components).
 - The wire load model applied to the net (the first entry is the net name and the second entry is the wire load model applied to that net). If you assigned wire load models to sub-nets of the clock tree, they would be listed here as well.

- Descriptions of the instances and net that were traced as part of the clock tree.
- The clock tree inferencing that occurred including buffers and integrated clock gating cells that were inferenced.
- The clock gating summary that provides statistics on the number of gated and ungated registers and the number of integrated clock gating cells either instantiated or inferenced in your design.

To better understand the content of this report and to see a sample, see the [Clock Domain Power Consumption](#) section in the analysis tutorial.

- **Total Power Per Supply:** lists each power supply used in the design with the power consumed from each supply. This section is generated when you specify the option “-average_report_options V” to the [CalculatePower](#) command.

To create a report where the internal and pad power are divided into their static and dynamic components, you can specify the option “-average_report_options s” to the [CalculatePower](#) command.

Power per supply is also reported as static and dynamic components.

- **Vertical Report:** reports power by cell. You can generate/manipulate this section by specifying the following [CalculatePower](#) options:

```
-vertical_report_instances [{inst1 inst2 inst3...}]  
-detailed_veritcal_report true  
-vertical_report_sort_mode alphabetical | power
```

Available Power Report Formats

The power report is available in plain text or HTML. Plain text is always available. To generate an HTML report, specify the [CalculatePower](#) -average_html_report_title option. You can display these reports in any web browser.

Controlling the Contents of the Power Report

You can specify different report options to [CalculatePower](#) to control what is included in and the format of the instance power report in the Internal Power Consumption section in several ways. In addition, you can specify reporting options that generate additional sections in the power report.

This section provides many different sample reports that vary based on the specified report options.

- A typical design, with no report options specified, contains the following instance power section¹.

```
2. Internal power consumption  
=====
```

Component	Model	Power (Watts)
-----------	-------	---------------

-----	-----	-----
wavtab.cnt.#0	adder_rip	67.3uW
wavtab.cnt.#1	register	225uW
wavtab.dff.#0	register	194uW
wavtab.div.#0	adder_rip	246uW
wavtab.div.#1	register	851uW
wavtab.rom	rom	4.03mW
Total internal power		5.62mW

- If you specify `-average_report_options ip`, the non-leaf hierarchical elements are shown using indenting to indicate levels of hierarchy, rather than repeating all the non-leaf cell names.

Component	Model	Power (Watts)
-----	-----	-----
wavtab	user	5.62mW
cnt	user	293uW
#0	adder_rip	67.3uW
#1	register	225uW
dff	user	194uW
#0	register	194uW
div	user	1.1mW
#0	adder_rip	246uW
#1	register	851uW
rom	rom	4.03mW
Total internal power		5.62mW

- If you specify `-average_report_options mp`, the inferred elements whose names contain `#` are not shown. This makes the report much shorter and also shows only the instances you have actually created in your design. See the following example.

Component	Model	Power (Watts)
-----	-----	-----
wavtab	user	5.62mW
wavtab.cnt	user	293uW
wavtab.dff	user	194uW
wavtab.div	user	1.1mW
wavtab.rom	rom	4.03mW
Total internal power		5.62mW

- Another way of reducing the size of the report is to use `-instance_power_threshold float` option, which eliminates from the report any instance consuming a small amount of power. Normally you might set this to 1

1. Note that the report samples that have just one Power(Watts) column were generated using the “f” reporting option that combines the static and dynamic power values into one.

percent (-instance_power_threshold 1). For the example shown here, the elimination threshold was set to 10 percent (-instance_power_threshold 10), a very high value.

Component	Model	Power (Watts)
-----	-----	-----
wavtab.div.#1	register	851uW
wavtab.rom	rom	4.03mW
Total internal power		5.62mW

It is important to notice that in each case, the total power reported is the same. The report options only change which modules are shown in the report, and do not change the power computation.

- If you specify -average_report_options a, the area estimation section will be included in the report.

6. Area				
=====				
Component	Width	Height	Regs	Gates
-----	-----	-----	-----	-----
top.core1.a1.#241	33.8um	33.8um	0	69
top.core1.a1.#242	30.5um	30.5um	0	56
top.core1.j1.#1923	10.8um	10.8um	2	7
...				
Total Counts			697	6382
Total Net Routing Area		542p (m^2)		
Total Area (Gates+Routing)		106n (m^2)		

- If you specify -average_report_options N, PowerArtist creates an additional section that reports net transition time information.

6. Transition time of nets		
=====		
	Transition time (Sec.)	
Net	Rise	Fall
---	---	---
top.Pclk	1.070e-10	1.070e-10
top.clk	1.070e-10	1.070e-10
...		
top.core1.s1.#843	1.070e-10	1.070e-10
top.core1.s1.#491	1.070e-10	1.070e-10

- If you specify `-average_report_options V`, PowerArtist creates an additional section that reports power dissipation per power supply as well as how the estimation of library voltages is determined.

2. Total power per supply
=====

Supply	Estimation	Library	Power(Watts)		
	Voltage(V)	Voltage(V)	Static	Dynamic	Total
-----	-----	-----	-----	-----	-----
clock_gating.vdd	1.45	1.45	41.5nW	83.9uW	84uW
Estimation Voltage Source = nom_voltage					
Library Voltage Source = nom_voltage					
DP512x32.WW_Vdd	2.5	2.5	0W	344mW	344mW
Estimation Voltage Source = nom_voltage					
Library Voltage Source = nom_voltage					
std_cell_lib.vdd	2.5	2.5	142uW	2.72mW	2.86mW
Estimation Voltage Source = nom_voltage					
Library Voltage Source = nom_voltage					
txrxios.Vdd	3.3	3.3	2.64uW	94.8mW	94.8mW
Estimation Voltage Source = nom_voltage					
Library Voltage Source = nom_voltage					
Total power			145uW	442mW	442mW

- If you specify `-average_report_options e`, scientific notation is used to report power numbers, as shown in the following report.

2. Internal power consumption
=====

Component	Model	Power(Watts)		
		Static	Dynamic	Total
-----	-----	-----	-----	-----
top.core1.a1.#241	decoder	1.93e-06	0.00e+00	1.93e-06
top.core1.a1.#242	mux21	1.76e-06	0.00e+00	1.76e-06

- If you specify `-average_report_options P`, PowerArtist creates an additional section that reports pin transition time information.

6. Transition time of pins
=====

Pin	Transition time(Sec.)	
	Rise	Fall
---	---	---
top.sclk1.inpad.PAD	1.040e-10	1.040e-10

top.sclk1.inpad.C	1.040e-10	1.040e-10
top.core1.u1.#133.in[0]	1.070e-10	1.070e-10
top.core1.u1.#133.out[0]	1.070e-10	1.070e-10

- If you specify `-average_report_options g`, PowerArtist creates an additional section that reports net frequency and RTL glitch information, as shown in the following report.

6. Net frequencies			
=====			
Net	Type	Glitch	Edge rate
---	----	-----	-----
decode.metgen_top.afifo8x17.#858	Int	0.00%	31.7MHz
decode.metgen_top.afifo8x17.re_cnter	Sync	0.00%	18MHz
decode.i_ry[0]	PI	0.00%	4.42MHz

Possible net types include: Int (internal), Sync (synchronous), and PI (primary input).

- If you specify the `-average_report_options t`, PowerArtist prints combined static and dynamic instance power. This is applicable to all sections in the report file.

2. Internal power consumption		
=====		
Component	Model	Power (Watts)
-----	-----	-----
top.core1.a1.#241	decoder	1.93uW
top.core1.dpmem.m2.m2	DP512x32	1.43mW

- If you specified the `-average_report_options v`, PowerArtist replaces the gate name with `vendor_gate` in the report as shown in the following sample.

2. Internal power consumption				
=====				
Component	Model	Power (Watts)		
		Static	Dynamic	Total
-----	-----	-----	-----	-----
top.core1.a1.#241	decoder	1.93uW	0W	1.93uW
top.core1.p1.#1918	register	7.33uW	821uW	828uW
top.core1.r1.dpmem.m1.m1	vendor_gate	0W	47.9mW	47.9mW
...				

- The following example of a vertical report includes entries for each model type only if you request a vertical report by specifying the `-vertical_report_instances` and `-detailed_vertical_report` options to `CalculatePower`:

Power dissipation by model/gate type:
=====

		Cell	
Component	Model	Count	Power (Watts)
-----	-----	----	-----
top.core1		8	24mW
Register power		2	4mW
Latch power		0	0mW
Memory power		0	0mW
Other power		6	20mW
	adder_pg	2	8mW
	comparator	1	1mW
	decoder	1	5mW
	mux21	2	6mW
	register	2	4mW
=====			
top.core2		12	36mW
Register power			...
...			

Note that vertical reports do not include instances that are reported as part of clock tree power.

Also, it is the number of instances being reported and not the number of bits. You might have 1 register instance in the design that is 64-bit wide. The cell count will be 1 not 64. If you performed clock gating in your design, you can obtain the number of register bits from the Clock Gating Summary as shown here:

Clock Gating Summary:

Clock net: top.clk
Number of inferred clock gating cells: 10
Number of registers gated by inferred clock gating cells: 376
Number of registers enhanced gated by inferred clock gating cells: 0
Number of instantiated clock gating cells: 0
Number of registers gated by instantiated clock gating cells: 0
Total number of gated registers: 376
Total number of ungated registers: 77

Notice the line “Number of registers enhanced gated by inferred clock gating cells”. This line appears if you request enhanced clock gating be performed (using the “-enhanced_cg true” option to the [SetClockNet](#) command. For more information on enhanced clock gating, see [Performing Enhanced Clock Gating](#),

- If you specify the `-average_report_options u`, PowerArtist excludes vendor gates from power and area reports. Use this option to reduce the size of your gate-level power report. If you don't use this option, every standard cell instantiation in your design will be reported, which could add up to millions of lines. An excerpt of a report with and without `-average_report_options u` is shown below. Power variations are highlighted in red.

Sample Report with `-average_report_options u`

```
2. Internal power consumption
=====
```

Component	Model	Power (Watts)		
		Static	Dynamic	Total
-----	-----	-----	-----	-----
...				
top.core1.al.#241	decoder	1.93uW	0W	1.93uW
top.core1.pl.#1918	register	7.33uW	821uW	828uW
...				

Sample Report without `-average_report_options u`

```
2. Internal power consumption
=====
```

Component	Model	Power (Watts)		
		Static	Dynamic	Total
-----	-----	-----	-----	-----
...				
top.core1.al.#241	decoder	1.93uW	0W	1.93uW
top.core1.pl.#1918	register	7.33uW	821uW	828uW
...				
top.core1.dpmem.m2.m1	DP512x32	0W	14.3mW	14.3mW
...				

- If you specify `-average_report_options c`, PowerArtist moves power associated with clock switched-cap for registers or latches into the clock report. Excerpts of a report with and without `-average_report_options c` are shown below. Power variations are highlighted in red. If you specify an uppercase “C” option, then you will get the clock switched-cap power for memories and IP blocks in your report.

Sample Report With `-average_report_options c`

```
1. Total power consumption
=====
```

Power contribution	Power (Watts)		
	Static	Dynamic	Total
-----	-----	-----	-----
Internal power			
Internal register power	38.4uW	921uW	959uW
Internal latch power	3.53uW	0W	3.53uW
Internal memory power	0W	344mW	344mW
Other internal power	100uW	964uW	1.06mW

Total internal power		142uW	346mW	346mW
IP Core power		0W	0W	0W
Pad power		0W	88.7mW	88.7mW
Clock power		2.69uW	7.1mW	7.1mW
Inferred Buffer power		139nW	4.9uW	5.04uW
Total power		145uW	442mW	442mW
2. Internal power consumption				
=====				
		Power (Watts)		
Component	Model	Static	Dynamic	Total
-----	-----	-----	-----	-----
...				
top.core1.t1.s1.#970	register	496nW	318nW	814nW
...				
Total internal power		142uW	346mW	346mW
5. Clock power consumption				
=====				
		Power (Watts)		
Net	Type	Static	Dynamic	Total
---	---	-----	-----	-----
...				
top.clk	Inferred	2.46uW	5.97mW	5.97mW
...				

Sample Report Without -average_report_options c

1. Total power consumption				
=====				
		Power (Watts)		
Power contribution		Static	Dynamic	Total
-----		-----	-----	-----
Internal power				
Internal register power		38.4uW	1.76mW	1.8mW
Internal latch power		3.53uW	0W	3.53uW
Internal memory power		0W	344mW	344mW
Other internal power		100uW	964uW	1.06mW
Total internal power		142uW	347mW	347mW
IP Core power		0W	0W	0W
Pad power		0W	88.7mW	88.7mW
Clock power		2.69uW	6.26mW	6.26mW
Total power		145uW	442mW	442mW

```

2. Internal power consumption
=====

Component      Model
-----      -
...
top.core1.t1.s1.#970  register
...
Total internal power

Power(Watts)
Static  Dynamic  Total
-----  -
496nW   31.1uW   31.6uW
...
142uW   347mW   347mW

5. Clock power consumption
=====

Net            Type
---            -
...
top.clk        Inferred
...

Power(Watts)
Static  Dynamic  Total
-----  -
2.46uW   5.69mW   5.69mW
...

```

- If you specify `-average_report_options 0`, PowerArtist creates an additional section that reports internal driver power (the power required to toggle the net). Excerpts of reports with and without `-average_report_options 0` are shown below. Power variations are highlighted in red. Note with `-average_report_options 0` that the internal load power is not reported as internal power consumption.

Sample Report With `-average_report_options 0`

```

1. Total power consumption
=====

Power contribution
-----
Internal power
  Internal register power
  Internal latch power
  Internal memory power
  Other internal power
  Total internal power
IP Core power
Pad power
Clock power
Internal load power
Total power

Power(Watts)
Static  Dynamic  Total
-----  -
38.4uW   999uW   1.04mW
3.53uW      0W   3.53uW
0W      344mW   344mW
100uW   143uW   243uW
142uW   345mW   345mW
0W      0W      0W
0W   88.7mW   88.7mW
2.69uW   6.26mW   6.26mW
0W   1.62mW   1.62mW
145uW   442mW   442mW

2. Internal power consumption
=====

```

		Power (Watts)		
Component	Model	Static	Dynamic	Total
-----	-----	-----	-----	-----
top.core1.r1.a1.#1721	xor	55.1nW	61.7nW	117nW
top.core1.r1.a1.#1722	xor	55.1nW	61.7nW	117nW
...				
Total internal power		142uW	347mW	347mW

6. Internal load power consumption

=====

		Power (Watts)			
Net	Estimator	Cap (F)	Static	Dynamic	Total
---	-----	-----	-----	-----	-----
top.rx_rq	Point to point	71fF	0W	13.2nW	13.2nW
top.updout[0]	Point to point	79.1fF	0W	6.94uW	6.94uW
top.core1.s1.#309	Point to point	6.9fF	0W	0W	0W
top.core1.s1.#310	Point to point	6.9fF	0W	0W	0W

Note: Point to Point = Wireload model estimation.

Exact = SPEF or Capacitance file input

Note that the -r 0 option filters out the following nets:

- Nets that are traced as part of a clock network defined in your clock file
- Nets that do not have any capacitance value attached to them
- Nets that do not have any drivers or are driven by a connect or connect_inv instance
- Nets that are the output of IO pads
- Nets with no output pins (that is, primary inputs)

Sample Report Without -average_report_options 0

1. Total power consumption				
=====				
		Power (Watts)		
Power contribution		Static	Dynamic	Total
-----		-----	-----	-----
Internal power				
Internal register power		38.4uW	1.76mW	1.8mW
Internal latch power		3.53uW	0W	3.53uW
Internal memory power		0W	344mW	344mW
Other internal power		100uW	964uW	1.06mW
Total internal power		142uW	347mW	347mW
IP Core power		0W	0W	0W
Pad power		0W	88.7mW	88.7mW
Clock power		2.69uW	6.26mW	6.26mW
Total power		145uW	442mW	442mW

2. Internal power consumption				
=====				
		Power (Watts)		
Component	Model	Static	Dynamic	Total
-----	-----	-----	-----	-----
...				
top.core1.r1.a1.#1721	xor	55.1nW	1.15uW	1.21uW
top.core1.r1.a1.#1722	xor	55.1nW	1.15uW	1.21uW
...				
Total internal power		142uW	347mW	347mW

- If you specify the `-average_report_options d` (and you provided a power diff file), the new delta on parents in the power diff is reported for each section.

5. Internal Power difference to abc.res					
=====					
Component			This (W)	Ref (W)	Diff (W)
-----			-----	-----	-----
top.core1.a1.#241			1.93uW	1.93uW	-1.27E-21W
top.core1.a1.#242			1.76uW	1.76uW	-1.27E-21W
6. Pad Power difference to abc.res					
=====					
Component	Pad type	Cap (F)	This (W)	Ref (W)	Diff (W)
-----	-----	-----	-----	-----	-----
top.sclk1.inpad	PDI	2.09pF	1.91mW	1.91mW	200pW
top.sclk2.inpad	PDI	1.94pF	1.91mW	1.91mW	-7.24uW
7. Clock Power difference to abc.res					
=====					
Net	Estimator	Cap (F)	This (W)	Ref (W)	Diff (W)
-----	-----	-----	-----	-----	-----
top.clk	Point to point	742fF	533uW	533uW	-207pW
top.pci_clk	Point to point	594fF	425uW	427uW	-1.62uW

- If you specify `-average_report_options r` (and you have provided a power diff file), the relative percentage delta on parents in the power diff is reported for each section.

5. Internal Power difference to abc.res			
=====			
Component	This (W)	Ref (W)	Diff (%)
-----	-----	-----	-----

top.core1.a1.#241			1.93uW	1.93uW	+0.000%
top.core1.p1.#1917			3.53uW	3.53uW	+0.000%
top.core1.p1.#1918			842uW	844uW	-0.207%
6. Pad Power difference to abc.res					
=====					
Component	Pad type	Cap(F)	This(W)	Ref(W)	Diff(%)
-----	----	-----	-----	-----	-----
top.sclk1.inpad	PDI	2.09pF	1.91mW	1.91mW	+0.000%
top.sclk2.inpad	PDI	1.94pF	1.91mW	1.91mW	-0.379%
7. Clock Power difference to abc.res					
=====					
Net	Estimator	Cap(F)	This(W)	Ref(W)	Diff(%)
-----	-----	-----	-----	-----	-----
top.clk	Point to point	742fF	533uW	533uW	+0.000%
top.pci_clk	Point to point	594fF	425uW	427uW	-0.379%

Analyzing Average Power Using a SAIF File

PowerArtist can perform an RTL, mixed RTL and gate or pure gate-level average power analysis using a SAIF file, instead of a VCD or FSDB activity file. SAIF is a well-defined ASCII format defined as part of the IEEE 1801 standard. SAIF files capture toggle information directly from simulators. Since a basic SAIF format contains only toggle counts, rather than value change sections as supplied by VCD or FSDB files, using SAIF files has the following benefits:

- Faster simulation time since far less data is created by a simulator
- Less disk space is required for simulation.
- The SAIF file is smaller than an equivalent VCD file and therefore takes less time to read during power analysis.

Despite these benefits, there are some drawbacks to using this format. The most important one being that you cannot analyze power over time. As noted previously, toggle counts are captured but not specific signal transitions and the time steps at which they occurred.

The second problem is that signal correlation is not possible, again because only toggle counts are stored. Therefore, state-dependent static power and dynamic power calculations require heuristics to determine which conditions get matched in your technology library. This results in less accurate power numbers when compared to a full arc power analysis. This may be especially true for large macros like compiled memories that have many pins and may have relatively complex power modeling.

There are two SAIF formats. The basic format captures signal toggle information and duty cycles. The State-Dependent/Path-Dependent (SDPD) format captures information equivalent to the PowerArtist arcs in the GAF file. PowerArtist supports both of these formats. If you want the most accurate power analysis results you should use the SDPD format. Support for the SDPD format is in beta. This will be especially true for a gate-level analysis. However, be aware that the power arc matching now done during the CalculatePower command will now be done during your simulation run. This your simulation will be slower than if you generate the basic format.

Flow Details

PowerArtist supports the SAIF format for average power analysis only. To use a SAIF file, you need to specify the `-saif_file` option to the [CalculatePower](#) command in addition to all of the other options you normally use, for example:

```
CalculatePower -analysis_type average -saif_file design.saif ...
```

PowerArtist automatically detects when the given SAIF file uses the SDPD format and so will perform arc-based estimation. For basic SAIF files, PowerArtist automatically performs pin-based analysis for RTL designs (and most gate-level designs).

Understanding Power Value Fluctuations Between SAIF and Other Formats

You should expect subtle differences in power numbers for instances in your design if you compare a SAIF-based power analysis with an equivalent VCD or FSDB-based analysis.

Duty cycle is defined as follows:

$$\text{duty_cycle} = T1 / (T1 + T0)$$

where T1 = the total time in which the signal remains in the 1 state

T0 = the total time in which the signal remains in the 0 state.

Any X or Z values generated during simulation need to be translated into duty cycle information. The equations used in the SAIF flow are:

$$\text{duty_cycle} = (T1 + (TX / 2) + (TZ / 2)) / (T1 + T0 + TX + TZ)$$

$$TC_{\text{new}} = TC_{\text{SAIF}} + IG$$

In this equation the following apply:

- T1, T0, TX and TZ are fields from a line in the SAIF file
- TX and TZ is the total time in which the signal remains in the X or Z state, respectively.
- TC_{new} is the newly calculated toggle counts
- TC_{SAIF} is the original toggle count in the SAIF file
- IG are the internal glitches

Analyzing Average Power Using Partial Stimulus Files

PowerArtist can perform an RTL, mixed RTL and gate or pure gate-level average power analysis without requiring that every user net name be present in an FSDB, IAF or VCD file with its associated toggle information. If a net is not present in the stimulus file but it is in the design, PowerArtist algorithmically estimates the activity of the net while performing an average power analysis. Once it has estimated activity values for all nets in the design, it then performs the power calculation. Note that the more nets that are “missing” from the stimulus file, the less accurate the power analysis. PowerArtist’s activity analysis allows you to trade-off the simulation time and stimulus file size with the accuracy of the power analysis.

There are three flows to consider:

- RTL design using a partial RTL stimulus file
- Mixed RTL and gate or complete gate-level design using a partial RTL stimulus file (see [Name Mapping Flow](#)).
- Gate-level design using a partial gate-level stimulus file (remember to use the “-mixed_sim_prob_estimation true” option setting).

In flows one and three, nothing special is required. If you are following the Palladium flow (see [Acquiring Simulation Data in Palladium Flows](#)) CalculatePower will check to make sure that the nets tagged as being selected in the scenario file are present in your stimulus file. If not, CalculatePower issues a warning message.

Name Mapping Flow

In the second flow, you need a synthesized portion of a design (or all of the design) to perform an average power analysis using an RTL stimulus file. You may find that simulating designs at the gate level is becoming more difficult, but you still want to perform a power analysis. There is one very important step that must be done before this type of analysis is possible—name mapping.

During gate-level synthesis, the RTL design hierarchy often becomes flattened and the RTL nets that will be inferred as registers are transformed into gate-level instances; therefore, you must map as many of your RTL net names as possible into gate-level equivalent names. Every gate-level net that is successfully mapped will have its activity determined by CalculatePower using its RTL toggles. This toggle information will be written out to the GAF file by the [CalculatePower](#) command. RTL nets in the design will be mapped automatically assuming that you have not changed either your RTL design hierarchy or your RTL modules between simulating the design and starting the CalculatePower run. The missing nets will have their activity calculated during average power analysis as in flows 1 and 3.

Currently, PowerArtist will do this gate-level mapping automatically using a mapping file generated by Conformal™ from Cadence. Suppose for example that your original design that you simulated was an RTL design. You would like to perform a power analysis on a fully synthesized gate-level design. The steps you will follow are:

1. Simulate your RTL design and create a stimulus file.
2. Synthesize your design to the gate level
3. Create a “do” file that runs Conformal in its “compare” mode.

Running Conformal in the compare mode versus its full equivalency check mode has two notable benefits. First, you can perform the mapping on your full-chip design. There is no need for a hierarchical equivalence checking methodology so the mapping step is much simpler to perform. Second, the performance is significantly better. During an equivalence checking run, one of the first steps is a compare operation that does the name mapping. Verifying the actual equivalence between your RTL and gate-level netlists is what takes the most time.

The following “do” script sample shows how a compare run is performed:

```
vpxmode
// preserve original RTL register names
set naming rule "%s_reg" -register -golden
// preserve hierarchical block names in RTL signal naming
set naming rule %s %L[%d].%s %s -instance
```

```
set system mode lec
remodel -seq_constant -repeat
add compare point -all
// compare
usage
report unmapped point > DESIGN.lec.unmapped
report compare data -class abort > DESIGN.lec.abort
report compare data -class noneq > DESIGN.lec.noneq
report mapped points > mapfile.dat
diag -sum
diag -all > DESIGN.lec.diag
exit -f
```

The key points to note in this script are:

- a. The “set naming rule” command is specified to preserve the original names of the RTL signals in the conformal mapping report.
 - b. The “compare” line is commented out. This tells Conformal to perform name mapping only, without equivalency checking.
 - c. The name mapping file PowerArtist needs is generated using the “report mapped points” command. The file created in this case, `mapfile.dat`, is what CalculatePower will process.
4. Create your gate-level scenario file then run CalculatePower in its name mapping mode to generate a GAF file and perform an average power analysis.
- a. Specify the [SetNameMapFile](#) command. The syntax of this command is:
`SetNameMapFile -map_file conformal_do_file_name -format conformal`
Use the output of the Conformal “do” file as your file name.
 - b. Specify [CalculatePower](#) using the “-use_rtl_sim_data true” option to indicate that you want to run in name mapping mode. This option indicates that the specified GAF file will be generated by a name-mapped run.

You can run this entirely from a command file such as the following:

```
Elaborate -top top \  
-gate_level_netlist true  
-scenario_file design.scn  
-verilog_startup_file design.vc  
SetNameMapFile -map_file mapFile.dat -format conformal \  
CalculatePower -analysis_type average \  
-top_instance testbench.top1 \  
-average_report_file design.rpt \  
-scenario_file design.scn \  
-activity_file ../designData/design_rtl.vcd \  
-gaf_file activities.gaf \  
-synlib_files ../designData/flop.lib \  
-use_rtl_sim_data true \  
-gate_level_netlist true \  

```

The critical points are highlighted in red. You need to set the `use_rtl_sim_data` variable to true to force the name mapping mode to occur. Notice that both the [Elaborate](#) and [CalculatePower](#) commands are run in the gate netlist flow (that is, they both specify the “-gate_level_netlist true” option).

The `-use_rtl_sim_data` option has a side effect of turning on the `-mixed_sim_prob_estimation` and the `-pin_based_estimation` options. Without `-mixed_sim_prob_estimation` turned on, missing nets will be flagged as errors. The `-pin_based_estimation` option indicates that many nets are missing from the design, so a complete arc-based power analysis is not possible.

Advanced Flows

At this point in time, PowerArtist supports name mapping files created by Conformal. If you have Formality™ from Synopsys or have the ability to create your own name mapping files, please contact your Apache Application Engineer who can answer questions about how to handle such a flow. In the future, Apache plans to support Formality.

Re-Using a Stimulus File from a Previous Run

Stimulus file processing typically takes a significant amount of time for any design of substantial size. If you want to do some what-if experiments without re-reading your stimulus file, you have to indicate this to the [CalculatePower](#) command. By default, during a simulation-based, average power analysis, your stimulus file gets re-read. If you want to prevent that from happening and you want to re-use the existing GAF file created by a previous run, you must specify the following option:

```
-use_existing_gaf true
```


Chapter 11

Analyzing Time-Based Power

Introduction

PowerArtist allows you to obtain power and current waveforms as a function of time for RTL, gate-level or mixed RTL and gate designs.

Before you run time-based power analysis, it is highly recommended that you read [Chapter 8, *Preparing for Power Analysis*](#), which describes prerequisite steps you need to perform before you begin any power analysis. In addition, if you are a new user, it is recommended that you run the full-chip tutorial. For details see, [Running an RTL Full-Chip Power Analysis Using Command Files](#).

Overall Design Flow

You should implement the following high-level design flow to perform time-based power analysis:

1. Simulate your design and generate simulation data in the FSDB, IAF, or VCD formats.
2. Build a scenario file using the [Elaborate](#) command. For details, see [Getting Your Design into PowerArtist](#).
3. Run vector analysis using the [GenerateActivityWaveforms](#) command. For details, see [Analyzing Simulation Activity](#).
4. Run the [CalculatePower](#) -analysis_type time_based command. Details for this process are documented in this chapter.
5. View the reports, in text format, created by the CalculatePower command or view the results, in the PowerCanvas, that you created by saving the power database.

Chapter Organization

The following topics are covered in this chapter:

- [Understanding the Inputs for a Time-Based Power Analysis](#)
- [Controlling Your Time-Based Power Analysis](#)
- [Understanding and Reviewing Outputs and Results of the Time-Based Analysis](#)
- [Monitoring Flop Clock Activity](#)

Understanding the Inputs for a Time-Based Power Analysis

To perform time-based power analysis you will need the following inputs:

- The [CalculatePower](#) -analysis_type time_based command with various other options required for a time-based analysis. For complete syntax, see [Syntax for Time-Based Power Analysis](#).
- A scenario file representing the design generated by the [Elaborate](#) command. For details, see [Getting Your Design into PowerArtist](#).
- Libraries in Liberty format.
- Simulation traces stored in either FSDB, IAF or VCD formats.
- You can use any of the power-aware Tcl commands to control your power analysis. Such commands include the [SetClockGatingStyle](#) and [SetClockNet](#). These commands and their functions are described in this chapter.
- Further, you may want to use the [MonitorInstances](#) and [MonitorToggleInstances](#) commands to monitor various design elements. This chapter describes how you would use these commands in your time-based power analysis.

Controlling Your Time-Based Power Analysis

As with all PowerArtist programs, you control the operation of your power analysis by specifying the appropriate commands in your PowerArtist command file or on the command line. To perform either an RTL, mixed, or gate-level time-based analysis, you need to specify the [CalculatePower](#) command with the arguments required for the type of analysis you want to perform. There are arguments that are required for all time-based analyses, some for gate-level only and some for RTL only.

Required Options for all Time-Based Analyses

Use these arguments to provide various types of required inputs:

- -activity_file *your_simulation_file*
 This option specifies an input stimulus file generated due to a functional simulator run. The file may be in FSDB, VCD or IAF (generated by Apache PLI routines) format. You must specify either -activity_file or -vectorless_input_file for an average power analysis. CalculatePower automatically determines the type of the simulation activity file.
- -scenario_file *your_scenario_file*
 This option specifies the scenario file you generated using the [Elaborate](#) command. For more information, see [Getting Your Design into PowerArtist](#).
- -synlib_files {*file_name1 file_name2 ...*}
 Adds the specified file or Tcl list of files to the list of Liberty technology files.

Although not required, you will most likely want to use the `start_time` and `-finish_time` options. For details see, [Setting Timing Windows for Time-Based Power Analysis](#).

`-top_instance top_simulation_instance`

Specifies the full hierarchical name of the top-level module in the simulation hierarchy.

Additional Arguments Required for Gate-Level Only

In addition to the options in the previous section, you must specify the following arguments if you are running a gate-level analysis:

■ `-gate_level_netlist true`

Specify this option if you want to perform a gate-level analysis.

■ `-interval_size seconds`

Specifies the minimum time interval in seconds for which power will be reported. The way in which the time-based analyzer calculates power depends on this value.

If you specify an interval of size $T \geq 1$ ns, the time-based analyzer:

- 1 Breaks up the time-steps in your activity file into buckets of width T .
- 2 Calculates the total dynamic energy for every arc that matches during all time steps processed for that interval.
- 3 Divides the dynamic energy by T .
- 4 Adds in the static power. This is the total power that will be displayed for that interval.

If you do not specify an interval, then PowerArtist performs an instantaneous analysis. For each time step in the activity file, PowerArtist:

- 1 Calculates the total dynamic energy for every arc that matches during that time step.
- 2 Divides the energy by 1 ns to get power.
- 3 Adds in the static power. This is the total power that will be displayed for that time step.

If you specify an interval of size $T < 1$ ns, PowerArtist generates a warning and performs an instantaneous power analysis instead.

Additional Arguments Required for RTL and Mixed Analysis Only

In addition to the required options for all analyses, you must specify the following arguments if you are running an RTL or mixed RTL-gate analysis:

■ `-reference_clock clock_name`

Specifies the reference clock that controls when a clock starts and the length of its period.

- `-active_edge auto | positive | negative`

Specifies the edge of the clock that defines the start point for the first interval.

- `positive`: starts the clock when the first clock transition goes to 1.
- `negative`: starts the clock when the first clock transition goes to 0.
- `auto`: starts clock when the first clock transition goes to either 0 or 1.

Default: `auto`

- `-num_clock_cycles int`

Sets the interval size as a number of clock cycles. The interval size is the period of the block into which PowerArtist will split the simulation. It is recommended that you choose an interval greater than 1% of the total simulation time.

Default: 0

For additional options, see the [CalculatePower](#) entry in the *PowerArtist Reference Manual*.

Setting Timing Windows for Time-Based Power Analysis

When performing a time-based power analysis, it is recommended that you make sure that the `-start_time` and `-finish_time` options are set to be the leading edge of a clock and the trailing edge of a clock, respectively. Then you need to select the interval size used by your analysis. This is controlled by the `-interval_size` for gate-level netlists and the `-reference_clock_active_edge` and `-num_clock_cycles` options for everything else.

In general, the interval size should be a multiple of the clock period. The number of clock periods impacts the performance and granularity of your analysis. Smaller intervals take longer but give you more data.

At the gate level, you should, in general, set the `-interval_size` to be no less than one clock cycle. For everything else, you will be setting the `-num_clock_cycles` option. Every interval will require an average power analysis of the entire design. The size should be no less than one clock cycle. The resulting waveforms will look more realistic than if you had used arbitrary values for `-start_time`, `-finish_time`, and `-interval_size`.

Running the Analysis

It is recommended that you use a PowerArtist command file to run your time-based power analysis. This analysis is controlled by the [CalculatePower](#) command.

Sample Command File for Gate-Level Time-Based Power Analysis

```
CalcualtePower -analysis_type time_based \  
-activity_file design.vcd \  
-clock_file clocks.tcl \  
-finish_time 20ns \  
-gate_level_netlist true \  
-scenario_file design.scn \  
-start_time 10ns \  
-synlib_files ../libs/memories.lib ../libs/core.lib ../libs/ios.lib \  
-time_based_report_file design.rpt
```

You can also run this command from the ptshell command line. The following three examples assume that you are running with “-gate_level_netlist true” set; your top_instance, scenario file and Liberty files have been specified. So these have not been shown for clarity.

Example 1

```
CalculatePower -analysis_type time_based -interval_size 1e-9
```

This command performs a time-based analysis breaking your simulation time into 1 ns bins. In addition to generating design.rpt, it will generate a power waveform file in CalculatePower.ptcl. The design.rpt file will contain a simple report for the top-level instance in the design.

Example 2

```
CalculatePower -analysis_type time_based -interval_size 1e-9  
-fsdb_output_file design.fsdb -ptcl_output_file design.ptcl  
-output_current true
```

This command performs the same time-base analysis as Example 1 but will generate current over time waveforms that will be stored in design.fsdb (fsdb format) and design.ptcl (ptcl format).

Example 3

```
MonitorInstances -name top.core1  
CalculatePower -analysis_type time_based -interval_size 1e-9  
-fsdb_output_file design.fsdb -ptcl_output_file design.ptcl  
-output_current true
```

These two commands performs a similar time-based analysis as in Example 2, but the report and the waveform file outputs will include only hierarchical instance top.core1. Note that all children instances will have their values included in the

information generated for top.core1. This is controlled by the [MonitorInstances](#) command. For further control, you can also use the [MonitorFast](#) and [MonitorArcs](#) commands.

■ Sample Command File for RTL Time -Based Power Analysis

```
CalculatePower -analysis_type time_based \  
-active_edge positive \  
-activity_file ../design_data/rtl_sim/activities.vcd \  
-calculate_log CalculatePowerTimeBasedBatch.log \  
-default_output_load 3.9e-11 \  
-finish_time 12135580ps \  
-fsdb_output_file $design.TimeBasedBatch.fsdb \  
-num_clock_cycles 20 \  
-power_db_name $design.TimeBasedBatch.pdb \  
-reference_clock top.clk \  
-save_clock_trees_netlist true \  
-scenario_file $design.Batch.scn \  
-start_time 6071580ps \  
-time_based_report_file $design.TimeBasedBatch.rpt \  
-time_based_write_power_db true \  
-top_instance txrx_tst.top1 \  
-use_scan_flops true \  
-wireload_library hvt
```

This example shows you how to perform a time-based analysis on a design that is not entirely gate level. In this case, you do not specify the `-gate_level_netlist true` option.

Understanding and Reviewing Outputs and Results of the Time-Based Analysis

After the analysis is complete, the following outputs are generated:

- A log file whose name you can set using the [CalculatePower](#) `-calculate_log` option.
- A report file containing an ASCII representation of the power analysis results.
- Waveforms in either the FSDB format or PowerArtist Tcl-based format that holds the current and power over time information you requested. Waveforms are not stored in the report file.

Contents of the ASCII Report File

If you specify, “`-time_based_report_file mychip.rpt`”, `CalculatePower` will generate an ASCII report that contains the following information:

- A header section that provides the following information:
 - The date of the power analysis

- The particular version of the software used to perform the analysis
- The options applied during this run
- Summary information on the simulation run including simulation duration, start and finish times
- The power supply values that were used
- The initial input transition time
- Monitor file name
- Wire load mode and model information
- Section 1: Power Contribution. This section is divided into the following sub-sections:
 - Category Power Summary. This group power section lists for each group:
 - The category name
 - Average power for all elements in the category broken into static, dynamic and total power
 - The maximum power in Watts
 - One or more time steps in your simulation at which the maximum power occurred.
 - Instance Power Summary. This section lists for each monitored instance:
 - Average power for all children of the instance (or just the instance itself if it were a cell in a library) broken into static, dynamic and total power
 - The maximum power in Watts
 - One or more time steps in your simulation that the maximum power occurred
 - The instance name at the end of the line because this field may be arbitrarily long.
- Section 2: Detailed Instance Power. This section provides a per instance power consumption report.
- Section 3: Clock Power Consumption. For details on this section, see [Clock Power Consumption](#).
- Section 4: Inferred Buffer Tree Power. If your design required buffer trees be generated for high fanout nets, this section describes the affected nets and how the buffer tree was created.

For a sample time-based power report, you can run through the analysis tutorial and produce either the top.BatchTimeBased.rpt (generated by the command-line flow) or the top.TimeBased.rpt files (generated by the wizard flow).

Generating and Viewing Waveforms

The [CalculatePower](#) -analysis_type time_based command can generate waveforms of current and power over time for the categories and instances that you specified

using the [MonitorInstances](#) command. You have your choice of two formats: FSDB or PTCL.

If you specified the FSDB format (by specifying the `CalculatePower -fsdb_output_file` option) you can load and display the waveforms in PowerArtist using the Apache Waveform Viewer available in the PowerCanvas (select **Tools > Waveform Viewer**). A sample waveform is shown in Figure 81.

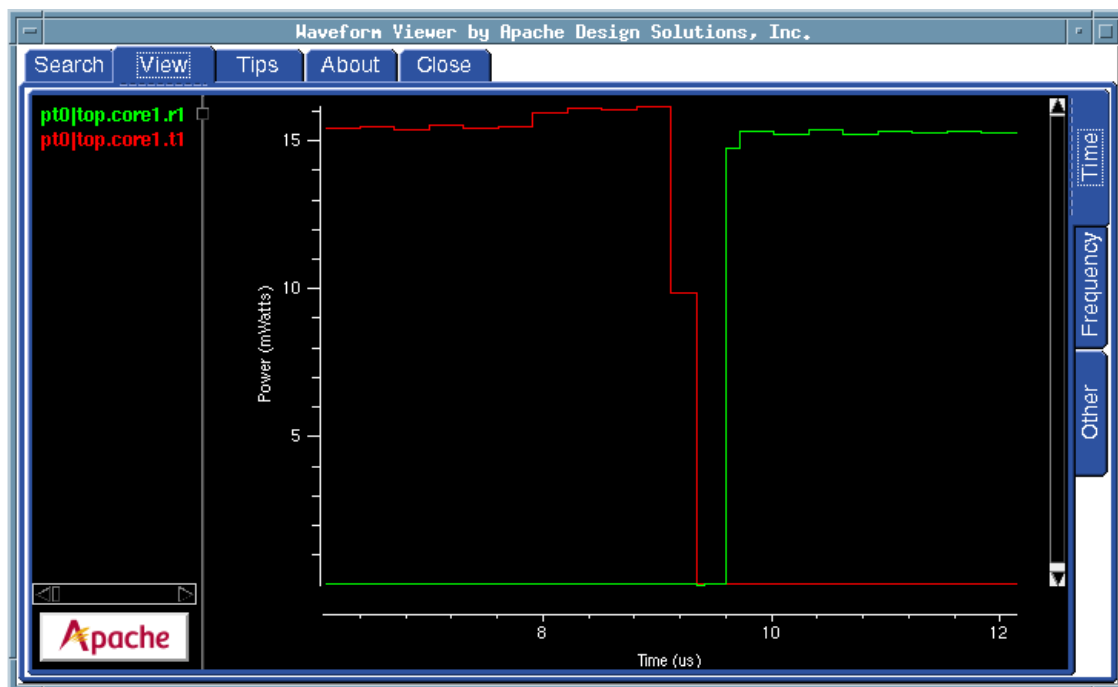


Figure 81 Sample Power-Over-Time Waveform

Measuring the Distance Between Two Data Points

Once you are viewing a particular waveform of interest, you can measure the distance between two data points on the waveform. To do so, you can use the ruler feature as follows:

1. Place your cursor over an area on the waveform and press the “r” button to start the ruler. You can now drag the ruler to any data point on the waveform and mark it using the “m” key. A colored box (same color as the waveform) will appear at the marked data point. This box includes the coordinates of the data point and the delta x (dx) and delta y (dy) values from the first data point to second data point. The dx value = (X of the end point) - (X of the start point), which the dy value = Y of the end point) - (Y of the start point). Also, a copy of the ruler is recorded on the

plot. You can continue measuring the distance to additional data points, or if you are done, simply press the “r” key again to turn off the active ruler. Marked ruler(s) will stay on the plot.

The following figure shows one measurement from a single data point.

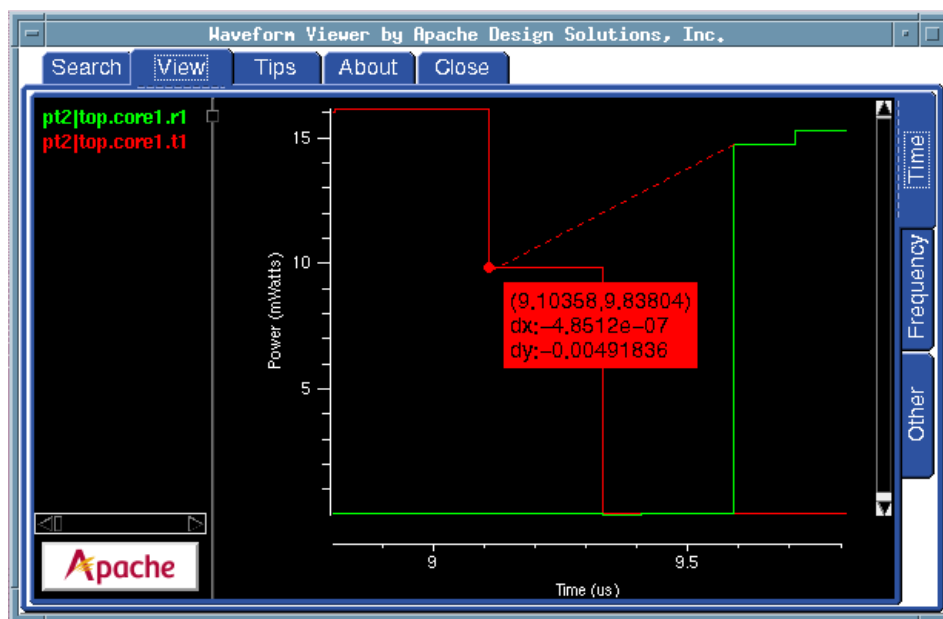


Figure 82 Measuring Distance between Data Points

This feature is useful for presenting data in a presentation or in printed form. If you want to present waveform data in printed form, it is recommended that you change the background color to white (to save ink when printing) by selecting **Toggle background color** from the options menu.

For more information on how to use the waveform viewer, see [Using the Waveform Viewer](#).

If you specified an `-interval_size` that is less than 1ns for a gate-level design, PowerArtist will perform an instantaneous power computation.

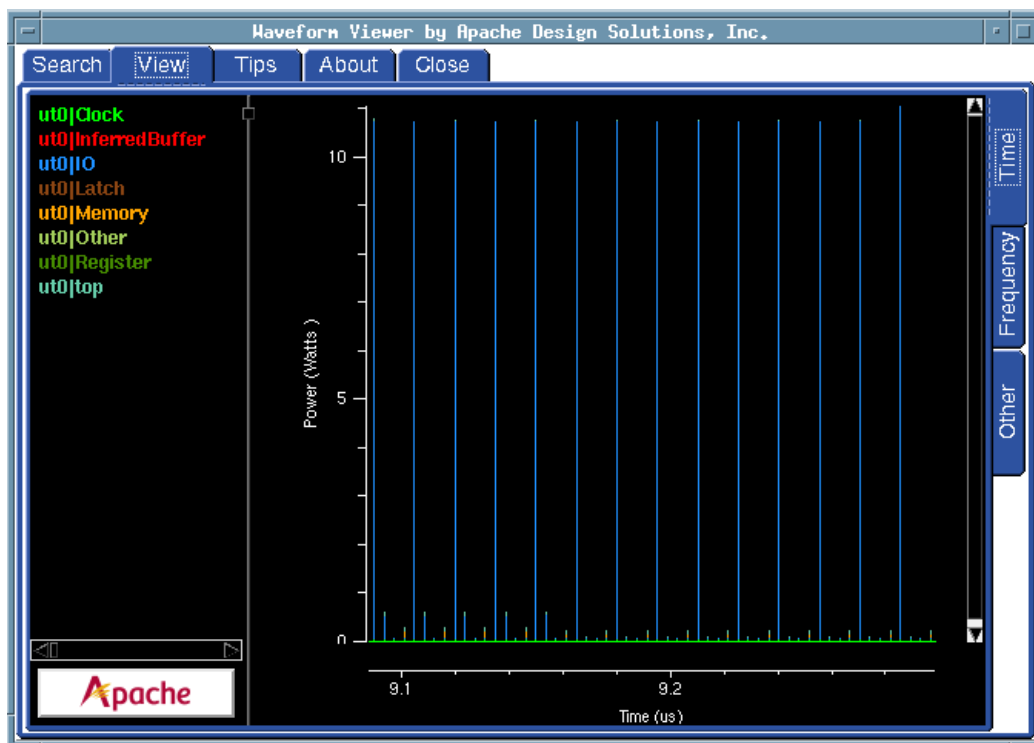


Figure 83 Sample Waveform in PTCL Format for Instantaneous Analysis

If you specified the FSDB format (using the `-fsdb_output_file` option) you can also view that file in the Apache Waveform Viewer (or use Verdi to view your waveforms).

Using Results

You can use the results from a time-based power analysis in a number of ways. First, peak power and current information can be used during the physical design process to size your power busses. By selecting various hierarchical instances in your design that will correspond to physical blocks, you can get a good idea of the power grid needs on a block-by-block basis. Second, the total peak power and current values can give you some idea of the power supply needs your chip will have. Third, by examining areas of the waveform that have large swings in power or current from one time step to the next, you can get an idea of any di/dt issues. The following section describes how you can use the waveforms if you are a CoolTime user.

Using Peak Power or Current Information in CoolTime

Currently, if you want to perform a dynamic voltage drop analysis using CoolTime, you must study the activity of your simulation trace using PowerArtist's vector analysis. You will typically choose the clock cycle that has the highest activity as your clock cycle of interest. This generally will correspond to your peak power cycle as well.

Using the time-based power analyzer, you can view waveforms that show either power or current over time and determine the best clock cycle to analyze. You can then use this information when performing your vector analysis to select the cycle of interest.

Monitoring Flop Clock Activity

Flop clock activity (FCA) is a measure of the efficiency of the clock gating in your design and can be monitored using time-based power analysis. The FCA feature monitors the activity of clock pins on registers in your design. A low FCA number implies that clocks are well gated. Conversely, a high FCA number indicates that the clocks have not been well gated (due to either inefficient enables or non-enabled registers) or the mode captured in the simulations requires clocks to toggle.

The FCA feature works at the RT level of abstraction only for both feedback MUX topologies and for instantiated clock gates. The reports are in the form of PTCL graphs (flop clock activity and average flop clock activity as a function of time) as well as a text version which includes minimum, maximum and average counts of flops whose clocks toggled during an interval expressed as a number and a percentage. It also includes a total flop count per hierarchical instance per clock domain. You can specify a list of hierarchical instances that you want FCA to monitor using the [MonitorToggleInstances](#) command as shown:

```
MonitorToggleInstances -instances {top top.core1 top.core1.u1  
    top.core1.t1 top.core1.r1}
```

This example tells PowerArtist to monitor the five specified instances. It then generates results for each hierarchical instance you specified per clock domain as a waveform in either the PTCL or FSDB file you specify.

To use this feature you must specify the following [CalculatePower](#) options:

```
-analysis_type time_based  
-num_clock_cycles int  
-reference_clock clock_name  
-flop_clock_activity file_name_prefix
```

The interval size is the same across all clocks and determined by the arguments to the `-reference_clock` and `-num_clock_cycles` options. The sample results waveforms and text file on the following pages were generated with the full-chip analysis tutorial using the following [CalculatePower](#) options:

```
-analysis_type time_based  
-num_clock_cycles 20  
-reference_clock top.clk  
-flop_clock_activity fca
```

As an example, if you specify “fca” as the `file_name_prefix` argument to the `-flop_clock_activity` option, the time-based analyzer creates the following three files:

- **fca.txt**

This is a text report that provides minimum/maximum/average statistics per clock domain per specified instances. This is for both flop clock activity and average flop clock activity metrics (see definitions below). The report includes both a number and a percentage as a function of total flops. It also reports the total number of flops (bits) for each instance per clock-domain. For a sample text report, see [Contents of the ASCII Report File](#).

- **fca_avg.ptcl**

This is a PTCL graph that shows averaged flop clock activity over time for all monitored clocks and instances. The x-axis is the simulation time and the y-axis is the average clock activity for the inferred register bits contained in that instance. This differs from the `fca.ptcl` in that the numbers here are derated by the total number of flops in the block.

- **fca.ptcl**

This is a PTCL graph that shows absolute flop clock activity over time for all monitored clocks and instances. The x-axis is simulation time and the y-axis is the absolute number of inferred register bits whose clocks toggled for that hierarchical instance in that time step.

It is recommended that you use the two PTCL graphs together to investigate any potential power bugs. You can open both the `fca_avg.ptcl` and the `fca.ptcl` graphs place them both on the View tab as shown in the following sample.

Sample Flop Clock Activity Graphs

The following figure shows both the average flop clock activity graph (on top) and the absolute graph.



Figure 84 Sample Waveform in PTCL Format for Instantaneous Analysis

Looking at the average flop clock activity in the top graph, you can see that the transmit clock domain (t1_top.clk) in brown is active when the receive clock domain (r1_top.clk) in green is inactive and vice-versa. You can also see that the pci clock domain (top.pci_clk) is always on, indicating that this could be a good candidate for clock gating. To determine whether it's worth your time to clock gate the pci clock, you would need to check the absolute waveforms (displayed on the bottom half of this figure). Looking at them here, you can see that there are approximately 20 flops being clocked in the receive block (ut0|top.core1.r1_top.pci_clk) and even more in the transmit block (ut0|top.core1.t1_top.pci_clk). Therefore, this could indicate a power bug. To see the exact values, you can either hover your mouse on the waveforms or read the text report (as shown on the next page).

Sample Flop Clock Activity Report

```
Flop Clock Activity Report
=====
Date : Wed Apr 13 11:21:15 2011
Program : PowerTheater Time Based Engine (64 Bit Linux) 2010.2.3PreAlpha (11 Apr 2011)

Interval Size      : 303 ns
Reference Clock    : top.clk
Number of clock cycle(s) : 20 clock cycle(s)

Min, Max, Average Clocked Flop Statistics
=====

Instance: top
  Clock: top.clk
    Number of Flops      : 453
    Flop Clock Activity:
      Minimum : 206 (45.4%)
      Maximum : 249 (55%)
      Average  : 230 (50.8%)

      Average Flop Clock Activity:
        Minimum : 0.454 (45.4%)
        Maximum : 0.408 (55%)
        Average  : 0.368 (50.8%)

    <snip>

Instance: top.core1.r1
  Clock: top.clk
    Number of Flops      : 77
    Flop Clock Activity:
      Minimum : 1 (1.3%)
      Maximum : 13 (16.9%)
      Average  : 5.98 (7.77%)

    Average Flop Clock Activity:
      Minimum : 0.013 (1.3%)
      Maximum : 0.169 (16.9%)
      Average  : 0.0777 (7.77%)

  Clock: top.pci_clk
    Number of Flops      : 19
    Flop Clock Activity:
      Minimum : 19 (100%)
      Maximum : 19 (100%)
      Average  : 19 (100%)

    Average Flop Clock Activity:
      Minimum : 1 (100%)
      Maximum : 1 (100%)
      Average  : 1 (100%)
```

There are 453 flops defined in hierarchical instance “top” and its children that are driven by top.clock. top and its children.

There existed one or more intervals where:
the minimum # of registers whose clock toggled was 206; the maximum was 249;

On average over all of the intervals, the minimum of # of clocks that toggled was 45.4%; the maximum was 55%; and the average was 50.8%.

```
<snip>

Instance: top.core1.t1
  Clock: top.clk
    Number of Flops      : 77
    Flop Clock Activity:
      Minimum : 1 (1.3%)
      Maximum : 13 (16.9%)
      Average  : 7.12 (9.25%)
    Average Flop Clock Activity:
      Minimum : 0.013 (1.3%)
      Maximum : 0.169 (16.9%)
      Average  : 0.0925 (9.25%)

  Clock: top.pci_clk
    Number of Flops      : 87
    Flop Clock Activity:
      Minimum : 87 (100%)
      Maximum : 87 (100%)
      Average  : 87 (100%)
    Average Flop Clock Activity:
      Minimum : 1 (100%)
      Maximum : 1 (100%)
      Average  : 1 (100%)

<snip>
```


Chapter 12

Analyzing Vectorless Average Power

Flow Overview

You can use PowerArtist to run a vectorless power analysis. To do so, you need to employ the following flow:

1. Run the [Elaborate](#) command to build a scenario file.
2. Create a Vectorless Activity File (VAF) that contains the activity information.
3. Run the [CalculatePower](#) command with the “-vectorless_input_file *file_name.vaf*” option.

Vectorless power analysis is a convenient way to quickly generate “what-if” scenarios that give you a good idea of what your power may be in various situations. Accurate power numbers come from simulation-based analysis.

Creating the VAF File

The VAF is used to specify activity and duty cycle information for critical signals in the design. This section describes the commands used in the VAF. For complete syntax and examples for all of the commands in this section, see the individual command descriptions in the *PowerArtist Reference Manual*.

You can use wild cards for net names and port names in the commands used in the VAF. It may be easiest to create an OADB script that automatically generates the commands. You can start with the sample scripts available in the following directory:

```
$POWERTHEATER_ROOT/examples/OpenAccess
```

Setting the Frequency and Duty Cycle for Clock Nets

You must specify the frequency or activity and duty cycle information for every clock net in the design using the [SetNetStimulus](#) command.

Example

```
SetNetStimulus -net top.clock -frequency 1e+08 -duty 0.6
```

This example sets the frequency for the net top.clock to 100 MHz. It also sets the duty cycle to 0.6 (the default is 0.5).

If you performed clock gating using integrated clock gating cells, you should also set the gated clock output frequency. To set the frequency for gated clock nets, you would use the SetNetStimulus command.

Setting the Frequency for Primary Inputs

You must specify the frequency or activity for all of the primary inputs of the design using the [SetNetStimulus](#) command or the [SetPortStimulus](#) command. If a net is connected to a module, you can set the frequency using either the SetNetStimulus or SetPortStimulus. For the SetPortStimulus command, you specify the hierarchical path to the instance and then the port name. For the SetNetStimulus command, you specify the hierarchical path to the net.

Example 1

```
SetPortStimulus -instance top -port addr -frequency 1e+08
```

This example sets a frequency of 100 Mhz to all bits of port addr of instance top.

Example 2

```
SetNetStimulus -net {top.we} -frequency 1e+6 -duty .8
```

This example sets a frequency of 1 Mhz and a duty cycle of .8. to net top.we.

You may be inclined to set the same frequency for all of your primary inputs and doing so may be fine, depending on the goals you are trying to achieve. However, you should give this careful consideration as setting all of the primary inputs to one value will greatly impact accuracy.

Setting the Frequency for Ports of a Specific Type

You can use the [SetPortStimulus](#) command with the -type option to set the frequency (and duty cycle) on ports of a specific type—input, output or both.

Example

```
SetPortStimulus -port b* -type input -frequency 1e+08
```

This example sets a frequency of 100 Mhz for all input ports that start with b.

Setting the Duty Cycle of Critical Control Signals that are Not Primary Inputs

Suppose that there are critical signals that are tied to a particular value that you want to remain constant. These constant values can be controlled by supplying a duty cycles of 0 (constant 0) or 1 (constant 1).

Example

```
SetNetStimulus -net {top.alu.globalEn} -duty 1
```


This example sets the duty cycle of `top.alu.globalEn` to be a constant 1. Using the `SetNetStimulus` command in this way accomplishes what case analysis does for static timing analysis.

Setting the Activity and Duty Cycle for Busses Driven by Tri-Stated Signals

If a signal has multiple drivers that are tri-states, you should set the desired activity and duty cycle for the bus that represents the combination of all the tri-stated signals using the `SetNetStimulus` command.

Setting the Frequency and Duty Cycle on Leaf-Level Instances

You can use the `SetPortStimulus` command to set the frequency and duty cycle on all of the input and output nets of a specified leaf-level instance.

Example

```
SetPortStimulus -instance inst -port * -frequency f -duty d
```

Setting the Frequency for Memories

You must specify the port frequencies for critical ports on your memories. You can use the `SetNetStimulus` or `SetPortStimulus` commands for this. Setting these is critical because memories account for a significant portion of the power of your chip. Output port frequencies must be set as well to enable correct power calculation and because the `CalculatePower` command cannot propagate activities and duty cycles through memories. Instances in the downstream cone of logic from a memory need these values to compute their power correctly.

Example

```
SetPortStimulus -instance rxchan.dpmem.m0.m1 -port *  
-frequency 2.76e+7
```

This example sets the average frequency for all ports of memory instance `rxchan.dpmem.m0.m1` to 27.6 Mhz.

Handling Blackbox Instances

PowerArtist cannot propagate activity or duty cycles during a power analysis on any instances that were black-boxed during the Elaborate run; therefore, for every output port of the instance, you must specify the activity, frequency, and duty cycle using the `SetPortStimulus` command.

Example

```
SetPortStimulus -instance top.myblackbox -port * -type output  
-frequency 1e+5 -duty .7
```

This example sets a frequency of .1 Mhz and the duty cycle of .7 to all output ports on blackbox instance `top.myblackbox`.

Understanding Common Warning Messages

If you do not use your VAF commands carefully, you may create certain conditions where conflicting settings are being attempted on the same object. In these cases, PowerArtist will generate an warning message indicating the conflict and how it is being resolved. This section provides some examples of conditions (cases) under which you will see certain warning messages.

Example 1: Mixing Commands or Using SetInstanceStimulus Commands

If you specify the SetInstanceStimulus command, you will see a warning message indicating that it is now obsolete. Also, if the command setting conflict, you will see a message that indicates how PowerArtist is handling the conflicting settings. For example, if you had the following command settings:

```
SetPortStimulus -port P1 -frequency f1 -duty d1 -instance inst1
SetInstanceStimulus -instance inst2 -frequency f2 -duty d2
SetInstanceStimulus -instance inst3 -frequency f3 -duty d3
```

you would see the following warning messages:

```
The 'SetInstanceStimulus' command will no longer be supported in a future release.
Please start using 'SetNetStimulus' and 'SetPortStimulus'
```

```
Following commands are trying to set frequency/activity/duty values on the same net 'N'
SetPortStimulus for inst1.p1 frequency 'f1' duty 'd1' ( command 1 : foo.vaf)
SetInstanceStimulus for inst2 frequency 'f2' duty 'd2' ( command 2 : foo.vaf)
SetInstanceStimulus for inst3 frequency 'f3' duty 'd3' ( command 3 : foo.vaf)
last specified data, frequency 'f3' duty 'd3', will be used.
```

Example 2: Two Net Names that Resolve to the Same Net

If you have two command settings with net names that resolve the same net you will get an warning message. For example, if you have the following command settings:

```
SetNetStimulus -net top.a -frequency f1 -duty d1
SetNetStimulus -net top.module.a -frequency f2 -duty d2
```

you will see the following warning message:

```
Following commands are trying to set frequency/activity/duty values on the same net 'N'
SetNetStimulus for top.a frequency 'f1' duty 'd1' ( command 1 : foo.vaf)
SetINetStimulus for top.module.a frequency 'f2' duty 'd2' ( command 2 : foo.vaf)
last specified data, frequency 'f2' duty 'd2', will be used.
```

Example 3: Conflicting Value Setting for the Same Object (Net/Port)

If you have two commands that are trying to set different values on the same object (net/port) you will get a warning message. For example, if you have the following command settings:

```
SetPortStimulus -port P1 -frequency f1 -duty d1 -instance inst1
SetPortStimulus -port P1 -frequency f2 -duty d2 -instance inst1
```

you will see the following warning message:

```
Following commands are trying to set frequency/activity/duty values on the same net 'N'
  SetPortStimulus for inst1.P1 frequency 'f1' duty 'd1' ( command 1 : foo.vaf)
  SetPortStimulus for inst1.P1 frequency 'f2' duty 'd2' ( command 2 : foo.vaf)
last specified data, frequency 'f2' duty 'd2', will be used.
```

Example 4: Conflicts Caused by Wild Cards

If you use a wild card with your commands, you could create a situation where there are conflicting settings for the same object. For example, if you have the following command settings:

```
SetPortStimulus -port P* -frequency f1 -duty d1 -instance inst1
SetPortStimulus -port P2 -frequency f2 -duty d2 -instance inst2
SetNetStimulus -net N - frequency f3 -duty d3
```

you will see the following warning message:

```
Following commands are trying to set frequency/activity/duty values on the same net 'N'
  SetPortStimulus for inst1.P* frequency 'f1' duty 'd1' ( command 1 : foo.vaf)
  SetPortStimulus for inst2.P2 frequency 'f2' duty 'd2' ( command 2 : foo.vaf)
  SetNetStimulus for N frequency 'f3' duty 'd3' ( command 3 : foo.vaf)
last specified data, frequency 'f3' duty 'd3', will be used.
```


Chapter 13

Examining and Implementing Power Reduction Opportunities

Introduction

Just as software engineers use performance analysis tools to determine their program's run-time bottlenecks, you should use PowerArtist to determine your power bottlenecks. The tree display in PowerArtist's PowerCanvas user interface allows you to quickly see where the power in your design is going.

Once a software engineer determines how to change an algorithm to improve performance, he/she relies on an optimizing compiler to increase performance. The same holds true for power. Once you change your algorithm to significantly reduce power, you can use PowerArtist and tools later in the design flow to further reduce your chip's power.

PowerArtist contains power reduction modules called "PowerBots". These PowerBots scan your design looking for a specific design feature, such as an enable signal that can be constructed to turn off unobserved register toggles. A PowerBot performs an analysis of the topology of your design and activities on nets and then makes recommendations on potential changes you should make to your design. It reports either power savings numbers or power wastage numbers to help you make your decision. If it can determine an example change to make, it will provide code snippets as well. Each of the available PowerBots are discussed in detail in this chapter.

For a hands-on example of running power reduction, it is highly recommended that you run through the [PowerArtist Tutorial Part II: Power Reduction](#) chapter.

Chapter Organization

The following topics are covered in this chapter:

- [Recommended Flow for Implementing Power Reductions](#)
- [Performing Power Reduction Analysis](#)
- [Viewing Reduction Results](#)

- [PowerBot Overview](#)
- [Power Reduction PowerBots](#)
- [Power Linter PowerBots](#)
- [Rewriting Your RTL](#)
- [Running Hierarchical Rewrite](#)

Recommended Flow for Implementing Power Reductions

It is recommended that you use the following flow when implementing reductions:

1. Run the [Elaborate](#) command (or the HDL Inferencing wizard) to perform logic inferencing on your design.
2. Perform vector analysis using PowerArtist on the available RTL simulation data and inspect the activity profiles. You can either use a scripted flow that includes the [GenerateActivityWaveforms](#) command or the Vector Analysis wizard. For details see, [Running an RTL Full-Chip Power Analysis Using Command Files](#) or [Running Vector Analysis Using the Vector Wizard](#).

3. Choose a set of vectors and intervals with a reasonable level of activity that is representative of normal device operation.

The vectors you choose for power reduction analysis with PowerArtist would be different from those you might choose to stress test the power distribution network with voltage drop analysis. For the latter you might choose to maximize activity, but if you used these vectors for power reduction they would leave PowerArtist with fewer opportunities on which to apply the LNR reduction to registers with normally low activity. These high-activity vectors might also cause existing clock enables to always be enabled, therefore, propagating them forward with Prism would appear to save little power. On the other hand, if you choose a vector set with too little activity PowerArtist might insert too many clock gating opportunities, which would cause physical layout problems for clock networks at the P&R phase or may increase power consumption.

4. Run the [ReducePower](#) command (or the Power Reduction wizard) with the vector set(s) you chose.

You will likely run reduction more than once. On your initial run, enable all reductions. PowerArtist will attempt to select the power reductions that will provide the optimum power savings. For example, it will choose the ODC reduction in preference to LNR. Also, by enabling all reductions you will see the power savings that are likely in memories and, potentially, any power you might be wasting (with the linters) due to inefficient clock enables. After you get a profile of the potential power savings from each class of reduction, you can re-run PowerArtist and disable reduction classes you don't want to consider.

5. When you are done with your reduction runs, inspect the .srpt file (see [Sample Reduction Report File](#)) in the following manner:
 - a. Take note of the clock gate percentage.

Is it about what you expected? If it's greater than 85%, the clock-based power savings will start to be limited. If the clock gating percentage is very high, it could indicate that you will encounter problems with clock tree synthesis and P&R so you might want to use PowerArtist to see if you could reduce the number of gated registers with only a minimal impact on power savings. You should use the synthesis constraint generation feature of the LEC PowerBot (see Step 3 in [Running PowerArtist Clock PowerBots](#)) to specify registers that should not be clock gated to your synthesis tool of choice.
 - b. Focus on the dynamic power savings in the core to see what is worth doing. If you simply look for the total power savings you might think there is nothing worth doing because real power savings are masked by high static power or high I/O power.
 - c. Take note of which reductions PowerArtist found and which of those were auto-accepted. If you see a lot of savings in your memories, it might be best to redesign the memory architecture and usage rather than rewriting the RTL.
6. If you want to control which reductions are rewritten, invoke the PowerArtist PowerCanvas and use the reduction dialogs to reject or accept reductions for rewrite. You can easily do this by filtering on specific reductions or modules. For details, see [Filtering Reduction Results](#) in the reduction tutorial.
7. Run [RewriteRTL](#) from the command line (or use the RTL Rewrite wizard in the PowerCanvas).

When it is complete, inspect the rewrite log file summary to ensure that a high percentage of reduction opportunities were rewritten.
8. Re-run simulation using the same setup and vectors on the rewritten RTL. You can do this easily because PowerArtist creates a modified Verilog control file that includes both the un-modified and rewritten RTL.
9. Run PowerArtist again to verify the power savings.

Controlling Clock Domains During Reduction

During reduction, you may want PowerArtist to analyze certain clock domains but not automatically rewrite them. The `-reduction_dont_touch_clocks` option to the `ReducePower` command allows you to specify a list of clock domains to be analyzed but not automatically rewritten. This option accepts a Tcl list of hierarchical net names:

```
-reduction_dont_touch_clocks {clock_net1 clock_net2 ...}
```

The effect of this option depends on the setting of the `-gate_clock` option to the [SetClockNet](#) command.

- If you specify “-gate_clock true” then:
 - The LEC PowerBot will not be run for any clock domain. You must set “-gate_clock false” (which is also the default if it is not specified) on all domains in order to run LEC on any clock net. See the [Local Explicit Clock Enable](#) description (“Usage” sub-section) for details.
 - All other reductions that are discovered that can be automatically rewritten will be marked as accepted in the reduction dialogs.
- If you specify “-gate_clock false” then:
 - The LEC PowerBot will run for that clock domain assuming that all other clocks have been marked as “-gate_clock false”.
 - All reductions that are found will not be accepted for automatic rewrite in the reduction dialogs.
- If you specify “-gate_clock true” and “-reduction_dont_touch_clocks” for the same net name, then no reductions will be accepted for automatic rewrite in the reduction dialogs. This behavior allows you to indicate that synthesis-based clock gating will be performed on the net and you don’t want any reductions automatically accepted for rewrite for that net. You will also notice in your *design.srpt* file that reductions that could have been accepted for rewrite are marked as potential reductions. This can be seen in the following .srpt excerpt for the LNR reduction technique:

Core Techniques	Dynamic Saving		Total Saving	

Low activity non-enabled register :				
Auto-accepted (width <= 16bits) :	0W	0.00%	0W	0.00%
Potential :	18.8uW	89.77%	18.7uW	89.52%

In this case, the user had specified “-reduction_dont_touch_clocks clk” which prevented a number of LNR opportunities from being auto-accepted.

If you specify a root clock with the -reduction_dont_touch_clocks option, then not only will that clock not be processed, but PowerArtist will not process any children clocks traceable from that clock.

Performing Power Reduction Analysis

You can run power reduction either from a command file in a scripted flow or using the PowerCanvas wizard (recommended for new users). For details on how to run both flows, see [PowerArtist Tutorial Part II: Power Reduction](#).

Viewing Reduction Results

It is recommended that you review the reduction results using the PowerCanvas. From the main menu, you would select one of the last three items under the **View** menu: Simple Power Reductions, Linter Power Reductions and Advanced Power Reductions. For more information on using these dialogs, see [Examining Simple Power Reductions](#), [Examining Power Linter Reduction Results](#) and [Examining Advanced Power Reduction Results](#) in the reduction tutorial.

PowerBot Overview

The following diagram describes how the PowerBots fit into PowerArtist.

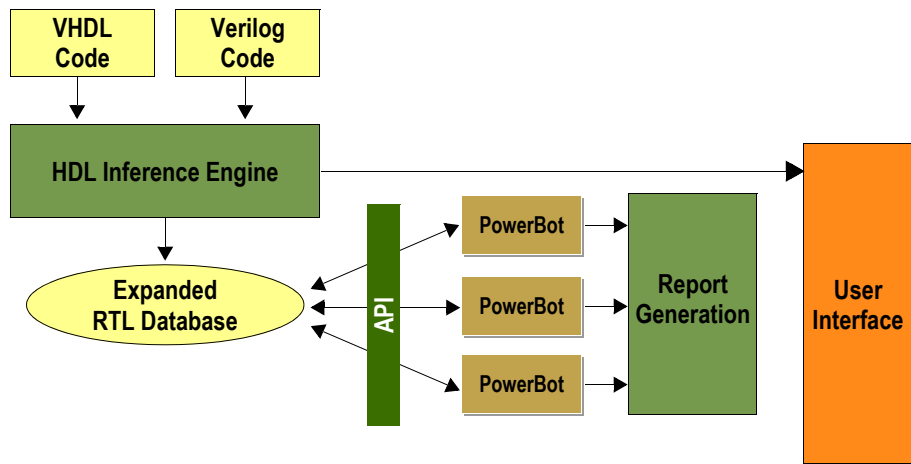


Figure 85 Overview of PowerBots

The following tables lists the PowerBots that are described in the remainder of this chapter. Notice that there are two types of PowerBots: *Power Reduction PowerBots* and *Power Linter PowerBots*.

PowerBot	Acronym
<i>Power Reduction PowerBots</i>	
Low-Activity Non-Enabled Register	lnr
Datapath Operator Isolation	doi
Local Explicit Clock Enable	lec
Split Memory Words (Beta)	smw
Gate Memory Clock	gmc
Prism	fce
Observability Don't Care	odc
<i>Power Linter PowerBots</i>	
Memory Power Linter	mem

PowerBot	Acronym
MUX Power Linter	mux
Register Power Linter	reg
Clock Enable Condition Linter	cec

Running PowerArtist Clock PowerBots

Reducing clock power is one of the easiest opportunities you will have to reduce the power of your design. First, you can take advantage of tools later in your design flow to automatically insert Integrated Clock Gating cells. Second, you can provide these tools constraints indicating which registers should be gated. Third, if your circuit is missing enable signals or has inefficient enable signals, PowerArtist can either recommend design modifications or automatically output new RTL that contains the recommendations. In general, the recommended methodology to run PowerArtist to reduce your clock power is as follows:

1. Leverage existing enables to create new enables.
 - a. Set “gate_clock yes” using the [SetClockNet](#) command for the appropriate nets in your clock file.
 - b. Disable the [Low-Activity Non-Enabled Register](#) PowerBot and run a reduction using [Prism](#), CEC and other PowerBots.
 - c. Perform RTL rewrite (see [RewriteRTL](#) command).
2. Generate completely new enables and leverage those throughout your design.
 - a. Keep the “gate_clock yes” setting to the [SetClockNet](#) command.
 - b. Analyze the RTL written in step 1.
 - c. Run a reduction analysis using both the Low-Activity Non-Enabled Register and [Prism](#) PowerBots.
 - d. Perform RTL rewrite using the [RewriteRTL](#) command.
3. Generate synthesis constraints to tell PowerCompiler the optimal registers to clock gate.
 - a. Set the “gate_clock no” option to the [SetClockNet](#) command.
 - b. Run [Local Explicit Clock Enable](#) PowerBot.
 - c. Execute the [CreateGraph](#) command to generate a cumulative savings versus number of reductions curve to determine which register should be gated.
 - d. Generate the constraints using the [WriteClockGatingConstraints](#) command.

Synthesis constraint generation is described in more detail in [Generating Synthesis Constraints](#).

Generating Synthesis Constraints

The simplest synthesis flows make clock gating decisions based on topological constraints such as minimum register bank bit width. Gating decisions do not take power into consideration. Furthermore, while some register clock gating might save power, the power saving is offset by the increased difficulty to close timing.

In contrast, PowerArtist analyzes the power of your design (when you run the [Local Explicit Clock Enable](#) PowerBot) and determines the optimal registers to clock gate based on the activity files you provide. PowerArtist then generates “-exclude” constraints to prevent PowerCompiler from gating registers when there would be little or no savings. To run this feature, apply the use model described in the following section.

Use Model

To generate clock gating constraints, use the following flow:

1. Run power reduction with the Local Explicit Clock Enable PowerBot. All clocks specified in your clock file must have the “-gate_clock false” SetClockNet option set (this is the default) for LEC to run on any clock.

When you run this PowerBot, PowerArtist identifies the registers that are likely to result in positive savings if the register is clock gated. Before you decide to perform clock gating, you should consider how clock gating impacts timing and physical design closure. If the savings is small, clock gating may have a negative impact on the design as a whole. This can be an important consideration since the bulk of the power savings from clock gating comes from a relatively small number of opportunities. When the Local Explicit Clock Enable PowerBots finishes running, you will have a .pdb file that contains the reductions that LEC considers to be worthwhile.

2. Run the [CreateGraph](#) command from the command line to generate a .ptcl file that you can view using the Apache Waveform Viewer. This command has the following syntax:

```
CreateGraph -class clock [-graph_output_file file_name]
               -power_db_name file_name [-graph_log file_name]
               [-graph_type power_savings]
```

Be sure to specify an output file name to make it easier to select in the Apache Waveform Viewer. Note that you will need to specify the “ptshell -artist” to start the shell and execute the CreateGraph command.

Example

```
% ptshell -artist
ptshell % CreateGraph -class clock -power_db_name my.pdb
               -graph_output_file power_savings.ptcl
```

This command generates a PTCL file where the X-axis is the total number of reduction opportunities and the Y-axis is the cumulative savings opportunity. The reduction opportunities are sorted so that the opportunities that provide the highest savings are closest to the Y-axis. The typical graph will quickly rise and then asymptotically approach 100% of your total savings. You will see quite often that 80% of your total savings are saved in a small fraction of the total gating opportunities.

3. Now, you will need to view the waveform using the following process:
 - a. Type “PowerArtist” to bring up the PowerCanvas.
 - b. From the PowerArtist main menu, select **Tools > Waveform Viewer**.
 - c. Delete the default *.fsdb in the file browser that appears and replace it with *.ptcl and click **Enter**.
 - d. Select the .ptcl file you just created. The waveform will automatically load.
PowerArtist displays the effectiveness curve in the View tab.
4. Examine the graph and look for a good cut-off point where you think the additional clock gating opportunities, while potentially saving power, are not worth the downstream design impacts. This is typically at the “knee of the curve”. The point on the Y-axis is the number you specify with the -constraints_savings option to the

[WriteClockGatingConstraints](#) command.

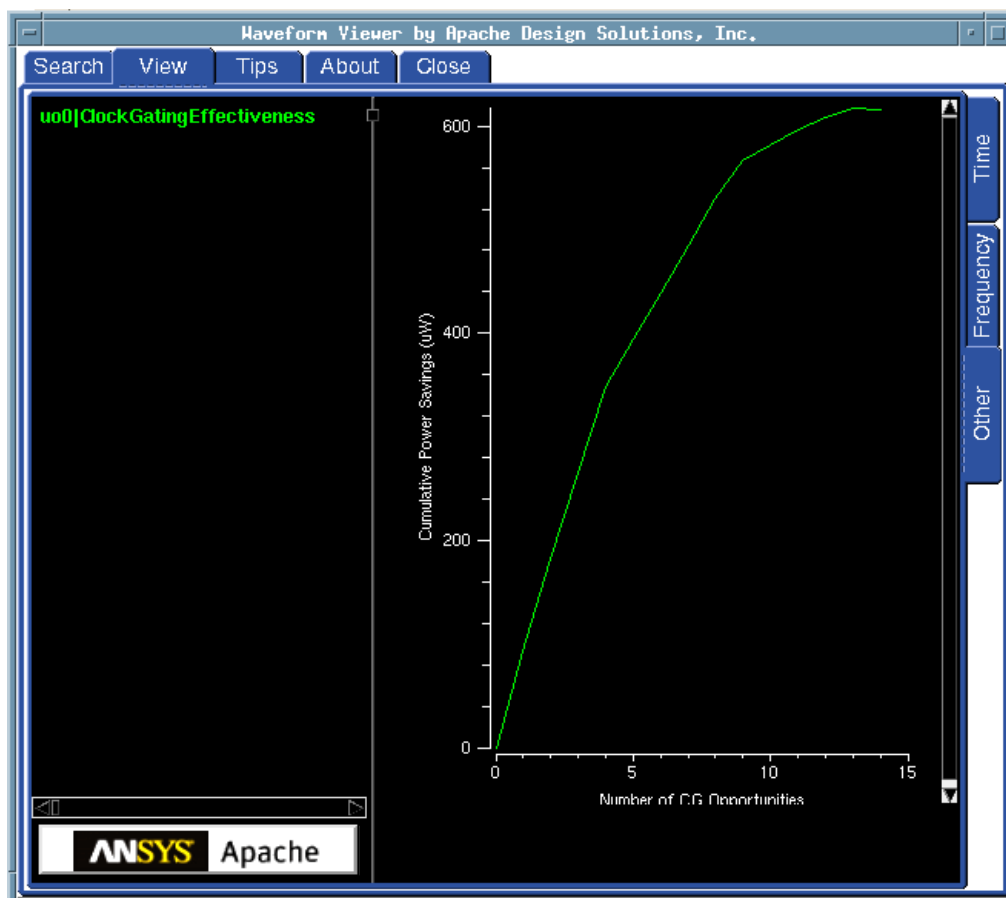


Figure 86 Sample Clock Gating Effectiveness Waveform

In the sample waveform in this figure, the selected point has coordinates (14.0, 618.517). Therefore, you would specify 618.517 as the value to the `-constraints_savings` option to the `WriteClockGatingConstraints` command.

5. Run the [WriteClockGatingConstraints](#) command to generate the constraints. The syntax is as follows:

```
WriteClockGatingConstraints -power_db_name pdb_file  
-constraints_savings float -top module_name  
[-constraints_bus_naming_style string]  
[-constraints_log file_name] [-constraints_output_file file_name]  
[-constraints_synthesis_tool PC]
```

For full descriptions of each of the options for this command, see the description in the reference manual.

Note that whenever you run the ReducePower command (or the Power Reduction wizard) in PowerArtist, reduction data is stored in a power database (.pdb) file. This is the file you specify with the -power_db_name option.

When WriteClockGatingConstraints runs, it creates a constraint file that can be read by your synthesizer. For PowerCompiler™, constraints are in the form of -exclude_instances constraints to the set_clock_gating_registers command.

Command Example 1

```
WriteClockGatingConstraints -constraints_savings 30e-06  
-power_db_name my.pdb -constraints_output_file my.con
```

Given this command, PowerArtist sorts the reduction opportunities from highest to lowest power savings and selects 30 uW as the cut-off point. This means that once the constraints savings value of 30 uW is reached, by the top X opportunities, all other opportunities are excluded from the resulting constraint file. The .pdb file to be analyzed is my.pdb and the constraints will be output in the my.con file.

Command Example 2

```
WriteClockGatingConstraints -bus_naming_style %s_%d -power_db_name my.pdb  
-constraints_output_file my.con
```

In this example, other constraints that you will supply to PowerCompiler will be controlling the bus_naming_style. The -exclude constraints generated by the WriteClockGatingConstraints command must match that naming style. In this case, the naming style is %s_%d.

Sample Output Constraint Files

Example 1: Output using the default bus naming style for specified constraints savings

```
set_clock_gating_registers -exclude_instances { cnt3_reg[11] cnt3_reg[10]  
cnt3_reg[9] cnt3_reg[8] cnt3_reg[7] cnt3_reg[6] cnt3_reg[5] }  
set_clock_gating_registers -exclude_instances { cnt1_reg[15] cnt1_reg[14]  
cnt1_reg[13] cnt1_reg[12] cnt1_reg[11] cnt1_reg[10] cnt1_reg[9] cnt1_reg[8]  
cnt1_reg[7] cnt1_reg[6] cnt1_reg[5] cnt1_reg[4] cnt1_reg[3] cnt1_reg[2] cnt1_reg[1]  
cnt1_reg[0] }
```

Example 2: Output using specified bus naming style for maximum constraints savings

```
set_clock_gating_registers -exclude_instances { cnt3_reg_11 cnt3_reg_10 cnt3_reg_9  
cnt3_reg_8 cnt3_reg_7 cnt3_reg_6 cnt3_reg_5 }
```

In the constraint file, you may see some constraints commented out using the # character, as shown in the following example.

Example 3: Output that includes lines that are commented out

```
set_clock_gating_registers -exclude_instances { q3_reg_11 q3_reg_10 q3_reg_9 }  
# set_clock_gating_registers -exclude_instances { q1_reg[2] q1_reg[1] q1_reg[0] }
```

The commented constraints are those registers that are below min bit width.

PowerArtist does not explicitly exclude these registers because a synthesis tool, with its better optimization, may find them to be part of larger register bank and therefore decide not to exclude them. If PowerArtist were to exclude these registers, synthesis would never be able to include them. They are present in the file only for reference.

The uncommented constraints are those that PowerArtist has determined to exclude because they do not save power and their bit-width is above the min_bit_width.

Power Reduction PowerBots

This section describes, in detail, each of the power reduction PowerBots that are available to you.

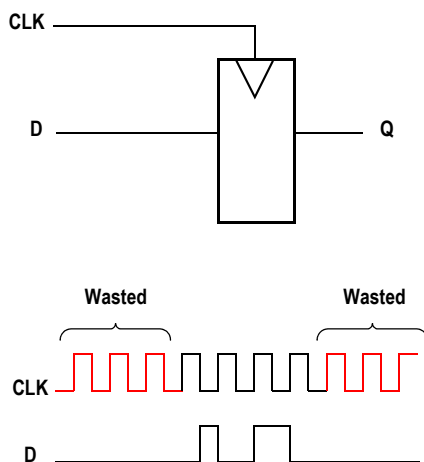
Low-Activity Non-Enabled Register

Design Parts: datapath, clock

The Low-Activity Non-Enabled Register (LNR) PowerBot finds registered buses in the design that change infrequently and are not enabled, and estimates the savings gained if a clock enable is generated by detecting changes on the bus.

Definition

The following schematic and timing diagram shows a register where the data input is active during one phase of operation only, and does not change for a long period of time.

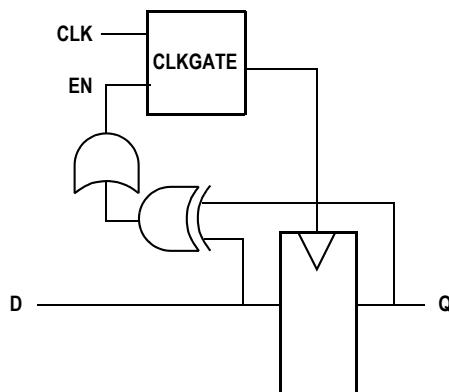


Implementation

In this circuit, power is wasted by the clock driver as well as by circuitry inside the register because most of the clock toggles are not needed. That is, driving the clock to this register when the data is not changing does not change the circuit behavior.

One way to construct an enable signal is by detecting actual changes on the bus. In the following circuit, an XOR is used to determine whether the next state of each bit is the same as the previous, and then a single N -bit OR is used to determine if any bits changed. If no bits on the bus are about to change state, then there is no reason

to enable the clock. The clock gate at the top of the schematic safely disables the clock without allowing any glitches on the detector output to reach the register clock.



For example, if the original code for which LNR is flagged is as follows:

```
always @(posedge clk)
```

```
    q <= d;
```

You can generate an enable “en” by modifying the code as follows:

```
en = | (q ^ d);
```

```
always @(posedge clk)
```

```
    if (en)
```

```
        q <= d;
```

Trade-offs

Clearly, there can be a large area impacted by adding the XORs and OR for a wide bus. The levels of logic between the data and the enable can also impact timing. If a change arrives late at the data inputs, it requires some time to propagate into the enable to allow the clock through.

Several variations of this technique can be used to reduce the area and delay impact. The most important variation is using human knowledge to locate an existing enable. For example, if a register is only active in one particular mode, there is no reason to build a bus-specific enable—you can simply use the mode control signal as the enable.

A second variation involves the selection of the bits upon which to detect changes. PowerArtist selects bits based on an activity threshold, which may result in the selection of only a few bits out of a bus. Depending on the actual activity of the bus, it may be profitable to enable fewer of the bits, or it may be simpler to understand the result if more of the bits are enabled.

Usage

This PowerBot is enabled by default. To disable it, specify the “-skip_reduction_list lnr” option to the ReducePower command. Note that this PowerBot also works in a vectorless mode.

Generating Debug Information

When you run the ReducePower command, you can generate debug information for the LNR PowerBot by specifying the -reduction_debug_flags option with any of the following values:

- 2, 5, 4—prints debug information for the LNR analysis (to the screen).
- &—generates ASCII output in the lnr.oadb.red file containing a detailed reduction report for LNR.

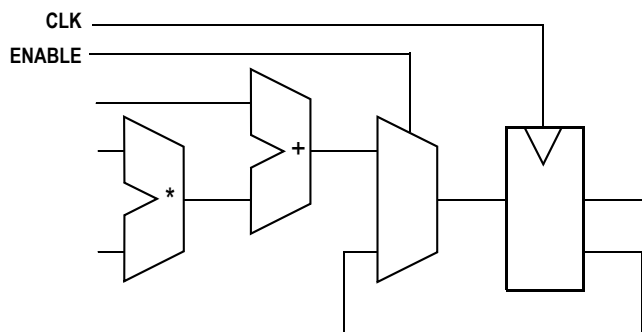
Datapath Operator Isolation

Design Parts: datapath

When a clock enable is present in the RTL, and there are datapath operators such as a multiply or add in front of the enabled register, the Datapath Operator Isolation (DOI) PowerBot estimates the power savings gained from latching the datapath inputs when the output is not being read.

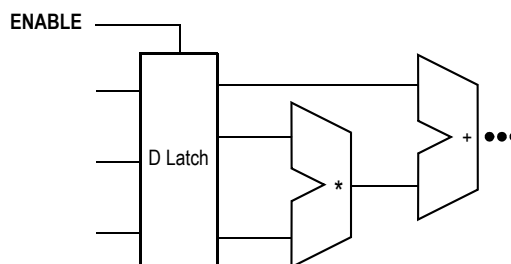
Definition

The following schematic shows datapath operators going into a register that has an explicit clock enable. When the enable is off, the datapath output is ignored. In this situation, the datapath is consuming power to compute a result that is not used.



Implementation

The following schematic shows a way to reduce power for the datapath operators. When the datapath result is not being used, the datapath can be kept quiet by isolating it from its inputs with a latch. The power savings are due to lowered activity in the datapath operators.



Trade-offs

When this change is made, the power dissipated in the datapath goes down. There is a penalty due to the latches that are added. These latches consume a small amount of power and area, and they also add some delay to the combinational path. If the datapath is on a critical path, this extra delay may not be acceptable.

Usage

This PowerBot is enabled by default. To disable it, specify the “-skip_reduction_list doi” option to the ReducePower command. Note that this PowerBot also works in a vectorless mode.

Generating Debug Information

When you run the ReducePower command, you can generate debug information for the DOI PowerBot by specifying the -reduction_debug_flags option with any of the following values:

- 6—prints debug information for the DOI analysis (to the screen).
- &—generates ASCII output in the doi.oadb.red file containing a detailed reduction report for DOI.

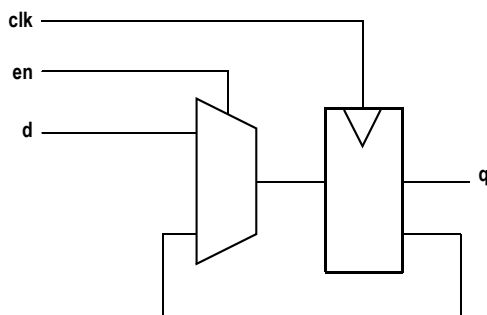
Local Explicit Clock Enable

Design Parts: control, clock, datapath

For clock enables already present in your HDL, the Local Explicit Clock Enable (LEC) PowerBot estimates the power savings gained by using a gated clock rather than a mux in a feedback loop.

Definition

The following schematic shows a local explicit clock enable. The q output is updated on a rising edge of clk, but only when the signal en is high.



Example in Verilog

```
module enable_example (d, clk, en, q);
    input d, clk, en;
    output q;
    reg q;
    always @(posedge clk)
        if (en)
            q = d;
endmodule
```

Example in VHDL

```
entity enable_example is port (
    d, clk, en: in std_logic;
    q: out std_logic);
end enable_example;
architecture rtl of enable_example is begin
    process (clk) begin
        if (clk'event and clk = '1') then
            if (en = '1') then
                q <= d;
            end if;
        end if;
    end process;
end architecture;
```

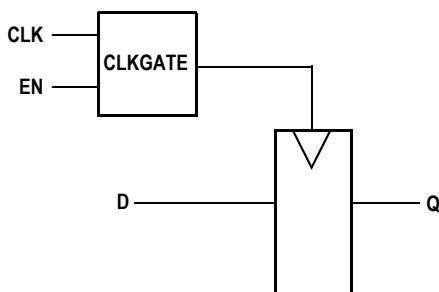
```
end if;  
end process;  
end rtl;
```

Implementation

Normally logic synthesis will implement the indicated circuit with a 2:1 mux, or a muxed D flip-flop if there is one available in the target technology. If the enable is low for a significant percentage of the circuit operation, and if d and q are multi-bit buses, then a substantial amount of power dissipated by the clock driver is wasted. That is, driving the clock to this register when the enable is low does not change the circuit behavior.

Another way to implement this circuit is to gate the clock. Simply replacing the clock input to the flip-flop with an AND gate whose inputs are the clock and the enable is not safe. Consider the situation when the clock is high and transitions occur on the enable: edges will appear on the register clock input, possibly causing incorrect circuit behavior.

To avoid these edges, an Integrated Clock Gating Cell (ICGC) must also be inserted so that when the clock is high, no activity on the enable will be transferred to the clock input. The following schematic illustrates the needed additional circuitry.



Many logic synthesizers have an ability to insert ICGCs into your logic and automatically replace the feedback multiplexor. PowerArtist can be used to predict that style of clock gate insertion using the [SetClockGatingStyle](#) command in your clock file.

If you want to control clock gate insertion, then you have two choices. You can manually insert your own clock gates or you constrain your synthesizer to clock gate only those registers you have chosen.

Manually Controlling Clock Gate Insertion Via RTL Code Changes

The following two code fragments illustrate the required changes for Verilog and VHDL, each showing the instantiation method where it is assumed that the required gate is CLKGATE.

The following code fragment is in Verilog, instantiated:

```
module enable_example (d, clk, en, q);
    input d, clk, en;
    output q;
    reg q;
    wire local_clk;
    CLKGATE cg1 (clk, en, local_clk)
    always @(posedge local_clk)
        q = d;
endmodule
```

The following code fragment is in VHDL, instantiated:

```
architecture rtl of enable_example is
    component CLKGATE ...
    signal local_clk, en: std_logic;
begin
    cg1
    CLKGATE (clk, en, local_clk);
    process (local_clk) begin
        if (local_clk'event and local_clk = '1') then
            q <= d;
        end if;
    end process;
end rtl;
```

Caution

This type of zero-delay adjustment of the clock could lead to simulation differences, especially in VHDL. Consider register A driving register B, where B has a clock gate. In the original non-gated version, A and B are activated on the same simulator delta, and data will require two clock cycles to flow from the input of A to the output of B. In a zero delay environment, if B is gated, the clock to B appears one delta after the clock to A, and data may accidentally flow from the input of A to the output of B in only a single cycle.

To avoid this problem, you might want to use a unit delay environment, where each assignment includes a delay, such as

```
q = #1 d;
```

in Verilog or

```
q <= d after 1 ps;
```

in VHDL.

Trade-offs

When this type of design change is made, the power dissipated on the clock edge goes down. Part of this power decrease occurs in the register itself, and part occurs

in the clock driver. At the same time, the new clock gating circuitry consumes power. PowerArtist estimates the power due to both of these factors. If the power of the new circuit is higher, no change is suggested. In general, for enables that are mostly active, and bus widths that are small, the power of the clock gating circuitry outweighs the possible savings. PowerArtist suggests this change usually for wide registers that are frequently disabled.

Other trade-offs involve area and timing. The area trade-off is similar to the power trade-off. The clock gating circuitry is added, but offset by the savings in either a simple flop instead of the larger muxed flop, or the removal of the mux for libraries without muxed flop cells. The key trade-off is delay. This design change can actually improve the delay along the path of data, because the mux has been removed. However, it could have an impact on the clock distribution.

In general, advanced software tools should be able to incorporate this logic into the clock tree with little impact on clock skew. However, there is a risk that some clock buffer insertion methods might not be able to hide the effect, resulting in additional clock skew. It is recommended that you perform static timing analysis after clock buffer insertion to ensure that no timing problems have appeared.

Usage

By default, this PowerBot simulates clock-gating by building a cell from your power libraries using an AND and a LATCH as a clock gating cell. You will get this behavior if you do not specify clock gating commands in your Tcl script for your reduction analysis runs.

If you want to perform clock gating using an ICGC, apply the following guidelines:

- Use the [SetClockGatingStyle](#) command to specify your selected clock gating cell type. Note that the LEC PowerBot ignores the `-min_bit_width` option if it is present.
- Use [SetClockBuffer](#) commands to define the clock buffers to be used to build a buffer tree. The tree could be used to reduce the load on the gated clock output of the ICGC. If no [SetClockBuffer](#) commands are provided, then no buffer tree will be created.
- Do not specify [SetClockNet](#) commands or specify the `“-gate_clock false”` option to each command if they are present; otherwise, the LEC PowerBot will be disabled. Consequently, you must be careful when using the same clock file for the LEC PowerBot while you are doing power analysis with automatic clock gate prediction.

The LEC PowerBot reports the savings achieved as follows:

The “2-1 multiplexor removal” savings is considered as part of the mux instance saving in the reduction file. The savings in the power of the net from the mux to data pin of the register is also considered as part of the mux instance savings. The register instance savings includes the gating logic power. This is different from the ReducePower flow where it becomes part of the clock power of the design. The

power due to the load on the output of the inserted ICGC will be considered as part of the clock power of the design. The savings in the clock power and the load power is considered as part of the top component savings.

The user interface displays the following information for every savings opportunity reported by the LEC PowerBot:

- Parent Module: the name of module containing the clock-gated register
- Enable net: the clock-enable signal name. If it is an RTL net name defined by the user in the design then it is the boolean expression showing the enable expression.
- Clock net: the clock signal name.
- Duty Cycle: the ratio of the new clock duty cycle to the original clock duty cycle.
- Gated Nets: the gated register net names
- Clock Gate Used: the cell name and the library containing the chosen ICGC

Note that this PowerBot also works in a vectorless mode.

Generating Debug Information

When you run the ReducePower command, you can generate debug information for the LEC PowerBot by specifying the `-reduction_debug_flags` option with any of the following values:

- 2, 6, 7 and c—print debug information for the LEC analysis (to the screen).

Split Memory Words (Beta)

Note on Using this Beta Feature

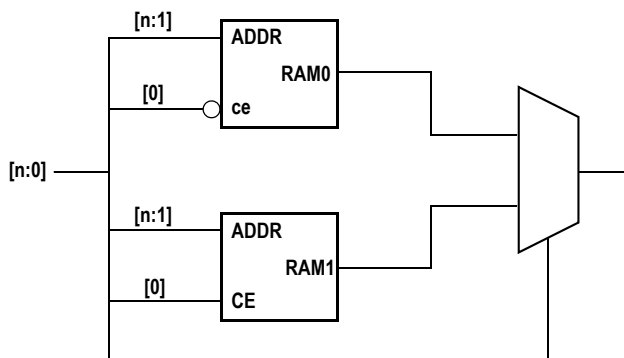
Before trying this beta feature/flow, please contact your Apache Design support team for assistance.

Design Parts: memory

In today's memory circuits, a memory array is usually partitioned into several segments. Only one segment of the array is active at a given point in time, thereby reducing the overall power consumption of that memory. This means that only a few addresses in memory are frequently accessed. Therefore, allocating these frequently executed sequences of memory references into smaller-size, low-power memories reduces the total energy of the memory.

Definition

Currently the Split Memory Words (SMW) PowerBot is designed to reduce dynamic memory power by splitting a memory into smaller symmetrical parts and asymmetrical parts when the `-reduction_max_memory_split` [ReducePower](#) option is set to an odd value. To be considered as a potential candidate for memory splitting, the smaller memories must have identical power rail definitions as the original large memory. The following schematic shows a memory with a large number of words split into two smaller memories, each with half the number of words.



Assume that the power of the full-size memory is F_d (dynamic) and F_s (static) so that the total power (P_f) of the full-size memory is:

$$P_f = F_d + F_s$$

The SMW PowerBot looks for a memory in the .lib that is half the size of the original memory and calculates the dynamic and static power of this half-size memory—call them H_d and H_s . It assumes that only one of these half-size memories is active at any time so that the total power (P_h) of both memories is calculated to be:

$$P_h = H_d + 2 * H_s$$

This means that the total power of both memories is one half-size dynamic power plus two times the leakage of the half-size memory. If SMW does not find a half-size memory in the .lib, uses the values defined using the [DefineHalfMemScalingFactor](#) to scale the static and dynamic power values of the full-size memory to approximate a suitable Ph.

PowerArtist also accounts for the power consumed by the additional circuitry (Pc). The total power consumed by the additional circuitry and the split memories (Ph) is calculated to be:

$$P_t = P_h + P_c$$

SMW accepts the memory substitution only if $P_t < P_f$.

Regarding area calculations, if the half-size memory is found in the .lib, the new memory area is 2 * the area of the half-size memory. The total area includes the area of the smaller memories plus the area of additional instances like a MUX, gates, a register etc. which are part of the additional circuitry that replace the original memory instance. If SMW does not find the half-size memory in the .lib, PowerArtist uses the area penalty scaling factor (defined using the [DefineHalfMemScalingFactor](#) command) to approximate a suitable new area value.

For most memory architectures, the power of a read or write to the half-size memory is far less than the power of a read or write to the full-size memories. Even though the same number of reads and writes are occurring in the new architecture, the power is reduced because each read or write is on only one of the smaller memories.

The SMW PowerBot partitions a memory based on the following considerations:

- the activity on the Most Significant Bit (MSB) and the Least Significant Bit (LSB) of the address bus
- the state of the chip enable—it must be active
- whether there is activity on any other bits of the memory

It selects either the MSB or LSB of the address bus—whichever has lower activity. In the figure on the previous page, bit 0 could be either the MSB or LSB of the address bus. At the same time it checks that the chip is enabled and checks that there is some activity on other address bits of the memory. Moreover, the selected MSB/LSB address bit should be in a stable state (that is, the signal maintains the state of 0 or 1) for more than a specified number of clock cycles (as defined by the [DefineMemActivityThreshold](#) command). When these conditions are met, SMW will partition the memory.

To reduce power, SMW splits memories in a tree-like manner. It internally performs analysis for all combinations of memory splits. By default, the ReducePower command splits the original memory into two half-size memories. You can increase the number of splits allowed using the -reduction_max_memory_split option. If you set this option to “3”, then one of the half-size memories is further split into two parts, resulting in a total of three smaller-size memories.

For example, if you have a memory with an original size of 2048x32 that needs to be split into 3 smaller-size memories, PowerArtist will split that memory into the following smaller-size memories: 1024x32, 512x32 and 512x32. PowerArtist will try the two possible split combinations and recommend the one that results in the maximum power reduction. It also reports the area overhead, due to the placement of instances and additional nets in the added circuitry, associated with the recommended split.

Usage

Use the following flow to run reduction using the SMW PowerBot:

1. Set the following environment variable:

```
PT_APPLY_SMW 1
```

This enables the SMW PowerBot for reduction. It is disabled by default.

2. Create a Tcl script that includes the following commands:
 - **DefineMemory**—to define your memory properties
 - **DefineHalfMemScalingFactor**—to define scaling factors for memory splitting. This is optionally required if you have not predefined a complete memory library that includes a variety of memory sizes.
 - **DefineMemActivityThreshold**—to specify the number of clock cycles during which a memory (of a given type and size) must maintain a stable state (1 or 0) for memory splitting. In addition to the MSB/LSB of a memory address maintaining a stable state, some activity must also happen on other address bits. Therefore, this count is the count of clock cycles where there is activity on other address bits. For example, if the MSB/LSB address is in state 0 for 100 clock cycles, but in those 100 cycles, there is activity on other address bits for 10 cycles, this would be equivalent to a threshold value of 10.

If you do not specify the last two commands, PowerArtist will use the default values for them.

3. Create a run script that includes the **ReducePower** command with the appropriate options specified. Two critical options are:
 - **-activity_file**: specifies the file containing your simulation vectors (VCD or FSDB).
 - **-synlib_files**: specifies your Liberty libraries. You should specify memories of varying sizes. In absence of half-size memories, PowerArtist will compute a half-size memory using the value of the **DefineHalfMemScalingFactor** (specified in step 2).
4. Run the **Elaborate** command to generate the scenario file.
5. Source the .tcl script that contains the Define* commands.

6. Run/source the reduction Tcl script that contains the ReducePower command. Internally this performs the following steps:
 - a. Analyzes your simulation input file (generates a GAF file) by using the information in your DefineMemory commands to:
 - Monitor the clocks of all recognized memory instances in your design.
 - Collect a variety of data for your memories.
 - Write out statistics on your memories into a side Tcl file (.stcl), which is read in the next step. SMW entries in the .stcl file begin with “splSMWLint”.The GAF file is required because memory splitting will only happen when the memory activity profile indicates that a particular memory instance is a candidate for memory splitting. ReducePower will then run the SMW PowerBot, along with the other PowerBots, with the parameters you specified in step 2.
 - b. Performs the power reduction analysis:
 - Reads the data from the side Tcl file you created in step 1.
 - Identifies memory splitting opportunities.
 - Calculates power savings based on the memory splitting opportunities.
 - Writes reduction data into a textual report and the power data base.
7. Review the reduction data either by reading the text report or viewing it as part of the Simple Reduction Dialog as one of the potential SMW opportunities.
8. Run subsequent **ReducePower** commands with the “-use_existing_gaf true” option set. This option tells PowerArtist not to re-run the GAF generation process, thereby reducing the time to run reduction. You can apply different combinations of the SMW-related parameters for each reduction run to see the effects of different types of memory splitting. These parameters include: the -max_mem_split command-line option, and the **DefineHalfMemScalingFactor** and **DefineMemActivityThreshold** commands.

The SMW PowerBot is disabled by default. If you want to run SMW, you need to specify the skip_reduction_list variable as follows:

```
pt_set skip_reduction_list { }
```

Furthermore, if you want to disable other PowerBots and the SMW PowerBot, you need to list all of them explicitly, for example:

```
pt_set skip_reduction_list {smw odc}
```

Trade-offs

From an area and layout perspective, this power reduction may increase the routing congestion. Since memories tend to be large and often do not allow routing above them, increasing the number of memory blocks in the design may make the routing more difficult.

From a delay perspective, the mux on the output adds some delay to the path from address change to data output. However, the actual memory access time for the smaller memory will decrease. Generally, the access time savings is more than the delay of a mux, so the path will become faster. However, if this path is timing critical, you should review the change for its impact on timing.

Understanding the ASCII Output of the SMW PowerBot

The SMW PowerBot automatically generates the smw.reduction.analysis.red output ASCII report file. This file reports SMW reduction information for all possible memory split combinations. It contains information for all memory instances present in the design that are candidates for splitting. If a memory is a good candidate for a split, you will see the note:

Power saved. Selected as a reduction opportunity.
appended to the end of the Total Power line. If not, you will see the following note:
No power saved. Not selected as a reduction opportunity.

Sample smw.reduction.analysis.red File

```
SMW Reduction Report
-----

SMW analysis for memory instance: 'top.core1.r1.dpmem.m0b.m1'
-----

1.  Original Memory Cell Name      : DP256x32
    Original Memory Size           : 256x32
    Original Memory Inst Total Power : 2.51mW
    Original Memory Inst Total Area : 161029
    Original Memory Library Name    : Memories
    Chip Enable Pin Name           : CENB
    Number of Memory Splits         : 2
    Memory Activity Threshold       : 1
    Total Power                     : 2.49mW (Power saved. Selected as a reduction
                                     opportunity)

    Total Dynamic Power             : 2.27mW
    Total Static Power              : 217uW
    Total Area                      : 264445
    Design Net Area                 : 18760.1
    Design Insts Area               : 245685

Memory Clock Specific Details:
-----
Index  Clk Port Name Clk Polarity  Avg  Cnt   Addr MSB Name Clk Net Name   Addr MSB Net Name
=====
1      CLKB          Pos Edge     1.0  33   AB[7]          CLKB1      AB1[0]
2      CLKA          Pos Edge     1.0  28   AA[7]          CLKA1      AA1[0]
```

Split Memory Details:

SMW analysis for memory instance: 'top.core1.r1.dpmem.m0b.m2'

Memory Clock Specific Details:

Split Memory Details:

Index	Split Mem Cell	Memory Size	Enable	Expression
=====				
1	DP128x32	128x32	CENB2	~AB2 [0]
2	DP128x32	128x32	CENB2	AB2 [0]

Note: The formatting in this sample report has been slightly modified to fit the available area.

Typical Example of Additional Circuitry

The following is an example of the additional circuitry introduced when a memory is split into two smaller memories. In this example, the original ra2shd_4kx32 memory is split into two ra2shd_2kx32 memories.

```
module submod (
    OA, IA, AA, CLKA, CENA, WENA,
    OB, IB, AB, CLKB, CENB, WENB
);
    parameter DATMAX = 31;
    parameter ADRMAX = 11;
    output [DATMAX:0] OA, OB;
    input  [DATMAX:0] IA, IB;
    input  [ADRMAX:0] AA, AB;
    input          CLKA, CLKB;
    input          CENA, CENB;
    input          WENA, WENB;

    wire [204:0] net;
    reg          sel1, sel2;

    assign net[0] = ~AA[0];
    assign net[1] = net[0] | CENA;

    assign net[2] = AA[0] | CENA;

    always @ (posedge CLKA) begin
        sel1 <= AA[0];
    end

    assign net[4] = ~AB[11];
    assign net[5] = net[4] | CENB;

    assign net[6] = AB[11] | CENB;

    always @ (posedge CLKB) begin
        sel2 <= AB[11];
    end
end
```



```
    ra2shd_2kx32 mem1(.CLKA(CLKA), .CENA(net[1]), .WENA(WENA), .AA(AA[11:1]),
    .DA(IA), .QA(net[41:10]),
    .CLKB(CLKB), .CENB(net[5]), .WENB(WENB), .AB(AB[10:0]), .DB(IB),
    .QB(net[73:42]));

    ra2shd_2kx32 mem2(.CLKA(CLKA), .CENA(net[2]), .WENA(WENA), .AA(AA[11:1]),
    .DA(IA), .QA(net[107:76]),
    .CLKB(CLKB), .CENB(net[6]), .WENB(WENB), .AB(AB[10:0]), .DB(IB),
    .QB(net[139:108]));

    assign net[171:140] = (sel1) ? net[41:10] : net[107:76];
    assign net[203:172] = (sel2) ? net[73:42] : net[139:108];

    assign OA = net[171:140];
    assign OB = net[203:171];

endmodule
```

Implementation

To implement this change, replace the single memory with two memories, each having half the number of words. The enable expressions identified in the `smw.reduction.analysis.red` file define the select signal that will operate as a bank select. This bit should also be used as the select input to a new multiplexor on the memory outputs. This MUX selects the appropriate memory bank.

Gate Memory Clock

Design Parts: memory

The Gate Memory Clock (GMC) PowerBot performs one of two methods of power reduction for memories. It first applies Method 1 and then applies Method 2.

Method 1

For this method, GMC determines a way to disable the clock in a redundant read access mode. A read access is redundant when:

- The read data at the memory output does not change, or
- The read data is not observed in the downstream cone of logic. This is an example of an observability-don't-care condition identical to what the ODC PowerBot finds for registers.

For this method to apply, the memories must fit into one of the following architectures:

- Single clock/multi port SRAM
- SRAMs with bit-write support
- Single port register file
- A two-port register file with memory and write enables
- ROM

In addition, a memory must meet the following constraints:

- At least one clock must support a read interface.
- The memory must be synchronous.
- ROMs must be single port
- The memory Liberty model has the required dynamic energy arcs.

If the cell does not meet these criteria, this method will not be applied. GMC will attempt Method 2.

Method 2

For this method, GMC examines your Liberty files and locates models that lack internal clock gating. It then turns off the clock when the memory is not being accessed. This will happen when the memory select signal is not asserted.

Setting the Type of Memory Gating

PowerArtist can gate memories using either an integrated clock gating cells (ICGCs) or a discrete circuit. Most applications will use explicit ICGCs, which you can define in one of two ways:

- You can use the [SetMemoryGatingCell](#) command to define a list of cells from specific libraries that should be used.

- If you do not specify one or more `SetMemoryGatingCell` commands, PowerArtist will use the ICGCs identified by any `SetClockGatingStyle` commands that you specified.

The ICGC are used as part of the algorithms to estimate power savings as well as being rewritten as instantiated cells in your RTL.

If you do not specify either the `SetMemoryGatingCell` or the `SetClockGatingStyle` command, PowerArtist will use a discrete gating circuit for estimation as well as for rewrite. The Rewrite process will create the `pa_modules.v` file and instantiate one of the defined modules as the clock gating logic.

Usage

Use the following process to run power reduction using the GMC PowerBot:

1. Run the `Elaborate` command to generate the scenario file.
2. Use `DefineMemory` commands to define critical ports for the memories used in your design.
3. Optionally, use `SetMemoryGatingCell` commands to define the clock gating cells that should be used for memory clock gating. Alternatively, you can specify the `SetClockGatingStyle` command to define your integrated clock gating cells.
4. Set options for the `ReducePower` command. The two critical options are:
 - `-reduction_dont_touch_modules {module_name1 module_name2 ...}`
Specify a list of memory cell names that you don't want to analyze.
 - `-reduction_memory_disable_edge_detection true | false`
If you do not want GMC to generate address stability checking logic, then set this option to true. The detection circuit created is similar to the XOR-OR based circuit used by the `Low-Activity Non-Enabled Register` PowerBot. In addition, registers are added to remember the previous address.
5. Run the `ReducePower` command. Internally this performs the following steps:
 - a. Analyzes your simulation input file by using the information in your `DefineMemory` commands to:
 - Monitor the clocks of all recognized memory instances in your design.
 - Collect a variety of data for your memories.
 - Write out statistics on your memories into a side Tcl file (`.stcl`), which is read in the next step.
 - b. Performs the power reduction analysis:
 - Reads the data from the side Tcl file.
 - Identifies power reduction opportunities.
 - Calculates power savings using the ICGCs specified using the `SetMemoryGatingCell` or `SetClockGatingStyle` commands.

- Writes reduction data into a textual report and the power data base.
6. Review the reduction data either by reading the textual report or viewing it as part of the Simple Reduction Dialog as one of the potential GMC opportunities.

Calculations Performed

The power savings calculations depend on the memory architecture found by the GMC PowerBot. A redundant memory read analysis performs three critical calculations. First, it estimates the power saved by implementing the potential reduction. Second, it estimates the penalty power to implement the potential reduction. This includes the circuitry needed to generate the enable, the clock tree for the additional registers required for an address line stability check, and clock gating circuitry. Finally it estimates an area penalty.

If the memory architecture does not meet the criteria for a redundant memory read analysis, as described earlier, GMC predicts power savings by gating the memory clock when the memory is inactive. In this case, the savings will be the internal power consumption due to the clock switching in the memory disabled state. If power will be saved, then the memory cell enable signal(s) will be used to gate the actual clock signal(s). The enabled clock(s) will then drive the clock input(s) of the memory.

Reduction is performed using the ICGCs specified using the cells defined by the SetMemoryGatingCell or SetClockGatingStyle commands. If none are specified, the gating logic is modeled using a more traditional circuit consisting of a latch and an AND.

Rewrite Considerations

You can schedule any opportunities found by the GMC PowerBot for either manual or automatic rewrite. It follows the same conventions as the other automated PowerBots like [Observability Don't Care](#), [Low-Activity Non-Enabled Register](#) and Prism.

Prism

Design Parts: registers

The Prism PowerBot looks for chains of registers where a register early in the chain is enabled, registers later in the chain are not enabled, and the enable can be used to gate later registers.

To efficiently use this PowerBot, you should be familiar with the following terms:

- **Clock Gating:** A power efficient implementation where register banks are disabled during some clock cycles. The register feedback loop and the multiplexer are removed. The clock gating circuitry is inserted into the clock path, creating a derated clock that reduces unnecessary register clock activity.

- **Gated Register:** A register bank that has either all of its bits enabled through a feedback multiplexer or has its clock driven by an instantiated integrated clock gating cell.
- **Candidate Register:** A register bank for which both of the following conditions are true:
 - none of its bits have a mux feedback path.
 - its datapath width is greater than or equal to user defined datapath width using `SetDatapathWidth` command.
- **Un-Gateable Register:** A register bank for which both of the following conditions are true:
 - none of its bits have a mux feedback path.
 - its width is less than the user defined data path width. This is not the same as the minimum bit width required for clock gating.

Prism uses the following flow:

1. It tries to find candidate registers in the design that are downstream to gated registers.
2. For such a candidate register, it then tries to determine if the upstream gated register enable signal can be used to gate it.
3. It estimates power savings and penalties to determine the effectiveness of the gating.
4. It writes the results into the OpenAccess database for later viewing.

During inferencing, vectored registers are broken out into their own registers. This makes for easy visualization of data paths. It also recognizes that data paths are often manipulated the same way across all bits.

Prism creates enables for non-enabled registers that feed other registers downstream. Chains of registers that had ungated registers as the first register in the chain will be enabled using XOR enable generation and all subsequent register stages would use a propagated enable. This XOR enable generation capability is the same used by the [Low-Activity Non-Enabled Register](#) PowerBot. In addition, Prism strengthens power-inefficient (weak) enables so that clocks become turned off for longer durations saving more power. It does this by reusing existing enables of upstream registers. This enable quality analysis is the same used by the [Clock Enable Condition Linter](#) PowerBot.

Before running power reduction with Prism, you can define the analysis conditions by providing specific inputs using the `-tcl` option or using the PowerArtist command file. You can set the following conditions:

- First, you can set the data path width. As previously mentioned, a data path is a vectored signal inferred as a register instance. You can specifically indicate how many bits wide a data path must be before it should be considered a candidate register. You do this using the [SetDatapathWidth](#) command. The default width is 8

bits.

- Second, you can specify the names modules you don't want to analyze. To do this use the [SetExcludeModules](#) command (which is similar to the `-black_box_modules` option to the [Elaborate](#) command).
- In addition, you can control the min and max bit widths for XOR enable creation as part of Prism. This is controlled using the `-min_bit_width` and `-max_bit_width` options to the [ReducePower](#) command.

For registers that are not part of a Prism chain and have acceptable power savings:

- Any register `< min_bit_width` wide are completely ignored as an LNR opportunity
- Any register `>= min_bit_width` are written to the reduction database as an LNR opportunity.
- Any register `>= min_bit_width` and `<= max_bit_width` are automatically accepted as an LNR opportunity in the reduction database.

For registers that are part of a Prism chain the behavior remains the same:

- Prism generates an enable for registers `<= max_bit_width`.
- If the register is `>= min_bit_width`, Prism also clock gates it.
- If the register is `< min_bit_width`, Prism will not clock gate it.

If the savings are reasonable, Prism will automatically accept changes.

Usage

This PowerBot is enabled by default. If you would like to disable the it, specify the `"-skip_reduction_list prism"` to the [ReducePower](#) command. This PowerBot also works in a vectorless mode.

Generating Debug Information

When you run the [ReducePower](#) command, you can specify the `-reduction_debug_flags` option with any of the following values:

- `#`—generates ASCII output in the `Forward_{$DataPathWidth}.red` file containing a detailed reduction report for Prism. [ReducePower](#) always writes reduction information to the OpenAccess database. Without specifying the `#` debug flag, you will need to review the results in the PowerCanvas.

Understanding the ASCII Output of the Prism PowerBot

The `Forward_{$DataPathWidth}.red` file allows you to perform what-if experiments with different datapath widths, saving the results in different `.red` files.

The Prism output is organized into five sections: Design Summary, Forward Chains, Register Details, Reduction Report and Cumulative Savings.

Section 1: Design Summary

This section summarizes the impact that making the changes suggested by Prism will have on your design and your development schedule. It tells you the number of register bits in your design, how many of those bits have been gated and how difficult it will be to make changes to reduce power. See the following sample section for details on the information it provides.

```
1. Design Summary
-----

Total number of register bits : 758
Number of gated register bits : 481
Number of gated register bits forming the head of a Prism chain : 224
Number of candidate register bits: 128
  Easy bits (only gated upstream) : 32
  Medium bits (no ungateable count upstream) : 32
  Hard bits (everything else) : 64
```

Section 2: Forward Chains

This section presents all the chains starting from the gated registers of the design. A chain describes the gated register and the downstream candidate registers. The candidate register description captures the following upstream counts:

1. gated count (g). This is the number of upstream gated register links in the chain. They are sources of enables that stand a very high probability of being useful.
2. candidate count (c). This is the number of upstream candidate register links in the chain. These have at least one gate register upstream link.
3. ungateable count (u). The following upstream links are include in the count:
 - a. Primary inputs. If the primary input is a bus, its count will still be 1.
 - b. Ungateable registers. These are ones that have no mux feedback path, have fewer bits than required and whose clock is not driven by a clock gate.
 - c. Complex vendor gates. A vendor gate gets counted once.
 - d. Upstream links to a candidate register that don't have a gated register link.

Consider the following forward chain sample:

```
G0 : top.upper.upReg.#13
  -> C1(top.upper.downReg.#15) |<- 1g, 0c, 0u
  -> C4(top.CReg.#15) |<- 0g, 2c, 0u
G2 : top.lower.upReg.#13
  -> C3(top.lower.downReg.#15) |<- 1g, 0c, 0u
  -> duplicate C4(top.CReg.#15) |<- 0g, 2c, 0u
```

Here, there are two chains:

- G0: top.upper.upReg.#13
- G2: top.lower.upReg.#13

The G# means that G = a gated register. # is an index number so that the chain can be easily referred to when FCE analyzes a large block.

The C1 line, which is indented below G1, represents top.upper.downReg.#15 and shows it has:

- 1g meaning 1 gated register upstream and that would be G0.
- 0c meaning no upstream candidate registers.
- 0u meaning no upstream ungated registers.

Similarly, C3 has G2 as its only upstream dependency.

C4 which represents top.CReg.#15 shows it has 2 upstream candidate dependencies which are C1 and C3. So, unless C1 and C3 can be gated, it can't be gated.

Section 3: Register Details

The Register Details section presents information about the participating instances in the above chains described in Section 2. For a G type instance, it will tell you its width, clock and enable. Consider the following detailed sample data for a gated register:

```
G5: core1.s1.#646
-----
Register core1.s1.wdtmr[0:31] is in module stats
Width: 32
Clock Net: top.clk
Local Clock: clk
Enable Net: top.core1.en_wdtmr, Local Expression: en_wdtmr
Clock Enable duty: 1, Optimal: 0.0512821
Percent clock-gated: 0%, Optimal: 94.8718%
CDC: false
It drives 1 candidate registers in 1 module
```

For the gated register in this excerpt, PowerArtist provides the following information:

- **Register:** the full path to the register and the module in which it is defined (G5: core1.s1.#646)
- **Width:** the number of bits in the register (32)
- **Clock Net:** the full path name to the register clock net (top.clk)
- **Local Clock:** the net name of the clock using its local scope name so you can more easily find it in the source file (clk)
- **Enable net:** the full path to the enable net (top.core1.en_wdtmr) and the Local Expression for this enable net (en_wdtmr).
- **Clock Enable duty:** These are the “normal” and “optimal” duty cycles of the enable signals. The normal duty cycle is the percentage of time the existing enable is at 1. The optimal value is calculated by looking at when the data inputs to the register are changing and algorithmically determining how long the enable would

have to be one to ideally match the data changes. In this example, the normal duty cycle is 1 and the optimal is 0.0512821.

- **Percent clock gated:** the normal and optimal clock gating percentages. This represents the percentage of clock cycles the clock is disabled. The optimal is calculated based on the percentage of clock cycles that data changed and the register would have been enabled. In this example, the normal value is 0% for normal and Optimal is 94.8718%.
- **CDC:** false means this register does not cross clock domains—it is in one clock domain. True would mean that it is in more than one clock domain.

For this instance, it also says that there is only one upstream enable and that one enable is available locally.

For a C type instance, it will tell you its clock and the enable information that can be used to gate the clock of the instance. It also tells you the bits of the data path that can be gated. Consider the following sample for a C type instance.

```
C10: core1.s1.#645
-----
Register core1.s1.dout[0:31] is in module stats
Width: 32
Clock Net: top.clk
Local Clock: clk
Clock Enable duty: 1, Optimal: 1
Percent clock-gated: 0%, Optimal: 0%
CDC: false
It could be gated by OR'ing 5 enable(s).
5 enables are available locally.
```

The information given for this C-type instance is similar to that given for the previous gated register. Further, the details for this instance (C10: core1.s1.#645) indicate that it can be gated by OR'ing 5 enable(s), which are available locally.

Section 4: Reduction Report

The reduction report summarizes each opportunity from a power savings point of view. The columns are:

- **Reg Power:** the power of the register bits before clock gating.
- **Saved Total:** the total power (register + clock - penalty) saved by gating the clock to the register.
- **Saved Reg:** the register power saved by gating the clock to the register. It includes the penalty due to the enable generation and propagation.
- **Saved Clock:** the clock tree power saved by gating the clock to the register. It includes the gating circuitry penalty.
- **Ideal Reg:** the register power saved by gating the clock to the register. It doesn't include the penalty due to the enable generation and propagation.

- **Ideal Clock:** the clock tree power saved by gating the clock to the register. It doesn't include the penalty due to the gating circuitry.

This section is further broken down by Analysis type (either normal or optimal) as shown in the following excerpt.

Analysis type = Normal						
Reg Power	Saved Total	Saved Reg	Saved Clock	Ideal Reg	Ideal Clock	Instance Name
-----	-----	-----	-----	-----	-----	-----
16.2uW	-4.92uW	-592nW	-4.33uW	0W	26.9uW	core1.s1.#645 (C10)
16.9uW	-4.05uW	-591nW	-3.46uW	0W	27.7uW	core1.s1.#644 (C11)
78.7uW	124uW	68.5uW	56uW	69.1uW	56.3uW	core1.pl.#1847 (C36)
Analysis type = Optimal						
Reg Power	Saved Total	Saved Reg	Saved Clock	Ideal Reg	Ideal Clock	Instance Name
-----	-----	-----	-----	-----	-----	-----
16.2uW	-4.92uW	-592nW	-4.33uW	0W	26.9uW	core1.s1.#645 (C10)
16.9uW	-4.05uW	-591nW	-3.46uW	0W	27.7uW	core1.s1.#644 (C11)
78.7uW	124uW	68.5uW	56uW	69.1uW	56.3uW	core1.pl.#1847 (C36)

The Normal section represents the power savings you would achieve given the simulation vectors used to perform the reduction analysis. The Optimal section represents the power savings you would achieve if your enable was “optimal”. You can switch between these values in the Prism dialog in the PowerCanvas For more information on how to use this feature see, [Examining Advanced Power Reduction Results](#).

Section 5: Cumulative Savings

This section provides the cumulative power savings using existing enables and new enables. It also is broken down by analysis type: Normal and Optimal.

5. Cumulative Savings	

Analysis type = Normal	
Using existing enables :	0W
Using new enables :	124uW
Analysis type = Optimal	
Using existing enables :	0W
Using new enables :	124uW

Observability Don't Care

Design Parts: registers

The Observability Don't Care (ODC) PowerBot generates enable signals by examining the topology of your circuit and determining the conditions under which the outputs of your registers are not observable by downstream registers. These conditions are then used as clock enable signals on the upstream register. Depending on your design, this may save a significant amount of dynamic power at the cost of increased area and slight timing impacts (see [Trade-offs](#)).

ODC goes through the following process.

1. It locates register banks that are not clock gated. These are the candidate registers.
2. If the candidate register meets user supplied constraints, it then locates all of the registers in the downstream cone of logic.
3. It then locates all 2-1 muxes, unencoded muxes and tri-states in the paths connecting a candidate register to all downstream registers. These instances form the critical steering logic that determines if a register output can be observed by downstream registers.
4. It examines the select lines for each steering logic instance to determine the conditions under which the register output is not observable by any downstream register.
5. If such a condition exists, it becomes a potential enable for the candidate register. If the signals in the enable expression are all direct outputs of registers and not boolean combinations of other signals, this means that the enable signal can be easily made available “one clock cycle earlier” with no penalty to precompute the logic. If the signals do not meet this “no precompute required” constraint, the opportunity is discarded as a potential ODC candidate.
6. If you are running the analysis with the “-reduction_topology true” setting to the [ReducePower](#) command, then it automatically selects the candidate register and records it in the power database (.pdb). Otherwise, it computes the power savings and power penalties to determine if the opportunity will ultimately save power. If it does save power, it automatically accepts the candidate register and records it in the power database.

You can review the results in the Simple Power Reduction dialog in the PowerArtist GUI to see if you agree with the automatic selections and deselect those that you do not want to accept. As the final step, you will run the RTL rewrite which will generate new RTL that implements the final list of selected candidates. ODC also recognizes opportunities for enable strengthening, but it does not automatically accept them.

Understanding Power Savings and Penalty Calculations in ODC

The ODC enable expression is the OR of all the select signals along one path ANDed with the enable expressions for all other paths. The penalty power that will happen will include:

- The boolean expression required to implement the enable signal.
- The ICGC required to implement the actual clock gate.
- Any buffers required to meet the ICGC fanout.
- Increased capacitive load on the signals in the enable expression.

The power savings comes from:

- Decreased buffers in the clock tree that were driving the candidate register bits directly.
- Reduced clock power in the candidate register due to the reduced number of times the clock pin toggles on all the candidate register bits.
- The savings in the cone of logic downstream from the candidate register due to fewer toggles appearing at the outputs of the candidate register.

Area impacts primarily occur due to the extra circuitry needed to implement the new clock enable and the clock distribution network due to the additional ICGC that will be synthesized.

Usage

This PowerBot is enabled by default and it works in a vectorless mode.

Disabling ODC

ODC is enabled by default when you perform a reduction run. You can disable ODC by doing any of the following:

- To skip this reduction altogether, specify the acronym “odc” with the `-skip_reduction_list` option to `ReducePower`.
- If you want to exclude all clock-related PowerBots, you can exclude “clock” in the `-reduction_classes` option to the `ReducePower` command. This will disable the ODC PowerBot as well as LEC, LNR and Prism.
- To exclude certain registers from consideration by ODC, you can set the following `-reduction_min_bit_width` option to `ReducePower`. The ODC PowerBot ignores registers whose bit widths do not meet the specified minimum bit width constraint. Furthermore, ODC ignores the `-reduction_max_bit_width` constraint.

Trade-offs

As with any clock gating technique, there will be area and delay trade-offs you will have to make versus the power saved. Area and delay may both increase due to the combinational logic that needs to be added to your circuit to create the enable signal.

In general, as long as the number of bits being clock gated is reasonable, which is probably 3 or more, the savings will outweigh the area impact.

Reporting Results

The Simple Power Reduction Dialog displays ODC opportunities. To display only ODC opportunities, simply filter on “Reductions contain ODC”. If you select “Show Downstream Cone” on an ODC candidate register in the Simple Reductions dialog, the schematic will show the downstream cone of the candidate register and also the downstream cones of the flops that participate in the ODC expression.

The .srpt file contains an entry in Section 4, “Power Reduction by Technique” that looks like the following file excerpt:

```
Observability Don't Care :  
    Auto-accepted : Topology based auto-accept#  
    Potential :    92.5uW    6.60%    92.4uW    6.54%
```

Note that this output was attained by performing a topology-based analysis.

Generating Debug Information

When you run the ReducePower command, you specify the `-reduction_debug_flags` option with any of the following values:

- 8—provides details of the ODC power estimation phase. This goes into the .srpt file and gives the Apache Design, Inc. engineering team an idea of how the algorithm performed.
- 0—provides details of the constraint analysis phase of ODC (to the screen).
- +—generates ASCII output in a new file, `ODC_min_bit_width.red`. This file contains a detailed reduction report for ODC.

Handling Overlaps Between ODC and Other PowerBots

The ODC PowerBot will find some of the same registers that other PowerBots may also find. Therefore, when an overlap occurs PowerArtist must select one PowerBot as the final implementation methodology. In making this selection, the following rules apply:

- An ODC reduction takes priority over an LNR reduction. In general, the LNR reduction xor tree has a higher penalty power. Also, ODC works for register banks of any size while it is recommended that you not have LNR consider register banks > 16 bits wide due to a high power and timing penalty.
- Different PowerBots work together to provide more savings than you may otherwise expect. For example, Prism may detect a large savings opportunity but there is no enable signal for the first register in one of the chains. In situations where Prism needs to generate an enable it will use LNR

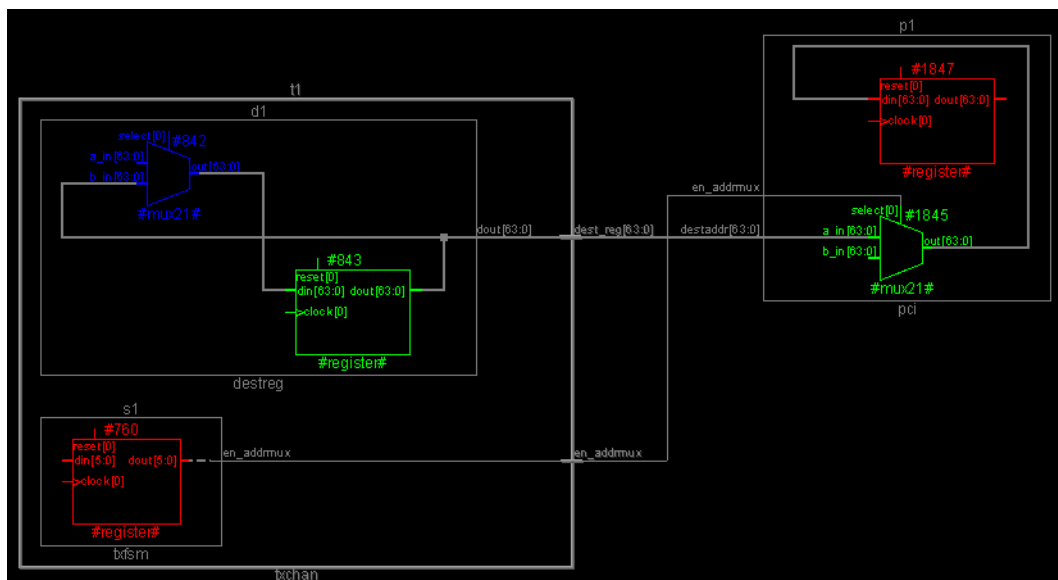
There is a special case. If the register being enabled is the last register in a chain, PowerArtist will compare the power savings between the two implementation choices and choose the one that saves more power.

ODC Flow

In general, after a rewrite where ODC is involved, you should re-simulate your design. Otherwise, when you perform a new power estimation of the rewritten RTL, you will get the original activities in the cone of logic downstream from the candidate register rather than derated activities. This means you will see no savings in the fanout network in the rewritten RTL. Therefore, your final power number will be higher than expected.

Performing Downstream Clock Tracing in ODC

Downstream tracing in the Simple Reduction Dialog for ODC is slightly different than the other reductions. For the other reductions, right clicking on an instance of a reduction and selecting **Show Downstream Cone** will show the downstream cone from the instance pins in the schematic. For ODC you will also see the upstream cones from “steering logic”. Steering logic instances include 2-1 muxes, unencoded mux and tri-state. Their enable or select pins are used in ODC expressions to generate clock gating expressions. The schematic will “zoom fit” to the gates in the steering logic cones to focus your attention on this information. The following figure shows this for the ODC reduction in the tutorial.



In this figure, gate #843 is the candidate register to which ODC gating could be applied. The steering logic is the mux #1845 and it's select pin upstream cone terminates at instance #760.

Power Linter PowerBots

If increased power consumption is the cost of achieving certain logic functionality and meeting timing in a design, then the goal of a low-power design is to achieve the required functionality and timing while consuming the least amount of power. In this case, power dissipation that does not add to functional completeness or aid in meeting (or exceeding) the timing budget is wasted power. PowerArtist's power linter PowerBots help you identify the areas in your design where power is wasted.

Since power analyses at the RT level of abstraction are made through net toggles, monitoring wasted power is equivalent to monitoring extraneous net toggles, and estimating the amount of power saved if the unneeded toggles were eliminated. The PowerBots in this section find the amount of wasted power at the inputs of three very common design components. They determine whether a certain toggle at the input of the device was used, or ignored. Based on the wasted toggle data collected during the reduction analysis process, it generates the wasted power numbers. You can then use the PowerCanvas to display the wasted power results and then decide what changes you may want to make to your design. These results include: an estimate of the amount of power wasted in each case, the kind of wasted power, the instance in which the power is getting wasted, and which nets form the input of that instance. The Mux Power Linter provides some critical additional information. It traces “cones of logic” for the inputs that have wasted power and presents this information in the PowerCanvas.

Limitations of the Linter PowerBots

The power linter PowerBots do not recommend a means of recovering the wasted power, as the method of recovering power depends heavily on the design and test pattern. For example, if the input net to a multiplexor toggles much more frequently than the frequency of its selection, you could reduce the number of toggles using one of several methods—gating the input with the select, changing some upstream logic so that the input data changes only before it will be selected, etc. However, there is no fixed “optimal” solution.

Input Nets with Multiple Instances Connected

If the instance being monitored by a power linter PowerBot is not the only instance connected to an input net, that input net will not be monitored for wasted power because that net's activity could be used by another device. In short, only nets with one source and one sink are considered for extraneous activity monitoring.

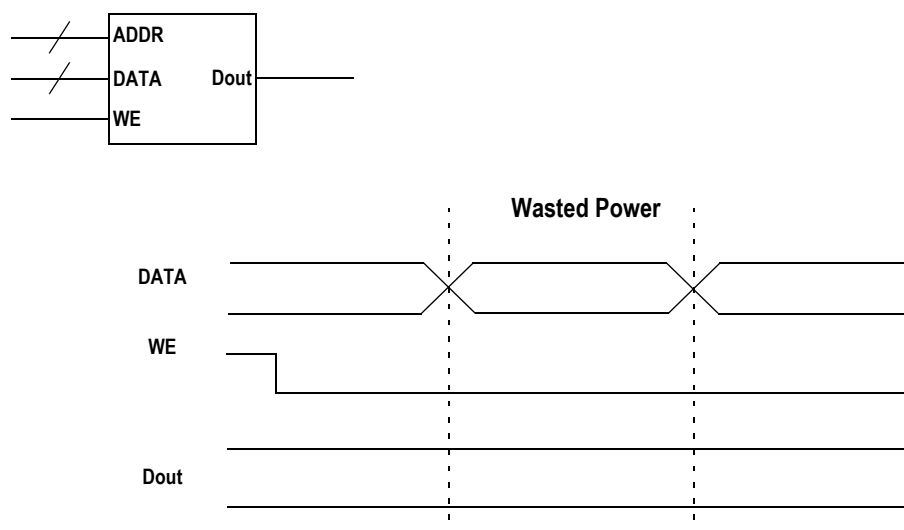
Memory Power Linter

Design Parts: memory

The Memory Power Linter (MEM) PowerBot finds the memories in the design and monitors the data input to the memories to see if changes in the data input ports were wasted because the memory was not selected for a write access. This PowerBot also estimates the amount of power lost due to the wasted activity on the input data bus.

Definition

If the input data bus to a memory changes one or more times and that memory is not selected for a write access before the input data line toggles again, the toggles on the input data bus of the memory can be considered as extraneous toggles, which cause power to be wasted. The following schematic shows data changing at the DATA input to the memory. However, the write enable (WE) to the memory is not valid during the data change and will consequently is not written into the memory at ADDR. Therefore, this extraneous transition of the data causes power to be wasted.



Implementation

Unused toggles on the memory input data bus can be desirable or undesirable. Depending on your design, there could be several ways to implement more efficient circuitry to reduce extraneous activity on the input data bus for the memory. This PowerBot is essentially an analysis tool that points out the potential areas of concern. The analysis is very much design and simulation dependent. The PowerBot will monitor the input data bus of the memory only if the bits of the input data bus and

the write enable are monitored by your simulator. Inferred nets are not monitored for extraneous activity/wasted power.

Generating Debug Information

When you run the ReducePower command, you can generate debug information for the MEM power linter PowerBot by specifying the `-reduction_debug_flags` option with either of the following values:

- `r`—prints details of the MEM analysis to the screen.
- `&`—generates ASCII output in the `mem.pl.oadb.red` file containing a detailed reduction report for the MEM linter.

MUX Power Linter

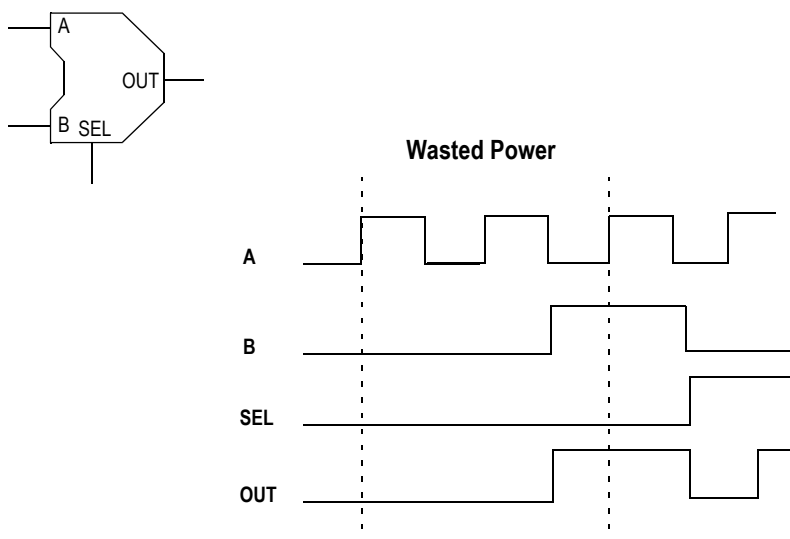
Design Parts: multiplexors

The MUX Power Linter (MUX) PowerBot locates all multiplexors in the design and monitors their inputs to see if certain input toggles are unnecessary due to the input not being selected. It also estimates the power in the cones of logic leading up to the inputs with the wasted power. This estimate allows you to determine whether you will see a significant benefit from reducing the power upstream in the cone.

Definition

If the input net to a multiplexor toggles one or more times and that input is not selected by the multiplexor before it toggles again, the toggles on that input of the multiplexor can be considered wasted. This means that there is activity that consumes power on that pin of the multiplexor, but that activity is meaningless. The following schematic shows a multiplexor which initially selects the B input. The power

consumed by the transitioning that appears on A during the B select period is considered wasted power as it will not be relayed to any other part of the design.



Implementation

The unused toggles on the multiplexor can be desirable or undesirable. Depending on your design, there could be many ways of implementing more efficient circuitry upstream from the multiplexor which would reduce wasted activity at the multiplexor. This PowerBot is essentially an analysis tool that points out the potential areas of concern. The analysis is very much design and simulation dependent.

The PowerBot will monitor an input of a multiplexor only if the associated net is monitored by your simulator. Inferred nets are not monitored for extraneous activity/ wasted power.

Generating Debug Information

When you run the ReducePower command, you can generate debug information for the MUX power linter PowerBot by specifying the `-reduction_debug_flags` option with either of the following values:

- `r`—prints details of the MUX analysis to the screen.
- `&`—generates ASCII output in the `mux.pl.oadb.red` file containing a detailed reduction report for the MUX linter.

Understanding the PowerCanvas Data for the MUX Linter

The PowerCanvas displays two different types of information related to the MUX Power Linter, as shown in the following figure. First, the Linter Reductions Dialog contains special columns in the top pane that provide data that can help you

determine whether the power savings would be worth the effort to implement the power reduction opportunity revealed by the technique. In the tabbed pane under the Detail tab, you will see detailed information that will help you determine the specific changes you will need to make (see the following figure).

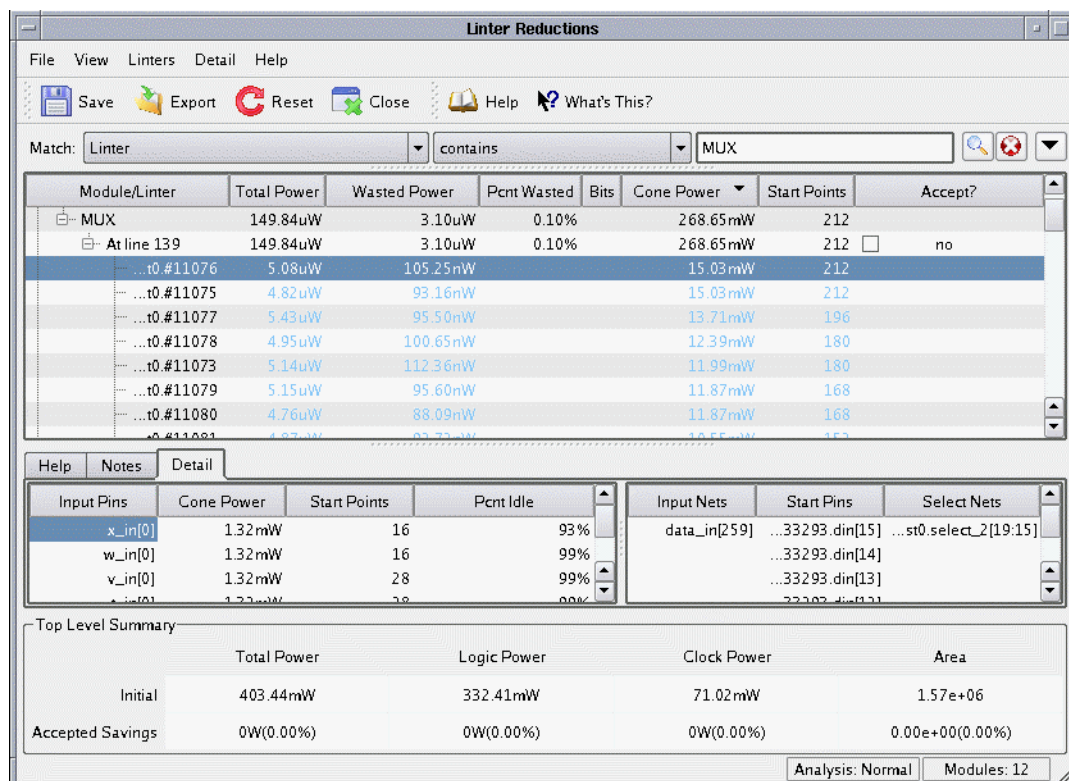


Figure 87 Linter Reduction Dialog Showing MUX Results

After reviewing the dialog data, if determine that there is a reduction opportunity worth investigating further, you can display logic cones in the schematic that will help you to better understand the scope of the changes required. To do this, select an input pin in the Detail tab and use the right-click pop-up menu to display the logic cones.

Linter Reduction Display for the MUX Linter

The MUX Linter points out power that is wasted in muxes due to toggles happening on mux data inputs that never get passed to the mux output because the select line is selecting other data inputs when those toggles happen. There is certainly wasted power in the mux itself. However, the cone of logic that generates the ignored toggles is a potential source of far more wasted power. The data provided in the Mux Power Linter shows you the magnitude of the power that could be wasted.

Information in the Upper Pane

One of the columns for the Mux Power Linter in the top pane is Cone Power. This represents the power of all the instances and nets in one or more exclusive logic cones associated with the instance. An exclusive logic cone is the cone of logic that drives a particular pin on a 2-1 or unencoded mux instance that does not fanout to any other pin.

The cone of logic stops when it reaches a sequential instance (register, latch, register file, latch file, memory), a blackbox instance, a bus pin of vendor gate instance, a primary input to the design or an inferred instance whose input pin is driven by an instance that fans out outside the cone. The cone of logic is traced through the select pins of 2-1 and unencoded muxes and tri-state enable pins.

Larger cone power numbers normally represent excellent opportunities to reduce power. If you are able to suppress toggles entering the exclusive cone, then those toggles will not ripple through the cone and cause excessive dynamic power.

The other column is Start Points. This represents a count of the number of entry points into the exclusive cone of logic. These are the places that logic has to be inserted to suppress toggles in the exclusive cone. Generally, the smaller the number of Start Points, the easier it will be to determine logic to suppress excess toggles.

The start points are determined as follows:

- For inferred registers, latches, register files and latch files, the start point is the din pin that generates the dout value; therefore, if the exclusive cone is traced back to dout[i], tracing will stop at this instance and din[i] will be listed as a start point. For sequential vendor gates, all of the input pins are listed as a start point.
- For a blackbox, tracing will stop at those instances and no start point is listed.
- For a combinational vendor gate, tracing will stop at the vector pin, if any, of the instances and all the input pins are listed as start point. In an RTL design, it should be rare that you see these instances.
- For a primary input, it will be the input pin of the instance driven by the primary input.
- For all other Start Points, it will be the input pin of the instance reached where exclusivity stops.

In short, you need to consider both the cone power number (whether it is high) and the number of Start Points when you determine the reduction opportunities to pursue first.

Information in the Tabbed Pane

Once you select an instance in a mux linter opportunity in the upper pane and then click the Detail tab, you will see much more information. You will see two side-by-side sub-panes. The left-hand side sub-pane controls the right-hand sub-pane display. The four columns in the left sub-pane are:

- **Input_pins:** this is the list of pins on the 2-1 or unencoded mux that have wasted power. It may be a single pin or a vector.

- **Cone Power:** the power of the exclusive cone of logic leading to that pin.
- **Start Points:** the number of start points associated with the exclusive cone of logic for that point.
- **Pcnt Idle:** the percentage of time that the inputs pins were idle. Therefore, it represents potential wasted power.

If you add up the Cone Power and the Start Points for all the pins, these will match their corresponding accumulated number in the top pane. If you select a particular Input Pin, the following columns in the right-hand sub-pane will be filled in:

- **Input Nets:** this is a list of nets attached to the Input Pins. This may be a scalar net, a vector or a combination of the two.
- **Start Pins:** this is the list of Start Pins for the exclusive cone leading to this pin. The number of Start Pins is equal to the Start Point count.
- **Select Nets:** these are the select lines that control the Input Pin.

Each of the following has an associated pop-up menu:

- **Input Pin:** If you select an input pin and right-click, you can display the full logic cone or the exclusive logic cone for that pin in the schematic display.
- **Input Net:** If you select an input net and right-click, you can display a logic waveform.
- **Start Pin:** If you select a start pin and right-click, you can display the full logic cone or the exclusive logic cone for that pin in the schematic display.
- **Select Net:** If you select a select net, you can display a logic waveform.

Schematic Display for the Mux Power Linter

You can generate a logic cone or an exclusive cone for a pin as noted in the previous section. The following figure shows an exclusive cone of logic that includes select paths and datapath coloring.

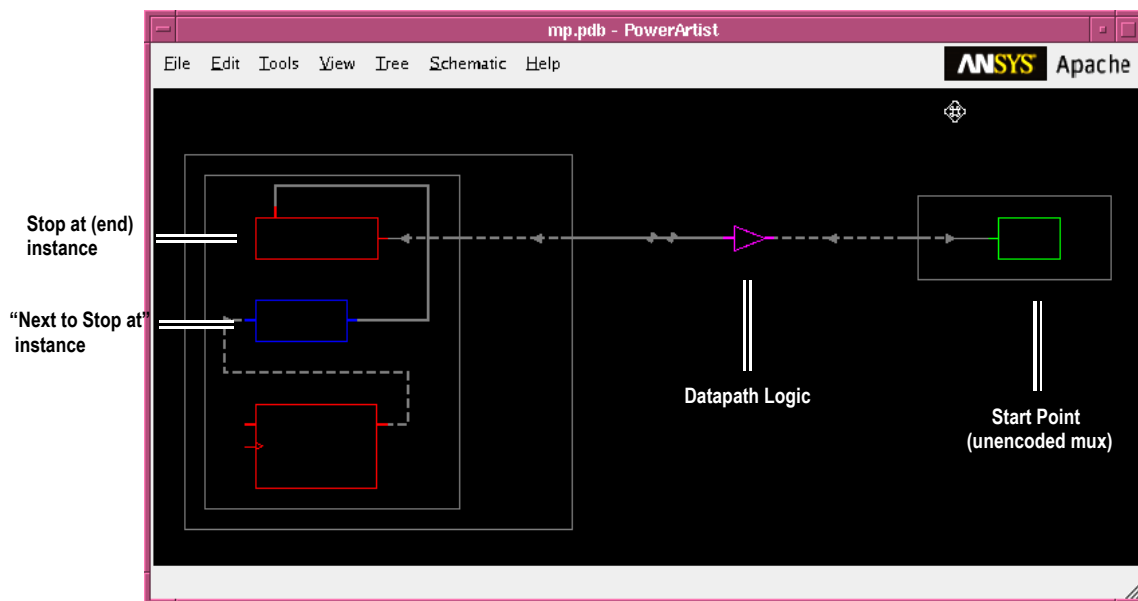


Figure 88 Exclusive Cone of Logic in Schematic (Colored by Connectivity)

If you have **Schematic > Colorized By > Connectivity** turned on, the following color scheme is applied to exclusive cones in the schematic display:

- **Green:** this is a 2-1 or unencoded mux from which the exclusive cone of logic is traced. All the instances in the exclusive cone generate logic for an input pin of this instance.
- **Magenta:** these are data path logic elements.
- **Red:** these are the “stop at” instances. These are instances that have one or more “start pins”. They will be registers, latches, register files, latch files, blackboxes, sequential vendor gates and instances where the exclusive cone stops.
- **Blue:** these are instances that are “next to last” “stop at” instances. You must insert logic between the “stop at” instance and the “next to last” instance to suppress wasted toggles.
- **Yellow:** these are elements in the select path that have not been colored as a start, end or “next to last” “stop at” instance. In the previous figure, the select path elements are colored blue and red.

The **View > Schematic Legend** menu displays the coloring. If you have asked for datapaths logic or select logic to be displayed and colored uniquely using the options in the **Schematic > When Tracing** menu options, you will see additional colors that

are explained in the legend. The legend uses the word “penultimate” which means “next to last” and applies to “next to last” “stop at” instance.

Details on Tracing Exclusive Cones

When the tracer reaches the output of an instance while tracing upstream for an exclusive cone, it continues from the corresponding input point. The choice of continuation point depends on the type of instance:

- adders and multipliers: If out[i] is reached, then:
 - a_in[0] up to a_in[i] will be a continuation point.
 - b_in[0] up to b_in[i] will be a continuation point.
- decoders and comparators: All of the inputs will be continuation points.
- buffer, not, connect, connect_inv: If out[i] is reached then in[i] will be a continuation point.
- OR, NOR, AND, NAND, XOR, XNOR: All the inputs will be continuation points.
- 2-1 mux and unencoded mux: If out[i] is reached then a_in[i], b_in[i], c_in[i], etc. will be continuation points. The select pins for a 2-1 mux and an unencoded mux will also be continuation points.

Details on Cone Power

The exclusive cone power is the sum of the power of all of the nets and instances in the cone except for instances that are vectored. For vectored instances, the instance total power number is divided by the number of output bits of the device and it is added to the exclusive cone power. Examples of vectored instances are registers, latches, register files, latch files adders, multipliers, tri-states, buffers, nots, unencoded muxes and 2-1 muxes.

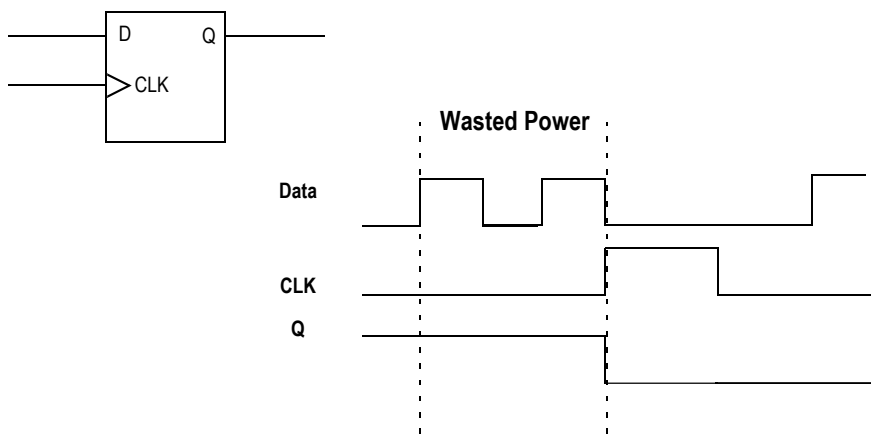
Register Power Linter

Design Parts: registers/flip-flops, clock

The Register Power Linter (REG) power linter PowerBot finds registers in the design and monitors the inputs to see if certain toggles were unnecessary because the clock did not cycle. It also estimates the amount of power lost due to the extraneous activity on the inputs.

Definition

If the input net to a register or a flip-flop toggles two or more times before the clock of the register completes one cycle, the toggles on the inputs could all be wasted. This means that there is activity that consumes power on the input of the register, but that activity is meaningless. The following schematic shows a register that is clocked infrequently (CLK) compared to the data changing at its D input. The transitioning occurring on the D input between the clocking of the register is considered extraneous in that it causes power to be wasted.



Implementation

The unused toggles on the register input can be desirable or undesirable. If it is undesirable, there may be many ways of implementing a more efficient circuitry upstream from the register, which would reduce wasted activity of the register. This PowerBot is essentially an analysis tool that points out the potential areas of concern. This analysis is very much design dependent and simulation dependent. This PowerBot will monitor the input of the register only if the associated net is monitored by your simulator. Inferred nets are not monitored for extraneous activity/wasted power.

Generating Debug Information

When you run the `ReducePower` command, you can generate debug information for the REG power linter PowerBot by specifying the `-reduction_debug_flags` option with either of the following values:

- `r`—prints details of the REG analysis to the screen.
- `&`—generates ASCII output in the `reg.pl.oadb.red` file containing a detailed reduction report for the REG linter.

Clock Enable Condition Linter

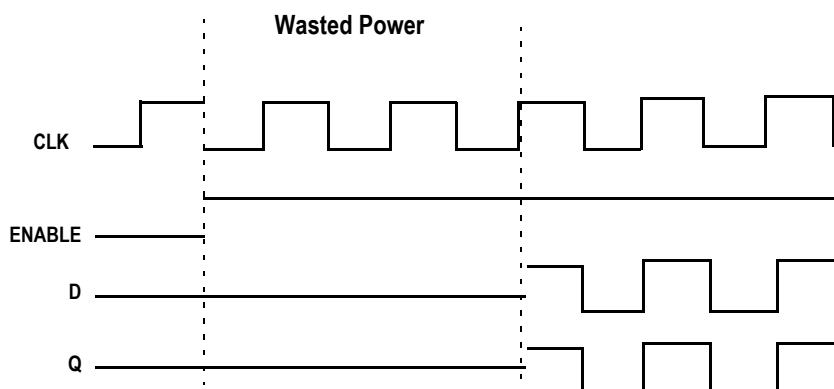
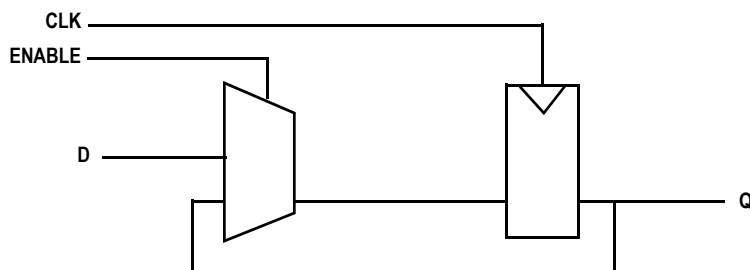
Design Parts: registers/flip-flops and multiplexers in a feedback loop, clock

The Clock Enable Condition Linter (CEC) PowerBot detects clock gating situations where the data input to the register is driven by a feedback mux. The goal is to determine situations where the mux select line, which will act as the clock gate enable signal, is not optimally designed. A simple example is where the clock is enabled but that data input is not changing.

Definition

The following schematic shows a mux-flop feedback loop topology, which is a potential candidate for clock gating. The power benefit of clock gating is only as good as the clock enable condition (ENABLE). If the input D does not cycle when the mux is enabled, the corresponding clock toggles are extraneous, and therefore contribute

to wasted power. If this topology is replaced with a gated clock with the same enable condition, the wasted power would remain the same.



Implementation

The unused clock toggles on the register clock pin can be desirable or undesirable. If they are undesirable, there may be ways of implementing more efficient circuitry that would reduce wasted activity of the clock. For example, you could replace the topology in the previous schematic with a gated clock such that the gated clock enable signal is in sync with the data activity of the register.

Usage

This PowerBot is enabled by default. To disable it, specify the “-skip_reduction_list cec” option to the ReducePower command.

Generating Debug Information

When you run the ReducePower command, you can generate debug information for the CEC power linter PowerBot by specifying the -reduction_debug_flags option with either of the following values:

- r—prints details of the CEC analysis to the screen.
- &—generates ASCII output in the cec.pl.oadb.red file containing a detailed reduction report for the CEC linter.

Rewriting Your RTL

As described in the [Scheduling Automatic RTL Changes](#) section of the PowerArtist Power Reduction tutorial, you have the ability to have PowerArtist automatically make your changes or make the edits yourself in an editor. This section describes how you can use the [RewriteRTL](#) Tcl command to perform the scheduled rewrites from the command line.

There are two types of power reduction opportunities: local and hierarchical (those that cross module boundaries). PowerArtist can perform an automatic rewrite of the RTL for both of these types of opportunities, given certain constraints. For more information on hierarchical RTL rewrite, see [Running Hierarchical Rewrite](#).

Inputs

The rewrite process requires the following inputs:

- The location of your original Verilog files.
This information is retrieved from the Verilog startup file that you specified with `-verilog_startup_file` option to the [Elaborate](#) command. This means that the rewrite process does not support a flow where Verilog files are specified using the [CompileFile](#) command.
- The name of your power data database that you have optionally updated, interactively, with the rewrites you want done automatically.
- Critical options to the Elaborate command you ran during the inferencing step. The options are:
 - `-black_box_modules`
 - `-case_sensitive`
 - `-case_insensitive`
 - `-ignore_translate_off_files`
 - `-top`
 - `-verilog_2001`
- The name of an empty directory. The rewrite process will create links to unchanged files and the new rewritten source files in that directory.
- The name of your new Verilog startup file that will closely resemble the original Verilog startup file but will instead have the rewritten files and the linked file names rather than the original file names.

Flow for Rewriting the RTL

The RewriteRTL command generates new RTL based on information in your power database that indicates which changes you want automatically written. Use the following flow:

1. Run the [Elaborate](#) and the [ReducePower](#) commands with the appropriate arguments. See the `run_Elaborate.tcl` and `run_ReducePower.tcl` Tcl scripts in the reduction tutorial for sample settings.
2. Once you have reviewed the results of the reduction analysis in the PowerCanvas, determined which modifications you want rewritten and then stored the results in your power database, you can execute a Tcl command file that includes the following example RewriteRTL command:

```
...
RewriteRTL \
  -elaborate_write_power_db true \
  -rewrite_log RTLRewriteBatch.log \
  -output_rtl_dir_name ./rewrite \
  -power_db_name $design.Batch.pdb \
  -rewrite_report_file rewriteBatch.rpt \
  -top $design \
  -verilog_2001 true \
  -verilog_startup_file txrx.vc
```

Note that the `-output_rtl_dir_name` option must specify either an empty directory or a non-existent directory. If you have already run this command (script) and the directory contains RTL code, you cannot run it again with the same directory setting. You will either need to change the directory setting or delete/move the directory that was already written. You could also add a line to the beginning of the script that deletes the rewrite directory first such as:

```
exec rm -r ./rewrite
```

If you do this, you can re-run the rewrite process over and over (without keeping the results from the previous rewrite runs).

Understanding the Internal Process

When you run the RewriteRTL command, it does the following (in the order listed):

1. Checks to ensure that your output directory is empty. If not, it will stop.
2. Determines if the power database has any scheduled rewrites. If not, it will stop.
3. Reads the files from your Verilog startup file.
4. Creates a new complete directory hierarchy in the output directory that mimics what is in your Verilog startup file.
 - a. It creates links to your original files for those that have not changed.
 - b. It stores new versions of modified files for the rewritten files.
 - c. It writes the output startup file to the user-specified file (or if none is specified, to the `rewrite.vc` file) in the output directory.

- d. Generates a report file with a user-specified name (or if none is specified, as the *startup_filename.rpt*) in the output directory.
5. Updates the power database to record which reduction opportunities have been successfully rewritten.

RewriteRTL Debug Switches

You can add the following debug options to your [RewriteRTL](#) command to diagnose any issues with the rewrite process.

`-rewrite_debug_flags F`

Prints to your screen information about the current processing steps performed by rewrite.

`-rewrite_debug_flags g`

Prints to your screen information related to failed rewrites. This will also generate files that begin with `rw_`. Example files include: `rw_details.txt`, `rw_failed.txt`, and `rw_stats.txt`.

Your Apache Design application engineer may ask you to run the [RewriteRTL](#) command with one or both of these two options.

Directory Organization

It is important to understand the directory structure created by the rewrite process. Assume that you specified the following option setting to the RewriteRTL command:

`-output_rtl_dir_name rewrite`

This sets the name of the output directory to “rewrite”. If you change into this directory, you will first notice that there is a subdirectory called EXTERNAL. From there you will see a fully rooted directory hierarchy that represents the location of your original Verilog files. If you are running the tutorial from a directory hierarchy: `./dir1/dir2`, you will see a hierarchy, depending on how you copied over the PowerArtist tutorial, similar to the following directory structure:

`rewrite/EXTERNAL/dir1/dir2/PowerArtist/rtl_analysis/rtl/verilog.`

Inside the Verilog directory, you would see a number of symbolic links that point to the original files. You would then see, assuming you chose a Gated Memory Clock example to be rewritten, an entirely new file `RR2P.v`. The new Verilog startup file is put wherever you wanted it to be. The new file will have all new path names to your files. They will reference the directory structure created underneath the rewrite directory. For example, in the tutorial, you would see in the newly created Verilog startup file names like:

```
rewrite/EXTERNAL/dir1/dir2/PowerArtist/rtl_analysis/rtl/verilog/clogic.h
rewrite/EXTERNAL/dir1/dir2/PowerArtist/rtl_analysis/rtl/verilog/top.v
<snip>
```

With very few modifications, you should be able to easily turn this Verilog startup file into a full simulation startup file for verification of your changes.

Controlling the Format of Rewritten Code

The rewrite process makes as few changes as possible to your original Verilog RTL. If you do a “diff” between the original and the rewritten file, you will notice a significant portion of the file probably has not changed at all. Spacing, indentation and comments are maintained. You do have a key choice to make on the code modifications made by the rewrite process.

By default, rewrite makes the source code changes directly in line. However, by specifying the “-rewrite_no_inline_modules true” option to the [RewriteRTL](#) command, you can force some of the changes into a module that is instantiated. There are two benefits of module instantiation. First, the resulting code may be easier for you to read. Second, it is more straightforward to do a post synthesis gate-level simulation. The trade-off is that your synthesis scripts may have to change to correctly manipulate the additional hierarchy level.

In general, it is recommended that you use the default. By default, all PowerBots creating rewritable RTL will make changes in-line.

As noted above, the rewrite process tries to maintain the formatting of your Verilog. However, the more complicated the optimization, the less possible it is to make your Verilog as readable as you originally wrote it. This becomes especially true when the rewrite spans module boundaries for a particular reduction opportunity. In this case, the rewrite process may need to add new module ports. It becomes further complicated when the module being optimized is instantiated multiple times and different instantiations result in different reduction opportunities.

Form of the Code Modifications for LNR

The Low Activity Non-Enabled Register (LNR) rewrite happens as follows. Given the following original code:

```
reg r1;  
always @(posedge clock)  
    r1 <= sig1;
```

The following two sections describe the RTL generated for both the inline and the module instantiation approach.

Inline Approach

Using the inline approach, the code will be rewritten to look like the following:

```
reg r1;

// Begin PowerArtist
wire pa_r1_lnr_rhs;
wire pa_r1_lnr_enable;
assign pa_r1_lnr_rhs = sig1;
assign pa_r1_lnr_enable = (0 != (|(r1^pa_r1_lnr_rhs)));

always (@posedge clock)
// Begin PowerArtist
if (pa_r1_lnr_enable)
    r1 <= pa_r1_lnr_rhs;
```

The `!=` operator is very critical. At the RT level, when this is simulated, if the result of the expression `|(r1^pa_r1_lnr_rhs)` is equal to X, then `pa_r1_lnr_enable` is assigned 1. Therefore, the value of `pa_r1_lnr_enable` is only set to 0 (that is clock gated) if the input and the output of the register are the same and are not X. This prevents X latch-up from occurring at the RT level when simulating.

When you synthesize your design, the `!=` operator is interpreted to be a `==` operator. When you try to simulate the resulting gate-level design, you will most likely experience X latch-up. If you follow the inline approach and you want to do gate-level simulation, you must now follow the methodology described in “LNR Gate-Level Simulation”.

Module Instantiation Approach

When the rewrite process generates module instantiations, the following events happen.

First, PowerArtist creates a new file `pa_modules.v` in the top-level of your rewrite directory hierarchy. This file contains the following module definition:

```
// This file is automatically generated by PowerArtist. It contains module
// definitions for use by LNR, Prism and GMC PowerBot rewritten Verilog.

//
// Module ADS_PA_COMPARE_WITH_ZERO
// PowerArtist instantiates this module whenever it performs an LNR
// rewrite. It contains two key `defines that you must manipulate.
// For Synthesis:
//     1) To preserve ADS_PA_COMPARE_WITH_ZERO module definition and
//        prevent it from being optimized out, supply a dont_touch constraint for it.
//        Please refer to the user guide for more details on how to do this.
//     2) By doing (1) if your synthesizer removes the != syntax during synthesis, you need
```

```
//      to black box this module. Define ADS_PA_SYNTHESIS_BLACKBOX
//      while reading this file during synthesis.
//      Please refer to the user guide for more details on how to do this.
//      3) Define SYNTHESIS during synthesis.
//      Otherwise you will likely get a warning message about != not being a
//      synthesizable construct.

// For Simulation at both RT and Gate Level:
//      1) Do not define ADS_PA_SYNTHESIS_BLACKBOX
//      2) Do not define SYNTHESIS.
//      This causes the following behavior for the statement
//          assign match = (1'b0 != value);
//      match is set to 1 if value = X or 1.
//      otherwise match is set to 0.
//      Thus if value goes to X for any reason, the clock will be enabled.
//

`ifndef ADS_PA_SYNTHESIS_BLACKBOX
    // Please provide rewrite_2009.4ER_no_inline/pa_modules.v again during simulation without
    // ADS_PA_SYNTHESIS_BLACKBOX define
`else
module ADS_PA_COMPARE_WITH_ZERO( match, value );
    output match;
    input value;

    // Do not define the variable SYNTHESIS to your simulator of choice.
    // By not defining it, you will get code that
    // prevents X's from propagating out through enable signals.

    `ifndef SYNTHESIS
        assign match = (1'b0 != value);
    `else
        assign match = value;
    `endif

endmodule

`endif
```

The ADS_PA_COMPARE_WITH_ZERO module definition name is fixed. If the rewrite process finds that a module called ADS_PA_COMPARE_WITH_ZERO already exists, it will reuse that definition and not create a definition for it in the pa_modules.v file. Therefore, you can supply your own definition for ADS_PA_COMPARE_WITH_ZERO if you like as long as the module port signature remains the same.

Second, the rewritten code will look like:

```
reg r1;
// Begin PowerArtist
wire pa_r1_lnr_rhs;
wire pa_r1_lnr_enable;
assign pa_r1_lnr_rhs = sig1;
ADS_PA_COMPARE_WITH_ZERO pa_compare_inst_0 (.match(pa_r1_lnr_enable),
.value(|(r1^pa_r1_lnr_rhs)));
// End PowerArtist

always @(posedge clock)
// Begin PowerArtist
    r1 <= pa_r1_lnr_enable?pa_r1_lnr_rhs /* PowerArtist */ :r1;
// End PowerArtist
```

Additionally, you will see that the modified Verilog source file will be identical to the original with the exception of a few changed lines. Your original lines have been deleted. Newly added lines are emphasized by being preceded by a `// Begin PowerArtist` comment. The extent will then be marked with an `// End PowerArtist`.

The `ADS_PA_COMPARE_WITH_ZERO` performs a critical task. If “value” is equal to X, its output is 1. Therefore, the value of `pa_r1_lnr_enable` is only set to 0 (that is, clock gated) if the input and the output of the register are the same and are not X.

If you are simulating the resulting gate-level design, you have to deal with potential X states. The reason that these arise is that the outputs of the registers will be initialized to X. If you don’t use the `ADS_PA_COMPARE_WITH_ZERO` module (or its equivalent) at the gate level, your registers will get locked in an X state.

The comments at the beginning of the module explain how to use the ``define` and synthesis constraints to generate the gate-level netlist you need. When you are synthesizing re-written using RTL Compiler™ from Cadence, keep the following points in mind:

- For post-synthesis simulation, you need to use the module definition of `ADS_PA_COMPARE_WITH_ZERO` that is given in the `pa_modules.v` file. This module definition will handle potential X states. To do this, you would use a script like the following:

```
set MODULE <top_module>
set HDLfiles { <list of files> }

# Read pa_modules.v file
read_hdl -define ADS_PA_SYNTHESIS_BLACKBOX ../<rewritedir>/pa_modules.v

read_hdl ${HDLfiles}
elaborate ${MODULE}

# For timing-related optimizations during synthesis, you may want to preserve
```

```
# the ADS_PA_COMPARE_WITH_ZERO module definition in the netlist to
# prevent the "assign value = match from being optimized away

set_attribute boundary_opto false [ find /des* \
    subdesign ADS_PA_COMPARE_WITH_ZERO ]
read_sdc <file.sdc>
synthesize -to_mapped -effort <medium|high>
write_hdl > netlist.v
```

- If you are using DesignCompiler™ from Synopsys, then you will use a script like the following to synthesize your design:

```
read_verilog [list pa_modules.v ...]
current_design <top>
link
# dont_touch across all instances of ADS_PA_COMPARE_WITH_ZERO
# to prevent them from being uniquified
set_dont_touch [get_designs ADS_PA_COMPARE_WITH_ZERO]
compile -ungroup_all
# do not generate ADS_PA_COMPARE_WITH_ZERO as part of the netlist
remove_design ADS_PA_COMPARE_WITH_ZERO
write -format verilog -hier -out <netlist>
```

- When using DesignCompiler and the above script fragment, you may get buffers inferred for the ADS_PA_COMPARE_WITH_ZERO module. This is most likely because you are using the set_fix_multiple_port_nets command to prevent feedthroughs from occurring. Physical design will optimize these buffer away, but it will lead to a slight loss in gate-level power analysis accuracy.
- If you want to do gate-level simulation and you tried to preserve the ADS_PA_COMPARE_WITH_ZERO module definition and the “assign match = (1'b0 != value);” statement was optimized away, you must restore it. There are two ways you can do this:
 - Edit the resulting ADS_PA_COMPARE_WITH_ZERO module definition in an editor.
 - OR
 - Replace the resulting ADS_PA_COMPARE_WITH_ZERO module definition with pa_modules.v.
- Finally, if you want to perform a post place and route gate-level simulation, because the ADS_PA_COMPARE_WITH_ZERO module is no longer in the netlist, you will again suffer from X-latch-up. You will have to follow the same methodology outlined in [LNR Gate-Level Simulation](#) (below) that is also used for the inline rewrite methodology.
- When it is time for physical design, you can replace pa_modules.v with one of your choice or potentially turn it into a macro definition that turns it into a continuous assignment statement.

LNR Gate-Level Simulation

At the RT level, new LNR code is protected by the `!==` operator by either the in-lined code or the instantiated `ADS_PA_COMPARE_WITH_ZERO` module. If you instantiated modules and preserved them through synthesis, the post synthesis netlist does not have the problem. However, if you are trying to simulate a gate-level netlist that was either synthesized from LNR code that was written in-line or was generated post place and route, there is no protection from X latch-up. In this case, you must follow the methodology outlined in [Preventing X Latch-up using ICGC Simulation Modules](#) in the *PowerArtist Library Developer's Guide*.

The goal of the methodology is to:

1. Create new simulation models for your ICGCs that have X latch-up protection built in. This step is well automated for you.
2. Use the new simulation models during gate-level simulation rather than the original ones supplied by your library provider.

Form of the Code Modifications for GMC

GMC rewrite is unique when compared with the other PowerBots because it may rewrite your RTL using instantiated ICGC cells from your Liberty files. All other PowerBots create gating logic using discrete logic circuits. The discrete logic circuits may be in-line or may instantiate modules defined in your `pa_modules.v` file.

GMC will use instantiated ICGC cells if you specified either the `SetClockGatingCell` or `SetMemoryGatingCell` commands for your `ReducePower` run and if `ReducePower` actually located and successfully used any ICGCs it found. In such a case, it will record information about the ICGC used for a particular opportunity in your power database file (`.pdb`). `RewriteRTL` then examines your power database to locate data associated with a GMC reduction opportunity. If it locates ICGC information, it will use that to create logic for an instantiated cell; otherwise, it will instantiate a clock gating subcircuit that is defined in the `pa_modules.v` file.

To best understand the code modifications, first examine the creation of a clock gating subcircuit. The following is a sample `pa_modules.v` file written by the Gated Memory Clock (GMC) PowerBot:

```
`ifndef ADS_PA_GATE_ANALYSISIS

// For Gate Level Analysis:
//     define ADS_PA_GATE_ANALYSISIS
// This will hide this version of icgc and will allow usage of the version
// created by the synthesizer in the gate level netlist for gate-level-analysis.
//
// Module ADS_PA_POSEDGED_ICGC
// PowerArtist instantiates this module whenever it performs an GMC rerwrite that
// requires a positive edge triggered integrated clock gating cell (ICGC). You will
```

```
// then need to use synthesis directives to map this module to the ICGC of choice.

module ADS_PA_POSEDGED_ICGC( ck_in, enable, test, ck_out );
    output ck_out;
    input  ck_in;
    input  enable;
    input  test;
    reg    genable;

    wire    tenable;
    assign tenable = enable | test;

    always @(ck_in or tenable) begin
        if(!ck_in)
            genable <= tenable;
    end

    assign ck_out = genable & ck_in;

endmodule

//
// Module ADS_PA_NEGEDGED_ICGC
// PowerArtist instantiates this module whenever it performs an GMC rewrite that
// requires a negative edge triggered integrated clock gating cell (ICGC). You will
// then
// need to use synthesis directives to map this module to the ICGC of choice.

//

module ADS_PA_NEGEDGED_ICGC( ck_in, enable, test, ck_out );
    output ck_out;
    input  ck_in;
    input  enable;
    input  test;
    reg    genable;

    wire    tenable;
    assign tenable = enable | test;

    always @(ck_in or tenable) begin
        if(ck_in)
            genable <= tenable;
    end

    assign ck_out = ~genable | ck_in;
```

```
endmodule
`endif
```

If PowerArtist finds both LNR and GMC reduction opportunities, `pa_modules.v` will contain all three module definitions.

Further, a sample `RR2P.v` file after a Gated Memory Clock (GMC) rewrite looks like the following:

```
// Begin PowerArtist
    wire pa_ml_gmc_clk, pa_ml_gmc_gclk, pa_ml_gmc_enable ;
    assign pa_ml_gmc_clk = CLKA1;
    assign pa_ml_gmc_enable = ~( CENA1 );
    ADS_PA_POSEDGED_ICGC pa_ml_gmc_icgc_inst(.ck_in(pa_ml_gmc_clk),
.enable(pa_ml_gmc_enable), .test(1'b0), .ck_out
t(pa_ml_gmc_gclk));
    DP512x32 m1 ( .QA(QA1), .CENSA(CENSA1), .OENSA(OENSA1),
.WENSA(WENSA1),
                .ASQA(ASQA1), .TQA(TQA1), .CLKA(pa_ml_gmc_gclk /* PowerArtist
*/), .CENA(CENA1), .OENA(OENA1),
                .WENA(WENA1), .AA(AA1), .DA(DA1), .TISA(TISA1), .TMSA(TMSA1),
                .TCENA(TCENA1), .TOENA(TOENA1), .TQOENA(TQOENA1),
.WENA(TWENA1),
                .TAA(TAA1), .TDA(TDA1), .QB(QB1), .CENSB(CENSB1),
.OENSB(OENSB1),
                .WENSB(WENSB1), .ASQB(ASQB1), .TQB(TQB1), .CLKB(CLKB1),
.CENB(CENB1), .OENB(OENB1), .WENB(WENB1), .AB(AB1), .DB(DB1),
.TISB(TISB1),
                .TMSB(TMSB1), .TCENB(TCENB1), .TOENB(TOENB1),
.TQOENB(TQOENB1), .TWENB(TWENB1), .TAB(TAB1), .TDB(TDB1) );
    // End PowerArtist
```

Note the instantiation of the `ADS_PA_POSEDGED_ICGC`. This performs the clock gating function and should be replaced during synthesis by the actual ICGC if one is used.

In the second case where the `ReducePower` command located and applied an ICGC during power savings computation, you will see your RTL rewritten like the following:

```
// Begin PowerArtist
assign pa_ml_gmc_clk = CLKA1;
assign pa_ml_gmc_tenable = !(CENA1);
`ifdef SYNTHESIS
    assign pa_ml_gmc_enable = pa_ml_gmc_tenable ;
`else
    assign pa_ml_gmc_enable = (0 != pa_ml_gmc_tenable);
`endif

`ifndef ADS_PA_FV
```

```

        SEQTLATNTSCAX2MTH pa_m1_gmc_icgc_inst(.CK(pa_m1_gmc_clk),
        .E(pa_m1_gmc_enable), .SE(1'b0), .ECK(pa_m1_gmc_gclk));
    `else
        assign pa_m1_gmc_gclk = pa_m1_gmc_clk ;
    `endif

    DP512x32 m1 ( .QA(QA1), .CENSQA(CENSQA1),
                  .OENSQA(OENSQA1), .WENSQA(WENSQA1),
                  .CLKA(pa_m1_gmc_gclk /* PowerArtist */),
                  .CENA(CENA1), .OENA(OENA1),
                  <snip>
                  .TAB(TAB1), .TDB(TDB1) );

// End PowerArtist

```

The ``ifdef SYNTHESIS` is the inlined `ADS_PA_COMPARE_WITH_ZERO` module.

If you want to perform formal verification of the rewritten RTL, most formal tools will give non-matching errors for inserted registers. To make the registers transparent and pass formal verification, you must define `ADS_PA_FV`:

```
`define ADS_PA_FV
```

before performing your formal verification run. This replaces the register with a constant wire. The constant wire turns off clock gating for the candidate register, which duplicates the original behavior of the design.

Form of Code Modifications for Prism

The Prism PowerBot generates additional registers in your design that delay the required enable signal by one clock cycle. The register may be either positively or negatively edge triggered. Further, it optionally may be reset. If it is reset, the reset may be active low or high and will be either synchronous or asynchronous.

Rather than add this complexity to your RTL by adding this logic directly in line, Prism instantiates one of a variety of Verilog modules that encapsulates this logic. Only those module definitions that have instantiations are written to the `pa_modules.v` file.

The following is a sample instantiation of a positive edge triggered flip-flop as it might appear in your rewritten RTL:

```

ADS_PA_POSEDGED_DELAY pa_cdreg_q_fce_0_delay(
    .ck_in(clk),
    .next((en)),
    .enable(pa_cdreg_q_fce_rpclk) );

```

If every type of delay register was written in your design, your `pa_modules.v` file would contain the following module definitions.

```
// This file is automatically generated by PowerArtist. It contains module
// definitions for use by LNR, Prism and GMC PowerBot rewritten Verilog.

`ifndef ADS_PA_GATE_ANALYSIS

// For Gate Level Analysis:
//   define ADS_PA_GATE_ANALYSIS
// This will hide this version of module and will allow usage of the version
// created by the synthesizer in the gate level netlist for gate-level-analysis.

//
// Module ADS_PA_POSEDGED_DELAY
// PowerArtist instantiates this module whenever it performs an FCE re-write that
// requires a positive edge triggered delay register.
//

module ADS_PA_POSEDGED_DELAY( ck_in, next, enable );
    input  ck_in;
    input  next;
    output enable;

    `ifndef ADS_PA_FV
        assign enable = 1'b1;
    `else
        reg    enable;
        always @(posedge ck_in) begin
            enable <= next;
        end
    `endif

endmodule

//
// Module ADS_PA_NEGEDGED_DELAY
// PowerArtist instantiates this module whenever it performs an FCE re-write that
// requires a negative edge triggered delay register.
//

module ADS_PA_NEGEDGED_DELAY( ck_in, next, enable );
    input  ck_in;
    input  next;
    output enable;
    `ifndef ADS_PA_FV
        assign enable = 1'b1;
    `else
```

```
    reg    enable;
    always @(negedge ck_in) begin
        enable <= next;
    end
`endif
endmodule

//
// Module ADS_PA_POSEDGED_SDELAY
// PowerArtist instantiates this module whenever it performs an FCE re-write that
// requires a positive edge triggered delay register.
// requires an active low reset.
//

module ADS_PA_POSEDGED_SDELAY( ck_in, next, reset, enable );
    input  ck_in;
    input  next, reset;
    output enable;
    `ifdef ADS_PA_FV
        assign enable = 1'b1;
    `else
        reg    enable;
        always @(posedge ck_i) begin
            if(~reset)
                enable <= 1'b1;
            else
                enable <= next;
        end
    `endif
endmodule

//
// Module ADS_PA_NEGEDGED_SDELAY
// PowerArtist instantiates this module whenever it performs an FCE re-write that
// requires a negative edge triggered delay register.
// requires an active low reset.
//

module ADS_PA_NEGEDGED_SDELAY( ck_i, next, reset, enable );
    input  ck_in;
    input  next, reset;
    output enable;
    `ifdef ADS_PA_FV
        assign enable = 1'b1;
    `else
        reg    enable;
        always @(negedge ck_in) begin
            if(~reset)
                enable <= 1'b1;
```



```
        else
            enable <= next;
        end
    `endif
endmodule

//
// Module ADS_PA_POSEDGED_ADELAY
// PowerArtist instantiates this module whenever it performs an FCE re-write that
// requires a positive edge triggered delay register.
// requires an asynchronous active low reset.
//

module ADS_PA_POSEDGED_ADELAY( ck_i, next, reset, enable );
    input  ck_in;;
    input  next, reset;
    output enable;
    `ifdef ADS_PA_FV
        assign enable = 1'b1;
    `else
        reg    enable;
        always @(posedge ck_i or negedge reset) begin
            if(~reset)
                enable <= 1'b1;
            else
                enable <= next;
            end
        `endif
endmodule

//
// Module ADS_PA_NEGEDGED_ADELAY
// PowerArtist instantiates this module whenever it performs an FCE re-write that
// requires a negative edge triggered delay register.
// requires an asynchronous active low reset.
//

module ADS_PA_NEGEDGED_ADELAY( ck_i, next, reset, enable );
    input  ck_in;
    input  next, reset;
    output enable;
    `ifdef ADS_PA_FV
        assign enable = 1'b1;
    `else
        reg    enable;
        always @(negedge ck_in or negedge reset) begin
            if(~reset)
                enable <= 1'b1;
            else
                enable <= next;
            end
        `endif
endmodule
```

```
        enable <= next;  
    end  
`endif  
endmodule  
  
`endif
```

There are additional points to note about this file. First, notice the following line:

```
`ifndef ADS_PA_GATE_ANALYSIS ... `endif
```

This piece of code encloses all of the delay register definitions. These are needed for RTL simulation.

General Form of Modifications for Different Code Types

In general, the rewrite process divides the rewritten RTL code into three sections: a combinational section that generates the logic that will serve as input to the rewritten register, an enable section that generates an enable signal and a sequential section that implements the enabled register. Consider the following snippet of RTL code for which the LNR PowerBot identified a reduction opportunity:

```
always @(posedge clk or negedge rst) begin  
    if (~rst)  
        reg_test <= 4'd0;  
    else if (data_read)  
        reg_test <= a  
    else  
        reg_test <= b;  
end
```

The schematic implementation of this code is as follows:



In this example:

- The mux has inputs A and B and the register output is REG_TEST.
- The wire connecting the mux output with the register is inferred and is not a named signal in the RTL. In PowerArtist, inferred signals are represented using names of the form `#[integer]`. In the above example, the inferred signal is named `#1`.

When PowerArtist flags an LNR reduction opportunity for this register, during the LNR rewrite process PowerArtist needs to XOR the input and output of the register and then OR the XOR output to generate the enable. Further in this example, the register input signal (#1) is not a user-defined signal and, therefore, PowerArtist needs to generate a named input.

PowerArtist will rewrite the always block in the previous code example as follows:

Sample Combinational Section

```
always @(reg_test or data_read)
begin
    pa__reg_test_comb = reg_test;
    begin
        if (data_read)
            pa__reg_test_comb <= a
        else begin
            pa__reg_test_comb <= b;
        end
    end
end
pa__reg_test_comb becomes the named output of the mux after rewrite.
```

Sample Enable Section

For LNR, the enable is generated by XORing the input and output of the register as in the following:

```
`ifndef SYNTHESIS
    assign pa_reg_test_lnr_enable = (0 != (( | (reg_test ^ pa__reg_test_comb)
));
`else
    assign pa_reg_test_lnr_enable = ( | (reg_test ^ pa__reg_test_comb) ;
`endif
```

Sample Sequential Section

```
always @(posedge clk or negedge rst) begin
    if (~rst)
        reg_test <= 4`d0;
    else begin
        if (pa_reg_test_lnr_enable)
            reg_test <= pa__reg_test_comb;
    end
end
```

PowerArtist can rewrite many different types of code. The following sections describe sample transformations made to various Verilog constructs during both LNR and Prism rewrites.

Concatenations

PowerArtist can rewrite registers that are involved in concatenations. For example, given the following original code:

```
reg [0:15] out3, out5 ;

always @ (posedge clk) begin
    {out3, out5} <= {16{in1}};
end
```

PowerArtist writes out the following transformed code:

```
reg [0:15] out3,out5 ;
reg [0:15] pa__out3_comb ;
reg [0:15] pa__out5_comb ;

always @( in1 or out3 or out5 ) begin
    pa__out3_comb = out3 ; // Avoid latch behavior
    pa__out5_comb = out5 ; // Avoid latch behavior
    begin
        { pa__out3_comb, pa__out5_comb } = {16{in1}} ;
    end
end

ADS_PA_COMPARE_WITH_ZERO pa__compare_inst_1(
    .match(pa__out3_lnr_enable),
    .value((( out3 ^ pa__out3_comb )))) ; // out3
ADS_PA_COMPARE_WITH_ZERO pa__compare_inst_2(
    .match(pa__out5_lnr_enable),
    .value((( out5 ^ pa__out5_comb )))) ; // out5

always @( posedge clk ) begin
    if ( pa__out3_lnr_enable )
        out3 = pa__out3_comb ;
    if ( pa__out5_lnr_enable )
        out5 = pa__out5_comb ;
end
```

Code with Assignments to Multiple Registers

PowerArtist can rewrite code with assignment to multiple registers. Given the following original code:

```
reg [0:15] regA, regB;
always @(posedge clk)
    if(enable) begin
        regB <= foo2;
```

```
    regA <= foo;
end
else if(check)
    regA <= bar;
else
    regA <= car;
```

PowerArtist writes out the following transformed code:

```
reg [0:15] regA, regB;
reg [0:15] tmp_regA, tmp_regB;
wire pa_lnr_regA_enable;
always @(enable or check or foo or bar or car or regA or regB) begin
    tmp_regA = regA; // Avoid latch behavior
    tmp_regB = regB; // Avoid latch behavior
    if(enable) begin
        tmp_regB = foo2;
        tmp_regA = foo;
    end
    else if(check)
        tmp_regA = bar;
    else
        tmp_regA = car;
end

ADS_PA_COMPARE_WITH_ZERO pa_inst1(pa_lnr_regA_enable,
                                   |(tmp_regA^regA));

always @(posedge clk) begin
    regB <= tmp_regB;
    if(pa_lnr_regA_enable) // regA is lnrred
        regA <= tmp_regA;
end
```

Multiple Registers in Case Statements

PowerArtist can rewrite code containing multiple registers in case statement (as well as other complex “Case” styles). For example, given the following original code:

```
reg [0:15] regA, regB;
always @(posedge clk)
    case (select_bits)
        2'b00:
            regA <= ina0;
            regB <= inb0;
        2'b01:
            regA <= ina1;
            regB <= inb1;
        2'b10:
```

```
    regA <= ina2;  
    regB <= inb2;  
2'b11:  
    regA <= ina3;  
endcase
```

PowerArtist writes out the following transformed code:

```
reg [0:15] regA, regB;  
reg [0:15] tmp_regA, tmp_regB;  
wire pa_lnr_regA_enable;  
always @(select_bits or ina0 or ina1 or ina2 or ina3 or  
    inb1 or inb1 or inb2 or regA or regB) begin  
    tmp_regA = regA; // Avoid latch behavior  
    tmp_regB = regB; // Avoid latch behavior  
    case (select_bits)  
        2'b00:  
            tmp_regA = ina0;  
            tmp_regB = inb0;  
        2'b01:  
            tmp_regA = ina1;  
            tmp_regB = inb1;  
        2'b10:  
            tmp_regA = ina2;  
            tmp_regB = inb2;  
        2'b11:  
            tmp_regA = ina3;  
    endcase  
end  
  
ADS_COMPARE_WITH_ZERO pa_inst1(pa_lnr_regA_enable,  
    | (tmp_regA^regA));  
  
always @(posedge clk) begin  
    regB <= tmp_regB;  
    if(pa_lnr_regA_enable) // regA is lnrred  
        regA <= tmp_regA;  
end
```

Code with a Bit Sliced Assignment on the Left-Hand Side

PowerArtist can rewrite code with a bit-sliced assignment on the left-hand side. For example, given the following original code:

```
reg [0:15] regA;  
always @(posedge clk) begin  
    regA[0:3] <= foo;
```

```
    regA[4:12] <= bar;  
    if(en) regA[13:15] <= car;  
end
```

PowerArtist writes out the following transformed code:

```
reg [0:15] regA;  
reg [0:15] tmp_regA;  
wire pa_lnr_regA_enable;  
always @(foo or bar or en or car or regA) begin  
    tmp_regA = regA; // Avoid latch behavior  
    tmp_regA[0:3] = foo;  
    tmp_regA[4:12] = bar;  
    if(en) tmp_regA[13:15] = car;  
end  
  
ADS_PA_COMPARE_WITH_ZERO pa_inst1(pa_lnr_regA_enable,  
                                   |(tmp_regA[9:12]^regA[9:12]));  
always @(posedge clk) begin  
    regA[0:8] <= tmp_regA[0:8];  
    if(pa_lnr_regA_enable)  
        regA[9:12] <= tmp_regA[9:12];  
    regA[13:15] <= tmp_regA[13:15];  
end
```

Formal Verification of Rewritten Code

Most LEC formal verification tools are unable to verify changes performed by Prism, ODC or GMC. This is due to the sequential logic changes created by additional registers inserted into the design. As described in the [Form of the Code Modifications for GMC](#) section, you can do a partial verification by making the registers transparent to ensure that no other functionality changes have occurred. A Conformal setup for changes made by the LNR PowerBot follows:

```
// Set up mode of operation  
set system mode setup  
// Read in design files  
read design -file design.vc -golden  
read design -file rewrite/rewrite.vc -revised -lastmod  
// Set the root module  
set root module top -both  
// Switch mode of operation  
set system mode lec  
// Add all compare points  
add compared points -all  
// Compare  
compare -all  
// Report
```

```
report unmapped points > unmapped_points.rpt
report mapped points > mapped_points.rpt
report compare data > compared_points.rpt
usage
exit -f
```

The file, `compared_points.rpt`, would contain the status of your conformal run. The `rewrite/rewrite.vc` is the new Verilog startup file generated by the rewrite process.

Performing formal verification for changes made by GMC requires more work. Complex macros like memories that are represented by simulation models and a corresponding power model can not be formally verified. Instead they are “black boxed”. So changes affecting their behavior cause match point problems. Therefore, you can manually verify the changes, re-simulate that portion of the design and verify that the behavior is the same or you can replace the simulation model with a very simple synthesizable model in your rewritten Verilog startup file.

For example, in the tutorial, a DP512x32 gets rewritten as described above after you have accepted the rewritten recommendation by the GMC PowerBot. The memory you would have to generate would look like the following and be saved in a file called `simpleRR2P.V`:

```
module DP512x32 ( QA, CENSQA, OENSQA, WENSQA,
                ASQA, TQA, CLKA, CENA, OENA,
                WENA, AA, DA, TISA, TMSA,
                TCENA, TOENA, TQOENA, TWENA,
                TAA, TDA, QB, CENSQB, OENSQB,
                WENSQB, ASQB, TQB, CLKB,
                CENB, OENB, WENB, AB, DB, TISB,
                TMSB, TCENB, TOENB,
                TQOENB, TWENB, TAB, TDB ) ;

input  [31:0] DA, TDA, TQA ;
input  [31:0] DB, TDB, TQB;
input  [8:0]  AA, TAA, ASQA;
input  [8:0]  AB, TAB, ASQB;
input  CENSQA, OENSQA, WENSQA, CLKA, CENA, OENA, WENA;
input  CENSQB, OENSQB, WENSQB, CLKB, CENB, OENB, WENB;
input  TISA, TMSA, TCENA, TOENA, TQOENA, TWENA;
input  TISB, TMSB, TCENB, TOENB, TQOENB, TWENB;

output [31:0] QA;
output [31:0] QB;
reg     [31:0] QA;
reg     [31:0] QB;

reg     [31:0] myRamA [511:0] ;
reg     [31:0] myRamB [511:0] ;
```



```
always @( posedge CLKA )
begin
  if(!CENA)
  begin
    if(!WENA)
    begin
      myRamA[AA] <= DA;
    end
    else if(WENA)
    begin
      QA <= myRamA[AA];
    end
  end
end

always @( posedge CLKB )
begin
  if(!CENB)
  begin
    if(!WENB)
    begin
      myRamB[AB] <= DB;
    end
    else if(WENB)
    begin
      QB <= myRamB[AB];
    end
  end
end

endmodule
```

You then have to use a more complex Conformal script to verify that the rewrite was correct.

```
// Set the mode of operation
set system mode setup
// how to handle x's
set x conversion E -both
// set all undefined blocks to be black boxes
// this is because conformal does not support alf
set undefined cell black_box
// Model clock gating
set flatten model -gated_clock
// read in libraries
Read library -verilog simpleRR2P.v -both
// read in designs
read design -file design.vc -golden
read design -file rewrite/rewrite.vc -revised -lastmod
```

```
// set top module
set root module top -both
// switch mode of operation
set system mode lec
add compared points -all
compare -all
report unmapped points > unmapped_points.rpt
report mapped points > mapped_points.rpt
report compare data > compared_points.rpt
usage
exit -f
```

Consult your Conformal manual for a detailed description of how these commands operate.

Errors, Warnings and Notes Generated by the Rewrite Process

When performing an RTL rewrite, you may see one or more of the following warnings:

- **Warning 7067: LNR Transformations were applied on #1 registers, #2 registers were skipped.**
This message provides the number of LNR opportunities that were successfully implemented (#1) and those that did not meet the criteria and were skipped (#2).
- **Note 7068: LNR Transformations were applied on # registers.**
This message provides the number of LNR opportunities that met the code rewrite criteria and were successfully rewritten.
- **Warning 7076: Register *register_name* in module *module_name* is a `defined register. Skipping LNR...**
This message tells you that the LNR opportunity was not implemented because the rewrite process encountered a `define statement that was then used that somewhere in an always block. PowerArtist will not rewrite this portion of the design.
- **Warning 7082: Skipping inferred register output *register_#* in module *module_name* from PDB.**
You may see this message if you are trying to rewrite multi-dimensional registers. PowerArtist might not successfully rewrite them in this case.
- **Warning 7088: Register *register_name* in module *module_name* has been inferred from a case statement. Skipping LNR...**
This means that the rewrite processes encountered a case statement. Rewriting case statements so that they are readable as simple transformations of your existing design is very difficult. Therefore, in order to preserve the legibility of your design, PowerArtist does not automatically implement an LNR rewrite that

involves modifications to case statements. In this case, you should look into the RTL and do a manual edit.

- **Warning 7090: Register *register_name* in module *module_name* does not have an associated constant set/reset. Skipping LNR...**

Though you selected this LNR opportunity to be automatically rewritten, PowerArtist is ignoring the opportunity because the given register is set or reset either asynchronously or synchronously.

- **Warning 7092: Could not apply LNR transformations for register *register_name* in module *module_name* since it is driven by more than one signal.**

Registers driven by more than one signal are not eligible for LNR transformations.

Running Hierarchical Rewrite

You can use the hierarchical rewrite feature of PowerArtist to make automated RTL changes that cross module hierarchy. There are separate algorithms for hierarchical rewrite that co-exist with the algorithms for local opportunities (the default algorithms).

Instantiation-specific and hierarchical rewrite currently supports the following PowerBots for automatic rewrite:

- [Gate Memory Clock](#) (GMC)
- [Observability Don't Care](#) (ODC)
- [Low-Activity Non-Enabled Register](#)
- [Prism](#) (both fce and bce)

Understanding the Hierarchical Rewrite Methodology

Hierarchical rewrite is a much more complex technique than local rewrite for the following reasons:

- To implement a rewrite opportunity hierarchically, enable signals generated by one module must be able to reach another module. This means that ports will need to be added to modules to export and to import enable signals.
- If a module has multiple instantiations, some instantiations of the module may be involved in an optimization and others may not.
- A given module may be involved in multiple separate reduction opportunities. Each of those opportunities may or may not be involved in a particular module instantiation.

The last two bullets are critical. If a module has been instantiated N different times in a fully flattened design tree, there may be N different versions of the module. This complexity almost guarantees that the resulting RTL will be much harder to read than RTL that is hand coded by a design engineer.

One way to develop the N versions of a module is to create N unique copies of the module with each version representing one unique combination of reduction opportunities. This means that if an ECO needs to be made to a common portion of the N modules, the designer needs to edit N different module definitions. To reduce this complexity, the rewrite process will create one module definition and use “parameter” and “defparam” statements to create an enable bit that can selectively turn on and off an optimization. In addition, if a future ECO removes the gating logic, commenting out the defparams for the particular optimization disables that optimization.

Consider the following simple example, which has much of the logic eliminated to highlight just the most critical pieces.

```
module top(...)

    mid1 mid1(..., enable);
    mid2 mid2(..., enable);

endmodule

module mid1(..., enable);

    input enable;

    always @(posedge clk)
        reg2 <= din; // gate this with a one cycle earlier "enable"

    always @(posedge clk)
        if (enable)
            reg1 <= reg2;

endmodule

module mid2(..., enable);

    output enable;

    always @(posedge clk)
        enable <= enable_prev; // enable_prev is the one cycle early enable

endmodule
```

In this example, reg2 can be clock gated if it is possible to find a 1-cycle earlier version of enable. The mid2 module contains such an enable called enable_prev. It needs to be exported from mod2 and imported into mod1.

A designer could rewrite this import/export in the following manner (with changes highlighted in red):

```
module top(...)

    mid1 mid1(..., enable, enable_prev);
    mid2 mid2(..., enable, enable_prev);

endmodule

module mid1(..., enable, enable_prev);

    input enable, enable_prev;
```

```
always @(posedge clk)
    if (enable_prev)
        reg2 <= din; // gate this with a one cycle earlier "enable"

always @(posedge clk)
    if (enable)
        reg1 <= reg2;

endmodule

module mid2(..., enable, enable_prev);

output enable, enable_prev;

always @(posedge clk)
    enable <= enable_prev; // enable_prev is the one cycle early enable

endmodule
```

Now assume that mid1 has been instantiated many times and in only some of those cases is it possible to clock gate reg2. In the remaining cases, the clock gating needs to be turned off. The code as written defines a parameter that selectively turns on the optimization. The changes to the code sample are shown in red in the following sample:

```
module top(...)

    defparam mid1.enable_bit = 1'b1;

    mid1 mid1(..., enable, enable_prev);
    mid2 mid2(..., enable, enable_prev);

endmodule

module mid1(..., enable, real_enable);
    output real_enable
    parameter enable_bit = 1'b0;

    assign real_enable = (enable_bit == 1'b1) ? enable_prev : 1'b1;

    always @(posedge clk)
        if (real_enable)
            reg2 <= din; // gate this with a one cycle earlier "enable"

    ... the remaining code is the same
endmodule
```

Note the following additions to the above code:

- A new parameter, `enable_bit`, gets defined (the default is off.)
- If `enable_bit` is true, this means that for this version of `mid1` that clock gating is enabled. Therefore, `real_enable` is the 1 cycle earlier `enable_prev`.
- If the `enable_bit` is false, then for this version of `mid1` clock gating is disabled. The `real_enable` must be set to true. `reg2` is always set to `din`.
- The `defparam` then sets hierarchical instance path `mid1.enable_bit` to true showing that for the `mid1` instantiation, this particular optimization is turned on.

As described in the [Form of the Code Modifications for GMC](#) section, it is possible to make all register enables transparent to ensure that standard formal verification tools that can only do combinational equivalence checking, can verify that the fundamental logic was not changed during the rewrite process. This process makes use of the `ADS_PA_FV` definition. This causes further modifications to the previous code sample. The following sample shows these modifications in red:

```
module mid1(..., enable, real_enable);
output real_enable

parameter enable_bit = 1'b0;
wire true_enable;

assign real_enable = (enable_bit == 1'b1) ?
                    transparent_enable_prev : 1'b1;

always @(posedge clk)
    if (real_enable)
        reg2 <= din; // gate this with a one cycle earlier "enable"

`ifdef ADS_PA_FV
assign transparent_enable_prev = 1'b1;
`else
assign transparent_enable_prev = enable_prev;
`endif
```

Finally, to eliminate name collisions, the simple names in this code sample are replaced by more complex names. Specifically, the following substitutions have been made:

- The enable signal for an ODC candidate register will be of the form `pa__ODC_candidate_register_enable`. In the code sample, `real_enable` actually will become `pa__reg2_enable`. The `reg2` replaces the `ODC_candidate_register`.
- The right hand side of the ODC candidate register assignment statement may have to be modified. The name of the variable created will be of the form `pa__ODC_candidate_register_rhs` or `pa__ODC_candidate_register_comb`. In the code sample, `din` actually will become `pa_reg2_rhs`. The `reg2` replaces the `ODC_candidate_register`.

- There may be many unique reduction opportunities requiring unique enable signals to be imported into the module or exported from the module. Rather than have a separate port for each, the input enable signal will be a bit vector with the name `pa__hier_in`. Each bit is assigned a unique enable signal. In the code sample, `enable_prev` will become `pa__hier_in[0]`. Similar modifications will be made to create new output port names using the bit vector name `pa__hier_out`.

The following sample shows these final modifications in red.

```
module mid1(..., enable, pa__hier_in);

input pa__hier_in;
parameter enable_bit = 1'b0;
wire pa_reg2_enable, pa_reg2_rhs;

assign pa_reg2_enable = (enable_bit == 1'b1) ?
                        transparent_enable_prev : 1'b1;

always @(posedge clk)
    if (pa_reg2_enable)
        reg2 <= pa_reg2_rhs; // gate this with a one cycle earlier "enable"

assign pa_reg2_rhs = din;
`ifdef ADS_PA_FV
assign transparent_enable_prev = 1'b1;
`else
assign transparent_enable_prev = pa__hier_in;
`endif
```

The rewrite process also transforms the names `enable_bit` and `transparent_enable_prev` into unique names to prevent name collisions.

Chapter 14

Performing Functional Verification of RTL Changes

Introduction

You can perform functional verification of changes made to the RTL by power reduction using one of three verification methods. You may choose to use one of the industry-standard formal verification tools, you may use the nCompare™ utility from SpringSoft or, you may want to perform a functional simulation of your design.

Using Standard Verification Tools by Defining ADS_PA_FV

PowerArtist encapsulates sequential changes inside an `ifdef ADA_PA_FV conditional compilation directive such as the following:

```
`ifdef ADS_PA_FV
    assign enable = 1'b1;
`else
    reg    enable;
    always @(posedge ck_in)
    begin
        enable <= next;
    end
`endif
```

If you want to use the industry-standard tools, such as Conformal™ or Formality™, you can set the ADS_PA_FV conditional compilation directive in your tool of choice. Defining `ADS_PA_FV in your Conformal or Formality setup replaces the delay register with a constant wire, which stops the formal verification from producing an error when encountering additional registers.

Limitations of the this Approach

The value of this approach may be limited. Although it does verify that PowerArtist RTL changes have not impacted any other functionality in the design, apart from the new enables, it does not verify the impact of the new enables on the functionality.

Generating a Rule File to use in the nCompare Utility (Beta)

Note on Using this Beta Feature

Before trying this beta feature/flow, please contact your Apache Design support team for assistance.

The Verdi™ debug system from SpringSoft includes a powerful waveform comparison tool called nCompare™. This approach involves generating a rule file from the PowerArtist power database that you can then load into nCompare. If you usually use Verdi for functional verification, you may find this method useful. Using this method, you do not need to compare all of the signals as you would without a rule file. To generate this rule file, you need to run the [WriteReductionCompareFile \(Beta\)](#) command.

Use Model

Use the following flow to run a functional verification with a rule file:

1. Run the [ReducePower](#) command to generate a power database.
2. This step is optional. If you want to compare signal changes due to all accepted reductions (not only those that are rewritten) you can generate a rule file at this point by specifying the [WriteReductionCompareFile \(Beta\)](#) command with the “-reduction_compare_type auto” command-line option¹. You are also required to specify the power database, the top instance name, and the name for the output rule file. For example,

```
WriteReductionCompareFile -power_db_name design.pdb
                          -top_instance test.top
                          -reduction_compare_output_rule_file design.pdb.rc
                          -reduction_compare_type auto
```

You can also use the optional arguments to the WriteReductionCompareFile to perform various functions as described here:

- reduction_compare_error_report_file *error_report_file_name*
Specifies the name of the file into which nCompare will output any mismatch error messages. You can use the nce2report SpringSoft utility to generate a readable text/html report as follows

```
nce2report -i error_report_file_name ...
```
- reduction_compare_golden_sim_file *file_name*
Specifies the path to the original FSDB file. If you do not specify this, PowerArtist will use a place holder variable (set GoldenFSDB *golden_fsdb_name*) in the rules file that you will later need to edit (see step 6).

1. As with all command file commands, you can use pt_set variables in your run script instead of command-line options.

```
-reduction_compare_secondary_sim_file file_name
```

Specifies the path to the new FSDB file that you get when you re-simulate your design using the rewritten RTL. If you do not specify this option, PowerArtist will use a place holder variable (set SecondaryFSDB *fsdb_file_name*) in the rule file that you will later need to edit (see step 6).

If you specify these options in the command, you will not need to perform hand editing of the rule file (as described in step 6). For a full list of options, see the [WriteReductionCompareFile \(Beta\)](#) command description.

If you want to generate a rule file for only the rewritten reductions, you can skip this step.

3. Run the [RewriteRTL](#) command. You will use the resulting Verilog start up file (.vc) with the accepted RTL changes that you accepted.
4. Run the WriteReductionCompareFile command at this point to generate a rule file for the reductions that were rewritten by the RewriteRTL command. You can set the “-reduction_compare_type rw” option (or not specify it at all since “rw” is the default).
5. Re-simulate your design using the .vc file generated by the rewrite process. This will produce the updated FSDB file that you will then specify in your rule file.
6. Perform any necessary hand editing of the rule file.

For example, you may have to add the name of the updated FSDB file in the SecondaryFSDB variable setting.

```
set SecondaryFSDB <Specify_Secondary_Fsdb_Here>
```

You can also specify this name when you generate the rule file (if you know it already) by specifying the -reduction_compare_secondary_sim_file option, to the WriteReductionCompareFile command. Doing so will automatically set the SecondaryFSDB variable in the rule file to the correct name.

7. Run the nCompare utility on the rule file using the following command:

```
nCompare -rule rule_file
```

Note: You can generate a formatted report that is easy to read in either text or HTML format using the nce2report utility:

```
nce2report -i error_file ... additional_options
```

You can also view the results using the nWave™ tool, also from SpringSoft.

Flow Diagram

The following flow diagram illustrates the process for generating an nCompare rule file to for functional verification of RTL changes (as detailed in the previous section).

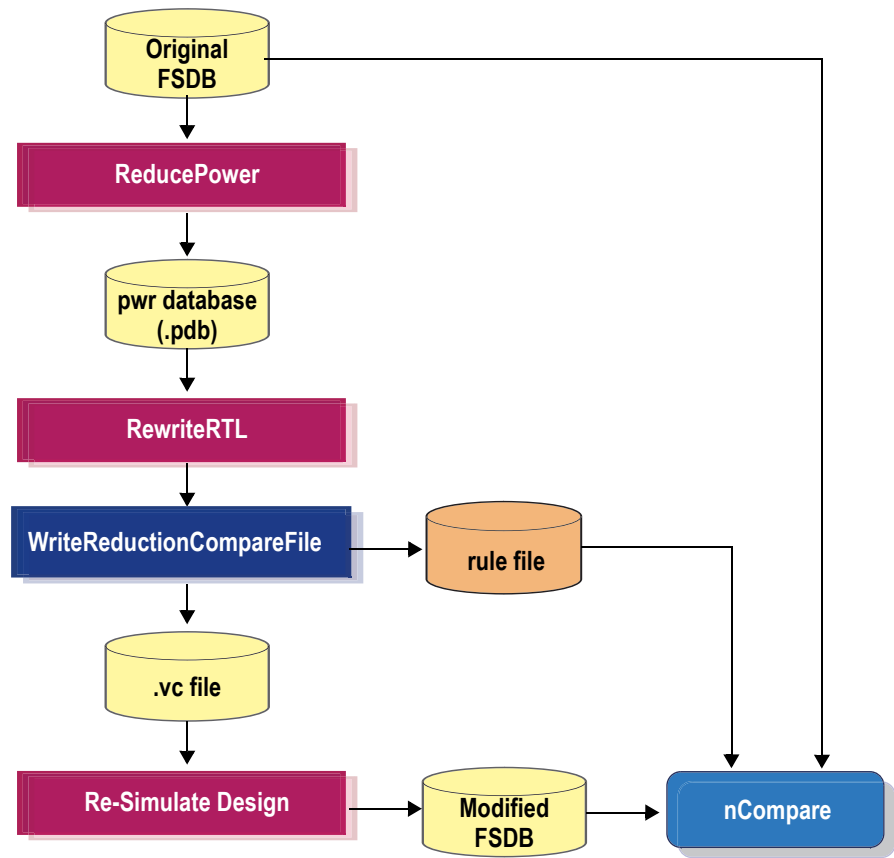


Figure 89 Typical RTL Verification Flow Using the nCompare Utility

Sample Output Rule File

The following sample rule file was generated by the [WriteReductionCompareFile \(Beta\)](#) command.

```
#####
# Apache Design, Inc.
#
# File Type       : PowerArtist/XP nCompare waveform comparison rule file
# Version        : 2010.1.3PreAlpha (64 Bit Linux) 31 Aug 2010
# Design Name    : top
# Date of Generation : 31 Aug, 2010 14:16:02 IST
```

```
#
# Options used -
#   -power_db_name                : design.pdb
#   -reduction_compare_error_report_file : design.pdb.rc.nce
#   -reduction_compare_golden_sim_file   : test.orig.fsdb
#   -reduction_compare_output_rule_file  : design.pdb.rc
#   -reduction_compare_secondary_sim_file : test.post.fsdb
#   -reduction_compare_type             : auto; all the signals due to
accepted reductions
#   -top                             : top
#   -top_instance                    : test.top
#
#####

#
# open golden and revised simulation files
#
set GoldenFSDB      test.orig.fsdb --> If this is not set, you will need to set it
before using nCompare.
set SecondaryFSDB   test.post.fsdb --> If this is not set, you will need to set it
before using nCompare.

cmpOpenFsdb $GoldenFSDB $SecondaryFSDB

#
# set default hierarchy delimiter as "."
#
cmpSetDelimiter .

#
# set maximum errors to 1500 and maximum per signal errors to 10
#
cmpSetCmpOption -MaxError 1500 -MaxErrorPerSignal 10

#
# compare input and output ports
#

cmpSetSignalPair test.top.clk
cmpSetSignalPair test.top.we
cmpSetSignalPair test.top.me
cmpSetSignalPair test.top.sel_mem_next
cmpSetSignalPair test.top.addr\[0\]
<snip> ...
cmpSetSignalPair test.top.qout\[31\]

#
# compare GMC registers
#
```

```
cmpSetSignalPair test.top.mem01.Q\[31\]
cmpSetSignalPair test.top.mem01.Q\[30\]
<snip> ...
cmpSetSignalPair test.top.mem01.LSI_SCAN_OUT

#
# all mismatch error messages will be output to design.pdb.rc.nce
#
cmpSetReport design.pdb.rc.nce

#
# start comparision
#
cmpCompare
```

Performing Functional Simulation

If you want to perform functional simulation of your rewritten design, you will want to know the impact of the power reduction techniques that PowerArtist applied. The code modifications performed vary with each PowerBot. For details on the code modifications done by the LNR, GMC and Prism PowerBots, see the following sections:

- [Form of the Code Modifications for LNR](#)
- [Form of the Code Modifications for GMC](#)
- [Form of Code Modifications for Prism](#)

Chapter 15

Analyzing the Effects of Power Gating with Proprietary Commands

Introduction

Defining voltage domains and performing power gating can help to aggressively reduce power. Early visibility into design trade-offs involving these techniques at the RT level of abstraction can be valuable. There are two methods for exploring multiple voltage domains and performing power gating in PowerArtist—using only PowerArtist proprietary commands or using a combination of proprietary commands and CPF or UPF commands. This chapter describes a method for exclusively using PowerArtist proprietary commands to explore multiple voltage domains, define power domains and perform either a simulation-based or a vectorless analysis of power domains.

Using Common File Formats

For information on using the CPF or UPF file formats, see [Using Standard File Formats](#). Note that if you intend to use UPF as input, you still need to thoroughly read this chapter. The [Using a UPF Input Flow \(Beta\)](#) and [Using a CPF Input Flow \(Beta\)](#) sections describe only that portion of the overall flow that UPF or CPF can replace, the remaining steps in the flow are described only in the following sections in this chapter.

Chapter Organization

The following topics are covered in this chapter:

- [Required Inputs for Power Gating](#)
- [Special Option to the Elaborate Command](#)
- [Setting up Your Command File for Power Gating](#)
- [Performing Simulation-Based Power Analysis with Power Gating](#)
- [Performing Vectorless Power Analysis with Power Gating](#)
- [Understanding the Output Reports for Power Gating Analysis](#)

Required Inputs for Power Gating

The following inputs are required for this flow:

- Libraries in Liberty format with required power gating attributes. For more information, see [Defining Libraries](#).
- RTL source files in which Verilog begin blocks and VHDL processes are named where you want to control the default cell selection for retention flops and latches. For more information, see [Creating Source Files](#).
- A PowerArtist command file
The command file contains all of the Tcl commands used to specify inputs such as clocks, defines libraries to be used in various portions of your design, virtual supplies, voltage and power domains and wire load models. In a vectorless analysis, while defining a virtual supply, the ON condition must be a constant 0 or constant 1. In simulation-based analysis, that condition can be any legal boolean expression of the design nets in the target language.
- A scenario file representing the RTL or mixed RTL and gate design
Use the [Elaborate](#) command to create your scenario file. If you are performing a vectorless power analysis, the amount of instantiated gate-level logic should be small (to almost non-existent) as to minimize any problems with propagating activity through gates. For complete details on creating a scenario file, see [Getting Your Design into PowerArtist](#). In addition, see the [Special Option to the Elaborate Command](#) for information specific to power gating. You may also analyze an entire gate-level design. However, you must not use the `-gate_level_netlist` switch on either the `Elaborate` or `CalculatePower` commands. Otherwise, you will be prevented from performing a power gating analysis.
- If you are using a simulation-based flow, you will also need an activity file in either FSDB, VCD or IAF file formats. The VCD format is generated by Apache PLI routines. For more information on activity file formats, see [Acquiring Simulation Data](#).
- The reference clock name
If you are using a simulation-based flow, you must specify a reference clock. This is the fastest clock in the design. It is used to calculate activity factors. All other signals will be assumed to be toggling slower than this signal.
- The number of clock cycles per calculation interval
If you are using a simulation-based flow, once the number of clock cycles is reached, the time-based analyzer performs an average power calculation for that interval. This process is repeated from the `-start_time` to the `-finish_time` giving you a power-over-time curve.
- Vectorless input file
This is required if you are running a vectorless flow. For details, see [Performing Vectorless Power Analysis with Power Gating](#).

Defining Libraries

To use the power gating feature, you must add the following attribute to the appropriate Liberty library.

- `power_gating_cell`

This is a cell-level attribute that indicates that the given cell is a retention flop or latch. You will use the `MapRetentionCell` command to map a type of retention cell specified by the value of this attribute to a particular always block.

A portion of a typical Liberty retention flip-flop cell definition might look like:

```
cell (retflop) {
    rail_connection (PVDD, VDD);
    rail_connection (PVDDC, VDDC);
    power_gating_cell : LOW;
    leakage_power () {
        value : 2.72E+01;
        when : "RET";
        power_level : VDDC;
    }
    leakage_power () {
        value : 5.91E+03;
        when : "RET";
        power_level : VDD;
    }
    leakage_power () {
        value : 2.89E+01;
        when : "!RET";
        power_level : VDDC;
    }
    leakage_power () {
        value : 5.89E+03;
        when : "!RET";
        power_level : VDD;
    }
    pin (RET) {
        internal_power () {
            power_level : VDDC;
            ...
        }
        internal_power {} {
            power_level : VDD;
            ...
        }
    }
    ...
}
```

Creating Source Files

Apache has established a convention that allows you to control how and when retention flops and latches will be selected when doing default cell selection. This convention relies on the use of named begin blocks for Verilog or named process statements for VHDL.

In your library, you will have a variety of retention cells. By understanding how your design operates, you may want to control which latches or flops from your power libraries are chosen at a very fine level of granularity. In Verilog, this will normally be down to the always @ block level and to the process level in VHDL. The following sample fragments of Verilog and VHDL demonstrate its use:

Sample Verilog

```
module top(...);  
...  
always @(posedge clock)  
begin : tag1  
    out1 = in1;  
end  
  
always @(posedge clock)  
begin : tag2  
    out2 = in2;  
end  
  
always @(posedge clock)  
begin : tag3  
    out3 = in3;  
end  
endmodule
```

Sample VHDL

```
architecture synth of top is  
...  
begin  
    tag1: process (clock)  
    begin  
        if (clock = '1' and clock'event) then  
            out1 = in1;  
        end if  
    end process  
  
    tag2: process (clock)  
    begin  
        if (clock = '1' and clock'event) then
```

```
        out2 = in2;
    end if
end process

tag3:process (clock)
begin
    if (clock = '1' and clock'event) then
        out3 = in3;
    end if
end process
end synth;
```

Later sections will show how to control default cell selection so that out1 can be one type of retention flop, out2 can be another and out3 can be a third type.

Special Option to the Elaborate Command

To use power gating, you will need to specify the “-tag_blocks true” option to the [Elaborate](#) command in your Tcl file. The resulting tags will be used later in the [MapRetentionCell](#) command to control retention cell default cell selection.

Setting up Your Command File for Power Gating

When running any type of PowerArtist analysis, you need to do the following:

- Define how you want your clocks inferred using the [SetClockNet](#) command.
- Ensure that the wire load models are being selected correctly to ensure that interconnect capacitance is being estimated correctly.
- Define output loads. For details see [Using Apache Default Wire Load Models for Capacitance Analysis](#).

In addition, for a power gating flow you will need to do the following:

- Define which libraries are used by which hierarchical instances.
- Define virtual supplies (design rails) that are associated with library power rails.
- Define power domains. For this, you need to:
 - Define the virtual supplies associated with a hierarchical instance.
 - Define the power gating condition that controls the virtual supply.
- Optionally, define the mapping for retention cells.

Defining Library Associations

As part of the power gating flow, you need to define which libraries will be used for power analysis for various hierarchical instances using the [SetLibrary](#) command. Assigning one or more libraries to an instance controls the following:

- Default cell selection for the RTL power models;
- Wire load model selection for use in estimating interconnect capacitance for the instance (if your libraries contain wire load models);
- Gate-level power models for instantiated cells;
- Power rail names that are available for use by voltage and power domains.

For complete details on how to use the SetLibrary command, including syntax and examples, see [Handling Designs with Multiple Libraries](#).

Defining Virtual Supplies

As part of the power gating flow, you need to define virtual supplies. Virtual supplies are used by PowerArtist to perform “what-if” experiments with respect to voltage islands or derating the voltages in libraries. They are also used for power gating applications. To define virtual supplies, you need to specify the [CreateVirtualSupply](#) command. For more information on using this command see, For complete details, see [Creating a Virtual Supply](#).

Defining Power Domains

To use power gating, you will have to create voltage and power domains. To do this, you need to use the [CreateDomain](#) command. For complete details, see [Assigning a Virtual Supply to a Hierarchical Instance](#).

Defining Retention Cell Mapping

As part of the power gating flow, you need to define the assignment of retention flops and latches to particular inferred register and latch instances. This is done by naming Verilog begin blocks or VHDL process statements (as described in [Creating Source Files](#)) and then providing the [MapRetentionCell](#) command for the named begin blocks or process statements.

Sample Command File for a Power Gating Flow

The following sample command file includes the set up commands, library definitions, virtual supply definitions, etc. for a power gating flow:

```
# Power Gating Settings
SetLibrary -instance top -library {hvt RETENTION_EXAMPLE_LIB Memories}
#SetLibrary -instance {top.core1.u1 top.core1.a1 top.core1.s1} -library {hvt lvt}

# Power gating specific settings #
CreateVirtualSupply -supply vdd -virtual_supply VDDR_X -on top.rx_rq
CreateVirtualSupply -supply vdd -virtual_supply VDDTX -on top.tx_rq
...
CreateDomain -instance top.core1.r1 -virtual_supply {VDDR_X RX_VDDNWS RX_VRET}
```

```
CreateDomain -instance top.core1.t1 -virtual_supply {VDDTX TX_VDDNWS TX_VRET}  
  
MapRetentionCell -instance {top.core1.r1} -attribute CK_LOW  
MapRetentionCell -instance {top.core1.t1} -attribute CK_LOW -tag NoMap -notag true
```

Performing Simulation-Based Power Analysis with Power Gating

To run a time-based power analysis that includes power gating, create a command file that contains the [CalculatePower](#) command with the appropriate options for time-based analysis, for example:

```
# Power analysis for simulation-based power gating flow #  
CalculatePower -analysis_type time_based \  
-time_based_upf_in_file my.upf \  
-activity_file my_file.iaf \  
-synlib_files {my_lib1.lib my_lib2.lib} \  
-top_instance top \  
-start_time 607158ps \  
-finish_time 12135580ps \  
-num_clock_cycles 20 \  
-reference_clock myclock.clk \  
-time_based_report_file time_based.rpt \  
-time_based_write_power_db true
```

You can just add this CalculatePower command specification to the command file you created in the following section (that includes the SetLibrary, CreateVirtualSupply, etc. commands) or you can source that .tcl file from this one using the “source” command.

Performing Vectorless Power Analysis with Power Gating

To perform a vectorless power analysis that takes into account power gating, you need all of the inputs listed in the section [Required Inputs for Power Gating](#) except those specific to time-based analysis. In addition, you need to create a vectorless activity file that you then specify to the CalculatePower command in your command file using the -vectorless_input_file options, for example:

```
# Power analysis for vectorless power gating flow #  
CalculatePower -analysis_type average \  
-vectorless_input_file my_file.vaf \  
-synlib_files {my_lib1.lib my_lib2.lib} \  
-top_instance top \  
-average_report_file average.rpt \  
-average_write_power_db true
```

Creating a Vectorless Activity File

Vectorless power analysis requires a Vectorless Activity File (.vaf). The .vaf file is an ASCII format file created using your favorite text editor whose format is defined in [Analyzing Vectorless Average Power](#).

To get reasonable power analyses, the .vaf file must:

- Accurately define the frequency and duty cycle for your clocks and primary IOs of your design.
- For clock gating, specify the frequency and duty cycles of enable signals.
- Specify the read/write frequencies for the memories in your design.

Understanding the Output Reports for Power Gating Analysis

Reports generated when analyzing the effects of power gating include additional sections over the basic power report.

Sample Report for Simulation-Based Analysis

The Time Based Power Report contains the Detailed Instance Power section that identifies the power and voltage domains with the type name “domain”.

2. Detailed Instance Power

=====

Average Power(Watts)			Type	Model	Instance
Static	Dynamic	Total	Name	Name	Name
-----	-----	-----	----	----	----
236.97uW	26.314mW	26.551mW	domain		domain_0
0W	0W	0W	user		-inst1
0W	0W	0W	user		-inst2
0W	0W	0W	CELL1		--inst_imp
0W	0W	0W	user		-inst_switch
0W	0W	0W	SWITCH1		--inst2_switch
275.01nW	0W	275.01nW	user		-io_inst
33pW	0W	33pW	and		--#381
< snip >					

After the Total power per supply section (not shown), there a section called “Power Per Domain”. This section provides power numbers for each defined power domain, both for average power and On Power. The average power is the overall average which includes power consumption for times when the power domain is on and off.

Conversely, the “On Power” is the average power consumed only when the power domain is switched on.

4. Power Per Domain

=====

Average Power (Watts)			On Power (Watts)			Domain
Static	Dynamic	Total	Static	Dynamic	Total	Name
-----	-----	-----	-----	-----	-----	----
118.74uW	24.841mW	24.96mW	118.81uW	24.856mW	24.975mW	domain_0

Lastly, there a section called “Power Domain Summary”. This section provides information on the power domains set up for this design. Information includes library names and details of the virtual supplies (including On condition setting, analysis voltage, and static and dynamic power numbers).

5. Power Domain Summary

=====

```
Domain domain_0
-----
Library typical
  File: ../typical.lib
Library typical1
  File: ../typical1.lib
Library typical2
  File: ../typical2.lib
Virtual Supply: VDDSW_0
Library Supplies:
  typical1.VDD
  typical2.VDD
  typical3.VDD
  typical4.VDD
  typical.VDD
Estimation Voltage: 1.2 V (from Tcl file)
On condition: domain1_on & ! domain2_on
Average Static Power: 116.33uW
Average Dynamic Power: 23.404mW
On Static Power: 116.41uW
On Dynamic Power: 23.419mW
Virtual Supply: VDDSW_1
Library Supplies:
  typical4.VDDNW
  typical.VDDNW
Estimation Voltage: 1.2 V (from Tcl file)
On condition: domain1_on & domain2_on
Average Static Power: 799.43nW
```

Average Dynamic Power: 1.253mW

On Static Power: 799.93nW

On Dynamic Power: 1.2538mW

Sample Report for Vectorless Analysis

Vectorless analysis reports contain two changes beyond normal time-based analysis. First, the Internal Power Consumption section highlights the power for each domain by setting the Model to “domain”. This is shown in the following excerpt.

```
3. Internal power consumption
=====
Power (Watts)
Component      Model      Supply      Static      Dynamic      Total
-----
instance_1     user       VDD_typ     26.9nW      226mW        226mW
  #48          and        VDD_typ     35.9pW      1.64nW        1.68nW
  #55          comparator VDD_typ     506pW        3mW           3mW
  #56          comparator VDD_typ     502pW      2.92mW        2.92mW
  ral          domain     VDD_typ     3.56nW      40.5mW        40.5mW
    #1         latch     VDD_typ     568pW      13.9mW        13.9mW
    #2         latch     VDD_typ     1.14nW      17.3mW        17.3mW
<snip>
  Total internal power              26.9nW      226mW        226mW
```

Secondly, this report also includes a section called “Power Domain Summary”. This section provides information on the power domains defined for this design. Information includes library names and details of the virtual supplies (including On condition setting, analysis voltage, and static and dynamic power numbers associated with the supply).

```
6. Power Domain Summary
=====

Domain instance_1.ra2
-----

Library typical
  File: typical.lib
Library typical2.db
  File: typical2.small.lib
Virtual Supply: VDD_typ
Library Supplies:
  typical.vdd
Estimation Voltage: 2.5 V (from Tcl file)
On condition: 1
Static Power: 26.9nW
Dynamic Power: 226mW
<snip>
```



```

Virtual Supply: VDDNSW
Library Supplies:
    typical2.db.VDDNW
Estimation Voltage: 2.5 V (from Tcl file)
On condition: On
Static Power: 0W
Dynamic Power: 0W

Domain instance_1.ral
-----
Library typical
File: typical.lib
Library typical2.db
File: typical2.small.lib
Virtual Supply: VDD_typ
Library Supplies:
    typical.vdd
Estimation Voltage: 2.5 V (from Tcl file)
On condition: 1
Static Power: 26.9nW
Dynamic Power: 226mW
<snip>
Virtual Supply: VDDNSW1
Library Supplies:
    typical2.db.VDDNW
Estimation Voltage: 2.5 V (from Tcl file)
On condition: 1
Static Power: 0W
Dynamic Power: 0W

```


Chapter 16

Using Standard File Formats

Introduction

PowerArtist supports three power formats:

- Proprietary PowerArtist commands
- CPF commands
- UPF commands

The proprietary commands are a superset of the supported commands in both UPF and CPF formats. These proprietary commands create a superset data model that gets extended as PowerArtist supports more commands in either the CPF or UPF standards. CPF and UPF are not translated in the proprietary commands but instead are mapped directly to the underlying data model. This chapter describes the CPF and UPF formats. It also includes a small section on using SDC.

Chapter Organization

The following topics are covered in this chapter:

- *Using a CPF Input Flow (Beta)*
- *Using a CPF Output Flow (Beta)*
- *Using a UPF Input Flow (Beta)*
- *Using an SDC Input Flow*

Using a CPF Input Flow (Beta)

Note on Using this Beta Feature

Before trying this beta feature/flow, please contact your Apache Design support team for assistance.

PowerArtist allows you to specify multiple voltage domains and power gating design intent/constraints in the Common Power Format (CPF). You can perform power gating analysis by creating multiple power domains in CPF and then providing the nominal condition for these domains. PowerArtist performs an analysis based on the power gating constraints that you specify. While you can mix and match CPF and the PowerArtist proprietary commands, you can't mix UPF and CPF commands.

CPF Input Use Model

Follow these high-level steps to perform power gating analysis with multiple power domains in CPF:

1. Define the power domains, nominal conditions, library sets and other low-power intent in the CPF format.
2. Specify power-related constraints using the CPF commands where available. For example, to specify retention cell definitions, you need to use the `define_state_retention_cell`, `create_state_retention_rule` and `update_state_retention_rules` CPF commands.
3. Specify other power-related constraints using the PowerArtist proprietary commands. For example, if you want to do a mixed-Vt analysis, you need to use the [SetVT](#) command.
4. Run RTL power analysis with or without simulation vectors. For simulation-based analysis, you will run the time-based analysis engine (`CalculatePower -analysis_type time_based`). For vectorless analysis, run the average power analysis engine (`CalculatePower -analysis_type average`).
5. To specify constraints in the CPF format, you must specify the any of the following `CalculatePower` command options, depending on the type of analysis you are performing:
 - `-average_cpf_in_file cpf_file_name`
 - `-reduction_cpf_in_file cpf_file_name`
 - `-time_based_cpf_in_file cpf_file_name`

An example would be:

```
CalculatePower -time_based_cpf_in_file my.upf
```

Sample CalculatePower Command for a Simulation-Based Analysis

```
CalculatePower -analysis_type time_based \
  -time_based_cpf_in_file my.upf \
  -activity_file my_file.fsdb \
  -synlib_files {my_lib1.lib my_lib2.lib}
  -top_instance top \
  -finish_time 20ns -start_time 20ns \
  -num_clock_cycles 20 \
  -reference_clock top.clk \
  -time_based_report_file time_based.rpt \
  -time_based_write_power_db true
```

Supported CPF 1.1 Commands

Though the software supports a subset of the total CPF 1.1 command set, it will read CPF files that use all of the available commands. Commands not supported are ignored. Similarly, the software ignores any command options that are specified but not supported. PowerArtist supports the following CPF 1.1 commands (with the listed options only):

<code>create_global_connection -net <i>net_name</i> -pins <i>pin_list</i> [-domain <i>domain</i>]</code>
<code>create_ground_nets -nets <i>net_list</i> [-voltage <i>float</i>] [-external_shutoff_condition <i>expression</i>]</code>
<code>create_nominal_condition -name <i>string</i> -voltage <i>float</i></code>
<code>create_power_domain -name <i>power_domain</i> [-default -default -instances <i>instance_list</i> -instances <i>instance_list</i> [-shutoff_condition <i>expression</i>]</code>
<code>create_power_mode -name <i>string</i> -domain_conditions <i>domain_condition_list</i> [-default]</code>
<code>create_power_nets -nets <i>net_list</i> [-voltage <i>float</i>] [-external_shutoff_condition <i>expression</i> -internal]</code>
<code>create_state_retention_rule -name <i>string</i> {-domain <i>power_domain</i> -instances <i>instance_list</i>} [-exclude <i>instance_list</i>]</code>
<code>define_library_set -name <i>library_set</i> -libraries <i>library_list</i></code>
<code>define_state_retention_cell -cells <i>cell_list</i> [-library_set <i>library_set</i>] [-cell_type <i>string</i>] {-restore_function <i>expression</i> -save_function <i>expression</i> -restore_function <i>expression</i> -save_function <i>expression</i>}</code>
<code>end_design</code>
<code>set_array_naming_style <i>string</i></code>
<code>set_cpf_version <i>value</i></code>

set_design module
set_hierarchy_separator <i>character</i>
set_power_target {-leakage <i>float</i> -dynamic <i>float</i> -leakage <i>float</i> -dynamic <i>float</i> }
set_power_unit [pW nW uW mW W]
update_nominal_condition -name <i>condition</i> -library_set <i>library_set</i>
update_power_domain -name domain {-primary_power_net <i>net_name</i> -primary_ground_net <i>net_name</i> }
update_state_retention_rules -names <i>rule_list</i> {-cell_type <i>string</i> -cells <i>cell_list</i> }

Sample CPF Input

The following example illustrates the use of CPF in PowerArtist.

```
# general commands
set_cpf_version 1.1
set_design top
set_array_naming_style \[%d\]
set_hierarchy_separator .
set_power_unit uW

# define the library sets
define_library_set -name LibSet_0 -libraries { ../../libs/apache/alf/alf_common/hvt.alf \
                                                ../../libs/apache/alf/alf_common/retn.alf \
                                                ../../libs/apache/alf/alf_vlog/mem.alf }

# create nominal conditions
create_nominal_condition -name NC_0 -voltage 1.10

# associate library sets with nominal conditions
update_nominal_condition -name NC_0 -library_set LibSet_0

# create power domains
create_power_domain -name PD_0 -default
create_power_domain -name PD_1 -instances { core1.r1 } -shutoff_condition { !rx_rq }
create_power_domain -name PD_2 -instances { core1.t1 } -shutoff_condition { !tx_rq }

# create power/ground nets
create_power_nets -nets {VDDRX RX_VDDNWS} -voltage 1.1 -external_shutoff_condition {!rx_rq}
create_ground_nets -nets RX_VRET -voltage 1.1 -external_shutoff_condition {rx_rq}
create_power_nets -nets { VDDTX TX_VDDNWS} -voltage 1.1 -external_shutoff_condition \
    {!tx_rq}
create_ground_nets -nets TX_VRET -voltage 1.1 -external_shutoff_condition {tx_rq}
create_power_nets -nets VDDRX_new -voltage 1.1
```

```

create_global_connection -net VDDR_X -pins vdd -domain PD_1
create_global_connection -net VDDTX -pins vdd -domain PD_2
create_global_connection -net RX_VDDNWS -pins {VDDNW VDD } -domain PD_1
create_global_connection -net TX_VDDNWS -pins {VDDNW VDD } -domain PD_2
create_global_connection -net RX_VRET -pins VRET -domain PD_1
create_global_connection -net TX_VRET -pins VRET -domain PD_2
create_global_connection -net VDDR_X_new -pins vdd -domain PD_0

# create power modes
create_power_mode -name PM_0 -domain_conditions { PD_0@NC_0 PD_1@NC_0 PD_2@NC_0 } \
    -default
create_power_mode -name PM_1 -domain_conditions { PD_0@NC_0 PD_1@NC_0 }
create_power_mode -name PM_2 -domain_conditions { PD_0@NC_0 PD_2@NC_0 }
create_power_mode -name PM_3 -domain_conditions { PD_0@NC_0 }

#retention cells
define_state_retention_cell -cell_type CK_LOW -cells NRTCLDFFBQ_F1 -restore_function "RET"
create_state_retention_rule -name RULE1 -instances {core1.t1 core1.r1} -exclude \
    {core1.t1.l1.* core1.r1.d1.*}
update_state_retention_rules -names RULE1 -cell_type CK_LOW

end_design

```

Sample PowerArtist Report Section for the CPF Flow

Using CPF to specify your power intent impacts the Power Domain Summary portion of your report file. The input in the previous section may generate a report section that looks like the following:

```

4. Power Domain Summary
=====

Domain top.PD_0
-----

Library hvt
  File: ../../libs/apache/alf/alf_common/hvt.alf
Library RETENTION_EXAMPLE_LIB
  File: ../../libs/apache/alf/alf_common/retention.alf
Library Memories
  File: ../../libs/apache/alf/alf_vlog/mem.alf
Virtual Supply: top.PD_0.VDDR_X_new
Library Supplies:
  hvt.vdd
  lvt.vdd
  Memories.vdd
  RETENTION_EXAMPLE_LIB.vdd
Estimation Voltage: 1.1 V (from TCL file)
Average Static Power: 4.4158uW
Average Dynamic Power: 963.97uW

```

```

    On Static Power: 4.4158uW
    On Dynamic Power: 963.97uW
Virtual Supply: OtherVirtualSupplies_top.PD_0_1.32
Library Supplies:
    iopad.VDD_1.320
    Estimation Voltage: 1.32 V (from library power_supply)
    On condition: 1
    Average Static Power: 46.145mW
    Average Dynamic Power: 29.74mW
    On Static Power: 46.145mW
    On Dynamic Power: 29.74mW
Virtual Supply: OtherVirtualSupplies_top.PD_0_2.7
Library Supplies:
    iopad.VDD_2.700
    Estimation Voltage: 2.7 V (from library power_supply)
    On condition: 1
    Average Static Power: 0W
    Average Dynamic Power: 727.1mW
    On Static Power: 0W
    On Dynamic Power: 727.1mW

Domain top.PD_1
-----
Library hvt
    File: ../../libs/apache/alf/alf_common/hvt.alf
Library RETENTION_EXAMPLE_LIB
    File: ../../libs/apache/alf/alf_common/retention.alf
Library Memories
    File: ../../libs/apache/alf/alf_vlog/mem.alf
Virtual Supply: top.PD_1.VDDRX
Library Supplies:
    hvt.vdd
    lvt.vdd
    Memories.vdd
    RETENTION_EXAMPLE_LIB.vdd
    Estimation Voltage: 1.1 V (from TCL file)
    On condition: top.rx_rq
    Average Static Power: 812.95uW
    Average Dynamic Power: 5.3168mW
    On Static Power: 2.026mW
    On Dynamic Power: 13.25mW
Virtual Supply: top.PD_1.RX_VDDNWS
Library Supplies:
    hvt.VDD
    lvt.VDD
    Memories.VDD
    RETENTION_EXAMPLE_LIB.VDD
    RETENTION_EXAMPLE_LIB.VDDNW
    Estimation Voltage: 1.1 V (from TCL file)

```



```
On condition: top.rx_rq
Average Static Power: 380.62pW
Average Dynamic Power: 1.6759uW
On Static Power: 948.59pW
On Dynamic Power: 4.1767uW
Virtual Supply: top.PD_1.RX_VRET
Library Supplies:
    RETENTION_EXAMPLE_LIB.VRET
Estimation Voltage: 1.1 V (from TCL file)
On condition: !top.rx_rq
Average Static Power: 2.2995nW
Average Dynamic Power: 57.093nW
On Static Power: 3.8404nW
On Dynamic Power: 95.354nW

Domain top.PD_2
-----
Library hvt
    File: ../../libs/apache/alf/alf_common/hvt.alf
Library RETENTION_EXAMPLE_LIB
    File: ../../libs/apache/alf/alf_common/retention.alf
Library Memories
    File: ../../libs/apache/alf/alf_vlog/mem.alf
Virtual Supply: top.PD_2.VDDTX
Library Supplies:
    hvt.vdd
    lvt.vdd
    Memories.vdd
    RETENTION_EXAMPLE_LIB.vdd
Estimation Voltage: 1.1 V (from TCL file)
On condition: top.tx_rq
Average Static Power: 1.1263mW
Average Dynamic Power: 7.3124mW
On Static Power: 2.0265mW
On Dynamic Power: 13.157mW
Virtual Supply: top.PD_2.TX_VDDNWS
Library Supplies:
    hvt.VDD
    lvt.VDD
    Memories.VDD
    RETENTION_EXAMPLE_LIB.VDD
    RETENTION_EXAMPLE_LIB.VDDNW
Estimation Voltage: 1.1 V (from TCL file)
On condition: top.tx_rq
Average Static Power: 1.8383nW
Average Dynamic Power: 6.1384uW
On Static Power: 3.3075nW
On Dynamic Power: 11.044uW
Virtual Supply: top.PD_2.TX_VRET
```

```
Library Supplies:  
    RETENTION_EXAMPLE_LIB.VRET  
Estimation Voltage: 1.1 V (from TCL file)  
On condition: !top.tx_rq  
Average Static Power: 6.0916nW  
Average Dynamic Power: 739.44nW  
On Static Power: 13.713nW  
On Dynamic Power: 1.6646uW
```

Key Points

In the power domain section of this report, note the following information:

- The name of a power domain specified in the CPF file is qualified with the scope in which it is created.
- The name of a virtual supply specified in the CPF file is qualified with the power domain for which it is created.
- The estimation voltage is the voltage of the power/ground nets specified with the `create_power_nets/create_ground_nets` commands.

The “on condition” of the virtual supply is the on state condition of the domain if the domain has its own on state condition. If the domain does not have its own on state condition, then the on state condition of power/ground nets will be used.

Using a CPF Output Flow (Beta)

Note on Using this Beta Feature

Before trying this beta feature/flow, please contact your Apache Design support team for assistance.

As a PowerArtist user, you may be accustomed to performing what-if analysis to explore different power and voltage domain architectures and then trying to determine the best possible combination to meet your power budget. If you are using Multiple Supply Voltage (MSV) and Power Shut-Off (PSO) low power techniques in your design, you can have PowerArtist write-out a file containing Common Power Format (CPF) constraints. This CPF-out flow uses CPF constraints to capture your MSV and PSO design intent.

You can then use these constraints with other tools like Cadence's RTL Compiler™. The CPF constraints are written in an ASCII format, allowing you to add to or modify the generated CPF commands to fit your needs. Note that these commands may not capture all of the information needed to synthesize the design and perform physical design.

To use the CPF-out flow, you will need to specify the name of the CPF output using either the `-average_cpf_output_file` or the `-time_based_cpf_output_file` to the `CalculatePower` command.

Example

```
CalculatePower -analysis_type average  
              -average_cpf_output_file example.cpf
```

When using a CPF-out flow, you will need to use particular options to existing MSV and PSO Tcl commands in PowerArtist. These options correspond to the options supplied in the CPF equivalent commands:

- `-name domain_name` option to the [CreateDomain](#) command
Specifies the name of the domain being created.
- `-domain domain_name(s)` option to the [SetLibrary](#) command
Specifies a list of domains in which all instances use cells from the specified library list. This option is required for a CPF-out flow.

Example

```
SetLibrary -domain {PD0} -library {typical_1 typical_2}  
SetLibrary -domain {PD1 PD2} -library {typical_3 typical_4}
```

The first command in the example specifies that all instances in the PD0 domain use cells from the two libraries: `typical_1` and `typical_2`. The second command specifies that all instances in domains PD1 and PD2 use cells from the two libraries: `typical_3` and `typical_4`.

- `-default on | off` option to the [CreateVirtualSupply](#) command

Use this option to specify the state of the main power net of the domain (and therefore the state of the domain) in the default power mode in CPF. It is optional and the default value for this option is “on”.

The [SetPowerTarget](#) command is also used in a CPF-out flow.

Mapping PowerArtist Tcl Commands to Generated CPF Commands

The following table shows lists the PowerArtist commands and the corresponding CPF commands that are generated as part of the CPF-out flow.

PowerArtist Command	Generated CPF Commands
SetLibrary -domain <i>domain_name(s)</i> -library <i>lib_name(s)</i> If -instance is given, it is ignored.	define_library_set -name <i>Libset_name</i> -libraries <i>lib_name(s)</i> Notes: ■ A unique <i>Libset_name</i> string is created. ■ <i>library_file_name</i> is the library file name, along with its path, that has the specified library.
	create_nominal_condition -name <i>name</i> -voltage <i>value</i> Notes: ■ A unique <i>name</i> string is created. Also, the voltage value is the estimation voltage of the library set given with the SetLibrary command.
	update_nominal_condition -name <i>NName</i> -library_set <i>Libset_name</i> Notes: ■ This command is generated for each define_library_set. The <i>Libset_name</i> is the same as that in the define_library_set command. Also, an <i>NName</i> corresponding to the <i>Libset_name</i> will be used.

PowerArtist Command	Generated CPF Commands
CreateVirtualSupply -supply <i>power_rail_name</i> -virtual_supply <i>supply_name</i> [-voltage <i>voltage</i>] [-on <i>condition</i>] [-default on off] Note: The power rail name must not be qualified by library name, for the CPF-out flow.	create_power_nets -nets <i>virtual_supply_name</i> -voltage <i>value</i> [-external_shutoff_condition <i>value</i>] Notes: ■ A separate create_power_nets command is generated for each CreateVirtualSupply and voltage value as given in the command. ■ -external_shutoff_condition: — added for those virtual supplies with the -on conditions that are not the main power supply of a domain. Its value is an inverted -on value. — added for those virtual supplies with the -on condition that are main power supplies of a domain that do not have any same voltage secondary supply without an -on condition. Its value is an inverted -on value.
CreateDomain -name <i>domain_name</i> -instance <i>instance_name(s)</i> -virtual_supply <i>supply_name(s)</i>	create_power_domain -name <i>domain_name</i> [-instances <i>instance_name(s)</i>] [-default] Notes: ■ Generated for each CreateDomain command. ■ -default: Only the domain with the top module name will have this.

PowerArtist Command	Generated CPF Commands
CreateDomain <i>cont.</i>	update_power_domain -name <i>domain_name</i> -internal_power_net <i>virtual_supply_name</i> Notes: ■ The <i>domain_name</i> is same as that of the create_power_domain command. ■ The -internal_power_net value is the name of the first virtual supply in the list given by the CreateDomain command. This net is treated as the main power supply of the domain.
	create_power_switch_rule -name <i>name</i> -domain <i>domain_name</i> -external_power_net <i>virtual_supply_name</i> Notes: ■ A unique <i>name</i> string is created. ■ It is generated for the main power supply of a domain, if: — this main power supply has a “-on” condition — there exists an always on (that does not have any -on condition) secondary power supply in the domain, that has the same voltage as the voltage of this power supply. ■ The -external_power_net value is the secondary virtual supply that has no -on condition specified.
	create_global_connection -domain <i>domain_name</i> -net <i>virtual_supply_name</i> -pins <i>power_rail_name</i> Notes: ■ Generated for each virtual supply name in the list specified with the CreateDomain command. ■ The <i>power_rail_name</i> is the power rail for that virtual supply.

PowerArtist Command	Generated CPF Commands
CreateDomain <i>cont.</i>	create_power_mode -name <i>mode_name</i> -domain_conditions <i>domain_condition(s)</i> [-default] Notes: ■ An MSV design will have only one create_power_domain command which will have -default specified. ■ A PSO design will contain 2ⁿ modes and therefore will create 2ⁿ create_power_mode commands. Where n is the number of the domain that has a -shutoff_condition. ■ -default: Specifies that this mode is the mode in which domains will be working using the default value of their main power supply as mentioned in their corresponding CreateVirtualSupply command
	create_isolation_rule -name <i>name</i> -isolation_condition <i>expression</i> -from <i>domain_list</i> -isolation_target from Notes: ■ Created for all of the domains with a -shutoff_condition. ■ A unique <i>name</i> string is created. ■ -isolation_condition is the power domain -shutoff_condition ■ -isolation_target is always “from”
	create_level_shifter_rule -name <i>name</i> -from <i>domain_list</i> -to <i>domain_list</i> Notes: ■ Generated for all domains with a lower nominal_condition voltage value as -from domain list to domains with a higher nominal_condition voltage value as -to domain list. ■ A unique <i>name</i> string is created.
CpfIncludeFile -name <i>file_name</i>	source <i>file_name</i> Note: ■ The file path can be relative or absolute.

PowerArtist Command	Generated CPF Commands
SetPowerTarget [-instance <i>instance_name(s)</i>] -static <i>static_value</i> -dynamic <i>dynamic_value</i> Note: ■ If you do not specify -instance, the power target is set for the top module instance and only the set_power_target command is generated. ■ You must specify either one or both of the static/dynamic values.	set_instance <i>instance_name_relative_to_top</i>
	set_design module_name_of_instance
	set_power_target -leakage <i>static_value</i> -dynamic <i>static_value</i>
	end_design
These CPF commands are automatically generated by PowerArtist. PowerArtist generates these commands as part of the header commands in the CPF file. There are no equivalent commands in PowerArtist.	set_cpf_version 1.0
	set_design <i>top_module</i> Note: ■ The <i>top_module</i> is the top module in the PowerArtist scenario file.
	set_array_naming_style \[%d\]
	set_hierarchy_separator .
	set_power_unit mW

Quick Reference Checklist for CPF Output Flow

This section provides a list of items to keep in mind while running a CPF-out flow.

- You must specify the name of the CPF output file using either the -average_cpf_output_file or -time_based_cpf_output_file options to the CalculatePower command (either in average or time_based mode, respectively).

Example for Time-Based Analysis

```
CalculatePower -analysis_type time_based -time_based_cpf_output_file example.cpf
```

- You must have a [CreateDomain](#) command for the top-level module in the design.
- Each domain—including the domain of the top-level module—must be associated with a [SetLibrary](#) command. The SetLibrary command must have the -domain option.
- In the [CreateVirtualSupply](#) command, the supply name must not be qualified by any library name. For example:

```
# The following will not work properly.
# CreateVirtualSupply -supply {Lib1.vdd} -virtual_supply {VDD_top}

# The following will work
CreateVirtualSupply -supply vdd -virtual_supply {VDD_top}
```


- If you do not specify the `-voltage` option with the `CreateVirtualSupply` command, the following will happen:
 - The voltage value of the supply (power rail) corresponding to the domain's library set is selected as the voltage value for that net.

Example

```
CreateVirtualSupply -supply vdd -virtual_supply {VDD_top}  
CreateDomain -name top -instance top -virtual_supply {VDD_top}  
# Assume for the example that lib1 has a VDD power rail with a value of 2.5V  
SetLibrary -domain {top} -library {lib1 lib2}  
# Generated "create_power_nets " rule for "VDD_top" will have 2.5V as its  
voltage value  
create_power_nets -nets VDD_top -voltage 2.5
```

- For nets not associated with any Power domain, the voltage value is selected as the first matching library supply (power rail) voltage from the libraries specified in the in your command file.
- The first virtual supply in the list of virtual supplies specified with the `CreateDomain` command should have a `-voltage` value matching the estimation voltage of the library set to which that domain is associated. If a primary net has a non-matching voltage, a `create_nominal_condition` CPF command with the virtual supply's voltage value is generated. This `nominal_condition` will not have a corresponding `update_nominal_condition` and the following warning is issued:

```
Warning 1556: PrimaryNet VDD_tpy1, of domain PD1 has voltage 2.5,  
which is different than domain's library set estimation voltage  
1.5 Creating a Nominal condition NC_2 with voltage 2.5 which will  
not have library set associated to it through  
update_nominal_condition in cpf_out file
```
- If you have multiple `CreateVirtualSupply` commands with same `-virtual_supply` name, the following will happen:
 - if the `-supply` values are different, then the same virtual supply is associated with all of the supplies (power rail) from different `CreateVirtualSupply` commands.
 - the `-voltage` and `-on` values are taken only from the first `CreateVirtualSupply` command.

Sample Input and Output for a CPF Flow

This section provides a sample of PowerArtist Tcl commands for a CPF output flow and the corresponding CPF file that is generated.

PowerArtist Tcl Commands

```
CpfIncludeFile -name ../LibDef.cpf

SetPowerTarget -static 0.030 -dynamic 0.250

# Note -domain rather than -instance for CPF out.
SetLibrary -domain {PD1 PD2 } -library {Lib1 Lib2}
SetLibrary -domain {PD3} -library {Lib3 Lib4}

CreateVirtualSupply -supply vdd -virtual_supply VDD -voltage 1.1 -default on
CreateVirtualSupply -supply vdd -virtual_supply VDD1 -voltage 1.1 -on \
{!(pcu_inst.pau[2])} -default on
CreateVirtualSupply -supply vdd -virtual_supply VDD2 -voltage 2.5 -on \
{!(pcu_inst.plu[2])} -default on

CreateDomain -name PD1 -instance {core core.alu_inst} -virtual_supply {VDD}
CreateDomain -name PD2 -instance {core.alu_inst.aui core.alu_inst.lui} \
-virtual_supply {VDD1 VDD}
CreateDomain -name PD3 -instance core.rf_inst -virtual_supply { VDD2}

MapRetentionCell -net {core.rf_inst.rf\\[7\\]* } -attribute CLK_FREE
```

Corresponding Generated CPF

```
set_cpf_version 1.0
source ../LibDef.cpf
set_design core
set_array_naming_style [%d\]
set_hierarchy_separator .

# define the library sets
define_library_set -name LibSet_0 -libraries {../libs/Lib1.lib ../libs/Lib2.lib}
define_library_set -name LibSet_1 -libraries {../libs/Lib3.lib ../libs/Lib4.lib}

# create nominal conditions
create_nominal_condition -name NC_0 -voltage 1.1
create_nominal_condition -name NC_1 -voltage 2.5

# associate library sets with nominal conditions
update_nominal_condition -name NC_0 -library_set LibSet_0
update_nominal_condition -name NC_1 -library_set LibSet_1

# create power domains
```

```
create_power_domain -name PD2 -instances { alu_inst.aui alu_inst.lui } \
-shutoff_condition { pcu_inst.pau[2] } \
-default_restore_edge { !(pcu_inst.pau[2]) } -default_save_edge { pcu_inst.pau[2] }
create_power_domain -name PD3 -instances { rf_inst } -shutoff_condition \
{ pcu_inst.plu[2] } -default_restore_edge { !(pcu_inst.plu[2]) } \
-default_save_edge { pcu_inst.plu[2] }
create_power_domain -name PD1 -instances { alu_inst } -default

# declare power and ground nets
create_power_nets -nets VDD -voltage 1.1
create_power_nets -nets VDD1 -voltage 1.1 -internal
create_power_nets -nets VDD2 -voltage 2.5 \
-external_shutoff_condition { pcu_inst.plu[2] }

# create power modes
create_power_mode -name Mode_PD2_0_PD3_0 -domain_conditions { PD1@NC_0 }
create_power_mode -name Mode_PD2_1_PD3_0 -domain_conditions { PD2@NC_0 PD1@NC_0 }
create_power_mode -name Mode_PD2_0_PD3_1 -domain_conditions { PD3@NC_1 PD1@NC_0 }
create_power_mode -name Mode_PD2_1_PD3_1 -domain_conditions \
{ PD2@NC_0 PD3@NC_1 PD1@NC_0 } -default

# create rule for level shifter insertion
create_level_shifter_rule -name LS_0 -from { PD2 PD1 } -to { PD3 }

# create rule for isolation cell insertion
create_isolation_rule -name ISO_0 -isolation_condition { pcu_inst.pau[2] } \
-from { PD2 } -isolation_target from
create_isolation_rule -name ISO_1 -isolation_condition { pcu_inst.plu[2] } \
-from { PD3 } -isolation_target from

#create rule for power switch insertion
create_power_switch_rule -name SW_0 -domain PD2 -external_power_net VDD

# create global connections
create_global_connection -domain PD1 -net VDD -pins { vdd }
create_global_connection -domain PD2 -net VDD1 -pins { vdd }
create_global_connection -domain PD2 -net VDD -pins { vdd }
create_global_connection -domain PD3 -net VDD2 -pins { vdd }

# update power domains
update_power_domain -name PD2 -internal_power_net VDD1
update_power_domain -name PD3 -internal_power_net VDD2
update_power_domain -name PD1 -internal_power_net VDD

# specify power targets
set_power_unit mW
set_power_target -leakage 30 -dynamic 250
```

```
# create rule for state retention insertion
create_state_retention_rule -name CSRR_CLK_FREE -instances { \
rf_inst.rf[7][0] rf_inst.rf[7][1] rf_inst.rf[7][2] }
end_design
```

Using a UPF Input Flow (Beta)

Note on Using this Beta Feature

Before trying this beta feature/flow, please contact your Apache Design support team for assistance.

This flow uses a combination of Unified Power Format (UPF) commands and proprietary commands. PowerArtist allows you to specify multiple voltage domains and power gating design intent/constraints in the UPF format. You can perform power gating analysis by creating multiple power domains in UPF and then establishing the power and ground supply connections for these domains. You can selectively switch these on and off using power switches.

PowerArtist performs an analysis based on the power gating constraints that you specify. Keep in mind that if you use UPF to define virtual supplies and power domains, you still need to use the proprietary commands to define libraries, source files, and other information. It is recommended that you read [Analyzing the Effects of Power Gating with Proprietary Commands](#) before reading this section so that you understand the scope of what UPF can replace within that greater flow.

UPF Input Use Model

Follow these high-level steps to perform power gating analysis with multiple power domains in UPF:

1. Define the power domains, nominal conditions, library sets and other low-power intent in the UPF format.
2. Specify power-related constraints using the UPF commands where available. For example, to specify retention cell definitions, you need to use the `define_state_retention_cell`, `create_state_retention_rule` and `update_state_retention_rules` UPF commands.
3. Specify other power-related constraints using the PowerArtist proprietary commands. For example, if you want to do a mixed-Vt analysis, you need to use the [SetVT](#) command.
4. Run RTL power analysis with or without simulation vectors. For simulation-based analysis, you will run the time-based analysis engine (`CalculatePower -analysis_type time_based`). For vectorless analysis, run the average power analysis engine (`CalculatePower -analysis_type average`).
5. For vectorless analysis, you should set the `on_state` option of the [create_power_switch](#) UPF command to 0, 1, true or false. You should not provide a net name as an `on_state` value because that means the switch will be considered to be always on. In vectored analysis however, you can specify an `on_state` with the net name.

6. To specify constraints in the UPF format, you must specify the any of the following CalculatePower command options, depending on the type of analysis you are performing:
- -average_upf_in_file upf_file_name

■ -reduction_upf_in_file upf_file_name

■ -time_based_upf_in_file upf_file_name

An example would be:

```
CalculatePower -time_based_upf_in_file my.upf
```

Sample CalculatePower Command for a Simulation-Based Analysis

```
CalculatePower -analysis_type time_based \  
-time_based_upf_in_file my.upf \  
-activity_file my_file.fsdb \  
-synlib_files {my_lib1.lib my_lib2.lib} \  
-top_instance top \  
-finish_time 20ns -start_time 20ns \  
-num_clock_cycles 20 \  
-reference_clock top.clk \  
-time_based_report_file time_based.rpt \  
-time_based_write_power_db true
```

Supported UPF Commands and Options

PowerArtist supports the following UPF commands:

create_power_domain domain_name [-elements list] [-include_scope] [-scope inst_name]
create_supply_port port_name [-domain domain_name] [-direction in out]
create_supply_net net_name -domain domain_name [-reuse]
create_power_switch switch_name -domain domain_name -output_supply_port {port_name supply_net_name} -input_supply_port {port_name supply_net_name} -control_port {port_name net_name} ... -on_state {state_name input_supply_port {boolean_function}}
connect_supply_net net_name [-ports port_list] [-domain domain_name] [-rail_connection rail_type]
set_domain_supply_net domain_name -primary_power_net supply_net_name -primary_ground_net supply_net_name
add_port_state port_name {-state {name (nom min nom max o)}} [-state ...]
set_scope inst_name

Sample UPF Input

This file sample shows commands that can be used in a PowerArtist UPF file.

```
set_scope top_module/conb1

create_power_domain PD_top -elements {ra2} -scope top_module/conb1
create_power_domain PD_Hamsa -elements {ra1} -include_scope

add_domain_elements PD_top -elements ra3

create_supply_net VDD_core -domain PD_top
create_supply_net VDD_Hamsa -domain PD_top
create_supply_net VDD_Hamsa -domain PD_Hamsa -reuse
create_supply_port VSS_port -direction in

add_port_state VSS_port -state {OFF off}

connect_supply_net VDD_core -rail_connection VDDNW -domain PD_top
connect_supply_net VDD_Hamsa -rail_connection VDD -domain PD_top

create_power_switch sw_Hamsa \
  -domain PD_Hamsa \
  -input_supply_port {vin VDD_core} \
  -output_supply_port {vout VDD_Hamsa} \
  -control_port {ctrl 0} \
  -control_port {abc 1} \
  -on_state {on_state vin {ctrl && abc}}

create_supply_net VSS -domain PD_top
create_supply_net VSS -domain PD_Hamsa -reuse

connect_supply_net VSS -rail_connection VDDC -domain PD_top
connect_supply_net VSS -ports VSS_port

add_port_state sw_Hamsa/vout -state {ON 0.99 1.2 3}

set_domain_supply_net PD_Hamsa -primary_power_net VDD_Hamsa -primary_ground_net VSS
set_domain_supply_net PD_top -primary_power_net VDD_core -primary_ground_net VSS
```

Sample PowerArtist Report Section for the UPF Flow

Using UPF to specify your power intent impacts the Power Domain Summary portion of your report file. The input in the previous section may generate a report section that looks like the following:

```
4. Power Domain Summary
=====
```

```
Domain top_level.PD_m1
-----
Library sc9_cmos11lp_base_rvt_tt_nominal_max_1p10v_25c
  File: ../libs_converted/sc9_cmos11lp_base_rvt_tt_nominal_max_1p10v_25c.lib
Library sc9_cmos11lp_pmk_rvt_tt_nominal_max_1p10v_25c
  File: ../libs_converted/sc9_cmos11lp_pmk_rvt_tt_nominal_max_1p10v_25c.lib
<snip>
Virtual Supply: top_level.PD_m1:VDD
  Library Supplies:
    sc9_cmos11lp_pmk_hvt_tt_nominal_max_1p10v_25c.VDDG
    sc9_cmos11lp_pmk_rvt_tt_nominal_max_1p10v_25c.VDDG
  Estimation Voltage: 1.1 V (from TCL file)
  Average Static Power: 0W
  Average Dynamic Power: 0W
  On Static Power: 0W
  On Dynamic Power: 0W
Virtual Supply: top_level.PD_m1:VSS
  Library Supplies:
    sc9_cmos11lp_base_hvt_tt_nominal_max_1p10v_25c.VSS
    sc9_cmos11lp_base_rvt_tt_nominal_max_1p10v_25c.VSS
  <snip>
  Estimation Voltage: 0V (from TCL file)
  Average Static Power: 0W
  Average Dynamic Power: 0W
  On Static Power: 0W
  On Dynamic Power: 0W
Virtual Supply: top_level.PD_m1:VDD_m1
  Library Supplies:
    sc9_cmos11lp_base_hvt_tt_nominal_max_1p10v_25c.VDD
    sc9_cmos11lp_base_rvt_tt_nominal_max_1p10v_25c.VDD
  <snip>
  Estimation Voltage: 1.1 V (from TCL file)
  On condition: top_level.m1_power_ctrl
  Average Static Power: 152.03nW
  Average Dynamic Power: 17.683mW
  On Static Power: 228.04nW
  On Dynamic Power: 26.525mW
```

Key Points

In the power domain section of this report, note the following information:

- The name of a power domain specified in the UPF file is qualified with the scope in which it is created.
- The name of a supply net specified in the UPF file is qualified with the scope and domain name in which it is created.
- The estimation voltage is the voltage of the UPF port to which the supply net is connected to in the UPF file.

- The “on condition” of the virtual supply is the on state condition of the output port of the power switch to which the supply net is connected to in UPF.
- The `connect_supply_net` command specifies the library rail connection of the virtual supply to the library rail in that domain. This corresponds to the Library Supplies section in the report for each virtual supply.

Additional items to note:

- The rail connection of all primary power supply nets of the domains must be present in the UPF file. This connects the library rail to the supply net for the domain where the primary power net is the supply net.
- PowerArtist does not support the power estimation of special cells such as switch cells, isolation cells and level-shifter cells.

Using an SDC Input Flow

The SDC (Synopsys Design Constraints) Format is part of most methodologies. If your flow relies on SDC, chances are that you already have captured some of the information required for Apache power tools. Apache provides the [ReadSDC](#) utility command to translate SDC information into PowerArtist Tcl commands.

Example

```
ReadSDC -sdc_files {clocks.sdc datapath.sdc}  
        -power_db_name top.pdb
```

Given this command, the software reads in two SDC files (`clocks.sdc` and `datapath.sdc`); opens and reads power database `rx40.pdb` to validate netlist information; writes out Tcl command file `ReadSDC.scr`; writes out transition time file `ReadSDC.tt`; writes out capacitance file `ReadSDC.caps` and writes out a `ReadSDC.load` file. Note that `ReadSDC.tt` and `ReadSDC.caps` are included in `ReadSDC.scr`.

Using ReadSDC Output with a PowerArtist Command File

Using output generated by `ReadSDC` is as simple as using the command and sourcing `ReadSDC.scr` in a Tcl file as shown below.

```
% ReadSDC -sdc_files {clocks.sdc datapath.sdc} -power_db_name top.pdb  
% source ReadSDC.scr
```

Mapping SDC commands to PowerArtist Tcl Commands

The following table shows a mapping of SDC 1.7 commands to PowerArtist Tcl commands.

SDC Command	PowerArtist Tcl Command
set_units	PowerArtist uses this unit value to define units for the rest of the numbers in the current SDC file.
create_clock	SetClockNet -name <> -mode infer -frequency <>
create_generated_clock	SetClockNet -name <> -mode trace -frequency <>
set_ideal_network	SetClockNet -mode infer
set_propagated_clock	SetClockNet -mode trace
set_clock_gating_check	SetClockNet -gate_clock yes
set_ideal_transition	CalculatePower -default_transition_time
set_input_transition	CalculatePower -transition_time_file
set_load	CalculatePower -capacitance_file
set_wire_load_mode	SetWireLoadMode
set_wire_load_model	SetWireLoadModel

Sample SDC Input File

```
set_units -time ns -capacitance pF
create_clock -name clk -period 2 -waveform { 0 1 } [get_ports {clk}]
set_propagated_clock [get_clocks {clk}]
set_clock_transition 0.1 clk
create_generated_clock -name gen_clk1 -source [get_ports {clk}] -multiply_by 2 \
-master_clock [get_clocks {clk}] [get_pins {myRtl1/out}]
create_generated_clock -name gen_clk2 -source [get_ports {clk}] -multiply_by 4 \
[get_pins -hsc @ -hierarchical -regexp myRtl2@out]
set_clock_gating_check -setup 0.5 -hold 0.1 [get_clocks gen_clk]
set_ideal_transition 0.4 clk
set_input_transition 0.25 [get_ports pIn1]
set_load 0.08 [get_ports pOut]
set_wire_load_mode segmented
set_wire_load_model -name small {r t}
```

Sample Output Files

There are three output files generated by the ReadSDC command: a PowerArtist command file a transition time file and a capacitance file.

Sample PowerArtist Command File

The [ReadSDC](#) utility generates, as one of its output files, a PowerArtist command file, which may look similar to the following:

```
SetClockNet -name myRtl1.out -frequency 1e+09 -mode infer
SetClockNet -name myRtl2.out -frequency 2e+09 -mode infer
SetClockNet -name clk -frequency 5e+08 -mode trace
pt_set default_transition_time 4e-10
pt_set transition_time_file ReadSDC.tt
pt_set capacitance_file ReadSDC.caps
SetWireLoadMode enclosed
SetWireLoadModel -name small -net r t
```

You may have to edit the resulting command file, particularly if you are using it at the RT level of abstraction. For example, you may want to add information on clock gating constraint to the [SetClockNet](#) command or add the [SetClockGatingStyle](#) command that is not present in SDC files.

Note also that the ReadSDC command uses the pt_set command when generating the command file. Using pt_set variables is an alternative technique to specifying options to commands, in this case the [CalculatePower](#) command. It is recommended that you not delete these and replace them with options to the CalculatePower command.

Sample Transition Time File

```
sdc_top.pIn1 2.5e-10
```

Sample Cap File

```
sdc_top.pOut 8e-14
```

Chapter 17

Acquiring Simulation Data

Introduction

This chapter describes how to acquire data from a simulation testbench so that power analysis can provide the most accurate results. PowerArtist provides the three approaches to acquiring of simulation data, all of which produce a simulation activity file. In general, the approach that you choose will depend on your design language (VHDL or Verilog), the level of abstraction and the simulator you are using. If you are doing RTL VHDL designs, you should choose either FSDB or IAF since these formats have the capability to store the composite data structure information that is typical of VHDL designs. If you are doing RTL Verilog, then any approach will work. If you are doing large gate-level designs (most likely in Verilog), the FSDB format is preferred because it more efficiently stores toggle information. In general, if you can create an FSDB file, use that approach. It is fast, compressed, complete and consistent across all simulators in its naming conventions.

If you have instantiated gates in your design, you need to pay particular attention to the amount of detail you capture in your format of choice. An instantiated gate may be as simple as a flip-flop or as complex as a memory. Simply, if it has a power model in Liberty format it is a gate. When your simulation runs, it will monitor and write out the nets in your design. If the instance being monitored is a gate-level instance, some simulators do not capture the nets local to the instance. Some of these local nets represent the ports of the instance. Whether or not ports are monitored for gate-level instances is very critical to know when performing power analysis.

If you are going to perform an average power analysis, then you should be monitoring the ports of all gate-level instances. If you are going to perform a time-based power analysis, this is not required. You lose a little accuracy when trying to perform an analysis of tri-state gates, but the improvement in performance is quite significant.

PowerArtist supports the following simulations and simulation data formats:

Simulator	Supported Formats
All Cadence Simulators	FSDB, VCD, IAF
Mentor ModelSim	FSDB, VCD, IAF
Synopsys VCS	FSDB and VCD

Chapter Organization

The following topics are covered in this chapter:

- [Using an FSDB Approach](#)
- [Using the Standard VCD Approach](#)
- [Using an IAF Approach](#)
- [Acquiring Simulation Data in Palladium Flows](#)
- [Troubleshooting Tips](#)

Using an FSDB Approach

It is recommended that you use FSDB files to get simulation data into PowerArtist. You need to generate the FSDB (.fsdb) file using the Verdi™ product from SpringSoft to obtain activity information from your simulator of choice. Using standard features available with Verdi, you will capture your simulator trace in an FSDB file.

Memory and MDA (Multi-Dimensional Arrays) value changes are captured by the following group of Novas system tasks:

```
$fsdbDumpMem
```

```
$fsdbDumpMDA
```

You can use the \$fsdbDumpvars task to dump all scalar, vector and memory/MDAs on struct signals. For example:

```
$fsdbDumpvars(top, "+all");  
$fsdbDumpfile
```

dumps all scalar, vector, memory/MDA and struct signals under the “top” instance and all of its hierarchical children. In contrast, VCD files cannot capture memory, MDA or struct signal types. If these language features are used in the RTL, a VCD will have poorer signal coverage for the design, impacting the accuracy of the RTL power analysis.

If your FSDB file contains either a larger time window than necessary or levels of hierarchy above the top module of the power analysis, fsdbextract can create a smaller FSDB file, enabling faster performance from CalculatePower.

Example

```
% fsdbextract my.fsdb -bt 100us -et 200us -s /testbench/top  
-level 0 -o my_100us-200us_top.fsdb
```

Once you have an FSDB file, you need to specify it using the `-activity_file` command-line option when running the [CalculatePower](#) command.

Improving FSDB Performance

If you have FSDB files that are greater than 500MB in size, you may experience performance problems with the processing of them. This will be especially true for gate-level designs, most of which are Verilog. For such cases, you can improve performance by running a SpringSoft utility called fsdb2vcd in the background. You can either have PowerArtist call the SpringSoft utility automatically or you can do it manually.

Automatic Technique for Running fsdb2vcd

If your design is written in either Verilog or Verilog 2001 and if you have fsdb2vcd in your path, the CalculatePower command automatically runs fsdb2vcd. It will not run fsdb2vcd for VHDL or System Verilog designs because the resulting VCD file will not adequately capture the equivalent activity information as was in your FSDB file.

If your design was Verilog or Verilog 2001 and you do not have fsdb2vcd in your path, CalculatePower will generate a Warning message that lets you know you may be missing an opportunity to improve your performance. Therefore, before running CalculatePower, you should make sure that fsdb2vcd is in your path. Typically your .cshrc file should include the following entry:

```
set path = (SPRINGSOFT_INSTALL_DIR/bin $path)
```

where `SPRINGSOFT_INSTALL_DIR` is the full path to your SpringSoft installation directory. A specific example would be:

```
set path = (/system/pkg/springsoft/61v1/bin $path)
```

Manual Technique for Running fsdb2vcd

Use the following process to manually execute the fsdb2vcd utility:

1. Create the FSDB file
2. Use the SpringSoft fsdb2vcd utility to create a VCD file

Since you don't want to store the VCD file on disk, you will need to use named pipes. To do this, see [Creating a Named Pipe to Manually Compress VCD Files](#). A sample set of commands would be:

```
% ptshell
mknod myout p
fsdb2vcd my.fsdb -o myout &
CalculatePower -activity_file myout
```

Note: fsdb2vcd Utility Must be Valid for Autospawn to Work Properly

If you are using the fsdb2vcd autospawn feature (where the fsdb2vcd utility from SpringSoft is autospawned for Verilog FSDB file processing) the fsdb2vcd utility must be valid, otherwise CalculatePower will hang with the message “wwgaf: Note 2103: Design prefix”. In such cases, the fsdb2vcd.log file present in the current directory will be empty. If this happens, you will need to kill the GAF creation process manually, correct the fsdb2vcd binary and then re-run CalculatePower.

Improving FSDB Capacity

If you are not manually running fsdb2vcd but allowing PowerArtist to read the FSDB file directly, you can control the size of the FSDB file that must be loaded by the database routines provided by SpringSoft. Specifically, you can reduce the total time interval that must be loaded. To do this, you need to specify the `-start_time/-finish_time` [CalculatePower](#) options.

Syntax

`-start_time` *string*

`-finish_time` *string*

Given these settings, only the value changes that occur between these times will be loaded. This is the preferred method.

Generating a VCDe File from an FSDB File

There are times when you might want to see an ASCII representation of the FSDB file. VCD is the ASCII representation of choice for most users. However, the VCD informal standard does not have a way to represent the complex data structures that System Verilog, Verilog 2001 or VHDL support. You must be aware of this if you use the standard SpringSoft utility fsdb2vcd. Data may not be represented in the resulting VCD file. If this is the case, you should run the ptFsdb2VcdPlus utility supplied as part of the PowerArtist distribution kit. This utility takes in an FSDB file and outputs the data in the Apache-proprietary enhanced VCD format, VCDe.

The following sample flow runs the ptFsdb2VcdPlus utility and then uses the resulting file as an input to CalculatePower.

1. Use the [ptFsdb2VcdPlus](#) utility of PowerArtist to convert the FSDB file to a VCDe file. The basic syntax of this utility is:


```
ptFsdb2VcdPlus -in fsdb_file_name -out vcde_file_name  
-topinst top_inst_name -start start_time_for_analysis  
-finish finish_time_for_analysis
```

If you intend to use the `-start_time` and `-finish_time` options in the `CalculatePower` run, you should use it in VCD generation itself instead—not in the `CalculatePower` run. This reduces the size of the resulting VCD file by eliminating the unneeded transitions. Note that for this utility, the option names are `-start` and `-finish`, which are different from the equivalent `CalculatePower` options.

2. Use the VCD file to generate a GAF file and run power analysis using the following command:

```
CalculatePower -enhanced_vcd true -activity_file vcde_file_name  
-gaf_file gaf_file_name -synlib_files path_to_libraries  
-top_instance top_inst_name
```

For details on the VCD file format, see [VCD File Format](#) in the *PowerArtist Reference Manual*.

Using the Standard VCD Approach

In the standard VCD (Value Change Dump) approach, you use commands provided with the simulator to create a VCD file. The standard VCD approach uses standard simulator facilities—for Verilog no special commands or options are required. While Verilog simulators have standardized on one set of commands to create VCD files, each VHDL simulator is slightly different. Also, because this approach creates a large file, it is recommended that you compress the resulting VCD file while the simulation is executing using a named pipe. To do this, see [Creating a Named Pipe to Manually Compress VCD Files](#).

Writing a VCD File from a Verilog Simulator

The easiest way to write a VCD file is to add the following two lines to your Verilog testbench:

```
$dumpvars;  
$dumpfile ("your_vcd_filename");
```

Writing a VCD File from ModelSim

Use the following procedure to write a VCD file using the Mentor (MTI) ModelSim simulator:

1. Execute the following commands before beginning simulation:

```
vcd file your_vcd_file_name  
vcd add -r *
```

2. Execute the following command when the simulation is finished to ensure that all data has been written to the VCD file:

```
vcd flush
```

Writing a VCD File from the Cadence NC-Sim Simulator

Use the following procedure to write a VCD file using the Cadence NC-Sim simulator:

1. Execute the following commands before beginning simulation, either from the command line or as part of a tcl input file:

```
database -open your_vcd_file_name -vcd  
probe -database your_vcd_file_name -all -depth all
```

2. Execute the following command when the simulation is complete:

```
database -close your_vcd_file_name
```

Creating a Named Pipe to Manually Compress VCD Files

If you are a UNIX user, you've most likely used a pipe to pass data from one tool to another. For example, the command line

```
ls | more
```

passes data from the first tool to the second using a pipe. This method does not work if the first tool requires the name of a file to write data into, or if the second tool requires the name of a file to read data from. Fortunately, UNIX provides a slightly different method for that case, called a named pipe. You can execute a UNIX command to create a special file that is actually a pipe. As the first tool writes data to this file, it is buffered and sent to the second tool, just as if the pipe were specified on the command line.

The named pipe lets you run the simulator and a file compression program (for example, `compress` or `gzip`) at the same time, passing data using a pipe. It is recommended that you use `gzip`, because it is automatically detected by the `wwgaf` conversion utility.

The Process

Use the following process to create a compressed VCD file using named pipe:

1. Edit the .v files to direct output to a file to be used as a named pipe, for example:

```
$dumpfile("my_pipe")
```
2. Compile the various files.
3. In the simulation directory, execute the following UNIX command:

```
mknod my_pipe p
```

The `mknod` command is located in a system directory that may not be in your execution path, usually `/etc` or `/usr/sbin`.

4. Execute the simulator and start your compression program. Both must be done in the background.

Example

```
mysim -f startup_file &  
gzip < my_pipe > dump.vcd.gz &
```

In this example, the name of the simulator executable is `mysim` and `gzip` is your compression program. You can delete `my_pipe` when the compression program is finished. In the example the characters “<” and “>” are UNIX redirection characters. Using these characters redirects the output of `my_pipe` to the input of `gzip`. The output of `gzip` is directed into a file called `dump.vcd.gz`.

Note:

The `wwgaf` utility automatically recognizes `gzip` files, therefore, you do not need to pipe gzipped files in `wwgaf`. If you use any other compression program, you will have to pipe data into `wwgaf`, for example:

```
uncompress -c my_vcd_file.vcd | wwgaf -iaf -
```

Using an IAF Approach

In the IAF (Intermediate Activity File) approach, you need to link special purpose code provided by PowerArtist into one of the supported simulators to create an IAF file. In this approach, you need to link one of the supported VHDL or Verilog simulators with PowerArtist utilities to create an IAF file. If you want to create an IAF, contact Apache Design for instructions.

Acquiring Simulation Data in Palladium Flows

Many of today's largest designs create tens of Gigabytes of simulation data. Even when capturing the data in the FSDB file format, the data size can be enormous. When a simulator generates this much data, it not only creates huge files that need to be stored on disk, but it significantly decreases the performance of the simulator. Hardware emulators and accelerators exacerbate this problem because you can run much larger test patterns on hardware than you can with a software simulator.

The Palladium™ series of accelerators/emulators from Cadence has the ability to output toggle information in FSDB or VCD file formats. They also have the ability to take a list of nets that should be monitored and record only the toggle information for those nets in the resulting FSDB or VCD file. PowerArtist requires critical nets to be monitored to perform an accurate average power analyses. The [Elaborate](#) command accepts the following option:

```
-list_required_traces file_name
```

The specified file will contain the nets that need to be monitored. This file will be formatted so that it contains one net name per line. The net name contains its hierarchical instance name path in the design. For example, if you specify the following option to Elaborate:

```
-list_required_traces trace.dat
```

the file trace.dat will be created in your current working directory and will have the following format:

```
top.in[1]  
top.in[2]  
top.clk
```

You will then write a simple script that post-processes this file into the format your Palladium hardware requires and include the resulting file as part of your Palladium setup environment. The resulting FSDB or VCD file will only contain the toggle information for those nets.

This flow works for both RTL, mixed RTL and gate, and pure gate-level designs. The following nets are monitored in your design:

- Nets connected to all of the primary ports (inputs and outputs) of the design.
- Nets connected to all of the ports of instantiated gates that are recognized as:
 - Memories
 - IO cells
 - Flip-flops
 - Latches
 - ICGCs (Integrated Clock Gating Cells)
 - Macro cells with pin counts greater than 10 and that are not flip-flops

- Macro cells with bussed pins
- Tri-states
- MUX select line (not output or input ports)
- Nets connected to all the ports of inferred elements (RTL components) that are recognized as:
 - Registers
 - Latches
 - Tri-states
 - Regfiles
 - Latchfiles
 - MUX/UNMUX select line (not output or input ports)

If your design contains RTL code, then perhaps the net that needs to be monitored is an inferred net that would not be present in your design source code. Therefore, what would need to be monitored would be all the named nets that form the immediate fan-in cone of logic for the inferred net. Therefore, Elaborate will trace back from an inferred net through inferred combinational logic instances (and, or, nand, nor, xor, xnor, connect and connect_inv) until it reaches the set of named signals that would be needed to capture the toggling activity for that signal.

Palladium will convert everything to uppercase so you need to run Elaborate with the `-case_insensitive` option as well as with the `-list_required_traces` option. The output list will contain IOs, state points and clock enable signals for monitoring in the Palladium simulation. You will need to convert the output list to uppercase. You can do this easily by using the `"tr [a-z] [A-Z]"` command. The resulting FSDB output from Palladium will be 15-20X smaller in size without sacrificing accuracy. The additional advantage for Palladium is that the simulations are now much faster and there no need to maintain terabytes of data. After running Elaborate, run CalculatePower with the default pin-based estimation (that is, the `-arc_based_estimation` option is set to `"false"` as it is by default).

For additional information, see [Analyzing Average Power Using a SAIF File](#).

Troubleshooting Tips

Missing 'timescale in Verilog

The most common error in using Verilog to acquire simulation data is leaving out the timescale specification for the simulator.

The first line of your Verilog testbench should set the simulation time scale, as shown in the following example.

```
'timescale 1 ns / 1 ns
```

If your Verilog file does not include a specification of the simulation time, the simulator uses a default value. For some simulators, the default is one second per clock cycle. If your signal changes every clock cycle but the simulator time step defaults to one second, when you import this data and run an analysis with a clock of 10 Mhz, the simulation data for the signal indicates that it changes only once per 10 million clocks. This results in a very low estimate of power for that net and the modules it drives.

Zero Length activities.iaf File

If you are performing a small simulation using the Intermediate Activity File approach and the activities.iaf file contains no data, exit the simulator. The simulator might not be closing the .iaf file until it exits. This means that there is still buffered data in the gzip process which is not written until its input file is closed.

Problems with ModelSim

ModelSim uses an internal environment variable to point to the binary executables for its tools. The \$MODEL_TECH environment variable is maintained by the tool and should not be manually reset (as noted in the ModelSim documentation). If you were to set it, you could have problems trying to use make_mti_mapfile executable because the modelsim.ini in the ModelSim installation uses this environment variable to point to other modelsim.ini files.

```
others = $MODEL_TECH/../../modeltech/modelsim.ini
```

Therefore, if you had set \$MODEL_TECH it is possible that the above path would not resolve to the correct location. If you run into this situation, simply unset the environment variable:

```
unsetenv MODEL_TECH
```

The make_mti_mapfile executable will then use ModelSim utilities to return a value for \$MODEL_TECH.

Zero Delay Simulation

If you are performing a mixed RTL/gate simulation (for example when pads are instantiated in your design) you need to know if a zero delay simulation run created your simulation results. There are three reasons for this.

First, it will cause inputs and outputs of cells that have power models to change in the same time step. Simulators often don't ensure that when they create simulation traces either via PLI routines or VCD files, that input changes will always appear before output changes.

When PowerArtist executes the “CalculatePower -analysis_type average” command when performing an arc-based analysis, it checks to see if certain transition sequences happen in the VCD file and matches those to power vectors in your .lib libraries. It does not sort all of the changes that happen in a time step to ensure that inputs change before outputs—doing so for most gate-level designs would be prohibitively expensive. Taking this method into account, it's possible in this scenario for an output to toggle but have no corresponding vector; therefore, power will be underestimated.

If you perform unit delay simulation or back-annotate delay information to your simulation, the output will change in a later time step and a power vector will be matched, assuming that one is present in your library files. This problem does not occur when you specify the “CalculatePower -analysis_type time_based -zero_delay true” command. The “-zero_delay true” option setting forces the time-based power calculator to try to re-order events that happen in each time step so output changes follow input changes. This switch will improve your results but you will incur increased run time due to the sorting of signal toggles for every time step in your simulation file.

A second reason for not using zero delay simulation is that zero-duration glitches will occur. These are multiple toggles on nets that occur during the same time cycle. These may result in too many power arcs being matched and power being over estimated. This problem is also addressed if you specify the “CalculatePower -analysis_type time_based -zero_delay true” command.

To accurately analyze power, you must at a minimum use unit delay simulation and at best back-annotate delays and simulate your design.

The following example is a section of a VCD file that exhibits both problems:

```
#40551
0T"
0S"
#136520
1S"
1T"
0S"
0T"
```

In this example, S” represents an input to a buffer. T” is the output. Notice at timestep #40551 that the output toggles before the input so a potential power arc match is missed here. Timestep #136520 shows the same net toggling multiple times in one time step. In this case, the arc missed in #40551 is actually recorded as being recognized twice here.

A third reason for not using zero delay simulation only occurs if you are using VCS, the function simulator from Synopsys. VCS eliminates simple buffers like those found in the clock network. The generated VCD file has all nets in the clock network sharing the same VCD id code, which causes poor performance when running “CalculatePower -analysis_type average”. When this happens, the CalculatePower log file contains an error message that all nets will not be monitored.

Chapter 18

Generating Etcl Files for CoolTime

Introduction

If you use Apache's CoolTime product and want to do simulation-based voltage drop analysis, you will need an Etcl file (also called a seed vector) generated by PowerArtist. You can generate an Etcl file for use in CoolTime by using one of two methods. You can generate it while performing a vector analysis (activity-based) or while performing a time-based power analysis (power-based). The design you are analyzing must be at the gate level of abstraction.

Chapter Organization

This chapter is divided into the following sections:

- [Generating an Activity-Based Etcl File](#)
- [Generating a Power-Based Etcl File](#)

For detailed information on the format of the Etcl file, see [Etcl File Format](#) in the *PowerArtist Reference Manual*.

Generating an Activity-Based Etcl File

Generating an activity-based Etcl file requires you to perform three steps:

1. Perform a vector analysis using the [GenerateActivityWaveforms](#) and [DefineGroup](#) commands. Before performing this step, you will need to [Elaborate](#) your design.
2. Examine the resulting waveforms in the Apache Waveform Viewer. Select the clock cycle(s) of interest noting the start and finish simulation times that encloses the cycle(s).
3. Generate the Etcl file using the [GenerateEtclFile](#) command.

Performing a Vector Analysis

When you perform the vector analysis, you can use either the clock-cycle mode or the time-based mode. In selecting the mode to run, consider that you will want to do the analysis at a very fine level of granularity. Most likely you will want your analysis interval size to be 1 clock cycle. This will allow you to see rapid changes in activity which could indicate a potential di/dt issue. The clock-cycle mode is probably the easiest way to perform this analysis.

It is recommended that you create a script called, for example, gaw.tcl. In this script, you will first need to define at least one group using the DefineGroup command. At a minimum, you should have one group represent the top module of your design. The following sample script performs vector analysis using the clock-cycle mode:

```
DefineGroup top { top }
set design top
GenerateActivityWaveforms \
  -activity_file ../design_data/rtl_sim/activities.vcd \
  -activity_waveform_clock_edge pos \
  -activity_waveform_clock_name top.clk \
  -activity_waveform_group_list { top } \
  -activity_waveform_graph_type activity_per_cycle \
  -activity_waveform_cycles_per_interval 1 \
  -activity_waveform_number_of_intervals 1000 \
  -activity_waveform_start_time 52ns \
  -fsdb_output_file top_activity.fsdb \
  -activity_waveform_log GenerateActivityWaveformBatch.log \
  -scenario_file top.Batch.scn \
  -top_instance txrx_tst.top1
```

The GenerateActivityWaveforms command in this script starts its processing at the first positive edge of top.clk after the start time of 52 ns. It performs a clock-cycle mode analysis for the next 1000 clock cycles because the cycles per interval is set to 1.

Depending on the options you specify to GenerateActivityWaveforms, this process will generate either an FSDB or a PTCL file with one waveform captured for each

group you defined using the [DefineGroup](#) command(s). Based on the sample script, there will be one waveform generated (for group top) in FSDB format (specified - fsdb_output_file option).

You would then run this script using the following command:

```
ptshell -tcl gaw.tcl
```

When the analysis is complete, you will see the resulting output files (GenerateActivityWaveformBatch.log, top_activity.fsdb, etc.) in your run directory.

Analyzing the Waveform

Examining the resulting waveform(s) requires you to load the resulting waveform output file into the Apache Waveform Viewer. At this point, you now need to decide what problem you want to analyze answering the following questions:

- Are you concerned about a high average sustained current with a potential high peak current?
- Are you concerned about a rapid shift from a low current to a much higher current which may cause a di/dt problem?

While activity does not completely correlate with power, the shape of the activity waveform should give you some insight into how you can narrow-down your analysis window. As a results of this analysis, you want to come away with a start and finish time that encompass the clock cycles you want to analyze.

Controlling the Precision of the Data in the Etcl File

The precision of data output in the Etcl file will be determined by extracting the precision data from the VCD file. If this is not adequate, you can set the environment variable PA_PRECISION to override these values, for example:

```
setenv PA_PRECISION 15
```

sets the Etcl precision to be in fempto seconds.

Generating the Etcl File

Once you have the start and finish times you recorded while analyzing your waveforms, the process of generating the Etcl file is easy. Take the start and finish time for the clock cycles of interest and apply them to the GenerateEtclFile command. To do so, you would create a Tcl script called, for example, gef.tcl. Given a start time of 152ns and a finish time of 160nS, your script might look like the following sample:

```
ReadLibrary lib1.lib
ReadLibrary lib2.lib

GenerateEtclFile \
```

```
-activity_file ../design_data/rtl_sim/activities.vcd \  
-etcl_finish_time 160ns \  
-etcl_file top.etcl \  
-etcl_start_time 152ns \  
-mixed_sim_prob_estimation true \  
-scenario_file top.Batch.scn
```

You would then run this script using the following command:

```
ptshell -tcl gef.tcl
```

Standard activity analysis parameters like Liberty libraries, scenario files are required. The resulting file top.etcl is what you would pass to CoolTime.

Using a Name Mapped Flow

You can also use an RTL simulation file to act as the stimulus file for Etcl generation. The [Name Mapping Flow](#) section describes how to do this in detail. The only difference is that before the [GenerateActivityWaveforms](#) and [GenerateEtclFile](#) commands, you have to specify the [SetNameMapFile](#) command to supply the name mapping file and then you have to supply the `-use_rtl_sim_data` option for both commands. See the following extended sample scripts from the following section.

Using a name mapping flow, the sample gaw.tcl file becomes:

```
SetNameMapFile -mapfile mapfile.dat -format conformal  
  
DefineGroup top { top }  
set design top  
GenerateActivityWaveforms \  
-activity_file ../design_data/rtl_sim/activities.vcd \  
-activity_waveform_clock_edge pos \  
-activity_waveform_clock_name top.clk \  
-activity_waveform_group_list { top } \  
-activity_waveform_graph_type activity_per_cycle \  
-activity_waveform_cycles_per_interval 1 \  
-activity_waveform_number_of_intervals 1000 \  
-activity_waveform_start_time 52ns \  
-fsdb_output_file top_activity.fsdb \  
-log GenerateActivityWaveformBatch.log \  
-scenario_file top.Batch.scn \  
-top_instance txrx_tst.top1 \  
-use_rtl_sim_data true
```

Using a name mapping flow, the sample gef.tcl becomes:

```
SetNameMapFile -mapfile mapfile.dat -format conformal  
ReadLibrary lib1.lib  
ReadLibrary lib2.lib  
GenerateEtclFile \  

```

```
-activity_file ../design_data/rtl_sim/activities.vcd \  
-etc1_finish_time 160ns \  
-etc1_file top.etc1 \  
-etc1_start_time 152ns \  
-mixed_sim_prob_estimation true \  
-scenario_file top.Batch.scn \  
-use_rtl_sim_data true
```

Generating a Power-Based Etcl File

To generate a power-based Etcl file, you have two choices. You can choose to have PowerArtist automatically select the clock cycle with the highest activity and then generate an Etcl file for that single clock cycle. Alternatively, you can perform a time-based power analysis, look at the waveforms interactively and then run the [GenerateEtclFile](#) command to generate an Etcl file that covers multiple clock cycles.

Automatically Selecting the Highest Activity Clock Cycle

If you want to generate an Etcl file for the clock cycle with the highest activity, you can use this method to have PowerArtist do so automatically.

1. Run a time-based power analysis using the [CalculatePower](#) -analysis_type time_based command. The key CalculatePower options needed to perform this operation are:
 - -etcl_file
 - -peak_cycle_file
 - -peak_cycle_processing_mode auto
 - -reference_clock

Using the tutorial design as in the activity-base Etcl generation section, a sample command would be:

```
ReadLibrary lib1.lib
ReadLibrary lib2.lib

CalculatePower -analysis_type time-based \
  -active_edge pos \
  -activity_file ../design_data/rtl_sim/activities.vcd \
  -etcl_file top.etcl \
  -peak_cycle_file pcf.tcl \
  -peak_cycle_processing_mode auto \
  -reference_clock top.clk \
  -scenario_file top.Batch.scn \
  -top_instance txrx_tst.top1
```

The “peak_cycle_processing_mode auto” instructs PowerArtist to identify the peak cycle automatically. When this command completes, you will see the pcf.tcl file in your working directory. This file contains a studio_setup_time_based command. This command has the following syntax:

```
studio_setup_time_based start_time end_time etcl_file_name
```

The start_time and end_time is the start/end of the cycle of highest activity, for example:

```
studio_state_setup_time_based 15ns 25ns top.etcl
```

The start time of the cycle of interest is 15ns and the end time is 25ns. The `etcl_file_name` value is the value of the `etcl_file` passed as an option to the `CalculatePower` command.

2. Run the `GenerateEtclFile` command as instructed in the activity-based flow. Here, you will have to feed forward the data in the `pcf.tcl` file into the same `GenerateEtclFile` command. The example here would be:

```
ReadLibrary lib1.lib
ReadLibrary lib2.lib

GenerateEtclFile \
  -activity_file ../design_data/rtl_sim/activities.vcd \
  -etcl_finish_time 25ns \
  -etcl_file top.etcl \
  -etcl_start_time 15ns \
  -mixed_sim_prob_estimation true \
  -scenario_file top.Batch.scn
```

The `start_time` (15ns) and `end_time` (25ns) values in the `pcf.tcl` file are used here in the `GenerateEtclFile` command (`end_time` in this command is called `finish_time`). The results file, `top.etcl`, is what you would specify to CoolTime.

Manually Selecting Single or Multiple Clock Cycles of Interest

If you want to perform a voltage drop analysis over multiple clock cycles, you can use this manual method to generate an Etcl file that captures the clock cycles of interest.

1. Run a time-based power analysis using the `CalculatePower -analysis_type time_based` command.

If you run in the auto mode as in the previous example, `CalculatePower` determines the peak power cycle. But if you want to analyze di/dt issues, sustained worst case power or analyze the results over multiple cycles, you will need to interactively analyze the power-over-time waveforms generated by a time-based power calculation. Use `CalculatePower -analysis_type time_based` and determine your start and finish time that encloses your clock cycle(s) of interest. Be sure to specify the “`-peak_cycle_processing_mode interactive`” option to `CalculatePower`.

2. Run the `GenerateEtclFile` command as instructed in the activity-based flow. At the end of step 1, you have your start and end time, now just run the `GenerateEtclFile` command as instructed in the activity-based flow.

Chapter 19

Writing OpenAccess Database Applications

Introduction

This chapter describes how you can develop OpenAccess database (OADB) applications and how to apply them in PowerArtist. Note that you can find OpenAccess programming samples in the \$POWERTHEATER_ROOT/examples/OpenAccess directory. For a list of OADB commands, see [Open Access Database Access Utilities](#).

Chapter Organization

The following topics are covered in this chapter:

- [Introduction to OADB Programming](#)
- [Using the PowerArtist API to Write an OpenAccess Application](#)
- [PowerArtist Netlist Properties](#)
- [Writing a Native OpenAccess Application](#)

Introduction to OADB Programming

OpenAccess is a collaborative effort managed by the Silicon Integration Initiative™, commonly referred to as Si2. A key goal of the effort is to provide an open EDA standard for IC design access and supporting software. A critical deliverable is a production proven database that all EDA vendors could use as the repository for their design data. The PowerArtist analyzers (controlled by the CalculatePower and ReducePower commands) generate an OpenAccess database (OADB) that contains the design netlist translated from the scenario file created by the Elaborate command, which they then augment with power information. The power information contains:

- Netlists that represent the clock networks in your design and the design itself.
- Reduction results data.
- Properties that represent calculated or extracted power data associated with the nets, instances and pins of your design.

OpenAccess provides an Application Programming Interface (API) that software developers may use to develop C++ applications. OpenAccess also provides a Tcl binding for those developers who wish to write Tcl applications rather than C++ applications. PowerArtist contains a Tcl API layer that makes use of the OpenAccess API.

If you decide to develop a native OpenAccess application, the OpenAccess community provides a tremendous amount of documentation that developers must consult to develop applications. It is beyond the scope of Apache Design to train end users to develop OpenAccess applications. However, as an Si2 member and an OpenAccess Coalition member, Apache is allowed to share this documentation with end users. If you want to write native OpenAccess applications, please contact your local sales team and they will make the documentation available to you. You may also join Si2 to support the OpenAccess effort and to gain other benefits as well. Visit www.openeda.org for more information.

PowerArtist Netlist Properties

As described earlier, PowerArtist stores netlists in OpenAccess. It also stores properties attached to instances, pins and nets. Once you have a handle to an instance, pin or net, you can use the following command to extract a particular attribute:

```
ptoa::getPropVal object_handle property_name
```

To retrieve the handle, use an appropriate “get” command. For example, for a net, you can use the following commands:

```
set net [oa::getNet $terminal]
set dutyCycle [ptoa::getPropVal $net Duty_Cycle]
```

For an occurrence instance, you can use the following commands:

```
set topOccurrence [oa::getTopOccurrence $design]
set occurrenceInstance [oa::getOccInst $topOccurrence]
set power [ptoa::getPropVal $occurrenceInstance Total_Power]
```

Apache is continually expanding the legal attributes you can retrieve off of each of the element types. The attributes names are case sensitive and must be used exactly as shown.

Instance Properties

PowerArtist stores the following instance properties:

- **Float/Double Values**
 - **Static_Power**: static power for the instance (in Watts). For a hierarchical instance, this is the sum of static power values of all of the children leaf and hierarchical instances.
 - **Dynamic_Power**: dynamic power for the instance in watts. For a hierarchical instance, this is the sum of dynamic power values of all of the children leaf and hierarchical instances.
 - **Total_Power**: the sum of the static and dynamic power values. For a hierarchical instance, it's the sum of static and dynamic power values of all the child instances (leaf as well as hierarchical) in that instance.
 - **Clock_Power**: inferred clock tree power consumed by the instance.
 - **Leaf_Total_Power**: the sum of the total power of all of the leaf-level instances in the hierarchical instance. As an example, suppose that module “top” has two children leaf-level instances top.#25 (an inferred instance) and top.mem1 (an instantiated memory) and one child hierarchical instance, top.block1. Given this information, then the Leaf_Total_Power of “top” = Total_Power of top.#25 plus Total_Power of top.mem1. Furthermore, the Total_Power of “top” = the

Total_Power of top.#25 plus the Total_Power of top.mem1 plus Total_Power of top.block1.

— **Forced_Power_Values**: power value specified by the SetPower command.

- String Values

— **Wireload_Model**: the wire load model used to estimate the capacitance for the instance.

— **File_Name**: the file name containing the instance definition if a hierarchical instance, or a gate-level instantiation, or the file containing the module that caused the inferred instance.

— **Clock_Gated**: the clock gating status, which can have any of the following values:

- None—the register instance is not clock gated.
- Inferred—the register has an inferred ICGC.
- Instantiated—the register instance has an instantiated ICGC.
- Inferred and Instantiated—the register instance has an inferred and an instantiated ICGC.

- Integer Values

— **Line_Number**: the line number of the instance in File_Name.

— **File_Type**: the type of HDL used to define the instance, which can have any of the following values:

- 0—invalid
- 1—defined in a Liberty file
- 2—verilog
- 3—system verilog
- 4—vhdl
- 5—blackbox

Net Properties

PowerArtist stores the following net properties:

- Float/Double Values

— **Wire_Cap**: estimated wire capacitance in farads

— **Duty_Cycle**: calculated duty cycle

— **Avg_Activity**: average activity value

— **Frequency**: frequency number in Hertz

— **Transition_Time**: transition time of the net

— **Net_Power**: power consumed by the net

- String Values
 - **Cap_Source**: the source of the net capacitance; it can be a wire load model if net capacitance was calculated or the name of a user specified file, such as a SPEF, if net capacitance was annotated.

Pin Properties

PowerArtist stores the following pin properties:

- Float/Double Values
 - **Cap**: pin capacitance if the pin is an input pin; load capacitance (that is, the sum of wire capacitance and capacitance of input load pins) if the pin is an output pin.

Module Properties

PowerArtist stores the following module properties:

- String Values
 - **Func_Type**: module function type
 - **File_Name**: the name of file where the module is defined
- Integer Values
 - **File_Type**: the type of HDL used to define the instance, which can have any of the following values:
 - 0—invalid
 - 1—defined in a Liberty file
 - 2—verilog
 - 3—system verilog
 - 4—vhdl
 - 5—blackbox
 - **Func_Type**: for inferred and instantiated leaf level modules
 - 0—invalid
 - 1—AND
 - 2—NAND
 - 3—NOR
 - 4—OR
 - 5—XOR
 - 6—XNOR
 - 7—INV
 - 8—BUFFER

- 9—MUX
- 10—FULL_ADDER
- 11—MEMORY
- 12—PAD
- 13—FLOP
- 14—LATCH
- 15—CORE
- 16—HALF_ADDER
- 17—ICGC
- **Line_Num**: the line number where the module is defined in File_Name
- Float/Double Values
 - **Area**: the sum of the area of all the instances in the module

Hierarchical Instance Properties

PowerArtist stores the following hierarchical instance properties:

- Float/Double Values
 - **Register_Static_Power**: this is the sum of the static power values of all registers in the instance and all registers of the instance's hierarchical children.
 - **Register_Dynamic_Power**: this is the sum of the dynamic power values of all registers in the instance and all registers of the instance's hierarchical children.
 - **Register_Load_Power**: this is the sum of the switching power values of all registers in the instance and all registers of the instance's hierarchical children.
 - **Latch_Static_Power**: this is the sum of the static power values of all latches in the instance and all latches of the instance's hierarchical children.
 - **Latch_Dynamic_Power**: this is the sum of the dynamic power values of all latches in the instance and all latches of the instance's hierarchical children.
 - **Latch_Load_Power**: this is the sum of the switching power values of all registers in the instance and all registers of the instance's hierarchical children.
 - **Memory_Static_Power**: this is the sum of static power values of all the memories in the instance and memories of the instance's hierarchical children
 - **Memory_Dynamic_Power**: this is the sum of dynamic power of all the memories of the instance and all memories of the instance's hierarchical children.
 - **Memory_Load_Power**: this is the sum of the switching power values of all memories in the instance and all memories in the instance's hierarchical children.
 - **Logic_Static_Power**: this is the sum of the static power values of all combinational elements in the instance and the instance's hierarchical children.

- **Logic_Dynamic_Power**: this is the sum of the dynamic power values of all combinational elements in the instance and the instance's hierarchical children.
- **Logic_Load_Power**: this is the sum of the switching power values of all combinational elements in the instance and the instance's hierarchical children.
- **Clock_Load_Power**: this is the sum of the switching power values of all clock tree elements in the instance and all clock tree elements of the instance's hierarchical children.
- **Pad_Static_Power**: this is the sum of the static power values of all pad elements in the instance and all pad elements in the instance's hierarchical children.
- **Pad_Dynamic_Power**: this is the sum of the dynamic power values of all pad elements in the instance all pad elements in the instance's hierarchical children.
- **Bbox_Static_Power**: this is the sum of static power values of all blackbox elements in the instance and instance's hierarchical children.
- **Bbox_Dynamic_Power**: this is the sum of dynamic power values of all the blackbox elements in the instance and the instance's hierarchical children.
- **Bbox_Load_Power**: this is the sum of switching power values of all the blackbox elements in the instance and the instance's hierarchical children.

Writing a Native OpenAccess Application

If you want to write an application using the API provided by Si2 as part of their OpenAccess package, this section describes how you can do it. You will most likely use the API layer described in [Using the PowerArtist API to Write an OpenAccess Application](#) to write your applications. This layer, while potentially slower, is much easier to use.

Contents of the Documentation Package

The documentation package you will receive from Apache (upon request) as a PowerArtist user contains the following:

- 50MBs of HTML help files that explain the C++ API and how to write Tcl applications.
- A PDF file, OpenAccess22fourthEdCD.si2.pdf, that is the OpenAccess Release 2.2 API tutorial. This contains over 600 pages of documentation on how to effectively write OpenAccess applications.
- An example that shows how to retrieve netlist and power information from an OpenAccess database created by PowerArtist using the Tcl binding.

There is a good amount of information available because the scope of OpenAccess is vast. It not only has to store logical information like netlists, but all the physical

information required to manufacture an IC. PowerArtist uses only a fraction of the total data model.

Basic Requirements for a Native OpenAccess Tcl Application

To write a native OpenAccess Tcl application, your development infrastructure must meet the following requirements:

- Have Tcl version 8.4 or greater installed on your system. This tclsh enables the Tcl OpenAccess API. Version 8.4.6 is the version that was used for testing.
- Have PowerArtist 2007.3 or greater installed. This version supports reading and writing OpenAccess databases from the power analyzers. It also contains libraries that your Tcl application must dynamically load that translate your Tcl procedure calls into C++ API calls to access the database.

Power data is stored as properties using OpenAccess Application Defined (AppDef) extensions. OpenAccess 2.2 does not support a generic method to access AppDef extensions. PowerArtist contains a shared library that you must load that provides a Tcl interface to access these properties.

Flow for Writing a Native Tcl Application

Use the following steps when writing a native Tcl Application. Note that there is a Bourne script available to do the first three steps for you (see [Using a Script](#)).

1. Make sure that you have access to the following runtime libraries. These are required to write even the most basic application:

- OpenAccess shared libraries.
- Apache Design shared library providing the AppDef extension.
- Standard gnu C++ shared libraries

All of these libraries are provided as part of the PowerArtist distribution. You must make these libraries part of your LD_LIBRARY_PATH. For example, you can do the following:

```
# pt-arch is an Apache script that returns the machine architecture
# to enable loading of platform dependent binaries and libraries.

setenv ADS_ARCH `pt-arch`
setenv LD_LIBRARY_PATH \ $POWERTHEATER_ROOT/lib/$ADS_ARCH:$POWERTHEATER_ROOT/lib/
oa/lib/$ADS_ARCH/opt:$LD_LIBRARY_PATH
```

Note: The “setenv LD_LIBRARY_PATH” command while very long must be on one line and not broken across multiple lines as shown here.

The C++ libraries are located in lib and the OpenAccess and AppDef extensions are located in lib/oa/lib.

2. Make sure the Tcl package autoloader knows the location of these routines. While there are several ways to do this, the easiest is to set the TCLLIBPATH environment variable as follows:

```
setenv TCLLIBPATH $POWERTHEATER_ROOT/lib/oa/lib/$ADS_ARCH/opt
```

3. Make sure that version 8.4 tclsh is in your path. For example, you could set it as:

```
set path = (/system/pkg/tcl/8.4.6/bin/i686-pc-linux-gnu $path)
```

The rest of this document assumes that you have become familiar with OpenAccess by reading the HTML and PDF documentation as well as the short example. A critical section for you to read in the HTML documentation is one called “Tcl Bindings for the OpenAccess API”. That is required reading before proceeding any further.

Using a Script

You may want to consider using the Bourne shell script, ptoatclsh, that is provided in the distribution. This script performs three separate functions:

- It implements the LD_LIBRARY_PATH changes described in Step 1 and sets the TCLLIBPATH in Step 2.
- It checks to make sure that the tclsh in your path is at least version 8.4 or above.
- It launches that tclsh with any parameters you pass it. A sample use would be:

```
> ptoatclsh  
%
```

This would put you into the tclsh command shell. Another use would be:

```
ptoatclsh my.tcl
```

This executes the Tcl file my.tcl and returns.

4. At the beginning of your Tcl program you need to load the Tcl packages. This can be done in the following manner:

```
package require oa  
package require ptoa
```

The package command loads the Tcl command extensions into the current Tcl interpreter. The “oa” in the first line represents the OpenAccess Tcl commands and “ptoa” represents the PowerArtist OpenAccess Tcl commands required to access power properties. If you just want to access netlist information, but don’t want power properties, you can eliminate this last line. To verify that the packages have been loaded correctly, run the following two commands:

```
info command oa::*  
info command ptoa::*
```

These should return a list of supported commands. If nothing gets returned, the packages have not loaded successfully.

5. If you want to access power properties and not just netlist information using the standard OpenAccess data model, then you need to do one final step. Just after you open the design, you need to do an “attach” that tells the PowerArtist routines the location of your OpenAccess design.

```
set design [oa::DesignOpen $libraryName $topCell $viewName $viewType r]  
ptoa::attach $design
```

Using the PowerArtist API to Write an OpenAccess Application

The PowerArtist API leverages knowledge you may have acquired writing applications for other EDA tools. It provides several layers of utilities that you can use to write a Tcl-based application that manipulates data stored in your power database that is itself an OADB. You can use these utilities to do the following:

- Map between PowerArtist, OpenAccess and industry standard synthesis product design representations of design object names.
- Perform design navigation functions similar to what Unix provides for manipulating the Unix file system. Example utilities and their Unix equivalents include [dcd](#) (cd), and [dpush/dpop](#) (pushd/popd).
- Query the power database. Example utilities include [getModule](#), [getModuleNets](#), and [getFlops](#).
- Traverse the netlist in complex ways. Example utilities include [getFanout](#) and [getSrcPin](#).
- Create detailed reports in the .csv format. Example utilities include [reportPath](#) and [reportCGEfficiency](#).

All of the utilities return 0, 1 or lists of design object path names.

With these utilities, you can access all of the netlist information contained in your design and extract data generated by a power analysis. If you want to write a Tcl application that uses the standard Si2 OpenAccess calls instead, please read section [Writing a Native OpenAccess Application](#).

Getting Started

Running a PowerArtist OADB application involves the following steps:

1. Create the power data using the [CalculatePower](#) command.
2. Launch ptshell.
3. While in the ptshell, source the power database file (.pdb) to locate the OpenAccess database you want to analyze.
4. Run your application by sourcing the appropriate Tcl files.

The following sections describe steps 3 and 4 in more detail.

Sourcing Your PDB File

Once you create the PDB file, launch the ptshell. As outlined in [Using the PowerArtist Shell](#), there are many ways to do this. The example below is merely an illustration of the simplest way. Suppose that you have run the analysis tutorial and

created a power database file called `top.AverageBatch.pdb`. The contents of the `.pdb` file look similar to the following:

```
# This file is a collection of other files.
#
# Primarily, the OADB directory and any other files required
# by openaccess.
#
# There is intentionally no scoping to the name, so that multiple
# Tcl namespaces can source this file and interpret the options
# appropriately.
#
# Generated by software version: 2010.2.2
setResults -oaDir top.BatchLib -topInst top
```

Now, in the directory containing `top.AverageBatch.pdb`, you would start the ptshell:

```
ptshell -artist
```

At this point, you will see the following type of information displayed in your Unix window:

```
PowerArtist/XP Shell (64 Bit Linux) R2010.2.2 (23 Mar 2011)
Copyright 1995-2011 by Apache Design, Inc.
Welcome to PowerArtist/XP
ptshell: Note 1093: Removing stacksize limit
#+++++
# Sourcing INI file: /pkg/pt/ptshell.ini
# Sourcing INI file: /projs00/analysis/full_chip/ptshell.ini
#+++++
Work Dir = /projs00/analysis/full_chip/ptshell_work
Log File = /projs00/analysis/full_chip/full_chip.log
Key File = /projs00/analysis/full_chip/full_chip.key
#+++++
full_chip_ptshell %
```

At this point, you will source the `.pdb` file as follows:

```
source top.AverageBatch.pdb
```

As the `pdb` file is processed, data structures are being built that ensure adequate API performance. When the process completes, you will see the following output in your Unix window:

```
# INFO: OADB: Initializing interface.
# INFO: OADB: Lib 'top.AverageBatchLib' read.
# INFO: OADB: Available views: 'netlist'
# INFO: OADB: Design 'top' opened in netlist view.
# INFO: OADB: Building module/inst cache for '/top'
# INFO: OADB: Read 79 modules, 1821 instances, 29558 pins
# INFO: OADB: Building cell primitives cache.
# INFO: OADB: Done.
0
```

The return code of 0 at the end can be tested to ensure automatically that the pdb file successfully loaded. If it did not load successfully, the return code would be 1. For instance, you can catch the failure by doing something like the following:

```
set passFail [catch {source top.AverageBatch.pdb}]
if {$passFail != 0} {
    puts "Failure!"
}
```

Sourcing your Application

At this point, you are ready to source your Tcl files that implement your particular application. A very simple application that you can run is one that returns the module type name for a hierarchical instance in the design. You can either enter the following commands in a file and source the file or type them directly at the ptshell prompt. For example, again using the tutorial design, you can type the following commands at the ptshell prompt:

```
set name [getModule /top/core1]
clogic
#+-----+
puts $name
clogic
#+-----+
```

Using the Design Navigation Utilities

In a power database, the design tree and inferred clock-trees are saved as top-level netlist-design objects. For the tutorial design, the top-level netlist-design objects are:

- design object, /top,
- clock-tree design objects /clk, /pci_clk, and /tck

The design object is hierarchical and the clock-tree design objects are flat. PowerArtist provides a set of design navigation utilities that mimic Unix directory traversal (cd) and listing (ls) utilities and allow you to navigate through the design hierarchy and list design contents. These are built on the notion of “current design directory” and “design root”. Similar concepts are supported in many Synopsys applications as well as the CPF and UPF standards. The current design directory is analogous to your current working directory in the Unix file system. The design root is analogous to your home directory. By setting the design root to a design object (be it your actual design or a clock design object) you can then set your current design directory to some point in your overall design hierarchy. Then all references to design elements can be specified relative to that point. Therefore, your design directory specification may be an absolute or a relative design path. These paths will correspond to a hierarchical instance in your design tree.

The following Design Navigation Utilities are available and are fully documented (full syntax and examples) in the *PowerArtist Reference Manual*:

Utility	Short Description
dcd	Sets the current design directory to the specified design directory
dpush	Pushes the current working directory to a directory stack and then sets the current working directory to be the design directory
dpop	Pops the directory stack and sets the current design directory to the directory that was on the top of the design stack
dirs	Lists the design directory stack
dpwd	Lists the current working directory
dls	Lists the contents of the design directory
show	Shows pseudo-Verilog for the specified instance path or module.

To view the full descriptions, simply click on the blue utility names (this will take you to the *PowerArtist Reference Manual*).

Using the Netlist Traversal Utilities

You can use the netlist traversal utilities to easily extract different types of data from from the netlist. For example, you can retrieve a list of pins for a specified net.

The following netlist traversal utilities are available and are fully documented (full syntax and examples) in the *PowerArtist Reference Manual*:

Utility	Short Description
getAssociatedNet	Returns the path name for the net associated with the specified pin path
getConnectedPins	Returns a list of pins connected to the specified net
getSrcPin	Returns the path name of the driving source pin of the specified pin or net.
getSinkPins	Returns a list of path names of the sink pins driven by the specified pin (or net)
getFanout	Returns a list of path names of fanout endpoints from a specified pin
getFanin	Returns a list of path names of fanin startpoints from a specified pin.

Using the Design Query Utilities

You can use the design query utilities to retrieve a variety of information from your design. For example, you can get a list of flop instances, module names, module port names, etc. The following design query utilities are available and are fully documented (full syntax and examples) in the *PowerArtist Reference Manual*:

Utility	Short Description
getFlops	Returns a list of flop instance paths
getModule	Returns the module name of the instance.
getModulePorts	Returns bit-blasted port names of the module in the form of a list of tuples <code>{bit_port_name dir}</code>
getModuleNets	Returns a list of net name paths that are local to the specified instance or module
getModuleInsts	Returns a list of instances of the specified inst path or module name. Names are relative to the specified inst path or module name
getInstsOfModuleType	Returns a list of instance path names with the specified module type name
getPinDirection	Returns a direction of the pin as input, output or inputOutput
getRelatedPins	Returns a list path names of related pins of this combinational instance pin
getPropVal	Returns the value of the specified property for the given object path (pin, net or instance)
isRoot	Determines if the specified instance or module is the root design root
isLeaf	Determines if the specified instance or module is a leaf
isFlop	Determines if the specified instance or module is a flop
isLatch	Determines if the specified instance or module a latch
isMemory	Determines if the specified instance or module is a memory
isSequential	Determines if the specified instance or module is a flop, latch or memory
isComb	Determines if the specified instance or module is a combinational device
isConnect	Determines if the specified instance or module is a connect primitive, buffer or inverter
isPin	Determines if the specified path is a pin

Utility	Short Description
getCGs	Returns a list of integrated clock gating cells (ICGCs)
getRegisterCGs	Returns a list of clock gating instances gating the register
getCGRegisters	Returns the register instances being gated by the specified clock gating instance

Using the Analysis Reporting Utilities

You can use the analysis reporting utilities to report power analysis information for various parts of your design. The following netlist traversal utilities are available and are fully documented (full syntax and examples) in the *PowerArtist Reference Manual*:

Utility	Short Description
reportPath	Reports the power along a specified path
reportPower	Returns the power associated with a given instance or all instances in a given module
reportCGEfficiency	Generates a comprehensive report of all inferred and instantiated ICGCs in the design

Using the Reduction Reporting Utilities

PowerArtist provides two reduction reporting utilities that you can use to collate multiple reduction reports: `readReductions` and `collateReductions`.

Utility	Short Description
readReductions	Reads one or more .csv report files generated by the <code>ReducePower</code> command and builds up an in-memory representation of them
collateReductions	Reads in CSV files and generates a collated report based on the data filter options you specify

These utilities work off of data generated by PowerArtist's `ReportReductions` command. Use the following flow to take advantage of these utilities:

1. Elaborate the design using the [Elaborate](#) command.
2. Perform a power reduction analysis using the [ReducePower](#) command.
3. Generate a .csv file of the reductions using the [ReportReductions](#) command.
4. Optionally iterate steps 2 and 3 using different stimulus files to obtain reduction data over a variety of different operating modes and conditions.

5. Run the [readReductions](#) utility once reading all CSV files generated in the sequence of steps 2-4.
6. Run the `collateReductions` command to read in the CSV files you created during step 5 and generate a formatted report. This report allows you to determine the best reductions to implement over all of the `ReducePower` scenarios you ran in step 3.

See the PowerArtist Reference Manual for complete syntax and examples of `readReductions` and `collateReductions` utilities.

Using the Power Database Mapping Utilities

The power database mapping utilities map names and objects between the PowerArtist namespace and the power database namespace defined by OpenAccess. You mostly likely will not need to use these utilities unless you are using API provided as part of the standard OpenAccess distribution. Utilities also exist that translate power database utilities to netlist names more closely associated with industry standard synthesis tools.

Utility	Short Description
pa2oa	Converts a PowerArtist name to a power database name
oa2pa	Converts a power database name to the equivalent PowerArtist name
oa2nl	Converts a power database name to a netlist name
nl2pa	Converts a netlist name to a PowerArtist name
pa2nl	Converts a PowerArtist name to a netlist name
nl2oa	Converts a netlist name to a power database name
getObject	Returns the power database object for the specified the object path
getOccObject	Returns the power database occurrence object for the specified object path

Index

Symbols

.ini (initialization file) [11](#)
 sample [12](#)
 .rpt (power analysis report) [71](#)
 .srpt (power report file) [129](#)
 sample [130](#)
 \$ADS_ARCH [452](#)
 \$EDITOR [39](#)
 \$LD_LIBRARY_PATH [452](#)
 \$MODEL_TECH [434](#)
 \$PA_PRECISION [439](#)
 \$POWERTHEATER_ROOT [5](#)
 \$PT_APPLY_SMW [87, 320](#)
 \$PT_DISABLE_APSH [10](#)
 \$PT_PACE_CLK_DONT_USE [222](#)
 \$PTSHELL_INI [12](#)
 \$TCLLIBPATH [453](#)

A

activity analysis
 creating graphs [245](#)
 design flow [245](#)
 ADS_ARCH environment variable [452](#)
 ADS_PA_FV, defining [381](#)
 Apache Waveform Viewer
 adding/deleting notes [191, 193](#)
 adjusting the height of plots [192](#)
 adjusting waveform thickness [192](#)
 comparing waveforms in one plot [193](#)
 deleting a plot [190](#)
 deleting waveforms [188](#)
 displaying waveforms from HSPICE [189](#)
 displaying waveforms from Spectre [189](#)
 marking data points [191](#)
 measurements and annotations [191](#)
 navigation menu [190](#)
 options menu [189](#)
 pan up, pan down [190](#)
 ruler feature [191](#)
 supported waveform sources [184](#)
 taking a snapshot [189](#)
 tearing tabs [193](#)
 toggle background color [192](#)

toggle grid [192](#)
 using zoom-in [188](#)
 viewing FSDB files [184](#)
 viewing PTCL files [188](#)
 waveform names [188](#)

C

capacitance estimation
 specifying wire load models [217](#)
 CGE improvement [178](#)
 in prism [178](#)
 in simple reductions [164](#)
 clock domain power consumption [77](#)
 Clock Enable Condition Linter (CEC) PowerBot [349](#)
 clock gating
 flow [237](#)
 clock gating efficiency, *See, CGE* [164](#)
 clock gating summary (in power report) [76](#)
 clock gating, enhanced [238](#)
 clock power analysis [233](#)
 clock power report [74](#)
 clock tracing order [76](#)

E

EDITOR environment variable [39](#)
 enhanced clock gating [238](#)
 environment variables
 ADS_ARCH [452](#)
 EDITOR [39](#)
 LD_LIBRARY_PATH [452](#)
 MODEL_TECH [434](#)
 PA_PRECISION [439](#)
 POWERTHEATER_ROOT [5](#)
 PT_APPLY_SMW [87, 320](#)
 PT_DISABLE_APSH [10](#)
 PT_PACE_CLK_DONT_USE [222](#)
 PTSH_INI [12](#)
 TCLLIBPATH [453](#)
 environment, setting the [4](#)
 etcl file, generating
 activity-based [438](#)
 power-based [442](#)

F

forward clock tracing, controlling [235](#)
 frequencies, net (reporting) [81](#)
 FSDB support
 automatic running of fsdb2vcd [427](#)
 improving capacity [428](#)
 improving performance [427](#)
 manual running of fsdb2vcd [427](#)
 functional verification [382](#)

G

Gate Memory Clock (GMC) PowerBot [326](#)
 gate-level power analysis [275](#)
 average analysis [253](#)
 time-based analysis [275](#)
 Graphical User Interface (GUI)
 See PowerCanvas GUI
 graphs, activity
 creating [245](#)
 viewing [67](#)

H

hierarchical RTL rewrite [376](#)
 hierarchy browser
 coloring by power [31](#)
 showing instances in schematic [32](#)
 HSPICE [184](#)

I

inferred buffer tree power report [80](#)
 initialization file (.ini) [11, 12](#)
 instantiated clock tree capacitance, estimating [237](#)

K

key file [12](#)
 changing the default name [12](#)

L

LD_LIBRARY_PATH environment variable
[452](#)

linter PowerBots [339](#)

M

Memory Power Linter (MEM) PowerBot [340](#)

metacomments, VHDL [203](#)

MODEL_TECH environment variable [434](#)

MUX Power Linter (MUX) PowerBot [341](#)

O

Observability Don't Care (ODC) PowerBot [335](#)

OpenAccess, programming [446](#)

P

pa_modules.v file [363](#)

PA_PRECISION environment variable [439](#)

PACE models [221](#)

capacitance models [221](#)

clock distribution models [221](#)

impact on output files [223](#)

precedence of [222](#)

peak power analysis

design flow [275](#)

power analysis

analyzing clock power [233](#)

average flow [249](#)

block-level tutorial [53](#)

command line

modes, simulation [250](#)

generating a clock tree [25, 56](#)

GUI tutorial, average flow [45](#)

GUI tutorial, time-based flow [49](#)

modal analysis [24](#)

peak analysis flow [275](#)

report (.rpt) [71](#)

time-based flow [25](#)

tutorial

See *also* tutorial [15](#)

using a SAIF file [269](#)

vectorless flow [291](#)

viewing results in the GUI [31](#)

with mixed-vt (threshold voltage) [229](#)

power macros [206](#)

built-in (default) macros [206](#)

default models [206](#)

macro map file [207](#)

-macro_directories option [207](#)

replacing simulation

models [206](#)

power reduction

design flow [298](#)

hierarchical clock gating [242](#)

inferring buffer trees for high-fanout nets [242](#)

recommended flow [298](#)

sample report file [130](#)

setting up clock gating [239](#)

algorithm [240](#)

flow [239](#)

techniques in power report [129](#)

tutorial [84](#)

viewing results [301](#)

power reduction clock gating report

description [132](#)

sample [133](#)

power reduction report

description [129](#)

sample [130](#)

power supplies

creating user [227](#)

power-aware models [206](#)

PowerBots

flow diagram [302](#)

flow for using [303](#)

for power reduction [309](#)

identifying wasted power [339](#)

PowerBots, running

CEC [349](#)

DOI [311](#)

GMC [326](#)

LEC [313](#)

LNR [309](#)

MEM [340](#)

MUX [341](#)

ODC [335](#)

Prism [328](#)

REG [348](#)

SMW [318](#)

PowerCanvas GUI

edit menu [142](#)

help menu [143](#)

manipulating the clock tree [161](#)

normal vs. optimal views for Prism dia-
log [181](#)

properties dialog [33](#)

smart source browser, using [151](#)

tools menu [142](#)

view menu [142](#)

waveform viewer [184](#)

POWERTHEATER_ROOT environment
variable [5](#)

Prism PowerBot [328](#)

properties dialog [33](#)

PT_APPLY_SMW environment variable

[87, 320](#)

PT_DISABLE_APSH environment variable
[10](#)

PT_PACE_CLK_DONT_USE environment
variable [222](#)

ptshell

basic features [10](#)

key file generation [12](#)

ptshKeyFile variable [12](#)

ptshLogFile variable [12](#)

starting the GUI from [9](#)

PTSHELL_INI environment variable [12](#)

ptshKeyFile Tcl variable [12](#)

ptshLogFile Tcl variable [12](#)

R

reduction, See power reduction

Register Power Linter (REG) PowerBot [348](#)

reports

power analysis report (.rpt) [71](#)

power reduction (.srpt) [129](#)

power reduction clock gating (_cg.srpt)
[132](#)

RTL rewrite

debug switches [353](#)

directory organization [353](#)

flow [351](#)

form of code modifications for LNR [354](#)

formal verification of rewritten code [371](#)

format of the rewritten code [354](#)

general form of code modifications [366](#)

GMC powerbot code modifications [359](#)

hierarchical flow [376](#)

inputs [351](#)

internal process flow [352](#)

LNR gate-level simulation [359](#)

Prism powerbot code modifications [362](#)

using the wizard [126](#)

warnings and errors [374](#)

RTL Rewrite wizard [127](#)

rule file, generating (for nCompare) [382](#)

sample rule file [384](#)

S

SAIF files [269](#)

basic format [269](#)

flow for using [269](#)

sdpd format [269](#)

schematic display

opening properties dialog [33](#)

using [32](#)

SetCellDefaultFanout command [235](#)

- SetClockBuffer command [233](#)
- SetClockNet command [233, 234](#)
- SetDefaultFanout command [234](#)
- SetHighFanoutNet command [242](#)
- SetNetProperty command [234](#)
- simple reductions dialog, column header definitions [153, 163](#)
- simple reductions dialog, in tutorial [97](#)
- simulation activity
 - creating analysis graphs [245](#)
 - selecting a vector analysis type [245](#)
 - defining design phase groups [244](#)
 - GenerateActivityWaveforms command [246](#)
 - running vector analysis in design flow [245](#)
- simulation activity file [425](#)
- simulation activity, analyzing [243](#)
- simulation testbench
 - acquiring data (troubleshooting) [434](#)
- SPEF processing
 - hierarchical [215](#)
- Split Memory Word (SMW) PowerBot [318](#)
- synthesis-aware models [206](#)
- System Verilog
 - licensing [2](#)

T

- TCLLIBPATH environment variable [453](#)
- threshold voltage, mixed (power analysis) [229](#)
- time-based power analysis [275](#)
- tutorial
 - power analysis
 - block-level [53](#)
 - gate-level [69](#)
 - RTL full-chip [19](#)
 - viewing results in the PowerCanvas [31](#)
 - with a mixed-vt cell library [61](#)
 - with clock gating [60](#)
 - with power gating [66](#)
 - with voltage islands [64](#)
 - reduction [84](#)
 - basic flow [87](#)
 - examining linter reduction results [118](#)
 - examining Prism results [109](#)
 - filtering reduction results [103](#)
 - reviewing results in the PowerCanvas [95](#)
 - scheduling auto RTL changes [106](#)
 - scheduling multiple RTL changes simultaneously [107](#)
 - scheduling Prism opportunities for auto RTL rewrite [117](#)
 - using the command files [86](#)
 - viewing cones of logic [111, 113](#)
 - wizard flow [121](#)

V

- VAF file
 - description [291](#)
 - sample from tutorial [56](#)
- VCD file
 - compressing manually [430](#)
 - creating
 - from Leapfrog [430](#)
 - from ModelSim [429](#)
 - from Verilog [429](#)
- vector analysis
 - batch flow [28, 246](#)
 - creating analysis graphs [245](#)
 - DefineGroup command [246](#)
 - defining groups [244](#)
 - design flow [245](#)
 - GenerateActivityWaveforms command [246](#)
 - GUI flow (tutorial) [42](#)
 - selecting a type [245](#)
 - viewing FSDB or PTCL files [248](#)
- verification, functional [381](#)
 - using a rule file [382](#)
- VHDL design
 - compiling into library [203](#)
 - metacomment processing [203](#)
- voltage values
 - characterization voltage [225](#)
 - estimation voltage [225](#)

W

- waveforms, viewing
 - See also Apache Waveform Viewer* [184](#)
- wire load models [217](#)
 - setting Tcl commands [218](#)
- WriteReductionCompareFile command [382](#)
- wgaf conversion utility [430](#)

Z

- zero delay simulation [435](#)

